# Homotopy Type Theory

*A Detailed Exposition of the HoTT Book*

Leandro Caniglia

November 2025

# Contents


**1 Preliminary Concepts** **1**
- 1.1 Introduction . . . 1
- 1.2 Basic Notions . . . 3
- 1.3 Function Types . . . 5
- 1.4 Dependent Function Types . . . 7
- 1.5 Product Types . . . 10
- 1.6 Dependent Pair Types . . . 17
- 1.7 Coproduct Types . . . 22
- 1.8 Boolean Type . . . 24
- 1.9 The Type of Natural Numbers . . . 27
- 1.10 Propositions as Types . . . 32
- 1.11 Identity Types . . . 34
- 1.12 Arithmetic of Natural Numbers . . . 42
- 1.13 Strong Induction . . . 46
- 1.14 Summary of Constructors and Eliminators . . . 47
- 1.15 Exercises . . . 49
- 1.16 Additional Exercises . . . 63

**2 Homotopy Type Theory** **67**
- 2.1 Functoriality and Path Algebra . . . 67
  - 2.1.1 Path Whiskering . . . 69
- 2.2 Pointed Types and Loop Spaces . . . 72
- 2.3 Fibrations and Transport . . . 75
- 2.4 Function Homotopies . . . 83
  - 2.4.1 The Homotopy Relation . . . 83
  - 2.4.2 Types as Categories . . . 84
  - 2.4.3 The Algebra of Homotopies . . . 85
  - 2.4.4 Evaluating a homotopy on another homotopy . . . 88
  - 2.4.5 Sections of Path Spaces . . . 88
- 2.5 Equivalences . . . 89
- 2.6 Observational Equality Framework . . . 98
  - 2.6.1 Observational Equality Framework . . . 98
  - 2.6.2 Extended Observational Framework . . . 99

2.7 Equality in Cartesian Products . . . 101
2.8 Equality in the Boolean Type . . . 108
2.9 Equality in Σ-Types . . . 110
2.10 Equality in the Unit Type . . . 122
2.11 Function Extensionality Axiom . . . 125
2.12 Naturality of Function Extensionality . . . 141
2.13 The Univalence Axiom . . . 146
2.14 Equality in Identity Types . . . 151
2.15 Equality in Coproducts . . . 157
2.16 Equality in Natural Numbers . . . 162
2.17 Equality in Semigroups . . . 169
2.18 Universal Properties . . . 171
2.19 Exercises . . . 178

**3 Sets & Mere Propositions 208**
3.1 Sets . . . 208
3.2 Mere Propositions . . . 215
3.3 Propositional Truncation . . . 228
3.4 Contractibility . . . 231
3.5 The Axiom of Choice . . . 239
3.6 Hedberg's Theorem . . . 242
3.7 Exercises . . . 244

**4 Equivalences 266**
4.1 Quasi-Inverses . . . 266
4.2 Half-Adjoint Equivalences . . . 276
4.3 Bi-invertible Maps . . . 281
4.4 Contractible Fibers . . . 282
4.5 Surjections and Embeddings . . . 284
4.6 Closure Properties of Equivalences . . . 286
4.7 The Object Classifier . . . 290
4.8 Univalence Implies Function Extensionality . . . 298
4.9 Exercises . . . 301

**5 Induction 316**
5.1 Intuition for Uniqueness . . . 316
5.2 Well-Founded Trees . . . 321
5.3 W-Types . . . 324
5.4 Initial Algebras . . . 330
5.5 General Recursion . . . 339
5.6 Homotopy-Inductive Types . . . 341
5.6.1 Bridging the Gap . . . 343
5.7 Strict Positivity . . . 344
5.7.1 Free Variables . . . 345
5.8 Inductive Type Families . . . 347
5.9 Mutual Inductive Types . . . 355

5.10 Inductive Type Families & W-Types . . . . . 357
5.11 Inductive-Inductive Definitions . . . . . 358
5.12 Identity Types and Identity Systems . . . . . 361
5.13 Exercises . . . . . 375

**6 Higher Inductive Types** **395**
6.1 Initial Algebras Revisited . . . . . 395
6.2 Induction Principles and Dependent Paths . . . . . 398
6.3 Elimination Rules . . . . . 398
6.4 The Interval . . . . . 404
6.5 Circles & Spheres . . . . . 405
6.5.1 Higher Dependent Loops . . . . . 409
6.5.2 Recursion . . . . . 413
6.6 Suspensions . . . . . 417
6.6.1 Functoriality . . . . . 428

**A Axiom Dependencies** **431**
A.1 Function Extensionality Dependencies . . . . . 431

**B Additional Notes** **433**
B.1 Universes and Type Families . . . . . 433
B.2 Yoneda's Lema . . . . . 436
B.3 The Slice Category . . . . . 437
B.4 Zorn's Lemma . . . . . 440

**Bibliography** **441**

# Chapter 1

# Preliminary Concepts

## 1.1 Introduction

This section describes basic concepts to establish an intuitive background that will be useful when providing formal definitions.

1. The syntactic nature of Type Theory. For a mathematician trained in classical structures (like groups or geometries), the definition of Homotopy Type Theory can seem unusual. Typically, mathematical structures are defined *axiomatically* or *descriptively*: we assume a background universe (usually ZFC Set Theory) and specify a list of properties that a collection of objects must satisfy (e.g., "A Projective Plane is a set of points and lines such that ..."). 

However, Type Theory is not a structure *within* mathematics; it is a *foundation for* mathematics. As such, it cannot rely on a pre-existing universe of sets. Instead, it must be defined *syntactically* or *generatively*, much like a programming language.

This approach relies on three components:

1. **The Alphabet:** A finite set of primitive symbols (e.g., variables $x$, $y$), separators (e.g., ':' , '(', ')'), and keywords like $\lambda, \Pi$).
2. **The Grammar (Syntax):** A set of structural rules that determine which strings of symbols are "well-formed" expressions.
3. **The Inference Rules (Logic):** A deductive system that defines how valid judgments can be derived from previous ones. For example, the rule for function application allows us to transition from having "a function $f$" and "an input $a$" to having "a term $f(a)$".

In this view, mathematical objects are not pre-existing entities that we capture with axioms; they are *constructions* that we build using the rules. A "proof" is

not an abstract truth value, but a valid derivation tree within this grammar.

Therefore, when we ask *What is a type in* HoTT?, the answer is not given by describing its properties in Set Theory, but by specifying the *inference rules* that govern how to construct it, how to construct its elements (introduction), and how to use them (elimination).

To put this generative process into action, the theory requires a set of *primitive objects* (or base types) to serve as the *seeds* of the universe. Just as a grammar needs terminal symbols to form sentences, Type Theory postulates specific starting points—such as the Empty type ($\mathsf{0}$), the Unit type ($\mathsf{1}$), and the Natural Numbers ($\mathbb{N}$).

These primitives are not constructed from simpler pieces; they are introduced by *fiat* through their own specific rules (Formation, Introduction, Elimination, Computation). They act as the *base cases* of the mathematical induction that builds the universe: once these primitives exist, the generative machinery (like $\Sigma$ and $\Pi$ types) can combine them to construct arbitrarily complex structures.

**The Problem: Runtime Errors vs. Mathematical Certainty.** Conventional programming languages permit runtime errors that are inconceivable in mathematical practice. A classic example is the "out-of-bounds" condition: an instruction like v[i] can fail because the type of i (e.g., integer) is too general to guarantee its value is within the valid domain of the array.

While software techniques like preconditions, runtime checks, or user-defined types can mitigate this, they do so at the cost of increased complexity or efficiency loss, without ever fully eliminating the problem.

This class of error is alien to mathematics because the validity of an expression is pre-established. An expression like $v_i$ is only well-formed if the index $i$ is known—explicitly or implicitly—to belong to the appropriate domain. In essence, every term in a mathematical expression carries with it the assumption or proof of its own applicability; the index $i$ is not just an integer, but a pair $(i, p)$ where $p$ justifies that $i$ is within bounds.

Type Theory seeks to integrate this mathematical property into the programming paradigm itself. However, the traditional approach based on Zermelo-Fraenkel (ZFC) set theory is not the best choice for this.

The reason is that ZFC operates in a two-layered system: a theory of *objects* (sets) and an *external* system of propositional logic used to make and prove claims *about* those objects. This separation introduces a complexity that scales poorly when building verified software.

Type Theory offers a more integrated solution by unifying these two concepts. In this paradigm, a *type* embodies both the object and the logic (the proposition). A term of that type is not just a value; it is a *witness* (or proof) that the

proposition it represents is true. We also say $a$ is an *element* of the type $A$ when $a\colon A$.

By constructing a term of a specific type, the construction itself *is* the proof that the term satisfies the required properties. This eliminates the problem of uncertain validity, not by checking for it, but by making it logically impossible to express. Since the proof of correctness is inherent in the construction of the terms, the solution is not an added burden that "scales" with software complexity—it *is* the paradigm itself.

## 1.2 Basic Notions

2. Atoms. Intuitively, *mathematical expressions* denoting *values* or *quantities* are composable under the grammar defined in a given theory. The building blocks of these expressions are *atomic*, while the well-formed expressions derived from them are *compound* (or *constructed*). Thus, $\emptyset$ is atomic and $\{\emptyset\}$ is compound.

3. Judgments. A *deductive system* is a set of *rules* that can be applied to derive *judgments*. For example, in the deductive system of first-order logic (on which Set Theory is based), there is only one kind of judgment: that a given proposition has a proof. Thus, in the deductive system of first-order, if $A$ and $B$ are propositions, then $A$ *has a proof* and $B$ *has a proof* are judgments and we can derive the judgment $A \wedge B$ *has a proof*.

The foundation of Type Theory is the deductive system. The theory operates fundamentally using a set of established *rules* for deriving judgments. The rules of Type Theory can be grouped into *type formers*. Each type former consists of a way to construct types (possibly making use of previously constructed types), together with rules for the construction and behavior of elements of that type (see 5. Witnesses).

4. Metatheory. The conceptual space where theorems about the deductive system are stated and proven is called the *metatheory*.

5. Witnesses. In Type Theory, the judgment $a\colon A$ is analogous to the notion that $A$ has a proof. This atomic expression is interpreted by saying that $a$ is a *witness* of the provability of the type $A$. Another analogy with Set Theory links $a\colon A$ with $a \in A$. However, $a \in A$ is a proposition not a judgment, whereas $a\colon A$ is a judgment, not a proposition.

The type-theoretic perspective treats proofs not merely as communicative means but as first-class mathematical objects in their own right, on par with familiar objects like numbers and functions. Calling the term $a$ a witness (or *evidence*) highlights that $a$ is a *construction*, consistent with the view of proofs as mathematical objects

The word "witness" helps maintain the distinction: $A$ *is provable* is a metatheo-

retic statement ($A$ is *inhabited*), while $a$ is the specific internal object providing the justification.

In summary, referring to $a$ as a *proof* is avoided. This emphasizes that $a$ is an object or construction (a witness) within the theory's types, not merely an abstract, metatheoretic declaration of provability.

Note also that in Type Theory, we cannot introduce a variable without specifying its type using an atomic judgment such as $a\colon A$.

6. Equalities. Since in Type Theory propositions are types, this means that equality is a type: for elements $a, b\colon A$ (that is, both $a\colon A$ and $b\colon A$) we have a type $a =_A b$. When $a =_A b$ is inhabited we say that $a$ and $b$ are *propositionally equal*.

The second concept of equality is that of *judgmental*, *computational*, or *definitional equality*, denoted by $a \equiv b\colon A$. This means that $a$ and $b$ are syntactically identical or reduce to the same form via the theory's *computation rules*, which allow automatic rewriting of expressions. For instance, if $f(x) = x^2$, then $f(3)$ and $3^2$ are equal by definition. The idea is that definitional equality is algorithmically decidable (though the algorithm is metatheoretic, not internal to the theory, i.e., is part of the implementation of the type checker in a computer language, not a function defined within the formal system itself).

The rule that links both concepts of equality establishes that $a\colon A$ and $A \equiv B$ allow us to derive $a\colon B$. In our example, if $p\colon 3^2 = 9$, then $p\colon f(3) = 9$.

We shall use $:\equiv$ to define a thing as (judgmentally) equal to another. For instance $f(x) :\equiv x^2$.

7. Precedence. Syntactically, the precedence of ':' and '$\equiv$' is the lowest in an expression. The reason is that both symbols are used for judgments, which cannot be put together into more complicated statements. Thus, $A \equiv x = y$ means $A \equiv (x = y)$ since $x = y$ is a type and a judgment (like $A \equiv x$) cannot be equal to anything.[1]

8. Assumptions. Judgments may depend on *assumptions* of the form $x\colon A$. The collection of assumptions is a *context*, which must be specified by an ordered list because any assumption might depend on a previous assumption in the list. For example $x, y\colon A$ and $p\colon x =_A y$ might form the context for $p^{-1}\colon y =_A x$. When the type involved represents a proposition, assumptions play the role of *hypotheses*.

9. Definitional substitution. Given that $x \equiv a$ is not a type, we cannot have it as an assumption. Instead we *substitute* $a\colon A$ *for* $x$ and obtain a more specific type or element. Even though not technically an assumption, the language "assume $x \equiv a$" will be used to refer to this process of substitution.

[1] In fact, judgments cannot be terms in any propositional equality.

10. PRINCIPLES. Some statements (whether a judgment, axiom, or theorem) that possess high generality and act as fundamental building blocks for reasoning within the theory are called *principles*. This is not a formal notion, but a pedagogical or communicational one aimed at underlying its importance in the deductive system.

Substitution (see above), for instance, is usually referred to as the *principle of substitution*. Note that this is a judgmental principle.

Other statements that correspond to rules, axioms or theorems are broadly called principles.

## 1.3 Function Types

11. FUNCTIONS. Given types $A$ and $B$, we can construct the type $A \to B$ of *functions with domain $A$ and codomain $B$*. We also sometimes refer to functions as *maps*. Unlike in Set Theory, functions are not defined as functional relations but as a primitive concept in Type Theory. We explain the function type by prescribing what we can do with functions, how to construct them and what equalities they induce. Note also that $f : A \to B$ stands for both the traditional notation and the type judgment. In both cases the expression is known as the *signature* of $f$.

Given a function $f : A \to B$ and an element of the domain $a : A$, we can *apply* the function to obtain an element of the codomain $B$, denoted $f(a)$ and called the *value of $f$ at $a$*. It is common in Type Theory to omit the parentheses and denote $f(a)$ simply by $f\,a$.

*Remark:* Functions are a primitive concept.

12. BINDERS. A *binder* (a.k.a. *variable binder*) is an *operator* $\nabla$ that can be applied to a variable $x$ of type $A$, and an expression $\Phi$ (possibly involving $x$), using the syntax

$$\nabla x : A.\, \Phi.$$

We say that $x$ is *bound* by $\nabla$ inside $\Phi$. A variable is *free* in an expression if it is not bound by any binder.

13. FUNCTION ABSTRACTION. There are two ways of defining a function. We define $f : A \to B$ by giving an equation

$$f(x) :\equiv \Phi$$

where $x$ is a (free) variable and $\Phi$ is an expression which may use $x$. In order for this to be valid, we have to check that $\Phi : B$ assuming $x : A$ (see 8. ASSUMPTIONS). For example, $f(x) :\equiv x^2$ defines a function $f : \mathbb{N} \to \mathbb{N}$, and so $f(3)$ is judgmentally equal to $3^2$.

We can also use the *$\lambda$-abstraction* binder

$$(\lambda x : A.\, \Phi) : A \to B.$$

For example, $(\lambda x\colon \mathbb{N}.\, x^2)\colon \mathbb{N} \to \mathbb{N}$ defines the same function as above. Another example is the *constant function* $(\lambda x\colon A.\, b)\colon A \to B$, where $b\colon B$. This function has the value $b$ at every $a\colon A$.

**Note.** The mathematical notation $f(x) = \Phi$, where $\Phi$ is an expression possibly involving $x$, binds the variable $x$ in the scope of $\Phi$. However, unlike other binders such as $\forall$, $\exists$, $\Pi$, $\Sigma$, etc., this one is named after the function. The $\lambda$-notation, *removes* this dependency, emphasizing the binding character of the entire expression.

14. Implicit notation. Whenever the type $A \to B$ is clear from the context, the $\lambda$-notation can be relaxed to $\lambda x.\, \Phi$. Another way to express the same is $(x \mapsto \Phi)\colon A \to B$.

15. Computation rule. Evaluation at $a$ takes the form

$$(\lambda x.\, \Phi)(a) \equiv \Phi',$$

where $\Phi'$ is obtained from $\Phi$ by substitution of $a$ for $x$ (see 9. Definitional substitution). This rule is also known as *$\beta$-conversion* or *$\beta$-reduction*.

16. Uniqueness principle for function types. This principle expresses that each function $f\colon A \to B$ is judgmentally equal to its $\lambda$-abstraction, i.e.,

$$f \equiv \lambda x.\, f(x),$$

meaning that $f$ is uniquely determined by its values. This expression is also known as *$\eta$-conversion* or *$\eta$-expansion*.

Note that this is an example of an *inference rule* (a judgment) that *qualifies* as a principle due to its importance and generality.

17. Bound variables. Consider the expression

$$f(x) :\equiv \lambda y.\, x + y.$$

Here the variable $y$ is *bound* to the scope of the expression in the rhs. It is, in other words, *dummy* in the sense that it can be replaced with another variable, say $z$, without changing the meaning of the expression. In particular,

$$f(x) \equiv \lambda z.\, x + z.$$

This allows us to perform the substitution of $y$ for $x$ in the first expression, by first using the second:

$$f(y) \equiv \lambda z.\, y + z.$$

This change of variable is intended to avoid the accidental *capture* of the substituted variable.

18. Curried form. If $f\colon A \times B \to C$ is a function, its *Curried form* is the function $g\colon A \to (B \to C)$ defined as $g(a)(b) \equiv f(a, b)$. The equivalence between

a function and its Curried form is extended inductively to the case where the domain is $A_1 \times \cdots \times A_n$.

In combination with $\lambda$-abstractions, for $f\colon A \to B \to C$ given by

$$f(x, y) :\equiv \Phi$$

where $\Phi\colon C$, assuming $x\colon A$ and $y\colon B$, corresponds to

$$f :\equiv \lambda x.\, \lambda y.\, \Phi.$$

From the viewpoint of Logic, Currying is the transformation of

$$A \wedge B \implies C$$

into

$$A \implies (B \implies C).$$

## 1.4 Dependent Function Types

19. Universes. As every term must have a type, types must have a type as well. To avoid paradoxical situations derived from the global universe, there is a family of universes $\mathcal{U}_0$, $\mathcal{U}_1$, $\mathcal{U}_2, \ldots$, such that $\mathcal{U}_i\colon \mathcal{U}_{i+1}$ for all $i \geq 0$.[2] Thus, $\mathcal{U}_0$ is the universe of *small* types, such as $\mathbb{N}$, etc., while $\mathcal{U}_{i+1}$ is the universe of types in the universe $\mathcal{U}_i$.[3]

Since there is no universe $\mathcal{U}_\infty$, the expression $\lambda i\colon \mathbb{N}.\, \mathcal{U}_i$ does not define a function (otherwise, its codomain would be a global universe). More precisely, no rule permits the definition of a function where the codomain depends on the value of the input in a way that transcends the universe hierarchy.

20. The cumulative property. The family of universes is *cumulative*, meaning that if $A\colon \mathcal{U}_i$, then $A\colon \mathcal{U}_{i+1}$. In particular, if $a$ is a witness of $A$ in $\mathcal{U}_{i+1}$, it is also a witness of $A$ in $\mathcal{U}_i$.

The following example illustrates the reason behind this rule.

**Example 1.4.1.** Consider a type constructor $\mathsf{L}$ that forms the type of lists containing elements of a specific type. If we want to create a list of types, we must determine the universe in which this list lives.

- Let $\mathbb{N}\colon \mathcal{U}_0$ and $\mathsf{Bool}\colon \mathcal{U}_0$.
- We can form the list $[\mathbb{N}, \mathsf{Bool}]$. The type of this list is $\mathsf{L}_{\mathcal{U}_0}$.
- However, since $\mathcal{U}_0\colon \mathcal{U}_1$, the type $\mathsf{L}_{\mathcal{U}_0}$ itself resides in $\mathcal{U}_1$.

[2] See also § B.1.

[3] The indexing here is distinct from the formal type of natural numbers. It is a meta-linguistic device; any finite alphabet containing symbols like '$\mathcal{U}$' and a successor mark such as '$'$' would suffice, allowing us to write sequences like $\mathcal{U}\colon \mathcal{U}'\colon \mathcal{U}''$ of arbitrary depth.

Now, suppose we have a more complex type $B$ such that $B : \mathcal{U}_1$. If we wish to construct a list containing both $\mathbb{N}$ and $B$, we face a potential universe mismatch. To include $\mathbb{N}$ in a list of type $\mathsf{L}_{\mathcal{U}_1}$, we must view $\mathbb{N}$ as an element of $\mathcal{U}_1$.

Therefore, the construction of the list $[\mathbb{N}, B]$ relies on the cumulative property:

$$\mathbb{N} : \mathcal{U}_0 \implies \mathbb{N} : \mathcal{U}_1.$$

This ensures that both $\mathbb{N}$ and $B$ are valid terms of type $\mathcal{U}_1$, making the expression $[\mathbb{N}, B]$ well-typed as a term of $\mathsf{L}_{\mathcal{U}_1}$. □

**Note.** To simplify the notation, the subindex $i$ is usually omitted and $\mathcal{U}$ is used instead of $\mathcal{U}_i$. When some universe $\mathcal{U}$ is assumed, its types are also called *small*.

21. Type Families. A *type family* is a function $B : A \to \mathcal{U}$ whose codomain is a universe.

The intuition behind a type family is that it provides a collection of types $B(x)$ indexed by the terms of a given type $A$. Defining this family as a function into $\mathcal{U}$ ensures that the resulting types do not escape the range of some predefined universe. For instance, we cannot define a family such as $\lambda n.\mathcal{U}_n$ for $n : \mathbb{N}$. Although this intuitively assigns a type to each natural number, the resulting types grow arbitrarily large and cannot be contained within any single, fixed universe $\mathcal{U}_k$. Therefore, it cannot be typed as a function $\mathbb{N} \to \mathcal{U}_k$.

There are two important examples. The first one is the *constant* type family defined by

$$(\lambda x.\, B) : A \to \mathcal{U}$$

under the assumption $B : \mathcal{U}$. The second is the family of *finite* sets[4]

$$\mathsf{Fin} : \mathbb{N} \to \mathcal{U},$$

where $\mathsf{Fin}(0)$ is the empty type (it has no elements), $\mathsf{Fin}(1)$ is a type with exactly one element[5], and so on. By its cumulative character (see 19. Universes), $\mathsf{Fin}(n)$ has $n$ elements denoted by $0_n, \ldots, (n-1)_n$. The idea behind this definition is to have integers $i$ equipped with a proof that $0 \leq i < n$.

The $\mathsf{Fin}$ function is relevant to define a function like

$$\mathsf{getElem} : \mathsf{Vect}\, A\, n \to \mathsf{Fin}(n) \to A,$$

whose second argument is guaranteed to be in the range $[\![0, n-1]\!]$.

22. Dependent functions (a.k.a. Π-types). Given a type $A : \mathcal{U}$ and a family of types $B : A \to \mathcal{U}$, we can construct the type of *dependent functions*

$$\prod_{x:\, A} B(x) : \mathcal{U}$$

[4] Formally introduced in Exercise 1.15.6.
[5] See also 25. Unit type.

using the binder $\Pi x.\, B(x)$ (see 12. BINDERS).

This construction generalizes the ordinary function type $A \to B$, which is recovered by choosing the constant family $\lambda x\colon A.\, B$. This yields the definitional equality

$$\left(\prod_{x\colon A} B\right) \equiv A \to B. \tag{1.1}$$

As with ordinary functions, applying a dependent function $f\colon \prod_{x\colon A} B(x)$ to an argument $a\colon A$ yields an element $f(a)\colon B(a)$. This application satisfies the following computation rule: if $a\colon A$ and $f :\equiv \lambda x.\, \Phi$, then both $f(a) \equiv \Phi'$ and $(\lambda x.\, \Phi)(a) \equiv \Phi'$. Here, the term $\Phi'$ is the expression obtained by replacing all free occurrences of $x$ in $\Phi$ with $a$, performing appropriate renaming to avoid variable capture (see 17. BOUND VARIABLES).[6]

An example that will become useful later is the dependent function[7]

$$\mathsf{fmax}\colon \prod_{n\colon \mathbb{N}} \mathsf{Fin}(\mathsf{succ}(n))$$

that selects the last element of each nonempty finite type (see 21. TYPE FAMILIES)

$$\mathsf{fmax}(n) :\equiv n_{n+1}.$$

**Note.** When not limited by parentheses, the scope of $\Pi$ extends over the rest of the expression. In particular,

$$\prod_{x\colon A} A \to A \quad \text{stands for} \quad \prod_{x\colon A} (A \to A).$$

23. POLYMORPHIC FUNCTIONS. A *polymorphic* function is one which takes a type as one of its arguments, and then acts on elements of that type (or of other types constructed from it).

An example is the *polymorphic identity* function

$$\mathsf{id} \equiv \prod_{A\colon \mathcal{U}} A \to A, \quad \mathsf{id} :\equiv \lambda A\colon \mathcal{U}.\, \lambda x\colon A.\, x,$$

or $\mathsf{id}_A(x) :\equiv x$ for short.

When using the Curried form to define dependent functions with several arguments, the second domain may depend on the first one, and the codomain may depend on both. That is, given $A\colon \mathcal{U}$ and type families $B\colon A \to \mathcal{U}$ and $C\colon \prod_{x\colon A} B(x) \to \mathcal{U}$, we may construct the type

$$\prod_{x\colon A} \prod_{y\colon B(x)} C(x, y)$$

[6] This capture-avoiding substitution is usually denoted by $\Phi[a/x]$, meaning $f(a) \equiv \Phi[a/x]$.
[7] Exercise 1.15.6.

of functions with two arguments.

In the case where $B$ is constant and equal to $A$, we may condense the notation and write $\prod_{x,y:\, A} C(x, y)$.

Consider, for instance, the function

$$\mathsf{swap}\colon \prod_{A,B,C:\,\mathcal{U}} (A \to B \to C) \to (B \to A \to C)$$

defined as

$$\mathsf{swap}(A, B, C, f) :\equiv \lambda b.\, \lambda a.\, f(a, b)$$

that swaps the order of the arguments and may also be denoted by

$$\mathsf{swap}_{A,B,C}(f)(b, a) \equiv f(a, b).$$

Note that given $f\colon \prod_{x:\, A} \prod_{y:\, B(x)} C(x, y)$, we have $f(a, b)\colon C(a, b)$.

## 1.5 Product Types

24. Products. Given types $A, B\colon \mathcal{U}$ we introduce their *cartesian product* $A \times B\colon \mathcal{U}$.

For now, we only declare $A \times B$ as a (new) type, and specify that all pairs $(a, b)$ with $a\colon A$ and $b\colon B$ satisfy $(a, b)\colon A \times B$. In 28. Products revisited, we will further establish the additional rules for defining this new type.

Even though the pairs $(a, b)$ are of type $A \times B$, the definition of this type makes no attempt at specifying that all the elements of $A \times B$ are pairs (as is the case in Set Theory). By contrast, we will derive this fact as a theorem.

25. Unit type. The *unit type* is the nullary product type (i.e., the cartesian product of zero types), denoted by $\mathsf{1}\colon \mathcal{U}$. The *canonical* element is $\star\colon \mathsf{1}$.

While this definition is consistent with $\mathsf{Fin}(1)$, a fundamental difference is that the unit type does not rely on $\mathbb{N}$. In particular, it does not depend on the definition of the entire $\mathsf{Fin}$ family.

26. Type specification. When introducing a new type, we must specify the following

i) *Formation rules.* Indicate how to form new types. Example: $A \to B$, $\prod_{x:\, A} B(x)$, etc.

ii) *Constructors.* Define how to introduce elements of that type. Example: $f(x) :\equiv x^2$, $\lambda x.\, x^2$, for $f\colon A \to B$ and $x\colon A$.

III) *Eliminators.*[8] These rules specify how to use (or operate on) elements of the type. Example: function application $f(a)$.

IV) *Computation rules.* Also known as $\beta$-reductions, prescribe how eliminators operate on constructors. Example: $(\lambda x\colon A.\, \Phi)(a) \equiv \Phi'$, where $a\colon A$, and $\Phi'$ is obtained from $\Phi$ by substitution of $a$ for $x$.

V) *Uniqueness principle*$^*$ (optional). Also known as $\eta$-expansions, express how constructors act on eliminators, dually to the computation rule. Example: $f \equiv \lambda x.\, f(x)$.

Sometimes, the uniqueness principle is propositional, meaning that the element is propositionally equal to another element obtained from other rules for the type. Similarly, some types include *propositional* computation rules.

In some cases the uniqueness principle indicates that it is not required to specify $f\colon A \to B$ on all elements of type $A$, and that it suffices to specify it on all the constructors of $A$.

27. CANONICAL ELEMENTS. While a constructor is the rule that enables the creation of new elements of a type, an element formed by the direct application of such a rule is called *canonical*. For instance, the constructor for the type $A \times B$ is the rule that creates the pair $(a, b)\colon A \times B$ from $a\colon A$ and $b\colon B$. The pair $(a, b)$ itself is a canonical element of $A \times B$.

28. PRODUCTS REVISITED. Type specification for products works as follows:

I) *Formation rules.* Given two types $A$ and $B$ we can form $A \times B$. Moreover, given zero types we have the unit type $\mathbf{1}$.

II) *Constructors.* Given $a\colon A$ and $b\colon B$ we have $(a, b)\colon A \times B$. In addition, $\star$ is the unique element of type $\mathbf{1}$.

III) *Eliminators.* If $f\colon A \times B \to C$, the prescription for evaluating $f$ is given by providing a function $g\colon A \to B \to C$ and defining

$$f((a, b)) :\equiv g(a)(b),$$

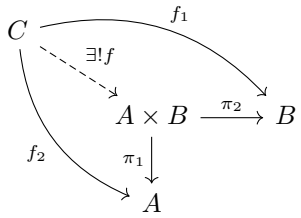
where $a\colon A$ and $b\colon B$.[9] Note how this rule *deconstructs* the pair $(a, b)$ into its coordinates, operating in the reverse direction of the constructor.

IV) *Computation rule.* Let us first recall that the universal property of the

[8] The term *elimination* refers to the syntactic transformation from premises to conclusion. In formal logic, when comparing the Abstract Syntax Trees (AST) of the judgments, the *main descriptor* (or root node) of the type being eliminated is replaced by the descriptor characteristic of the conclusion's type.

[9] This does not assume that all elements with type $A \times B$ are necessarily pairs. It only specifies that function evaluation is defined by how it works on pairs.

product is summarized in the following commutative diagram




(1.1)

However, this definition relies on the existence of the projections $\pi_1$ and $\pi_2$. Therefore, we need to define them first. Consider the Π-type

$$\prod_{C:\mathcal{U}} A \to B \to C.$$

It represents the type of all functions $f$ satisfying $\mathrm{dom}(f) \equiv A \times B$, of which $\pi_1$ and $\pi_2$ are instances. So, we can define the *recursor* function

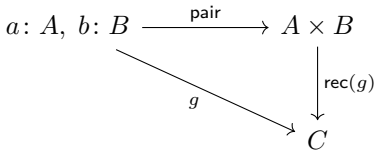
$$\mathsf{rec}_{A\times B}\colon \prod_{C:\mathcal{U}} (A \to B \to C) \to A \times B \to C \tag{1.2}$$

specified as

$$\mathsf{rec}_{A\times B}(C, g)((a, b)) :\equiv g(a)(b), \tag{1.3}$$

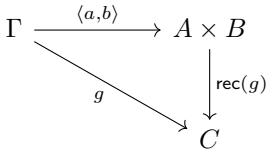
where we are using the uniqueness principle that establishes that, to specify a function $f\colon A \times B \to C$, it is enough to specify it on the pairs $(a, b)$ (see 26. TYPE SPECIFICATION V).

In a diagram[10]




where $\mathsf{rec}$ stands for $\mathsf{rec}_{A\times B}(C)$.

If we introduce the context $\Gamma :\equiv (a\colon A,\ b\colon B)$, the diagram becomes




The first diagram is *functional*: it depicts the constructor $\mathsf{pair}$ combining inputs to form the product, and how $\mathsf{rec}$ factors the Curried function $g$.

[10] Strictly speaking, the domain of the Curried function $g\colon A \to B \to C$ does not appear as a single object in the diagram, so we represent the input context explicitly.

The second is *categorical*: it treats the inputs as a context object $\Gamma$, where the arrow $\langle a, b\rangle$ represents the specific term (substitution) in that context mapping to the product.

We can derive the projections as

$$\pi_1 :\equiv \mathsf{rec}_{A\times B}(A.\, \lambda a.\, \lambda b.\, a)$$
$$\pi_2 :\equiv \mathsf{rec}_{A\times B}(B.\, \lambda a.\, \lambda b.\, b),$$

which satisfy $\pi_1((a, b)) \equiv a$ and $\pi_2((a, b)) \equiv b$, and we can consider them the computation rules associated with the product.

More generally, the function $\mathsf{rec}_{A\times B}$, called the *recursor* for product types, allows us to define functions of type $A \times B \to C$, a property referred to as the *recursion principle*.

Note that the judgment

$$\mathsf{rec}_{A\times B}(C, g) \equiv \lambda x.\, g(\pi_1(x))(\pi_2(x))$$

shows that $\mathsf{rec}$ is also derivable from the projections.

The viewpoint under which $\mathsf{rec}$ is introduced is not the one expressed in the universal property of the product (1.1), but the fact that the functors $F_B(A) = A \times B$ and $G_B(C) = \mathrm{Hom}(B, C)$ satisfy $F_B \dashv G_B$, i.e., $\mathrm{Hom}(F_B(A), C) \cong \mathrm{Hom}(A, G_B(C))$. In this regard, the universal property leads us to view the product as a function codomain, while the adjoint relation leads us to consider it as a domain.

The version of $\mathsf{rec}$ for the empty product type $\mathsf{1}$ (where $A$ and $B$ are missing) is

$$\mathsf{rec}_{\mathsf{1}} \colon \prod_{C:\mathcal{U}} C \to \mathsf{1} \to C$$

specified as

$$\mathsf{rec}_{\mathsf{1}}(C, c) :\equiv \lambda s.\, c.$$

Since every $g \colon \mathsf{1} \to C$ is equivalent to choosing one element $c \colon C$, we have

$$\mathsf{rec}_{\mathsf{1}}(C, c)(\star) \equiv c.$$

29. Recursion significance. While projections and recursion are mutually definable for product types, this equivalence does not hold in general. As we will see, recursion is the more fundamental concept; in fact, it is a primitive of Type Theory. For instance, even though $\mathsf{rec}_{\mathsf{1}}$ is well-defined, the Unit type (the empty product) has no projections.

In colloquial terms, while projections merely extract the parts so that we may operate on them, recursion asks what calculation we wish to perform *using* the parts, and then carries it out.

In Object-Oriented Programming, recursion is mediated by encapsulation: an object exposes methods that recursively delegate work to the methods of its

constituent parts. In Structured Programming, by contrast, processing operates directly on the data stored in record fields rather than through behavior exported by the data itself.

Note also that $\mathsf{rec}$ implements *Uncurrying* [cf. 18. CURRIED FORM], namely

$$\mathrm{Hom}(A, B \to C) \to \mathrm{Hom}(A \times B, C).$$

In Logic, this corresponds to the transformation

$$A \implies (B \implies C) \quad \text{into} \quad A \wedge B \implies C.$$

**Note.** More formally, the definition of every new type $T : \mathcal{U}$ includes a *recursion map*, usually denoted by $\mathsf{rec}_T$. This map generates functions of type $T \to C$ for any result type $C : \mathcal{U}$, based on values specified for the canonical elements of $T$.

For example, $\mathsf{rec}_{A\times B}$ assigns to every type $C : \mathcal{U}$ a function $A \times B \to C$ for each curried function $g : A \to B \to C$, as indicated in (1.2), which satisfies the computation rule (1.3).

As we shall see [cf. 32. INDUCTION], an analogous mechanism must be provided for dependent functions with domain $T$; this is the role of the *induction map*.

30. IDENTITY TYPES. Given a type $A$ and terms $x, y : A$ we can form the type $x =_A y$. If the type $x =_A y$ is inhabited (has a term), the equality is true. If the type $x =_A y$ is empty (has no terms), the equality is false.

The type for equality is called the *identity type*, written $\mathsf{Id}_A(x, y)$ or $x =_A y$. So, $\mathsf{Id}_A(x, y)$ represents the *type of all proofs that $x$ equals $y$*.

The *unique* canonical term inhabiting the identity type $\mathsf{Id}_A(x, x)$ is $\mathsf{refl}_x$. Denoted as $\mathsf{refl}_x : \mathsf{Id}_A(x, x)$, this term witnesses that $x$ is equal to itself.

The fact that $\mathsf{Id}_A(x, x)$ has a unique canonical witness is the key to the identity type's elimination rule (see 59. GENERAL PATH INDUCTION), which states that to prove something for all equalities $p : x =_A y$, we only need to prove it for the case $p \equiv \mathsf{refl}_x$. Note however that this does not mean that $\mathsf{refl}_x$ is the only element of $\mathsf{Id}(x, x)$; in fact, there are cases where there are infinitely many, as well as cases where there is only one.

31. DEPENDENT FUNCTIONS OVER PRODUCT. According to the specification of Π-types, given $A, B : \mathcal{U}$ and a type family $C : A \times B \to \mathcal{U}$, a dependent function

$$f : \prod_{z:\, A\times B} C(z)$$

is defined by means of a function

$$g : \prod_{x:\, A} \prod_{y:\, B} C((x, y))$$

as

$$f((x, y)) :\equiv g(x)(y).$$

Thus, $f$ is well defined on (any element of) $A \times B$ as soon as its values are specified in the *canonical* elements of the product, i.e., the pairs.

Since, given $(a, b) \colon A \times B$, we have (see 28. PRODUCTS REVISITED)

$$\pi_1((a, b)) \equiv a \quad \text{and} \quad \pi_2((a, b)) \equiv b,$$

we deduce that

$$\big(\pi_1((a, b)), \pi_2((a, b))\big) \equiv (a, b).$$

In consequence, the types

$$(a, b) =_{A\times B} (a, b) \quad \text{and} \quad \big(\pi_1((a, b)), \pi_2((a, b))\big) =_{A\times B} (a, b)$$

are judgmentally equal. Since $\mathsf{refl}_{A\times B}$ inhabits the former, it also inhabits the latter, i.e.,

$$\mathsf{refl}_{(a,b)} \colon (\pi_1((a, b)), \pi_2((a, b))) =_{A\times B} (a, b)$$

Given that to construct a witness for the uniqueness principle

$$\mathsf{uniq}_{A\times B} \colon \prod_{x\colon A\times B} (\pi_1(x), \pi_2(x)) =_{A\times B} x.$$

it suffices to specify it on canonical elements, the definition

$$\mathsf{uniq}_{A\times B}((a, b)) :\equiv \mathsf{refl}_{(a,b)}$$

meets this requirement.

**Note.** This function $\mathsf{uniq}_{A\times B}$ is a witness (formal proof) of the uniqueness principle for products (see 16. UNIQUENESS PRINCIPLE FOR FUNCTION TYPES). As we will see in 32. INDUCTION below, this fulfills the claim from 28. PRODUCTS REVISITED that all terms of type $A \times B$ are (propositionally) pairs.

In the case of $\mathsf{1}$, we have

$$\mathsf{uniq}_{\mathsf{1}} \colon \prod_{x\colon \mathsf{1}} x =_{\mathsf{1}} \star$$

with

$$\mathsf{uniq}_{\mathsf{1}}(\star) :\equiv \mathsf{refl}_{\star},$$

which shows that $\star$ is the unique element of $\mathsf{1}$.

32. INDUCTION. The *principle* by which defining a function on a product $A \times B$ amounts to specifying its value on each pair, can be formalized by introducing the induction map

$$\mathsf{ind}_{A\times B} \colon \prod_{C\colon A\times B\to\mathcal{U}} \Big(\prod_{x\colon A} \prod_{y\colon B} C(x, y)\Big) \to \prod_{z\colon A\times B} C(z),$$

given by

$$\mathsf{ind}_{A\times B}(C)(g)\big((a,b)\big) :\equiv g(a)(b),$$

or, in abbreviated form,

$$\mathsf{ind}_{A\times B}(C,g,(a,b)) :\equiv g(a)(b).$$

**Interpretation.** The function $\mathsf{ind}_{A\times B}$ transforms any proof that a proposition $C$ holds for all canonical pairs $(x,y)$ into a proof of $C(z)$ for an arbitrary element $z\colon A\times B$.

When the family $C$ is constant we obtain

$$\prod_{x\colon A}\prod_{y\colon B} C((x,y)) \equiv A\to B\to C \quad \text{and} \quad \prod_{z\colon A\times B} C(z) \equiv A\times B\to C,$$

Hence, $\mathsf{ind}_{A\times B}$ becomes $\mathsf{rec}_{A\times B}$. Consequently, induction is the (*dependent*) *eliminator* and recursion the *non-dependent eliminator*.

In the case of the unit type (see 25. UNIT TYPE), where types $A$ and $B$ are both missing, the eliminator is

$$\mathsf{ind}_{\mathsf{1}}\colon \prod_{C\colon \mathsf{1}\to\mathcal{U}} \Big(C(\star)\to \prod_{x\colon \mathsf{1}} C(x)\Big) \tag{1.4}$$

specified by

$$\mathsf{ind}_{\mathsf{1}}(C,g,\star) :\equiv g,$$ [11]

**Interpretation.** Given $C\colon \mathsf{1}\to\mathcal{U}$ and a proof $g$ that $C(\star)$ is true, the resulting term $\mathsf{ind}_{\mathsf{1}}(C,g)$ —i.e., $\mathsf{ind}_{\mathsf{1}}(C)(g)$— is a dependent function $f\colon \prod_{x\colon \mathsf{1}} C(x)$ that serves as a proof for $C(x)$ for all $x\colon \mathsf{1}$.

In summary, we have:

I) *Type:* There is a type called $\mathsf{1}$.

II) *Constructor:* There is a term $\star\colon \mathsf{1}$.

III) *Eliminator:* There is a function $\mathsf{ind}_{\mathsf{1}}$ with type signature (1.4).

IV) *Computation Rule:* $\mathsf{ind}_{\mathsf{1}}(C,g,\star) :\equiv g$.

If for $C\colon \mathsf{1}\to\mathcal{U}$ we take

$$C :\equiv \lambda x.\,(x =_{\mathsf{1}} \star)$$

and for $g\colon C(\star)$ we take $\mathsf{refl}_\star$, then the result is

$$\mathsf{ind}_{\mathsf{1}}((\lambda x.\, x =_{\mathsf{1}} \star), \mathsf{refl}_\star, \star) \equiv \mathsf{refl}_\star,$$

which is judgmentally equal to $\mathsf{uniq}_{\mathsf{1}}(\star)$ (see 31. DEPENDENT FUNCTIONS OVER PRODUCT). Thus,

$$\mathsf{ind}_{\mathsf{1}}\big(\lambda x.\, x =_{\mathsf{1}} \star,\ \mathsf{refl}_\star\big) \equiv \mathsf{uniq}_{\mathsf{1}}. \tag{1.5}$$

[11] A more precise notation is $\mathsf{ind}_{\mathsf{1}}(C)(g)(\star) :\equiv g$.

Note that $\mathsf{ind}_{\mathbf{1}}$ is strictly more general than $\mathsf{uniq}_{\mathbf{1}}$, since it is defined for an arbitrary family $C\colon \mathbf{1} \to \mathcal{U}$. Equation (1.5) shows that $\mathsf{uniq}_{\mathbf{1}}$ is simply a specific instance of $\mathsf{ind}_{\mathbf{1}}$. Thus the fact that $\mathbf{1}$ is a singleton type is subsumed by the more general induction principle.

33. UNIVERSE POLYMORPHISM The principle that mathematical constructions and elimination rules apply uniformly across the entire universe hierarchy, from the smallest upward, is called *universe polymorphism*.

This principle combines with the convention of typical ambiguity, by which universe indices are systematically suppressed.

To handle this formally, the primitive type $\mathsf{Level}$ is introduced to index the universes. Thus, when induction and recursion principles are stated for the universe $\mathcal{U}$, the simplified quantification

$$\prod_{C:\mathcal{U}} \cdots$$

must be understood as

$$\prod_{i:\,\mathsf{Level}} \prod_{C:\mathcal{U}_i} \cdots$$

By quantifying over a level $i\colon \mathsf{Level}$, we ensure that every induction and recursion principle holds for all universes. This allows us to apply standard structural principles to target types residing in any universe $\mathcal{U}_i$.

## 1.6 Dependent Pair Types

34. DEPENDENT PAIRS (A.K.A. $\Sigma$-TYPES). In Set Theory the *disjoint union* of a family $(A_i)_{i\in I}$ of sets is defined as

$$\bigsqcup_{i\in I} \{i\} \times A_i,$$

where the indexes in the first coordinate ensure that

$$\{i\} \times A_i \cap \{j\} \times A_j = \emptyset$$

whenever $i \neq j$, even though $A_i$ and $A_j$ might overlap (or coincide).

The equivalent notion in Type Theory is the *dependent pair type* (a.k.a. $\Sigma$-*type*), which highlights the dependence of the second component on the first one. More precisely, given a type $A\colon \mathcal{U}$ and a type family $B\colon A \to \mathcal{U}$, the binder $\Sigma$ is used to define the *dependent pair* type

$$\Big(\sum_{x:\,A} B(x)\Big)\colon \mathcal{U}.$$

As it is the case with $\lambda$ and $\Pi$, this binder also scopes over the rest of the expression, e.g.,

$$\sum_{x\colon A} B(x) \to C \quad \text{stands for} \quad \sum_{x\colon A} (B(x) \to C).$$

The constructor takes $a\colon A$ and $b\colon B(a)$ and produces

$$\langle a, b\rangle\colon \sum_{x\colon A} B(x).$$

In this regard, $\Sigma$-types resemble the existential binder $\exists$ of Set Theory in that its elements $\langle a, b\rangle$ consist of a first component $a$ and a second $b$, which is a proof of $B(a)$. In particular, in the case where $B$ is constant, we have

$$\sum_{x\colon A} B \equiv A \times B.$$

35. Generalized Currying. Recall from 18. Curried form that standard Currying establishes an equivalence

$$(A \times B) \to C \iff A \to B \to C.$$

In the context of dependent types, the $\Sigma$-type acts as the generalization of the Cartesian product (see 34. Dependent pairs (a.k.a. $\Sigma$-types)).

Consequently, the Currying principle extends naturally to this setting: mapping out of a dependent sum is equivalent to mapping out of the components sequentially.

Specifically, a function

$$f\colon \Big(\sum_{x\colon A} B(x)\Big) \to C$$

is equivalent to a dependent function

$$\hat{f}\colon \prod_{x\colon A} B(x) \to C,$$

where the scope of the binder $\Pi$ includes $C$.

To create a nondependent function

$$f\colon \Big(\sum_{x\colon A} B(x)\Big) \to C$$

we have to provide a function

$$g\colon \prod_{x\colon A} B(x) \to C$$

and specify

$$f(\langle a, b\rangle) :\equiv g(a)(b).$$

36. Induction in Σ-types. The first projection of a Σ-type can be defined as

$$\pi_1 \colon \Big(\sum_{x\colon A} B(x)\Big) \to A$$

with specification

$$\pi_1(\langle a, b\rangle) :\equiv a,$$

which, with the notation of 34. Dependent pairs (a.k.a. Σ-types), corresponds to

$$g(a)(b) :\equiv a.$$

However, the second projection is the dependent function

$$\pi_2 \colon \prod_{p\colon \sum_{x\colon A} B(x)} B(\pi_1(p)) \tag{1.1}$$

specified as

$$\pi_2(\langle a, b\rangle) :\equiv b.$$

More generally, we need the *induction principle* of Σ-types, which says that, given a type family

$$C \colon \sum_{x\colon A} B(x) \to \mathcal{U},$$

to define a dependent function

$$f \colon \prod_{p\colon \sum_{x\colon A} B(x)} C(p)$$

we must provide a function

$$g \colon \prod_{a\colon A} \prod_{b\colon B(a)} C(\langle a, b\rangle)$$

and specify

$$f(\langle a, b\rangle) :\equiv g(a)(b).$$

Thus, in the case where $C(p) :\equiv B(\pi_1(p))$, we can take

$$g(a)(b) :\equiv b$$

for $\langle a, b\rangle \colon \sum_{x\colon A} B(x)$, and obtain (1.1). This is valid because

$$C(\langle a, b\rangle) \equiv B(\pi_1(\langle a, b\rangle)) \equiv B(a)$$

and $b \colon B(a)$.

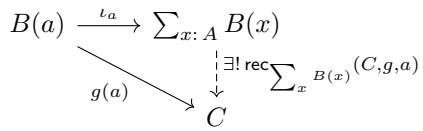
37. Recursion in $\Sigma$-types. Consider the recursion

$$\mathsf{rec}_{\sum_{x:A} B(x)} : \prod_{C:\mathcal{U}} \Big( \prod_{x:A} B(x) \to C \Big) \to \Big( \sum_{x:A} B(x) \Big) \to C$$

specified as

$$\mathsf{rec}_{\sum_{x:A} B(x)}(C, g, \langle a, b \rangle) :\equiv g(a)(b), \tag{1.2}$$

This can be represented by a collection of diagrams that commute for all $a : A$




where $\iota_a :\equiv \lambda y : B(a).\, \langle a, y \rangle$.

For the induction principle

$$\mathsf{ind}_{\sum_{x:A} B(x)} : \prod_{C:\left(\sum_{x:A} B(x)\right) \to \mathcal{U}} \Big( \prod_{a:A} \prod_{b:B(a)} C(\langle a, b \rangle) \Big) \to \prod_{p:\sum_{x:A} B(x)} C(p)$$

where

$$\mathsf{ind}_{\sum_{x:A} B(x)}(C, g, \langle a, b \rangle) :\equiv g(a)(b).$$

Note that when $C$ is constant $\mathsf{ind}$ reduces to $\mathsf{rec}$.

**Interpretation.** To prove $C(p)$ for every $p : \sum_{x:A} B(x)$ it suffices to show a proof of $C(\langle a, b \rangle)$ for every canonical pair $\langle a, b \rangle$.

38. Relations. Let $A$ and $B$ be types, $a : A$ and $R : A \to B \to \mathcal{U}$. The type

$$\sum_{y:B} R(a, y)$$

represents the fiber at $a$ of the relation $R$. Its elements are pairs $\langle b, \omega \rangle$ where $b : B$ and $\omega : R(a, b)$ is a witness that $b$ is related to $a$.

For instance, let $R(2, n)$ stand for the type representing the relation "$2 \mid n$", where $n : \mathbb{N}$. The elements of the type $\sum_{n:\mathbb{N}} R(2, n)$ represent the even numbers paired with the evidence of their evenness. Thus, if $\omega$ is a witness that $2 \cdot 3 =_{\mathbb{N}} 6$, we have

$$\langle 6, \omega \rangle : \sum_{n:\mathbb{N}} R(2, n).$$

However, there is no term $\omega$ for which $\langle 3, \omega \rangle : \sum_{n:\mathbb{N}} R(2, n)$, because the type $R(2, 3)$ is uninhabited (there is no proof of $2 \mid 3$).

39. Type-theoretic Axiom of Choice. Now consider the following principle, where $A$ and $B$ are types and $R : A \to B \to \mathcal{U}$:

$$\mathsf{ac} : \Big( \prod_{x:A} \sum_{y:B} R(x, y) \Big) \to \Big( \sum_{f:A \to B} \prod_{x:A} R(x, f(x)) \Big).$$

To specify it, we assume a witness $g$ for the domain:

$$g\colon \prod_{x\colon A}\sum_{y\colon B} R(x,y).$$

For $x\colon A$, the term $g(x)$ is a pair $\langle y,\omega\rangle$ where $y\colon B$ and $\omega\colon R(x,y)$. We can therefore define the *choice function* $f\colon A\to B$ as

$$f :\equiv \lambda x.\, \pi_1(g(x)).$$

Since $\pi_2(g(x))\colon R(x,\pi_1(g(x)))$, we obtain $\pi_2(g(x))\colon R(x,f(x))$. In addition,

$$\omega :\equiv \lambda x.\, \pi_2(g(x)).$$

Finally, we specify

$$\mathsf{ac} :\equiv \langle \overbrace{\lambda x.\, \pi_1(g(x))}^{f\equiv}, \overbrace{\lambda x.\, \pi_2(g(x))}^{\omega\equiv}\rangle.$$

**Note.** Even though both the conventional existence binder $\exists\, y\, R(x,y)$ and the dependent pair type binder $\sum_y R(x,y)$ express the existence of an element $y$, their semantics differ. While the former predicates the mere existence of $y$, the latter identifies pairs $\langle y,w\rangle$, where $w$ *proves* $R(x,y)$. Thus, in Type Theory, when we evaluate the antecedent function $g$ at $x$, the result $g(x)\equiv\langle y,w\rangle$ already contains the element $y$ whose existence is claimed. Consequently, the *choice* is trivial in this setting. This explains why, even though the translation to Logic reads as the classical Axiom of Choice, the Type Theoretic version is a theorem whose proof is a direct consequence of the definitions.

Incidentally, there is a backward map defined by

$$\langle f,h\rangle :\equiv \lambda x.\, \langle f(x),h(x)\rangle,$$

which is well-typed because $f(x)\colon B$ and $h(x)\colon R(x,f(x))$.

40. Mathematical structures. Many mathematical structures are compositions of the simplest one, called magma, that consists of a set $A$ and an operation $m\colon A\times A\to A$. In Type Theory notation the pair $\langle A,m\rangle$ is an element of a dependent pair type

$$M :\equiv \sum_{A\colon \mathcal{U}} A\to A\to A.$$

In more specific cases, such as pointed magmas, where a distinguished element $e$ must be identified, we would have,

$$PM :\equiv \sum_{A\colon \mathcal{U}} (A\to A\to A)\times A$$

and, consequently, we will have expressions like $\langle A,(m,e)\rangle\colon PM$. By abuse of notation, however, we will sometimes simply say *let $A$ be a magma/group/etc.* as it is usually expressed in the regular mathematical jargon.

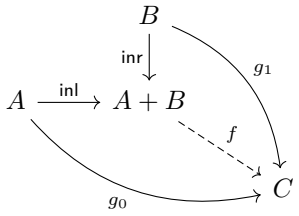
## 1.7 Coproduct Types

41. Coproducts. Given types $A, B : \mathcal{U}$ their *coproduct* is $A + B : \mathcal{U}$, a type that corresponds to the *disjoint union* in Set Theory. Its constructors consist of two maps

$$\begin{aligned} \mathsf{inl} &: A \to A + B && \text{(\textit{left injection})} \\ \mathsf{inr} &: B \to A + B && \text{(\textit{right injection})} \end{aligned}$$

There is also a nullary version, the *empty coproduct* (i.e., the coproduct of no types), denoted by $\mathsf{0} : \mathcal{U}$, which has no constructors.

Given $C : \mathcal{U}$, to define a function $f : A + B \to C$ we have to provide two maps, as illustrated in the following diagram




This means that to define $f$ we need to specify $g_0$ and $g_1$, which will satisfy

$$f(\mathsf{inl}(a)) \equiv g_0(a) \quad \text{and} \quad f(\mathsf{inr}(b)) \equiv g_1(b),$$

for $a : A$ and $b : B$.

In the case of the null coproduct, which is known as the *empty type*, given $C : \mathcal{U}$ there is a unique function $f : \mathsf{0} \to C$. Since the type $\mathsf{0}$ has no elements, this function has no further specification.

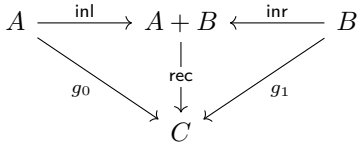
42. Coproduct recursion. Consistent with the way functions with domain in a coproduct are defined, we introduce the recursor

$$\mathsf{rec}_{A+B} : \prod_{C:\mathcal{U}} (A \to C) \to (B \to C) \to A + B \to C$$

with specification

$$\begin{aligned} \mathsf{rec}_{A+B}(C, g_0, g_1, \mathsf{inl}(a)) &\equiv g_0(a), \\ \mathsf{rec}_{A+B}(C, g_0, g_1, \mathsf{inr}(b)) &\equiv g_1(b). \end{aligned}$$

We can also express this recursion with the definitionally commutative diagram



where $\mathsf{rec} :\equiv \mathsf{rec}_{A+B}(C, g_0, g_1)$.

Additionally,

$$\mathsf{rec}_0 \colon \prod_{C \colon \mathcal{U}} 0 \to C \tag{1.1}$$

constructs the unique function $0 \to C$ for every $C \colon \mathcal{U}$.

**Digression.** In Gentzen-style notation, the empty coproduct is denoted by $\bot$ and the $\mathsf{rec}_0$ function is called $\mathsf{abort}$. In that framework, the rule corresponding to (1.1) is the elimination rule, which is presented as an inference rule in fractional form:

$$\frac{p \colon \bot}{\mathsf{abort}_A\, p \colon A} \quad (\bot \text{ Elim}).$$

This structural representation closely mirrors an Abstract Syntax Tree (AST). However, it may not properly scale with the complexity of the expressions.

43. Coproduct induction. Given a type family $C \colon (A + B) \to \mathcal{U}$, to define a dependent function

$$f \colon \prod_{s \colon A+B} C(s)$$

we have to provide two dependent functions

$$g_0 \colon \prod_{x \colon A} C(\mathsf{inl}(x)),$$
$$g_1 \colon \prod_{y \colon B} C(\mathsf{inr}(y))$$

and specify, for $a \colon A$ and $b \colon B$,

$$f(\mathsf{inl}(a)) \equiv g_0(a) \quad \text{and} \quad f(\mathsf{inr}(b)) \equiv g_1(b).$$

As a result, we obtain the *induction principle for coproducts*

$$\mathsf{ind}_{A+B} \colon \prod_{C \colon (A+B)\to\mathcal{U}} \Big(\prod_{x \colon A} C(\mathsf{inl}(x))\Big) \to \Big(\prod_{y \colon B} C(\mathsf{inr}(y))\Big) \to \prod_{s \colon A+B} C(s). \tag{1.2}$$

In the case of the empty type $0$ (the nullary coproduct), there are no constructors to handle. Thus, the induction principle for the empty type (a.k.a. eliminator) reduces to

$$\mathsf{ind}_0 \colon \prod_{C \colon 0\to\mathcal{U}} \prod_{s \colon 0} C(s).$$

This function asserts that if we assume an element of the empty type, we can derive any conclusion $C(s)$ (the principle of *ex falso quodlibet*).[12]

44. Case Analysis on Coproducts. When reasoning informally on a prop-

[12] In classical logic, intuitionistic logic, and similar logical systems, the *principle of explosion* is the law according to which any statement can be proven from a contradiction. That is, from a contradiction, any proposition (including its negation) can be inferred; this is known as deductive *explosion*. Note however the existence of the field of Inconsistent Mathematics, a theory that explains and studies paradoxes and paraconsistency.

erty $P(s)$ for an arbitrary element $s\colon A + B$ it is usual to apply *case analysis*: one considers the scenario where $s$ is of the form $\mathsf{inl}(a)$ for some $a\colon A$, and separately the scenario where $s$ is of the form $\mathsf{inr}(b)$ for some $b\colon B$.

This intuitive procedure is formally justified by the induction principle of the coproduct. Specifically, to construct a global witness:

$$f\colon \prod_{s\colon A+B} P(s),$$

it suffices to provide two independent case branches:

- a proof for left injections: $g_0\colon \prod_{a\colon A} P(\mathsf{inl}(a))$, and
- a proof for right injections: $g_1\colon \prod_{b\colon B} P(\mathsf{inr}(b))$.

The witness $f$ is then formally defined as

$$f :\equiv \lambda s.\, \mathsf{ind}_{A+B}(P, g_0, g_1, s).$$

The computation rules:

$$f(\mathsf{inl}(a)) \equiv g_0(a)$$
$$f(\mathsf{inr}(b)) \equiv g_1(b)$$

guarantee that evaluating $f$ on any canonical element reduces to the proof provided for that specific case. Thus, what manifests informally as *pattern matching on constructors* is the production of a dependent function via induction.

## 1.8 Boolean Type

45. The type of booleans. The *boolean type* is denoted $\mathsf{2}\colon \mathcal{U}$. Its constructors yield $0_\mathsf{2}$ and $1_\mathsf{2}$. To derive a function $f\colon \mathsf{2} \to C$ we have to specify two elements $c_0, c_1\colon C$ and declare

$$f(0_\mathsf{2}) :\equiv c_0 \quad \text{and} \quad f(1_\mathsf{2}) :\equiv c_1.$$

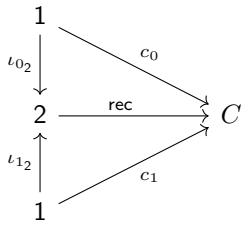
Therefore, the recursor

$$\mathsf{rec}_\mathsf{2}\colon \prod_{C\colon \mathcal{U}} C \to C \to \mathsf{2} \to C, \tag{1.1}$$

is specified by

$$\mathsf{rec}_\mathsf{2}(C, c_0, c_1, 0_\mathsf{2}) \equiv c_0$$
$$\mathsf{rec}_\mathsf{2}(C, c_0, c_1, 1_\mathsf{2}) \equiv c_1,$$

which means that $f\colon 2 \to C$ is defined by a pair of elements of type $C$. We can capture this in a definitionally commutative diagram




where $\mathsf{rec} :\equiv \mathsf{rec}_2(C, c_0, c_1)$.

In the case of dependent functions, given a type family $C\colon 2 \to \mathcal{U}$, we define

$$f\colon \prod_{b\colon 2} C(b)$$

specifying $c_0\colon C(0_2)$ and $c_1\colon C(1_2)$ and setting

$$f(0_2) :\equiv c_0 \quad \text{and} \quad f(1_2) :\equiv c_1.$$

For a dependent function

$$f\colon \prod_{b\colon 2} C(b),$$

where $C\colon 2 \to \mathcal{U}$, we have

$$\mathsf{ind}_2\colon \prod_{C\colon \mathcal{U}} C(0_2) \to C(1_2) \to 2 \to \prod_{b\colon 2} C(b)$$

along with

$$\mathsf{ind}_2(C, c_0, c_1, 0_2) :\equiv c_0,$$
$$\mathsf{ind}_2(C, c_0, c_1, 0_2) :\equiv c_1.$$

46. Exactly two booleans. Here we will use $\mathsf{ind}_2$ to prove that the boolean type 2 has exactly two elements, namely, $0_2$ and $1_2$.

To see this we use the type family

$$C\colon 2 \to \mathcal{U}, \quad C :\equiv \lambda x.\, (x =_2 0_2) + (x =_2 1_2).$$

Now, evaluation at the constructors yields

$$C(0_2) \equiv (0_2 =_2 0_2) + (0_2 =_2 1_2),$$
$$C(1_2) \equiv (1_2 =_2 0_2) + (1_2 =_2 1_2).$$

Therefore, we have valid witnesses

$$\mathsf{inl}(\mathsf{refl}_{0_2})\colon C(0_2) \quad \text{and} \quad \mathsf{inr}(\mathsf{refl}_{1_2})\colon C(1_2).$$

And so we obtain the proof:

$$\mathsf{ind}_2\big(C, \mathsf{inl}(\mathsf{refl}_{0_2}), \mathsf{inr}(\mathsf{refl}_{1_2})\big) \colon \prod_{x\colon 2}(x =_2 0_2) + (x =_2 1_2).$$

47. Coproducts as Indexed Disjoint Unions. For $A, B \colon \mathcal{U}$ consider the recursor $\mathsf{rec}_2$ as defined in (1.1) with $C :\equiv \mathcal{U}$ and

$$\begin{aligned} f \colon 2 &\to \mathcal{U} \\ 0_2 &\mapsto A, \\ 1_2 &\mapsto B. \end{aligned}$$

Thus,

$$\sigma :\equiv \mathsf{rec}_2(\mathcal{U}, A, B) \colon 2 \to \mathcal{U},$$

with

$$\sigma(0_2) \equiv A \quad \text{and} \quad \sigma(1_2) \equiv B.$$

We can then introduce what we aim to prove as being equivalent to $A + B$,

$$A \oplus B :\equiv \sum_{x\colon 2} \sigma(x)$$

along with

$$\begin{aligned} \mathsf{in}_A &\colon A \to A \oplus B, \quad a \mapsto \langle 0_2, a\rangle \\ \mathsf{in}_B &\colon B \to A \oplus B, \quad b \mapsto \langle 1_2, b\rangle. \end{aligned} \tag{1.2}$$

To see that these are well-defined specifications, note that for any $a \colon A$, the pair $\langle 0_2, a\rangle$ has type $\sum_{x\colon 2} \sigma(x)$ because the second component is of the required type $\sigma(0_2) \equiv A$. A similar argument holds for $\mathsf{in}_B$ since $\sigma(1_2) \equiv B$.

It remains to be shown that $A + B \simeq A \oplus B$. To see this first define

$$\phi \colon A + B \to A \oplus B \tag{1.3}$$

as the only map that satisfies

$$\phi(\mathsf{inl}(a)) \equiv \mathsf{in}_A(a) \quad \text{and} \quad \phi(\mathsf{inr}(b)) \equiv \mathsf{in}_B(b)$$

for $a \colon A$ and $b \colon B$. Second, to define a map in the other direction, we have to use the recursor evaluated at the coproduct $A + B$

$$\mathsf{rec}_{\sum_{x\colon 2}\sigma(x)}(A + B) \colon \Big(\prod_{x\colon 2} \sigma(x) \to A + B\Big) \to \Big(\sum_{x\colon 2} \sigma(x)\Big) \to A + B$$

and apply the result to a well-chosen dependent map

$$h \colon \prod_{x\colon 2} \sigma(x) \to A + B.$$

Take $h$ to be the one defined by

$$h(0_2) :\equiv \mathsf{inl}\colon A \to A + B,$$
$$h(1_2) :\equiv \mathsf{inr}\colon B \to A + B,$$

which is valid because $\sigma(0_2) \equiv A$ and $\sigma(1_2) \equiv B$.

Now we can define

$$\psi :\equiv \mathsf{rec}_{\sum_{x:2} \sigma(x)}(A + B, h). \tag{1.4}$$

Finally, we have to check the following two equalities

$$\phi(\psi(p)) =_{A \oplus B} p \quad (p\colon A \oplus B)$$
$$\psi(\phi(s)) =_{A+B} s \quad (s\colon A + B)$$

To verify the first equation, by the definition of $A \oplus B$, it suffices to consider the cases $p :\equiv \langle 0_2, a\rangle$ and $p :\equiv \langle 1_2, b\rangle$, with $a\colon A$ and $b\colon B$, since these exhaust all the canonical pairs of $A \oplus B$.

- For $p :\equiv \langle 0_2, a\rangle$, we have $\psi(\langle 0_2, a\rangle) \equiv h(0_2)(a) \equiv \mathsf{inl}(a)$. Thus,

$$\phi(\psi(p)) \equiv \phi(\mathsf{inl}(a)) \equiv \mathsf{in}_A(a) \equiv \langle 0_2, a\rangle \equiv p.$$

- For $p :\equiv \langle 1_2, b\rangle$, we have $\psi(\langle 1_2, b\rangle) \equiv h(1_2)(b) \equiv \mathsf{inr}(b)$. Thus,

$$\phi(\psi(p)) \equiv \phi(\mathsf{inr}(b)) \equiv \mathsf{in}_B(b) \equiv \langle 1_2, b\rangle \equiv p.$$

Conversely, for $s\colon A + B$, we check $\psi(\phi(s)) =_{A+B} s$ by induction on the coproduct:

- For $s :\equiv \mathsf{inl}(a)$, $\psi(\phi(s)) \equiv \psi(\mathsf{in}_A(a)) \equiv \psi(\langle 0_2, a\rangle) \equiv \mathsf{inl}(a) \equiv s$.
- For $s :\equiv \mathsf{inr}(b)$, $\psi(\phi(s)) \equiv \psi(\mathsf{in}_B(b)) \equiv \psi(\langle 1_2, b\rangle) \equiv \mathsf{inr}(b) \equiv s$.

This completes the proof.

## 1.9 The Type of Natural Numbers

48. NATURAL NUMBERS. The type $\mathbb{N}\colon \mathcal{U}$ is introduced with two constructors: the element *zero* $0\colon \mathbb{N}$ and a *successor* operator $\mathsf{succ}\colon \mathbb{N} \to \mathbb{N}$.

Predictably, we adopt the conventional notation $1 :\equiv \mathsf{succ}(0)$, $2 :\equiv \mathsf{succ}(1)$, etc.

The recursor

$$\mathsf{rec}_{\mathbb{N}}\colon \prod_{C:\mathcal{U}} C \to (\mathbb{N} \to C \to C) \to \mathbb{N} \to C$$

is specified by

$$\begin{aligned} \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, 0) &:\equiv c_0, \\ \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, \mathsf{succ}(n)) &:\equiv c_s(n, \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, n)). \end{aligned}$$

We can represent this in a definitionally commutative diagram

$$\begin{array}{ccccc} 1 & \xrightarrow{\quad 0 \quad} & \mathbb{N} & \xrightarrow{\quad \mathsf{succ} \quad} & \mathbb{N} \\ \downarrow{\scriptstyle (0,c_0)} & & \downarrow{\scriptstyle (\mathsf{id},\mathsf{rec})} & & \downarrow{\scriptstyle (\mathsf{id},\mathsf{rec})} \\ 1 \times C & \xrightarrow[0\times c_0]{} & \mathbb{N} \times C & \xrightarrow[(\mathsf{succ}\circ\pi_1, c_s)]{} & \mathbb{N} \times C \end{array} \tag{1.1}$$

where $\mathsf{rec}$ stands for $\mathsf{rec}_{\mathbb{N}}(C, c_0, c_s)$.

Note that this definition is different from the ones we introduced so far: it allows for $\mathsf{rec}$ to appear in the RHS. This is why we say that $\mathbb{N}$ is an *inductive* type.

The justification of this extension is *metatheoretic*: the expression on the RHS uses $n$, which is syntactically *shorter* than $\mathsf{succ}(n)$. This implies that well-formed terms effectively halt, after a finite number of iterations. More precisely, the syntactic reduction is *well-founded*, meaning that it has no infinite descending chains $x_1 > x_2 > \cdots > x_n > \cdots$.

Consequently, to define a function $f : \mathbb{N} \to C$, we provide two pieces of information:

$$\begin{gathered} \text{a starting point } c_0 : C, \\ \text{the next step function } c_s : \mathbb{N} \to C \to C \end{gathered} \tag{1.2}$$

and declare

$$\begin{aligned} f(0) &:\equiv c_0, \\ f(\mathsf{succ}(n)) &:\equiv c_s(n, f(n)), \end{aligned} \tag{1.3}$$

where the second equation corresponds to

$$c_s(n, f(n)) \equiv c_s(n, \underbrace{\mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, n)}_{f(n)}).$$

We say that $f$ is defined by *primitive recursion*.

49. NATURAL DOUBLING. A simple example of a function is $\mathsf{double} : \mathbb{N} \to \mathbb{N}$, which can be defined by

$$\begin{aligned} \mathsf{double}(0) &:\equiv 0, \\ \mathsf{double}(\mathsf{succ}(n)) &:\equiv \mathsf{succ}(\mathsf{succ}(\mathsf{double}(n))), \end{aligned}$$

which corresponds to $c_0 :\equiv 0$ and $c_s : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$, $c_s(n, d) :\equiv \mathsf{succ}(\mathsf{succ}(d))$.

Thus, for instance,

$$\begin{aligned}\mathsf{double}(1) &\equiv \mathsf{double}(\mathsf{succ}(0))\\ &\equiv \mathsf{succ}(\mathsf{succ}(\mathsf{double}(0)))\\ &\equiv \mathsf{succ}(\mathsf{succ}(0))\\ &\equiv \mathsf{succ}(1)\\ &\equiv 2\end{aligned}$$

and

$$\begin{aligned}\mathsf{double}(2) &\equiv \mathsf{double}(\mathsf{succ}(1))\\ &\equiv \mathsf{succ}(\mathsf{succ}(\mathsf{double}(1)))\\ &\equiv \mathsf{succ}(\mathsf{succ}(2))\\ &\equiv \mathsf{succ}(3)\\ &\equiv 4.\end{aligned}$$

Note that even though the expression grows in length in some of the intermediate steps, it eventually succeeds in removing any circular reference to $\mathsf{rec}$, which is the precise meaning of "halting".

50. Addition of natural numbers. The natural recursor allows the definition of multi-variable functions, such as addition, by letting the return type $C$ be a function type itself. We define

$$\mathsf{add} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$$

using primitive recursion with $C :\equiv \mathbb{N} \to \mathbb{N}$, and specifying after (1.2)

$$\begin{aligned} &c_0 : \mathbb{N} \to \mathbb{N}, && c_0 :\equiv \lambda n.\, n\\ &c_s : \mathbb{N} \to (\mathbb{N} \to \mathbb{N}) \to (\mathbb{N} \to \mathbb{N}), && c_s :\equiv \lambda m.\, \lambda g.\, \lambda n.\, \mathsf{succ}(g(n)),\end{aligned}$$

where $g : \mathbb{N} \to \mathbb{N}$ acts as a placeholder for the recursive result. When the recursor computes, $g$ will be instantiated with the function $n \mapsto m + n$.

We define $\mathsf{add} :\equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N} \to \mathbb{N}, c_0, c_s)$. By the computation rules of the recursor, this definition satisfies the standard equations:

$$\begin{aligned}\mathsf{add}(0, n) &\equiv c_0(n) \equiv n,\\ \mathsf{add}(\mathsf{succ}(m), n) &\equiv c_s(m, \mathsf{add}(m))(n)\\ &\equiv \mathsf{succ}(\mathsf{add}(m, n)),\end{aligned}$$

which specify $\mathsf{add}$ by recursion on the first variable.

51. Multiplication of natural numbers. To define

$$\mathsf{mult} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$$

we apply the recursion principle for $\mathbb{N}$ with $C :\equiv \mathbb{N} \to \mathbb{N}$ and

$$\begin{aligned} c_0 &:\equiv \lambda n.\, 0, \\ c_s &:\equiv \lambda m.\, \lambda g.\, \lambda n.\, \mathsf{add}(g(n), n). \end{aligned}$$

As a result, $\mathsf{mult} :\equiv \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s)$ satisfies the following equations:

$$\begin{aligned} \mathsf{mult}(0, n) &\equiv c_0(n) \equiv 0, \\ \mathsf{mult}(\mathsf{succ}(m), n) &\equiv c_s(m, \mathsf{mult}(m))(n) \\ &\equiv \mathsf{add}(\mathsf{mult}(m, n), n). \end{aligned}$$

52. Pattern matching. When we introduce a function $f\colon \mathbb{N} \to C$ such as

$$\begin{aligned} f(0) &:\equiv \Phi_0, \\ f(\mathsf{succ}(n)) &:\equiv \Phi_s, \end{aligned}$$

where $\Phi_0$ is an expression of type $C$, and $\Phi_s$ is an expression of type $C$ which may involve the variable $n$ and also the symbol "$f(n)$", we may translate it to a formal definition

$$f :\equiv \mathsf{rec}_{\mathbb{N}}(C, \Phi_0, \lambda n.\, \lambda g.\, \Phi'_s),$$

where $\Phi'_s$ is obtained from $\Phi_s$ by replacing all occurrences of "$f(n)$" by the new variable $g$.

This style of defining functions by recursion (or, more generally, dependent functions by induction) is called definition by *pattern matching*. We frequently use it for the sake of convenience.

**Example 1.9.1.** Let us define the factorial function $\mathsf{fact} : \mathbb{N} \to \mathbb{N}$.

In the convenient pattern matching style, we write:

$$\begin{aligned} \mathsf{fact}(0) &:\equiv 1, \\ \mathsf{fact}(\mathsf{succ}(n)) &:\equiv \mathsf{succ}(n) \cdot \mathsf{fact}(n), \end{aligned}$$

where the usual '$\cdot$' symbol denotes multiplication.

Here, our components correspond to the general schema as follows:

a) The target type is $C \equiv \mathbb{N}$.

b) The base case is $\Phi_0 \equiv 1$.

c) The successor expression is $\Phi_s \equiv \mathsf{succ}(n) \cdot \mathsf{fact}(n)$. Note that this involves both the variable $n$ and the recursive call $\mathsf{fact}(n)$.

d) $\Phi'_s :\equiv \mathsf{succ}(n) \cdot g$.

e) $\mathsf{fact} \equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N}, 1, \lambda n.\, \lambda g.\, \mathsf{succ} \cdot g)$.

53. Induction principle. Assume as given a family $C\colon \mathbb{N} \to \mathcal{U}$, an element $c_0\colon C(0)$, and a function $c_s\colon \prod_{n:\mathbb{N}} C(n) \to C(\mathsf{succ}(n))$; then we can construct a dependent function $f\colon \prod_{n:\mathbb{N}} C(n)$ with the defining equations:

$$\begin{aligned} f(0) &:\equiv c_0, \\ f(\mathsf{succ}(n)) &:\equiv c_s(n, f(n)). \end{aligned}$$

We formalize this with the induction function

$$\mathsf{ind}_{\mathbb{N}}\colon \prod_{C:\mathbb{N}\to\mathcal{U}} C(0) \to \Big(\prod_{n:\mathbb{N}} C(n) \to C(\mathsf{succ}(n))\Big) \to \prod_{n:\mathbb{N}} C(n)$$

plus the equations

$$\begin{aligned} \mathsf{ind}_{\mathbb{N}}(C, c_0, c_s, 0) &:\equiv c_0, \\ \mathsf{ind}_{\mathbb{N}}(C, c_0, c_s, \mathsf{succ}(n)) &:\equiv c_s(n, \mathsf{ind}_{\mathbb{N}}(C, c_0, c_s, n)). \end{aligned}$$

As usual, we will refer to $C$ as the *theorem*, to $c_0$ as the *base case*, and to $c_s$ as the *inductive step*. More precisely,

$$c_s :\equiv \lambda n.\, \lambda h.\, \mathit{witness},$$

where $n$ is a generic natural, $h\colon C(n)$ is our *inductive hypothesis* and *witness* is an element of $C(\mathsf{succ}(n))$.[13]

54. Blocked computation. As usual, we will denote $\mathsf{add}(m, n)$ by $m+n$ and $\mathsf{mult}(m, n)$ by $m \cdot n$. In the first case, computation on the first argument implies the definitional equalities:

$$\begin{aligned} 0 + n &\equiv n, \\ \mathsf{succ}(m) + n &\equiv \mathsf{succ}(m + n). \end{aligned} \tag{1.4}$$

In particular, we can derive:

$$\begin{aligned} 1 + n &\equiv \mathsf{succ}(0) + n \\ &\equiv \mathsf{succ}(0 + n) \\ &\equiv \mathsf{succ}(n). \end{aligned}$$

However, the equation $n =_{\mathbb{N}} n + 0$ does not hold definitionally for an arbitrary variable $n$. It is a proposition that must be proven as a theorem.

The reason is found in the operational semantics: a *machine* evaluating the expression $\Phi + 0$ proceeds by pattern matching on $\Phi$ (the first argument). It checks if the *head*[14] of $\Phi$ matches the constructor 0 or matches the constructor $\mathsf{succ}$.

In our case, $\Phi :\equiv n$. Since $n$ is a variable and not a constructor, neither reduction rule applies. The machine cannot simplify the expression further, and the computation *blocks*. Thus, we cannot rely on automatic reduction. In 67. Additive identity we construct an explicit proof term.

[13] In other words, $c_s$ is a function that produces a proof of $C(\mathsf{succ}(n))$ from a proof of $C(n)$.

[14] The word 'head' refers to the root of the Abstract Syntactic Tree (AST) of the expression.

## 1.10 Propositions as Types

55. Propositional Logic. There is an interpretation that assigns constructive logic semantics to type-theoretical types, which can be summarized as:

| | **Constructive proposition** | **Type Theory's type** |
|---|---|---|
| 1 | true | $\mathbf{1}$ |
| 2 | false | $\mathbf{0}$ |
| 3 | $A$ and $B$ | $A \times B$ |
| 4 | $A$ or $B$ | $A + B$ |
| 5 | if $A$ then $B$ | $A \to B$ |
| 6 | $A$ if, and only if, $B$ | $(A \to B) \times (B \to A)$ |
| 7 | not $A$ | $A \to \mathbf{0}$ |

Let us review them one by one, assuming that types $A$ and $B$ represent propositions and their elements represent proofs.

1. Since $\star \colon \mathsf{1}$, we see that $\mathsf{1}$ has a proof (it is always inhabited).
2. Since $\mathsf{0}$ has no elements, there is no proof for $\mathsf{0}$ (it is false).
3. A proof $x$ of $A \times B$ produces a proof $\pi_1(x)$ of $A$ and a proof $\pi_2(x)$ of $B$. Conversely, if $a$ is a proof of $A$ and $b$ is a proof of $B$, then the pair $(a, b)$ is a proof of $A \times B$.
4. A proof of $A + B$ (propositionally) equals either $\mathsf{inl}(a)$, for a proof $a$ of $A$, or $\mathsf{inr}(b)$, for a proof $b$ of $B$. The converse clearly holds.
5. A proof of the implication $A \to B$ is a function $f$. This function provides a method to transform any proof $a$ of $A$ into a proof $f(a)$ of $B$.
6. This follows immediately from parts 3 and 5.
7. By definition, a proof of $\neg A$ is a function $f \colon A \to \mathsf{0}$. If such a function exists, $A$ cannot have any proof, otherwise applying $f$ to that proof would yield a proof of $\mathsf{0}$ (which is impossible, see part 2).

56. From English to types. Using the table from 55. Propositional Logic we can translate a classical proof into Type Theory. Consider De Morgan's law:

*if not $A$ and not $B$, then not ($A$ or $B$).*

This corresponds to the type

$$(A \to \mathsf{0}) \times (B \to \mathsf{0}) \to (A + B \to \mathsf{0}).$$

To construct an inhabitant $f$ of this type, we assume a witness for the antecedent, which is a pair $(x, y)$ where $x \colon A \to \mathsf{0}$ and $y \colon B \to \mathsf{0}$.

We must construct a return value of type $A + B \to \mathsf{0}$. We do this using the recursion principle for coproducts with the target type $C :\equiv \mathsf{0}$.

The recursor requires two functions to handle the cases:

- A function $A \to \mathsf{0}$. Our assumption $x$ is exactly this function.
- A function $B \to \mathsf{0}$. Our assumption $y$ is exactly this function.

Therefore, we can define $f$ as (see 42. Coproduct recursion)

$$f((x, y)) :\equiv \mathsf{rec}_{A+B}(\mathsf{0}, x, y).$$

Thus, for $a\colon A$ and $b\colon B$, we have

$$f((x, y))(\mathsf{inl}(a)) :\equiv x(a) \quad \text{and} \quad f((x, y))(\mathsf{inr}(b)) :\equiv y(b),$$

which is well defined because $x(a), y(b)\colon \mathsf{0}$. Therefore, $f$ proves $(A + B) \to \mathsf{0}$ in each of the two possible cases derived from $A + B$.

In contrast, the other law

$$\textit{if not } (A \textit{ and } B), \textit{ then } (\textit{not } A) \textit{ or } (\textit{not } B),$$

which translates into

$$((A \times B) \to \mathsf{0}) \to (A \to \mathsf{0}) + (B \to \mathsf{0})$$

is not generally valid because there is no method to decide which of the two propositions, $A$ or $B$, is false from a proof of $\neg(A \times B)$. However, to exhibit an element of $(A \to \mathsf{0}) + (B \to \mathsf{0})$, we must decide whether it will be a value of $\mathsf{inl}$ or of $\mathsf{inr}$. For instance, consider the case where $B :\equiv \neg A$. We do know that $A \wedge \neg A$ is false[15] but not if this is because $A$ is false or $\neg A$ is false (as it happens with unsolved conjectures).

57. Predicate logic. The following table extends the propositions as types interpretation (see 55. Propositional Logic) to quantifiers

| | **Predicate logic** | **Type Theory's type** |
|---|---|---|
| 1 | for all $x\colon A$, $P(x)$ holds | $\prod_{x\colon A} P(x)$ |
| 2 | there exists $x\colon A$ such that $P(x)$ | $\sum_{x\colon A} P(x)$ |

For example, the logical distributivity law

$$\textit{if for all } x\colon A, P(x) \textit{ and } Q(x), \textit{ then } (\textit{for all } x\colon A, P(x)) \textit{ and } (\textit{for all } x\colon A, Q(x))$$

translates into the type

$$\Big(\prod_{x\colon A}(P(x) \times Q(x))\Big) \to \Big(\prod_{x\colon A} P(x)\Big) \times \Big(\prod_{x\colon A} Q(x)\Big).$$

[15] A witness of this proposition is a pair $(a, b)$ with $a\colon A$ and $b\colon A \to \mathsf{0}$. It cannot exist since otherwise $b(a)\colon \mathsf{0}$.

Its proof consists of constructing a witness $f$ for this type.

Take a dependent function $r$ in the domain. We must construct a pair of dependent functions $(p, q)$, where $p\colon \prod_{x\colon A} P(x)$ and $q\colon \prod_{x\colon A} Q(x)$. For any $x\colon A$, the term $r(x)$ is a pair in $P(x) \times Q(x)$. Using standard projections, we could define the result directly as:

$$f(r) :\equiv \big(\lambda x.\, \pi_1(r(x)),\ \lambda x.\, \pi_2(r(x))\big).$$

To derive this using the recursor, we must separate the components by providing the appropriate step functions. For the first component $p$, the handler must select the first argument:

$$p(r) :\equiv \lambda x.\, \mathsf{rec}_{P(x)\times Q(x)}(P(x),\ \lambda u.\, \lambda v.\, u,\ r(x)).$$

Similarly, for the second component $q$, the handler selects the second argument:

$$q(r) :\equiv \lambda x.\, \mathsf{rec}_{P(x)\times Q(x)}(Q(x),\ \lambda u.\, \lambda v.\, v,\ r(x)).$$

Finally, we assemble $f$ as the pair of these lambda abstractions:

$$f :\equiv \lambda r.\, (p(r), q(r)).$$

## 1.11 Identity Types

In this section we go in further detail about identity family types. Concretely, if $A\colon \mathcal{U}$, then $\mathsf{Id}_A\colon A \to A \to \mathcal{U}$ is the type family dependent on two copies of $A$, whose notation is

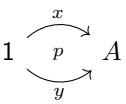
$$\mathsf{Id}_A(a, b) :\equiv a =_A b.$$

The witnesses of this type are regarded as *paths in the space $A$ from the point $a$ to the point $b$*. The constructor of elements of this type is the dependent *reflexivity* function

$$\mathsf{refl}\colon \prod_{a\colon A} a =_A a.$$

In particular, if $b \equiv a$, then $\mathsf{refl}_a\colon a =_A b$.

**Remark 1.11.1.** A point $x\colon A$ can be interpreted as a function from the unit type, $x\colon 1 \to A$. Under this convention, a path $p\colon x =_A y$ can be seen as a commutative diagram: a homotopy between the maps $x$ and $y$.




This representation links the topological view with the algebraic one of higher categories, allowing us to consider points and paths under the standard notions of functor and natural transformation.

58. Path recursion. Consider the type of *free* paths

$$A^I :\equiv \sum_{x,y:\, A} x =_A y.$$

While this represents the space of all paths in $A$, the fundamental recursion principle is stated for paths starting at a fixed element $x\colon A$.

Given a type $B$, the recursion principle for the *based* path space at $x$ defines functions that effectively ignore the path. It has the signature:

$$\mathsf{rec}_{=_A}\colon \prod_{B:\,\mathcal{U}} B \to \prod_{y:\,A} (x =_A y) \to B.$$

It satisfies the computation rule:

$$\mathsf{rec}_{=_A}(B, c_{\mathsf{refl}}, x, \mathsf{refl}_x) \equiv c_{\mathsf{refl}},$$

where $c_{\mathsf{refl}}\colon B$ is the value assigned to the reflexivity path.

Since $x$ is fixed in the context, the value $b$ may depend on it. For example, taking $B :\equiv A$ and $b :\equiv x$, we obtain the function

$$f\colon \prod_{y:\,A} (x =_A y) \to A$$
$$f :\equiv \lambda y.\, \lambda p.\, x,$$

which is formally defined as $f :\equiv \mathsf{rec}_{=_A}(A, x)$, and satisfies $f(x, p) =_A x$ for any path $p$ with origin in $x$.[16]

59. General path induction. In the case of identity types, the induction principle (a.k.a. *path induction*) conceptually requires a motive defined on the total space of paths:

$$\Big(\sum_{x,y:\,A} x =_A y\Big) \to \mathcal{U}.$$

In Type Theory, however, we specify this map using its Curried[17] form:

$$C\colon \prod_{x,y:\,A} (x =_A y) \to \mathcal{U}.$$

This distinction explains why we write $C(x, y, p)$ rather than $C(\langle (x, y), p\rangle)$.

Given such a motive $C$ and a base case function $c\colon \prod_{x:\,A} C(x, x, \mathsf{refl}_x)$, the principle establishes the existence of a dependent function

$$f\colon \prod_{x,y:\,A} \prod_{p:\, x =_A y} C(x, y, p)$$

[16] Compare this with the function $g :\equiv \lambda x.\, \lambda p.\, x$ which satisfies $g(x) \equiv x$ for $x\colon A$.

[17] See 35. Generalized Currying.

such that $f(x, x, \mathsf{refl}_x) :\equiv c(x)$ for $x\colon A$. In other words, the induction principle for identity types is defined as

$$\mathsf{ind}_{=_A}\colon \prod_{C\colon \prod_{x,y\colon A}(x=_A y)\to\mathcal{U}} \left(\prod_{x\colon A} C(x, x, \mathsf{refl}_x)\right) \to \prod_{x,y\colon A} \prod_{p\colon x=_A y} C(x, y, p) \quad (1.1)$$

satisfying the computation rule

$$\mathsf{ind}_{=_A}(C, c, a, a, \mathsf{refl}_a) :\equiv c(a)$$

for every $a\colon A$.[18]

60. Path symmetry. The following lemma formalizes the symmetry of the *equality relation* between elements of the same type $A$

$$x \sim_A y \iff x =_A y \text{ is inhabited.} \quad (1.2)$$

**Lemma 1.11.2.** *For every type $A$ and every $x, y\colon A$, there is a function*

$$(x =_A y) \to (y =_A x),$$

*denoted $p \mapsto p^{-1}$, such that $\mathsf{refl}_x^{-1} \equiv \mathsf{refl}_x$ for each $x\colon A$.*

*Proof.* Given a type $A$, define the *motive* family[19]

$$C\colon \prod_{x,y\colon A} (x =_A y) \to \mathcal{U}$$
$$C(x, y, p) :\equiv (y =_A x).$$

For the base case, we need a function of type $\prod_{x\colon A} C(x, x, \mathsf{refl}_x)$. Since

$$C(x, x, \mathsf{refl}_x) \equiv (x =_A x),$$

we can take

$$c :\equiv \lambda x.\, \mathsf{refl}_x.$$

Then, by general path induction, we define

$$p^{-1} :\equiv \mathsf{ind}_{=_A}(C, c, x, y, p)$$

with

$$\mathsf{refl}_x^{-1} \equiv \mathsf{ind}_{=_A}(C, c, x, x, \mathsf{refl}_x) \equiv c(x) \equiv \mathsf{refl}_x,$$

which completes the proof. □

[18] This mirrors the categorical Yoneda's lemma B.2.1, which states that natural transformations from a representable functor $\mathrm{Hom}(x, -)$ are completely determined by evaluating at the identity morphism on $x$.

[19] The type family whose role is to identify the theorem we are about to prove.

61. Path concatenation. The following lemma shows that the equality relation is transitive.

**Lemma 1.11.3.** *For every type $A$ and every $x, y, z\colon A$ there is a concatenation function*

$$(x =_A y) \to (y =_A z) \to (x =_A z),$$

*written*

$$p \mapsto q \mapsto p \cdot q \qquad \begin{array}{ccccc} & & x & & \\ & \overset{p}{\nearrow} & & \overset{p}{\searrow} & \\ 1 & \xrightarrow{\quad} & y & \xrightarrow{\quad} & A, \\ & \underset{q}{\searrow} & & \underset{q}{\nearrow} & \\ & & z & & \end{array}$$

*such that* $\mathsf{refl}_x \cdot q \equiv q$ *for any* $x\colon A$ *and* $q\colon x =_A z$.

*Proof.* Let $D\colon \prod_{x,y\colon A}(x =_A y) \to \mathcal{U}$ be the motive family

$$D(x, y, p) :\equiv \prod_{z\colon A} \prod_{q\colon y =_A z} x =_A z.$$

Since $x =_A z$ does not depend on the path $q\colon y =_A z$, by (1.1) we can rewrite $D$ as

$$D(x, y, p) :\equiv \prod_{z\colon A} (y =_A z) \to (x =_A z).$$

For the base case we take

$$d(x)\colon D(x, x, \mathsf{refl}_x) \equiv \prod_{z\colon A} (x =_A z) \to (x =_A z), \quad d(x) :\equiv \lambda z.\, \lambda q.\, q.$$

By path induction, we obtain

$$\mathsf{ct}(x, y, p) :\equiv \mathsf{ind}_{=_A}(D, d, x, y, p)\colon D(x, y, p).$$

We can define the concatenation $p \cdot q$ by applying this function:

$$p \cdot q :\equiv \mathsf{ct}(x, y, p, z, q).$$

The computation rule for path induction implies

$$\mathsf{ct}(x, x, \mathsf{refl}_x) \equiv d(x).$$

Therefore,

$$\mathsf{refl}_x \cdot q \equiv \mathsf{ct}(x, x, \mathsf{refl}_x, z, q) \equiv d(x)(z, q) \equiv q,$$

which completes the proof. □

62. Right unit law. The term $\mathsf{refl}$ also acts as a right unit for concatenation; however, in this case, the equality is propositional.

**Lemma 1.11.4.** *For every type $A$, every $x, y\colon A$ and every path $p\colon x =_A y$, we have a path*

$$\mathsf{ru}(p)\colon p \cdot \mathsf{refl}_y =_{x=_A y} p.$$

*Proof.* We perform path induction on $p$. Let $C$ be the motive family

$$C :\equiv \prod_{x,y\colon A} \prod_{p\colon x=_A y} (p \cdot \mathsf{refl}_y =_{x=_A y} p).$$

For the base case, we assume $y \equiv x$ and $p \equiv \mathsf{refl}_x$. We must construct a term of type

$$\mathsf{refl}_x \cdot \mathsf{refl}_x =_{x=_A x} \mathsf{refl}_x.$$

By Lemma 1.11.3, we have the definitional equality $\mathsf{refl}_x \cdot \mathsf{refl}_x \equiv \mathsf{refl}_x$. Thus, the goal simplifies to $\mathsf{refl}_x =_{x=_A x} \mathsf{refl}_x$, which is inhabited by $c(x) :\equiv \mathsf{refl}_{\mathsf{refl}_x}$.

Applying path induction, we obtain the desired term

$$\mathsf{ru}(p) :\equiv \mathsf{ind}_{=_A}(C, c, x, y, p).$$

□

63. Further concatenation properties.

**Lemma 1.11.5.** *Let $A\colon \mathcal{U}$, $x, y, z, w\colon A$ and $p\colon x =_A y$. Then, the following types are inhabited:*

a) $p^{-1} \cdot p =_{y=_A y} \mathsf{refl}_y$ *and* $p \cdot p^{-1} =_{x=_A x} \mathsf{refl}_x$.

b) $(p^{-1})^{-1} =_{x=_A y} p$.

c) $p \cdot (q \cdot r) =_{x=_A w} (p \cdot q) \cdot r$, *for* $q\colon y =_A z$ *and* $r\colon z =_A w$.

*Proof.*

a) Consider the motive

$$C :\equiv \prod_{x,y\colon A} \prod_{p\colon x=_A y} p^{-1} \cdot p =_{y=_A y} \mathsf{refl}_y.$$

We must show $C(x, x, \mathsf{refl}_x)$. Since $\mathsf{refl}_x^{-1} \equiv \mathsf{refl}_x$ and $\mathsf{refl}_x \cdot \mathsf{refl}_x \equiv \mathsf{refl}_x$, we have

$$\mathsf{refl}_x^{-1} \cdot \mathsf{refl}_x \equiv \mathsf{refl}_x.$$

Thus, we can take $c(x) :\equiv \mathsf{refl}_{\mathsf{refl}_x}$ and obtain

$$\mathsf{ind}_{=_A}(C, c, x, y, p)\colon C(x, y, p).$$

The other equation is proved similarly.[20]

[20] Alternatively, it can be derived from part b) and the equation just proved, applied to $p^{-1}$ in the place of $p$.

b) By path induction on $p$, it suffices to consider the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In that case, the statement holds definitionally because $\mathsf{refl}_x^{-1} \equiv \mathsf{refl}_x$.

c) By repeated path induction on $p$, $q$, and $r$, it suffices to consider the case where they are all $\mathsf{refl}_x$. In that case, the statement holds because $\mathsf{refl}_x \cdot \mathsf{refl}_x \equiv \mathsf{refl}_x$. □

64. TRANSPORT. The principle of *indiscernibility of identicals* establishes that the equality constructor is compatible with *type transport* (a.k.a. *type substitution*).[21]

**Theorem 1.11.6.** *Let $P\colon A \to \mathcal{U}$ be a family of types. Given $x, y\colon A$ and a path $p\colon x =_A y$, there is a function*

$$\mathsf{transport}^P(p)\colon P(x) \to P(y).$$

*Moreover,* $\mathsf{transport}^P(\mathsf{refl}_x) \equiv \lambda z.\, z$.

*Proof.* Consider the motive $C\colon \prod_{x,y:A}(x =_A y) \to \mathcal{U}$

$$C(x, y, e) :\equiv P(x) \to P(y).$$

For the base case $c\colon C(x, x, \mathsf{refl}_x) \equiv P(x) \to P(x)$ take the identity function $c :\equiv \lambda z.\, z$. Then define

$$\mathsf{transport}^P(x, y, e) :\equiv \mathsf{ind}_{=_A}(C, c, x, y, e).$$

□

65. BASED PATH INDUCTION. A different approach to path induction, which we will show is equivalent to the general one, consists of fixing an element $a\colon A$ before considering a type family $D$ specific to $a$,

$$D\colon \prod_{x:A}(a =_A x) \to \mathcal{U}.$$

Given an element $d\colon D(a, \mathsf{refl}_a)$, we then obtain a dependent function

$$f\colon \prod_{x:A} \prod_{p:a=_A x} D(x, p).$$

This means that to define an element of the family $D$ for all $x$ and $p$, it suffices to consider the case where $x$ is $a$ and $p$ is $\mathsf{refl}_a$.

As a function, based path induction becomes

$$\mathsf{ind}'_{=_A}\colon \prod_{a:A} \prod_{D:\prod_{x:A}(a=_A x)\to\mathcal{U}} D(a, \mathsf{refl}_a) \to \prod_{x:A} \prod_{p:a=_A x} D(x, p) \tag{1.3}$$

[21] See also Exercise 1.16.1.

with the equality

$$\mathsf{ind}'_{=_A}(a, D, d, a, \mathsf{refl}_a) :\equiv d.$$

**Note.** The difference between both forms of path induction is that in the based one we are free to choose and fix an element $a\colon A$ and then (1) formulate a property $D(x, p)$ for every $p\colon a =_A x$, and (2) exhibit a witness of $D(a, \mathsf{refl}_a)$. In general path induction, we formulate $C(x, y, p)$ for generic $p\colon x =_A y$, and then prove $C(x, x, \mathsf{refl}_x)$, also for a generic $x\colon A$. While the general approach may look natural in some special cases, the based one is often simpler because the generalization $D(x, p)$ it requires is closer to the concrete base case $D(a, \mathsf{refl}_a)$, with $a$ fixed, than the full generalization $C(x, y, p)$ required by the $\mathsf{ind}$ function. As we will see next, both notions are equivalent.

66. Path induction equivalence. Here we will see that the two path induction principles are equivalent. First, observe that general path induction $\mathsf{ind}_{=_A}$ can be obtained from based path induction $\mathsf{ind}'_{=_A}$ by treating the starting point $x$ as the parameter for the based version. We define:

$$\mathsf{ind}_{=_A}(C, c, x, y, p) :\equiv \mathsf{ind}'_{=_A}(x, \lambda y.\, \lambda p'.\, C(x, y, p'), c(x), y, p).$$

To see that this is well defined, consider the types required by $\mathsf{ind}_{=_A}$:

$$\begin{aligned} C\colon & \prod_{x,y\colon A} (x =_A y) \to \mathcal{U} \\ c\colon & \prod_{x\colon A} C(x, x, \mathsf{refl}_x) \\ x, y\colon & A \\ p\colon & x =_A y \end{aligned}$$

By fixing the first argument to $x$, we obtain the specific arguments required by $\mathsf{ind}'_{=_A}$ with base point $x$:

$$\begin{aligned} & D :\equiv \lambda y.\, \lambda p'.\, C(x, y, p')\colon \prod_{y\colon A} (x =_A y) \to \mathcal{U} \\ & d :\equiv c(x)\colon C(x, x, \mathsf{refl}_x) \\ & y\colon A \\ & p\colon x =_A y, \end{aligned}$$

which honors the signature of $\mathsf{ind}'_{=_A}$. Moreover,

$$\mathsf{ind}_{=_A}(C, c, x, x, \mathsf{refl}_x) \equiv \mathsf{ind}'_{=_A}(x, D, d, x, \mathsf{refl}_x) \equiv d \equiv c(x),$$

For the converse, fix $a\colon A$ and suppose we are given

$$D\colon \prod_{x\colon A} (a =_A x) \to \mathcal{U}, \quad \text{and} \quad d\colon D(a, \mathsf{refl}_a).$$

In order to derive $D$ using $\mathsf{ind}$ we define the motive $C$ as

$$C(u, v, q\colon u =_A v) :\equiv \prod_{s\colon a =_A u} D(u, s) \to D(v, s \cdot q). \tag{1.4}$$

For the base case $c(u)$ we need a term of type

$$C(u, u, \mathsf{refl}_u) \equiv \prod_{s\colon a =_A u} D(u, s) \to D(u, s \cdot \mathsf{refl}_u).$$

By 62. RIGHT UNIT LAW, there is a path $\mathsf{ru}(s)\colon s \cdot \mathsf{refl}_u =_{a =_A u} s$. Then, its inverse path satisfies $\mathsf{ru}(s)^{-1}\colon s =_{a =_A u} s \cdot \mathsf{refl}_u$.

Since we need to transform terms of $D(u, s)$ into terms of $D(u, s \cdot \mathsf{refl}_u)$, we define $P_u :\equiv \lambda s.\, D(u, s)$ and use the function $\mathsf{transport}^{P_u}$

$$c(u) :\equiv \lambda s.\, \mathsf{transport}^{P_u}(\mathsf{ru}(s)^{-1})\colon C(u, u, \mathsf{refl}_u). \tag{1.5}$$

By path induction, we obtain

$$E(u, v, q) :\equiv \mathsf{ind}_{=_A}(C, c, u, v, q)\colon C(u, v, q),$$

which yields a witness for the based induction:

$$f(x, p) :\equiv E(a, x, p)(\mathsf{refl}_a)(d).$$

This is well-typed because, according to (1.4), $E(a, x, p)(\mathsf{refl}_a)$ defines a map $D(a, \mathsf{refl}_a) \to D(x, \mathsf{refl}_a \cdot p)$, and since $\mathsf{refl}_a \cdot p \equiv p$, the codomain is precisely $D(x, p)$. Finally, we apply this map to $d\colon D(a, \mathsf{refl}_a)$ to get an element of $D(x, p)$.

To verify the computation rule, we have to prove that

$$f(a, \mathsf{refl}_a) \equiv d.$$

Expanding the definition,

$$\begin{aligned} f(a, \mathsf{refl}_a) &\equiv \mathsf{ind}_{=_A}(C, c, a, a, \mathsf{refl}_a)(\mathsf{refl}_a)(d) \\ &\equiv c(a)(\mathsf{refl}_a)(d). \end{aligned}$$

We reduce the term $c(a)(\mathsf{refl}_a)$:

$$\begin{aligned} c(a)(\mathsf{refl}_a) &\equiv \mathsf{transport}^{P_a}(\mathsf{ru}(\mathsf{refl}_a)^{-1}) && ;\ (1.5) \\ &\equiv \mathsf{transport}^{P_a}(\mathsf{refl}_a^{-1}) && ;\ \mathsf{ru}(\mathsf{refl}_a) \equiv \mathsf{refl}_a \\ &\equiv \mathsf{transport}^{P_a}(\mathsf{refl}_a) && ;\ \mathsf{refl}_a^{-1} \equiv \mathsf{refl}_a \\ &\equiv \lambda z.\, z. \end{aligned}$$

Hence,

$$f(a, \mathsf{refl}_a) \equiv (\lambda z.\, z)(d) \equiv d,$$

as desired.

## 1.12 Arithmetic of Natural Numbers

67. Additive identity. Since our definition of addition in $\mathbb{N}$ is by induction in the first operand, we know that $0 + n \equiv n$ for $n : \mathbb{N}$. It remains to show the following

**Lemma 1.12.1.** *For every $n : \mathbb{N}$ the type $n + 0 =_{\mathbb{N}} n$ is inhabited.*

*Proof.* We proceed by induction on $n$.

- **Base case ($n :\equiv 0$):** We must show $0 =_{\mathbb{N}} 0 + 0$. Since $0 + 0 \equiv 0$, this holds by reflexivity.
- **Inductive step:** We assume the hypothesis $h : n =_{\mathbb{N}} n + 0$ and must show $\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(n) + 0$. By the computation rules, the RHS reduces: $\mathsf{succ}(n) + 0 \equiv \mathsf{succ}(n + 0)$. Thus, the goal becomes $\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(n + 0)$.

  To complete the proof, we need to apply the function $\mathsf{succ}$ to both sides of the hypothesis $h$. This relies on the *principle of congruence* that states that functions respect equality.[22]

  However, we can also use 64. Transport as follows. Consider the type family $P : \mathbb{N} \to \mathcal{U}$ defined by

  $$P :\equiv \lambda k.\, \mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(k).$$

  Since $\mathsf{refl}_{\mathsf{succ}(n)} : P(n)$, we can evaluate

  $$\mathsf{transport}^P(h) : P(n) \to P(n + 0)$$

  at $\mathsf{refl}_{\mathsf{succ}(n)}$ to construct an inhabitant of $\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(n + 0)$, as desired.

□

68. Associativity of addition. Proving the associativity of the addition of natural numbers entails defining a function

$$\mathsf{assoc} : \prod_{i,j,k:\mathbb{N}} i + (j + k) =_{\mathbb{N}} (i + j) + k.$$

We proceed by induction on $i$. The type family $C : \mathbb{N} \to \mathcal{U}$ is given by:

$$C(i) :\equiv \prod_{j,k:\mathbb{N}} i + (j + k) =_{\mathbb{N}} (i + j) + k.$$

[22] Exercise 1.16.1.

To define the function $\mathsf{assoc} :\equiv \mathsf{ind}_{\mathbb{N}}(C, c_0, c_s)$, we need to specify:

$$\begin{aligned} c_0 &: C(0), \\ c_s &: \prod_{i:\mathbb{N}} C(i) \to C(\mathsf{succ}(i)). \end{aligned}$$

For the base case $c_0$, we need a witness of $0 + (j + k) =_{\mathbb{N}} (0 + j) + k$. From the definition of addition we know that $0 + n \equiv n$. Therefore, the equality reduces to $j + k =_{\mathbb{N}} j + k$, and we can define the witness by reflexivity:

$$c_0 :\equiv \lambda j.\, \lambda k.\, \mathsf{refl}_{j+k}.$$

For the inductive step $c_s$, we assume $i : \mathbb{N}$ and the inductive hypothesis $h : C(i)$. We must construct a proof for $\mathsf{succ}(i)$.

$$c_s :\equiv \lambda i.\, \lambda h.\, \lambda j.\, \lambda k.\, \ldots$$

where $h(j, k) : i + (j + k) =_{\mathbb{N}} (i + j) + k$.

Computing the goal, we obtain

$$\begin{aligned} \mathsf{succ}(i) + (j + k) &\equiv \mathsf{succ}(i + (j + k)) \\ &=_{\mathbb{N}} \mathsf{succ}((i + j) + k) \qquad ; \text{ by } h(j, k) + \text{congruence} \\ &\equiv \mathsf{succ}(i + j) + k \\ &\equiv (\mathsf{succ}(i) + j) + k. \end{aligned}$$

The equality in the second line is justified by applying the function $\mathsf{succ}$ to the path provided by the hypothesis $h(j, k)$. This relies on the *principle of congruence* ($\mathsf{ap}$), which we prove in Exercise 1.16.1. Thus, the expression for $c_s$ is

$$c_s :\equiv \lambda i.\, \lambda h.\, \lambda j.\, \lambda k.\, \mathsf{ap}_{\mathsf{succ}}(h(j, k)).$$

69. Successor injectivity. The *predecessor* function $\mathsf{pred} : \mathbb{N} \to \mathbb{N}$ is inductively defined by

$$\begin{aligned} \mathsf{pred}(0) &:\equiv 0, \\ \mathsf{pred}(\mathsf{succ}(n)) &:\equiv n. \end{aligned}$$

**Lemma 1.12.2.** *For every* $n : \mathbb{N}$, $\mathsf{succ}(n) \neq 0$.

*Proof.* We must construct a map of signature

$$\prod_{n:\mathbb{N}} (\mathsf{succ}(n) =_{\mathbb{N}} 0) \to \mathsf{0}.$$

Define a type family $P\colon \mathbb{N} \to \mathcal{U}$ by recursion:

$$P(0) :\equiv \mathsf{1}$$
$$P(\mathsf{succ}(n)) :\equiv \mathsf{0}.$$

Take $n\colon \mathbb{N}$ and suppose we have a path $p\colon \mathsf{succ}(n) =_{\mathbb{N}} 0$. By transport along the inverse path $p^{-1}\colon 0 =_{\mathbb{N}} \mathsf{succ}(n)$, for $\star\colon \mathsf{1}$, we obtain:

$$\mathsf{transport}^P(p^{-1})\colon P(0) \to P(\mathsf{succ}(n)).$$

The definition of $P$ implies that the desired map is $(n, p) \mapsto \mathsf{subst}_{\mathsf{P}}(p^{-1})(\star)$. □

**Lemma 1.12.3.** *For $n, m\colon \mathbb{N}$, if $\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(m)$ is inhabited, then $n =_{\mathbb{N}} m$ is inhabited.*

*Proof.* Fix $m\colon \mathbb{N}$. We have to define a function of type

$$\prod_{n\colon \mathbb{N}} (\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(m)) \to (n =_{\mathbb{N}} m).$$

This is accomplished by applying the congruence principle $\mathsf{ap}_{\mathsf{pred}}$ to every given path, namely

$$\lambda n.\, \lambda p.\, \mathsf{ap}_{\mathsf{pred}}(p).$$

□

70. The order relation on $\mathbb{N}$. The translation table (see 57. Predicate logic) implies that the order relations can be defined as

$$n \leq m :\equiv \sum_{k\colon \mathbb{N}} n + k =_{\mathbb{N}} m$$

and

$$n < m :\equiv (n \leq m) \times \neg(n =_{\mathbb{N}} m).$$

**Lemma 1.12.4.** *The following properties hold*

a) $0 \leq n$

b) $n \leq \mathsf{succ}(n)$

c) $n \leq n$

d) $n \leq m$ *and* $m \leq n$ *imply* $n =_{\mathbb{N}} m$

e) $n \leq m$ *and* $m \leq r$ *imply* $n \leq r$

f) $n \leq m$ *or* $m \leq n$

g) $n < 0$ *does not hold*

h) $\neg(n =_{\mathbb{N}} 0)$ *implies* $n =_{\mathbb{N}} \mathsf{succ}(\mathsf{pred}(n))$

*Proof.* (Sketch)

a) Take $k :\equiv n$. By the definition of addition (see 50. ADDITION OF NATURAL NUMBERS), we have the definitional equality $0 + n \equiv n$. Thus, $\mathsf{refl}_n$ has type $0 + n =_{\mathbb{N}} n$. The proof term is $\langle n, \mathsf{refl}_n \rangle$.

b) This relies on the equality $\mathsf{succ}(n) =_{\mathbb{N}} n+1$, which can be proved by induction following a similar approach to the one we sketched in 67. ADDITIVE IDENTITY for proving $n + 0 =_{\mathbb{N}} n$ .

c) This is because $n + 0 =_{\mathbb{N}} n$.

d) The standard mathematical argument goes like this: Take $k$ and $h$ satisfying $n + k =_{\mathbb{N}} m$ and $m + h =_{\mathbb{N}} n$. Then $n + (k + h) =_{\mathbb{N}} n$ and in order to derive $h =_{\mathbb{N}} 0$ and $k =_{\mathbb{N}} 0$, we need two theorems

$$n + s =_{\mathbb{N}} n \to s =_{\mathbb{N}} 0 \quad \text{and}$$
$$k + h =_{\mathbb{N}} 0 \to (k =_{\mathbb{N}} 0) \times (h =_{\mathbb{N}} 0).$$

All of this can be easily expressed and proved formally.

The first theorem follows by induction on $n$. The base case is trivial since $0 + s \equiv s$. The inductive step consists in showing a witness of

$$\mathsf{succ}(n) + s =_{\mathbb{N}} \mathsf{succ}(n) \to \mathsf{succ}(n + s) =_{\mathbb{N}} \mathsf{succ}(n),$$

which follows from $\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(m) \to n =_{\mathbb{N}} m$.

The second theorem follows from induction on $k$. The base case, with $k \equiv 0$, is trivial. The inductive step is also trivial because

$$\mathsf{succ}(k) + h \equiv \mathsf{succ}(k + h)$$

and so, by Lemma 1.12.2, there is no inhabitant of $\mathsf{succ}(k) + h =_{\mathbb{N}} 0$.

e) Take $k$ and $h$ so that $n + k =_{\mathbb{N}} m$ and $m + h =_{\mathbb{N}} r$. Then,

$$(n + k + h =_{\mathbb{N}} m + h) \times (m + h =_{\mathbb{N}} r)$$

is inhabited, and the result is a consequence of the transitivity of the equality.

f) We proceed by induction on $n$.

The base case $n =_{\mathbb{N}} 0$ is covered by part a).

For the inductive step, assume that for a fixed $n\colon \mathbb{N}$ and all $m\colon \mathbb{N}$, we have $(n \leq m) + (m \leq n)$. We must show that for all $m\colon \mathbb{N}$, we have

$$(\mathsf{succ}(n) \leq m) + (m \leq \mathsf{succ}(n)).$$

We proceed by induction on $m$.

For $m =_{\mathbb{N}} 0$, it suffices to note that $0 \leq \mathsf{succ}(n)$.

For the inductive step, assume the result holds for $m$, so we want to show

$$(\mathsf{succ}(n) \leq \mathsf{succ}(m)) + (\mathsf{succ}(m) \leq \mathsf{succ}(n)). \tag{1}$$

By the outer inductive hypothesis applied to $m$, we have a witness of

$$(n \leq m) + (m \leq n). \tag{2}$$

To construct a map from the type defined in (2) to the type defined in (1), it suffices to define two maps, one from $n \leq m$ and another from $m \leq n$, both of which are easily deduced by applying $\mathsf{succ}$.

This completes the induction on $m$, and therefore the induction on $n$.

g) By definition $(n < 0) \equiv (n \leq 0) \times \neg(n =_{\mathbb{N}} 0)$. Thus, if $u\colon n < 0$ then $\pi_1(u)\colon n \leq 0$ and, by part d), there exists $p\colon n =_{\mathbb{N}} 0$. Thus, $\pi_2(u)(p)\colon \mathsf{0}$, proving $\neg(n < 0)$.

h) Let $u\colon (n =_0) \to \mathsf{0}$. We proceed by induction on $n\colon \mathbb{N}$.

In the case $n \equiv 0$, the goal is obtained as

$$\mathsf{rec}_{\mathsf{0}}\big(0 =_{\mathbb{N}} \mathsf{succ}(\mathsf{pred}(0)), u(\mathsf{refl}_0)\big).$$

For the inductive step, assume the property holds for $n\colon \mathbb{N}$. We must construct a function for $\mathsf{succ}(n)$ with signature

$$\neg(\mathsf{succ}(n) =_{\mathbb{N}} 0) \to \big(\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(\mathsf{pred}(\mathsf{succ}(n)))\big).$$

Assume $u\colon \neg(\mathsf{succ}(n) =_{\mathbb{N}} 0)$. By definition, $\mathsf{pred}(\mathsf{succ}(n)) \equiv n$. Therefore, our goal reduces definitionally to:

$$\mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(n),$$

which is trivially inhabited by $\mathsf{refl}_{\mathsf{succ}(n)}$, regardless of $u$ and the inductive hypothesis. □

## 1.13 Strong Induction

**Theorem 1.13.1** [Strong Induction]. *Let $P\colon \mathbb{N} \to \mathcal{U}$ be a type family over the natural numbers. The principle of strong induction states that there exists a term of type:*

$$\Big(\prod_{n\colon \mathbb{N}}\Big(\prod_{m\colon \mathbb{N}} m < n \to P(m)\Big) \to P(n)\Big) \to \prod_{n\colon \mathbb{N}} P(n)$$

*where $m < n$ is the standard strict inequality type defined on $\mathbb{N}$.*

*Alternatively, to avoid explicit propositions for the inequality relation, one can recursively define a type family $P^{<}\colon \mathbb{N} \to \mathcal{U}$ that accumulates the sequence of previous values:*

$$\begin{aligned} P^{<}(0) &\equiv \mathsf{1} \\ P^{<}(\mathsf{succ}(n)) &\equiv P^{<}(n) \times P(n). \end{aligned}$$

*In this formulation, the strong induction principle provides a term of type:*

$$\Big(\prod_{n:\mathbb{N}} P^{<}(n) \to P(n)\Big) \to \prod_{n:\mathbb{N}} P(n).$$

*Proof.* Let $h\colon \prod_{n:\mathbb{N}} P^{<}(n) \to P(n)$ be given. We first construct a dependent function $g\colon \prod_{n:\mathbb{N}} P^{<}(n)$ by ordinary induction.

For the base case $n \equiv 0$, we must provide a term of type $P^{<}(0) \equiv \mathsf{1}$. We set $g(0) :\equiv \star$.

For the inductive step, let $n\colon \mathbb{N}$ and $x\colon P^{<}(n)$. We must construct a term of type $P^{<}(\mathsf{succ}(n)) \equiv P^{<}(n) \times P(n)$. This is given by the pair $(x, h(n)(x))$.

Applying the standard induction principle for $\mathbb{N}$ yields the desired function $g\colon \prod_{n:\mathbb{N}} P^{<}(n)$, namely

$$g :\equiv \mathsf{ind}_{\mathbb{N}}\big(P^{<}, \star, \lambda n.\, \lambda x.\, (x, h(n)(x))\big).$$

Finally, to construct the required term of type $\prod_{n:\mathbb{N}} P(n)$, let $n\colon \mathbb{N}$. Since $g(n)\colon P^{<}(n)$, we can define $\lambda n.\, h(n)(g(n))$, which concludes the proof.

□

## 1.14 Summary of Constructors and Eliminators

- $A \times B$

  Constructors: $(a, b)$

  Eliminators: $\pi_1, \pi_2$

$$\beta\colon \begin{cases} \pi_1(a,b) \equiv a \\ \pi_2(a,b) \equiv b \end{cases}$$
$$\eta\colon z \equiv (\pi_1(z), \pi_2(z))$$

- $\prod_{x:A} B(x)$

  Constructors: $\lambda x.\, \Phi$

ELIMINATORS: $\mathsf{ev}(f, a) \equiv f(a)$

$$\beta\colon (\lambda x.\, \Phi)(a) \equiv \Phi[x \leftarrow a]$$
$$\eta\colon f \equiv \lambda x.\, f(x)$$

- $\sum_{x:A} B(x)$

  CONSTRUCTORS: $\langle a, b \rangle$

  ELIMINATORS: $\pi_1, \pi_2$

$$\beta\colon \begin{cases} \pi_1(\langle a, b \rangle) \equiv a \\ \pi_2(\langle a, b \rangle) \equiv b \end{cases}$$
$$\eta\colon z \equiv \langle \pi_1(z), \pi_2(z) \rangle$$

- $A + B$

  CONSTRUCTORS: $\mathsf{inl}, \mathsf{inr}$

  ELIMINATORS: $\mathsf{ind}_{A+B}$ (case analysis)

$$\beta\colon \begin{cases} \mathsf{ind}_{A+B}(\mathsf{inl}(a), l, r) \equiv l(a) \\ \mathsf{ind}_{A+B}(\mathsf{inr}(b), l, r) \equiv r(b) \end{cases}$$
$$\eta\colon z \equiv \mathsf{ind}_{A+B}(z, \mathsf{inl}, \mathsf{inr})$$

- $2$

  CONSTRUCTORS: $0_2, 1_2$

  ELIMINATORS: $\mathsf{ind}_2$ (if-else)

$$\beta\colon \begin{cases} \mathsf{ind}_2(0_2, c_0, c_1) \equiv c_0 \\ \mathsf{ind}_2(1_2, c_0, c_1) \equiv c_1 \end{cases}$$
$$\eta\colon x \equiv \mathsf{ind}_2(x, 0_2, 1_2)$$

- $\mathbb{N}$

  CONSTRUCTORS: $0, \mathsf{succ}$

  ELIMINATORS: $\mathsf{rec}_{\mathbb{N}}$ (recursion)

$$\beta\colon \begin{cases} \mathsf{rec}_{\mathbb{N}}(0, c_0, c_s) \equiv c_0 \\ \mathsf{rec}_{\mathbb{N}}(\mathsf{succ}(n), c_0, c_s) \equiv c_s(n, \mathsf{rec}_{\mathbb{N}}(n, c_0, c_s)) \end{cases}$$
$$\eta\colon n \equiv \mathsf{rec}_{\mathbb{N}}(n, 0, \mathsf{succ})$$

## 1.15 Exercises

**Exercise 1.15.1.**

a) *Given* $f\colon A \to B$ *and* $g\colon B \to C$, *define their composite* $g \circ f\colon A \to C$.

b) *Show that we have* $h \circ (g \circ f) \equiv (h \circ g) \circ f$.

*Solution.*

a) We define the composition $g \circ f\colon A \to C$ as

$$g \circ f :\equiv \lambda x.\, g(f(x)).$$

To check that this is valid we have to verify that $g(f(x))\colon C$, assuming $x\colon A$. But, given that $f\colon A \to B$, we know that $f(x)\colon B$. And since $g\colon B \to C$, we obtain $g(f(x))\colon C$.

b) Based on part a), we define the polymorphic composition operator as

$$\circ\colon \prod_{A,B,C:\mathcal{U}} \prod_{g:\, B\to C} \prod_{f:\, A\to B} A \to C,$$

with specification

$$\circ :\equiv \lambda A.\, \lambda B.\, \lambda C.\, \lambda g.\, \lambda f.\, \lambda x.\, g(f(x)).$$

Therefore, assuming $h\colon C \to D$, $g\colon B \to C$ and $f\colon A \to B$, we obtain

$$g \circ_{ABC} f\colon A \to C \quad \text{and} \quad h \circ_{BCD} g\colon B \to D.$$

Therefore,

$$h \circ_{ACD} (g \circ_{ABC} f)\colon A \to D \quad \text{and} \quad (h \circ_{BCD} g) \circ_{ABD} f\colon A \to D.$$

Moreover,

$$\begin{aligned} h \circ_{ACD} (g \circ_{ABC} f) &:\equiv \lambda x.\, h((g \circ f)(x)) \\ &\qquad \lambda x.\, h((\lambda y.\, g(f(y)))(x)) \\ &\qquad \lambda x.\, h(g(f(x)) \qquad\qquad \text{; by part a)} \end{aligned}$$

and

$$\begin{aligned} (h \circ_{BCD} g) \circ_{ABD} f &:\equiv \lambda x.\, (h \circ g)(f(x)) \\ &\qquad \lambda x.\, (\lambda y.\, h(g(y)))(f(x)) \\ &\qquad \lambda x.\, h(g(f(x)) \qquad\qquad \text{; } h \circ g :\equiv \lambda y.\, h(g(y)) \end{aligned}$$

show that $h \circ_{ACD} (g \circ_{ABC} f) \equiv (h \circ_{BCD} g) \circ_{ABD} f$.

□

**Exercise 1.15.2.** *Derive the recursion principle for products* $\mathsf{rec}_{A\times B}$ *using only the projections, and verify that the definitional equalities are valid. Do the same for* $\Sigma$*-types.*

*Solution.* Define

$$\mathsf{rec}_{A\times B}\colon \prod_{C:\mathcal{U}} (A \to B \to C) \to A \times B \to C$$

specified as

$$\mathsf{rec}_{A\times B}(C, g) :\equiv \lambda x.\, g(\pi_1(x))(\pi_2(x)).$$

Given $x\colon A\times B$ we have $\pi_1(x)\colon A$ and $\pi_2(x)\colon B$. It follows that $g(\pi_1(x))\colon B \to C$ and so $g(\pi_1(x))(\pi_2(x))\colon C$. Hence, the function is well defined. Moreover, if we evaluate the function at a canonical element $(a, b)$, since $\pi_1((a, b)) \equiv a$ and $\pi_2((a, b)) \equiv b$, we obtain

$$g(\pi_1(a, b))(\pi_2((a, b))) \equiv g(a)(b).$$

Then,

$$\mathsf{rec}_{A\times B}(C, g)((a, b)) \equiv g(a)(b),$$

which is the definition of the recursor function (cf. 28. Products revisited).

For $\Sigma$-types, we define

$$\mathsf{rec}_{\sum_{x:A} B(x)}\colon \prod_{C:\mathcal{U}} \Big(\prod_{x:A} B(x) \to C\Big) \to \Big(\sum_{x:A} B(x)\Big) \to C$$

and specify it as

$$\mathsf{rec}_{\sum_{x:A} B(x)} :\equiv \lambda C.\, \lambda g.\, \lambda p.\, g(\pi_1(p))(\pi_2(p)).$$

To see that this is well defined, let us first observe that $g$ can be applied to $\pi_1(p)$. We have

$$g\colon \prod_{x:A} B(x) \to C.$$

The signature of $\pi_1$ implies that $\pi_1(p)\colon A$. Hence, $g(\pi_1(p))\colon B(\pi_1(p)) \to C$.

Second, since the signature of $\pi_2$ guarantees that $\pi_2(p)\colon B(\pi_1(p))$, we also see that $g(\pi_1(p))$ can be applied to $\pi_2(p)$ to get an element of $C$.

It remains to be shown that the recursion works as expected. To verify this we have to evaluate the function at canonical elements. In this case it suffices to compute the $\lambda$-expression when $p \equiv (a, b)$ with $b\colon B(a)$. Here we have

$$\begin{aligned}\mathsf{rec}_{\sum_{x:A} B(x)}(C, g)(\langle a, b\rangle) &\equiv \big(\lambda p.\, g(\pi_1(p))(\pi_2(p))\big)(\langle a, b\rangle)\\ &\equiv g(\pi_1(\langle a, b\rangle))(\pi_2(\langle a, b\rangle))\\ &\equiv g(a)(b),\end{aligned}$$

because $\pi_1(\langle a, b\rangle) \equiv a$ and $\pi_2(\langle a, b\rangle) \equiv b$. Since this coincides with (1.2), our definition of $\mathsf{rec}$ recovers the definition given in 37. RECURSION IN $\Sigma$-TYPES.

□

**Exercise 1.15.3.** *Assuming as given only the iterator for natural numbers*

$$\mathsf{iter}\colon \prod_{C:\mathcal{U}} C \to (C \to C) \to \mathbb{N} \to C$$

*with the defining equations*

$$\begin{aligned}\mathsf{iter}(C, c_0, c_s, 0) &:\equiv c_0,\\ \mathsf{iter}(C, c_0, c_s, \mathsf{succ}(n)) &:\equiv c_s(\mathsf{iter}(C, c_0, c_s, n)),\end{aligned}$$

*derive a function having the type of the recursor* $\mathsf{rec}_{\mathbb{N}}$. *Show that the defining equations of the recursor hold propositionally for this function, using the induction principle for* $\mathbb{N}$.

*Solution.* Fix $C\colon \mathcal{U}$. We have to define

$$\mathsf{rec}\colon \prod_{C:\mathcal{U}} C \to (\mathbb{N} \to C \to C) \to \mathbb{N} \to C$$

using $\mathsf{iter}$. Suppose we are given $c_0\colon C$ and $c_s\colon \mathbb{N} \to C \to C$. We have to produce a function $\mathsf{rec}(C, c_0, c_s)\colon \mathbb{N} \to C$ such that

$$\begin{aligned}\mathsf{rec}(C, c_0, c_s, 0) &:\equiv c_0,\\ \mathsf{rec}(C, c_0, c_s, \mathsf{succ}(n)) &:\equiv c_s(n, \mathsf{rec}(C, c_0, c_s, n)).\end{aligned}$$

Let us first try to derive the function $\mathsf{pred}\colon \mathbb{N} \to \mathbb{N}$ from $0\colon \mathbb{N}$, $\mathsf{succ}\colon \mathbb{N} \to \mathbb{N}$ and $\mathsf{iter}$. For consider the case where $C$ is $\mathbb{N} \times \mathbb{N}$. We have to define a function

$$p_s\colon (\mathbb{N} \times \mathbb{N}) \to (\mathbb{N} \times \mathbb{N})$$

and an element $p_0\colon \mathbb{N} \times \mathbb{N}$. Invoking $\mathsf{iter}$, with these two pieces of data we obtain a function $\mathsf{q}\colon \mathbb{N} \to (\mathbb{N} \times \mathbb{N})$, given by $\lambda n.\, \mathsf{iter}(\mathbb{N} \times \mathbb{N}, p_0, p_s, n)$ which satisfies

$$\begin{aligned}\mathsf{q}(0) &:\equiv p_0\\ \mathsf{q}(\mathsf{succ}(n)) &:\equiv p_s(\mathsf{q}(n)).\end{aligned}$$

Let $p_0 :\equiv (0, 0)$ and $p_s(n, m) :\equiv (\mathsf{succ}(n), n)$. Using the projections we can write $\mathsf{q}(n) \equiv (q_1(n), q_2(n))$, where $q_1, q_2\colon \mathbb{N} \to \mathbb{N}$. Therefore,

$$\begin{aligned}q_1(0) &\equiv 0\\ q_2(0) &\equiv 0\\ q_1(\mathsf{succ}(n)) &\equiv \mathsf{succ}(q_1(n))\\ q_2(\mathsf{succ}(n)) &\equiv q_1(n).\end{aligned}$$

These equations suggest that $q_1$ is the identity and $q_2$ is $\mathsf{pred}$. However, as we will see below, such equalities are propositional.

To define $\mathsf{rec}$, we will apply $\mathsf{iter}$ to $\hat{C} :\equiv \mathbb{N} \to C$. For $\hat{c}_0$ we take the constant function $\lambda n.\, c_0$. For $\hat{c}_s$

$$\begin{aligned} \hat{c}_s &: (\mathbb{N} \to C) \to (\mathbb{N} \to C) \\ \hat{c}_s(\theta) &:\equiv \lambda n.\, c_s(n)(\theta(q_2(n))). \end{aligned}$$

This is well defined: given $n : \mathbb{N}$, we have $c_s(n) : C \to C$ and $\theta(q_2(n)) : C$, and so $c_s(n)(\theta(q_2(n))) : C$.

Now we can define

$$\mathsf{rec}(C, c_0, c_s, n) :\equiv \mathsf{iter}(\hat{C}, \hat{c}_0, \hat{c}_s, q_1(n))(q_2(q_1(n))),$$

which satisfies

$$\begin{aligned} \mathsf{rec}(C, c_0, c_s, 0) &\equiv \hat{c}_0(0) \\ &\equiv (\lambda n.\, c_0)(0) \\ &\equiv c_0. \end{aligned}$$

and

$$\begin{aligned} \mathsf{rec}(C, c_0, c_s, \mathsf{succ}(n)) &\equiv \mathsf{iter}(\hat{C}, \hat{c}_0, \hat{c}_s, q_1(\mathsf{succ}(n)))(q_2(q_1(\mathsf{succ}(n))) \\ &\equiv \mathsf{iter}\big(\hat{C}, \hat{c}_0, \hat{c}_s, \mathsf{succ}(q_1(n))\big)(q_1(q_1(n))) \\ &\equiv \hat{c}_s\big(\mathsf{iter}(\hat{C}, \hat{c}_0, \hat{c}_s, q_1(n))\big)(q_1(q_1(n))) \\ &\equiv c_s\big(q_1(n)\big)\big(\mathsf{iter}(\hat{C}, \hat{c}_0, \hat{c}_s, q_1(n))\big)(q_2(q_1(q_1(n)))) \\ &\equiv c_s\big(q_1(n)\big)\big(\mathsf{rec}(C, c_0, c_s, q_1(n))\big) \\ &\equiv c_s\big(q_1(n), \mathsf{rec}(C, c_0, c_s, q_1(n))\big). \end{aligned}$$

Thus, the question of proving that $\mathsf{rec}$ is propositionally equal to the recursion principle $\mathsf{rec}_{\mathbb{N}}$ for $\mathbb{N}$, would follow from the fact that a function $f : \mathbb{N} \to \mathbb{N}$ that satisfies $f(0) \equiv 0$ and $f(\mathsf{succ}(n)) \equiv \mathsf{succ}(n)$ is propositionally equal to the identity, i.e., $f =_{\mathbb{N}\to\mathbb{N}} \mathsf{id}_{\mathbb{N}}$.

However, strictly speaking, that would be too much for our needs, since the equality $q_1(n) =_{\mathbb{N}} n$ for $n : \mathbb{N}$ would imply by congruence[23] that $\mathsf{rec}$ satisfies the recursion principle equations propositionally, which is what we want.

In other words, we need a witness of

$$\prod_{n:\mathbb{N}} q_1(n) =_{\mathbb{N}} n.$$

To construct it, we use the induction principle 32. INDUCTION for the type family

$$C : \mathbb{N} \to \mathcal{U}, \quad C(n) :\equiv q_1(n) =_{\mathbb{N}} n,$$

[23] See Exercise 1.16.1.

which requires a constant $c_0 : C(0)$ and a step function

$$c_s : \prod_{n:\mathbb{N}} C(n) \to C(\mathsf{succ}(n)).$$

The induction principle yields

$$\mathsf{ind}_{\mathbb{N}}(C, c_0, c_s) : \prod_{n:\mathbb{N}} C(n).$$

For $c_0$, we define it as $\mathsf{refl}_0$ because $q_1(0) \equiv 0$. For the inductive step, given $n : \mathbb{N}$ and a hypothesis $h : C(n)$ (i.e., $h : q_1(n) =_{\mathbb{N}} n$), we define $c_s(n, h)$ by applying $\mathsf{succ}$ to $h$ via congruence. This provides a witness for $q_1(\mathsf{succ}(n)) =_{\mathbb{N}} \mathsf{succ}(n)$, since $q_1(\mathsf{succ}(n)) \equiv \mathsf{succ}(q_1(n))$.

□

**Exercise 1.15.4.** *Show that if we define* $A + B :\equiv \sum_{x:2} \mathsf{rec}_2(\mathcal{U}, A, B, x)$*, then we can give a definition of* $\mathsf{ind}_{A+B}$ *for which the definitional equalities stated in* 43. Coproduct induction *hold.*

*Solution.* This is a complement to 47. Coproducts as Indexed Disjoint Unions, where the notation for the coproduct is $A \oplus B$. To formulate the induction principle, we must first define the injection functions for this specific Σ-type construction. The definitional equalities

$$\mathsf{rec}_2(\mathcal{U}, A, B, 0_2) \equiv A \quad \text{and} \quad \mathsf{rec}_2(\mathcal{U}, A, B, 1_2) \equiv B,$$

imply that we can define the injections into $A \oplus B$ as

$$\mathsf{in}_A(x) :\equiv \langle 0_2, x\rangle \quad \text{and} \quad \mathsf{in}_B(y) :\equiv \langle 1_2, y\rangle.$$

The induction function we have to define is

$$\mathsf{ind}_{A\oplus B} : \prod_{C:\, A\oplus B\to\mathcal{U}} \prod_{x:\, A} C(\mathsf{in}_A(x)) \to \prod_{y:\, B} C(\mathsf{in}_B(y)) \to \prod_{s:\, A\oplus B} C(s).$$

Fix $C : A \oplus B \to \mathcal{U}$. Given two dependent functions

$$g_0 : \prod_{x:\, A} C(\mathsf{in}_A(x)) \quad \text{and} \quad g_1 : \prod_{y:\, B} C(\mathsf{in}_B(y)),$$

we need to define a function with signature

$$f : \prod_{s:\, A\oplus B} C(s).$$

To do this, we must use the induction principle of the Σ-type, which requires (see 37. Recursion in Σ-types) constructing a function $g$ with the following signature:

$$g : \prod_{t:\, 2} \prod_{z:\, \mathsf{rec}_2(\mathcal{U},A,B,t)} C(\langle t, z\rangle).$$

As a result, we will obtain

$$f :\equiv \mathsf{ind}_{\sum_{t:2} \mathsf{rec}_2(\mathcal{U},A,B,t)}(C, g).$$

We can construct this $g$ using boolean induction:

$$g :\equiv \lambda t\colon 2.\, \mathsf{ind}_2(D, g_0, g_1, t),$$

where the motive $D\colon 2 \to \mathcal{U}$ is given by

$$D(t) :\equiv \prod_{z:\, \mathsf{rec}_2(\mathcal{U},A,B,t)} C(\langle t, z\rangle).$$

Applying the definitional equalities of $\mathsf{rec}_2$, we obtain $D(0_2) \equiv \prod_{x:\, A} C(\mathsf{in}_A(x))$ and $D(1_2) \equiv \prod_{y:\, B} C(\mathsf{in}_B(y))$.

Finally, we must verify that the definitional equalities hold. For any $x\colon A$, the evaluation rules for the $\Sigma$-type and the boolean type yield

$$\begin{aligned} f(\mathsf{in}_A(x)) &\equiv f(\langle 0_2, x\rangle) \\ &\equiv g(0_2, x) \\ &\equiv \mathsf{ind}_2(D, g_0, g_1, 0_2)(x) \\ &\equiv g_0(x). \end{aligned}$$

By a symmetric argument, for any $y\colon B$, we have $f(\mathsf{in}_B(y)) \equiv g_1(y)$. This completes the construction. □

**Exercise 1.15.5.**

a) *Define multiplication and exponentiation using* $\mathsf{rec}_{\mathbb{N}}$.

b) *Verify that* $(\mathbb{N}, +, 0, -, 1)$ *is a semiring using only* $\mathsf{ind}_{\mathbb{N}}$.

c) *Verify that* $m^{n+n'} =_{\mathbb{N}} m^n \cdot m^{n'}$.

*Solution.*

a) Multiplication was defined in 51. Multiplication of natural numbers.

For exponentiation, we take $C :\equiv \mathbb{N} \to \mathbb{N}$ and define

$$\begin{aligned} &c_0\colon \mathbb{N} \to \mathbb{N} \\ &c_0 :\equiv \lambda m.\, 1, \\ &c_s\colon \mathbb{N} \to (\mathbb{N} \to \mathbb{N}) \to (\mathbb{N} \to \mathbb{N}) \\ &c_s :\equiv \lambda n.\, \lambda g\colon \mathbb{N} \to \mathbb{N}.\, \lambda m.\, g(m) \cdot m. \end{aligned}$$

From (1.1), we obtain

$$\mathsf{rec}_{\mathbb{N}}(C, c_0, c_s)\colon \mathbb{N} \to (\mathbb{N} \to \mathbb{N}).$$

We define exponentiation as

$$\mathsf{exp} :\equiv \lambda n.\, \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, n).$$

This definition satisfies

$$\begin{aligned}
\mathsf{exp}(0, m) &\equiv \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, 0)(m) \\
&\equiv c_0(m) \\
&\equiv 1, \\
\mathsf{exp}(\mathsf{succ}(n), m) &\equiv \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, \mathsf{succ}(n))(m) \\
&\equiv c_s(n, \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, n))(m) \\
&\equiv \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, n)(m) \cdot m \\
&\equiv \mathsf{exp}(n, m) \cdot m.
\end{aligned}$$

b) From 67. Additive identity and 68. Associativity of addition, we know that 0 is additive identity of $\mathbb{N}$ and that addition is associative. Therefore, we have to prove the following propositions[24]

1. *Annihilation.* Since $0 \cdot n \equiv n$, it suffices to show that $n \cdot 0 =_{\mathbb{N}} n$ by induction on $n \colon \mathbb{N}$.

   The base case is trivial because $0 \cdot 0 \equiv 0$ by definition.

   For the inductive step, take $p \colon n \cdot 0 =_{\mathbb{N}} 0$. We have to see that $\mathsf{succ}(n) \cdot 0 =_{\mathbb{N}} 0$ is inhabited. Since $\mathsf{succ}(n) \cdot 0 \equiv n \cdot 0 + 0$, the conclusion follows from path concatenation: take $q \colon n \cdot 0 + 0 =_{\mathbb{N}} n \cdot 0$ and obtain $q \cdot p \colon \mathsf{succ}(n) \cdot 0 =_{\mathbb{N}} 0$.

2. *Left distributivity.* Define the motive $C \colon \mathbb{N} \to \mathcal{U}$ by

   $$D(i) :\equiv \prod_{j,k \colon \mathbb{N}} i \cdot (j + k) =_{\mathbb{N}} i \cdot j + i \cdot k.$$

   Then

   $$D(0) \equiv \prod_{j,k \colon \mathbb{N}} 0 \cdot (i + j) =_{\mathbb{N}} 0 \cdot i + 0 \cdot j,$$

   which is judgmentally equal to $\prod_{i,j \colon \mathbb{N}} 0 =_{\mathbb{N}} 0$ because $0 \equiv 0 + 0$. Thus, the constant map $d_0 \colon \mathbb{N} \times \mathbb{N} \to \mathsf{refl}_0$ is a witness.

   For the inductive step take $p \colon D(i)$ and observe that

   $$\begin{aligned}
   \mathsf{succ}(i) \cdot (j + k) &\equiv i \cdot (j + k) + (j + k) \\
   &=_{\mathbb{N}} (i \cdot j + i \cdot k) + (j + k) && ; \text{ by } p \\
   &=_{\mathbb{N}} (i \cdot j + j) + (i \cdot k + k) && ; +\text{assoc \& } +\text{comm} \\
   &=_{\mathbb{N}} \mathsf{succ}(i) \cdot j + \mathsf{succ}(i) \cdot k,
   \end{aligned}$$

[24] For commutativity of addition see Exercise 1.15.12.

where we have used congruence[25] in obvious ways. The conclusion follows from 61. PATH CONCATENATION.

3. *Multiplicative identity.* First observe that

$$1 \cdot n \equiv \mathsf{succ}(0) \cdot n \equiv 0 \cdot n + n \equiv 0 + n \equiv n.$$

To prove $n \cdot 1 =_{\mathbb{N}} n$, we proceed by induction on $n$. The case $n \equiv 0$ is trivial: $n \cdot 1 \equiv 0 \cdot 1 \equiv 0 \equiv n$. Suppose $p\colon n \cdot 1 =_{\mathbb{N}} n$. Then

$$\begin{aligned} \mathsf{succ}(n) \cdot 1 &\equiv n \cdot 1 + 1 \\ &=_{\mathbb{N}} n + 1 && ;\ p + \text{congruence} \\ &=_{\mathbb{N}} 1 + n && ;\ \text{commutativity} \\ &\equiv \mathsf{succ}(n). \end{aligned}$$

4. *Multiplication commutativity.* Define the motive $C\colon \mathbb{N} \to \mathcal{U}$ by

$$C(i) :\equiv \prod_{j:\mathbb{N}} i \cdot j =_{\mathbb{N}} j \cdot i.$$

The base case, where $i :\equiv 0$, follows from annihilation (part 1).

For the inductive step, take $p\colon i \cdot j =_{\mathbb{N}} j \cdot i$. We have to show that $\mathsf{succ}(i) \cdot j =_{\mathbb{N}} j \cdot \mathsf{succ}(i)$. Using $\mathsf{succ}(i) \cdot j \equiv i \cdot j + j$ and left distributivity (part 2), we obtain

$$\begin{aligned} i \cdot j + j &=_{\mathbb{N}} j \cdot i + j && ;\ \text{by } p \\ &=_{\mathbb{N}} j \cdot 1 + j \cdot i && ;\ +\text{commutativity \& mult. identity} \\ &=_{\mathbb{N}} j \cdot (1 + i) && ;\ \text{left distributivity} \\ &\equiv j \cdot \mathsf{succ}(i), \end{aligned}$$

and the conclusion follows from path concatenation.

5. Right distributivity. Directly follows from commutativity (part 4) and left distributivity (part 2).

6. *Multiplication associativity.* Define the motive $A\colon \mathbb{N} \to \mathcal{U}$ given by

$$A(i) :\equiv \prod_{j,k:\mathbb{N}} (i \cdot j) \cdot k =_{\mathbb{N}} i \cdot (j \cdot k).$$

The base case is

$$A(0) \equiv \prod_{j,k:\mathbb{N}} (0 \cdot j) \cdot k =_{\mathbb{N}} 0 \cdot (j \cdot k),$$

which is trivial because, by definition, $0 \cdot n \equiv n$ for $n\colon \mathbb{N}$.

[25] Exercise 1.16.1.

In the inductive step we assume $p\colon A(i)$ and must exhibit a witness of

$$(\mathsf{succ}(i) \cdot j) \cdot k =_{\mathbb{N}} \mathsf{succ}(i) \cdot (j \cdot k).$$

For

$$\begin{aligned}(\mathsf{succ}(i) \cdot j) \cdot k &\equiv (i \cdot j + j) \cdot k \\ &=_{\mathbb{N}} (i \cdot j) \cdot k + j \cdot k \qquad \text{; right distributivity} \\ &=_{\mathbb{N}} i \cdot (j \cdot k) + j \cdot k \qquad \text{; by } p \\ &\equiv \mathsf{succ}(i) \cdot (j \cdot k).\end{aligned}$$

c) Consider the motive $E\colon \mathbb{N} \to \mathcal{U}$ given by

$$E :\equiv \prod_{n,m,n'\colon \mathbb{N}} m^{n+n'} =_{\mathbb{N}} m^n \cdot m^{n'}.$$

Fix $m$ and $n'$. To construct a witness of $E(m, n')$ we proceed by induction on $n$. For the base case, we have

$$\begin{aligned}m^{0+n'} &\equiv m^{n'} \qquad \text{; (1.4)} \\ &\equiv 1 \cdot m^{n'} \\ &\equiv m^0 \cdot m^{n'} \qquad \text{; part a)}\end{aligned}$$

Suppose that $p\colon E(m, n')(n)$. Fix $m, n'\colon \mathbb{N}$ and put

$$q :\equiv p(m, n')\colon m^{n+n'} =_{\mathbb{N}} m^n \cdot m^{n'}.$$

We have to construct a path for

$$m^{\mathsf{succ}(n)+n'} =_{\mathbb{N}} m^{\mathsf{succ}(n)} \cdot m^{n'}.$$

Now

$$\begin{aligned}m^{\mathsf{succ}(n)+n'} &\equiv m^{\mathsf{succ}(n+n')} \\ &\equiv m^{n+n'} \cdot m \\ &=_{\mathbb{N}} (m^n \cdot m^{n'}) \cdot m \qquad \text{; by } q \text{ + congruence} \\ &=_{\mathbb{N}} (m^n \cdot m) \cdot m^{n'} \qquad \text{; associativity \& commutativity} \\ &=_{\mathbb{N}} m^{\mathsf{succ}(n)} \cdot m^{n'}.\end{aligned}$$

□

**Exercise 1.15.6.** *Define the type family* $\mathsf{Fin}\colon \mathbb{N} \to \mathcal{U}$ *mentioned at the end of* 21. Type Families*, and the dependent function* $\mathsf{fmax}\colon \prod_{n\colon \mathbb{N}} \mathsf{Fin}(\mathsf{succ}(n))$ *mentioned in* 22. Dependent functions (a.k.a. Π-types).

*Solution.* The $\mathsf{Fin}$ type family requires a motive $C$ such that $\mathsf{Fin}(n) : C(n)$. Since $\mathsf{Fin}(n)$ is a type, $C$ must be $\mathcal{U}$.

To define $\mathsf{Fin}$ using the recursion principle, we specify $c_0 : \mathcal{U}$ and $c_s : \mathbb{N} \to \mathcal{U} \to \mathcal{U}$ as follows

$$\begin{aligned} c_0 &:\equiv \mathsf{0}, && \text{; empty coproduct} \\ c_s &:\equiv \lambda n.\, \lambda A.\, A + \mathsf{1}. && \text{; } A + \text{empty product} \end{aligned}$$

As a result, we can define $\mathsf{Fin} :\equiv \mathsf{rec}_{\mathbb{N}}(\mathcal{U}, c_0, c_s) : \mathbb{N} \to \mathcal{U}$, which satisfies

$$\mathsf{Fin}(0) :\equiv \mathsf{0} \quad \text{and} \quad \mathsf{Fin}(\mathsf{succ}(n)) \equiv \mathsf{Fin}(n) + \mathsf{1}.$$

To define $\mathsf{fmax}$ we simply declare

$$\begin{aligned} \mathsf{fmax} : \mathbb{N} &\to \mathsf{Fin}(n) + \mathsf{1} \\ n &\mapsto \mathsf{inr}_n(\star), \end{aligned}$$

where $\mathsf{inr}_n$ denotes the $n$th right inclusion of $\mathsf{1}$ into $\mathsf{Fin}(n) + \mathsf{1}$. This completes the proof because $\mathsf{Fin}(n) + \mathsf{1} \equiv \mathsf{Fin}(\mathsf{succ}(n))$.

□

**Exercise 1.15.7.** *Show that the Ackermann function* $\mathsf{ack} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ *is definable using only* $\mathsf{rec}_{\mathbb{N}}$ *satisfying the following equations:*

$$\begin{aligned} \mathsf{ack}(0, n) &\equiv \mathsf{succ}(n), \\ \mathsf{ack}(\mathsf{succ}(m), 0) &\equiv \mathsf{ack}(m, 1), \\ \mathsf{ack}(\mathsf{succ}(m), \mathsf{succ}(n)) &\equiv \mathsf{ack}(m, \mathsf{ack}(\mathsf{succ}(m), n)). \end{aligned}$$

*Solution.* Take the motive $C :\equiv \mathbb{N} \to \mathbb{N}$. To use the $\mathsf{rec}_{\mathbb{N}}$ function we need to define

$$c_0 : \mathbb{N} \to \mathbb{N}, \quad \text{and} \quad c_s : \mathbb{N} \to (\mathbb{N} \to \mathbb{N}) \to (\mathbb{N} \to \mathbb{N}).$$

According with equations (1.3) for the case where $f$ is $\mathsf{ack}$, by defining

$$\mathsf{ack}(m, -) :\equiv \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, m)$$

we obtain

$$\begin{aligned} \mathsf{ack}(0, n) &\equiv c_0(n) \\ \mathsf{ack}(\mathsf{succ}(m), -) &\equiv c_s(m, \mathsf{ack}(m, -)) \end{aligned}$$

The equation $\mathsf{ack}(0, n) \equiv \mathsf{succ}(n)$, that we must honor, makes the definition of $c_0$ straightforward: $c_0 :\equiv \mathsf{succ}$.

To define $c_s(m, g) : \mathbb{N} \to \mathbb{N}$, we fix $m : \mathbb{N}$ and $g : \mathbb{N} \to \mathbb{N}$, with $g$ playing the role of $\mathsf{ack}(m, -)$, and proceed by recursion on $n$. We need to provide two pieces of information

$$\begin{aligned} d_0 &: \mathbb{N} \\ d_s &: \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \end{aligned}$$

where $d_0 \equiv c_s(m, g)(0)$. Since $c_s$ is the step function on $m$ for $\mathsf{ack}(m, -)$, we deduce that

$$d_0 \equiv \mathsf{ack}(\mathsf{succ}(m), 0) \equiv \mathsf{ack}(m, 1).$$

Therefore, we must specify $d_0 :\equiv g(1)$.

Since $c_s(m, g)$ represents $\mathsf{ack}(\mathsf{succ}(m), -)$, and

$$d_s(n, c_s(m, g)(n)) \equiv c_s(m, g)(\mathsf{succ}(n)),$$

we deduce that $d_s(n, c_s(m, g)(n))$ must be a representative of

$$\mathsf{ack}(\mathsf{succ}(m), \mathsf{succ}(n)) \equiv \mathsf{ack}(m, \mathsf{ack}(\mathsf{succ}(m), n)).$$

Therefore, if we define

$$d_s :\equiv \lambda n, r.\, g(r), \tag{1.1}$$

we obtain

$$c_s :\equiv \lambda m, g.\, \mathsf{rec}_{\mathbb{N}}(\mathbb{N}, g(1), d_s). \tag{1.2}$$

Now we can define the full function $\mathsf{ack}$ using the outer recursion on $m$:

$$\mathsf{ack} :\equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N} \to \mathbb{N}, \mathsf{succ}, c_s).$$

Let us verify the three properties of the Ackermann function.

$\to$ $\mathsf{ack}(0, n) \equiv \mathsf{succ}(n)$

$$\begin{aligned} \mathsf{ack}(0, n) &\equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N} \to \mathbb{N}, c_0, c_s, 0)(n) \\ &\equiv c_0(n) \\ &\equiv \mathsf{succ}(n). \end{aligned}$$

$\to$ $\mathsf{ack}(\mathsf{succ}(m), 0) \equiv \mathsf{ack}(m, 1)$

$$\begin{aligned} \mathsf{ack}(\mathsf{succ}(m), 0) &\equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N} \to \mathbb{N}, c_0, c_s, \mathsf{succ}(m))(0) \\ &\equiv c_s(m, \mathsf{ack}(m))(0) \\ &\equiv \big(\lambda g.\, \mathsf{rec}_{\mathbb{N}}(\mathbb{N}, g(1), d_s)\big)(\mathsf{ack}(m))(0) \qquad ; (1.2) \\ &\equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N}, \mathsf{ack}(m)(1), d_s, 0) \\ &\equiv \mathsf{ack}(m)(1) \\ &\equiv \mathsf{ack}(m, 1). \end{aligned}$$

$\to$ $\mathsf{ack}(\mathsf{succ}(m), \mathsf{succ}(n)) \equiv \mathsf{ack}(m, \mathsf{ack}(\mathsf{succ}(m), n))$

$$\begin{aligned}
\mathsf{ack}(\mathsf{succ}(m), \mathsf{succ}(n)) &\equiv c_s(m, \mathsf{ack}(m))(\mathsf{succ}(n)) \\
&\equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N}, \mathsf{ack}(m)(1), d_s, \mathsf{succ}(n)) \\
&\equiv d_s\big(n, \mathsf{rec}_{\mathbb{N}}(\mathbb{N}, \mathsf{ack}(m)(1), d_s, n)\big) \\
&\equiv d_s\big(n, c_s(m, \mathsf{ack}(m))(n)\big) && ; (1.2)
\end{aligned}$$

(at this point the variable $g$ of (1.1) is bound to $\mathsf{ack}(m)$; consequently)

$$\begin{aligned}
&\equiv d_s\big(n, \mathsf{ack}(\mathsf{succ}(m), n)\big) && ; \text{recursion} \\
&\equiv \mathsf{ack}(m)\big(\mathsf{ack}(\mathsf{succ}(m), n)\big) && ; (1.1) \\
&\equiv \mathsf{ack}(m, \mathsf{ack}(\mathsf{succ}(m), n)).
\end{aligned}$$

□

**Exercise 1.15.8.** *Show that for any type $A$, we have $\neg\neg\neg A \to \neg A$.*

*Solution.* Recall that $\neg A :\equiv A \to \mathsf{0}$. So, we have to find a function

$$f\colon (((A \to \mathsf{0}) \to \mathsf{0}) \to \mathsf{0}) \to (A \to \mathsf{0}).$$

Fix $Z\colon \mathcal{U}$ and $a\colon A$ and consider the evaluation maps

$$\begin{aligned}
\mathrm{ev}_a\colon (A \to Z) &\to Z \\
x &\mapsto x(a).
\end{aligned}$$

and

$$\begin{aligned}
\mathrm{ev}_{\mathrm{ev}_a}\colon ((A \to Z) \to Z) &\to Z \\
y &\mapsto y(\mathrm{ev}_a).
\end{aligned}$$

Then define

$$\begin{aligned}
\mathrm{ev}_{\mathrm{ev}_{\mathrm{ev}_a}}\colon (((A \to Z) \to Z) \to Z) &\to A \to Z \\
(z, a) &\mapsto z(\mathrm{ev}_{\mathrm{ev}_a}).
\end{aligned}$$

To conclude the proof, take $Z :\equiv \mathsf{0}$.

*Second solution:* Consider the evaluation $\mathrm{ev}\colon A \to \neg\neg A$ defined by $a \mapsto \lambda\zeta\colon \neg A.\, \zeta(a)$. Now observe the functoriality of $\neg$, i.e., every map $X \to Y$ produces by composition another map $\neg Y \to \neg X$. □

**Exercise 1.15.9.** *Using the propositions as types interpretation, derive the following tautologies.*

a) *If $A$, then (if $B$ then $A$).*

b) *If A, then not (not A).*

c) *If (not A or not B), then not (A and B).*

*Solution.*

a) This translates into the type $A \to (B \to A)$. An obvious inhabitant is $a \mapsto (b \mapsto a)$.

b) This corresponds to $A \to ((A \to \mathsf{0}) \to \mathsf{0})$. A function of this type is a particular case of

$$\begin{aligned} \mathrm{ev}_a &: A \to ((A \to Z) \to Z) \\ a &\mapsto (g \mapsto g(a)). \end{aligned}$$

when $Z :\equiv \mathsf{0}$.

c) Here the interpretation is $\neg A + \neg B \to \neg(A \times B)$.

$$(A \to \mathsf{0}) + (B \to \mathsf{0}) \to (A \times B) \to \mathsf{0}.$$

Therefore, it suffices to define

$$\begin{aligned} g_0 &: (A \to \mathsf{0}) \to (A \times B \to \mathsf{0}) \\ g_1 &: (B \to \mathsf{0}) \to (A \times B \to \mathsf{0}) \end{aligned}$$

which can be done respectively as

$$\begin{aligned} x &\mapsto ((a, b) \mapsto x(a)) \\ y &\mapsto ((a, b) \mapsto y(b)) \end{aligned}$$

□

**Exercise 1.15.10.** *Using propositions-as-types, derive the double negation of the principle of excluded middle, i.e. prove* $\neg\neg(P \vee \neg P)$.

*Solution.* We have to define a function

$$\big(P + (P \to \mathsf{0}) \to \mathsf{0}\big) \to \mathsf{0}.$$

More generally, let us define a function

$$f : \big(P + (P \to Z) \to Z\big) \to Z.$$

Fix a function $g : P + (P \to Z) \to Z$. It defines two functions

$$\begin{aligned} g_0 &: P \to Z & \qquad g_1 &: (P \to Z) \to Z \\ g_0 &:\equiv \lambda p.\, g(\mathsf{inl}(p)) & \qquad g_1 &:\equiv \lambda h.\, g(\mathsf{inr}(h)). \end{aligned}$$

Then define

$$f :\equiv \lambda g.\, g_1(g_0).$$

□

**Exercise 1.15.11.** *Explain why the induction principles for identity types do not allow us to construct a function*

$$f\colon \prod_{x\colon A}\prod_{p\colon x=_A x}(p =_{x=_A x} \mathsf{refl}_x)$$

*with the defining equation*

$$f(x, \mathsf{refl}_x) :\equiv \mathsf{refl}_{\mathsf{refl}_x}.$$

*Solution.* The construction is invalid because the expression does not fit the type signature required by the induction principle for identity types in either formulation (1.1) or (1.3).

The given expression only considers paths of the form $p\colon x =_A x$, between $x$ and itself, while the induction principle requires paths between two variables. □

**Exercise 1.15.12.** *Show that addition of natural numbers is commutative.*

*Solution.* We must exhibit a witness for the type

$$\prod_{i,j\colon \mathbb{N}} i + j =_{\mathbb{N}} j + i.$$

We proceed by induction on $i$. Define the motive $C\colon \mathbb{N} \to \mathcal{U}$ by

$$C(i) :\equiv \prod_{j\colon \mathbb{N}} i + j =_{\mathbb{N}} j + i.$$

**Base case:** We must provide a term $c_0\colon \prod_{j\colon \mathbb{N}} 0 + j =_{\mathbb{N}} j + 0$. By definition of addition, $0 + j \equiv j$. Thus, the goal reduces to showing $j =_{\mathbb{N}} j + 0$. We have already constructed a witness for this in 67. Additive identity.

**Inductive step:** Let $i\colon \mathbb{N}$ and assume the inductive hypothesis

$$h\colon \prod_{j\colon \mathbb{N}} i + j =_{\mathbb{N}} j + i.$$

We must show that for any $j$, $\mathsf{succ}(i) + j =_{\mathbb{N}} j + \mathsf{succ}(i)$.

$$\begin{aligned}
\mathsf{succ}(i) + j &\equiv \mathsf{succ}(i + j) \\
&=_{\mathbb{N}} \mathsf{succ}(j + i) && ; h(j) \text{ and congruence} \\
&=_{\mathbb{N}} (j + i) + 1 && ; \mathsf{succ}(n) =_{\mathbb{N}} n + 1 \\
&=_{\mathbb{N}} j + (i + 1) && ; \text{associativity} \\
&=_{\mathbb{N}} j + \mathsf{succ}(i) && ; n + 1 =_{\mathbb{N}} \mathsf{succ}(n).
\end{aligned}$$

□

## 1.16 Additional Exercises

**Exercise 1.16.1.** *In* 68. Associativity of addition, *we relied on the principle of congruence (applying a function to both sides of an equality).*

a) *Define the application*[26] *function* $\mathsf{ap}_f$*, which takes a function* $f\colon A \to B$ *and a path* $p\colon x =_A y$ *and produces a path* $f(x) =_B f(y)$*.*

b) *Show that* $\mathsf{ap}_f$ *distributes over path concatenation, i.e.,*

$$\mathsf{ap}_f(p \cdot q) =_{f(x)=_B f(z)} \mathsf{ap}_f(p) \cdot \mathsf{ap}_f(q),$$

*for* $x, y, z\colon A$*,* $p\colon x =_A y$*, and* $q\colon y =_A z$*.*

*Solution.*

a) Fix the types $A$ and $B$, and a function $f\colon A \to B$. Consider the motive family

$$C_f\colon \prod_{x,y\colon A} (x =_A y) \to \mathcal{U}$$
$$C_f :\equiv \lambda x.\, \lambda y.\, \lambda p.\, f(x) =_B f(y).$$

To apply the induction principle, we need a base case function $c_f$ of type

$$\prod_{x\colon A} C_f(x, x, \mathsf{refl}_x).$$

Since $C_f(x, x, \mathsf{refl}_x) \equiv f(x) =_B f(x)$, we can define this function as

$$c_f :\equiv \lambda x.\, \mathsf{refl}_{f(x)}.$$

Then, applying the induction principle $\mathsf{ind}_{=_A}$ to $C_f$ and $c_f$, as specified in (1.1) yields a function

$$\mathsf{ind}_{=_A}(C_f, c_f)\colon \prod_{x,y\colon A} \prod_{p\colon x=_A y} f(x) =_B f(y),$$

which satisfies

$$\mathsf{ind}_{=_A}(C_f, c_f, x, x, \mathsf{refl}_x) \equiv c_f(x) \equiv \mathsf{refl}_{f(x)}.$$

Hence, we can introduce the *action of paths* function as

$$\mathsf{ap}_f :\equiv \mathsf{ind}_{=_A}(C_f, c_f).$$

The complete notation $\mathsf{ap}_f(x, y, p)$ is usually simplified to $\mathsf{ap}_f(p)$, or even to $\tilde{f}(p)$. In particular,

$$\tilde{f}(\mathsf{refl}_x) \equiv \mathsf{refl}_{f(x)}.$$

[26] A.k.a. *action on paths*.

b) Fix $f\colon A \to B$ and consider the motive

$$D_f\colon \prod_{x,y\colon A} (x =_A y) \to \mathcal{U}$$
$$D_f(x,y,p) :\equiv \prod_{z\colon A} \prod_{q\colon y=_A z} \tilde{f}(p \cdot q) =_{f(x)=_B f(z)} \tilde{f}(p) \cdot \tilde{f}(q).$$

Using part a) and the fact that $\mathsf{refl}$ is the definitionally left concatenation unit, we deduce

$$\begin{aligned} D_f(x,x,\mathsf{refl}_x) &\equiv \prod_{z\colon A} \prod_{q\colon x=_A z} \tilde{f}(\mathsf{refl}_x \cdot q) =_{f(x)=_B f(x)} \tilde{f}(\mathsf{refl}_x) \cdot \tilde{f}(q) \\ &\equiv \prod_{z\colon A} \prod_{q\colon x=_A z} \tilde{f}(q) =_{f(x)=_B f(x)} \tilde{f}(q). \end{aligned}$$

Consequently, we can take

$$d_f(x) :\equiv \lambda z.\, \lambda q.\, \mathsf{refl}_{\tilde{f}(q)},$$

and apply the induction principle to obtain

$$\mathsf{ind}_{=_A}(D_f, d_f, x, y, p)\colon D_f(x,y,p).$$

□

**Exercise 1.16.2.** *Consider the boolean type* $2$.

a) *Define the negation function* $\mathsf{not}\colon 2 \to 2$.

b) *Prove that negation is an involution, i.e., construct a witness for*

$$\prod_{b\colon 2} \mathsf{not}(\mathsf{not}(b)) =_2 b.$$

*Solution.*

a) It is enough to use the recursion principle (1.1) for the motive $C :\equiv 2$. For the constants $c_0, c_1\colon 2$, we take

$$c_0 :\equiv 1_2 \quad \textit{and} \quad c_1 :\equiv 0_2.$$

Then we can define

$$\mathsf{not}\colon 2 \to 2, \quad \mathsf{not} :\equiv \lambda b.\, \mathsf{rec}_2(2, c_0, c_1, b),$$

which satisfies $\mathsf{not}(0_2) \equiv 1_2$ and $\mathsf{not}(1_2) \equiv 0_2$.

b) Consider the motive $C\colon 2 \to \mathcal{U}$ given by

$$C :\equiv \prod_{b\colon 2} \mathsf{not}(\mathsf{not}(b)) =_2 b.$$

Using the induction principle for 2, we need to specify two paths $c_0\colon C(0_2)$ and $c_1\colon C(1_2)$, which we choose as $\mathsf{refl}_{0_2}$ and $\mathsf{refl}_{1_2}$. These are well defined because

$$\mathsf{not}(\mathsf{not}(0_2)) \equiv \mathsf{not}(1_2) \equiv 0_2,$$

and similarly for $1_2$. As a result, we can define

$$\lambda b.\, \mathsf{ind}_2(C, c_0, c_1, b).$$

□

**Exercise 1.16.3.** *Prove that transport over a constant type family acts as the identity function. That is, given a type $B$ and the constant family $P(x) :\equiv B$, construct a witness for*

$$\prod_{x,y\colon A} \prod_{p\colon x =_A y} \prod_{b\colon B} \mathsf{transport}^P(p, b) =_B b.$$

*Solution.* The motive $C$ used in 64. TRANSPORT to define the transport function $\mathsf{transport}^P$ is given by

$$C(x, y, p) :\equiv P(x) \to P(y),$$

which, in this case, reduces to

$$C(x, y, p) :\equiv B \to B,$$

while the base case $c(x)\colon C(x, x, \mathsf{refl}_x)$ is the identity function $c(x) :\equiv \lambda b\colon B.\, b$. Let us also recall that $\mathsf{transport}^P(p) \equiv \mathsf{ind}_{=_A}(C, c, x, y, p)$.

To construct the required witness, consider the motive

$$D(x, y, p) :\equiv \prod_{b\colon B} \mathsf{transport}^P(p, b) =_B b.$$

Because the expression is well-typed we can proceed by path induction on $p$. This reduces the goal to the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. For this base case, we need a dependent function $d$ of type $\prod_{x\colon A} D(x, x, \mathsf{refl}_x)$. Since

$$\mathsf{transport}^P(\mathsf{refl}_x) \equiv \mathsf{id}_B,$$

and $\mathsf{id}_B(b) \equiv b$, evaluating the motive at the base yields

$$D(x, x, \mathsf{refl}_x) \equiv \prod_{b\colon B} b =_B b.$$

Thus, we may define $d(x) :\equiv \lambda b.\, \mathsf{refl}_b$. Path induction then provides the desired function

$$\mathsf{ind}_{=_A}(D, d) : \prod_{x,y:\, A} \prod_{p:\, x =_A y} D(x, y, p).$$

□

**Exercise 1.16.4.** *Let $f : A \to B$ be a function between types $A$ and $B$. Show that the action on paths $\mathsf{ap}_f$ can be defined strictly in terms of the transport function* transport*. Specifically, given $x, y : A$ and a path $p : x =_A y$, construct a term of type $f(x) =_B f(y)$ using only* transport *and* refl*.*

*Solution.* Fix $x : A$ and consider the family $P : A \to \mathcal{U}$ defined by

$$P(y) :\equiv f(x) =_B f(y).$$

By transport, we know that there is a function

$$\mathsf{transport}^P : \prod_{y:\, A} \prod_{p:\, x =_A y} P(x) \to P(y).$$

Since $\mathsf{refl}_{f(x)} : P(x)$, given a path $e : x =_A y$, we can apply the transport function to obtain

$$\mathsf{transport}^P(y, e)(\mathsf{refl}_{f(x)}) : P(y).$$

By definition of $P$, this term is an inhabitant of $f(x) =_B f(y)$, as required. □

# Chapter 2

# Homotopy Type Theory

## 2.1 Functoriality and Path Algebra

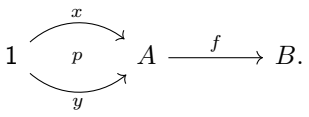
Recall from Exercise 1.16.1 that given $f\colon A \to B$, the congruence function

$$\mathsf{ap}_f\colon \prod_{x,y\colon A} \prod_{p\colon x =_A y} f(x) =_B f(y)$$

transforms paths in $A$ to paths in $B$. More precisely

$$\mathsf{ap}_f(x,y,p)\colon f(x) =_B f(y), \quad \text{for } p\colon x =_A y,$$

where the notation $\mathsf{ap}_f(x,y,p)$ is usually simplified to $\mathsf{ap}_f(p)$ or $\tilde{f}(p)$. The following diagram illustrates the setting




**Proposition 2.1.1.** *Let $A, B, C\colon \mathcal{U}$. Given functions $f\colon A \to B$ and $g\colon B \to C$ and paths $p\colon x =_A y$ and $q\colon y =_A z$, we have:*

a) $\mathsf{ap}_f(\mathsf{refl}_x) \equiv \mathsf{refl}_{f(x)}$.

b) $\mathsf{ap}_f(p \cdot q) =_{f(x)=_B f(z)} \mathsf{ap}_f(p) \cdot \mathsf{ap}_f(q)$.

c) $\mathsf{ap}_f(p^{-1}) =_{f(y)=_B f(x)} \mathsf{ap}_f(p)^{-1}$.

d) $\mathsf{ap}_g(\mathsf{ap}_f(p)) =_{g\circ f(x)=_C g\circ f(y)} \mathsf{ap}_{g\circ f}(p)$.

e) $\mathsf{ap}_{\mathsf{id}_A}(p) =_{x=_A y} p$.

*Proof.*

a) Exercise 1.16.1.

b) Exercise 1.16.1.

c) This easily follows by path induction on $p$. However, here we will illustrate how to deduce it from what we have proven so far.

First observe that

$$\begin{aligned}
\mathsf{ap}_f(p) \cdot \mathsf{ap}_f(p^{-1}) &=_{f(x)=_B f(x)} \mathsf{ap}_f(p \cdot p^{-1}) && \text{; by part b)} \\
&=_{f(x)=_B f(x)} \mathsf{ap}_f(\mathsf{refl}_x) && \text{; by } \mathsf{ap}_{\mathsf{ap}_f}\text{(inverse law)} \\
&\equiv \mathsf{refl}_{f(x)} && \text{; by part a).}
\end{aligned}$$

The second equality applies $\mathsf{ap}_{\mathsf{ap}_f}$ to the witness of the inverse law, i.e., to the path $p \cdot p^{-1} =_{x=_A x} \mathsf{refl}_x$.

Now consider the function

$$\begin{aligned}
g\colon (f(x) =_B f(x)) &\to (f(y) =_B f(x)) \\
g(r) &:\equiv \mathsf{ap}_f(p)^{-1} \cdot r.
\end{aligned}$$

Let $\gamma$ be the witness of the equality we just proved. Applying $\mathsf{ap}_g$ to $\gamma^{-1}$ yields

$$\begin{aligned}
\mathsf{ap}_f(p)^{-1} &=_{f(y)=_B f(x)} \mathsf{ap}_f(p)^{-1} \cdot \big(\mathsf{ap}_f(p) \cdot \mathsf{ap}_f(p^{-1})\big) && \text{; } \mathsf{ap}_g(\gamma^{-1}) \\
&=_{f(y)=_B f(x)} \big(\mathsf{ap}_f(p)^{-1} \cdot \mathsf{ap}_f(p)\big) \cdot \mathsf{ap}_f(p^{-1}) && \text{; associativity} \\
&=_{f(y)=_B f(x)} \mathsf{ap}_f(p^{-1}) && \text{; left inverse.}
\end{aligned}$$

where the last equality can be proven by mirroring the deduction of the first one.

d) Since $p\colon x =_A y$, we know that $\mathsf{ap}_f(p)\colon f(x) =_B f(y)$. Therefore,

$$\mathsf{ap}_g(\mathsf{ap}_f(p))\colon g(f(x)) =_C g(f(y)).$$

The type of the term above is definitionally equal to $g \circ f(x) =_C g \circ f(y)$. Consider the case where $p \equiv \mathsf{refl}_x$. We have

$$\begin{aligned}
\mathsf{ap}_g(\mathsf{ap}_f(\mathsf{refl}_x)) &\equiv \mathsf{ap}_g(\mathsf{refl}_{f(x)}) \\
&\equiv \mathsf{refl}_{g(f(x))} \\
&\equiv \mathsf{refl}_{g \circ f(x)} \\
&\equiv \mathsf{ap}_{g \circ f}(\mathsf{refl}_x).
\end{aligned}$$

To conclude that $\mathsf{ap}_g(\mathsf{ap}_f(p)) =_{g \circ f(x) =_C g \circ f(y)} \mathsf{ap}_{g \circ f}(p)$, we apply path induction to the motive

$$C(x, y, p) :\equiv \mathsf{ap}_g(\mathsf{ap}_f(p)) =_{g \circ f(x) =_C g \circ f(y)} \mathsf{ap}_{g \circ f}(p),$$

with the base case $c(x)\colon C(x, x, \mathsf{refl}_x)$ defined as $\mathsf{refl}_{\mathsf{ap}_{g \circ f}(\mathsf{refl}_x)}$.

e) For $p \equiv \mathsf{refl}_x$, since $\mathsf{id}_A(x) \equiv x$, we have

$$\begin{aligned}\mathsf{ap}_{\mathsf{id}_A}(\mathsf{refl}_x) &\equiv \mathsf{refl}_{\mathsf{id}_A(x)} \\ &\equiv \mathsf{refl}_x.\end{aligned}$$

Therefore, the conclusion follows by path induction applied to the motive

$$C(x, y, p) :\equiv \mathsf{ap}_{\mathsf{id}_A}(p) =_{x=_A y} p$$

with base case $c(x) :\equiv \mathsf{refl}_{\mathsf{refl}_x} : C(x, x, \mathsf{refl}_x)$.

□

**Lemma 2.1.2.** *Let $A, B : \mathcal{U}$ and $b : B$. Given the constant function $f : A \to B$ defined by $f(x) :\equiv b$, for any path $p : x =_A y$, we have*

$$\mathsf{ap}_f(p) =_{b=_B b} \mathsf{refl}_b.$$

*Proof.* By path induction on $p$ it suffices to prove the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$.

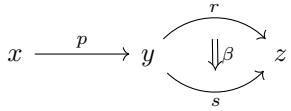
- LHS: $\mathsf{ap}_f(\mathsf{refl}_x) \equiv \mathsf{refl}_{f(x)} \equiv \mathsf{refl}_b$.
- RHS: $\mathsf{refl}_b$.

Since both sides are definitionally equal to $\mathsf{refl}_b$, the equality is inhabited by $\mathsf{refl}_{\mathsf{refl}_b}$. □

### 2.1.1 Path Whiskering

**Definition 2.1.3.** Let $A$ be a type and $x, y, z : A$ points.

a) Given a 1-path $p : x =_A y$ and a 2-path $\beta : r =_{y=_A z} s$ between 1-paths $r, s : y =_A z$,

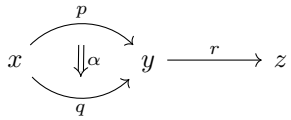



we define the *left whiskering*

$$p \triangleleft \beta : (p \cdot r) =_{x=_A z} (p \cdot s)$$

by path induction on $\beta$: if $r \equiv s$ and $\beta \equiv \mathsf{refl}_r$, then $p \triangleleft \mathsf{refl}_r :\equiv \mathsf{refl}_{p \cdot r}$.

b) Given a 2-path $\alpha : p =_{x=_A y} q$ between paths $p, q : x =_A y$ and a path $r : y =_A z$,



we define the *right whiskering*

$$\alpha \triangleright r : (p \cdot r) =_{x =_A z} (q \cdot r)$$

by induction on $\alpha$: if $q \equiv p$ and $\alpha \equiv \mathsf{refl}_p$, then $\mathsf{refl}_p \triangleright r :\equiv \mathsf{refl}_{p \cdot r}$.

**Remark 2.1.4.** By definition,

$$\begin{aligned} \alpha \triangleright r &\equiv \mathsf{ap}_{\mathsf{rct}_r}(\alpha) \\ p \triangleleft \beta &\equiv \mathsf{ap}_{\mathsf{lct}_p}(\beta), \end{aligned}$$

where $\mathsf{rct}_r :\equiv x \mapsto x \cdot r$ and $\mathsf{lct}_p :\equiv x \mapsto p \cdot x$.

**Proposition 2.1.5.** *Given a* 2*-path* $\alpha : p = q$ *and appropriately composable* 1*-paths* $r$ *and* $s$*, we have the following homotopies witnessing the associativity of whiskering:*

a) $(\alpha \triangleright r) \triangleright s = \alpha \triangleright (r \cdot s)$.

b) $p \triangleleft (q \triangleleft \alpha) = (p \cdot q) \triangleleft \alpha$.

c) $p \triangleleft (\alpha \triangleright r) = (p \triangleleft \alpha) \triangleright r$.

*Proof.* For a) observe that

$$\begin{aligned} (\alpha \triangleright r) \triangleright s &\equiv \mathsf{ap}_{\mathsf{rct}_r}(\mathsf{ap}_{\mathsf{rct}_r}(\alpha)) \\ &= \mathsf{ap}_{\mathsf{rct}_s \circ \mathsf{rct}_r}(\alpha) \\ &\equiv \mathsf{ap}_{\mathsf{rct}_{r \cdot s}}(\alpha) \\ &\equiv \alpha \triangleright (r \cdot s). \end{aligned}$$

The proof of b) is similar.

For c) compare left and right actions:

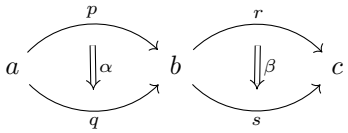
$$\begin{aligned} \text{LHS} &\equiv \mathsf{ap}_{\mathsf{lct}_p}(\mathsf{ap}_{\mathsf{rct}_r}(\alpha)) \\ &= \mathsf{ap}_{\mathsf{lct}_p \circ \mathsf{rct}_r}(\alpha). \\ \text{RHS} &\equiv \mathsf{ap}_{\mathsf{rct}_r}(\mathsf{ap}_{\mathsf{lct}_p}(\alpha)) \\ &= \mathsf{ap}_{\mathsf{rct}_r \circ \mathsf{lct}_r p}(\alpha). \end{aligned}$$

The conclusion follows because

$$\mathsf{rct}_r \circ \mathsf{lct}_p = \mathsf{lct}_p \circ \mathsf{rct}_r$$

since concatenation of 1-paths is propositionally associative. □

**Definition 2.1.6.** Let $A$ be a type and $a, b, c\colon A$. Consider paths $p, q\colon a =_A b$ and $r, s\colon b =_A c$, along with homotopies $\alpha\colon p =_{a=_A b} q$ and $\beta\colon r =_{b=_A c} s$, as depicted below:




The *horizontal composition* of $\alpha$ and $\beta$ is the 2-path

$$\alpha \bullet \beta\colon (p \cdot r) =_{a=_A c} (q \cdot s)$$

defined by path induction on $\alpha$ and $\beta$. Explicitly, if $q \equiv p$ and $\alpha \equiv \mathsf{refl}_p$, we define

$$\mathsf{refl}_p \bullet \beta\colon (p \cdot r) =_{a=_A c} (p \cdot s)$$

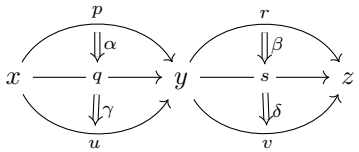
by induction on $\beta$ as follows: if $s \equiv r$ and $\beta \equiv \mathsf{refl}_r$, then

$$\mathsf{refl}_p \bullet \mathsf{refl}_r :\equiv \mathsf{refl}_{p \cdot r}. \tag{2.1}$$

**Definition 2.1.7.** Given 2-paths $\alpha\colon p =_{x=_A y} q$ and $\gamma\colon q =_{x=_A y} r$, their *vertical composition* is their concatenation as paths in the identity type $x =_A y$:

$$\alpha \cdot \gamma\colon p =_{x=_A y} r.$$

**Lemma 2.1.8.** [Interchange Law] *Let $A$ be a type. Consider the configuration of paths and homotopies depicted in the following commutative diagram:*




*The horizontal and vertical compositions satisfy:*

$$(\alpha \bullet \beta) \cdot (\gamma \bullet \delta) = (\alpha \cdot \gamma) \bullet (\beta \cdot \delta).$$

*Proof.* Let us first observe that the types match. We have:

$$\alpha \bullet \beta\colon (p \cdot r) =_{x=_A z} (q \cdot s)$$
$$\gamma \bullet \delta\colon (q \cdot s) =_{x=_A z} (u \cdot v).$$

Thus, the LHS is a path of type $(p \cdot r) =_{x=_A z} (u \cdot v)$. Similarly, for the right-hand side components:

$$\alpha \cdot \gamma\colon p =_{x=_A y} u$$
$$\beta \cdot \delta\colon r =_{y=_A z} v.$$

Thus, the RHS is also a path of type $(p \cdot r) =_{x=_A z} (u \cdot v)$.

Since the types agree, we proceed by path induction.

STEP 1. Induction on $\alpha$. We assume $\alpha$ is $\mathsf{refl}_p$, which definitionally forces the endpoint $q$ to be $p$. Consequently, the type of $\gamma$ becomes $p =_{x=_A y} u$.

STEP 2. Induction on $\gamma$. We assume $\gamma$ is $\mathsf{refl}_p$, which forces $u$ to be $p$.

STEP 3. Induction on $\beta$, and then on $\delta$. Applying the same logic to the right column, we assume $\beta$ and $\delta$ are $\mathsf{refl}_r$, which forces $s \equiv r$ and $v \equiv r$.

STEP 4. Computation. In this base case, all paths are definitionally reflexivity. We compute both sides:

$$\begin{aligned}
\text{LHS} &\equiv (\mathsf{refl}_p \bullet \mathsf{refl}_r) \cdot (\mathsf{refl}_p \bullet \mathsf{refl}_r) \\
&\equiv \mathsf{refl}_{p\cdot r} \cdot \mathsf{refl}_{p\cdot r} \\
&\equiv \mathsf{refl}_{p\cdot r}. && ; \ (2.1) \\
\text{RHS} &\equiv (\mathsf{refl}_p \cdot \mathsf{refl}_p) \bullet (\mathsf{refl}_r \cdot \mathsf{refl}_r) \\
&\equiv \mathsf{refl}_p \bullet \mathsf{refl}_r \\
&\equiv \mathsf{refl}_{p\cdot r}. && ; \ (2.1)
\end{aligned}$$

Since both sides reduce to the same term $\mathsf{refl}_{p\cdot r}$ in the base case, the path induction principle ensures they are equal in the general case. □

## 2.2 Pointed Types and Loop Spaces

**Definition 2.2.1.** A *pointed type* $(A, a)$ is a type $A : \mathcal{U}$ together with a point $a : A$, called its *basepoint*. We write

$$\mathcal{U}_\bullet :\equiv \sum_{A:\mathcal{U}} A$$

for the type of pointed types in the universe $\mathcal{U}$.

**Definition 2.2.2.** Given a pointed type $(A, a)$, we define the *loop space* of $(A, a)$ as the following pointed type:

$$\begin{aligned}
\Omega &: \mathcal{U}_\bullet \to \mathcal{U}_\bullet \\
\Omega(\langle A, a\rangle) &:\equiv \langle a =_A a, \mathsf{refl}_a\rangle.
\end{aligned}$$

An element of the underlying type $a =_A a$ is called a *loop* at $a$.

For $n : \mathbb{N}$, we define the *loop space operation* $\Omega^n : \mathcal{U}_\bullet \to \mathcal{U}_\bullet$ by recursion as follows:

$$\begin{aligned}
C &:\equiv \mathcal{U}_\bullet \to \mathcal{U}_\bullet \\
c_0 &:\equiv \mathsf{id}_{\mathcal{U}_\bullet} \\
c_s &: \mathbb{N} \to C \to C \\
c_s(n, f) &:\equiv f \circ \Omega \\
\Omega^n &:\equiv \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, n).
\end{aligned}$$

Then, using the definitional equation

$$\mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, \mathsf{succ}(n)) \equiv c_s(n, \mathsf{rec}_{\mathbb{N}}(C, c_0, c_s, n)),$$

we obtain

$$\Omega^{n+1}(\langle A, a\rangle) \equiv \Omega^n(\Omega(\langle A, a\rangle)).$$

Since $\Omega^0$ is the identity, $\Omega^1 \equiv \Omega$.

**Lemma 2.2.3.** *For any external natural number $n \in \mathbb{N}$ and any pointed type $\langle A, a\rangle$, we have the judgmental equality*

$$\Omega^n(\Omega(\langle A, a\rangle)) \equiv \Omega(\Omega^n(\langle A, a\rangle)). \tag{2.1}$$

*Proof.* Since a judgmental equality cannot be expressed as a type family, we proceed by metatheoretic induction on $n$.

For the base case $n = 0$, the equality holds trivially because $\Omega^0 \equiv \mathsf{id}_{\mathcal{U}_\bullet}$.

For the inductive step, assume the judgmental equality holds for $n$. Applying the inductive hypothesis to the pointed type $\Omega(\langle A, a\rangle)$, we have

$$\begin{aligned}\Omega^{n+1}(\Omega(\langle A, a\rangle)) &\equiv \Omega^n(\Omega(\Omega(\langle A, a\rangle))) \\ &\equiv \Omega(\Omega^n(\Omega(\langle A, a\rangle))) \\ &\equiv \Omega(\Omega^{n+1}(\langle A, a\rangle)).\end{aligned}$$

This completes the metatheoretic induction, establishing the judgmental equality for any external natural number $n$.

□

**Remark 2.2.4.** Let $c_n :\equiv \pi_2(\Omega^n(\langle A, a\rangle))$ be the *trivial $n$-loop*. Applying the projection $\pi_2$ to both sides of (2.1), we obtain

$$c_{n+1} \equiv \mathsf{refl}_{c_n} : c_n =_{\pi_1(\Omega^n(\langle A,a\rangle))} c_n,$$

with $c_0 \equiv a$. In particular $(c_n =_{\pi_1(\Omega^n(\langle A,a\rangle))} c_n) \equiv \pi_1(\Omega^{n+1}(\langle A, a\rangle))$.

**Notation 2.2.5.** *By abuse of notation, we will sometimes identify $\Omega(\langle A, a\rangle)$ with $\pi_1(\Omega(\langle A, a\rangle)) \equiv a =_A a$. The following definition is an example.*

**Definition 2.2.6.** Given a function $f : A \to B$ and a point $a : A$, the *induced map on loops* is the function

$$\Omega f : \Omega(\langle A, a\rangle) \to \Omega(\langle B, f(a)\rangle)$$

defined by

$$\Omega f(p) :\equiv \mathsf{ap}_f(p).$$

Note that by part a) of Proposition 2.1.1, we have $\Omega f(\mathsf{refl}_a) \equiv \mathsf{refl}_{f(a)}$, meaning that $\Omega f$ preserves the basepoint definitionally.

**Remark 2.2.7.** Since $\Omega^2(\langle A, a\rangle) \equiv \Omega(\langle a =_A a, \mathsf{refl}_a\rangle)$, it follows that

$$\Omega^2(\langle A, a\rangle) \equiv \langle \mathsf{refl}_a =_{a=_A a} \mathsf{refl}_a, \mathsf{refl}_{\mathsf{refl}_a}\rangle.$$

**Remark 2.2.8.** Sometimes the notation is abused and $\Omega(\langle A, a\rangle)$ is identified with $a =_A a$, i.e., as $\pi_1(\langle a =_A a, \mathsf{refl}_a\rangle)$.

Note however that the reason to define $\Omega\colon \mathcal{U}_\bullet \to \mathcal{U}_\bullet$ and not from $\mathcal{U}_\bullet$ to $\mathcal{U}$, is to enable the definition of $\Omega^n$ for $n\colon \mathbb{N}$.

In particular, it is customary to refer to *a point in the loop space* $\Omega(\langle A, a\rangle)$ as a path $p\colon a =_A a$.

Similarly, *a point* in the second loop space $\Omega^2(\langle A, a\rangle)$ refers to a homotopy

$$\alpha\colon \mathsf{refl}_a =_{a=_A a} \mathsf{refl}_a$$

between the trivial loop and itself.

By induction, *a point* in $\Omega^n(\langle A, a\rangle)$ is an $n$-dimensional path from the trivial $(n-1)$-dimensional path to itself.

**Theorem 2.2.9.** [Eckmann-Hilton] *The composition operation on the second loop space*

$$\Omega^2(A) \times \Omega^2(A) \to \Omega^2(A)$$

*is commutative:* $\alpha \cdot \beta =_{\Omega^2(A)} \beta \cdot \alpha$*, for any* $\alpha, \beta\colon \Omega^2(A)$.

*Proof.* Introduce the 2-loop

$$e :\equiv \mathsf{refl}_{\mathsf{refl}_a} : \mathsf{refl}_a =_{\Omega(A)} \mathsf{refl}_a,$$

which is the identity element of the second loop space $\Omega^2(A)$.

HORIZONTAL = VERTICAL: Consider the expression

$$(\alpha \bullet e) \cdot (e \bullet \beta).$$

Using the unit laws for horizontal composition (which hold propositionally by path induction), we have $\alpha \bullet e =_{a=_A a} \alpha$ and $e \bullet \beta =_{a=_A a} \beta$. Thus, we have a path:

$$(\alpha \bullet e) \cdot (e \bullet \beta) =_{a=_A a} \alpha \cdot \beta. \tag{2.2}$$

On the other hand, applying the Interchange Law Lemma 2.1.8 to the original expression yields:

$$(\alpha \bullet e) \cdot (e \bullet \beta) =_{a=_A a} (\alpha \cdot e) \bullet (e \cdot \beta).$$

Using the unit laws for vertical composition, namely

$$\alpha \cdot e =_{a=_A a} \alpha \quad \text{and} \quad e \cdot \beta =_{a=_A a} \beta,$$

the right-hand side becomes $\alpha \bullet \beta$. Comparing this result with (2.2), we conclude:

$$\alpha \bullet \beta =_{a=_A a} \alpha \cdot \beta. \tag{2.3}$$

COMMUTATIVITY: Now consider the expression with terms swapped:

$$(e \bullet \beta) \cdot (\alpha \bullet e).$$

Using the horizontal unit laws first:

$$(e \bullet \beta) \cdot (\alpha \bullet e) =_{a=_A a} \beta \cdot \alpha.$$

Alternatively, applying the Interchange Law first:

$$(e \bullet \beta) \cdot (\alpha \bullet e) =_{a=_A a} (e \cdot \alpha) \bullet (\beta \cdot e).$$

Using the vertical unit laws on the right side, this reduces to $\alpha \bullet \beta$.

Combining these chains of equalities with (2.3), we obtain:

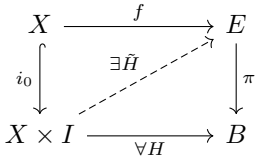
$$\beta \cdot \alpha =_{a=_A a} \alpha \bullet \beta =_{a=_A a} \alpha \cdot \beta,$$

as desired.

□

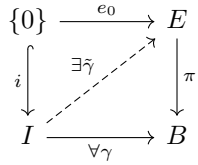
## 2.3 Fibrations and Transport

Recall from classical topology that a *fibration* is a continuous map $\pi\colon E \to B$ with the homotopy-lifting property:




In this context, $E$ is the *total space*[1] and $B$ the *base space* of the fibration.

*Serre fibrations* lift paths, path homotopies, and higher-dimensional volumes. For instance, in the case of paths, the diagram above becomes:




[1] After the French *Espace*.

In Homotopy Type Theory, given a type family $P\colon A \to \mathcal{U}$, the first projection $\pi_1\colon \sum_{x:A} P(x) \to A$ is analogous to a fibration with *total space* $\sum_{x:A} P(x)$ and *base space* $A$. This structural analogy is established by the theorem below.

Given $a, b\colon A$, the type family $P$ structurally relates the fibers $P(a)$ and $P(b)$. Although a point $u\colon P(a)$ cannot be directly compared with $v\colon P(b)$ due to type mismatch, it is possible to establish a connection between the pairs $\langle a, u\rangle$ and $\langle b, v\rangle$ within the $\Sigma$-type. Specifically, if there exists a path $p\colon a =_A b$, we can transport $u$ along $p$ to obtain an element $\mathsf{transport}^P(p, u)\colon P(b)$. Since this transported term inhabits the same fiber as $v$, the comparison becomes well-defined. More precisely:

**Theorem 2.3.1.** [Path Lifting Property] *Let $P\colon A \to \mathcal{U}$ be a type family over $A$ and let $E :\equiv \sum_{x:A} P(x)$ be its total space. Given $a, b\colon A$, there exists a function*

$$\mathsf{lift}\colon \prod_{u:P(a)} \prod_{p:a=_A b} \langle a, u\rangle =_E \langle b, \mathsf{transport}^P(p, u)\rangle$$

*satisfying the following properties:*

i) $\mathsf{lift}(u, p)\colon \langle a, u\rangle =_E \langle b, \mathsf{transport}^P(p, u)\rangle$.

ii) $\mathsf{lift}(u, \mathsf{refl}_a) \equiv \mathsf{refl}_{\langle a,u\rangle}$.

iii) $\mathsf{ap}_{\pi_1}(\mathsf{lift}(u, p)) =_{a=_A b} p$.

*where $\pi_1\colon E \to A$ denotes the first projection.*[2]

*Proof.* Consider the motive family $C\colon \prod_{x:A}(a =_A x) \to \mathcal{U}$, defined by

$$C(x, p) :\equiv \langle a, u\rangle =_E \langle x, \mathsf{transport}^P(p, u)\rangle.$$

Since $\mathsf{transport}^P(\mathsf{refl}_a, u) \equiv u$, we have

$$C(a, \mathsf{refl}_a) \equiv \langle a, u\rangle =_E \langle a, u\rangle,$$

and we can specify the base case $c :\equiv \mathsf{refl}_{\langle a,u\rangle}$ to define, by 65. Based path induction,

$$\mathsf{lift}(u, p) :\equiv \mathsf{ind}'_{=_A}(a, C, c, x, p).$$

We can use based path induction again to verify that

$$\mathsf{ap}_{\pi_1}(\mathsf{lift}(u, p)) =_{a=_A x} p.$$

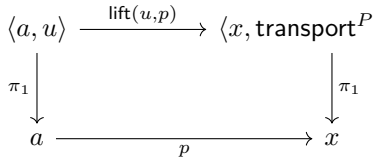
For the base case, we know definitionally that $\mathsf{lift}(u, \mathsf{refl}_a) \equiv \mathsf{refl}_{\langle a,u\rangle}$. Thus,

$$\mathsf{ap}_{\pi_1}(\mathsf{lift}(u, \mathsf{refl}_a)) \equiv \mathsf{ap}_{\pi_1}(\mathsf{refl}_{(a,u)}) \equiv \mathsf{refl}_{\pi_1(\langle a,u\rangle)} \equiv \mathsf{refl}_a.$$

□

[2] Since $\pi_1$ maps the total space to the base type $A$, the functorial action $\mathsf{ap}_{\pi_1}$ projects paths in $E$ to paths in $A$.

**Remark 2.3.2.** The lemma can be represented by the following diagram, where the horizontal arrows represent paths —i.e., terms of the propositional equalities between their ends— and the vertical arrows, functions:




$$;\ u\colon P(a),\ p\colon a =_A x.$$

Note also that a term $f\colon \prod_{x\colon A} P(x)$ can be regarded as a *section* of the fibration induced by $P$.

In particular, given that $\mathsf{transport}^P(\mathsf{refl}_a, u) \equiv u$, we deduce that yielding a witness of $\langle a, u\rangle =_E \langle a, v\rangle$ is exactly the same as yielding a witness of

$$u =_{P(a)} v. \tag{2.1}$$

**Notation 2.3.3.** *When $P$ is clear from the context we will simply write*

$$p_*(u) :\equiv \mathsf{transport}^P(p, u)\colon P(x).$$

*In particular,*

$$(\mathsf{refl}_a)_*(u) \equiv u. \tag{2.2}$$

**Definition 2.3.4.** Let $A, B\colon \mathcal{U}$. Given $b\colon B$ we define the *section* function

$$\mathsf{sec}_b\colon A \to A \times B, \quad a \mapsto (a, b).$$

Note that

$$\pi_1 \circ \mathsf{sec}_b \equiv \pi_1 \circ (\lambda x.\,(x, b)) \equiv \lambda x.\,\pi_1((x, b)) \equiv \lambda x.\,x \equiv \mathsf{id}_A.$$
$$\pi_2 \circ \mathsf{sec}_b \equiv \pi_2 \circ (\lambda x.\,(x, b)) \equiv \lambda x.\,\pi_2((x, b)) \equiv \lambda x.\,b.$$

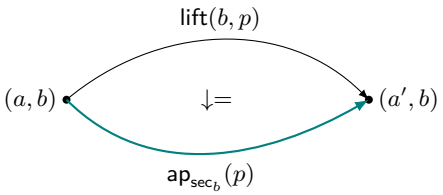
**Remark 2.3.5.** In the particular case where $P$ is a constant family $P(x) \equiv B$ for $x\colon A$, given $p\colon a =_A a'$, we have

i) $\mathsf{lift}(b, p) =_{(a,b) =_{A\times B} (a',b)} \mathsf{ap}_{\mathsf{sec}_b}(p)$,

ii) $\mathsf{lift}(b, \mathsf{refl}_a) \equiv \mathsf{refl}_{(a,b)}$,

iii) $\mathsf{ap}_{\pi_1}(\mathsf{lift}(b, p)) =_{a =_A a'} p$.

Part ii) is property ii) of Theorem 2.3.1. The equality of part i) follows from part ii) by induction on $p$.



For iii) observe that

$$\mathsf{ap}_{\pi_1}(\mathsf{lift}(b,p)) = \mathsf{ap}_{\pi_1}(\mathsf{ap}_{\mathsf{sec}_b}(p)) = \mathsf{ap}_{\pi_1 \circ \mathsf{sec}_b}(p) \equiv \mathsf{ap}_{\mathsf{id}_A}(p) = p.$$

**Lemma 2.3.6.** [Dependent map] *Let $P\colon A \to \mathcal{U}$. If $f\colon \prod_{x\colon A} P(x)$, then there is a map*

$$\mathsf{apd}_f\colon \prod_{x,y\colon A} \prod_{p\colon x =_A y} p_*(f(x)) =_{P(y)} f(y)$$

*satisfying* $\mathsf{apd}_f(\mathsf{refl}_x) \equiv \mathsf{refl}_{f(x)}$.

*Proof.* Fix $a\colon A$ and consider the motive

$$C\colon \prod_{a\colon A} (a =_A x) \to \mathcal{U}$$
$$C(x,p) :\equiv p_*(f(a)) =_{P(x)} f(x).$$

Since

$$\begin{aligned} C(a, \mathsf{refl}_a) &\equiv (\mathsf{refl}_a)_*(f(a)) =_{P(a)} f(a) \\ &\equiv f(a) =_{P(a)} f(a) && ;\ (2.2), \end{aligned}$$

we can specify $c :\equiv \mathsf{refl}_{f(a)}$ and obtain $\mathsf{apd}_f$ by based path induction. □

**Lemma 2.3.7.** *If $P\colon A \to \mathcal{U}$ is defined by $P(x) :\equiv B$ for a fixed $B\colon \mathcal{U}$, then for any $x, y\colon A$ and $p\colon x =_A y$ and $b\colon B$ we have a path*

$$\mathsf{transportconst}^B_p(b)\colon \mathsf{transport}^P(p,b) =_B b$$

*satisfying* $\mathsf{transportconst}^B_{\mathsf{refl}_x} :\equiv \lambda b.\, \mathsf{refl}_b$.

*Proof.* See Exercise 1.16.3.

□

**Lemma 2.3.8.** *Let $f\colon A \to B$ and define the constant family $P\colon A \to \mathcal{U}$ by $P(x) :\equiv B$. For any path $p\colon x =_A y$, we have*

$$\mathsf{apd}_f(p) =_{p_*(f(x)) =_B f(y)} \mathsf{transportconst}^B_p(f(x)) \cdot \mathsf{ap}_f(p).$$

*Proof.* First note that $C(x,p)$ is well defined because

$$\begin{aligned} \mathsf{apd}_f(p)&\colon p_*(f(x)) =_{P(y)} f(y) && ;\ \text{Lem. 2.3.6.} \\ \mathsf{transportconst}^B_p(f(x))&\colon p_*(f(x)) =_B f(x) && ;\ \text{Lem. 2.3.7.} \\ \mathsf{ap}_f(p)&\colon f(x) =_B f(y). \end{aligned}$$

Fix $a\colon A$ and let $C\colon \prod_{x\colon A}(a =_A x) \to \mathcal{U}$ be defined by

$$C(x,p) :\equiv \mathsf{apd}_f(p) =_{p_*(f(a))=_B f(x)} \mathsf{transportconst}_p^B(f(a)) \cdot \mathsf{ap}_f(p).$$

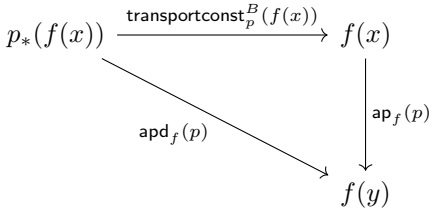
To verify the base case:

$$\begin{aligned} C(a,\mathsf{refl}_a) &\equiv \mathsf{apd}_f(\mathsf{refl}_a) =_{(\mathsf{refl}_a)_*(f(a))=_B f(a)} \mathsf{transportconst}^B_{\mathsf{refl}_a}(f(a)) \cdot \mathsf{ap}_f(\mathsf{refl}_a) \\ &\equiv \mathsf{refl}_{f(a)} =_{f(a)=_B f(a)} \mathsf{refl}_{f(a)} \cdot \mathsf{refl}_{f(a)} \\ &\equiv \mathsf{refl}_{f(a)} =_{f(a)=_B f(a)} \mathsf{refl}_{f(a)}. \end{aligned}$$

Therefore, we can specify the base case $c :\equiv \mathsf{refl}_{\mathsf{refl}_{f(a)}}$ to complete the proof by based path induction.

□

**Remark 2.3.9.** We can represent the previous lemma in a propositionally commutative path diagram, where "composition" means concatenation:




**Lemma 2.3.10.** *Let $P\colon A \to \mathcal{U}$ be a type family. Given paths $p\colon x =_A y$ and $q\colon y =_A z$, and a term $u\colon P(x)$, we have*

$$q_*(p_*(u)) =_{P(z)} (p \cdot q)_*(u).$$

*Proof.* First, observe that both sides of the equation inhabit $P(z)$. To prove equality, we fix $x\colon A$ and proceed by based path induction on $p$.

The motive of the induction is the family of types defined by

$$\begin{aligned} D&\colon \prod_{y\colon A}(x =_A y) \to \mathcal{U} \\ D(y,p) &:\equiv \prod_{u\colon P(x)} \prod_{z\colon A} \prod_{q\colon y=_A z} q_*(p_*(u)) =_{P(z)} (p \cdot q)_*(u). \end{aligned}$$

We must construct a term of type $D(x,\mathsf{refl}_x)$. Substituting $y$ with $x$ and $p$ with $\mathsf{refl}_x$, the goal becomes

$$\prod_{u\colon P(x)} \prod_{z\colon A} \prod_{q\colon x=_A z} q_*((\mathsf{refl}_x)_*(u)) =_{P(z)} (\mathsf{refl}_x \cdot q)_*(u).$$

Using the definitional equalities $(\mathsf{refl}_x)_*(u) \equiv u$ and $\mathsf{refl}_x \cdot q \equiv q$, this reduces to

$$\prod_{u:\, P(x)} \prod_{z:\, A} \prod_{q:\, x =_A z} q_*(u) =_{P(z)} q_*(u),$$

which is inhabited by $\lambda u, z, q.\, \mathsf{refl}_{q_*(u)}$.

□

**Lemma 2.3.11.** *Let $P\colon A \to \mathcal{U}$ be a type family. For any path $p\colon x =_A y$ and elements $u\colon P(x)$ and $v\colon P(y)$, there are paths*

$$\mathsf{transportinv}_l^P(p, u)\colon (p^{-1})_*(p_*(u)) =_{P(x)} u,$$
$$\mathsf{transportinv}_r^P(p, v)\colon p_*((p^{-1})_*(v)) =_{P(y)} v.$$

*Moreover,*

$$\mathsf{transportinv}_l^P(\mathsf{refl}_x, u) \equiv \mathsf{refl}_u, \quad \mathsf{transportinv}_r^P(\mathsf{refl}_x, v) \equiv \mathsf{refl}_v.$$

*Proof.* The expressions are well-typed: given $u\colon P(x)$ and $v\colon P(y)$, we have

$$p_*(u)\colon P(y), \qquad (p^{-1})_*(v)\colon P(x).$$

We can now proceed by path induction on $p$. It suffices to assume $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case, definitionally, $p^{-1} \equiv \mathsf{refl}_x$. Since transport along reflexivity acts as the identity function, the left-hand side of the first identity type reduces as follows:

$$(\mathsf{refl}_x)_*\big((\mathsf{refl}_x)_*(u)\big) \equiv (\mathsf{refl}_x)_*(u) \equiv u,$$

The calculation for $v$ is identical. Thus, in both cases, the goal is proved by $\mathsf{refl}$.

□

**Remark 2.3.12.** In calculus, the *Chain Rule* establishes how to compute the derivative of a composition $P \circ f$ in the direction of a vector $v$

$$d(P \circ f)_x(v) = dP_{f(x)}(df_x(v)).$$

In more advanced Differential Geometry, this corresponds to the definition of *pullback connection*: given a bundle $P$ over $B$ equipped with a connection (transport), and a map $f\colon A \to B$, we can create a *pullback bundle* $f^*P$ over $A$. The natural way to transport objects in this new bundle $f^*P$ is simply to map the path into $B$ and use the original transport there:

$$\nabla_v^{\text{pullback}} = \nabla_{f_* v}^{\text{original}}.$$

In HoTT, the analogous statement is given by the following

**Lemma 2.3.13.** *For a function $f\colon A \to B$, a type family $P\colon B \to \mathcal{U}$, and any $p\colon x =_A y$ and $u\colon P(f(x))$, we have*

$$\mathsf{transport}^{P\circ f}(p,u) =_{P(f(y))} \mathsf{transport}^{P}(\mathsf{ap}_f(p),u).$$

*Proof.* Since both sides of the equality are inhabitants of $P(f(y))$, we can proceed to proving it by path induction on $p$.

For the motive, define

$$C\colon \prod_{y\colon A}(x =_A y) \to \mathcal{U}$$

by

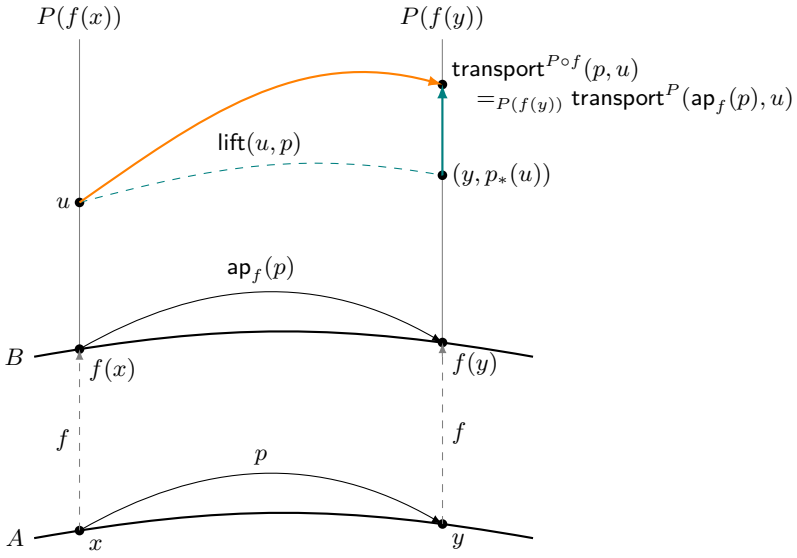
$$C(y,p) :\equiv \prod_{u\colon P(f(x))} \mathsf{transport}^{P\circ f}(p,u) =_{P(f(y))} \mathsf{transport}^{P}(\mathsf{ap}_f(p),u).$$

Evaluating at the base case, we have

$$\begin{aligned} C(x,\mathsf{refl}_x) &\equiv \prod_{u\colon P(f(x))} \mathsf{transport}^{P\circ f}(\mathsf{refl}_x,u) =_{P(f(x))} \mathsf{transport}^{P}(\mathsf{ap}_f(\mathsf{refl}_x),u) \\ &\equiv \prod_{u\colon P(f(x))} u =_{P(f(x))} \mathsf{transport}^{P}(\mathsf{refl}_{f(x)},u) \\ &\equiv \prod_{u\colon P(f(x))} u =_{P(f(x))} u, \end{aligned}$$

which is inhabited by $\lambda u\colon P(f(x)).\,\mathsf{refl}_u$. □

**Remark 2.3.14.** The lemma is illustrated in the following diagram



**Lemma 2.3.15.** *Let $P, Q\colon A \to \mathcal{U}$ be type families and $f\colon \prod_{x:A} P(x) \to Q(x)$ a family map. For any path $p\colon x =_A y$ and term $u\colon P(x)$, we have*

$$\mathsf{transport}^Q(p, f_x(u)) =_{Q(y)} f_y(\mathsf{transport}^P(p, u)).$$

*Equivalently, the following diagram is propositionally commutative*

$$\begin{array}{ccc} P(x) & \xrightarrow{\mathsf{transport}^P(p,-)} & P(y) \\ {\scriptstyle f_x}\downarrow & & \downarrow{\scriptstyle f_y} \\ Q(x) & \xrightarrow[\mathsf{transport}^Q(p,-)]{} & Q(y) \end{array}$$

*Proof.* Since $\mathsf{transport}^P(p, u)\colon P(y)$, both sides inhabit $Q(y)$. Therefore, we can proceed with based path induction on $p$ to proving it.

Fix $a\colon A$ and consider the motive

$$C\colon \prod_{x:A}(a =_A x) \to \mathcal{U}$$
$$C(x, p) :\equiv \prod_{u:P(a)} \mathsf{transport}^Q(p, f_a(u)) =_{Q(x)} f_x(\mathsf{transport}^P(p, u)).$$

For the base case we obtain

$$\begin{aligned} C(a, \mathsf{refl}_a) &\equiv \prod_{u:P(a)} \mathsf{transport}^Q(\mathsf{refl}_a, f_a(u)) =_{Q(a)} f_a(\mathsf{transport}^P(\mathsf{refl}_a, u)) \\ &\equiv \prod_{u:P(a)} f_a(u) =_{Q(a)} f_a(u), \end{aligned}$$

which is inhabited by $\lambda u\colon P(a).\, \mathsf{refl}_{f_a(u)}$. □

**Theorem 2.3.16.** *Let $A, A'\colon \mathcal{U}$ be types, let $B\colon A \to \mathcal{U}$ and $B'\colon A' \to \mathcal{U}$ be type families, and let $g\colon A \to A'$ be a function. Given a family of functions $h\colon \prod_{a:A} B(a) \to B'(g(a))$, a path $p\colon x =_A y$, and an element $u\colon B(x)$, there is a path*

$$\mathsf{ap}_g(p)_*(h(x, u)) =_{B'(g(y))} h(y, p_*(u)).$$

*Proof.* This theorem can be proved by combining Lemmas 2.3.13 and 2.3.15. However, we provide an alternative proof below using path induction.

First, let us verify that the equality is well-typed:

$$\mathsf{ap}_g(p)\colon g(x) =_{A'} g(y),$$
$$\mathsf{ap}_g(p)_* \equiv \mathsf{transport}^{B'}(\mathsf{ap}_g(p)).$$

Therefore,

$$\underbrace{\mathsf{ap}_g(p)_*(\underbrace{h(x,u)}_{B'(g(x))})}_{B'(g(y))} =_{B'(g(y))} \underbrace{h(y,\underbrace{p_*(u)}_{B(y)})}_{B'(g(y))}.$$

By path induction on $p$, we may assume $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case, $\mathsf{ap}_g(p)$ computes definitionally to $\mathsf{refl}_{g(x)}$. Since transport along reflexivity is the identity, both sides compute definitionally to $h(x,u)$, and the equality is witnessed by $\mathsf{refl}_{h(x,u)}$. □

## 2.4 Function Homotopies

### 2.4.1 The Homotopy Relation

**Notation 2.4.1.** *Given a type family $P\colon A \to \mathcal{U}$, we will sometimes denote the type of sections*[3] $\prod_{x\colon A} P(x)$ *by* $\Pi_A P$.

**Definition 2.4.2.** Let $f, g\colon \prod_{x\colon A} P(x)$ be two sections of the family $P\colon A \to \mathcal{U}$. A *homotopy* from $f$ to $g$ is a dependent function of type

$$f \sim g :\equiv \prod_{x\colon A} f(x) =_{P(x)} g(x).$$

**Example 2.4.3.** For any function $f\colon A \to B$, the dependent function

$$\lambda x.\, \mathsf{refl}_{f(x)} \;:\; \prod_{x\colon A} f(x) =_B f(x)$$

is a homotopy from $f$ to $f$. This is often called the *reflexive homotopy*.

**Example 2.4.4.** As defined in 31. Dependent functions over product,

$$\mathsf{uniq}_{A\times B}\colon (\pi_1, \pi_2) \sim \mathsf{id}_{A\times B},$$

where $(\pi_1, \pi_2) :\equiv \lambda x.\, (\pi_1(x), \pi_2(x))$.

**Proposition 2.4.5.** *Homotopy is an equivalence relation on each dependent function type $\prod_{x\colon A} P(x)$. That is, we have elements of the types*

$$\prod_{f\colon \Pi_A P} f \sim f$$
$$\prod_{f,g\colon \Pi_A P} (f \sim g) \to (g \sim f)$$
$$\prod_{f,g,h\colon \Pi_A P} (f \sim g) \to (g \sim h) \to (f \sim h).$$

[3] Remark 2.3.2.

*Proof.* We have the following inhabitants for each of the Π-types:

- *Reflexivity:* $\lambda f\colon \Pi_A P.\, \lambda x\colon A.\, \mathsf{refl}_{f(x)}$.
- *Symmetry:* $\lambda f, g\colon \Pi_A P.\, \lambda \eta\colon f \sim g.\, \lambda x\colon A.\, \eta(x)^{-1}$.
- *Transitivity:* $\lambda f, g, h\colon \Pi_A P.\, \lambda \eta\colon f \sim g.\, \lambda \vartheta\colon g \sim h.\, \lambda x\colon A.\, \eta(x) \cdot \vartheta(x)$.

□

**Definitions 2.4.6.**

- Given a homotopy $\eta\colon f \sim g$, its *inverse* is the homotopy $\eta^{-1}\colon g \sim f$ defined by
$$\eta^{-1} :\equiv \lambda x.\, \eta(x)^{-1}.$$
- Given homotopies $\eta\colon f \sim g$ and $\vartheta\colon g \sim h$, their *composition* (a.k.a. *vertical concatenation*) is the homotopy $\eta \cdot \vartheta\colon f \sim h$ defined by
$$\eta \cdot \vartheta :\equiv \lambda x.\, \eta(x) \cdot \vartheta(x).$$

### 2.4.2 Types as Categories

Types admit different interpretations: depending on the circumstances, they can be viewed as sets, propositions, or spaces. Another way to think of types is as categories. In this analogy we have

Objects. These are the elements of the type: $x, y\colon A$ represent the objects that populate the category $A$.

Arrows. Given two objects $x, y\colon A$, an arrow is a path $p\colon x =_A y$.

Composition. Path composition corresponds to concatenation. Note that for paths $p\colon x =_A y$ and $q\colon y =_A z$, the concatenation $p \cdot q\colon x =_A z$ is written in diagrammatic order, which reverses standard categorical notation. To align with categorical conventions, one may alternatively define $q \circ p :\equiv p \cdot q$. Furthermore, this concatenation is propositionally associative.

Identity. Given $x\colon A$ the identity for concatenation is $\mathsf{refl}_x$. Note that $\mathsf{refl}_x$ is the propositional unit of concatenation.

Functors. A function $f\colon A \to B$ acts as a functor between the corresponding categories:

| **Feature** | **Domain** | **Codomain** |
|---|---|---|
| Objects | $x\colon A$ | $f(x)\colon B$ |
| Morphisms | $p\colon x =_A y$ | $\tilde{f}(p)\colon f(x) =_B f(y)$ |
| Composition | $p \cdot q$ | $\tilde{f}(p) \cdot \tilde{f}(q)$ |

NATURAL TRANSFORMATIONS. These are the homotopies: given $f, g\colon A \to B$, a natural transformation between them is an inhabitant of the homotopy type $\eta\colon f \sim g$. To see this we need to show the (propositional) commutativity of the square

$$\begin{array}{ccc} f(x) & \xrightarrow{\mathsf{ap}_f(p)} & f(y) \\ {\scriptstyle \eta_x}\downarrow & & \downarrow{\scriptstyle \eta_y} \\ g(x) & \xrightarrow[\mathsf{ap}_g(p)]{} & g(y) \end{array} \tag{2.1}$$

i.e., for every $p\colon x =_A y$, the type

$$\mathsf{ap}_f(p) \cdot \eta_y =_{f(x)=_B g(y)} \eta_x \cdot \mathsf{ap}_g(p) \tag{2.2}$$

is inhabited. This follows by based path induction on $p$ using the motive $D\colon \prod_{y:A}(x =_A y) \to \mathcal{U}$ defined by

$$D(y,p) :\equiv \mathsf{ap}_f(p) \cdot \eta_y =_{f(x)=_B g(y)} \eta_x \cdot \mathsf{ap}_g(p),$$

which satisfies

$$\begin{aligned} D(x, \mathsf{refl}_x) &:\equiv \mathsf{ap}_f(\mathsf{refl}_x) \cdot \eta_x =_{f(x)=_B g(x)} \eta_x \cdot \mathsf{ap}_g(\mathsf{refl}_x) \\ &\equiv \mathsf{refl}_{f(x)} \cdot \eta_x =_{f(x)=_B g(x)} \eta_x \cdot \mathsf{refl}_{g(x)} \\ &\equiv \eta_x =_{f(x)=_B g(x)} \eta_x \cdot \mathsf{refl}_{g(x)}. \end{aligned}$$

By Lemma 1.11.4, $\mathsf{ru}(\eta_x)\colon \eta_x \cdot \mathsf{refl}_{g(x)} =_{f(x)=_B g(x)} \eta_x$. Hence, $\mathsf{ru}(\eta_x)^{-1}$ is an inhabitant of $D(x, \mathsf{refl}_x)$.[4]

### 2.4.3 The Algebra of Homotopies

**Proposition 2.4.7.** [Function whiskering] *The homotopy relation is compatible with right and left composition:*

RIGHT: *Given $\eta\colon f \sim g$ and a right composable function $h$, we have*

$$\eta \triangleright h :\equiv \lambda z.\, \eta(h(z))\colon f \circ h \sim g \circ h.$$

LEFT: *Given a left composable function $h$ and $\eta\colon f \sim g$, we have*

$$h \triangleleft \eta :\equiv \lambda x.\, \mathsf{ap}_h(\eta(x))\colon h \circ f \sim h \circ g.$$

*Proof.* Assume that $f, g\colon A \to B$ and let $\eta\colon f \sim g$ be a homotopy.

First, consider the case where $h\colon C \to A$. Given $z\colon C$, we have $h(z)\colon A$ and therefore a path $\eta(h(z))\colon f(h(z)) =_B g(h(z))$. Hence, the term $\lambda z.\, \eta(h(z))$ is a well-defined witness of $f \circ h \sim g \circ h$.

[4] To achieve independence of our definition of concatenation, we could also have used the left unit law $\mathsf{lu}$, that satisfies $\mathsf{lu}(p)\colon \mathsf{refl}_x \cdot p =_{x=_A y} p$, arriving at $\mathsf{lu}(\eta_x) \cdot \mathsf{ru}(\eta_x)^{-1}$.

For the case where $h\colon B \to C$, the function $\mathsf{ap}_h$ constructs a witness of the equality $h(f(x)) =_C h(g(x))$ from the witness $\eta(x)$ of $f(x) =_B g(x)$. Thus, the term $\lambda x.\, \mathsf{ap}_h(\eta(x))$ is a well-defined witness of $h \circ f \sim h \circ g$.

□

**Remark 2.4.8.** In the particular case where $f \equiv g$ and $\eta \equiv \mathsf{refl}_f$, defined by $x \mapsto \mathsf{refl}_{f(x)}$, we have

$$\begin{aligned} \mathsf{refl}_f \triangleright h &\equiv \lambda z.\, \mathsf{refl}_f(h(z)) \\ &\equiv \lambda z.\, \mathsf{refl}_{\mathsf{f(h(z))}} \\ &\equiv \mathsf{refl}_{f \circ h} \end{aligned}$$

and

$$\begin{aligned} h \triangleleft \mathsf{refl}_f &\equiv \lambda x.\, \mathsf{ap}_h(\mathsf{refl}_f(x)) \\ &\equiv \lambda x.\, \mathsf{ap}_h(\mathsf{refl}_{f(x)}) \\ &\equiv \lambda x.\, \mathsf{refl}_{h(f(x))} && \text{; Exr. 1.16.1 a)} \\ &\equiv \mathsf{refl}_{h \circ f}. \end{aligned}$$

**Proposition 2.4.9.** *Function whiskering*[5] *preserves inverses and vertical composition.*[6]

*More precisely; let $\eta\colon f \sim g$ and $\vartheta\colon g \sim h$, then*

Right Whiskering: *Given $k\colon A' \to A$, the following hold definitionally:*

- $(\eta^{-1}) \triangleright k \equiv (\eta \triangleright k)^{-1}$.
- $(\eta \cdot \vartheta) \triangleright k \equiv (\eta \triangleright k) \cdot (\vartheta \triangleright k)$.

Left Whiskering: *Given $k\colon B \to B'$, we have homotopies:*

- $k \triangleleft (\eta^{-1}) \sim (k \triangleleft \eta)^{-1}$.
- $k \triangleleft (\eta \cdot \vartheta) \sim (k \triangleleft \eta) \cdot (k \triangleleft \vartheta)$.

*Proof.* In the case of right whiskering, the properties follow immediately from $\lambda$-conversion:

$$\begin{aligned} (\eta^{-1} \triangleright k)(x) &\equiv \eta^{-1}(k(x)) \\ &\equiv \eta(k(x))^{-1} \\ &\equiv (\eta \triangleright k)(x)^{-1} \\ &\equiv (\eta \triangleright k)^{-1}(x). \end{aligned}$$

[5] Definition 2.4.6.

[6] Thus, function whiskering is a homomorphism of the groupoid structure of homotopies.

Similarly for composition:

$$\begin{aligned}((\eta \cdot \vartheta) \triangleright k)(x) &\equiv (\eta \cdot \vartheta)(k(x)) \\ &\equiv \eta(k(x)) \cdot \vartheta(k(x)) \\ &\equiv (\eta \triangleright k)(x) \cdot (\vartheta \triangleright k)(x) \\ &\equiv ((\eta \triangleright k) \cdot (\vartheta \triangleright k))(x).\end{aligned}$$

The properties for left whiskering follow pointwise from the functorial properties of $\mathsf{ap}_k$ (Proposition 2.1.1):

$$\begin{aligned}(k \triangleleft \eta^{-1})(x) &\equiv \mathsf{ap}_k(\eta(x)^{-1}) \\ &=_{k(g(x))=_{B'}k(f(x))} \mathsf{ap}_k(\eta(x))^{-1} \\ &\equiv (k \triangleleft \eta)(x)^{-1} \\ &\equiv (k \triangleleft \eta)^{-1}(x).\end{aligned}$$

And for composition:

$$\begin{aligned}(k \triangleleft (\eta \cdot \vartheta))(x) &\equiv \mathsf{ap}_k(\eta(x) \cdot \vartheta(x)) \\ &=_{k(f(x))=_{B'}k(h(x))} \mathsf{ap}_k(\eta(x)) \cdot \mathsf{ap}_k(\vartheta(x)) \\ &\equiv (k \triangleleft \eta)(x) \cdot (k \triangleleft \vartheta)(x) \\ &\equiv ((k \triangleleft \eta) \cdot (k \triangleleft \vartheta))(x).\end{aligned}$$

□

**Proposition 2.4.10.** *Given a homotopy $\eta$ and appropriately composable functions $h$ and $k$, we have*[7]

a) $(\eta \triangleright h) \triangleright k \equiv \eta \triangleright (h \circ k)$.

b) $k \triangleleft (h \triangleleft \eta) = (k \circ h) \triangleleft \eta$.

c) $h \triangleleft (\eta \triangleright k) \equiv (h \triangleleft \eta) \triangleright k$.

*Proof.* The first equation is a direct consequence of the definitions. The second follows from Proposition 2.1.1. For the third we have,

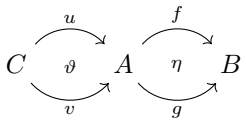
$$\begin{aligned}h \triangleleft (\eta \triangleright k) &\equiv \lambda x.\, \mathsf{ap}_h(\eta(k(x)) \\ &\equiv \lambda x.\, (h \triangleleft \eta)(k(x)) \\ &\equiv \lambda x.\, (h \triangleleft \eta) \circ k(x) \\ &\equiv (h \triangleleft \eta) \triangleright k.\end{aligned}$$

□

[7] Compare with Proposition 2.1.5.

### 2.4.4 Evaluating a homotopy on another homotopy

Suppose that in the context where we have two function $f, g\colon A \to B$ and a homotopy $\eta\colon f \sim g$, we also have a third type $C$, two functions $u, v\colon C \to A$ and another homotopy $\vartheta\colon u \sim v$, as depicted in the diagram




For every $z\colon C$, we can plug the path $\vartheta(z)$ into the naturality equation (2.2) using the endpoints $x :\equiv u(z)$ and $y :\equiv v(z)$, and obtain

$$\mathsf{ap}_f(\vartheta(z)) \cdot \eta(v(z)) = \eta(u(z)) \cdot \mathsf{ap}_g(\vartheta(z)).$$

Thus, abstracting $z$ and using the whiskering notation, we deduce that

$$(f \triangleleft \vartheta) \cdot (\eta \triangleright v) \sim (\eta \triangleright u) \cdot (g \triangleleft \vartheta). \tag{2.1}$$

is inhabited. This can be visualized as a commutative square where the vertices are functions and the edges homotopies:

$$\begin{array}{ccc} f \circ u & \xrightarrow{\eta \triangleright u} & g \circ u \\ {\scriptstyle f \triangleleft \vartheta}\downarrow & & \downarrow{\scriptstyle g \triangleleft \vartheta} \\ f \circ v & \xrightarrow[\eta \triangleright v]{} & g \circ v \end{array} \tag{2.2}$$

### 2.4.5 Sections of Path Spaces

**Definition 2.4.11.** Let $A$ be a type and $E :\equiv \sum_{x,y:\,A} x =_A y$ its total path space. The *reflexivity section* $\rho_A$ is the map that assigns to every point its trivial loop:

$$\rho_A\colon A \to E, \quad x \mapsto (x, x, \mathsf{refl}_x).$$

**Remark 2.4.12.** Let $C$ be a family over the path space $E$ of $A$. Given a dependent function $f$ defined on $E$

$$f\colon \prod_{\omega:\,E} C(\omega),$$

the composition of $f \circ \rho_A$ is the dependent function

$$f \circ \rho_A :\equiv \lambda x.\, f(x, x, \mathsf{refl}_x)\colon \prod_{x:\,A} C(x, x, \mathsf{refl}_x).$$

**Lemma 2.4.13.** *With the preceding notation, let* $C\colon E \to \mathcal{U}$ *be a type family. Then given two dependent functions* $f, g\colon \prod_{\omega\colon E} C(\omega)$*, if their compositions with* $\rho_A$ *are homotopic, i.e.,*

$$f \circ \rho_A \sim g \circ \rho_A,$$

*then* $f \sim g$*.*

*Proof.* By definition of homotopy, we must construct a term of type

$$f(x, y, p) =_{C(x,y,p)} g(x, y, p)$$

for all $x, y\colon A$ and $p\colon x =_A y$.

Fix $x\colon A$. By based path induction on $p$, it suffices to construct the term for the base case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case, the required type becomes

$$f(x, x, \mathsf{refl}_x) =_{C(x,x,\mathsf{refl}_x)} g(x, x, \mathsf{refl}_x),$$

which is exactly the type inhabited by the hypothesis. □

## 2.5 Equivalences

**Definitions 2.5.1.**

- A function $f\colon A \to B$ is an *equivalence* if the type $\mathsf{isequiv}(f)$ is inhabited, where
$$\mathsf{isequiv}(f) :\equiv \Big(\sum_{g\colon B\to A} f \circ g \sim \mathsf{id}_B\Big) \times \Big(\sum_{h\colon B\to A} h \circ f \sim \mathsf{id}_A\Big).$$
- A *quasi-inverse* of a function $f\colon A \to B$ is a triple $\langle g, (\eta, \vartheta)\rangle$ consisting of a function $g\colon B \to A$ and homotopies $\eta\colon f \circ g \sim \mathsf{id}_B$ and $\vartheta\colon g \circ f \sim \mathsf{id}_A$. Formally, it is an inhabitant of the type
$$\mathsf{qinv}(f) :\equiv \sum_{g\colon B\to A} (f \circ g \sim \mathsf{id}_B) \times (g \circ f \sim \mathsf{id}_A).$$

**Lemma 2.5.2.** *If* $f\colon A \to B$ *has two quasi-inverses*

$$\langle g, (\eta, \vartheta)\rangle\colon \mathsf{qinv}(f) \quad \textit{and} \quad \langle g', (\eta', \vartheta')\rangle\colon \mathsf{qinv}(f),$$

*then* $g \sim g'$*.*

*Proof.* Let $y\colon B$. Since $\eta'\colon f\circ g' \sim \mathsf{id}_B$, we have the path $\eta'(y)^{-1}\colon y =_B f(g'(y))$. Hence,

$$\begin{aligned} g(y) &=_A g(f(g'(y))) && ;\text{ by } \mathsf{ap}_g(\eta'(y)^{-1}) \\ &\equiv (g \circ f)(g'(y)) \\ &=_A g'(y) && ;\text{ by } \vartheta(g'(y)). \end{aligned}$$

Thus, $g \sim g'$. □

**Notation 2.5.3.** *Given a quasi-invertible function $f\colon A \to B$, we will write $f^{-1}$ to refer to an arbitrary quasi-inverse of $f$. By* Lemma 2.5.2*, this notation is unambiguous up to homotopy.*

**Lemma 2.5.4.** *If $f\colon A \to B$ and $g\colon B \to C$ are equivalences with inhabitants*

$$(\langle h_1, \eta_1\rangle, \langle k_1, \vartheta_1\rangle)\colon \mathsf{isequiv}(f) \quad \textit{and} \quad (\langle h_2, \eta_2\rangle, \langle k_2, \vartheta_2\rangle)\colon \mathsf{isequiv}(g),$$

*then their composition $g \circ f$ is an equivalence inhabited by*

$$(\langle h_1 \circ h_2, \eta\rangle, \langle k_1 \circ k_2, \vartheta\rangle)\colon \mathsf{isequiv}(g \circ f),$$

*where the homotopies are defined by whiskering:*

$$\begin{aligned} \eta &:\equiv (g \lhd \eta_1 \rhd h_2) \cdot \eta_2\colon g \circ f \circ h_1 \circ h_2 \sim \mathsf{id}_C \\ \vartheta &:\equiv (k_1 \lhd \vartheta_2 \rhd f) \cdot \vartheta_1\colon k_1 \circ k_2 \circ g \circ f \sim \mathsf{id}_A. \end{aligned} \tag{2.1}$$

*Proof.* The hypotheses provide the following homotopies:

$$\begin{aligned} &\eta_1\colon f \circ h_1 \sim \mathsf{id}_B, && \eta_2\colon g \circ h_2 \sim \mathsf{id}_C, \\ &\vartheta_1\colon k_1 \circ f \sim \mathsf{id}_A, && \vartheta_2\colon k_2 \circ g \sim \mathsf{id}_B. \end{aligned}$$

Whiskering these yields the intermediate steps:

$$\begin{aligned} g \lhd \eta_1 &\colon g \circ f \circ h_1 \sim g, \\ \vartheta_2 \rhd f &\colon k_2 \circ g \circ f \sim f. \end{aligned}$$

By Proposition 2.4.10 (associativity of whiskering), we obtain:

$$\begin{aligned} g \lhd \eta_1 \rhd h_2 &\colon g \circ f \circ h_1 \circ h_2 \sim g \circ h_2, \\ k_1 \lhd \vartheta_2 \rhd f &\colon k_1 \circ k_2 \circ g \circ f \sim k_1 \circ f. \end{aligned}$$

Finally, by Proposition 2.4.5 (homotopy transitivity), utilizing $\eta_2$ and $\vartheta_1$, we obtain the homotopies in (2.1). □

**Remark 2.5.5.** Notation 2.5.3 allows us to restate Lemma 2.5.2 and the symmetry of the quasi-inverse definition as

$$(f^{-1})^{-1} \sim f,$$

whenever $f$ has a quasi-inverse.

**Proposition 2.5.6.** *Let $f\colon A \to B$. Then:*

a) *There is a canonical map $\mathsf{qinv}(f) \to \mathsf{isequiv}(f)$*

$$\langle g, (\eta, \vartheta)\rangle \mapsto (\langle g, \eta\rangle, \langle g, \vartheta\rangle).$$

b) *There is a map* $\mathsf{isequiv}(f) \to \mathsf{qinv}(f)$.

*Consequently, the type* $\mathsf{qinv}(f)$ *is inhabited if, and only if,* $\mathsf{isequiv}(f)$ *is inhabited.*

*Proof.*

a) This follows immediately from the definition, setting $h :\equiv g$.

b) Let $(\langle g, \eta\rangle, \langle h, \vartheta\rangle) \colon \mathsf{isequiv}(f)$. By definition, we have homotopies:

$$\eta \colon f \circ g \sim \mathsf{id}_B \quad \text{and} \quad \vartheta \colon h \circ f \sim \mathsf{id}_A.$$

Using the whiskering operations, we obtain:

$$h \triangleleft \eta \colon h \circ f \circ g \sim h,$$
$$\vartheta \triangleright g \colon h \circ f \circ g \sim g.$$

Evaluating these at a point $y \colon B$ gives paths

$$\mathsf{ap}_h(\eta(y)) \colon h(f(g(y))) =_B h(y) \quad \text{and} \quad \vartheta(g(y)) \colon h(f(g(y))) =_A g(y).$$

We can therefore define a homotopy $\gamma \colon g \sim h$ by

$$\gamma(y) :\equiv \vartheta(g(y))^{-1} \cdot \mathsf{ap}_h(\eta(y)).$$

We now construct the required left homotopy $\vartheta' \colon g \circ f \sim \mathsf{id}_A$. For any $x \colon A$, we have the path $\gamma(f(x)) \colon g(f(x)) =_A h(f(x))$. Concatenating this with $\vartheta(x)$ yields

$$\gamma(f(x)) \cdot \vartheta(x) \colon g(f(x)) =_A x.$$

Thus, we define the map to be $\langle g, (\eta, \vartheta')\rangle$, where $\vartheta'(x) :\equiv \gamma(f(x)) \cdot \vartheta(x)$.

□

**Definition 2.5.7.** An *equivalence* from $A$ to $B$ is a function $f \colon A \to B$ together with an inhabitant of $\mathsf{isequiv}(f)$, i.e., a proof that it is an equivalence. We write $A \simeq B$ for the type of equivalences from $A$ to $B$, i.e., the type

$$A \simeq B :\equiv \sum_{f \colon A \to B} \mathsf{isequiv}(f).$$

**Remark 2.5.8.** Two types $A$ and $B$ are *logically equivalent* when the type

$$(A \to B) \times (B \to A)$$

is inhabited. While equivalent types are logically equivalent, the reciprocal is not generally true.[8]

[8] See, however, Lemma 3.2.4.

**Proposition 2.5.9.** *The following properties hold*

a) $\langle \mathsf{id}_A, (\mathsf{refl}_{\mathsf{id}_A}, \mathsf{refl}_{\mathsf{id}_A})\rangle : \mathsf{qinv}(\mathsf{id}_A)$

b) *Given* $f : A \to B$ *and* $g : B \to C$ *we have,*

$$\left.\begin{array}{l}\langle f^{-1}, (\eta_f, \vartheta_f)\rangle : \mathsf{qinv}(f)\\ \langle g^{-1}, (\eta_g, \vartheta_g)\rangle : \mathsf{qinv}(g)\end{array}\right\} \implies \langle g^{-1} \circ f^{-1}, (\eta, \vartheta)\rangle : \mathsf{qinv}(g \circ f),$$

*where*

$$\eta :\equiv (g \triangleleft \eta_f \triangleright g^{-1}) \cdot \eta_g : g \circ f \circ f^{-1} \circ g^{-1} \sim \mathsf{id}_C$$
$$\vartheta :\equiv (f^{-1} \triangleleft \vartheta_g \triangleright f) \cdot \vartheta_f : f^{-1} \circ g^{-1} \circ g \circ f \sim \mathsf{id}_A.$$

*Proof.*

a) By definition

$$\mathsf{qinv}(\mathsf{id}_A) \equiv \sum_{g:\, A\to A} (\mathsf{id}_A \circ g \sim \mathsf{id}_A) \times (g \circ \mathsf{id}_A \sim \mathsf{id}_A).$$

Since $\mathsf{id}_A \circ \mathsf{id}_A \equiv \mathsf{id}_A$, we can take $g \equiv \mathsf{id}_A$ and observe that

$$\mathsf{refl}_{\mathsf{id}_A} : \mathsf{id}_A =_{A\to A} \mathsf{id}_A$$

defines the homotopy $x \mapsto \mathsf{refl}_x$ between $\mathsf{id}_A$ and itself.

b) By Proposition 2.5.6, this is a direct consequence of Lemma 2.5.4 applied to the case where $h_1 \equiv k_1 \equiv f^{-1}$ and $h_2 \equiv k_2 \equiv g^{-1}$.

□

**Definition 2.5.10.** Part b) of Proposition 2.5.9, allows us to define the *composition* of quasi-inverses by

$$\langle f^{-1}, (\eta_f, \vartheta_f)\rangle \circ \langle g^{-1}, (\eta_g, \vartheta_g)\rangle :\equiv \langle g^{-1} \circ f^{-1}, (\eta, \vartheta)\rangle$$

with

$$\eta :\equiv (g \triangleleft \eta_f \triangleright g^{-1}) \cdot \eta_g$$
$$\vartheta :\equiv (f^{-1} \triangleleft \vartheta_g \triangleright f) \cdot \vartheta_f.$$

**Corollary 2.5.11.** *In the particular case where one of the functions is the identity, we have*

a) $\langle \mathsf{id}_A, (\mathsf{refl}_{\mathsf{id}_A}, \mathsf{refl}_{\mathsf{id}_A})\rangle \circ \langle f^{-1}, (\eta, \vartheta)\rangle \equiv \langle f^{-1}, (\eta, \vartheta \cdot \mathsf{refl}_{\mathsf{id}_A})\rangle$

b) $\langle f^{-1}, (\eta, \vartheta)\rangle \circ \langle \mathsf{id}_A, (\mathsf{refl}_{\mathsf{id}_A}, \mathsf{refl}_{\mathsf{id}_A})\rangle \equiv \langle f^{-1}, (\eta \cdot \mathsf{refl}_{\mathsf{id}_A}, \vartheta)\rangle$

*Proof.*

a) By definition,

$$\begin{aligned}
\text{LHS} &\equiv \langle f^{-1}, (\eta', \vartheta') \rangle \\
\eta' &\equiv (f \triangleleft \mathsf{refl}_{\mathsf{id}_A} \triangleright f^{-1}) \cdot \eta \\
&\equiv \mathsf{refl}_{f \circ f^{-1}} \cdot \eta && \text{; Rem. 2.4.8} \\
&\equiv \lambda x.\, \mathsf{refl}_{f \circ f^{-1}(x)} \cdot \eta(x) \\
&\equiv \lambda x.\, \eta(x) \\
&\equiv \eta. \\
\vartheta' &\equiv (\mathsf{id}_A \triangleleft \vartheta \triangleright \mathsf{id}_A) \cdot \mathsf{refl}_{\mathsf{id}_A} \\
&\equiv \vartheta \cdot \mathsf{refl}_{\mathsf{id}_A} && \text{; Prop. 2.4.7.}
\end{aligned}$$

b) Similarly,

$$\begin{aligned}
\text{RHS} &\equiv \langle f^{-1}, (\eta', \vartheta') \rangle \\
\eta' &\equiv (\mathsf{id}_A \triangleleft \eta \triangleright \mathsf{id}_A) \cdot \mathsf{refl}_{\mathsf{id}_A} \\
&\equiv \eta \cdot \mathsf{refl}_{\mathsf{id}_A}. && \text{; Prop. 2.4.7} \\
\vartheta' &\equiv (f^{-1} \triangleleft \mathsf{refl}_{\mathsf{id}_A} \triangleright f) \cdot \vartheta \\
&\equiv \vartheta && \text{; Idem part a)}
\end{aligned}$$

□

**Notation 2.5.12.** *We will sometimes abbreviate the equivalence*

$$\langle f, (\langle g, \eta \rangle, \langle h, \vartheta \rangle) \rangle : A \simeq B$$

*by writing simply* $f : A \simeq B$.

**Proposition 2.5.13.** *Type equivalence is an equivalence relation on* $\mathcal{U}$. *More specifically:*

a) *For any* $A$, *the identity function* $\mathsf{id}_A$ *is an equivalence; hence* $A \simeq A$.

b) *For any* $f : A \simeq B$, *we have an equivalence* $f^{-1} : B \simeq A$.

c) *For any* $f : A \simeq B$ *and* $g : B \simeq C$, *we have* $g \circ f : A \simeq C$.

*Proof.*

a) Let $\mathsf{refl} :\equiv \lambda x.\, \mathsf{refl}_x$. Since $\mathsf{id}_A \circ \mathsf{id}_A \equiv \mathsf{id}_A$, we clearly have a homotopy $\mathsf{refl} : \mathsf{id}_A \circ \mathsf{id}_A \sim \mathsf{id}_A$. Therefore,

$$(\langle \mathsf{id}_A, \mathsf{refl} \rangle, \langle \mathsf{id}_A, \mathsf{refl} \rangle) : \mathsf{isequiv}(\mathsf{id}_A).$$

Hence,

$$\langle \mathsf{id}_A, (\langle \mathsf{id}_A, \mathsf{refl} \rangle, \langle \mathsf{id}_A, \mathsf{refl} \rangle) \rangle : A \simeq A.$$

b) Suppose that $f\colon A \simeq B$. Then,

$$\begin{aligned}
\langle f, \omega\rangle &\colon A \simeq B && \text{; Notn. 2.5.12}\\
\omega \equiv (\langle g, \eta\rangle, \langle h, \vartheta\rangle) &\colon \mathsf{isequiv}(f) && \text{; Defns. 2.5.7 \& 2.5.1}\\
\langle g, (\eta, \vartheta')\rangle &\colon \mathsf{qinv}(f) && \text{; Prop. 2.5.6}\\
\langle f, (\vartheta', \eta)\rangle &\colon \mathsf{qinv}(g) && \text{; Lem. 2.5.2}\\
\langle g, (\langle f, \vartheta'\rangle, \langle f, \eta\rangle)\rangle &\colon B \simeq A && \text{; Lem. 2.5.2.}
\end{aligned}$$

c) Suppose that $f\colon A \simeq B$ and $g\colon B \simeq C$. This translates into

$$\langle f, (\langle h_1, \eta_1\rangle, \langle k_1, \vartheta_1\rangle)\rangle\colon A \simeq B, \qquad \langle g, (\langle h_2, \eta_2\rangle, \langle k_2, \vartheta_2\rangle)\rangle\colon B \simeq C,$$

with

$$\begin{aligned}
\eta_1 &\colon f \circ h_1 \sim \mathsf{id}_B, & \eta_2 &\colon g \circ h_2 \sim \mathsf{id}_C,\\
\vartheta_1 &\colon k_1 \circ f \sim \mathsf{id}_A, & \vartheta_2 &\colon k_2 \circ g \sim \mathsf{id}_B.
\end{aligned}$$

Then, we have the intermediate homotopies:

$$g \triangleleft \eta_1 \colon g \circ f \circ h_1 \sim g, \qquad \vartheta_2 \triangleright f \colon k_2 \circ g \circ f \sim f.$$

By Proposition 2.4.10 (associativity of whiskering), we obtain:

$$\begin{aligned}
g \triangleleft \eta_1 \triangleright h_2 &\colon g \circ f \circ h_1 \circ h_2 \sim g \circ h_2,\\
k_1 \triangleleft \vartheta_2 \triangleright f &\colon k_1 \circ k_2 \circ g \circ f \sim k_1 \circ f.
\end{aligned}$$

By Proposition 2.4.5 (homotopy transitivity), utilizing $\eta_2$ and $\vartheta_1$, we deduce that

$$(g \circ f) \circ (h_1 \circ h_2) \sim \mathsf{id}_C \quad \text{and} \quad (k_1 \circ k_2) \circ (g \circ f) \sim \mathsf{id}_A$$

are inhabited. Thus, according to Notation 2.5.12, we have $g \circ f\colon A \simeq C$.

□

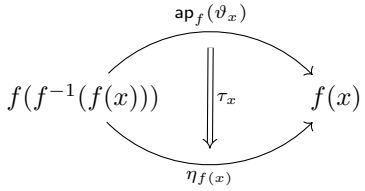
**Remark 2.5.14.** Given that $\langle g, (\eta, \vartheta)\rangle\colon \mathsf{qinv}(f) \iff \langle f, (\vartheta, \eta)\rangle\colon \mathsf{qinv}(g)$, we deduce that $\langle f, (\langle g, \eta\rangle, \langle g, \vartheta\rangle)\rangle^{-1} \equiv \langle g, (\langle f, \vartheta\rangle, \langle f, \eta\rangle)\rangle$. In particular,

$$\langle \mathsf{id}_A, (\langle \mathsf{id}_A, \mathsf{refl}\rangle, \langle \mathsf{id}_A, \mathsf{refl}\rangle)\rangle^{-1} \equiv \langle \mathsf{id}_A, (\langle \mathsf{id}_A, \mathsf{refl}\rangle, \langle \mathsf{id}_A, \mathsf{refl}\rangle)\rangle.$$

**Definition 2.5.15.** Let $f\colon A \to B$ be a function. A *coherent quasi-inverse* (a.k.a. *half-adjoint equivalence*) of $f$ consists of a quasi-inverse $\langle f^{-1}, (\eta, \vartheta)\rangle$ along with a higher homotopy

$$\tau\colon \prod_{x\colon A} \mathsf{ap}_f(\vartheta_x) =_{f(f^{-1}(f(x)))=_B f(x)} \eta_{f(x)}.$$

as illustrated next




Using function whiskering [cf. Proposition 2.4.7], the coherence term $\tau$ can be seen as an homotopy

$$\tau : f \triangleleft \vartheta \sim \eta \triangleright f.$$

**Theorem 2.5.16.** *Let $f : A \to B$ be an equivalence. If $\langle g, (\eta, \vartheta)\rangle$ is a quasi-inverse of $f$, there is an homotopy $\eta' : f \circ g \sim \mathsf{id}_B$, such that $\langle g, (\eta', \vartheta)\rangle$ is a coherent quasi-inverse.*

*Proof.* By definition, $g : B \to A$, $\eta : f \circ g \sim \mathsf{id}_B$, and $\vartheta : g \circ f \sim \mathsf{id}_A$.

We define the new right homotopy $\eta' : f \circ g \sim \mathsf{id}_B$ by requiring the following diagram of homotopies to commute:

$$\begin{array}{ccc} f \circ g \circ f \circ g & \xrightarrow{f \triangleleft \vartheta \triangleright g} & f \circ g \\ {\scriptstyle \eta \triangleright (f \circ g)} \downarrow & & \downarrow {\scriptstyle \eta} \\ f \circ g & \xrightarrow[\eta']{} & \mathsf{id}_B \end{array}$$

To complete the proof, we need to show an inhabitant of $f \triangleleft \vartheta \sim \eta' \triangleright f$. By right-whiskering the diagram defining $\eta'$ with $f$, we obtain:

$$\begin{array}{ccc} f \circ g \circ f \circ g \circ f & \xrightarrow{f \triangleleft \vartheta \triangleright (g \circ f)} & f \circ g \circ f \\ {\scriptstyle \eta \triangleright (f \circ g \circ f)} \downarrow & & \downarrow {\scriptstyle \eta \triangleright f} \\ f \circ g \circ f & \xrightarrow[\eta' \triangleright f]{} & f \end{array}$$

This reduces our goal to proving that $f \triangleleft \vartheta$ fits the bottom edge of this square.

Evaluating the naturality of $\eta$ on the homotopy $f \triangleleft \vartheta : f \circ g \circ f \sim f$, as defined in (2.2), yields the commutative square:

$$\begin{array}{ccc} f \circ g \circ f \circ g \circ f & \xrightarrow{f \triangleleft (g \circ f) \triangleleft \vartheta} & f \circ g \circ f \\ {\scriptstyle \eta \triangleright (f \circ g \circ f)} \downarrow & & \downarrow {\scriptstyle \eta \triangleright f} \\ f \circ g \circ f & \xrightarrow[f \triangleleft \vartheta]{} & f \end{array}$$

Comparing the last two diagrams reduces the goal to showing that their top edges are homotopic; that is,

$$f \triangleleft \vartheta \triangleright (g \circ f) \sim f \triangleleft (g \circ f) \triangleleft \vartheta. \tag{$*$}$$

Evaluating the naturality of $\vartheta$ on itself using (2.2) gives

$$\begin{array}{ccc} (g \circ f) \circ (g \circ f) & \xrightarrow{\vartheta \triangleright (g \circ f)} & g \circ f \\ {\scriptstyle (g \circ f) \triangleleft \vartheta}\downarrow & & \downarrow{\scriptstyle \vartheta} \\ g \circ f & \xrightarrow[\vartheta]{} & \mathsf{id}_A \end{array}$$

By canceling $\vartheta$, we deduce that $\vartheta \triangleright (g \circ f) \sim (g \circ f) \triangleleft \vartheta$. The conclusion $(*)$ follows by whiskering this homotopy with $f$ on the left. □

**Theorem 2.5.17.** [Transport as a coherent equivalence] *Let $A$ be a type. Given a path $p\colon x =_A y$ and a type family $P\colon A \to \mathcal{U}$, the transport function*

$$f :\equiv \mathsf{transport}^P(p, -)\colon P(x) \to P(y)$$

*is a half-adjoint equivalence with quasi-inverse*

$$g :\equiv \mathsf{transport}^P(p^{-1}, -)\colon P(y) \to P(x)$$

*and homotopies*

$$\eta\colon f \circ g \sim \mathsf{id}_{P(y)} \quad \textit{and} \quad \vartheta\colon g \circ f \sim \mathsf{id}_{P(x)}.$$

*together with the coherence condition*

$$\tau\colon f \triangleleft \vartheta \sim \eta \triangleright f.$$

*Proof.* By path induction on $p$, to construct $\eta$, $\vartheta$ and $\tau$, it suffices to assume that $y \equiv x$ and $p \equiv \mathsf{refl}_x$. Under this assumption, the inverse path $p^{-1}$ is also definitionally $\mathsf{refl}_x$. The functions $f$ and $g$ both reduce definitionally to $\mathsf{id}_{P(x)}$. Consequently, the compositions $f \circ g$ and $g \circ f$ evaluate to $\mathsf{id}_{P(x)}$, allowing us to define the homotopies as reflexivity:

$$\eta :\equiv v \mapsto \mathsf{refl}_v$$
$$\vartheta :\equiv u \mapsto \mathsf{refl}_u$$

Substituting these into the coherence condition leads to the goal

$$\mathsf{ap}_{\mathsf{id}_{P(x)}}(\mathsf{refl}_u) =_{u =_{P(x)} u} \mathsf{refl}_u.$$

Since $\mathsf{ap}_{\mathsf{id}_{P(x)}}(\mathsf{refl}_u) \equiv \mathsf{refl}_u$, this type reduces to $\mathsf{refl}_u =_{u =_{P(x)} u} \mathsf{refl}_u$, which is trivially inhabited by $\mathsf{refl}_{\mathsf{refl}_u}$. The conclusion follows. □

**Theorem 2.5.18.** *If $f\colon A \to B$ is an equivalence, then for any $a, a'\colon A$, the function*

$$\mathsf{ap}_f\colon (a =_A a') \to (f(a) =_B f(a'))$$

*is also an equivalence.*

*Proof.* Let $\langle f^{-1}, (\eta, \vartheta)\rangle$ be a quasi-inverse of $f$. By Theorem 2.5.16, we may assume this quasi-inverse is coherent, providing a homotopy $\tau\colon f \triangleleft \vartheta \sim \eta \triangleright f$.

For the inverse function $h\colon (f(a) =_B f(a')) \to (a =_A a')$ consider

$$h(q) :\equiv \vartheta_a^{-1} \cdot \mathsf{ap}_{f^{-1}}(q) \cdot \vartheta_{a'}.$$

To construct a right homotopy $\mathsf{ap}_f \circ h \sim \mathsf{id}_{f(a)=_B f(a')}$, let $q\colon f(a) =_B f(a')$. Using the fact that $\mathsf{ap}_f$ distributes over concatenation, we obtain

$$\mathsf{ap}_f(h(q)) =_{f(a)=_B f(a')} \mathsf{ap}_f(\vartheta_a)^{-1} \cdot \mathsf{ap}_{f\circ f^{-1}}(q) \cdot \mathsf{ap}_f(\vartheta_{a'}).$$

Substituting the coherence $\tau$ evaluated at $a$ and $a'$ yields

$$\mathsf{ap}_f(h(q)) =_{f(a)=_B f(a')} \eta_{f(a)}^{-1} \cdot \mathsf{ap}_{f\circ f^{-1}}(q) \cdot \eta_{f(a')}.$$

The naturality equation (2.2) for $\eta\colon f \circ f^{-1} \sim \mathsf{id}_B$ evaluated on $q$ states that

$$\mathsf{ap}_{f\circ f^{-1}}(q) \cdot \eta_{f(a')} =_{f(f^{-1}(f(a)))=_B f(a')} \eta_{f(a)} \cdot q.$$

Left-concatenating with $\eta_{f(a)}^{-1}$, we recover the RHS of the previous equation. Thus,

$$\mathsf{ap}_f(h(q)) =_{f(a)=_B f(a')} q.$$

To construct a left homotopy $h \circ \mathsf{ap}_f \sim \mathsf{id}_{a=_A a'}$, let $p\colon a =_A a'$. The definition of $h$ yields

$$h(\mathsf{ap}_f(p)) =_{a=_A a'} \vartheta_a^{-1} \cdot \mathsf{ap}_{f^{-1}\circ f}(p) \cdot \vartheta_{a'}.$$

The naturality of $\vartheta$ evaluated on $p$ states that

$$\mathsf{ap}_{f^{-1}\circ f}(p) \cdot \vartheta_{a'} =_{f^{-1}(f(a))=_A a'} \vartheta_a \cdot p.$$

Left-concatenating with $\vartheta_a^{-1}$, we obtain

$$\vartheta_a^{-1} \cdot \mathsf{ap}_{f^{-1}\circ f}(p) \cdot \vartheta_{a'} =_{a=_A a'} p.$$

Therefore, $h(\mathsf{ap}_f(p)) =_{a=_A a'} p$, which completes the proof. □

## 2.6 Observational Equality Framework

In classical Type Theory, equality is often regarded as *opaque*—essentially a mystery. It is defined as *intensional* (not to be confused with intentional), meaning that for two objects to be equal, they must follow the exact same construction.

Homotopy Type Theory attempts to bridge this gap. By interpreting the inhabitants of the type $x =_A y$ as paths from $x$ to $y$ in the *space* $A$, it becomes possible to determine when two objects behave equivalently, independent of their specific construction. This shared behavioral appearance is termed *observational equality*. The rest of this chapter is dedicated to describing the observational equality for each of the basic types introduced in Chapter 1.

Technically, the strategy involves using the $\mathsf{transport}$ function to construct a homotopy between the actual identity type (which remains intensional) and a description relative to the type's internal structure. This description acts as a specific *implementation*, encoding an equivalent equality that is no longer intensional (opaque), but *observational* (visible).

To achieve this for each basic type, we define a type $\mathsf{Code}(x, y)$ associated with the identity $x =_A y$. We then prove that $\mathsf{Code}(x, y) \simeq (x =_A y)$. This process requires defining an $\mathsf{encode}$ function (derived from $\mathsf{transport}$) and a quasi-inverse $\mathsf{decode}$ that maps in the opposite direction, from $\mathsf{Code}(x, y)$ to $x =_A y$.

This approach consists of the following

### 2.6.1 Observational Equality Framework

Code family. Define a family that structurally represents the identity type:

$$\mathsf{Code}\colon A \to A \to \mathcal{U}.$$

Base code. Select an element that corresponds to reflexivity:

$$c\colon \mathsf{Code}(x, x).$$

Encode function. For any $x, y\colon A$ and path $p\colon x =_A y$, define the map

$$\mathsf{encode}\colon (x =_A y) \to \mathsf{Code}(x, y).$$

Applying transport to the base code $c$ with $C_x :\equiv \lambda z.\, \mathsf{Code}(x, z)\colon A \to \mathcal{U}$, we obtain

$$\mathsf{encode}(p) :\equiv \mathsf{transport}^{C_x}(p, c)\colon C_x(y).$$

Note that, by definition of transport, $\mathsf{encode}(\mathsf{refl}_x) \equiv c$.

Decode function. Construct a quasi-inverse to $\mathsf{encode}$

$$\mathsf{decode}\colon \prod_{y\colon A} \mathsf{Code}(x, y) \to (x =_A y).$$

**Remark 2.6.1.** Given $x, y\colon A$, we have to verify that the functions

$$\mathsf{encode}(x,y)\colon (x =_A y) \to \mathsf{Code}(x,y)$$

and

$$\mathsf{decode}(x,y)\colon \mathsf{Code}(x,y) \to (x =_A y)$$

satisfy

$$\mathsf{decode}(x,y,\mathsf{encode}(x,y,p)) =_{x=_A y} p,$$

for $p\colon x =_A y$, and

$$\mathsf{encode}(x,y,\mathsf{decode}(x,y,d)) =_{\mathsf{Code}(x,y)} d,$$

for $d\colon \mathsf{Code}(x,y)$.

**Remark 2.6.2.** The decode function acts as an *introduction rule* (or constructor) for the identity type of points in the space $A$, as it generates paths from structured data. Establishing that encode and decode are quasi-inverses justifies an *induction principle*: to prove a property for all paths, it suffices to consider those constructed by decode. Conversely, the encode function serves as an *elimination rule*, extracting structural features from a given path.

### 2.6.2 Extended Observational Framework

There are two additional requirements to the Observational Equality Framework that imply further properties of the pair encode/decode. They are requirements to be fulfilled for arbitrary $x, y\colon A$.

| | |
|---|---|
| INVERSION MAP: | $\mathsf{inv}\colon \mathsf{Code}(x,y) \to \mathsf{Code}(y,x)$ |
| CONCATENATION: | $*\colon \mathsf{Code}(x,y) \to \mathsf{Code}(y,z) \to \mathsf{Code}(x,z)$ |
| UNIT LAWS: | $\mathsf{inv}(c) \equiv c \quad \& \quad c * d \equiv d$ |

where $c\colon \mathsf{Code}(x,x)$ is the base code and the unit laws hold for $d\colon \mathsf{Code}(x,y)$.[9]

**Definition 2.6.3.** [Groupoid] A *groupoid* is a category in which every morphism is an isomorphism.

**Remark 2.6.4.** In Homotopy Type Theory, every type $A$ constitutes a (weak) groupoid,[10] often called the *fundamental groupoid* of $A$:

- *Objects:* The terms $a, b\colon A$.
- *Morphisms:* The paths $p\colon a =_A b$.
- *Identity:* The reflexivity path $\mathsf{refl}_a$.

[9] In some cases the second unit law is relaxed to $c * d =_{\mathsf{Code}(x,z)} d$.
[10] See also Remark 2.4.2.

- *Composition:* Path concatenation $(p, q) \mapsto p \cdot q$.
- *Inverse:* Path inversion $p \mapsto p^{-1}$.

Note that in HoTT, the groupoid laws hold only *propositionally* (up to a path one level higher), not structurally (definitionally). This is why types are formally called $\infty$-*groupoids*.

**Theorem 2.6.5.** *Let the tuple* $(A, \mathsf{Code}, c, \mathsf{encode}, \mathsf{decode}, \mathsf{inv}, *)$ *be defined as in the* Extended Observational Equality Framework. *Then the* decode *function preserves the groupoid structure of identity types:*

*Reflexivity:* $\mathsf{decode}(x, x, c) =_{x =_A x} \mathsf{refl}_x$.

*Symmetry:* $\mathsf{decode}(y, x, \mathsf{inv}(d)) =_{y =_A x} \mathsf{decode}(x, y, d)^{-1}$.

*Transitivity:* $\mathsf{decode}(x, z, d * e) =_{x =_A z} \mathsf{decode}(x, y, d) \cdot \mathsf{decode}(y, z, e)$.

*Proof.*

*Reflexivity.* By the definition in the framework,

$$\mathsf{encode}(\mathsf{refl}_x) \equiv \mathsf{transport}^{C_x}(\mathsf{refl}_x, c) \equiv c.$$

Applying decode to both sides yields

$$\mathsf{refl}_x =_{x =_A y} \mathsf{decode}(x, x, c).$$

*Symmetry.* Consider the equality

$$\mathsf{encode}(p^{-1}) =_{\mathsf{Code}(x,y)} \mathsf{inv}(\mathsf{encode}(p)).$$

To prove it by path induction on $p$ it suffices to assume $y \equiv x$ and $p \equiv \mathsf{refl}_x$:

$$\text{LHS} \equiv \mathsf{encode}(\mathsf{refl}_x^{-1}) \equiv \mathsf{encode}(\mathsf{refl}_x) \equiv c.$$
$$\text{RHS} \equiv \mathsf{inv}(\mathsf{encode}(\mathsf{refl}_x)) \equiv \mathsf{inv}(c).$$

By the hypothesis $\mathsf{inv}(c) \equiv c$, the LHS and RHS are definitionally equal. Thus, encode preserves inverses. The result follows applying decode.

*Transitivity.* Consider

$$\mathsf{encode}(p \cdot q) =_{\mathsf{Code}(x,y)} \mathsf{encode}(p) * \mathsf{encode}(q).$$

By path induction on $p$, assume $y \equiv x$ and $p \equiv \mathsf{refl}_x$. Then

$$\text{LHS} \equiv \mathsf{encode}(\mathsf{refl}_x \cdot q) \equiv \mathsf{encode}(q).$$
$$\text{RHS} \equiv \mathsf{encode}(\mathsf{refl}_x) * \mathsf{encode}(q) \equiv c * \mathsf{encode}(q).$$

By the hypothesis $c * d \equiv d$, and so the RHS reduces to $\mathsf{encode}(q)$. Thus, encode preserves concatenation. Applying decode to both sides yields the conclusion. □

## 2.7 Equality in Cartesian Products

In this section we apply the observational framework 2.6.1 to the product $A \times B$.

Code family:

$$\begin{aligned} \mathsf{Code}\colon (A \times B) &\to (A \times B) \to \mathcal{U} \\ \mathsf{Code}(x, y) &:\equiv (\pi_1(x) =_A \pi_1(y)) \times (\pi_2(x) =_B \pi_2(y)). \end{aligned}$$

Base code:

$$c :\equiv (\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)})\colon \mathsf{Code}(x, x).$$

Equivalently,

$$c \equiv (\mathsf{ap}_{\pi_1}(\mathsf{refl}_x), \mathsf{ap}_{\pi_2}(\mathsf{refl}_x)).$$

Encode function:

For $C_x :\equiv \mathsf{Code}(x, -)$, the encode function becomes

$$\begin{aligned} \mathsf{encode}\colon (x =_{A\times B} y) &\to \mathsf{Code}(x, y) \\ \mathsf{encode}(p) &:\equiv \mathsf{transport}^{C_x}(p, c). \end{aligned}$$

In particular, for $p \equiv \mathsf{refl}_x$, we obtain

$$\begin{aligned} \mathsf{encode}(\mathsf{refl}_x) &\equiv \mathsf{transport}^{C_x}(\mathsf{refl}_x, c) \\ &\equiv c \\ &\equiv (\mathsf{ap}_{\pi_1}(\mathsf{refl}_x), \mathsf{ap}_{\pi_2}(\mathsf{refl}_x)), \end{aligned}$$

which implies, by induction on $p$, that

$$\mathsf{encode}(p) =_{\mathsf{Code}(x,y)} (\mathsf{ap}_{\pi_1}(p), \mathsf{ap}_{\pi_2}(p)).$$

We define the function

$$\begin{aligned} \mathsf{pair}^{\times}\colon \prod_{x,y\colon A\times B} (x =_{A\times B} y) &\to \mathsf{Code}(x, y) \\ (x, y, p) &\mapsto (\mathsf{ap}_{\pi_1}(p), \mathsf{ap}_{\pi_2}(p)) \\ (x, x, \mathsf{refl}_x) &\mapsto (\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)}), \end{aligned}$$

and adopt it as our implementation of $\mathsf{encode}$.

The following function *constructs* a path between pairs from paths between their components:

$$\mathsf{pair}^{=}\colon \mathsf{Code}(x, y) \to (x =_{A\times B} y).$$

It is defined as follows:

- By induction on $x$, it is enough to consider the case where $x$ is a pair $(a, b)$. In this case, we must define a function of type

$$\prod_{y:A\times B} \big((a =_A \pi_1(y)) \times (b =_B \pi_2(y))\big) \to \big((a, b) =_{A\times B} y\big).$$

- Now, by induction on $y$, we reduce to the case where $y$ is a pair $(a', b')$. That is, we must define a function of type

$$\big((a =_A a') \times (b =_B b')\big) \to \big((a, b) =_{A\times B} (a', b')\big).$$

- Since the domain is a product type, we can curry the function. This means we need to specify a dependent function of type

$$\prod_{p:\, a=_A a'} \prod_{q:\, b=_B b'} (a, b) =_{A\times B} (a', b').$$

- Path induction on $p$ allows us to reduce the definition to the case where $a' \equiv a$ and $p \equiv \mathsf{refl}_a$. Thus, we must define a dependent function of type

$$\prod_{q:\, b=_B b'} (a, b) =_{A\times B} (a, b').$$

- Finally, by path induction on $q$, this reduces to the case where $b' \equiv b$ and $q \equiv \mathsf{refl}_b$. We are left to define a single term of type

$$(a, b) =_{A\times B} (a, b).$$

- This is accomplished by choosing the reflexivity path

$$\mathsf{refl}_{(a,b)}.$$

**Lemma 2.7.1.** *The type*

$$\prod_{x,y:\, A\times B} \prod_{\omega:\, \mathsf{Code}(x,y)} P(x, y, \omega)$$

*is inhabited if, and only if, for all $a\colon A$ and $b\colon B$, the term*

$$P((a, b), (a, b), (\mathsf{refl}_a, \mathsf{refl}_b))$$

*has a witness.*

*Proof.* By induction on $x$, it suffices to consider the case where $x \equiv (a, b)$. The goal becomes

$$\prod_{y:\, A\times B} \prod_{\omega:\, (a=_A \pi_1(y))\times(b=_B \pi_2(y))} P((a, b), y, \omega).$$

Similarly, induction on $y$ reduces the goal to the case where $y \equiv (a', b')$

$$\prod_{\omega\colon (a =_A a') \times (b =_B b')} P((a,b),(a',b'),\omega).$$

The same reasoning allows us to decompose $\omega$ into components $(p, q)$ and find a section of:

$$\prod_{p\colon a =_A a'} \prod_{q\colon b =_B b'} P((a,b),(a',b'),(p,q)).$$

Path induction on $p$ contracts the endpoint $a'$ to $a$ and the path to $\mathsf{refl}_a$, yielding

$$\prod_{q\colon b =_B b'} P((a,b),(a,b'),(\mathsf{refl}_a, q)).$$

Finally, path induction on $q$ contracts $b'$ to $b$, leaving us with

$$P((a,b),(a,b),(\mathsf{refl}_a,\mathsf{refl}_b)),$$

as desired. □

**Theorem 2.7.2.** *Let $A, B\colon \mathcal{U}$ be two types. For any $x, y\colon A \times B$, the function $\mathsf{pair}^{\times}(x,y)$ is an equivalence with quasi-inverse $\mathsf{pair}^{=}(x,y)$. Consequently:*

$$(x =_{A\times B} y) \simeq \mathsf{Code}(x,y).$$

*Proof.* We must show that $\mathsf{pair}^{\times}$ and $\mathsf{pair}^{=}$ are quasi-inverses.

a) $\mathsf{pair}^{=}(x,y) \circ \mathsf{pair}^{\times}(x,y) \sim \mathsf{id}$.

We have to prove that

$$\prod_{x,y\colon A\times B} \prod_{p\colon x =_{A\times B} y} \big(\mathsf{pair}^{=}(x,y) \circ \mathsf{pair}^{\times}(x,y)\big)(p) =_{x =_{A\times B} y} p.$$

By path induction it suffices to consider the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$, which reduces the goal to

$$\big(\mathsf{pair}^{=}(x,x) \circ \mathsf{pair}^{\times}(x,x)\big)(\mathsf{refl}_x) =_{x =_{A\times B} x} \mathsf{refl}_x.$$

By the definitions of $\mathsf{pair}^{\times}$ and $\mathsf{pair}^{=}$, we have

$$\begin{aligned}\big(\mathsf{pair}^{=}(x,x) \circ \mathsf{pair}^{\times}(x,x)\big)(\mathsf{refl}_x) &\equiv \mathsf{pair}^{=}(x,x,\mathsf{pair}^{\times}(x,x,\mathsf{refl}_x))\\ &\equiv \mathsf{pair}^{=}(x,x,(\mathsf{refl}_{\pi_1(x)},\mathsf{refl}_{\pi_2(x)}))\\ &\equiv \mathsf{refl}_{(\pi_1(x),\pi_2(x))}.\end{aligned}$$

Let $\alpha\colon (\pi_1(x),\pi_2(x)) =_{A\times B} x$ be the path given by the uniqueness principle. The action on paths $\mathsf{ap}$ of the map $\mathsf{refl} :\equiv z \mapsto \mathsf{refl}_z$ applied to $\alpha$ yields:

$$\mathsf{ap}_{\mathsf{refl}}(\alpha)\colon \mathsf{refl}_{(\pi_1(x),\pi_2(x))} =_{x =_{A\times B} x} \mathsf{refl}_x.$$

b) $\mathsf{pair}^{\times}(x,y) \circ \mathsf{pair}^{=}(x,y) \sim \mathsf{id}$.

We have to prove that

$$\prod_{x,y:\, A\times B} \prod_{\omega:\,(\pi_1(x)=_A\pi_1(y))\times(\pi_2(x)=_B\pi_2(y))} \big(\mathsf{pair}^{\times}(x,y) \circ \mathsf{pair}^{=}(x,y)\big)(\omega) = \omega.$$

Applying Lemma 2.7.1, the goal reduces to

$$\begin{aligned}\big(\mathsf{pair}^{\times}((a,b),(a,b)) \circ \mathsf{pair}^{=}((a,b),(a,b))\big)(\mathsf{refl}_a,\mathsf{refl}_b)&\\ \equiv \mathsf{pair}^{\times}\big((a,b),(a,b),\mathsf{pair}^{=}((a,b),(a,b),(\mathsf{refl}_a,\mathsf{refl}_b))\big)&\\ \equiv \mathsf{pair}^{\times}((a,b),(a,b),\mathsf{refl}_{(a,b)})&\\ \equiv (\mathsf{refl}_a,\mathsf{refl}_b).&\end{aligned}$$

Thus, the output matches the input, as desired.

Since both composites are homotopic to the identity, $\mathsf{pair}^{=} : \mathsf{qinv}(\mathsf{pair}^{\times})$.

□

**Corollary 2.7.3.** *Given $x\colon A \times B$, we have*

$$x =_{A\times B} (\pi_1(x),\pi_2(x)).$$

*Proof.* Take $y :\equiv (\pi_1(x),\pi_2(x))\colon A \times B$. Then

$$\pi_1(y) \equiv \pi_1(x) \quad \text{and} \quad \pi_2(y) \equiv \pi_2(x).$$

Therefore,

$$(\mathsf{refl}_{\pi_1(x)},\mathsf{refl}_{\pi_2(x)})\colon (\pi_1(x) =_A \pi_1(y)) \times (\pi_2(x) =_B \pi_2(y)),$$

and the theorem implies that $\mathsf{pair}^{=}(\mathsf{refl}_{\pi_1(x)},\mathsf{refl}_{\pi_2(x)})\colon x =_{A\times B} y$.

□

**Extended framework**. To apply the extended framework 2.6.2 we set

$$\begin{aligned}\mathsf{inv}&\colon \mathsf{Code}(x,y) \to \mathsf{Code}(y,x)\\ \mathsf{inv}(p,q) &:\equiv (p^{-1},q^{-1})\end{aligned}$$

and

$$\begin{aligned}*&\colon \mathsf{Code}(x,y) \to \mathsf{Code}(y,z) \to \mathsf{Code}(x,z)\\ (p,q) * (p',q') &:\equiv (p \cdot p', q \cdot q'),\end{aligned}$$

which are clearly well defined. Moreover

$$\begin{aligned}\mathsf{inv}(c) &\equiv \mathsf{inv}(\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)}) \\ &\equiv (\mathsf{refl}^{-1}_{\pi_1(x)}, \mathsf{refl}^{-1}_{\pi_2(x)}) \\ &\equiv (\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)}) \\ &\equiv c\end{aligned}$$

and, for $d \equiv (p, q)$, we have

$$\begin{aligned}c * d &\equiv (\mathsf{refl}_{\pi_1(x)} \cdot p, \mathsf{refl}_{\pi_2(x)} \cdot q) \\ &\equiv (p, q).\end{aligned}$$

**Remark 2.7.4.** The following table is a summary of the functions introduced above:

| | **Function** | **Definition** |
|---|---|---|
| $\mathsf{pair}^{\times}$ | Encode | $(x =_{A\times B} y) \to \mathsf{Code}(x, y)$ |
| $c$ | Base code | $(\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)})$ |
| $\mathsf{pair}^{=}$ | Decode | $\mathsf{Code}(x, y) \to (x =_{A\times B} y)$ |
| $\mathsf{inv}$ | Inverse | $(p, q) \mapsto (p^{-1}, q^{-1})$ |
| $*$ | Concatenation | $(p, q) * (p', q') \equiv (p \cdot p', q \cdot q')$ |

**Proposition 2.7.5.** *Let $A, B : \mathcal{U}$. Then, for all $x, y, z : A \times B$, and paths*

$$\begin{aligned}p &: \pi_1(x) =_A \pi_1(y), \quad p' : \pi_1(y) =_A \pi_1(z), \\ q &: \pi_2(x) =_B \pi_2(y), \quad q' : \pi_2(y) =_B \pi_2(z),\end{aligned}$$

*we have*

a) $\mathsf{pair}^{=}(\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)}) \equiv \mathsf{refl}_x$

b) $\mathsf{pair}^{=}(p^{-1}, q^{-1}) =_{A\times B} \mathsf{pair}^{=}(p, q)^{-1}$

c) $\mathsf{pair}^{=}(p \cdot p', q \cdot q') =_{A\times B} \mathsf{pair}^{=}(p, q) \cdot \mathsf{pair}^{=}(p', q')$

*Proof.* This is a corollary of Theorem 2.6.5.[11] □

**Proposition 2.7.6.** *Let $A, B : \mathcal{U}$. Then, for all $x, y, z : A \times B$, the following equality types are inhabited:*

a) *For any $p : \pi_1(x) =_A \pi_1(y)$ and $q : \pi_2(x) =_B \pi_2(y)$,*

$$\mathsf{ap}_{\pi_1}(\mathsf{pair}^{=}(p, q)) =_A p$$

[11] Note also that parts b) and c) are easily derived from Lemma 2.7.1.

b) *For any* $p\colon \pi_1(x) =_A \pi_1(y)$ *and* $q\colon \pi_2(x) =_B \pi_2(y)$,

$$\mathsf{ap}_{\pi_2}(\mathsf{pair}^{=}(p,q)) =_B q$$

c) *For any* $r\colon x =_{A\times B} y$,

$$\mathsf{pair}^{=}(\mathsf{ap}_{\pi_1}(r), \mathsf{ap}_{\pi_2}(r)) =_{A\times B} r$$

d) *For any* $r\colon x =_{A\times B} y$,

$$\mathsf{pair}^{=}(\mathsf{ap}_{\pi_1}(r)^{-1}, \mathsf{ap}_{\pi_2}(r)^{-1}) =_{A\times B} r^{-1}$$

e) *For any* $r\colon x =_{A\times B} y$ *and* $s\colon y =_{A\times B} z$,

$$\mathsf{pair}^{=}(\mathsf{ap}_{\pi_1}(r) \cdot \mathsf{ap}_{\pi_1}(s), \mathsf{ap}_{\pi_2}(r) \cdot \mathsf{ap}_{\pi_2}(s)) =_{A\times B} r \cdot s$$

*Proof.*

a) We must show that $\mathsf{ap}_{\pi_1}(\mathsf{pair}^{=}(p,q)) =_A p$. By Lemma 2.7.1, it suffices to check the case where $x \equiv y \equiv (a,b)$, $p \equiv \mathsf{refl}_a$, and $q \equiv \mathsf{refl}_b$. The equality holds definitionally:

$$\begin{aligned}\mathsf{ap}_{\pi_1}(\mathsf{pair}^{=}(\mathsf{refl}_a, \mathsf{refl}_b)) &\equiv \mathsf{ap}_{\pi_1}(\mathsf{refl}_{(a,b)}) \\ &\equiv \mathsf{refl}_a.\end{aligned}$$

b) Analogous to i), replacing $\pi_1$ with $\pi_2$.

c) We proceed by path induction on $r$. It suffices to prove the case where $y \equiv x$ and $r \equiv \mathsf{refl}_x$:

$$\mathsf{pair}^{=}(\mathsf{ap}_{\pi_1}(\mathsf{refl}_x), \mathsf{ap}_{\pi_2}(\mathsf{refl}_x)) =_{x=_{A\times B}x} \mathsf{refl}_x.$$

This reduces to

$$\mathsf{pair}^{=}(\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)}) =_{x=_{A\times B}x} \mathsf{refl}_x,$$

whose inhabitation is granted by part a) of Proposition 2.7.5.

d) This follows from part b) of Proposition 2.7.5 by path induction on $r$.

e) We proceed by path induction on both $r$ and $s$. It suffices to consider the base case where $y \equiv x$ and $z \equiv x$, and both paths are the reflexivity path $\mathsf{refl}_x$. The goal reduces to proving

$$\mathsf{pair}^{=}(\mathsf{ap}_{\pi_1}(\mathsf{refl}_x) \cdot \mathsf{ap}_{\pi_1}(\mathsf{refl}_x), \mathsf{ap}_{\pi_2}(\mathsf{refl}_x) \cdot \mathsf{ap}_{\pi_2}(\mathsf{refl}_x)) =_{A\times B} \mathsf{refl}_x \cdot \mathsf{refl}_x.$$

By the definitional equalities for the action on paths and the concatenation of reflexivity paths, this simplifies to

$$\mathsf{pair}^{=}(\mathsf{refl}_{\pi_1(x)}, \mathsf{refl}_{\pi_2(x)}) =_{A\times B} \mathsf{refl}_x,$$

whose inhabitation is granted by part a) of Proposition 2.7.5.

□

**Notation 2.7.7.** *Given type families $A, B\colon Z \to \mathcal{U}$, we denote by $A \times B\colon Z \to \mathcal{U}$ the type family defined by $(A \times B)(z) :\equiv A(z) \times B(z)$.*

**Theorem 2.7.8.** *Fix $z\colon Z$ and $x\colon (A \times B)(z)$. Then, for $w\colon Z$ and $p\colon z =_Z w$, we have*

$$\mathsf{transport}^{A\times B}(p, x) =_{A(w)\times B(w)} (\mathsf{transport}^{A}(p, \pi_1(x)), \mathsf{transport}^{B}(p, \pi_2(x))).$$

*Proof.* By based path induction on $p$ it suffices to consider the case $w \equiv z$ and $p \equiv \mathsf{refl}_z$. According to 64. TRANSPORT, the goal becomes

$$x =_{A(z)\times B(z)} (\pi_1(x), \pi_2(x)).$$

Hence, the conclusion follows from Corollary 2.7.3.

□

**Definition 2.7.9.** Given functions $f\colon A \to A'$ and $g\colon B \to B'$, we define the *product* function

$$f \times g\colon A \times B \to A' \times B'$$

by its action on elements:

$$(f \times g)(x) :\equiv (f(\pi_1(x)), g(\pi_2(x))).$$

In particular, $f \times g(a, b) \equiv (f(a), g(b))$ for $a\colon A$ and $b\colon B$.

**Theorem 2.7.10.** *Let $f\colon A \to A'$ and $g\colon B \to B'$ be two functions. Given $x, y\colon A \times B$ and $\rho\colon (\pi_1(x) =_A \pi_1(y)) \times (\pi_2(x) =_B \pi_2(y))$, we have*

$$\mathsf{ap}_{f\times g}(\mathsf{pair}^{=}(\rho)) =_{A'\times B'} \mathsf{pair}^{=}(\mathsf{ap}_f(\pi_1(\rho)), \mathsf{ap}_g(\pi_2(\rho))).$$

*Proof.* By Lemma 2.7.1, it suffices to consider the case where $x \equiv y \equiv (a, b)$ and $\rho \equiv (\mathsf{refl}_a, \mathsf{refl}_b)$. In this case, the goal becomes

$$\mathsf{ap}_{f\times g}(\mathsf{pair}^{=}(\mathsf{refl}_a, \mathsf{refl}_b)) =_{A'\times B'} \mathsf{pair}^{=}(\mathsf{ap}_f(\mathsf{refl}_a), \mathsf{ap}_g(\mathsf{refl}_b)).$$

Computing the terms, we have:

- LHS: $\mathsf{ap}_{f\times g}(\mathsf{refl}_{(a,b)}) \equiv \mathsf{refl}_{(f(a),g(b))}$.
- RHS: $\mathsf{pair}^{=}(\mathsf{refl}_{f(a)}, \mathsf{refl}_{g(b)}) \equiv \mathsf{refl}_{(f(a),g(b))}$.

Thus, $\mathsf{refl}_{\mathsf{refl}_{(f(a),g(b))}}$ is a witness of the target identity type. □

## 2.8 Equality in the Boolean Type

To characterize the identity type of the booleans, we instantiate the Observational Equality Framework.

CODE FAMILY: We define the code family $\mathsf{Code}\colon 2 \to 2 \to \mathcal{U}$ by double induction on 2:

$$\begin{aligned} \mathsf{Code}(0_2, 0_2) &:\equiv 1 & \mathsf{Code}(0_2, 1_2) &:\equiv 0 \\ \mathsf{Code}(1_2, 0_2) &:\equiv 0 & \mathsf{Code}(1_2, 1_2) &:\equiv 1. \end{aligned}$$

BASE CODE: The base code $c_x\colon \mathsf{Code}(x, x)$ is defined by induction on $x\colon 2$ as[12]

$$c_{0_2} :\equiv \star \quad \text{and} \quad c_{1_2} :\equiv \star.$$

ENCODE FUNCTION:

$$\begin{aligned} \mathsf{encode}\colon \prod_{x,y\colon 2} (x =_2 y) &\to \mathsf{Code}(x, y) \\ (x, y, p) &\mapsto \mathsf{transport}^{C_x}(p, c_x), \end{aligned}$$

which is well-defined because $C_x(y) \equiv \mathsf{Code}(x, y)$.

DECODE FUNCTION: It proceeds by double induction on $x$ and $y$:

$$\begin{aligned} \mathsf{decode}\colon \prod_{x,y\colon 2} \mathsf{Code}(x, y) &\to (x =_2 y) \\ (0_2, 0_2, \star) &\mapsto \mathsf{refl}_{0_2} \\ (0_2, 1_2, \zeta) &\mapsto \mathsf{rec}_0(\zeta) \\ (1_2, 0_2, \zeta) &\mapsto \mathsf{rec}_0(\zeta) \\ (1_2, 1_2, \star) &\mapsto \mathsf{refl}_{1_2} \end{aligned}$$

**Theorem 2.8.1.** *For any $x, y\colon 2$, the functions $\mathsf{encode}(x, y)$ and $\mathsf{decode}(x, y)$ are quasi-inverses.*

*Proof.* Following Remark 2.6.1, we must verify that the compositions are pointwise homotopic to the respective identity functions.

First, let $p\colon x =_2 y$. By path induction on $p$, it suffices to check the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. By definition of $\mathsf{encode}$, we have

$$\mathsf{encode}(x, x, \mathsf{refl}_x) \equiv c_x.$$

[12] Since $\mathsf{Code}(x, x)$ is not definitionally equal to $\mathbf{1}$, and $c_x$ must be of type $\mathsf{Code}(x, x)$, we cannot define it as the constant map $\lambda x.\star$.

By induction on $x$:

- If $x \equiv 0_2$, then $c_{0_2} \equiv \star$, and $\mathsf{decode}(0_2, 0_2, \star) \equiv \mathsf{refl}_{0_2}$.
- If $x \equiv 1_2$, then $c_{1_2} \equiv \star$, and $\mathsf{decode}(1_2, 1_2, \star) \equiv \mathsf{refl}_{1_2}$.

Thus, $\mathsf{decode}(x, x, \mathsf{encode}(x, x, \mathsf{refl}_x)) = \mathsf{refl}_x$.

Second, let $d\colon \mathsf{Code}(x, y)$. We proceed by double induction on $x$ and $y$:

- If $x, y \equiv 0_2$, then $d\colon \mathsf{1}$ and induction on $d$ yields $\mathsf{decode}(0_2, 0_2, \star) \equiv \mathsf{refl}_{0_2}$. Thus,

$$\mathsf{encode}(0_2, 0_2, \mathsf{decode}(0_2, 0_2, \star)) \equiv \mathsf{encode}(0_2, 0_2, \mathsf{refl}_{0_2}) \equiv c_{0_2} \equiv \star.$$

- If $x, y \equiv 1_2$, then $d\colon \mathsf{1}$ and induction on $d$ yields $\mathsf{decode}(1_2, 1_2, \star) \equiv \mathsf{refl}_{1_2}$. Thus,

$$\mathsf{encode}(1_2, 1_2, \mathsf{decode}(1_2, 1_2, \star)) \equiv \mathsf{encode}(1_2, 1_2, \mathsf{refl}_{1_2}) \equiv c_{1_2} \equiv \star.$$

- If $x \equiv 1_2$ and $y \equiv 0_2$, or the other way around, then $d\colon \mathsf{0}$, and the goal is vacuously satisfied.

□

**Extended framework**. We define:

Inversion map:

$$\mathsf{inv}\colon \prod_{x,y\colon 2} \mathsf{Code}(x, y) \to \mathsf{Code}(y, x)$$

| $x$ | $y$ | $\zeta$ | $\mathsf{inv}(x, y, \zeta)$ |
|---|---|---|---|
| $0_2$ | $0_2$ | $\zeta$ | $\zeta$ |
| $1_2$ | $1_2$ | $\zeta$ | $\zeta$ |
| $0_2$ | $1_2$ | $\zeta$ | $\mathsf{rec}_{\mathsf{0}}(\zeta)$ |
| $1_2$ | $0_2$ | $\zeta$ | $\mathsf{rec}_{\mathsf{0}}(\zeta)$ |

Concatenation:

$$*\colon \prod_{x,y,z\colon 2} \mathsf{Code}(x, y) \to \mathsf{Code}(y, z) \to \mathsf{Code}(x, z)$$

$$u \mapsto v \mapsto u * v$$

| $x$ | $y$ | $u$ | $u * v$ |
|---|---|---|---|
| $0_2$ | $0_2$ | $\mathsf{1}$ | $v$ |
| $1_2$ | $1_2$ | $\mathsf{1}$ | $v$ |
| $0_2$ | $1_2$ | $\mathsf{0}$ | $\mathsf{rec}_{\mathsf{0}}(u)$ |
| $1_2$ | $0_2$ | $\mathsf{0}$ | $\mathsf{rec}_{\mathsf{0}}(u)$ |

Unit laws:

To show that $\mathsf{inv}(x, x, c_x) =_{\mathsf{Code}(x,x)} c_x$ it suffices to consider the cases $x \equiv 0_2$ and $x \equiv 1_2$. In both cases $c_x \equiv \star$, and $\mathsf{inv}(x, x, \star) \equiv \star$ by definition.

To show that $c_x * v =_{\mathsf{Code}(x,z)} v$, by induction on $x$, we must only verify the cases $x \equiv 0_2$ and $x \equiv 1_2$, where the concatenation satisfies $c_x * v \equiv v$ by definition.

## 2.9 Equality in Σ-Types

Let $A\colon \mathcal{U}$ be a type and $P\colon A \to \mathcal{U}$ a type family. Given a path $p\colon \omega =_E \omega'$ between points $\omega$ and $\omega'$ in the total space $E :\equiv \sum_{x:A} P(x)$, the behavior of the first and second projections is asymmetric. While $\pi_1$ yields a path $\mathsf{ap}_{\pi_1}(p)\colon \pi_1(\omega) =_A \pi_1(\omega')$, the second components $\pi_2(\omega)$ and $\pi_2(\omega')$ cannot be compared because they inhabit the fibers $P(\pi_1(\omega))$ and $P(\pi_1(\omega'))$, which are not necessarily the same type.

**Definition 2.9.1.** Let $P\colon A \to \mathcal{U}$ be a type family and let $E :\equiv \sum_{x:A} P(x)$ be its total space. For any $x\colon A$, the *fiber inclusion* is defined as

$$\begin{aligned} \mathsf{in}_x &\colon P(x) \to E \\ \mathsf{in}_x(u) &:\equiv \langle x, u \rangle. \end{aligned}$$

**Remark 2.9.2.** The fiber inclusion embeds the fiber $P(x)$ into the total space. Consequently, given a path $\gamma\colon u =_{P(x)} v$ in the fiber, the functorial action $\mathsf{ap}_{\mathsf{in}_x}$ yields a path $\mathsf{ap}_{\mathsf{in}_x}(\gamma)\colon \langle x, u \rangle =_E \langle x, v \rangle$ in the total space.

---

Applying the observational framework 2.6.1 to the total space $E$ we define the

Code family:

$$\begin{aligned} \mathsf{Code} &\colon E \to E \to \mathcal{U} \\ \mathsf{Code}(\omega, \omega') &:\equiv \sum_{q:\, \pi_1(\omega) =_A \pi_1(\omega')} \mathsf{transport}^P(q, \pi_2(\omega)) =_{P(\pi_1(\omega'))} \pi_2(\omega'). \end{aligned}$$

Base code:

$$c :\equiv \langle \mathsf{refl}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)} \rangle\colon \mathsf{Code}(\omega, \omega).$$

Another way to express the first component of $c$ is by observing that

$$\mathsf{refl}_{\pi_1(\omega)} \equiv \mathsf{ap}_{\pi_1}(\omega).$$

For the second component of $c$, since $\pi_2$ is a dependent function,[13] we have to use the dependent application map $\mathsf{apd}$.[14] Thus, $\mathsf{refl}_{\pi_2(\omega)} \equiv \mathsf{apd}_{\pi_2}(\mathsf{refl}_\omega)$ and $c$ can be written as

$$c \equiv \langle \mathsf{ap}_{\pi_1}(\mathsf{refl}_\omega), \mathsf{apd}_{\pi_2}(\mathsf{refl}_\omega)\rangle.$$

ENCODE FUNCTION: The transport family $C_\omega :\equiv \mathsf{Code}(\omega, -)$ yields

$$\begin{aligned}\mathsf{encode}&\colon (\omega =_E \omega') \to \mathsf{Code}(\omega, \omega')\\ \mathsf{encode}(p) &:\equiv \mathsf{transport}^{C_\omega}(p, c).\end{aligned}$$

In particular, for $\omega' \equiv \omega$ and $p \equiv \mathsf{refl}_\omega$, we obtain

$$\begin{aligned}\mathsf{encode}(\mathsf{refl}_\omega) &\equiv \mathsf{transport}^{C_\omega}(\mathsf{refl}_\omega, c)\\ &\equiv c\\ &\equiv \langle \mathsf{ap}_{\pi_1}(\mathsf{refl}_\omega), \mathsf{apd}_{\pi_2}(\mathsf{refl}_\omega)\rangle.\end{aligned}$$

Hence, path induction on $p$ implies that

$$\mathsf{encode}(p) =_{\mathsf{Code}(\omega,\omega')} \langle \mathsf{ap}_{\pi_1}(p), \mathsf{apd}_{\pi_2}(p)\rangle.$$

Therefore, we take the RHS to define the encode function

$$\begin{aligned}f&\colon \prod_{\omega,\omega'\colon E} (\omega =_E \omega') \to \mathsf{Code}(\omega, \omega')\\ f(\omega, \omega', p) &:\equiv \langle \mathsf{ap}_{\pi_1}(p), \mathsf{apd}_{\pi_2}(p)\rangle,\end{aligned}$$

which satisfies:

$$f(\mathsf{refl}_\omega) \equiv \langle \mathsf{refl}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)}\rangle.$$

DECODE FUNCTION:

We have to define a backward function

$$g\colon \mathsf{Code}(\omega, \omega') \to (\omega =_E \omega').$$

Let $u :\equiv \pi_2(\omega)$ and $v :\equiv \pi_2(\omega')$. The domain is the type of pairs $\langle q, \gamma\rangle$ where $q\colon \pi_1(\omega) =_A \pi_1(\omega')$ and $\gamma\colon q_*(u) =_{P(\pi_1(\omega'))} v$.

By induction on $\omega$ and $\omega'$, it suffices to consider the case where $\omega \equiv \langle x, u\rangle$ and $\omega' \equiv \langle y, v\rangle$. The domain is then the type of pairs $\langle q, \gamma\rangle$ where $q\colon x =_A y$ and $\gamma\colon q_*(u) =_{P(y)} v$.

We proceed in two steps:

1. By the Path Lifting Theorem 2.3.1, we have

$$\alpha :\equiv \mathsf{lift}(u, q)\colon \langle x, u\rangle =_E \langle y, q_*(u)\rangle.$$

[13] See 36. INDUCTION IN Σ-TYPES, equation (1.1).
[14] See Lemma 2.3.6.

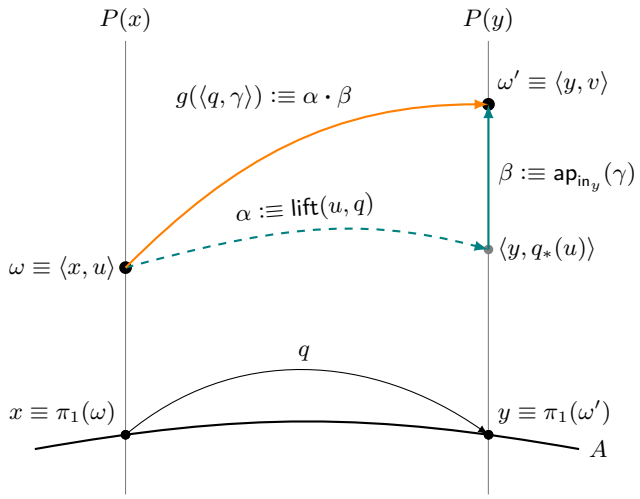
2. We need to connect $\langle y, q_*(u)\rangle$ to $\langle y, v\rangle$. Since these two points share the same first component, we can use the fiber inclusion function [cf. Remark 2.9.2] to obtain a vertical path:

$$\beta :\equiv \mathsf{ap}_{\mathsf{in}_y}(\gamma)\colon \langle y, q_*(u)\rangle =_E \langle y, v\rangle.$$

We can now define $g$ by concatenating both paths:

$$g(\langle q, \gamma\rangle) :\equiv \alpha \cdot \beta.$$

Here is a picture illustrating the construction




**Remark 2.9.3.** By Theorem 2.3.1 and Proposition 2.1.1, respectively, we know that $\mathsf{lift}(\langle u, \mathsf{refl}_x\rangle) \equiv \mathsf{refl}_{\langle x,u\rangle}$ and $\mathsf{ap}_{\mathsf{in}_x}(\mathsf{refl}_u) \equiv \mathsf{refl}_{\langle x,u\rangle}$. Therefore,

$$g(\langle \mathsf{refl}_x, \mathsf{refl}_u\rangle) \equiv \mathsf{refl}_{\langle x,u\rangle}. \tag{2.1}$$

**Note.** What follows is a generalization of Lemma 2.7.1 to $\Sigma$-types.

**Lemma 2.9.4.** *Let $E :\equiv \sum_{x\colon A} P(x)$. For any type family $Q(x, \omega', \varepsilon)$, the type*

$$\prod_{\omega,\omega'\colon E} \prod_{\varepsilon\colon \mathsf{Code}(\omega,\omega')} Q(\omega, \omega', \varepsilon)$$

*is inhabited if, and only if, for all $x\colon A$ and $u\colon P(x)$, the term*

$$Q(\langle x, u\rangle, \langle x, u\rangle, \langle \mathsf{refl}_x, \mathsf{refl}_u\rangle)$$

*has a witness.*

*Proof.* By induction on $\omega$, it suffices to consider the case $\omega \equiv \langle x, u\rangle$ with $x\colon A$ and $u\colon P(x)$. The goal becomes

$$\prod_{\omega'\colon E} \prod_{\varepsilon\colon \mathsf{Code}((\langle x,u\rangle),\omega')} Q(\langle x, u\rangle, \omega', \varepsilon).$$

Similarly, induction on $\omega'$ reduces the goal to the case $\omega' \equiv \langle y, v\rangle$ with $y\colon A$ and $v\colon P(y)$, which yields

$$\prod_{\varepsilon\colon \mathsf{Code}(\langle x,u\rangle,\langle y,v\rangle)} Q(\langle x, u\rangle, \langle y, v\rangle, \varepsilon).$$

Since

$$\mathsf{Code}(\langle x, u\rangle, \langle y, v\rangle) :\equiv \sum_{p\colon x =_A y} \mathsf{transport}^P(p, u) =_{P(y)} v,$$

we can replace $\varepsilon$ with $\langle p, q\rangle$, where $p\colon x =_A y$ and $q\colon \mathsf{transport}^P(p, u) =_{P(y)} v$. The goal becomes

$$Q(\langle x, u\rangle, \langle y, v\rangle, \langle p, q\rangle).$$

Now, path induction on $p$ reduces this to $a' \equiv x$ and $p \equiv \mathsf{refl}_x$. Thus, the type of $q$ reduces to $u =_{P(x)} v$. Finally, path induction on $q$ takes us to the conclusion. □

**Theorem 2.9.5.** *Let $A\colon \mathcal{U}$ be a type and $P\colon A \to \mathcal{U}$ a type family. Denote the total space $\sum_{x\colon A} P(x)$ by $E$. Then, for any $\omega, \omega'\colon E$, there is an equivalence*

$$(\omega =_E \omega') \simeq \mathsf{Code}(\omega, \omega').$$

*Proof.* We have to verify two compositions:

1. $f \circ g \sim \mathsf{id}$.

   We have to show that, given $\omega, \omega'\colon E$

   $$\prod_{\varepsilon\colon \mathsf{Code}(\omega,\omega')} f(g(\varepsilon)) =_{\mathsf{Code}(\omega,\omega')} \varepsilon.$$

   By Lemma 2.9.4 it suffices to prove that in the case $\omega \equiv \omega' \equiv \langle x, u\rangle$ and $\varepsilon \equiv \langle \mathsf{refl}_x, \mathsf{refl}_u\rangle$, it is

   $$f(g(\langle \mathsf{refl}_x, \mathsf{refl}_u\rangle)) \equiv \langle \mathsf{refl}_x, \mathsf{refl}_u\rangle.$$

   But

   $$\begin{aligned} f(g(\langle \mathsf{refl}_x, \mathsf{refl}_u\rangle)) &\equiv f(\mathsf{refl}_{\langle x,u\rangle}) && ;\ (2.1)\\ &\equiv \langle \mathsf{refl}_x, \mathsf{refl}_u\rangle && ;\ \text{Def. of } f. \end{aligned}$$

2. $g \circ f \sim \mathsf{id}$.

   We have to prove that for $\rho\colon \omega, \omega'\colon E$, we have

   $$g(f(\rho)) =_{\omega =_E \omega'} \rho.$$

   By based path induction on $\rho$ we can set $\omega' \equiv \omega$ and $\rho \equiv \mathsf{refl}_\omega$. Then

   $$\begin{aligned} g(f(\mathsf{refl}_\omega)) &\equiv g(\langle \mathsf{refl}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)} \rangle) && ;\ \text{Def. of } f \\ &\equiv \mathsf{refl}_{\langle \pi_1(\omega), \pi_2(\omega) \rangle} && ;\ (2.1) \\ &=_{\omega =_E \omega'} \mathsf{refl}_\omega && ;\ \mathsf{ap}_{\mathsf{refl}}. \end{aligned}$$

□

**Corollary 2.9.6.** *With the preceding notation, given $\omega\colon E$, we have*

$$\omega =_E \langle \pi_1(\omega), \pi_2(\omega) \rangle.$$

*Proof.* Applying the theorem with $\omega\colon E$ and $\omega' :\equiv \langle \pi_1(\omega), \pi_2(\omega) \rangle$, we obtain a map (the backward direction of the equivalence):

$$\sum_{p\colon \pi_1(\omega) =_A \pi_1(\omega)} p_*(\pi_2(\omega)) =_{P(\pi_1(\omega))} \pi_2(\omega) \to (\omega =_E \langle \pi_1(\omega), \pi_2(\omega) \rangle).$$

We select the pair $\langle \mathsf{refl}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)} \rangle$ in the domain. Note that the second component is well-typed because $(\mathsf{refl}_{\pi_1(\omega)})_*$ is the identity function. Evaluating the map at this pair yields the desired witness in the codomain. □

**Proposition 2.9.7.** *Let $A, B\colon \mathcal{U}$.*

a) *If $P\colon A \to \mathcal{U}$ is the constant family $P(x) \equiv B$, then for any $b\colon B$ and $p\colon a =_A a'$, we have*

$$\mathsf{lift}(b, p) =_{(a,b) =_{A \times B} (a', b)} \mathsf{pair}^{=}(p, \mathsf{refl}_b).$$

b) *For any $x\colon A$ and $\gamma\colon u =_B v$, the fiber inclusion $\mathsf{in}_x\colon B \to A \times B$ satisfies*

$$\mathsf{ap}_{\mathsf{in}_x}(\gamma) =_{(x,u) =_{A \times B} (x,v)} \mathsf{pair}^{=}(\mathsf{refl}_x, \gamma).$$

*Proof.* We proceed by path induction in each case.

a) By induction on $p$ it suffices to assume $a' \equiv a$ and $p \equiv \mathsf{refl}_a$.

   - LHS: $\mathsf{lift}(b, \mathsf{refl}_a) \equiv \mathsf{refl}_{(a,b)}$.
   - RHS: $\mathsf{pair}^{=}(\mathsf{refl}_a, \mathsf{refl}_b) \equiv \mathsf{refl}_{(a,b)}$.

   Thus, the equality is witnessed by $\mathsf{refl}_{\mathsf{refl}_{(a,b)}}$.

b) By induction on $\gamma$ it suffices to assume $v \equiv u$ and $\gamma \equiv \mathsf{refl}_u$.

- LHS: $\mathsf{ap}_{\mathsf{in}_x}(\mathsf{refl}_u) \equiv \mathsf{refl}_{\mathsf{in}_x(u)} \equiv \mathsf{refl}_{(x,u)}$.
- RHS: $\mathsf{pair}^{=}(\mathsf{refl}_x, \mathsf{refl}_u) \equiv \mathsf{refl}_{(x,u)}$.

Thus, the equality is witnessed by $\mathsf{refl}_{\mathsf{refl}_{(x,u)}}$. □

**Remark 2.9.8.** If the type family $P\colon A \to \mathcal{U}$ is constant, say $P(x) \equiv B$ for all $x\colon A$, then the total space $E$ becomes $A \times B$ and, by Lemma 2.3.7, we have $p_*(b) =_B b$ for $b\colon B$. Thus, the backward function of the equivalence reduces to

$$g\colon \sum_{q\colon \pi_1(\omega)=_A \pi_1(\omega')} \pi_2(\omega) =_B \pi_2(\omega') \to (\omega =_{A\times B} \omega'),$$

i.e.,

$$g\colon \prod_{\omega,\omega'\colon A\times B} (\pi_1(\omega) =_A \pi_1(\omega')) \times (\pi_2(\omega) =_B \pi_2(\omega')) \to (\omega =_{A\times B} \omega'),$$

which has the same signature as $\mathsf{pair}^{=}$. In fact, given $\omega, \omega'\colon A \times B$, we have

$$\begin{aligned}
g(q,\gamma) &\equiv \mathsf{lift}(\pi_2(\omega), q) \cdot \mathsf{ap}_{\mathsf{in}_{\pi_1(\omega')}}(\gamma) && \text{; Def. of } g\\
&=_{A\times B} \mathsf{pair}^{=}(q, \mathsf{refl}_{\pi_2(\omega)}) \cdot \mathsf{pair}^{=}(\mathsf{refl}_{\pi_1(\omega')}, \gamma) && \text{; Prop. 2.9.7}\\
&=_{A\times B} \mathsf{pair}^{=}(q \cdot \mathsf{refl}_{\pi_1(\omega')}, \mathsf{refl}_{\pi_2(\omega)} \cdot \gamma) && \text{; Prop. 2.7.5 c)}\\
&=_{A\times B} \mathsf{pair}^{=}(q, \gamma) && \text{; Unit laws.}
\end{aligned}$$

In consequence, for the general case, we introduce the following

**Definition 2.9.9.** The *dependent pair equality constructor*, denoted by $\mathsf{pair}^{=}$, is the backward function of the equivalence established in Theorem 2.9.5. It constructs a path in the total space $E :\equiv \sum_{x\colon A} P(x)$ from a path in the base $A$ and a path in the fiber. Its signature is:

$$\mathsf{pair}^{=}\colon \prod_{\omega,\omega'\colon E} \left( \sum_{p\colon \pi_1(\omega)=_A \pi_1(\omega')} p_*(\pi_2(\omega)) =_{P(\pi_1(\omega'))} \pi_2(\omega') \right) \to (\omega =_E \omega').$$

Moreover,

$$\mathsf{pair}^{=}(\langle p, \gamma\rangle) \equiv \mathsf{lift}(\pi_2(\omega), p) \cdot \mathsf{ap}_{\mathsf{in}_{\pi_1(\omega')}}(\gamma).$$

**Remark 2.9.10.** By definition, when $\omega' \equiv \omega \equiv \langle x, u\rangle$ and $p \equiv \mathsf{refl}_x$, we have

$$\begin{aligned}
\mathsf{pair}^{=}(\langle \mathsf{refl}_x, \gamma\rangle) &\equiv \mathsf{lift}(u, \mathsf{refl}_x) \cdot \mathsf{ap}_{\mathsf{in}_x}(\gamma)\\
&\equiv \mathsf{refl}_{\langle x,u\rangle} \cdot \mathsf{ap}_{\mathsf{in}_x}(\gamma)\\
&\equiv \mathsf{ap}_{\mathsf{in}_x}(\gamma).
\end{aligned}$$

In particular, for $\gamma \equiv \mathsf{refl}_u\colon u =_{P(x)} u$,

$$\mathsf{pair}^{=}(\langle \mathsf{refl}_x, \mathsf{refl}_u\rangle) \equiv \mathsf{refl}_{\langle x,u\rangle}.$$

**Lemma 2.9.11.** *Let $A$ be a type and $B\colon A \to \mathcal{U}$ a type family. Suppose $w_1, w_2\colon \sum_{x:A} B(x)$ with $w_1 \equiv \langle x_1, y_1\rangle$ and $w_2 \equiv \langle x_2, y_2\rangle$. Then, for any paths $p\colon x_1 =_A x_2$ and $q\colon p_*(y_1) =_{B(x_2)} y_2$, we have*

$$\mathsf{ap}_{\pi_1}(\mathsf{pair}^{=}(p,q)) =_{x_1 =_A x_2} p.$$

*Proof.* By path induction on $p$ we may assume that $x_2 \equiv x_1$ and $p \equiv \mathsf{refl}_{x_1}$. In this case, the transport $p_*(y_1)$ evaluates to $y_1$, and therefore $q$ is a path of type $y_1 =_{B(x_1)} y_2$. We can now proceed by path induction on $q$, which allows us to assume $y_2 \equiv y_1$ and $q \equiv \mathsf{refl}_{y_1}$.

Thus, the term $\mathsf{pair}^{=}(p,q)$ reduces definitionally to $\mathsf{refl}_{(x_1,y_1)}$. Applying the function $\pi_1$ to this path yields $\mathsf{ap}_{\pi_1}(\mathsf{refl}_{\langle x_1,y_1\rangle})$, which is definitionally equal to $\mathsf{refl}_{x_1}$. Since $p \equiv \mathsf{refl}_{x_1}$, both sides of the equation are definitionally equal to $\mathsf{refl}_{x_1}$. Thus, the identity path $\mathsf{refl}_{\mathsf{refl}_{x_1}}$ witnesses the required propositional equality. □

**Proposition 2.9.12.** [Σ-commutativity] *Let $B, C\colon \mathcal{U}$. Given a type family*

$$D\colon B \to C \to \mathcal{U},$$

*the map*

$$\begin{aligned}\chi\colon \Big(\sum_{b:B}\sum_{c:C} D(b,c)\Big) &\to \Big(\sum_{c:C}\sum_{b:B} D(b,c)\Big)\\ \langle b, \langle c, d\rangle\rangle &\mapsto \langle c, \langle b, d\rangle\rangle\end{aligned}$$

*induces an equivalence.*

*Proof.* The map is well-defined because, in both expression the requirement is $d\colon D(b,c)$ for $b\colon B$ and $c\colon C$. The verification of the invertibility of this function is straightforward.

□

**Proposition 2.9.13.** [Σ-associativity] *Let $X\colon \mathcal{U}$ be a type, $A\colon X \to \mathcal{U}$ a type family over $X$, and $B\colon \Sigma_X A \to \mathcal{U}$ a type family over the total space. Then, the map*

$$\begin{aligned}f\colon \sum_{x:X}\sum_{u:A(x)} B(\langle x,u\rangle) &\to \sum_{\omega:\Sigma_X A} B(\omega)\\ \langle x, \langle u, z\rangle\rangle &\mapsto \langle\langle x,u\rangle, z\rangle\end{aligned}$$

*induces an equivalence.*

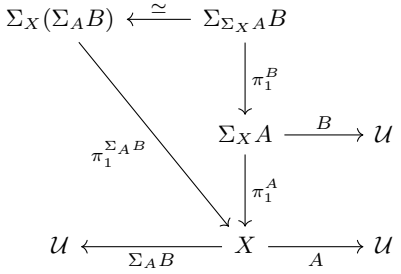
*Proof.* Define the backward map

$$g\colon \Big(\sum_{\omega\colon \Sigma_X A} B(\omega)\Big) \to \sum_{x\colon X}\ \sum_{u\colon A(x)} B(\langle x,u\rangle)$$
$$\langle\langle x,u\rangle,z\rangle \mapsto \langle x,\langle u,z\rangle\rangle.$$

Verifying that $f$ and $g$ are quasi-inverses is straightforward. □

**Theorem 2.9.14.** *Let $A\colon X \to \mathcal{U}$ be a type family with total space $\Sigma_X A$. Let $B\colon \Sigma_X A \to \mathcal{U}$ be a second type family. Define the composite type family $\Sigma_A B\colon X \to \mathcal{U}$ as $x \mapsto \sum_{u\colon A(x)} B(\langle x,u\rangle)$. If $\Sigma_{\Sigma_X A} B$ is the total space of $B$, then*

a) *By the associativity of Σ-types, $\Sigma_{\Sigma_X A} B$ is equivalent to the total space of the composite family, forming a tower of fibrations:*




b) *For any path $p\colon x =_X y$ and any $\langle u,z\rangle\colon \Sigma_A B(x)$, we have*

$$\mathsf{transport}^{\Sigma_A B}(p,\langle u,z\rangle) =_{\Sigma_A B(y)} \langle p_*(u), \mathsf{transport}^B(\mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle, z))\rangle,$$

*where $p_*(u) \equiv \mathsf{transport}^A(p)(u)$. Hence, if*

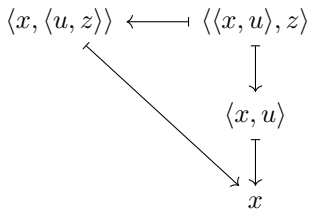
$$\sigma_p(\langle u,z\rangle) :\equiv \langle p_*(u), \mathsf{transport}^B(\mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle, z)),\rangle$$

*the following diagram commutes propositionally:*

$$\begin{array}{ccc}
\Sigma_A B(x) & \xrightarrow{\mathsf{transport}^{\Sigma_A B}(p)} & \Sigma_A B(y) \\
{\scriptstyle\mathrm{def}}\,\Big\| & & \Big\|\,{\scriptstyle\mathrm{def}} \\
\displaystyle\sum_{u\colon A(x)} B(x,u) & \xrightarrow[\sigma_p]{} & \displaystyle\sum_{v\colon A(y)} B(y,v)
\end{array}$$

*Proof.* The commutativity of the triangle follows easily from Proposition 2.9.13

and induction on the $\Sigma$-types:




since, by definition,

$$\Sigma_X(\Sigma_A B) \equiv \sum_{x:\, X} \sum_{u:\, A(x)} B(\langle x, u\rangle)$$

The proof of b) has two parts:

TYPE VERIFICATION. Let $\omega :\equiv \langle x, u\rangle : \Sigma_X A$ and $\omega' :\equiv \langle y, p_*(u)\rangle : \Sigma_X A$. Then, according to Definition 2.9.9, we have:

- $p : \pi_1(\omega) =_X \pi_1(\omega')$,
- $\mathsf{refl}_{p_*(u)} : p_*(u) =_{A(\pi_1(\omega'))} p_*(u)$,
- $\langle p, \mathsf{refl}_{p_*(u)}\rangle : \mathrm{dom}(\mathsf{pair}^{=})$,
- $\mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle) : \omega =_{\Sigma_X A} \omega'$,
- $\mathsf{transport}^{B}(\mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle)) : B(\omega) \to B(\omega')$.

Thus, we obtain

$$\text{LHS} :\equiv \underbrace{\underbrace{\mathsf{transport}^{\Sigma_A B}(p)}_{\Sigma_A B(x)\to\Sigma_A B(y)}\ \underbrace{(\langle u, z\rangle)}_{\Sigma_A B(x)}}_{\Sigma_A B(y)}$$

$$\text{RHS} :\equiv \big\langle \underbrace{p_*(u)}_{A(y)},\ \underbrace{\mathsf{transport}^{B}\big(\underbrace{\mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle)}_{\text{path in } \Sigma_X A}\big)\big(\ \underset{\sqcup}{z}\ \big)}_{B(\langle x,u\rangle)}\big\rangle,}_{B(\langle y, p_*(u)\rangle)}$$

which shows that both sides are elements of $\Sigma_A B(y)$. In the case of the RHS, this is because the first component lies in $A(y)$ and the second in $B(\langle y, v\rangle)$ with $v :\equiv p_*(u) : A(y)$, as required.

INHABITATION. Since the statement holds for every path $p : x =_X y$, by path induction, to prove that there is a witness of

$$\text{LHS} =_{\Sigma_A B(y)} \text{RHS}$$

it suffices to consider the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. The goal reduces as follows:

$$\begin{aligned}
\textsc{lhs} &\equiv \mathsf{transport}^{\Sigma_A B}(\mathsf{refl}_x)(\langle u, z\rangle) \\
&\equiv \langle u, z\rangle.
\end{aligned}$$

$$\begin{aligned}
\textsc{rhs} &\equiv \big\langle (\mathsf{refl}_x)_*(u), \mathsf{transport}^B(\mathsf{pair}^=(\langle \mathsf{refl}_x, \mathsf{refl}_{(\mathsf{refl}_x)_*(u)}\rangle))(z)\big\rangle \\
&\equiv \big\langle u, \mathsf{transport}^B(\mathsf{pair}^=(\langle \mathsf{refl}_x, \mathsf{refl}_u\rangle))(z)\big\rangle \\
&\equiv \big(u, \mathsf{transport}^B(\mathsf{refl}_{\langle x,u\rangle})(z)\big) \\
&\equiv \langle u, z\rangle.
\end{aligned}$$

Thus, $\textsc{lhs} \equiv \textsc{rhs}$ definitionally. □

**Corollary 2.9.15.** *Let $W$ and $X$ be types, and let $A\colon W \to \mathcal{U}$ be the constant type family defined by $A(w) :\equiv X$. Let $B\colon \Sigma_W A \to \mathcal{U}$ be a second type family. Define the composite type family $\Sigma_A B\colon W \to \mathcal{U}$ by $w \mapsto \sum_{x:X} B(\langle w, x\rangle)$.*

*For any path $p\colon w_1 =_W w_2$ and any element $\langle x, z\rangle\colon \Sigma_A B(w_1)$, we have*

$$\mathsf{transport}^{\Sigma_A B}(p)(\langle x, z\rangle) = \big\langle x, \mathsf{transport}^{w\mapsto B(\langle w,x\rangle)}(p)(z)\big\rangle.$$

*The following diagram commutes propositionally:*

$$\begin{array}{ccc}
\Sigma_A B(w_1) & \xrightarrow{\mathsf{transport}^{\Sigma_A B}(p)} & \Sigma_A B(w_2) \\
{\scriptstyle\mathrm{def}}\,\Big\| & & \Big\|\,{\scriptstyle\mathrm{def}} \\
\sum_{x:X} B(\langle w_1, x\rangle) & \xrightarrow[\sigma_p]{} & \sum_{x:X} B(\langle w_2, x\rangle)
\end{array}$$

*where the bottom map is defined component-wise as*

$$\sigma_p(\langle x, z\rangle) :\equiv \big\langle x, \mathsf{transport}^{w\mapsto B(\langle w,x\rangle)}(p)(z)\big\rangle.$$

*Proof.* We apply part b) of the theorem. Since $A$ is the constant family, by Exercise 1.16.3, $p_*(x) \equiv \mathsf{transport}^A(p)(x) =_X x$. Substituting this into the RHS of the theorem's formula, we obtain

$$\big\langle x, \mathsf{transport}^B\big(\mathsf{pair}^=(\langle p, \mathsf{refl}_x\rangle)\big)(z)\big\rangle.$$

The term $\mathsf{pair}^=(\langle p, \mathsf{refl}_x\rangle)$ represents a path in the total space $\Sigma_W A$ from $\langle w_1, x\rangle$ to $\langle w_2, x\rangle$. Transporting $z$ along this path in the family $B$ is definitionally equivalent to transporting $z$ along $p$ in the partially applied family $w \mapsto B(\langle w, x\rangle)$. This yields the desired equality. □

**Remark 2.9.16.** For $\omega, \omega'\colon E$, a canonical pair $\langle p, u\rangle\colon \mathsf{Code}(\omega, \omega')$ is given by

$$p\colon \pi_1(\omega) =_A \pi_1(\omega'), \quad u\colon p_*(\pi_2(\omega)) =_{P(\pi_1(\omega'))} \pi_2(\omega')$$

**Extended framework.** To apply the extended framework 2.6.2 to the $\Sigma$-type, we define

$$\begin{aligned} \mathsf{inv}\colon \mathsf{Code}(\omega,\omega') &\to \mathsf{Code}(\omega',\omega) \\ \mathsf{inv}(\langle p,\gamma\rangle) &:\equiv \big\langle p^{-1}, (p^{-1})_*(\gamma^{-1}) \cdot \mathsf{transportinv}_l^P(p,\pi_2(\omega))\big\rangle. \qquad (2.2)\\ *\colon \mathsf{Code}(\omega,\omega') &\to \mathsf{Code}(\omega',\omega'') \to \mathsf{Code}(\omega,\omega'') \\ \langle p,\gamma\rangle * \langle p',\gamma'\rangle &:\equiv \langle p\cdot p', p'_*(\gamma)\cdot\gamma'\rangle. \end{aligned}$$

To verify the unit laws, observe that

$$\begin{aligned} \mathsf{inv}(c) &\equiv \mathsf{inv}(\langle \mathsf{refl}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)}\rangle) \\ &\equiv \langle \mathsf{refl}^{-1}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)}\rangle && ;\text{ Lem. 2.3.11} \\ &\equiv c, \\ c * \langle p,\gamma\rangle &\equiv \langle \mathsf{refl}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)}\rangle * \langle p,\gamma\rangle \\ &\equiv \langle \mathsf{refl}_{\pi_1(\omega)}\cdot p, \mathsf{refl}_{\pi_2(\omega)}\cdot\gamma\rangle \\ &\equiv \langle p,\gamma\rangle. \end{aligned}$$

**Proposition 2.9.17.** *Given elements $\omega,\omega',\omega'' : E$ and paths*

$$\begin{aligned} p &: \pi_1(\omega) =_A \pi_1(\omega'), \quad \gamma : p_*(\pi_2(\omega)) =_{P(\pi_1(\omega'))} \pi_2(\omega'), \\ p' &: \pi_1(\omega') =_A \pi_2(\omega''), \quad \gamma' : p'_*(\pi_2(\omega')) =_{P(\pi_1(\omega''))} \pi_2(\omega''), \end{aligned}$$

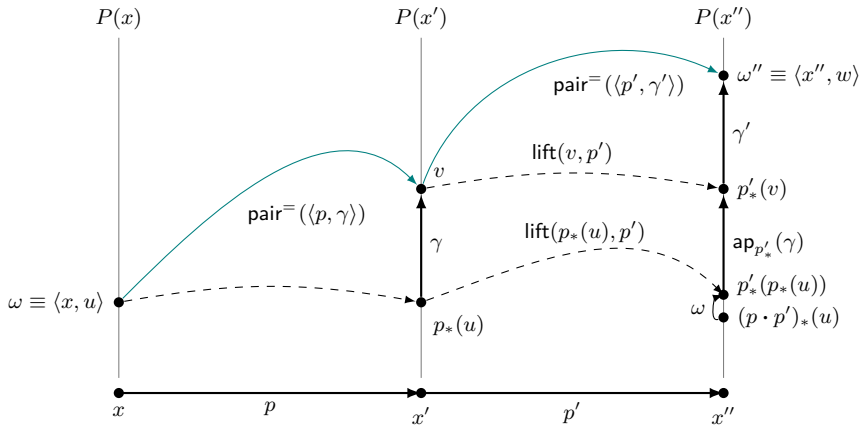
*we have*

a) $\mathsf{pair}^=(\langle \mathsf{refl}_{\pi_1(\omega)}, \mathsf{refl}_{\pi_2(\omega)}\rangle) \equiv \mathsf{refl}_\omega$

b) $\mathsf{pair}^=(\langle p^{-1}, (p^{-1})_*(\gamma^{-1}) \cdot \mathsf{transportinv}_l^P(p,\pi_2(\omega))\rangle) =_E \mathsf{pair}^=(\langle p,\gamma\rangle)^{-1}$

c) $\mathsf{pair}^=(\langle p\cdot p', \gamma\cdot\gamma'\rangle) =_E \mathsf{pair}^=(\langle p,\gamma\rangle)\cdot\mathsf{pair}^=(\langle p',\gamma'\rangle)$

*Proof.* This is a direct consequence of Theorem 2.6.5.[15]




[15] Note also that parts b) and c) are easily derived from Lemma 2.9.4.

□

**Corollary 2.9.18.** *Let $A : \mathcal{U}$ be a type and $B : A \to \mathcal{U}$ a type family. Given an element $\langle x, u\rangle : \Sigma_A B$ and a path $p : x =_A y$, we have*

$$\mathsf{pair}^{=}(\langle p^{-1}, \gamma\rangle)^{-1} =_{\Sigma_A B} \mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle),$$

*where $\gamma : (p^{-1})_*(p_*(u)) =_{B(x)} u$ is the canonical path that cancels the transport along inverse paths.*

*Proof.* The equation is well-typed because

$$\begin{aligned} p^{-1} &: y =_A x, \\ \mathsf{pair}^{=}(\langle p^{-1}, \gamma\rangle) &: \langle y, p_*(u)\rangle =_{\Sigma_A B} \langle x, u\rangle, \\ \text{LHS} &: \langle x, u\rangle =_{\Sigma_A B} \langle y, p_*(u)\rangle, \\ \text{RHS} &: \langle x, u\rangle =_{\Sigma_A B} \langle y, p_*(u)\rangle. \end{aligned}$$

To verify the equality, we proceed by path induction on $p$. Thus, we can assume that $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case, $\gamma$ becomes $\mathsf{refl}_u$, and both sides compute definitionally to $\mathsf{refl}_{(x,u)}$.

□

**Note.** What follows is a generalization of Theorem 2.7.10 to Σ-types. For simplicity, the statement is formulated for canonical dependent pairs; its extension to arbitrary terms is straightforward.

**Theorem 2.9.19.** *Let $A, A' : \mathcal{U}$ be types, and let $B : A \to \mathcal{U}$ and $B' : A' \to \mathcal{U}$ be type families. Given $g : A \to A'$ and $h : \prod_{a:A} B(a) \to B'(g(a))$, define*

$$\begin{aligned} f : \Big(\sum_{x:A} B(x)\Big) &\to \Big(\sum_{x':A'} B'(x')\Big) \\ \langle a, b\rangle &\mapsto \langle g(a), h(a, b)\rangle. \end{aligned}$$

*Then, for $\langle x, u\rangle, \langle y, v\rangle : \Sigma_A B$, $p : x =_A y$, and $\gamma : p_*(u) =_{B(y)} v$, we have*

$$\mathsf{ap}_f(\mathsf{pair}^{=}(\langle p, \gamma\rangle)) =_{f(x,u) =_{\Sigma_{A'} B'} f(y,v)} \mathsf{pair}^{=}(\langle \mathsf{ap}_g(p), \alpha \cdot \mathsf{ap}_{h(y,-)}(\gamma)\rangle),$$

*where*

$$\alpha : \mathsf{ap}_g(p)_*(h(\langle x, u\rangle)) =_{B'(g(y))} h(\langle y, p_*(u)\rangle)$$

*is the path given by* Theorem 2.3.16.

*Proof.* Let us first verify that the equality is well-typed:

$$\begin{aligned}
\mathsf{pair}^{=}(\langle p,\gamma\rangle)&\colon \langle x,u\rangle =_{\Sigma_A B} \langle y,v\rangle.\\
\text{LHS}&\colon \langle g(x),h(\langle x,u\rangle)\rangle =_{\Sigma_{A'}B'} \langle g(y),h(\langle y,v\rangle)\rangle.\\
&\equiv f(\langle x,u\rangle) =_{\Sigma_{A'}B'} f(\langle y,v\rangle).\\
\mathsf{ap}_g(p)&\colon g(x) =_{A'} g(y).\\
\mathsf{ap}_{h(\langle y,-\rangle)}(\gamma)&\colon h(\langle y,p_*(u)\rangle) =_{B'(g(y))} h(\langle y,v\rangle).\\
\alpha \cdot \mathsf{ap}_{h(\langle y,-\rangle)}(\gamma)&\colon \mathsf{ap}_g(p)_*(h(\langle x,u\rangle)) =_{B'(g(y))} h(\langle y,v\rangle).\\
\text{RHS}&\colon \langle g(x),h(\langle x,u\rangle)\rangle =_{\Sigma_{A'}B'} \langle g(y),h(\langle y,v\rangle)\rangle\\
&\equiv f(\langle x,u\rangle) =_{\Sigma_{A'}B'} f(\langle y,v\rangle).
\end{aligned}$$

Consider the path in $f(\langle x,u\rangle) =_{\Sigma_{A'}B'} f(\langle y,v\rangle)$

$$Q(\langle x,u\rangle,\langle y,v\rangle,\langle p,\gamma\rangle) :\equiv \mathsf{ap}_f(\mathsf{pair}^{=}(\langle p,\gamma\rangle)) = \mathsf{pair}^{=}(\langle \mathsf{ap}_g(p), \alpha \cdot \mathsf{ap}_{h(\langle y,-\rangle)}(\gamma)\rangle).$$

By Lemma 2.9.4 applied to $Q$, it suffices to exhibit an inhabitant of

$$Q(\langle x,u\rangle,\langle x,u\rangle,\langle \mathsf{refl}_x,\mathsf{refl}_u\rangle).$$

But this is trivial because both sides of the equation reduce to

$$\begin{aligned}
\text{LHS} &\equiv f(\langle x,u\rangle).\\
\alpha &\equiv \mathsf{refl}_{h(\langle x,u\rangle)}.\\
\mathsf{ap}_{h(\langle x,-\rangle)}(\mathsf{refl}_u) &\equiv \mathsf{refl}_{h(\langle x,u\rangle)}.\\
\text{RHS} &\equiv \mathsf{pair}^{=}(\langle \mathsf{refl}_{g(x)}, \mathsf{refl}_{h(x,u)}\rangle)\\
&\equiv \mathsf{refl}_{\langle g(x),h(\langle x,u\rangle)\rangle}\\
&\equiv f(\langle x,u\rangle),
\end{aligned}$$

which is witnessed by $\mathsf{refl}_{f(\langle x,u\rangle)}$. □

## 2.10 Equality in the Unit Type

To apply the framework 2.6.1 to the unit type $\mathsf{1}$, we proceed as in the previous cases.

Code family:

$$\begin{aligned}
\mathsf{Code}&\colon \mathsf{1} \to \mathsf{1} \to \mathcal{U}\\
\mathsf{Code}(x,y) &:\equiv \mathsf{1}.
\end{aligned}$$

Base code: For $c\colon \mathsf{Code}(x,x)$ we specify $c :\equiv \star$.

ENCODE FUNCTION:

$$\mathsf{encode}\colon (x =_1 y) \to \mathsf{Code}(x, y)$$
$$\mathsf{encode}(p) :\equiv \mathsf{transport}^{C_x}(p, c)$$

In particular, for $y \equiv x$ and $p \equiv \mathsf{refl}_x$, we have

$$\mathsf{encode}(\mathsf{refl}_x) \equiv c \equiv \star.$$

Therefore, by induction on $p$, we deduce that

$$\mathsf{encode}(p) =_1 \star$$

and we take the RHS to define the constant encode function $f :\equiv \lambda x, y.\, \lambda p.\, \star$.

DECODE FUNCTION: Let $g$ denote the backward $\mathsf{decode}$ map. Then

$$g\colon \prod_{x,y\colon 1} \mathsf{Code}(x, y) \to (x =_1 y).$$

To define it we proceed by induction on $x$. It suffices to construct $g(\star)$. The goal becomes a term of type

$$\prod_{y\colon 1} 1 \to (\star =_1 y).$$

Induction on $y$ further reduces the goal to $1 \to (\star =_1 \star)$, which is inhabited by the constant function $z \mapsto \mathsf{refl}_\star$. In particular,

$$\mathsf{decode}(\star) \equiv \mathsf{refl}_\star. \tag{2.1}$$

Thus, we have defined $g\colon \prod_{x,y\colon 1} 1 \to (x =_1 y)$ as the unique function that satisfies $g(\star, \star, \star) \equiv \mathsf{refl}_\star$.

**Theorem 2.10.1.** *For any $x, y\colon 1$, we have $(x =_1 y) \simeq 1$.*

*Proof.* To verify the homotopies, we need to prove

a) $f \circ g \sim \mathsf{id}_1$.

This translates into $f(g(x, y, u)) =_1 u$ for all $x, y, u\colon 1$. By the induction principle of 1, we may set $x, y, u \equiv \star$, which satisfies the goal because $f$ maps every element to $\star$.

b) $g \circ f \sim \mathsf{id}_{\prod_{x,y\colon 1}(x =_1 y)}$.

We must show $g(x, y, f(x, y, p)) =_{x=_1 y} p$. By path induction, it suffices to check the case $y \equiv x$ and $p \equiv \mathsf{refl}_x$. The goal becomes

$$g(x, x, f(x, x, \mathsf{refl}_x)) =_{x=_1 x} \mathsf{refl}_x.$$

By induction on $x$, it is enough to prove $g(\star, \star, f(\star, \star, \mathsf{refl}_\star)) =_{\star=_1 \star} \mathsf{refl}_\star$. Since $f(\star, \star, \mathsf{refl}_\star) \equiv \star$, the goal reduces to $g(\star, \star, \star) =_{\star=_1 \star} \mathsf{refl}_\star$, which holds by the definition of $g$. □

The theorem has the following consequences:

*Introduction rule.* We define the canonical element inhabiting the identity type of the unit for any two points

$$\mathsf{triv} : x =_1 y$$

as $\mathsf{triv} :\equiv \mathsf{decode}(x, y, \star)$.

*Elimination rule.* The dependent eliminator asserts that to define a function out of the identity type, we only need to specify its value on the canonical element.

$$\mathsf{ind}_{=_1} : \prod_{C : (x =_1 y) \to \mathcal{U}} C(\mathsf{triv}) \to \prod_{p : x =_1 y} C(p)$$

*Computation rule.* When the eliminator is applied to the canonical element $\mathsf{triv}$, it reduces propositionally[16] to the base case $c : C(\mathsf{triv})$.

$$\mathsf{ind}_{=_1}(C, c, \mathsf{triv}) =_{C(\mathsf{triv})} c.$$

*Uniqueness rule.* This rule asserts that the type is contractible (a singleton); any inhabitant $p$ is path-equal to the canonical element.

$$\mathsf{uniq}_{=_1} : \prod_{p : x =_1 y} p =_{x =_1 y} \mathsf{triv}.$$

To derive the elimination rule, we use Theorem 2.10.1, which gives us a homotopy:

$$\varepsilon_p : \mathsf{decode}(x, y, \mathsf{encode}(x, y, p)) =_1 p$$

for every path $p : x =_1 y$. Since, by definition, $\mathsf{encode}(x, y, p) \equiv \star$, the homotopy simplifies:

$$\mathsf{decode}(x, y, \mathsf{encode}(x, y, p)) \equiv \mathsf{decode}(x, y, \star) \equiv \mathsf{triv},$$

and so

$$\varepsilon_p : \mathsf{triv} =_{x =_1 y} p.$$

Now, given $C : (x =_1 y) \to \mathcal{U}$ and $c : C(\mathsf{triv})$, we want to construct $f(p) : C(p)$.

We define

$$\mathsf{ind}_{=_1}(C, c, p) :\equiv \mathsf{transport}^C(\varepsilon_p, c).$$

In the case $p \equiv \mathsf{triv}$, we have:

$$\varepsilon_{\mathsf{triv}} =_{x =_1 y} \mathsf{refl}_{\mathsf{triv}}. \tag{2.2}$$

Therefore:

$$\mathsf{ind}_{=_1}(C, c, \mathsf{triv}) =_{C(\mathsf{triv})} \mathsf{transport}^C(\mathsf{refl}_{\mathsf{triv}}, c) \equiv c. \tag{2.3}$$

The Uniqueness rule is derived by taking the symmetry (inverse) of $\varepsilon_p$:

$$\mathsf{uniq}_{=_1}(p) :\equiv \varepsilon_p^{-1}.$$

[16] This is weaker than usual. The equality at the base case is definitional if we accept the $\eta$-rule for the identity type [cf. The $\eta$-rule for the Identity of 1].

### The $\eta$-rule for the Identity of 1

The $\eta$-rule asserts that every inhabitant of the identity type of the unit is definitionally equal to reflexivity, namely

$$p \equiv \mathsf{refl}_\star \qquad (\text{for every } p\colon x =_1 y).$$

Since the $\eta$-rule for the Identity of $\mathsf{1}$ is structurally impossible to state (as a general rule for arbitrary $x, y$) without first assuming the $\eta$-rule for $\mathsf{1}$, this requires the $\eta$-rule for the unit type.

**Extended framework.** For the unit type identity, we have

$$\begin{aligned} \mathsf{inv}&\colon \mathsf{Code}(x,y) \to \mathsf{Code}(y,x) \\ \mathsf{inv}(u) &:\equiv u \end{aligned}$$

and

$$\begin{aligned} *&\colon \mathsf{Code}(x,y) \to \mathsf{Code}(y,z) \to \mathsf{Code}(x,z) \\ u * v &:\equiv v. \end{aligned}$$

The unit laws follow directly from the definitions above, namely

$$\begin{aligned} \mathsf{inv}(c) &\equiv c \\ c * u &\equiv u. \end{aligned}$$

**Proposition 2.10.2.** *Given $x, y, z\colon \mathsf{1}$, and $p\colon x =_1 y$ and $q\colon y =_1 z$, we have*

a) $\mathsf{decode}(x, x, \star) =_{x=_1 x} \mathsf{refl}_x$

b) $\mathsf{decode}(y, x, u) =_{y=_1 x} \mathsf{decode}(x, y, u)^{-1}$

c) $\mathsf{decode}(x, z, u * v) =_{x=_1 z} \mathsf{decode}(x, y, u) \cdot \mathsf{decode}(y, z, v)$

*Proof.* This is a direct consequence of Theorem 2.6.5.

□

## 2.11 Function Extensionality Axiom

In this section we address the question of the identity type for dependent (and non-dependent) functions. To establish the context, let $A\colon \mathcal{U}$ be a type and $B\colon A \to \mathcal{U}$ a type family. To analyze the identity in the section space[17]

$$\Pi_A B :\equiv \prod_{x\colon A} B(x)$$

[17] Consisting of all dependent functions $s\colon A \to E$, where $E$ is the total space $\sum_{x\colon A} B(x)$, such that $\pi_1(s(x)) =_A x$ for all $x\colon A$.

we apply the framework 2.6.1.

CODE FAMILY:

$$\begin{aligned} \mathsf{Code}\colon \Pi_A B \to \Pi_A B \to \mathcal{U} \\ \mathsf{Code}(f,g) :\equiv f \sim g \end{aligned}$$

i.e.,

$$\mathsf{Code}(f,g) :\equiv \prod_{x\colon A} f(x) =_{B(x)} g(x).$$

BASE CODE: For the reflexive case, we define the term $c\colon \mathsf{Code}(f,f)$ pointwise by reflexivity:

$$c :\equiv \lambda x\colon A.\, \mathsf{refl}_{f(x)}. \tag{2.1}$$

ENCODE FUNCTION:

$$\begin{aligned} \mathsf{encode}\colon (f =_{\Pi_A B} g) \to \mathsf{Code}(f,g) \\ \mathsf{encode}(p) :\equiv \mathsf{transport}^{C_f}(p,c). \end{aligned}$$

Since $C_f :\equiv \mathsf{Code}(f,-)$, we introduce the local family at a point $x\colon A$

$$C_{f,x} :\equiv \lambda h\colon \Pi_A B.\, f(x) =_{B(x)} h(x).$$

In particular, in the case where $g \equiv f$ and $p \equiv \mathsf{refl}_f$, we have

$$\begin{aligned} \mathsf{encode}(\mathsf{refl}_f)(x) &\equiv \mathsf{transport}^{C_f}(\mathsf{refl}_f, c)(x) \\ &\equiv c(x) \\ &\equiv \mathsf{refl}_{f(x)} \\ &\equiv \mathsf{transport}^{C_{f,x}}(\mathsf{refl}_f, \mathsf{refl}_{f(x)}). \end{aligned}$$

Hence, path induction implies

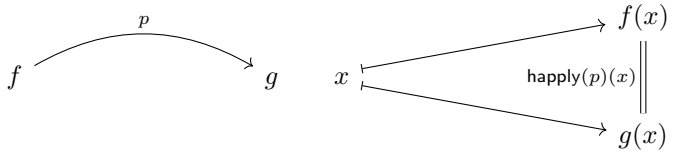
$$\mathsf{encode}(p)(x) =_{C_{f,x}(g)} \mathsf{transport}^{C_{f,x}}(p, \mathsf{refl}_{f(x)}),$$

for $p\colon f =_{\Pi_A B} g$ and $x\colon A$. We take the RHS to define the standard name for the encode function *homotopy application*:

$$\begin{aligned} \mathsf{happly}\colon (f =_{\Pi_A B} g) \to \mathsf{Code}(f,g) \\ \mathsf{happly}(p) :\equiv \lambda x\colon A.\, \mathsf{transport}^{C_{f,x}}(p, \mathsf{refl}_{f(x)}), \end{aligned} \tag{2.2}$$

which can be illustrated by

$$\Pi_A B \equiv \prod_{x:\,A} B(x) \qquad A \qquad B(x)$$




Path in function space Function homotopy

Because of the base code we defined in (2.1), we have

$$\mathsf{happly}(\mathsf{refl}_f) \equiv \lambda x.\, \mathsf{refl}_{f(x)}. \tag{2.3}$$

Since the basic rules of type theory do not provide a way to construct a quasi-inverse for happly, we introduce the Function Extensionality axiom to formally establish the equivalence between function equality and pointwise homotopy.

**Axiom.** [Function Extensionality]. The homotopy application function (2.2) is an equivalence.[18]

The axiom asserts that happly has a quasi-inverse

$$\mathsf{funext}\colon \mathsf{Code}(f,g) \to (f =_{\Pi_A B}\ g), \tag{2.4}$$

which, by definition, satisfies the following homotopies for any path $p\colon f =_{\Pi_A B}\ g$ and any pointwise homotopy $\eta\colon \mathsf{Code}(f,g)$:

$$\begin{aligned}\mathsf{funext}(\mathsf{happly}(p)) &=_{f=_{\Pi_A B} g}\ p\\ \mathsf{happly}(\mathsf{funext}(\eta)) &=_{\mathsf{Code}(f,g)}\ \eta.\end{aligned}$$

**Extended framework.** To apply the framework 2.6.2, we define[19]

$$\begin{aligned}\mathsf{inv}\colon \mathsf{Code}(f,g) &\to \mathsf{Code}(g,f)\\ \mathsf{inv}(\eta) &:\equiv \eta^{-1}\end{aligned}$$

and

$$\begin{aligned}*\colon \mathsf{Code}(f,g) &\to \mathsf{Code}(g,h) \to \mathsf{Code}(f,h)\\ \eta * \vartheta &:\equiv \eta \cdot \vartheta.\end{aligned}$$

[18] We will see in later chapters that this axiom follows both from univalence and from the interval type.

[19] See Definitions 2.4.6.

The unit laws follow immediately from the definitional properties of path inversion and concatenation applied pointwise:

$$\begin{aligned}\mathsf{inv}(c) &\equiv \lambda x.\, \mathsf{refl}_{f(x)}^{-1} \\ &\equiv \lambda x.\, \mathsf{refl}_{f(x)} \\ &\equiv c,\end{aligned}$$

and

$$\begin{aligned}c * \eta &\equiv (\lambda x.\, c(x)) \cdot \eta(x) \\ &\equiv \lambda x.\, \mathsf{refl}_{f(x)} \cdot \eta(x) \\ &\equiv \lambda x.\, \eta(x) \\ &\equiv \eta.\end{aligned}$$

**Notation 2.11.1.** *From now on, we will use the superscript* FE *to indicate that a result depends on the function extensionality axiom.*

**Proposition 2.11.2.**[FE] *Let $A$, $B$, and $\Pi_A B$ be as above. If $f, g \colon \Pi_A B$, then*

a) $\mathsf{funext}(f, f, \lambda x.\, \mathsf{refl}_{f(x)}) =_{f =_{\Pi_A B} f} \mathsf{refl}_f$

b) *For $\eta \colon f \sim g$, we have*

$$\mathsf{funext}(g, f, \eta^{-1}) =_{g =_{\Pi_A B} f} \mathsf{funext}(f, g, \eta)^{-1}$$

c) *Given $h \colon \Pi_A B$ and $\vartheta \colon g \sim h$, we have*

$$\mathsf{funext}(f, h, \eta \cdot \vartheta) =_{f =_{\Pi_A B} h} \mathsf{funext}(f, g, \eta) \cdot \mathsf{funext}(g, h, \vartheta)$$

*Proof.* This is an immediate consequence of Theorem 2.6.5. □

## Composition of Quasi-inverses

Recall from Definition 2.5.10 that quasi-inverses can be composed.

**Proposition 2.11.3.**[FE] *Let $f \colon A \to B$. Suppose that $\langle f^{-1}, (\eta, \vartheta)\rangle \colon \mathsf{qinv}(f)$. Then $\langle f^{-1}, (\eta, \vartheta \cdot \mathsf{refl}_{\mathsf{id}_A})\rangle \colon \mathsf{qinv}(f)$ and both quasi-inverses are (propositionally) equal.*

*Proof.* We construct the path in $\mathsf{qinv}(f)$ by pairs.

1. The path between the functions is $\mathsf{refl}_{f^{-1}}$.
2. The path between the homotopy pairs is given component-wise:
   - For the right homotopies: $\eta \equiv \eta$, so we use $\mathsf{refl}_\eta$.

- For the left homotopies: we need to show that

$$\vartheta \cdot \mathsf{refl}_{\mathsf{id}_A} =_{f^{-1} \circ f \sim \mathsf{id}_A} \vartheta.$$

  By function extensionality, this follows from the right unit law for paths.[20] Therefore,

$$\mathsf{ru}_\vartheta :\equiv \mathsf{funext}(\lambda x.\, \mathsf{ru}(\vartheta(x))).$$

Thus, the total identity path is

$$\mathsf{pair}^{=}(\langle \mathsf{refl}_{f^{-1}}, \mathsf{pair}^{=}(\mathsf{refl}_\eta, \mathsf{ru}_\vartheta)\rangle) \colon \langle f^{-1}, (\eta, \vartheta \cdot \mathsf{refl}_{\mathsf{id}_A})\rangle =_{\mathsf{qinv}(f)} \langle f^{-1}, (\eta, \vartheta)\rangle.$$

□

**Corollary 2.11.4.**[FE] *In the particular case where one of the functions is the identity, we have*

a) $\langle \mathsf{id}_A, (\mathsf{refl}_{\mathsf{id}_A}, \mathsf{refl}_{\mathsf{id}_A})\rangle \circ \langle f^{-1}, (\eta_f, \vartheta_f)\rangle =_{\mathsf{qinv}(f)} (f^{-1}, (\eta_f, \vartheta_f))$

b) $\langle f^{-1}, (\eta_f, \vartheta_f)\rangle \circ \langle \mathsf{id}_A, (\mathsf{refl}_{\mathsf{id}_A}, \mathsf{refl}_{\mathsf{id}_A})\rangle =_{\mathsf{qinv}(f)} \langle f^{-1}, (\eta_f, \vartheta_f)\rangle$

## Non-dependent Functions

The special case where $B \colon A \to \mathcal{U}$ is constant refers to the non-dependent function type: $F \equiv A \to B$. Here the general case computes as follows. For $f, g \colon A \to B$ and $p \colon f =_{A\to B} g$, we have

$$C_{f,x} :\equiv \lambda g.\, f(x) =_B g(x)$$
$$\mathsf{happly}(p) :\equiv \lambda x.\, \mathsf{transport}^{C_{f,x}}(p, \mathsf{refl}_{f(x)}).$$

Furthermore, evaluating $\mathsf{happly}(p)$ at a specific point $x \colon A$ is propositionally equal to the action on paths of the evaluation functional $\mathsf{ev}_x \colon (A \to B) \to B$, where $\mathsf{ev}_x(h) :\equiv h(x)$. That is, we have a path

$$\mathsf{happly}(p)(x) =_B \mathsf{ap}_{\mathsf{ev}_x}(p), \tag{2.5}$$

as it can be easily verified by path induction on $p$.

Function extensionality provides the quasi-inverse to $\mathsf{happly}$, mapping pointwise homotopies back to paths between functions:

$$\mathsf{funext} \colon (f \sim g) \to (f =_{A\to B} g).$$

[20] Lemma 1.11.4.

## Transport Decomposition

**Non-dependent case.** Let $X : \mathcal{U}$ be a type and $A, B : X \to \mathcal{U}$ be type families. Let $A \to B : X \to \mathcal{U}$ denote (by abuse of notation) the type family

$$(A \to B)(x) :\equiv A(x) \to B(x).$$

Given a path $p : x =_X y$ and a function $f : (A \to B)(x)$, we have the following diagram:

$$\begin{array}{ccc} A(x) & \xrightarrow{\quad f \quad} & B(x) \\ {\scriptstyle \mathsf{transport}^A(p)}\downarrow & & \downarrow{\scriptstyle \mathsf{transport}^B(p)} \\ A(y) & \xrightarrow[\mathsf{transport}^{A\to B}(p,f)]{} & B(y) \end{array} \tag{2.6}$$

**Proposition 2.11.5.** *The diagram* (2.6) *propositionally commutes in the function type* $A(y) \to B(y)$*. In particular,*

$$\mathsf{transport}^{A\to B}(p, f) =_{A(y)\to B(y)} \mathsf{transport}^B(p) \circ f \circ \mathsf{transport}^A(p^{-1}).$$

*Proof.* First, observe that the diagram is well-typed because

$$\mathsf{transport}^{A\to B}(p, f) : (A \to B)(y) \equiv A(y) \to B(y).$$

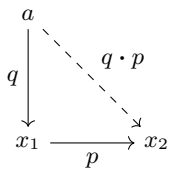
To prove the commutativity, by path induction, it suffices to consider the case $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case, we obtain the square

$$\begin{array}{ccc} A(x) & \xrightarrow{\quad f \quad} & B(x) \\ {\scriptstyle \mathsf{transport}^A(\mathsf{refl}_x)}\downarrow & & \downarrow{\scriptstyle \mathsf{transport}^B(\mathsf{refl}_x)} \\ A(x) & \xrightarrow[\mathsf{transport}^{A\to B}(\mathsf{refl}_x,f)]{} & B(x) \end{array}$$

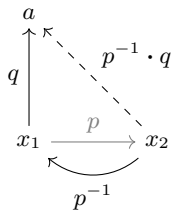
which trivially (and definitionally) commutes because all transports of reflexivity reduce to identity functions. □

**Proposition 2.11.6.** *Let* $A : \mathcal{U}$ *be a type and* $a : A$ *a term. Given* $p : x_1 =_A x_2$*, we have*

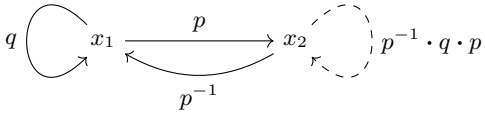
a) $\mathsf{transport}^{x\mapsto(a=_A x)}(p, q) = q \cdot p$*, for* $q : a =_A x_1$



b) $\mathsf{transport}^{x \mapsto (x =_A a)}(p, q) = p^{-1} \cdot q$, *for* $q \colon x_1 =_A a$




c) $\mathsf{transport}^{x \mapsto (x =_A x)}(p, q) = p^{-1} \cdot q \cdot p$, *for* $q \colon x_1 =_A x_1$.




*Proof.* For type verification, observe the following

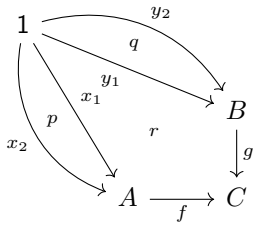
$$\begin{array}{lll} \text{LHS}(1) \colon a =_A x_2. & \text{LHS}(2) \colon x_2 =_A a. & \text{LHS}(3) \colon x_2 =_A x_2. \\ \text{RHS}(1) \colon a =_A x_2. & \text{RHS}(2) \colon x_2 =_A a. & \text{RHS}(3) \colon x_2 =_A x_2. \end{array}$$

By induction on $p$ it suffices to consider the case $x_2 \equiv x_1$ and $p \equiv \mathsf{refl}_{x_1}$. In that case

$$\begin{aligned} &\mathsf{transport}^{x \mapsto (a =_A x)}(\mathsf{refl}_{x_1}, q) \equiv q \equiv \mathsf{refl}_{x_1} \cdot q. \\ &\mathsf{transport}^{x \mapsto (x =_A a)}(\mathsf{refl}_{x_1}, q) \equiv q \equiv \mathsf{refl}_{x_1}^{-1} \cdot q. \\ &\mathsf{transport}^{x \mapsto (x =_A x)}(\mathsf{refl}_{x_1}, q) \equiv q \\ &\qquad\qquad =_{x =_A x} \mathsf{refl}_{x_1}^{-1} \cdot q \cdot \mathsf{refl}_{x_1}. \end{aligned}$$

□

**Theorem 2.11.7.** *Let* $f \colon A \to C$ *and* $g \colon B \to C$ *be two functions. Define the type family* $P \colon A \times B \to \mathcal{U}$ *by* $P(x, y) :\equiv f(x) =_C g(y)$. *Then, given paths* $p \colon x_1 =_A x_2$, $q \colon y_1 =_B y_2$, *and an element* $r \colon f(x_1) =_C g(y_1)$, *as illustrated by the paths in* $C$ *below*



*we have*

$$\mathsf{transport}^P\big(\mathsf{pair}^=(p,q),r\big) = \mathsf{ap}_f(p)^{-1} \cdot r \cdot \mathsf{ap}_g(q).$$

*Proof.* The transport on the LHS defines a map

$$(f(x_1) =_C g(y_1)) \to (f(x_2) =_C g(y_2)).$$

Hence, the LHS inhabits $f(x_2) =_C g(y_2)$. The RHS is a concatenation of paths $f(x_2) =_C f(x_1)$, $f(x_1) =_C g(y_1)$, and $g(y_1) =_C g(y_2)$, which also computes to a path in $f(x_2) =_C g(y_2)$.

By path induction on $p$ and then on $q$, it suffices to assume $x_2 \equiv x_1$, $p \equiv \mathsf{refl}_{x_1}$, $y_2 \equiv y_1$ and $q \equiv \mathsf{refl}_{y_1}$. In this case the LHS computes to $r$, as it does the RHS:

$$\mathsf{ap}_f(\mathsf{refl}_{x_1})^{-1} \cdot r \cdot \mathsf{ap}_g(\mathsf{refl}_{y_1}) \equiv \mathsf{refl}^{-1}_{f(x_1)} \cdot r \cdot \mathsf{refl}_{g(y_1)} =_{f(x_1)=_C g(y_1)} r.$$

□

**Dependent case.** To generalize the non-dependent case, we consider the situation where

$$B\colon \prod_{x\colon X} A(x) \to \mathcal{U}.$$

The components from the non-dependent context generalize as follows:

$$\begin{aligned} \Pi_A B &\colon X \to \mathcal{U} \\ \Pi_A B(x) &:\equiv \prod_{u\colon A(x)} B(x,u) \\ f &\colon \Pi_A B(x). \end{aligned}$$

We introduce the total space and the uncurried family:

$$\begin{aligned} \Sigma_X A &:\equiv \sum_{x\colon X} A(x) && \text{(total space)} \\ \hat{B} &\colon \Sigma_X A \to \mathcal{U} && \text{(uncurried } B\text{)} \\ \hat{B}(\omega) &:\equiv B(\pi_1(\omega), \pi_2(\omega)) && \text{(induction principle).} \end{aligned}$$

**Theorem 2.11.8.** *Let $A\colon X \to \mathcal{U}$ and $B\colon \prod_{x\colon X} A(x) \to \mathcal{U}$ be type families, $x, y\colon X$ and $p\colon x =_X y$. Given $f\colon \Pi_A B(x)$ and $u\colon A(x)$, define*

$$p_* :\equiv \mathsf{transport}^A(p), \quad \gamma :\equiv \mathsf{pair}^=(\langle p, \mathsf{refl}_{p_*(u)}\rangle)\colon \langle x,u\rangle =_{\Sigma_X A} \langle y, p_*(u)\rangle.$$

*Then the following diagram of elements commutes propositionally:*

$$\begin{array}{ccc} u & \xmapsto{\quad p_* \quad} & p_*(u) \\ {\scriptstyle f}\big\downarrow & & \big\downarrow{\scriptstyle \mathsf{transport}^{\Pi_A B}(p,f)} \\ f(u) & \xmapsto[\mathsf{transport}^{\hat{B}}(\gamma)]{} & \mathsf{transport}^{\Pi_A B}(p,f)(p_*(u)) \end{array}$$

*Explicitly, we have the equality*

$$\mathsf{transport}^{\hat{B}}(\gamma, f(u)) =_{B(y,p_*(u))} \big(\mathsf{transport}^{\Pi_A B}(p, f)\big)(p_*(u)).$$

*Proof.*

TYPE VERIFICATION. Note that, since $u\colon A(x)$, we have $f(u)\colon B(x,u)$. Hence,

$$\mathsf{transport}^{\hat{B}}(\gamma)\colon B(x,u) \to B(y,p_*(u))$$

and

$$\text{LHS} \equiv \mathsf{transport}^{\hat{B}}(\gamma, f(u))\colon B(y,p_*(u)).$$

Since

$$\mathsf{transport}^{\Pi_A B}(p)\colon \Pi_A B(x) \to \Pi_A B(y),$$

we obtain

$$\mathsf{transport}^{\Pi_A B}(p, f)\colon \Pi_A B(y),$$

and so

$$\text{RHS} \equiv \big(\mathsf{transport}^{\Pi_A B}(p, f)\big)(p_*(u))\colon B(y,p_*(u)).$$

Therefore, both sides of the equation are of type $B(y,p_*(u))$.

INHABITATION. By path induction, it suffices to consider the case $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In that case, we deduce that $\gamma \equiv \mathsf{refl}_{\langle x,u\rangle}$. Therefore,

$$\begin{aligned}\text{LHS} &\equiv \mathsf{transport}^{\hat{B}}(\mathsf{refl}_{\langle x,u\rangle}, f(u))\\ &\equiv f(u)\end{aligned}$$

and

$$\begin{aligned}\text{RHS} &\equiv \big(\mathsf{transport}^{\Pi_A B}(\mathsf{refl}_x, f)\big)(u)\\ &\equiv f(u),\end{aligned}$$

which establishes that LHS $\equiv$ RHS.

□

**Remark 2.11.9.** The previous theorem is to Π-types what Theorem 2.9.14 is to Σ-types. To better visualize the analogy, let us rewrite the equality of Theorem 2.9.14 using the path $\gamma :\equiv \mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle)$ in the total space.

Let

$$\Sigma_A B\colon X \to \mathcal{U}, \quad \Sigma_A B(x) :\equiv \sum_{u\colon A(x)} B(x,u).$$

Then, we have

$$\underbrace{\mathsf{transport}^{\Sigma_A B}(p)(\langle u,z\rangle)}_{\text{transport } \langle u,z\rangle \text{ along } p} =_{\Sigma_A B(y)} \Big\langle \underbrace{p_*(u)}_{\text{transport } u}, \underbrace{\mathsf{transport}^{B}(\gamma)(z)}_{\text{transport } z \text{ along } \gamma}\Big\rangle$$

$$\underbrace{\mathsf{transport}^{\hat{B}}(\gamma, f(u))}_{\text{transport } f(u) \text{ along } \gamma} =_{B(y,p_*(u))} \Big(\underbrace{\mathsf{transport}^{\Pi_A B}(p,f)}_{\text{transport } f \text{ along } p}\Big)(p_*(u)).$$

In words,

transport a dependent pair = pair the transports,
transport an evaluation = evaluate the transport.

This means that transport commutes with dependent pairing in the first case, and with function application in the second.

**Corollary 2.11.10.** *With the preceding notation, for $v\colon A(y)$ we have*

$$\mathsf{transport}^{\Pi_A B}(p,f)(v) =_{B(y,v)} \mathsf{transport}^{\hat{B}}\big(\delta^{-1}, f((p^{-1})_*(v))\big),$$

*where $\delta :\equiv \mathsf{pair}^{=}(\langle p^{-1}, \mathsf{refl}_{(p^{-1})_*(v)}\rangle)$.*

*Proof.* Even though this can be derived from the theorem, it is much simpler to use path induction and verify the definitional equality.

Type verification: First observe that, by definition,

$$\text{LHS} \equiv \mathsf{transport}^{\Pi_A B}(p,f)(v)\colon B(y,v).$$

To compute the RHS, we need to compute $\delta$. Let $w :\equiv (p^{-1})_*(v)\colon A(x)$. By Definition 2.9.9, we have

$$\mathsf{pair}^{=}\colon \left(\sum_{q\colon \pi_1(\omega)=_X \pi_1(\omega')} q_*(\pi_2(\omega)) =_{A(\pi_1(\omega'))} \pi_2(\omega')\right) \to (\omega =_{\Sigma_X A} \omega').$$

In our case, $\omega :\equiv \langle y, v\rangle$, $\omega' :\equiv \langle x, w\rangle$, and $q :\equiv p^{-1}$. Therefore,

$$\mathsf{pair}^{=}\colon \left(\sum_{q\colon y=_X x} q_*(v) =_{A(x)} w\right) \to (\langle y,v\rangle =_{\Sigma_X A} \langle x,w\rangle).$$

Thus, we can apply $\mathsf{pair}^{=}$ to the dependent pair $\langle q, r\rangle$, provided that $r\colon q_*(v) =_{A(x)} w$. Since $q_*(v) :\equiv (p^{-1})_*(v) \equiv w$, we may take $r :\equiv \mathsf{refl}_w$ to deduce that

$$\langle p^{-1}, \mathsf{refl}_w\rangle\colon \mathrm{dom}(\mathsf{pair}^{=}).$$

Moreover,

$$\delta :\equiv \mathsf{pair}^{=}(\langle p^{-1}, \mathsf{refl}_w\rangle)\colon \langle y,v\rangle =_{\Sigma_X A} \langle x,w\rangle.$$

Therefore,

$$\mathsf{transport}^{\hat{B}}(\delta^{-1})\colon \hat{B}(\langle x,w\rangle) \to \hat{B}(\langle y,v\rangle),$$

which is definitionally equal to

$$\mathsf{transport}^{\hat{B}}(\delta^{-1})\colon B(x,w) \to B(y,v).$$

Since $f\colon \Pi_A B(x)$, we have $f(w)\colon B(x,w)$. Hence,

$$\text{RHS} \equiv \mathsf{transport}^{\hat{B}}(\delta^{-1}, f(w))\colon B(y,v).$$

INHABITATION: By path induction on $p$, the goal reduces to the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. We obtain

$$\begin{aligned}
\text{LHS} &\equiv \mathsf{transport}^{\Pi_A B}(\mathsf{refl}_x, f)(v) \\
&\equiv f(v), \\
\delta &\equiv \mathsf{pair}^{=}\big(\langle \mathsf{refl}_x, \mathsf{refl}_{(\mathsf{refl}_x)_*(v)}\rangle\big) \\
&\equiv \mathsf{refl}_{\langle x,v\rangle} && \text{; Rem. 2.9.10} \\
\text{RHS} &\equiv \mathsf{transport}^{\hat{B}}\big(\mathsf{refl}_{\langle x,v\rangle}, f((\mathsf{refl}_x)_*(v))\big) && \text{; } \delta^{-1} \equiv \mathsf{refl}_{\langle x,v\rangle} \\
&\equiv f((\mathsf{refl}_x)_*(v)) \\
&\equiv f(v).
\end{aligned}$$

□

**Proposition 2.11.11.** *Let $X$ and $A$ be types, and $P\colon X \to A \to \mathcal{U}$ a type family. Then, given $x, y\colon X$ and $p\colon x =_X y$, for any dependent function*

$$f\colon \prod_{a\colon A} P(x,a),$$

*we have*

$$\mathsf{transport}^{\lambda z.\, \prod_{a\colon A} P(z,a)}(p,f) =_{\prod_{a\colon A} P(y,a)} \lambda a.\, \mathsf{transport}^{\lambda z.\, P(z,a)}(p, f(a)).$$

*Proof.* Consider the type families $E\colon X \to \mathcal{U}$ and, for each $a\colon A$, $P_a\colon X \to \mathcal{U}$ defined by

$$E(z) :\equiv \prod_{a\colon A} P(z,a), \quad P_a(z) :\equiv P(z,a).$$

The LHS transport at $p$ is a function from $E(x)$ to $E(y)$, mapping $f$ to a dependent function in $E(y)$.

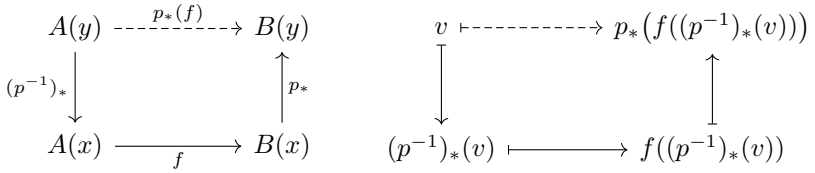
The RHS is also a dependent function in $E(y)$ because, for every $a\colon A$, the transport $\mathsf{transport}^{P_a}(p)$ is a map from $P(x,a)$ to $P(y,a)$.

To verify the propositional equality, by based path induction on $p$, it suffices to consider the case $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In that case, the LHS reduces to $f$ and the RHS to $\lambda a.\, f(a)$, which is definitionally equal to $f$. □

**Remark 2.11.12.** Suppose we have two type families $A, B\colon X \to \mathcal{U}$, a point $x\colon X$ and a function $f\colon A(x) \to B(x)$. Then, given a second point $y\colon X$ and a path $p\colon x =_X y$, to transport $f$ along $p$ for getting a new function

$$p_*(f)\colon A(y) \to B(y),$$

we can specify




**Lemma 2.11.13.** *Let $A, B\colon X \to \mathcal{U}$ be type families, and consider the family of function types $A \to B\colon X \to \mathcal{U}$ defined by $(A \to B)(x) :\equiv A(x) \to B(x)$. For any path $p\colon x =_X y$, function $f\colon A(x) \to B(x)$, and element $v\colon A(y)$, the transport of $f$ along $p$ evaluates as*

$$\mathsf{transport}^{A\to B}(p, f)(v) =_{B(y)} p_*\big(f((p^{-1})_*(v))\big),$$

*where $p_* \equiv \mathsf{transport}^{B}(p, -)$ and $(p^{-1})_* \equiv \mathsf{transport}^{A}(p^{-1}, -)$.*

*Proof.* By path induction on $p\colon x =_X y$, it suffices to consider the base case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. But in that case both sides are definitionally reduced to $f(v)$. □

**Lemma 2.11.14.** *Let $A\colon X \to \mathcal{U}$ be a type family and $p\colon x =_X y$ a path in $X$. Then, given a second type family $B\colon X \to \mathcal{U}$ and a non-dependent function $f\colon A(x) \to B(x)$, for any $u\colon A(x)$ there are two paths*

$$\begin{aligned} i(p, u)&\colon p_*(f)(p_*(u)) =_{B(y)} p_*\big(f(p_*^{-1}(p_*(u)))\big),\\ j(p, u)&\colon p_*\big(f(p_*^{-1}(p_*(u)))\big) =_{B(y)} p_*(f(u)),\end{aligned} \tag{2.7}$$

*where we use the abbreviations:*

$$\begin{aligned} p_*(f) &:\equiv \mathsf{transport}^{A\to B}(p, f),\\ p_*(f(u)) &:\equiv \mathsf{transport}^{B}(p, f(u)),\\ p_*(u) &:\equiv \mathsf{transport}^{A}(p, u).\end{aligned} \tag{2.8}$$

*Proof.* Define the motive

$$P\colon \prod_{y\colon X}(x =_X y) \to \mathcal{U}$$
$$P(y, p) :\equiv \prod_{u\colon A(x)} \mathsf{Eq}_i(y, p, u) \times \mathsf{Eq}_j(y, p, u),$$

where $\mathsf{Eq}_i$ and $\mathsf{Eq}_j$ are the identity types corresponding to $i(p,u)$ and $j(p,u)$.

By based path induction, it suffices to construct an inhabitant of $P(x, \mathsf{refl}_x)$. In this case, given $u\colon A(x)$, both equalities reduce to $f(u) =_{B(x)} f(u)$. Hence,

$$\lambda u.\, (\mathsf{refl}_{f(u)}, \mathsf{refl}_{f(u)})\colon P(x, \mathsf{refl}_x).$$

□

**Theorem 2.11.15.**[FE] *Let $A\colon X \to \mathcal{U}$ be a type family and $p\colon x =_X y$ a path in $X$. Then, given a second type family $B\colon X \to \mathcal{U}$ and two functions $f\colon A(x) \to B(x)$ and $g\colon A(y) \to B(y)$, there is an equivalence*

$$\big(p_*(f) =_{A(y)\to B(y)} g\big) \simeq (p_* \circ f \sim g \circ p_*),$$

*where $p_* \circ f$ and $g \circ p_*$ have signature*

$$p_* \circ f,\ g \circ p_*\colon A(x) \to B(y).$$

*Moreover, if $q\colon p_*(f) =_{A\to B} g$ corresponds under this equivalence to $\hat{q}$, then for any $u\colon A(x)$, the path*

$$\mathsf{happly}(q, p_*(u))\colon p_*(f)(p_*(u)) =_{B(y)} g(p_*(u))$$

*satisfies*

$$\mathsf{happly}(q, p_*(u)) =_{B(y)} i(p,u) \cdot j(p,u) \cdot \hat{q}(u),$$

*where $i(p,u)$ and $j(p,u)$ are the paths given by* (2.7).

*Proof.* To define the motive

$$P\colon \prod_{y\colon X} (x =_X y) \to \mathcal{U},$$

we proceed as follows:

- For $y\colon X$ and $p\colon x =_X y$, let

$$\mathsf{L}(y,p,g) :\equiv p_*(f) =_{A(y)\to B(y)} g,$$
$$\mathsf{R}(y,p,g) :\equiv p_* \circ f \sim g \circ p_*.$$

- Then define

$$P(y,p) :\equiv \prod_{g\colon A(y)\to B(y)} \sum_{e\colon \mathsf{L}\simeq\mathsf{R}} \mathsf{E}(y,p,g,e),$$

  where

$$\mathsf{E}(y,p,g,e) :\equiv \prod_{q\colon \mathsf{L}(y,p,g)} \prod_{u\colon A(x)} \mathsf{happly}(q, p_*(u)) =_{B(y)} i(p,u) \cdot j(p,u) \cdot e(q)(u),$$

and $i(p,u)$ and $j(p,u)$ are the paths defined in (2.7). The following diagram, where the arrows represent paths, shows that $\mathsf{E}$ is well defined:

$$\begin{array}{ccc} p_*(f)(p_*(u)) & \xrightarrow{\mathsf{happly}(q,p_*(u))} & g(p_*(u)) \\ {\scriptstyle i(p,u)}\downarrow & & \uparrow{\scriptstyle e(q)(u)} \\ p_*\big(f((p^{-1})_*(p_*(u)))\big) & \xrightarrow[j(p,u)]{} & p_*(f(u)) \end{array}$$

To see this, recall from (2.5) that $\mathsf{happly}(q,v) =_{B(y)} \mathsf{ap}_{\mathrm{ev}_v}(p)$, which shows that the top arrow represents the resulting path. Moreover, since $e\colon \mathsf{L} \simeq \mathsf{R}$ and $q\colon p_*(f) =_{A(y)\to B(y)} g$, by identifying $e$ with its first projection, we obtain $e(q)\colon \mathsf{R}$ and $e(q)(u)\colon p_*(f(u)) =_{B(y)} g(p_*(u))$.

By based path induction, i.e., by equation (1.3) applied to $P(y,p)$, we have to construct an inhabitant for the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case:

- $p_* \equiv \mathsf{id}$, $\mathsf{L}(x,\mathsf{refl}_x,g) \equiv f =_{A(x)\to B(x)} g$, and $\mathsf{R}(x,\mathsf{refl}_x,g) \equiv f \sim g$.
- $i(\mathsf{refl}_x,u) \equiv \mathsf{refl}_{f(u)}$ and $j(\mathsf{refl}_x,u) \equiv \mathsf{refl}_{f(u)}$.
- We choose $e :\equiv \mathsf{happly}$, which is an equivalence by the function extensionality axiom. The type $\mathsf{E}(x,\mathsf{refl}_x,g,\mathsf{happly})$ requires us to verify that for every $q\colon f =_{A(x)\to B(x)} g$ and $u\colon A(x)$,

$$\mathsf{happly}(q,u) =_{B(x)} \mathsf{refl}_{f(u)} \cdot \mathsf{refl}_{f(u)} \cdot \mathsf{happly}(q,u).$$

But this is inhabited by $\mathsf{refl}_{\mathsf{happly}(q,u)}$.

Thus, the map

$$g \mapsto (\mathsf{happly}, \lambda q.\, \lambda u.\, \mathsf{refl}_{\mathsf{happly}(q,u)})$$

inhabits $P(x,\mathsf{refl}_x)$, which completes the proof. □

What follows is a generalization of Lemma 2.11.14 and Theorem 2.11.15 for dependent functions. The notation is the one introduced in the Dependent case of section Transport Decomposition.

**Lemma 2.11.16.** *Let $A\colon X \to \mathcal{U}$ be a type family and $p\colon x =_X y$ a path in $X$. Then, given a second type family $B\colon \prod_{x:X} A(x) \to \mathcal{U}$ and a dependent function $f\colon \Pi_A B(x)$, we have two paths*

$$i(p,u)\colon p_*(f)(p_*(u)) =_{B(y,p_*(u))} \Phi((p^{-1})_*(p_*(u)))$$
$$j(p,u)\colon \Phi((p^{-1})_*(p_*(u))) =_{B(y,p_*(u))} \mathsf{Tr}(p,f,u),$$

*with*

$$\Phi(w) :\equiv \mathsf{transport}^{\hat{B}}\big(\mathsf{pair}^{=}(p^{-1},\mathsf{refl}_w)^{-1}, f(w)\big)$$
$$\mathsf{Tr}(p,f,u) :\equiv \mathsf{transport}^{\hat{B}}\big(\mathsf{pair}^{=}(p,\mathsf{refl}_{p_*(u)}), f(u)\big).$$

*Proof.*

TYPE VERIFICATION: Observe that[21]

$$u\colon A(x), \quad p_*(u)\colon A(y), \quad w \equiv (p^{-1})_*(p_*(u))\colon A(x).$$

Thus, as shown in Corollary 2.11.10, TYPE VERIFICATION, for $v :\equiv p_*(u)$:

$$\begin{aligned} \mathsf{pair}^{=}(p^{-1}, \mathsf{refl}_w)^{-1}\colon (x,w) =_{\Sigma_X A} (y, p_*(u)), \qquad & f(w)\colon B(x,w), \\ \Phi(w)\colon B(y, p_*(u)), \qquad & \mathsf{Tr}(p,f,u)\colon B(y,p_*(u)). \end{aligned}$$

Then all expressions are correctly typed.

INHABITATION: To construct the witnesses $i(p,u)$ and $j(p,u)$, define the motive

$$\begin{aligned} P\colon & \prod_{y\colon X} (x =_X y) \to \mathcal{U} \\ P(y,p) :\equiv & \prod_{u\colon A(x)} \mathsf{Eq}_i(y,p,u) \times \mathsf{Eq}_j(y,p,u), \end{aligned}$$

where $\mathsf{Eq}_i$ and $\mathsf{Eq}_j$ are the types of $i(p,u)$ and $j(p,u)$.

By based path induction, to show that $P(y,p)$ is inhabited it suffices to construct a witness of $P(x, \mathsf{refl}_x)$. In this case:

- $p$ and $p^{-1}$ reduce definitionally to $\mathsf{refl}_x$.
- Since the transport operations $p_*$ reduce to identity functions, we get $p_*(u) \equiv u$ and $p_*(f) \equiv f$.
- The fiber coordinate $w :\equiv (p^{-1})_*(p_*(u))$ reduces definitionally to $u$.
- $\Phi(w)$ becomes $f(u)$.
- Finally,

$$\begin{aligned} \mathsf{Tr}(\mathsf{refl}_x, f, u) &\equiv \mathsf{transport}^{\hat{B}}\big(\mathsf{pair}^{=}(\mathsf{refl}_x, \mathsf{refl}_u), f(u)\big) \\ &\equiv \mathsf{transport}^{\hat{B}}\big(\mathsf{refl}_{(x,u)}, f(u)\big) && \text{; Rem. 2.9.10} \\ &\equiv f(u). \end{aligned}$$

In consequence, for any $u\colon A(x)$, the goals become:

$$\begin{aligned} i(\mathsf{refl}_x, u)\colon f(u) =_{B(x,u)} f(u), \\ j(\mathsf{refl}_x, u)\colon f(u) =_{B(x,u)} f(u). \end{aligned}$$

These are clearly inhabited by $\mathsf{refl}_{f(u)}$. Thus, the term

$$\lambda u.\, (\mathsf{refl}_{f(u)}, \mathsf{refl}_{f(u)})$$

witnesses $P(x, \mathsf{refl}_x)$, completing the proof. □

[21] See the notation used in Theorem 2.11.8.

**Theorem 2.11.17.**[FE] *Let $A\colon X \to \mathcal{U}$ and $B\colon \prod_{x:X} A(x) \to \mathcal{U}$ be two type families. Given a path $p\colon x =_X y$ and two functions $f\colon \Pi_A B(x)$ and $g\colon \Pi_A B(y)$, there is an equivalence*

$$\big(p_*(f) =_{\Pi_A B(y)} g\big) \simeq \big(\mathsf{Tr}(p,f) \sim g \circ p_*\big),$$

*where*

$$\mathsf{Tr}(p,f,u) :\equiv \mathsf{transport}^{\hat{B}}\big(\mathsf{pair}^{=}(\langle p, \mathsf{refl}_{p_*(u)}\rangle), f(u)\big).$$

*Moreover, if $q\colon p_*(f) =_{\Pi_A B(y)} g$ corresponds under this equivalence to $\hat{q}$, then for any $u\colon A(x)$, the path*

$$\mathsf{happly}(q, p_*(u))\colon p_*(f)(p_*(u)) =_{B(y,p_*(u))} g(p_*(u))$$

*is equal to the concatenated path*

$$i(p,u) \cdot j(p,u) \cdot \hat{q}(u),$$

*where $i(p,u)$ and $j(p,u)$ are the paths given by* Lemma 2.11.16*, as illustrated in the following diagram:*

$$\begin{array}{ccc} p_*(f)(p_*(u)) & \xrightarrow{\mathsf{happly}(q,p_*(u))} & g(p_*(u)) \\ {\scriptstyle i(p,u)}\big\downarrow & & \big\uparrow{\scriptstyle \hat{q}(u)} \\ \Phi((p^{-1})_*(p_*(u))) & \xrightarrow[j(p,u)]{} & \mathsf{Tr}(p,f,u) \end{array}$$

*with $\Phi$ defined in* Lemma 2.11.16*.*

*Proof.*

Type verification: We need to verify that both sides of the equivalence are well-formed elements of $\mathcal{U}$.

For the lhs, observe that $p_*(f)$ and $g$ both inhabit the function type $\Pi_A B(y)$; thus, the identity type $p_*(f) =_{\Pi_A B(x)} g$ is well-formed.

For the rhs, we must ensure the terms inhabit the same fiber.

- By Lemma 2.11.16, $\mathsf{Tr}(p,f,u)\colon B(y, p_*(u))$.
- Since $g\colon \Pi_A B(y)$ and $p_*(u)\colon A(y)$, we have $g(p_*(u))\colon B(y, p_*(u))$.

Since both terms inhabit $B(y, p_*(u))$, the identity type on the right is also well-formed.

Inhabitation: To define the motive

$$P\colon \prod_{y:X} (x =_X y) \to \mathcal{U}$$

proceed as follows:

- For $y\colon X$ and $p\colon x =_X y$, let

$$\mathsf{L}(y,p,g) :\equiv p_*(f) =_{\prod_A B(y)} g,$$
$$\mathsf{R}(y,p,g) :\equiv \mathsf{Tr}(p,f) \sim g \circ p_*.$$

- Then define

$$P(y,p) :\equiv \prod_{g\colon \prod_A B(y)} \sum_{e\colon \mathsf{L}\simeq\mathsf{R}} \mathsf{E}(y,p,g,e),$$

where

$$\mathsf{E}(y,p,g,e) :\equiv \prod_{q\colon \mathsf{L}(y,p,g)} \prod_{u\colon A(x)} \mathsf{happly}(q,v) =_{B(y,v)} \delta(p,u)\cdot e(q)(u),$$

with $v :\equiv p_*(u)$ and $\delta(p,u) :\equiv i(p,u)\cdot j(p,u)$.

By based path induction, it suffices to specify an inhabitant for the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case:

- $\mathsf{L}(x,\mathsf{refl}_x,g) \equiv f = g$, $\mathsf{Tr}(\mathsf{refl}_x,f,u) \equiv f(u)$, $\mathsf{R}(x,\mathsf{refl}_x,g) \equiv f \sim g$.
- $i(\mathsf{refl}_x,u)$ and $j(\mathsf{refl}_x,u)$ reduce to $\mathsf{refl}_{f(u)}$.
- Take $e :\equiv \mathsf{happly}$, which is an equivalence by function extensionality. The type $\mathsf{E}(x,\mathsf{refl}_x,g,\mathsf{happly})$ requires us to verify that for every $q\colon f =_{\prod_A B} g$ and $u\colon A(x)$,

$$\mathsf{happly}(q,u) =_{B(x,u)} \mathsf{refl}_{f(u)} \cdot \mathsf{happly}(q,u).$$

This is inhabited by $\mathsf{refl}_{\mathsf{happly}(q,u)}$.

Thus, the map

$$g \mapsto (\mathsf{happly}, \lambda q.\, \lambda u.\, \mathsf{refl}_{\mathsf{happly}(q,u)})$$

inhabits $P(x,\mathsf{refl}_x)$, which completes the proof. □

## 2.12 Naturality of Function Extensionality

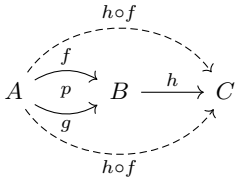
Let $f,g\colon A \to B$ be functions. Recall that a homotopy $\eta\colon f \sim g$ translates pointwise into a path $\eta(x)\colon f(x) =_B g(x)$ for each $x\colon A$, whereas an identity path $p\colon f =_{A\to B} g$ operates at the level of the function types. These two notions are related by the equivalence $\mathsf{happly}\colon (f = g) \simeq (f \sim g)$.

The left whiskering of a function $h\colon B \to C$ and a homotopy $\eta$ is defined pointwise as

$$(h \triangleleft \eta)(x) :\equiv \mathsf{ap}_h(\eta(x)) \qquad \text{(homotopy whiskering)}$$

For an identity path $p\colon f =_{A\to B} g$, we define left whiskering as

$$h \lhd p :\equiv \mathsf{ap}_{h\circ -}(p) \qquad \text{(function identity whiskering)}$$




**Lemma 2.12.1.** *Both notions of whiskering are related by*

$$\mathsf{happly}(h \lhd p)(x) =_C (h \lhd \mathsf{happly}(p))(x),$$

*Proof.* Note that $\mathsf{happly}$ maps $h \lhd p$ to a homotopy. Furthermore, since $\mathsf{happly}(p)$ is already a homotopy, the whiskering on the RHS is a homotopy whiskering.

By based path induction on $p$, it suffices to consider $g \equiv f$ and $p \equiv \mathsf{refl}_f$:

$$\begin{aligned}
\text{LHS} &\equiv \mathsf{happly}(h \lhd \mathsf{refl}_f)(x)\\
&\equiv \mathsf{happly}(\mathsf{ap}_{h\circ -}(\mathsf{refl}_f))(x)\\
&\equiv \mathsf{happly}(\mathsf{refl}_{h\circ f})(x)\\
&\equiv (\lambda y.\, \mathsf{refl}_{h\circ f(y)})(x)\\
&\equiv \mathsf{refl}_{h\circ f(x)}.\\
\text{RHS} &\equiv (h \lhd \mathsf{happly}(\mathsf{refl}_f))(x)\\
&\equiv \mathsf{ap}_h\big(\mathsf{happly}(\mathsf{refl}_f)(x)\big)\\
&\equiv \mathsf{ap}_h\big((\lambda y.\, \mathsf{refl}_{f(y)})(x)\big)\\
&\equiv \mathsf{ap}_h(\mathsf{refl}_{f(x)})\\
&\equiv \mathsf{refl}_{h\circ f(x)}.
\end{aligned}$$

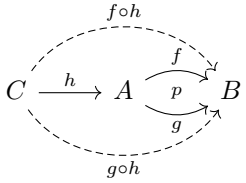
□

Analogously, based on the definition of right homotpy whiskering

$$\eta \rhd h :\equiv \lambda z.\, \eta(h(z))\colon f \circ h \sim g \circ h.$$

we introduce right function identity whiskering of a path $p$ with a function $k\colon C \to A$ as

$$p \rhd h :\equiv \mathsf{ap}_{-\circ h}(p),$$



**Lemma 2.12.2.** *Given $z\colon C$, we have*

$$\mathsf{happly}(p \triangleright h)(z) =_B \mathsf{happly}(p)(h(z)).$$

*Proof.* By based path induction on $p$, we can set $g \equiv f$ and $p \equiv \mathsf{refl}_f$:

$$\begin{aligned}
\text{LHS} &\equiv \mathsf{happly}(\mathsf{refl}_f \triangleright h)(z) \\
&\equiv \mathsf{happly}\big(\mathsf{ap}_{-\circ h}(\mathsf{refl}_f)\big)(z) \\
&\equiv \mathsf{happly}(\mathsf{refl}_{f\circ h})(z) \\
&\equiv \mathsf{refl}_{f\circ h(z)}. \\
\text{RHS} &\equiv \mathsf{happly}(\mathsf{refl}_f)(h(z)) \\
&\equiv \mathsf{refl}_{f(h(z))}.
\end{aligned}$$

□

**Theorem 2.12.3.**[FE] [Naturality of function extensionality] *With the preceding notation, given $\eta\colon f \sim g$, we have*

$$\mathsf{funext}(h \triangleleft \eta) = h \triangleleft \mathsf{funext}(\eta)$$

*and*

$$\mathsf{funext}(\eta \triangleright k) = \mathsf{funext}(\eta) \triangleright k.$$

*Proof.* Using the homotopy $\mathsf{happly} \circ \mathsf{funext} \sim \mathsf{id}$, we deduce that

$$\begin{aligned}
\mathsf{happly}(\mathsf{funext}(h \triangleleft \eta))(x) &= (h \triangleleft \eta)(x) \\
&= \mathsf{ap}_h(\eta(x))
\end{aligned}$$

and, since $\mathsf{funext}(\eta)\colon f =_{A\to B} g$,

$$\begin{aligned}
\mathsf{happly}(h \triangleleft \mathsf{funext}(\eta))(x) &= \big(h \triangleleft \mathsf{happly}(\mathsf{funext}(\eta))\big)(x) \qquad \text{; Lem. 2.12.1} \\
&= (h \triangleleft \eta)(x) \\
&= \mathsf{ap}_h(\eta(x)).
\end{aligned}$$

Hence, $\mathsf{happly}(\mathsf{funext}(h \triangleleft \eta))(x) = \mathsf{happly}(h \triangleleft \mathsf{funext}(\eta))(x)$. The first equality follows by applying $\mathsf{funext}$.

For the second equality, evaluating $\mathsf{happly}$ at a point $z\colon C$ yields

$$\begin{aligned}
\mathsf{happly}(\text{LHS})(z) &\equiv \mathsf{happly}(\mathsf{funext}(\eta \triangleright k))(z) \\
&= (\eta \triangleright k)(z) \\
&\equiv \eta(k(z))
\end{aligned}$$

and

$$\begin{aligned}\mathsf{happly}(\mathrm{RHS})(z) &\equiv \mathsf{happly}\big(\mathsf{funext}(\eta) \triangleright k\big)(z) \\ &= \mathsf{happly}(\mathsf{funext}(\eta))(k(z)) && ;\ \text{Lem. } 2.12.2 \\ &= \eta(k(z)).\end{aligned}$$

Since both sides are pointwise equal, $\mathsf{funext}$ yields the second identity.

□

In categorical terms, the maps $(h \circ -)$ and $(- \circ k)$ act as functors, and $\mathsf{happly}$ and $\mathsf{funext}$ act as natural isomorphisms between the Hom-functors defined by identity types and those defined by homotopies.

$$\begin{array}{ccc} f =_{A\to B} g & \xrightarrow{h\triangleleft -} & h\circ f =_{A\to C} h\circ g \\ {\scriptstyle\mathsf{happly}}\downarrow\uparrow{\scriptstyle\mathsf{funext}} & & {\scriptstyle\mathsf{funext}}\downarrow\uparrow{\scriptstyle\mathsf{happly}} \\ f \sim g & \xrightarrow[h\triangleleft -]{} & h\circ f \sim h\circ g \end{array} \qquad \begin{array}{ccc} f =_{A\to B} g & \xrightarrow{-\triangleright k} & f\circ k =_{C\to B} g\circ k \\ {\scriptstyle\mathsf{happly}}\downarrow\uparrow{\scriptstyle\mathsf{funext}} & & {\scriptstyle\mathsf{funext}}\downarrow\uparrow{\scriptstyle\mathsf{happly}} \\ f \sim g & \xrightarrow[-\triangleright k]{} & f\circ k \sim g\circ k \end{array}$$

**Remark 2.12.4.** If we formally interpret functions as paths and function composition as concatenation, function identity terms become 2-paths. Taking this analogy further, we can apply to it the general notion of left path whiskering introduced in Remark 2.1.4. It formally becomes

$$h \triangleleft p :\equiv \mathsf{ap}_{\mathsf{lct}_h}(p),$$

for $p\colon f =_{A\to B} g$, where $\mathsf{lct}_h :\equiv x \mapsto h \cdot x$. Interpreting concatenation analytically as function composition, we have $h \cdot x \equiv h \circ x$, so $\mathsf{lct}_h \equiv h \circ -$ and $h \triangleleft p \equiv \mathsf{ap}_{h\circ -}(p)$. Therefore, the definition of left whiskering introduced above matches precisely the general one.

Analogously, applying the general notion of right path whiskering yields

$$p \triangleright k :\equiv \mathsf{ap}_{\mathsf{rct}_k}(p),$$

where $\mathsf{rct}_k :\equiv x \mapsto x \cdot k$. Under the same interpretation, we have $x \cdot k \equiv x \circ k$, so $\mathsf{rct}_k \equiv - \circ k$ and $p \triangleright k \equiv \mathsf{ap}_{-\circ k}(p)$. Thus, the definition of right function identity whiskering also aligns perfectly with the general one.

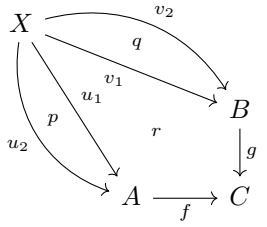
**Proposition 2.12.5.** *Let $h\colon B \to C$, $f, g\colon A \to B$, and $p\colon f =_{A\to B} g$. Then*

a) $h \triangleleft p^{-1} = (h \triangleleft p)^{-1}$

**Proposition 2.12.6.** *Let $f\colon A \to C$ and $g\colon B \to C$ be functions, and let $X$ be a type. Define the family*

$$\begin{aligned} W &\colon (X \to A) \times (X \to B) \to \mathcal{U} \\ W(u, v) &:\equiv f \circ u =_{X\to C} g \circ v. \end{aligned}$$

*Given paths* $p\colon u_1 = u_2$ *and* $q\colon v_1 = v_2$*, and a path* $r\colon f \circ u_1 = g \circ v_1$*, as illustrated below*

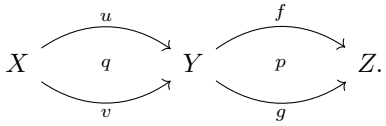



*we have*

$$\mathsf{transport}^W\big(\mathsf{pair}^{=}(p,q), r\big) = (f \lhd p)^{-1} \cdot r \cdot (g \lhd q).$$

*Proof.* This is a corollary of Theorem 2.11.7. □

**Proposition 2.12.7.** *Let* $X, Y, Z\colon \mathcal{U}$ *be types,* $f, g\colon Y \to Z$ *equal functions with* $p\colon f =_{Y\to Z} g$*, and* $u, v\colon X \to Y$ *equal functions with* $q\colon u =_{X\to Y} v$*, as depicted below*




*Then, we have the canonical equality*

$$(f \lhd q) \cdot (p \rhd v) =_{f\circ u =_{X\to Z} g\circ v} (p \rhd u) \cdot (g \lhd q).$$

*Proof.* This can be derived from § 2.4.4 by function extensionality. However, here we give an independent proof.

First observe that the path on the LHS is a concatenation of a path $f \circ u =_{X\to Z} f \circ v$ and a path $f \circ v =_{X\to Z} g \circ v$, which computes to a path that inhabits $f \circ u =_{X\to Z} g \circ v$.

In the RHS the first path inhabits $f \circ u =_{X\to Z} g \circ u$ while the second inhabits $g \circ u =_{X\to Z} g \circ v$. Thus, their concatenation also computes to a path of type $f \circ u =_{X\to Z} g \circ v$.

We proceed by path induction on both $p$ and $q$. It suffices to consider the case where $f \equiv g$ with $p \equiv \mathsf{refl}_f$, and $u \equiv v$ with $q \equiv \mathsf{refl}_u$.

Under these definitions, the left-hand side evaluates to

$$(f \lhd \mathsf{refl}_u) \cdot (\mathsf{refl}_f \rhd u) \equiv \mathsf{refl}_{f\circ u} \cdot \mathsf{refl}_{f\circ u} \equiv \mathsf{refl}_{f\circ u}.$$

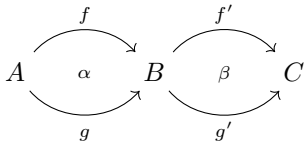
Similarly, the right-hand side evaluates to

$$(\mathsf{refl}_f \rhd u) \cdot (f \lhd \mathsf{refl}_u) \equiv \mathsf{refl}_{f\circ u} \cdot \mathsf{refl}_{f\circ u} \equiv \mathsf{refl}_{f\circ u}.$$

Since both sides compute to the reflexivity path $\mathsf{refl}_{f\circ u}$, they are judgmentally equal. □

### Homotopy Whiskering

Given equivalent functions $f, g\colon A \to B$ with $\alpha\colon f \sim g$, and $f', g'\colon B \to C$ with $\beta\colon f' \sim g'$,




the *horizontal composition* $\beta \bullet \alpha$ is defined using whiskering and homotopy concatenation[22] as

$$\beta \bullet \alpha :\equiv (\beta \triangleright f) \cdot (g' \triangleleft \alpha)\colon (f' \circ f) \sim (g' \circ g).$$

**Proposition 2.12.8.**[FE] *With the preceding notation,*

a) $\beta \bullet \alpha =_{(f' \circ f) \sim (g' \circ g)} (f' \triangleleft \alpha) \cdot (\beta \triangleright g)$

b) $\beta \bullet \mathsf{refl}_f = \beta \triangleright f$

c) $\mathsf{refl}_{f'} \triangleleft \alpha \equiv g' \triangleleft \alpha$

*Proof.* For part a) we have

$$\begin{aligned}
\mathsf{funext}(\beta \bullet \alpha) &= \mathsf{funext}(\beta \triangleright f) \cdot \mathsf{funext}(g' \triangleleft \alpha) && \text{; Prop. 2.11.2} \\
&= (\mathsf{funext}(\beta) \triangleright f) \cdot (g' \triangleleft \mathsf{funext}(\alpha)) && \text{; Thm. 2.12.3} \\
&= (f' \triangleleft \mathsf{funext}(\alpha)) \cdot (\mathsf{funext}(\beta) \triangleright g) && \text{; Prop. 2.11.2} \\
&= \mathsf{funext}((f' \triangleleft \alpha) \cdot (\beta \triangleright g)),
\end{aligned}$$

and the result follows by evaluation of $\mathsf{happly}$.

Parts b) and c) are direct consequences of Remark 2.4.8. □

## 2.13 The Univalence Axiom

To apply the observational framework 2.6.1 to the universe $\mathcal{U}$, we identify the observational equality with the type of equivalences.

Code family.

$$\begin{aligned}
\mathsf{Code}&\colon \mathcal{U} \to \mathcal{U} \to \mathcal{U} \\
\mathsf{Code}(A, B) &:\equiv A \simeq B.
\end{aligned}$$

Base code. We specify the base code $c\colon \mathsf{Code}(A, A)$ as $c :\equiv \mathsf{id}_A$. This is an abuse of notation [cf. Proposition 2.5.13] for the canonical equivalence

$$\langle \mathsf{id}_A, (\langle \mathsf{id}_A, \mathsf{refl} \rangle, \langle \mathsf{id}_A, \mathsf{refl} \rangle) \rangle\colon A \simeq A.$$

[22] See Definition 2.4.6.

ENCODE FUNCTION. To define

$$\mathsf{encode}\colon (A =_{\mathcal{U}} B) \to \mathsf{Code}(A, B),$$

with $C_A :\equiv \mathsf{Code}(A, -)$, we will use based path induction. Given $X : \mathcal{U}$ and $q\colon A =_{\mathcal{U}} X$, consider the motive

$$D(X, q) :\equiv A \to X.$$

To define the depending function

$$\prod_{X:\mathcal{U}} \prod_{q:\, A=_{\mathcal{U}} X} A \to X,$$

it suffices to specify $d\colon D(A, \mathsf{refl}_A) \equiv A \to A$. Then, for $d :\equiv \mathsf{id}_A$, we obtain

$$f(p) :\equiv \mathsf{ind}'_{=_A}(A, D, \mathsf{id}_A, B, p),$$

for $p\colon A =_{\mathcal{U}} B$.

To further characterize this map, consider the type family $\mathsf{id}_{\mathcal{U}}\colon \mathcal{U} \to \mathcal{U}$. Given $p\colon A =_{\mathcal{U}} B$, we have

$$\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}\colon (A =_{\mathcal{U}} B) \to (A \to B).$$

We claim that $f(p) =_{A\to B} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p)$. To prove this by path induction on $p$, it suffices to restrict ourselves to the case $B \equiv A$ and $p \equiv \mathsf{refl}_A$. In this case

$$\begin{aligned}
\text{LHS} &\equiv f(\mathsf{refl}_A) \\
&\equiv \mathsf{ind}'_{=_A}(A, D, \mathsf{id}_A, A, \mathsf{refl}_A) \\
&\equiv \mathsf{id}_A, \\
\text{RHS} &\equiv \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{refl}_A) \\
&\equiv \mathsf{id}_A \qquad\qquad \text{; Thm. 1.11.6}
\end{aligned}$$

**Theorem 2.13.1.** *Given $A, B : \mathcal{U}$, the encode function defined above is propositionally equal to the function*

$$\mathsf{idtoeqv}\colon (A =_{\mathcal{U}} B) \to (A \simeq B) \tag{2.1}$$

*induced by*

$$p \mapsto \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p),$$

*where for each path $p$, the quasi-inverse of the map $\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p)$ is given by $\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p^{-1})$, with right and left homotopies respectively provided by*

$$\mathsf{transportinv}_r^{\mathsf{id}_{\mathcal{U}}}(p) \quad \textit{and} \quad \mathsf{transportinv}_l^{\mathsf{id}_{\mathcal{U}}}(p).$$

*Proof.* For a given path $p\colon A =_{\mathcal{U}} B$, the verification that $\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p^{-1})$ acts as a quasi-inverse for $\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p)$ via the stated homotopies follows directly from Lemma 2.3.11. □

**Definition 2.13.2.** The theorem allows us to take $\mathsf{idtoeqv}$ as our encode function.

DECODE FUNCTION. To define the decoding map we need to add the following

**Axiom.** [Univalence] For any $A, B\colon \mathcal{U}$, the function (2.1) is an equivalence.

**Remark 2.13.3.** The Univalence Axiom asserts that $\mathsf{idtoeqv}$ is an equivalence. In particular, $(A =_{\mathcal{U}} B) \simeq (A \simeq B)$. Moreover, it allows us to establish the following rules

- the induction principle:

$$\mathsf{ua}\colon (A \simeq B) \to (A =_{\mathcal{U}} B),$$

- the elimination rule:

$$\mathsf{idtoeqv}\colon (A =_{\mathcal{U}} B) \to (A \simeq B),$$

- the propositional computation rule: for $f\colon A \simeq B$ and $x\colon A$,

$$\mathsf{idtoeqv}(\mathsf{ua}(f))(x) =_B f(x),$$

- the propositional uniqueness principle: for $p\colon A =_{\mathcal{U}} B$,

$$\mathsf{ua}(\mathsf{idtoeqv}(p)) =_{A=_{\mathcal{U}}B} p.$$

**Extended framework.** The groupoid structure corresponds to the groupoid structure of equivalences:

$$\begin{aligned} \mathsf{inv}&\colon (A \simeq B) \to (B \simeq A) \\ \mathsf{inv}(e) &:\equiv e^{-1} \end{aligned}$$

and

$$\begin{aligned} *&\colon (A \simeq B) \to (B \simeq C) \to (A \simeq C) \\ e * e' &:\equiv e' \circ e, \end{aligned}$$

with inverse and composition is defined in Proposition 2.5.13.

The first unit laws holds definitionally

$$\begin{aligned}\mathsf{inv}(c) &\equiv (\mathsf{id}_A, (\mathsf{id}_A, \mathsf{refl}, \mathsf{id}_A, \mathsf{refl}))^{-1}\\ &\equiv (\mathsf{id}_A, (\mathsf{id}_A, \mathsf{refl}, \mathsf{id}_A, \mathsf{refl}))\\ &\equiv c\end{aligned}$$

and the second propositionally, by Corollary 2.11.4

$$\begin{aligned}c * d &\equiv (\mathsf{id}_A, (\mathsf{id}_A, \mathsf{refl}, \mathsf{id}_A, \mathsf{refl})) \circ d\\ &=_{A\simeq B} d.\end{aligned}$$

**Lemma 2.13.4.** *Let $X$ and $C$ be types, and $P\colon X \to \mathcal{U}$ a type family. For any path $p\colon x =_X y$ and map $g\colon P(x) \to C$, we have a homotopy*

$$\mathsf{transport}^{\lambda z.\,(P(z)\to C)}(p, g) \sim g \circ \mathsf{transport}^{P}(p^{-1}).$$

*Proof.* The conclusion is a direct consequence of Lemma 2.11.13 applied to $P$ and the constant family $C$, and Exercise 1.16.3. □

**Proposition 2.13.5.**[UA] *Let $E, A, B\colon \mathcal{U}$ be types. Then, given an equivalence $e\colon E \simeq A$ and a function $g\colon E \to B$, we have*

$$\mathsf{transport}^{\lambda X.\,(X\to B)}(\mathsf{ua}(e), g) \sim g \circ e^{-1}.$$

*Proof.* Let $p :\equiv \mathsf{ua}(e)$. Then $p\colon E =_{\mathcal{U}} A$ and $p^{-1} =_{A=_{\mathcal{U}}E} \mathsf{ua}(e^{-1})$. By Lemma 2.11.13 applied to the identity type family $\mathsf{id}_{\mathcal{U}}$ and the constant $\lambda X.\,B$, given $a\colon A$, we have

$$\begin{aligned}\mathsf{transport}^{\lambda X.\,(X\to B)}(p, g)(a) &=_B \mathsf{transport}^{\lambda X.\,B}\big(p, g(\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p^{-1}, a))\big)\\ &\overset{(1)}{=}_B g(\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p^{-1}, a))\\ &\overset{(2)}{=}_B g(\mathsf{idtoeqv}(p^{-1})(a))\\ &=_B g(\mathsf{idtoeqv}(\mathsf{ua}(e^{-1}))(a))\\ &\equiv g(e^{-1}(a)),\end{aligned}$$

(1) holds because the transport of a constant family is the identity map [cf. Exercise 1.16.3].

(2) follows from Theorem 2.13.1.

□

**Proposition 2.13.6.** *Let $B\colon A \to \mathcal{U}$ be a type family and $x, y\colon A$. Given a path $p\colon x =_A y$ and an element $u\colon B(x)$, we have*

$$\begin{aligned}\mathsf{transport}^{B}(p, u) &=_{B(y)} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_B(p), u)\\ &\equiv \pi_1\big(\mathsf{idtoeqv}(\mathsf{ap}_B(p))\big)(u),\end{aligned}$$

*where*

$$\pi_1 : \Big( \sum_{f : B(x) \to B(y)} \mathsf{isequiv}(f) \Big) \to \big( B(x) \to B(y) \big)$$

*is the first projection of the* $\Sigma$*-type.*[23]

*Proof.* The types of both sides of the first equality are:

$$\begin{aligned} \mathsf{transport}^B(p) &: B(x) \to B(y),\ u : B(x) \implies \mathsf{transport}^B(p, u) : B(y) \\ \mathsf{ap}_B(p) &: B(x) \to B(y),\ u : B(x) \implies \mathsf{transport}^{\mathsf{id}\mathcal{U}}(\mathsf{ap}_B(p), u) : B(y), \end{aligned}$$

and the equality follows by path induction on $p$, since both sides reduce to $u$ when $y \equiv x$ and $p \equiv \mathsf{refl}_x$. □

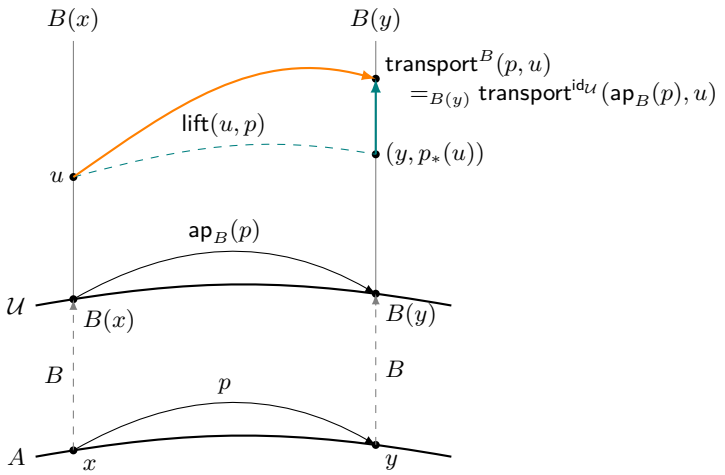
**Remark 2.13.7.** The first equality of Proposition 2.13.6 is a direct consequence of Lemma 2.3.13, for the case $f \equiv B$ and $P \equiv \mathsf{id}_{\mathcal{U}}$.

Conversely, Lemma 2.3.13 can be derived from the proposition applying it twice in different contexts.

More precisely,

$$\begin{aligned} \mathsf{transport}^{P \circ f}(p, u) &=_{P(f(y))} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_{P \circ f}(p), u) && ;\ B \equiv P \circ f,\ P \equiv \mathsf{id}_{\mathcal{U}} \\ &=_{P(f(y))} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_P(\mathsf{ap}_f(p)), u) && ;\ \mathsf{ap}_{P \circ f} = \mathsf{ap}_P \circ \mathsf{ap}_f \\ &=_{P(f(y))} \mathsf{transport}^P(\mathsf{ap}_f(p), u) && ;\ B \equiv P,\ p \equiv \mathsf{ap}_f(p). \end{aligned}$$

Here is an illustration. Compare with Remark 2.3.14.




[23] Notation 2.5.12 is sometimes used to identify $\mathsf{idtoeqv}(\mathsf{ap}_B(p)) : B(x) \simeq B(y)$ with its projection onto $B(x) \to B(y)$.

## Partial Summary

Before proceeding with other types, let us recap what we have seen so far.

| Type | Interface | Extensional Equality |
|---|---|---|
| $A \times B$ | $\pi_1$ and $\pi_2$ | $(\pi_1(u) = \pi_1(v)) \times (\pi_2(u) = \pi_2(v))$ |
| $\prod_{x:\,A} B(x)$ | Application: $f(x)$ | $\prod_{x:\,A} f(x) = g(x)$ |
| $\sum_{x:\,A} B(x)$ | $\pi_1$ and $\pi_2$ | $\sum_{p:\,\pi_1(u)=\pi_1(v)} p_*\pi_2(u) = \pi_2(v)$ |
| $\mathsf{1}$ | None | $x = \star$ |
| $\mathcal{U}$ | Equivalences | $A \simeq B$ |

## 2.14 Equality in Identity Types

The Observational Equality Framework 2.6.1 aims to implement *structural decomposition*: it characterizes the identity type of a compound type (such as $\Sigma$-types, Cartesian products, or $\Pi$-types) in terms of the identity types of its components.

The framework cannot be applied to an identity type itself. An identity type $a =_A a'$ presents no structural components to unpack. The type of paths between paths, $p =_{a=_A a'} q$, is instead part of the higher homotopy structure of the base type $A$.

Even if we fix a generic base type $A$, there is no universal formula to characterize its 2-paths. The complexity of these iterated identity types is precisely what gives the theory its rich homotopical character. We can only provide an observational characterization of $p =_{a=_A a'} q$ when $A$ is instantiated with a concrete type constructor (such as $B \times C$ or $\mathcal{U}$). In those cases, we simply re-apply the structural theorems already established for those specific types.

Nevertheless, we can use the Observational Equality Framework 2.6.1 to make the following

**Remark 2.14.1.** Suppose that, for some type $A$, we have a full characterization of $a =_A a'$ given by the equivalence

$$\mathsf{encode}\colon (a =_A a') \simeq \mathsf{Code}(a, a').$$

Then, by Theorem 2.5.18, the function application

$$\mathsf{ap}_{\mathsf{encode}}\colon (p =_{a=_A a'} q) \to (\mathsf{encode}(p) =_{\mathsf{Code}(a,a')} \mathsf{encode}(q))$$

is also an equivalence. Therefore, the equality $p =_{a=_A a'} q$ between paths $p, q\colon a =_A a'$ is fully characterized by the equality of their encoded representations in the structurally simpler type $\mathsf{Code}(a, a')$.

Before working the details of this observation in concrete cases, we need two results.

**Lemma 2.14.2.** [Transport in Identity Types] *Let $A$ and $B$ be types, and $f, g\colon A \to B$ functions. Consider the type family $P(x) :\equiv f(x) =_B g(x)$. Then, for any path $\gamma\colon x =_A y$ and any path $q\colon P(x)$, the transport of $q$ along $\gamma$ is given by*

$$\mathsf{transport}^P(\gamma, q) =_{P(y)} \mathsf{ap}_f(\gamma)^{-1} \cdot q \cdot \mathsf{ap}_g(\gamma).$$

*Proof.* The LHS of the equality inhabits $P(y)$. This is also the case with the RHS:

$$\underbrace{\mathsf{ap}_f(\gamma)^{-1}}_{f(y)=_B f(x)} \cdot \underbrace{q}_{f(x)=_B g(x)} \cdot \underbrace{\mathsf{ap}_g(\gamma)}_{g(x)=_B g(y)} : P(y).$$

Therefore, we can apply path induction on $\gamma$. It suffices to assume that $y \equiv x$ and $\gamma \equiv \mathsf{refl}_x$.

$$\begin{aligned}
\text{LHS} &\equiv \mathsf{transport}^P(\mathsf{refl}_x, q) \\
&\equiv q, \\
\text{RHS} &\equiv \mathsf{ap}_f(\mathsf{refl}_x)^{-1} \cdot q \cdot \mathsf{ap}_g(\mathsf{refl}_x) \\
&\equiv \mathsf{refl}_{f(x)} \cdot q \cdot \mathsf{refl}_{g(x)} \\
&=_{P(x)} q.
\end{aligned}$$

Since $\mathsf{refl}_q$ inhabits the identity type, the proof is complete. □

**Theorem 2.14.3.** *Let $A$ be a type and $B\colon A \to \mathcal{U}$ a type family. For any two points $\omega, \omega'\colon \Sigma_A B$ and any two paths $p, q\colon \omega =_{\Sigma_A B} \omega'$, let*

$$a :\equiv \pi_1(\omega), \quad u :\equiv \pi_2(\omega) \quad \textit{and} \quad a' :\equiv \pi_1(\omega'), \quad u' :\equiv \pi_2(\omega').$$

*Then, there is an equivalence*

$$\varepsilon\colon (p =_{\omega=_{\Sigma_A B}\omega'} q) \simeq \sum_{\alpha\colon p_1 = q_1} \mathsf{transport}^P(\alpha, p_2) =_{P(q_1)} q_2,$$

*where*

$$p_1 :\equiv \mathsf{ap}_{\pi_1}(p), \quad p_2 :\equiv \mathsf{apd}_{\pi_2}(p) \quad \textit{and} \quad q_1 :\equiv \mathsf{ap}_{\pi_1}(q), \quad q_2 :\equiv \mathsf{apd}_{\pi_2}(q),$$

*and the type family $P\colon (a =_A a') \to \mathcal{U}$ for $r\colon a =_A a'$ is defined as*

$$P(r) :\equiv T(r) =_{B(a')} u', \quad \textit{with} \quad T(r) :\equiv \mathsf{transport}^B(r, u).$$

*Proof.* Let us denote the total space of the encoded identity type as

$$E :\equiv \sum_{r\colon a =_A a'} P(r).$$

Section 2.9 Equality in Σ-Types establishes an equivalence

$$\mathsf{encode}\colon (\omega =_{\Sigma_A B} \omega') \xrightarrow{\simeq} E.$$

Theorem 2.5.18 applied to $\mathsf{encode}$ yields an equivalence:

$$\mathsf{ap}_{\mathsf{encode}}\colon (p =_{\omega=_{\Sigma_A B}\omega'} q) \xrightarrow{\simeq} (\mathsf{encode}(p) =_E \mathsf{encode}(q)),$$

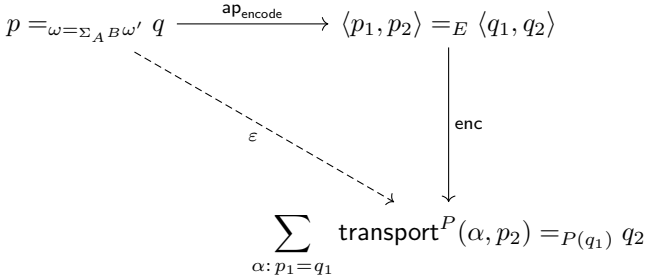
where $\mathsf{encode}(p) \equiv \langle p_1, p_2\rangle$ and $\mathsf{encode}(q) \equiv \langle q_1, q_2\rangle$. Hence:

$$\mathsf{ap}_{\mathsf{encode}}\colon (p =_{\omega=_{\Sigma_A B}\omega'} q) \xrightarrow{\simeq} (\langle p_1, p_2\rangle =_E \langle q_1, q_2\rangle).$$

Applying the characterization of equality in Σ-types a second time, this time to the Σ-type $E$ itself, we obtain a second encoding:

$$\mathsf{enc}\colon \big(\langle p_1, p_2\rangle =_E \langle q_1, q_2\rangle\big) \simeq \sum_{\alpha\colon p_1 =_{a=_A a'} q_1} \mathsf{transport}^P(\alpha, p_2) =_{P(q_1)} q_2.$$

Composing $\mathsf{ap}_{\mathsf{encode}}$ with $\mathsf{enc}$ yields the desired equivalence:




□

Here are examples of Remark 2.14.1.

a) **Product Types:** Consider two paths $p, q\colon \omega =_{A\times B} \omega'$ in the product type $A \times B$. Since the identity type $\omega =_{A\times B} \omega'$ is equivalent to the type of pairs of paths $(\pi_1(\omega) =_A \pi_1(\omega')) \times (\pi_2(\omega) =_B \pi_2(\omega'))$, the identity type $p =_{\omega=_{A\times B}\omega'} q$ is equivalent to the product of their respective identity types. Thus, an identity between $p$ and $q$ is equivalent to a pair of identities:

$$\mathsf{ap}_{\pi_1}(p) =_{\pi_1(\omega)=_A \pi_1(\omega')} \mathsf{ap}_{\pi_1}(q)$$

and

$$\mathsf{ap}_{\pi_2}(p) =_{\pi_2(\omega)=_B \pi_2(\omega')} \mathsf{ap}_{\pi_2}(q).$$

b) **Dependent Pair Types:** For a dependent pair type $\Sigma_A B$, consider two paths $p, q\colon \omega =_{\Sigma_A B} \omega'$. Let us denote the projections of the endpoints as

$$a :\equiv \pi_1(\omega), \quad u \quad :\equiv \pi_2(\omega)$$

and

$$a' :\equiv \pi_1(\omega'), \quad u' :\equiv \pi_2(\omega').$$

For any path $r\colon a =_A a'$, let us define the transport function

$$T(r) :\equiv \mathsf{transport}^B(r, u).$$

The projected paths of the endpoints are then given by

$$\begin{aligned} p_1, q_1 &\colon a =_A a' \\ p_2 &\colon T(p_1) =_{B(a')} u', \\ q_2 &\colon T(q_1) =_{B(a')} u'. \end{aligned}$$

As Theorem 2.14.3 establishes, calculating the equality between paths $p$ and $q$ reduces to providing a pair of identities $(\alpha, \beta)$, where

$$\begin{aligned} \alpha &\colon p_1 =_{a=_A a'} q_1 \\ \beta &\colon \mathsf{transport}^P(\alpha, p_2) =_{P(q_1)} q_2, \end{aligned}$$

with the identity family

$$\begin{aligned} P &\colon a =_A a' \to \mathcal{U} \\ P(r) &:\equiv T(r) =_{B(a')} u'. \end{aligned}$$

To understand the geometric meaning of $\beta$, we can compute the action of transport in the family $P$.

Applying Lemma 2.14.2 to our case with

$$\begin{aligned} x &:\equiv p_1 \\ y &:\equiv q_1 \\ \gamma &:\equiv \alpha\colon p_1 = q_1 \\ q &:\equiv p_2\colon T(p_1) = u' \\ f &:\equiv T\colon (a =_A a') \to B(a') \\ g &:\equiv r \mapsto u' \end{aligned}$$

yields

$$\begin{aligned} \mathsf{transport}^P(\alpha, p_2) &=_{P(q_1)} \mathsf{ap}_T(\alpha)^{-1} \cdot p_2 \cdot \mathsf{ap}_{r \mapsto u'}(\alpha) \\ &=_{P(q_1)} \mathsf{ap}_T(\alpha)^{-1} \cdot p_2 \cdot \mathsf{refl}_{u'} && \text{; Lem. 2.1.2} \\ &=_{P(q_1)} \mathsf{ap}_T(\alpha)^{-1} \cdot p_2. \end{aligned}$$

Concatenating the inverse of this computation with $\beta$ gives

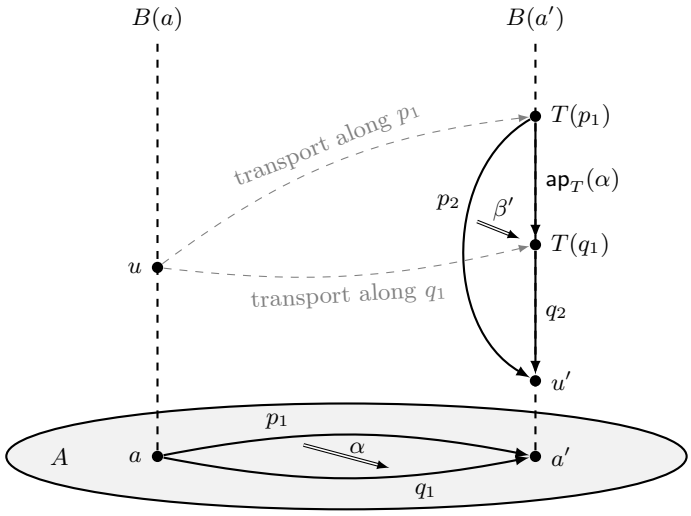
$$\beta' : \mathsf{ap}_T(\alpha)^{-1} \cdot p_2 =_{P(q_1)} q_2.$$

Redefining $\beta'$ as $\mathsf{ap}_T(\alpha) \cdot \beta'$ yields

$$\beta' : p_2 =_{P(q_1)} \mathsf{ap}_T(\alpha) \cdot q_2.$$

Since the composition $(\alpha, \beta) \mapsto (\alpha, \beta')$ is an equivalence, we obtain as result a decomposition of the 2-path $p =_{\omega =_{\Sigma_A B} \omega'} q$ into a 2-path $\alpha$ in the base type $A$ and a 2-path $\beta'$ in the target fiber $B(a')$, mirroring the decomposition of 1-paths in the total space.

The following picture illustrates this geometric construction. The direct path $p_2$ and the concatenated indirect path $\mathsf{ap}_T(\alpha) \cdot q_2$ form a triangle within the target fiber, and $\beta'$ acts as the homotopy between them.




c) **Function Types:** For a dependent function type $\Pi_A B :\equiv \prod_{x:\, A} B(x)$, consider two paths $p, q : f =_{\Pi_A B} g$. By function extensionality, the identity type $f =_{\Pi_A B} g$ is equivalent to the homotopy type $\prod_{x:\, A} f(x) =_{B(x)} g(x)$ via the function $\mathsf{happly}$. Therefore, an identity $p =_{f =_{\Pi_A B} g} q$ is equivalent to a second-order homotopy between their applications:

$$\prod_{x:\, A} \mathsf{happly}(p)(x) =_{f(x) =_{B(x)} g(x)} \mathsf{happly}(q)(x).$$

## Transport in Identity Families

When the Observational Equality Framework encodes a path, it usually generates a type containing the transport operator. For instance, when encoding an equality in dependent pairs, the second coordinate is a 2-path of type

$\mathsf{transport}^P(\alpha, p_2) = q_2$. This expression is "opaque" because $\mathsf{transport}$ is a primitive. It asserts that a path is moved between fibers to match types, but it does not specify what the resulting path looks like in terms of simpler operations.

To make sense of these abstract 2-paths, we must evaluate how paths are transported within families of the form $E(x) :\equiv f(x) = g(x)$.

The following results establish computational rules for these transport operations. Beginning with simple identity families where one or both endpoints are fixed, we then consider families defined by functions into a fixed codomain, and finally address the general case of dependent functions. These rules provide the computational engine that allows us to translate abstract transport terms into concrete path concatenations.

**Lemma 2.14.4.** *Let $A$ be a type, $a, x_1, x_2 : A$, and consider a path $p : x_1 =_A x_2$. Then, we have the following:*

a) *Let $E_1 : A \to \mathcal{U}$ be the type family $E_1(x) :\equiv a =_A x$. Then,*

   i) *Given $q : a =_A x_1$, transport evaluates to post-composition by $p$:*

$$\mathsf{transport}^{E_1}(p, q) =_{a =_A x_2} q \cdot p.$$

   ii) *Post-composition with $p$ is an equivalence $(a =_A x_1) \simeq (a =_A x_2)$.*

b) *Let $E_2 : A \to \mathcal{U}$ be the type family $E_2(x) :\equiv x =_A a$.*

   i) *Given $q : x_1 =_A a$, transport evaluates to pre-composition by $p^{-1}$:*

$$\mathsf{transport}^{E_2}(p, q) =_{x_2 =_A a} p^{-1} \cdot q.$$

   ii) *Pre-composition by $p^{-1}$ is an equivalence $(x_1 =_A a) \simeq (x_2 =_A a)$.*

c) *Let $E_3 : A \to \mathcal{U}$ be the type family $E_3(x) :\equiv x =_A x$.*

   i) *Given $q : x_1 =_A x_1$, transport evaluates to conjugation by $p$:*

$$\mathsf{transport}^{E_3}(p, q) =_{x_2 =_A x_2} p^{-1} \cdot q \cdot p.$$

   ii) *Conjugation by $p$ is an equivalence $(x_1 =_A x_1) \simeq (x_2 =_A x_2)$.*

*Proof.* In the first case, both sides inhabit $a =_A x_2$. In the second, they inhabit $x_2 =_A a$. In the third, $x_2 =_A x_2$. Therefore, all types are verified.

By path induction on $p$ we can assume that $x_2 \equiv x_1$ and $p \equiv \mathsf{refl}_{x_1}$. In this case, each of the transport operations on the left-hand sides evaluates definitionally to $q$. On the right-hand side, the expressions reduce propositionally (with the second case reducing definitionally) to $q$ by the unit laws for path concatenation.

Since the equations hold, the corresponding operations inherit the equivalence property of Theorem 2.5.17. □

**Theorem 2.14.5.** *Let $A$ be a type, $B\colon A \to \mathcal{U}$ a type family, and consider two dependent functions $f, g\colon \prod_{x:A} B(x)$. For any path $p\colon a =_A a'$ and any path $q\colon f(a) =_{B(a)} g(a)$, transport in the identity family $E(x) :\equiv f(x) =_{B(x)} g(x)$ computes as*

$$\mathsf{transport}^E(p, q) =_{f(a')=_{B(a')}g(a')} \mathsf{apd}_f(p)^{-1} \cdot \mathsf{ap}_{\mathsf{transport}^B(p)}(q) \cdot \mathsf{apd}_g(p).$$

*Proof.* The type of the LHS is clearly $f(a') =_{B(a')} g(a')$. For the RHS we have

$$\begin{aligned} \mathsf{apd}_f(p)^{-1}&\colon f(a') =_{B(a')} p_*(f(a)) \\ \mathsf{transport}^B(p)&\colon B(a) \to B(a') \\ \mathsf{ap}_{\mathsf{transport}^B(p)}(q)&\colon p_*(f(a)) =_{B(a')} p_*(g(a)) \\ \mathsf{apd}_g(p)&\colon p_*(g(a)) =_{B(a')} g(a'). \end{aligned}$$

By path induction on $p$, we may assume that $a' \equiv a$ and $p \equiv \mathsf{refl}_a$. In this case, the transport operation on the LHS evaluates definitionally to $q$. On the RHS, the dependent action on paths $\mathsf{apd}$ for both $f$ and $g$ evaluates to $\mathsf{refl}_{f(a)}$ and $\mathsf{refl}_{g(a)}$, respectively. Furthermore, transport along $\mathsf{refl}_a$ in $B$ is definitionally the identity function, so its action on the path $q$ is simply $q$. The expression reduces to $\mathsf{refl}_{f(a)}^{-1} \cdot q \cdot \mathsf{refl}_{g(a)}$, which propositionally reduces to $q$ by the unit laws for path concatenation. □

## 2.15 Equality in Coproducts

To apply the observational framework 2.6.1 to the coproduct $A + B$, we proceed as follows:

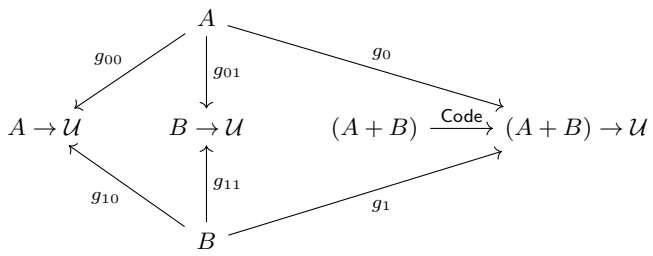
CODE FAMILY. By nesting the recursion principle $\mathsf{rec}_{A+B}$, we obtain

$$\begin{aligned} g_{00}&\colon A \to A \to \mathcal{U} \\ &\quad a \mapsto (a' \mapsto a =_A a') \\ g_{01}&\colon A \to B \to \mathcal{U} \\ &\quad a \mapsto (b \mapsto \mathsf{0}) \\ g_0&\colon A \to (A + B) \to \mathcal{U} \\ &\quad a \mapsto \mathsf{rec}_{A+B}(\mathcal{U}, g_{00}(a), g_{01}(a)) \end{aligned} \qquad \begin{aligned} g_{10}&\colon B \to A \to \mathcal{U} \\ &\quad b \mapsto (a \mapsto \mathsf{0}) \\ g_{11}&\colon B \to B \to \mathcal{U} \\ &\quad b \mapsto (b' \mapsto b =_B b') \\ g_1&\colon B \to (A + B) \to \mathcal{U} \\ &\quad b \mapsto \mathsf{rec}_{A+B}(\mathcal{U}, g_{10}(b), g_{11}(b)). \end{aligned}$$

Then, we apply the recursion principle one final time to define the outer family:

$$\begin{aligned} \mathsf{Code}&\colon (A + B) \to (A + B) \to \mathcal{U} \\ &\quad z \mapsto \mathsf{rec}_{A+B}((A + B) \to \mathcal{U}, g_0, g_1, z), \end{aligned}$$

which can be represented by:




More commonly in practice, the Code function can be defined using pattern matching on the four constructors of $A + B$:

$$\begin{aligned}
&\mathsf{Code}\colon (A+B) \to (A+B) \to \mathcal{U}\\
&\quad \mathsf{Code}(\mathsf{inl}(x), \mathsf{inl}(y)) :\equiv x =_A y\\
&\quad \mathsf{Code}(\mathsf{inr}(x), \mathsf{inr}(y)) :\equiv x =_B y\\
&\quad \mathsf{Code}(\mathsf{inl}(x), \mathsf{inr}(y)) :\equiv \mathsf{0}\\
&\quad \mathsf{Code}(\mathsf{inr}(x), \mathsf{inl}(y)) :\equiv \mathsf{0}.
\end{aligned}$$

Base code. Define the dependent function

$$c\colon \prod_{z\colon A+B} \mathsf{Code}(z, z)$$

by induction on $z$. The motive is the type-valued function

$$C :\equiv w \mapsto \mathsf{Code}(w, w).$$

For the base cases, specify:

$$f_0\colon \prod_{x\colon A} \mathsf{Code}(\mathsf{inl}(x), \mathsf{inl}(x)) \qquad f_1\colon \prod_{y\colon B} \mathsf{Code}(\mathsf{inr}(y), \mathsf{inr}(y))$$
$$f_0(x) :\equiv \mathsf{refl}_x \qquad f_1(y) :\equiv \mathsf{refl}_y.$$

Then, define

$$c(z) :\equiv \mathsf{ind}_{A+B}(C, f_0, f_1, z).$$

In other words,

$$c(z) :\equiv \begin{cases} \mathsf{refl}_x & \text{if } z \equiv \mathsf{inl}(x),\\ \mathsf{refl}_y & \text{if } z \equiv \mathsf{inr}(y). \end{cases}$$

Encode function. We have to define

$$\mathsf{encode}\colon \prod_{z,w\colon A+B} (z =_{A+B} w) \to \mathsf{Code}(z, w).$$

Fix $z\colon A+B$. Then, $C_z :\equiv \mathsf{Code}(z,-)$ and

$$\begin{aligned}\mathsf{encode}(z,w)\colon (z =_{A+B} w) &\to \mathsf{Code}(z,w)\\ p &\mapsto \mathsf{transport}^{C_z}(p, c(z)).\end{aligned}$$

In the case where $w \equiv z$ and $p \equiv \mathsf{refl}_z$, we have

$$\mathsf{encode}(z,w)(\mathsf{refl}_z) \equiv c(z).$$

DECODE FUNCTION. We have to define

$$\mathsf{decode}\colon \prod_{z,w\colon A+B} \mathsf{Code}(z,w) \to (z =_{A+B} w)$$

By double induction on $z, w\colon A+B$, we define

$$\begin{aligned}\mathsf{decode}\colon \prod_{z,w\colon A+B} &\mathsf{Code}(z,w) \to (z =_{A+B} w)\\ &(\mathsf{inl}(x), \mathsf{inl}(u)) \mapsto \mathsf{ap}_{\mathsf{inl}}\\ &(\mathsf{inr}(y), \mathsf{inr}(v)) \mapsto \mathsf{ap}_{\mathsf{inr}}\\ &(\mathsf{inl}(x), \mathsf{inr}(v)) \mapsto \mathsf{rec}_{\mathbf{0}}(\mathsf{inl}(x) =_{A+B} \mathsf{inr}(v))\\ &(\mathsf{inr}(y), \mathsf{inl}(u)) \mapsto \mathsf{rec}_{\mathbf{0}}(\mathsf{inr}(y) =_{A+B} \mathsf{inl}(u)).\end{aligned}$$

To verify that $\mathsf{decode}(z,w)$ is a right inverse of $\mathsf{encode}(z,w)$ for the left constructor, we must show that for any path $q\colon x =_A u$,

$$\mathsf{encode}(\mathsf{decode}(\mathsf{inl}(x), \mathsf{inl}(u), q)) =_{x=_A u} q.$$

Since $\mathsf{ap}_{\mathsf{inl}}(q)\colon \mathsf{inl}(x) =_{A+B} \mathsf{inl}(u)$, we have

$$\begin{aligned}\text{LHS} &\equiv \mathsf{encode}(\mathsf{ap}_{\mathsf{inl}}(q))\\ &\equiv \mathsf{transport}^{C_{\mathsf{inl}(x)}}(\mathsf{ap}_{\mathsf{inl}}(q), \mathsf{refl}_x).\end{aligned}$$

Based path induction on $q$ reduces us to the case $u \equiv x$, $q \equiv \mathsf{refl}_x$. Here the LHS becomes $\mathsf{refl}_x$, which is definitionally equal to $q$.

The verification for the right constructor is similar.

For the mixed case $z \equiv \mathsf{inl}(x)$ and $w \equiv \mathsf{inr}(v)$, the code $q$ inhabits the empty type, meaning $q\colon \mathbf{0}$. We must formally construct a term of type

$$\mathsf{encode}(\mathsf{inl}(x), \mathsf{inr}(v), \mathsf{decode}(\mathsf{inl}(x), \mathsf{inr}(v), q)) =_{\mathbf{0}} q.$$

We proceed by induction on $q$ using the empty type eliminator $\mathsf{ind}_{\mathbf{0}}$. We define our motive $P\colon \mathbf{0} \to \mathcal{U}$ as the identity family:

$$P(q) :\equiv \mathsf{encode}(\mathsf{inl}(x), \mathsf{inr}(v), \mathsf{decode}(\mathsf{inl}(x), \mathsf{inr}(v), q)) =_{\mathbf{0}} q.$$

Applying the eliminator to our motive $P$ and our element $q\colon \mathsf{0}$ directly yields the required proof term:

$$\mathsf{ind}_{\mathsf{0}}(P, q)\colon \mathsf{encode}(\mathsf{inl}(x), \mathsf{inr}(v), \mathsf{decode}(\mathsf{inl}(x), \mathsf{inr}(v), q)) =_{\mathsf{0}} q.$$

The symmetric mixed case is proven similarly.

For the Extended framework 2.6.2 we have to consider

INVERSION MAP: Defined by

$$\begin{aligned}\mathsf{inv}\colon \mathsf{Code}(z, w) &\to \mathsf{Code}(w, z)\\ p &\mapsto p^{-1}\end{aligned}$$

CONCATENATION: Defined by

$$\begin{aligned}*\colon \mathsf{Code}(\mathsf{x}, \mathsf{y} \to \mathsf{Code}(y, z) &\to \mathsf{Code}(x, z)\\ p * q &:\equiv p \cdot q.\end{aligned}$$

UNIT LAWS: Since $c(z)$ is a reflection path, we have:

$$\mathsf{inv}(c(z)) \equiv c(z)$$

and

$$c(z) * q \equiv q.$$

**Lemma 2.15.1.** *The type*

$$\prod_{x,y\colon A+B} \prod_{\omega\colon \mathsf{Code}(x,y)} P(x, y, \omega)$$

*is inhabited if, and only if, the type*

$$\Big(\prod_{a\colon A} P(\mathsf{inl}(a), \mathsf{inl}(a), \mathsf{refl}_a)\Big) \times \Big(\prod_{b\colon B} P(\mathsf{inr}(b), \mathsf{inr}(b), \mathsf{refl}_b)\Big)$$

*has a witness.*

*Proof.* By induction on $x$ and $y$, we may assume that both are constructed via the injections from $A$ or $B$. Since we are quantifying over an element $\omega\colon \mathsf{Code}(x, y)$, we may disregard the mixed cases. Thus, we can assume that both injections come from $A$ or both come from $B$. The goal becomes

$$\Big(\prod_{a,a'\colon A} \prod_{p\colon a =_A a'} P(\mathsf{inl}(a), \mathsf{inl}(a'), p)\Big) \times \Big(\prod_{b,b'\colon B} \prod_{q\colon b =_B b'} P(\mathsf{inr}(b), \mathsf{inr}(b'), q)\Big).$$

The conclusion then follows by path induction on $p$ and $q$. □

**Proposition 2.15.2.** *Let $A, B : \mathcal{U}$ be types. Given $x, y, z : A + B$ we have*

a) $\mathsf{decode}(x, x, c(x)) \equiv c(z)$

b) $\mathsf{decode}(y, x, q^{-1}) =_{y =_{A+B} x} \mathsf{decode}(x, y, q)^{-1}$

c) $\mathsf{decode}(x, z, q \cdot r) =_{x =_{A+B} z} \mathsf{decode}(x, y, q) \cdot \mathsf{decode}(y, z, r)$.

*Proof.* This is a direct consequence of Theorem 2.6.5.[24] □

**Theorem 2.15.3.** *Let $X$ be a type and $A, B : X \to \mathcal{U}$ type families. Then, given a path $p : x_1 =_X x_2$, we have*

$$\mathsf{transport}^{A+B}(p, \mathsf{inl}(a)) =_{A(x_2)+B(x_2)} \mathsf{inl}(\mathsf{transport}^{A}(p, a)),$$
$$\mathsf{transport}^{A+B}(p, \mathsf{inr}(b)) =_{A(x_2)+B(x_2)} \mathsf{inr}(\mathsf{transport}^{B}(p, b)),$$

*where $(A + B)(x) :\equiv A(x) + B(x)$ for $x : X$.*

*Proof.* Clearly both sides of both equalities are of type $A(x_2)+B(x_2)$. Therefore, we can proceed by induction on $p$. This reduces ourselves to the case $x_2 \equiv x_1$ and $p \equiv \mathsf{refl}_{x_1}$. Then all transport become identities and the equalities are trivially inhabited, the first by $\mathsf{refl}_{\mathsf{inl}(a)}$ and the second by $\mathsf{refl}_{\mathsf{inr}(b)}$. □

**Theorem 2.15.4.** *Let $f : A \to A'$ and $g : B \to B'$ be two functions.*

a) *For all $x_0, y_0 : A$ and paths $p : \mathsf{inl}(x_0) =_{A+B} \mathsf{inl}(y_0)$, we have*

$$\mathsf{ap}_{f+g}(p) =_{\mathsf{inl}(f(x_0))=_{A'+B'}\mathsf{inl}(f(y_0))} \mathsf{decode}(\mathsf{ap}_f(\mathsf{encode}(p))).$$

b) *For all $x_1, y_1 : B$ and paths $q : \mathsf{inr}(x_1) =_{A+B} \mathsf{inr}(y_1)$, we have*

$$\mathsf{ap}_{f+g}(q) =_{\mathsf{inr}(g(x_1))=_{A'+B'}\mathsf{inr}(g(y_1))} \mathsf{decode}(\mathsf{ap}_g(\mathsf{encode}(q))).$$

c) *For all $x_0 : A$, $y_1 : B$, and paths $r : \mathsf{inl}(x_0) =_{A+B} \mathsf{inr}(y_1)$, we have*

$$\mathsf{ap}_{f+g}(r) =_{\mathsf{inl}(f(x_0))=_{A'+B'}\mathsf{inr}(g(y_1))} \mathsf{ind}_0(\mathsf{encode}(r)).$$

d) *For all $x_1 : B$, $y_0 : A$, and paths $s : \mathsf{inr}(x_1) =_{A+B} \mathsf{inl}(y_0)$, we have*

$$\mathsf{ap}_{f+g}(s) =_{\mathsf{inr}(g(x_1))=_{A'+B'}\mathsf{inl}(f(y_0))} \mathsf{ind}_0(\mathsf{encode}(s)).$$

*Proof.*

[24] Parts b) and c) can also be easily deduced from Lemma 2.15.1.

a) By path induction on $p$, we may assume $y_0 \equiv x_0$ and $p \equiv \mathsf{refl}_{\mathsf{inl}(x_0)}$. Then,

$$\begin{aligned} \text{LHS} &\equiv \mathsf{refl}_{f+g(\mathsf{refl}_{\mathsf{inl}(x_0)})} \\ &\equiv \mathsf{refl}_{\mathsf{inl}(f(x_0))}. \\ \mathsf{encode}(p) &\equiv \mathsf{refl}_{x_0}. \\ \mathsf{ap}_f(\mathsf{encode}(p)) &\equiv \mathsf{refl}_{f(x_0)}. \\ \text{RHS} &\equiv \mathsf{refl}_{\mathsf{inl}(f(x_0))}. \end{aligned}$$

b) Similarly, by path induction on $q$ .

c) Since $r\colon \mathsf{inl}(x_0) =_{A+B} \mathsf{inr}(y_1)$ is a path between distinct constructors, $\mathsf{encode}(r)$ yields an element of the empty type $\mathsf{0}$. The equality is therefore vacuously inhabited via $\mathsf{ind}_{\mathsf{0}}$.

d) Similar to part c).

□

## 2.16 Equality in Natural Numbers

In this section we apply the Observational Framework 2.6.1 to the type of natural numbers.

Code family. We define the family $\mathsf{Code}\colon \mathbb{N} \to \mathbb{N} \to \mathcal{U}$ formally using nested recursion.

For the base case, we define

$$e_0 \colon \mathbb{N} \to \mathcal{U}; \quad e_0 :\equiv \mathsf{rec}_{\mathbb{N}}(\mathcal{U}, e_{00}, e_{0s}),$$

where

$$\begin{aligned} &e_{00} \colon \mathcal{U} && e_{0s} \colon \mathbb{N} \to \mathcal{U} \to \mathcal{U} \\ &e_{00} :\equiv \mathsf{1} && (m, X) \mapsto \mathsf{0}. \end{aligned}$$

By definition, $e_0$ satisfies

$$e_0(0) \equiv \mathsf{1} \quad \text{and} \quad e_0(\mathsf{succ}(m)) \equiv \mathsf{0}.$$

For the inductive step, we define the function

$$e_s \colon \mathbb{N} \to (\mathbb{N} \to \mathcal{U}) \to (\mathbb{N} \to \mathcal{U}).$$

Given $n\colon \mathbb{N}$ and a function $e\colon \mathbb{N} \to \mathcal{U}$ (representing the recursive evaluation at $n$), we define the next function $e_s(n, e)$ by recursion on its argument:

$$e_s(n, e) :\equiv \mathsf{rec}_{\mathbb{N}}(\mathcal{U}, e_{s0}, e_{ss}),$$

where

$$\begin{aligned} e_{s0} &: \mathcal{U} & e_{ss} &: \mathbb{N} \to \mathcal{U} \to \mathcal{U} \\ e_{s0} &:\equiv \mathsf{0} & (m, Y) &\mapsto e(m). \end{aligned}$$

By definition, we have $e_s(n, e)(0) \equiv \mathsf{0}$ and

$$\begin{aligned} e_s(n, e)(\mathsf{succ}(m)) &\equiv \mathsf{rec}_{\mathbb{N}}(\mathcal{U}, e_{s0}, e_{ss}, \mathsf{succ}(m)) \\ &\equiv e_{ss}(m, \mathsf{rec}_{\mathbb{N}}(\mathcal{U}, e_{s0}, e_{ss}, m)) \\ &\equiv e_{ss}(m, \underbrace{e_s(n, e)(m)}_{Y}) \\ &\equiv e(m). \end{aligned}$$

Finally, we define $\mathsf{Code}$ by applying the outer recursor with motive $\mathbb{N} \to \mathcal{U}$:

$$\mathsf{Code} :\equiv \mathsf{rec}_{\mathbb{N}}(\mathbb{N} \to \mathcal{U}, e_0, e_s).$$

In particular, we have

$$\begin{aligned} \mathsf{Code}(0, 0) &\equiv e_0(0) \equiv \mathsf{1} \\ \mathsf{Code}(0, \mathsf{succ}(m)) &\equiv e_0(\mathsf{succ}(m)) \equiv \mathsf{0} \\ \mathsf{Code}(\mathsf{succ}(n), 0) &\equiv e_s(n, \mathsf{Code}(n))(0) \\ &\equiv e_{s0} \\ &\equiv \mathsf{0} \\ \mathsf{Code}(\mathsf{succ}(n), \mathsf{succ}(m)) &\equiv e_s(n, \mathsf{Code}(n))(\mathsf{succ}(m)) \\ &\equiv \mathsf{Code}(n)(m) \\ &\equiv \mathsf{Code}(n, m). \end{aligned}$$

Base code. To define the base code we need the following

**Lemma 2.16.1.** *Given $n : \mathbb{N}$, the type $\mathsf{Code}(n, n) =_{\mathcal{U}} \mathsf{1}$ is inhabited.*

*Proof.* We proceed by induction on $n$, with the motive $C : \mathbb{N} \to \mathcal{U}$ specified as

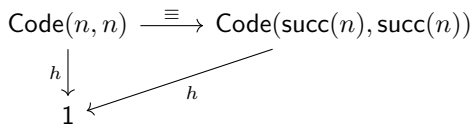
$$C(n) :\equiv \mathsf{Code}(n, n) =_{\mathcal{U}} \mathsf{1}.$$

For the base case we can take $\mathsf{refl}_{\mathsf{1}}$.

Given a witness of $C(n)$, i.e., a path $h : \mathsf{Code}(n, n) =_{\mathcal{U}} \mathsf{1}$, we define the step function

$$\begin{aligned} e_s &: \prod_{n:\mathbb{N}} C(n) \to C(\mathsf{succ}(n)) \\ e_s(n, h) &:\equiv h, \end{aligned}$$

which is well defined as the following diagram of paths demonstrates




Consequently, we have

$$e(n) :\equiv \mathsf{ind}_{\mathbb{N}}(C, \mathsf{refl}_1, e_s, n) \colon \mathsf{Code}(n,n) =_{\mathcal{U}} 1.$$

□

Note that, given $n \colon \mathbb{N}$, to define the base code $c(n) \colon \mathsf{Code}(n,n)$, we cannot simply select $\star \colon 1$ because that would not type-check. Instead, we need to transport $\star$ as follows:

$$c(n) :\equiv \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(e(n)^{-1}, \star),$$

where $e(n) \colon \mathsf{Code}(n,n) =_{\mathcal{U}} 1$ is the witness given by the lemma. In particular,

$$e(0) \equiv \mathsf{refl}_1 \quad \text{and} \quad c(0) \equiv \star.$$

More generally,

$$\begin{aligned} e(\mathsf{succ}(n)) &\equiv \mathsf{ind}_{\mathbb{N}}(C, \mathsf{refl}_1, e_s, \mathsf{succ}(n)) \\ &\equiv e_s(n, e(n)) \\ &\equiv e(n), \end{aligned}$$

which implies that $c(\mathsf{succ}(n)) \equiv c(n)$.

Encode function. Define

$$\mathsf{encode} \colon \prod_{n,m \colon \mathbb{N}} (n =_{\mathbb{N}} m) \to \mathsf{Code}(n,m)$$
$$\mathsf{encode}(n,m,p) :\equiv \mathsf{transport}^{C_n}(p, c(n)).$$

In particular, given $p \colon 0 =_{\mathbb{N}} 0$, since $C_0(0) \equiv \mathsf{Code}(0,0) \equiv 1$, we have

$$\mathsf{transport}^{C_0}(p) \colon 1 \to 1$$

and so

$$\begin{aligned} \mathsf{encode}(0,0,p) &\equiv \mathsf{transport}^{C_0}(p, \star) \\ &=_1 \star, \end{aligned} \tag{2.1}$$

where the last propositional equality holds because every inhabitant of the unit type 1 is propositionally equal to $\star$ via $\mathsf{uniq}_1$.

DECODE FUNCTION. To define

$$\mathsf{decode}\colon \prod_{n,m\colon\mathbb{N}} \mathsf{Code}(n,m) \to (n =_{\mathbb{N}} m)$$

we proceed by nested induction.

For the base case, we take the motive

$$\begin{aligned} D_0 &\colon \mathbb{N} \to \mathcal{U} \\ D_0(m) &:\equiv \mathsf{Code}(0,m) \to (0 =_{\mathbb{N}} m), \end{aligned}$$

and define

$$d_0 \colon \prod_{m\colon\mathbb{N}} D_0(m); \quad d_0 :\equiv \mathsf{ind}_{\mathbb{N}}(D_0, d_{00}, d_{0s}),$$

where

$$\begin{aligned} d_{00} &\colon \mathsf{Code}(0,0) \to (0 =_{\mathbb{N}} 0) \\ &\qquad \star \mapsto \mathsf{refl}_0 \\ d_{0s} &\colon \prod_{m\colon\mathbb{N}} D_0(m) \to D_0(\mathsf{succ}(m)) \\ &\qquad (m,d) \mapsto \mathsf{rec}_0(0 =_{\mathbb{N}} \mathsf{succ}(m)). \end{aligned}$$

In particular,

$$\begin{aligned} d_0(0) &\equiv \star \mapsto \mathsf{refl}_0 \\ d_0(\mathsf{succ}(m)) &\equiv d_{0s}(m, d_0(m)) \\ &\equiv \mathsf{rec}_0(0 =_{\mathbb{N}} \mathsf{succ}(m)). \end{aligned} \tag{2.2}$$

Now we introduce the motive, which fixes the variable $n$, by

$$\begin{aligned} D &\colon \mathbb{N} \to \mathcal{U} \\ D(n) &:\equiv \prod_{m\colon\mathbb{N}} \mathsf{Code}(n,m) \to (n =_{\mathbb{N}} m), \end{aligned}$$

and define

$$\begin{aligned} d_s &\colon \prod_{n\colon\mathbb{N}} D(n) \to D(\mathsf{succ}(n)) \\ d_s(n,h) &:\equiv \mathsf{ind}_{\mathbb{N}}(D_{nh}, d_{s0}, d_{ss}) \end{aligned}$$

where $h\colon D(n)$, $D_{nh}(m) :\equiv \mathsf{Code}(\mathsf{succ}(n), m) \to (\mathsf{succ}(n) =_{\mathbb{N}} m)$ and

$$\begin{aligned} d_{s0} &\colon D_{nh}(0) \\ d_{s0} &:\equiv \mathsf{rec}_0(\mathsf{succ}(n) =_{\mathbb{N}} 0) \\ d_{ss} &\colon \prod_{m\colon\mathbb{N}} D_{nh}(m) \to D_{nh}(\mathsf{succ}(m)) \\ d_{ss}(m,\omega,t) &:\equiv \mathsf{ap}_{\mathsf{succ}}(h(m,t)), \end{aligned}$$

as illustrated in the following diagram

$$\begin{array}{ccc} \mathsf{Code}(\mathsf{succ}(n), \mathsf{succ}(m)) & \xrightarrow{d_{ss}(m,\omega)} & \mathsf{succ}(n) =_{\mathbb{N}} \mathsf{succ}(m) \\ \equiv \Big\downarrow & & \Big\uparrow \mathsf{ap}_{\mathsf{succ}} \\ \mathsf{Code}(n, m) & \xrightarrow[h(m)]{} & n =_{\mathbb{N}} m \end{array}$$

In particular,

$$\begin{aligned} d_s(n, h, 0) &\equiv \mathsf{rec}_0(\mathsf{succ}(n) =_{\mathbb{N}} 0) \\ d_s(n, h, \mathsf{succ}(m), t) &\equiv d_{ss}(m, d_s(n, h, m), t) \\ &\equiv \mathsf{ap}_{\mathsf{succ}}(h(m, t)) \end{aligned} \tag{2.3}$$

This allows us to define

$$\mathsf{decode} :\equiv \mathsf{ind}_{\mathbb{N}}(D, d_0, d_s).$$

From equations (2.2) and (2.3), it follows

$$\begin{aligned} \mathsf{decode}(0, 0, \star) &\equiv d_0(0, \star) \\ &\equiv \mathsf{refl}_0. \\ \mathsf{decode}(0, \mathsf{succ}(m)) &\equiv d_0(\mathsf{succ}(m)) \\ &\equiv \mathsf{rec}_0(0 =_{\mathbb{N}} \mathsf{succ}(m)). \\ \mathsf{decode}(\mathsf{succ}(n), 0) &\equiv d_s(n, \mathsf{decode}(n), 0) \\ &\equiv \mathsf{rec}_0(\mathsf{succ}(n) =_{\mathbb{N}} 0). \\ \mathsf{decode}(\mathsf{succ}(n), \mathsf{succ}(m), t) &\equiv d_s(n, \mathsf{decode}(n), \mathsf{succ}(m), t) \\ &\equiv \mathsf{ap}_{\mathsf{succ}}(\mathsf{decode}(n, m, t)). \end{aligned} \tag{2.4}$$

**Theorem 2.16.2.** *For $n, m : \mathbb{N}$, we have*

$$\mathsf{encode}(n, m) \circ \mathsf{decode}(n, m) \sim \mathsf{id}_{\mathsf{Code}(n,m)}.$$

*Proof.* We proceed by double induction, examining the distinct cases for the generic pair $(n, m)$.

**Case** $(0, 0)$**:**

$$\begin{aligned} \mathsf{encode}(\mathsf{decode}(0, 0, \star)) &\equiv \mathsf{encode}(\mathsf{refl}_0) \\ &\equiv \mathsf{transport}^{C_0}(\mathsf{refl}_0, c(0)) \\ &\equiv c(0) \\ &\equiv \star. \end{aligned}$$

**Case** $(0, \mathsf{succ}(m))$**:** By (2.4) we know that $\mathrm{dom}(\mathsf{decode}(0, \mathsf{succ}(m))) \equiv \mathsf{0}$. Thus, for $c\colon \mathsf{0}$, we have

$$\mathsf{encode}(\mathsf{decode}(0, \mathsf{succ}(m), c)) =_{\mathsf{0}} c,$$

as it follows from $\mathsf{ind}_{\mathsf{0}}(E)\colon \prod_{x:\mathsf{0}} x =_{\mathsf{0}} c$, where $E\colon \mathsf{0} \to \mathcal{U}$ is given by $E(x) :\equiv x =_{\mathsf{0}} c$.

**Case** $(0, m)$**:** It follows by induction on $m$ from the two previous cases.

**Case** $(\mathsf{succ}(n), 0)$**:** This is similar to the previous case because, by (2.4), we know that $\mathrm{dom}(\mathsf{decode}(\mathsf{succ}(n), 0)) \equiv \mathsf{0}$.

**Case** $(n, 0)$**:** It follows from the cases $(0, 0)$ and $(\mathsf{succ}(n), 0)$ by induction on $n$.

**Case** $(\mathsf{succ}(n), \mathsf{succ}(m))$**:** By (2.4) and the definition of $\mathsf{encode}$, we have

$$\begin{aligned}
&\mathsf{encode}(\mathsf{decode}(\mathsf{succ}(n), \mathsf{succ}(m), t)) \\
&\quad \equiv \mathsf{transport}^{C_{\mathsf{succ}(n)}}(\mathsf{ap}_{\mathsf{succ}}(\mathsf{decode}(n, m, t)), c(\mathsf{succ}(n))) \\
&\quad =_{\mathsf{Code}(n,m)} \mathsf{transport}^{C_{\mathsf{succ}(n)} \circ \mathsf{succ}}(\mathsf{decode}(n, m, t), c(n)) \\
&\quad \equiv \mathsf{transport}^{C_n}(\mathsf{decode}(n, m, t), c(n)) \\
&\quad \equiv \mathsf{encode}(\mathsf{decode}(n, m, t)),
\end{aligned}$$

where we have used Lemma 2.3.13 and the identities

$$c(\mathsf{succ}(n)) \equiv c(n) \quad \text{and} \quad C_{\mathsf{succ}(n)} \circ \mathsf{succ} \equiv C_n.$$

**Case** $(n, m)$**:** We assemble the preceding cases into a single term using the induction principle for the natural numbers. The main motive is

$$E(n) :\equiv \prod_{m:\mathbb{N}} \prod_{t:\mathsf{Code}(n,m)} \mathsf{encode}(\mathsf{decode}(n, m, t)) =_{\mathsf{Code}(n,m)} t.$$

A section of this type family is $\mathsf{ind}_{\mathbb{N}}(E, d_0, d_s)$, where:

- The base case $d_0\colon E(0)$ is constructed by induction on $m$. Concretely, $d_0 :\equiv \mathsf{ind}_{\mathbb{N}}(E_0, d_{00}, d_{0s})$, with motive $E_0 :\equiv E(0)$, i.e.,

  $$E_0(m) :\equiv \prod_{t:\mathsf{Code}(0,m)} \mathsf{encode}(\mathsf{decode}(0, m, t)) =_{\mathsf{Code}(0,m)} t.$$

  The base term $d_{00}$ is the proof from Case $(0, 0)$, and the step term $d_{0s}$ is the proof from Case $(0, \mathsf{succ}(m))$.

- The step function $d_s\colon \prod_{n:\mathbb{N}} E(n) \to E(\mathsf{succ}(n))$ binds an index $n$ and the inductive hypothesis $h\colon E(n)$. Then $d_s(n, h)$ is defined by another induction on $m$, using the motive $E_s :\equiv E(\mathsf{succ}(n))$, i.e.,

  $$E_s(m) :\equiv \prod_{t} \mathsf{encode}(\mathsf{decode}(\mathsf{succ}(n), m, t)) =_{\mathsf{Code}(\mathsf{succ}(n),m)} t,$$

for $t\colon \mathsf{Code}(\mathsf{succ}(n), m)$.

The base case is provided directly by Case $(\mathsf{succ}(n), 0)$. The step function is given in Case $(\mathsf{succ}(n), \mathsf{succ}(m))$ that reduces the goal to proving that

$$\mathsf{encode}(\mathsf{decode}(n, m, t)) =_{\mathsf{Code}(n,m)} t,$$

which is inhabited by $h(m, t)$.

Thus, the total application of $\mathsf{ind}_{\mathbb{N}}$ yields the required dependent function for all $n$ and $m$.

□

**Theorem 2.16.3.** *For all $n, m\colon \mathbb{N}$, we have*

$$\mathsf{decode}(n, m) \circ \mathsf{encode}(n, m) \sim \mathsf{id}_{n=_{\mathbb{N}} m}.$$

*Proof.* Consider the motive

$$\prod_{n,m:\,\mathbb{N}} \prod_{p:\,n=_{\mathbb{N}} m} \mathsf{decode}(n, m, \mathsf{encode}(n, m, p)) =_{n=_{\mathbb{N}} m} p.$$

To show that it is inhabited we can use path induction and reduce it to the case $m \equiv n$ and $p \equiv \mathsf{refl}_n$. The goal becomes

$$D :\equiv \prod_{n:\,\mathbb{N}} \mathsf{decode}(n, n, \mathsf{encode}(n, n, \mathsf{refl}_n)) =_{n=_{\mathbb{N}} n} \mathsf{refl}_n.$$

We can now proceed by induction on $n$.

**Case** $0$**:**

$$\begin{aligned} \mathsf{decode}(0, 0, \mathsf{encode}(0, 0, \mathsf{refl}_0)) &\equiv \mathsf{decode}(0, 0, \star) && ;\ (2.1) \\ &\equiv \mathsf{refl}_0. && ;\ (2.4) \end{aligned}$$

which is inhabited by $\mathsf{refl}_{\mathsf{refl}_0} : D(0)$.

**Case** $\mathsf{succ}(n)$**:** First observe that

$$\begin{aligned} \mathsf{encode}(\mathsf{succ}(n), \mathsf{succ}(n), \mathsf{refl}_{\mathsf{succ}(n)}) &\equiv \mathsf{transport}^{C_{\mathsf{succ}(n)}}(\mathsf{refl}_{\mathsf{succ}(n)}, c(\mathsf{succ}(n))) \\ &\equiv c(\mathsf{succ}(n)) \\ &\equiv c(n) \\ &\equiv \mathsf{transport}^{C_n}(\mathsf{refl}_n, c(n)) \\ &\equiv \mathsf{encode}(n, n, \mathsf{refl}_n). \end{aligned}$$

Therefore,

$$\begin{aligned} D(\mathsf{succ}(n)) &\equiv \mathsf{decode}(\mathsf{succ}(n), \mathsf{succ}(n), \mathsf{encode}(n, n, \mathsf{refl}_n)) \\ &\equiv \mathsf{ap}_{\mathsf{succ}}(\mathsf{decode}(n, n, \mathsf{encode}(n, n, \mathsf{refl}_n))) && ;\ (2.4) \end{aligned}$$

Hence, if $h\colon D(n)$, then $\mathsf{ap}_{\mathsf{ap}_{\mathsf{succ}}}(h)\colon D(\mathsf{succ}(n))$, which defines the step function $D(n) \to D(\mathsf{succ}(n)$.

□

## 2.17 Equality in Semigroups

Let $A\colon \mathcal{U}$ be a type. Using the type family

$$\begin{aligned} M &\colon \mathcal{U} \to \mathcal{U} \\ X &\mapsto (X \to X \to X) \end{aligned}$$

we define an *operation* on $A$ as a function

$$m\colon M(A).$$

The operation $m$ is *associative* if the type

$$\mathsf{Assoc}(A, m) :\equiv \prod_{x,y,z\colon A} m(x, m(y, z)) =_A m(m(x, y), z)$$

is inhabited.

With this we can define a *semigroup structure* type on $A$ as

$$\begin{aligned} \mathsf{SemigroupStr} &\colon \mathcal{U} \to \mathcal{U} \\ \mathsf{SemigroupStr}(A) &:\equiv \sum_{m\colon M(A)} \mathsf{Assoc}(A, m). \end{aligned}$$

More generally, the *Semigroup* type is

$$\mathsf{Semigroup} :\equiv \sum_{A\colon \mathcal{U}} \mathsf{SemigroupStr}(A).$$

By viewing associativity as the dependent function

$$\mathsf{Assoc}\colon \prod_{X\colon \mathcal{U}} M(X) \to \mathcal{U},$$

we can uncurry it to obtain the type family

$$\begin{aligned} \mathsf{Assoc}_\Sigma &\colon \Sigma_{\mathcal{U}} M \to \mathcal{U} \\ \langle A, m\rangle &\mapsto \mathsf{Assoc}(A, m), \end{aligned}$$

where $\Sigma_{\mathcal{U}} M$ is the total space $\sum_{X\colon \mathcal{U}} M(X)$. With this notation we have

$$\mathsf{SemigroupStr}(A) \equiv \sum_{m\colon M(A)} \mathsf{Assoc}_\Sigma(A, m) \equiv \Sigma_M \mathsf{Assoc}_\Sigma(A)$$

and

$$\mathsf{Semigroup} \equiv \Sigma_{\mathcal{U}}(\Sigma_M \mathsf{Assoc}_\Sigma).$$

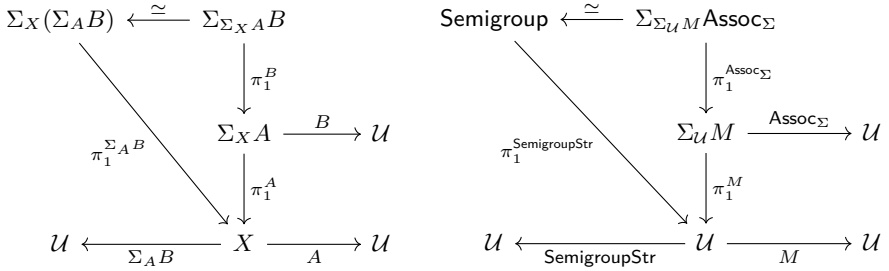
Now suppose we are given a path $p\colon A =_{\mathcal{U}} B$ between the types $A$ and $B$. Then we can transport the operation $m$ on $A$ to an operation $m'$ on $B$ by

$$m' :\equiv \mathsf{transport}^M(p, m).$$

By path induction on $p$ it is easy to see that $m'$ is associative whenever $m$ is. However, we can go deeper if we apply Theorem 2.9.14 as follows:

| **Role** | **Theorem** | **Semigroup** |
|---|---|---|
| Base space | $X$ | $\mathcal{U}$ |
| First family | $A\colon X \to \mathcal{U}$ | $M\colon \mathcal{U} \to \mathcal{U}$ |
| Total space | $\Sigma_X A$ | $\Sigma_{\mathcal{U}} M$ |
| Second family | $B\colon \Sigma_X A \to \mathcal{U}$ | $\mathsf{Assoc}_\Sigma$ |
| Composite family | $\Sigma_A B\colon X \to \mathcal{U}$ | $\mathsf{SemigroupStr}$ |
| Equivalent family | $\Sigma_X(\Sigma_A B)$ | $\mathsf{Semigroup}$ |

with diagram correspondence:




Given a path $p\colon A =_{\mathcal{U}} B$ between types, part b) of the theorem establishes the equality

$$\mathsf{transport}^{\Sigma_A B}(p)(\langle u, z\rangle) =_{\Sigma_A B(B)} \big(p_*(u), \mathsf{transport}^B(\mathsf{pair}^=(\langle p, \mathsf{refl}_{p_*(u)}\rangle))(z)\big),$$

which, according to our table above, translates into

$$\mathsf{transport}^{\mathsf{SemigroupStr}}(p)(\langle m, a\rangle) =_{\mathsf{SemigroupStr}(B)} (\langle m', a'\rangle)$$

where

$$\begin{aligned} m' &:\equiv \mathsf{transport}^M(p, m) \\ a' &:\equiv \mathsf{transport}^{\mathsf{Assoc}_\Sigma}\big(\mathsf{pair}^=(p, \mathsf{refl}_{m'}), a\big). \end{aligned}$$

Note that

$$\mathsf{pair}^{=}(p, \mathsf{refl}_{m'})\colon (\langle A, m\rangle) =_{\Sigma_{\mathcal{U}} M} (\langle B, m'\rangle)$$

serves as the base path to transport the associativity proof $a$ into its new fiber $\mathsf{Assoc}_{\Sigma}(B, m')$.

We have proven the following

**Theorem 2.17.1.**[UA] *Let $A$ and $B$ be types, and let $\langle m, a\rangle$ be a semigroup structure on $A$. Given an equivalence $e\colon A \simeq B$, the univalence axiom induces a semigroup structure $(m', a')$ on $B$ defined by*

$$\begin{aligned} m' &:\equiv \mathsf{transport}^{M}(\mathsf{ua}(e), m) \\ a' &:\equiv \mathsf{transport}^{\mathsf{Assoc}_{\Sigma}}\big(\mathsf{pair}^{=}(\langle \mathsf{ua}(e), \mathsf{refl}_{m'}\rangle), a\big). \end{aligned}$$

## 2.18 Universal Properties

In category theory, universal properties are typically expressed by the unique existence of morphisms that complete commutative diagrams. In homotopy type theory, however, these properties are interpreted homotopically as equivalences between types.

These universal properties can generally be categorized by whether they characterize functions *into* a type (reflecting its *introduction* rules) or functions *out of* a type (reflecting its *elimination* or *induction* principles).

As we discussed in 28. PRODUCTS REVISITED, the cartesian product $A \times B$ admits two characterizations: one by means of a function into $A \times B$, and another by means of a function out of $A \times B$. In each of these cases, the associated commutative diagram can be represented as an equivalence of types. In this section, we generalize this approach to different basic types.

**Theorem 2.18.1.**[FE] *Let $X, A, B\colon \mathcal{U}$ be types. Then the function*[25]

$$\begin{aligned} \sigma\colon (X \to A \times B) &\to (X \to A) \times (X \to B) \\ f &\mapsto (\pi_1 \circ f, \pi_2 \circ f) \end{aligned}$$

*is an equivalence.*

*Proof.* Define the map

$$\begin{aligned} \tau\colon (X \to A) \times (X \to B) &\to (X \to A \times B) \\ \tau(u, v) &:\equiv \lambda x.\, \big(u(x), v(x)\big). \end{aligned}$$

[25] By an abuse of notation, we omit irrelevant parentheses.

Evaluating $\sigma$ at $\tau(u, v)$ yields

$$\begin{aligned}\sigma(\tau(u,v)) &\equiv \sigma\big(\lambda x.\,(u(x), v(x))\big)\\ &\equiv \big(\pi_1 \circ (\lambda x.\,(u(x), v(x))), \pi_2 \circ (\lambda x.\,(u(x), v(x)))\big)\\ &\equiv \big(\lambda x.\,\pi_1(u(x), v(x)), \lambda x.\,\pi_2(u(x), v(x))\big)\\ &\equiv \big(\lambda x.\,u(x), \lambda x.\,v(x)\big)\\ &\equiv (u, v).\end{aligned}$$

On the other hand, evaluating $\tau$ at $\sigma(f)$ yields

$$\begin{aligned}\tau(\sigma(f)) &\equiv \tau(\pi_1 \circ f, \pi_2 \circ f)\\ &\equiv \lambda x.\,\big(\pi_1(f(x)), \pi_2(f(x))\big).\end{aligned}$$

By Corollary 2.7.3, there is a path in $(\pi_1(f(x)), \pi_2(f(x))) =_{A\times B} f(x)$ for each $x\colon X$. Therefore, applying the function extensionality axiom to this pointwise equality yields the path

$$\tau(\sigma(f)) =_{X\to A\times B} f,$$

which completes the proof by showing that $\tau$ is a quasi-inverse of $\sigma$. □

**Theorem 2.18.2.**[FE] *Let $X\colon \mathcal{U}$ be a type, and let $A, B\colon X \to \mathcal{U}$ be type families. Then the function*

$$\begin{aligned}\sigma\colon \left(\prod_{x\colon X} A(x) \times B(x)\right) &\to \left(\prod_{x\colon X} A(x)\right) \times \left(\prod_{x\colon X} B(x)\right)\\ f &\mapsto \big(\lambda x.\,\pi_1(f(x)), \lambda x.\,\pi_2(f(x))\big)\end{aligned}$$

*is an equivalence.*

*Proof.* Define the map

$$\begin{aligned}\tau\colon \left(\prod_{x\colon X} A(x)\right) \times \left(\prod_{x\colon X} B(x)\right) &\to \prod_{x\colon X} A(x) \times B(x)\\ (u, v) &\mapsto \lambda x.\,(u(x), v(x)).\end{aligned}$$

Evaluating $\sigma \circ \tau$ at $(u, v)$, we obtain

$$\begin{aligned}\sigma(\tau(u,v)) &\equiv \sigma\big(\lambda x.\,(u(x), v(x))\big)\\ &\equiv \big(\lambda x.\,\pi_1(u(x), v(x)), \lambda x.\,\pi_2(u(x), v(x))\big)\\ &\equiv \big(\lambda x.\,u(x), \lambda x.\,v(x)\big)\\ &\equiv (u, v).\end{aligned}$$

Evaluating $\tau \circ \sigma$ at $f$ yields

$$\begin{aligned}\tau(\sigma(f)) &\equiv \tau\big(\lambda x.\,\pi_1(f(x)), \lambda x.\,\pi_2(f(x))\big)\\ &\equiv \lambda x.\,\big(\pi_1(f(x)), \pi_2(f(x))\big).\end{aligned}$$

By Corollary 2.7.3, for each $x\colon X$, the type $(\pi_1(f(x)), \pi_2(f(x))) =_{A(x)\times B(x)} f(x)$ is inhabited. Thus, function extensionality implies that

$$\tau(\sigma(f)) =_T f,$$

where

$$T :\equiv \prod_{x\colon X} A(x) \times B(x).$$

□

**Theorem 2.18.3.**[FE] [Axiom of Choice] *Let $X : \mathcal{U}$ be a type, $A\colon X \to \mathcal{U}$ a type family, and $P\colon \prod_{x\colon X} A(x) \to \mathcal{U}$ a dependent type family. Then the function*

$$\mathsf{ac}\colon \left(\prod_{x\colon X} \sum_{y\colon A(x)} P(x,y)\right) \to \sum_{g\colon \prod_{x\colon X} A(x)} \prod_{x\colon X} P(x, g(x))$$
$$f \mapsto \left\langle \lambda x.\, \pi_1(f(x)), \lambda x.\, \pi_2(f(x))\right\rangle$$

*is an equivalence.*

*Proof.* First observe that $\mathsf{ac}$ is well defined:

$$f(x)\colon \sum_{y\colon A(x)} P(x,y)$$

and so

$$\pi_1(f(x))\colon A(x)$$
$$\pi_2(f(x))\colon P(x, \pi_1(f(x))).$$

Consider the function

$$\mathsf{ac}^{-1}\colon \left(\sum_{g\colon \prod_{x\colon X} A(x)} \prod_{x\colon X} P(x, g(x))\right) \to \prod_{x\colon X} \sum_{y\colon A(x)} P(x,y)$$
$$\langle g, \varepsilon\rangle \mapsto \lambda x.\, \langle g(x), \varepsilon(x)\rangle.$$

Then,

$$\begin{aligned}\mathsf{ac}(\mathsf{ac}^{-1}(\langle g, \varepsilon\rangle)) &\equiv \mathsf{ac}\big(\lambda x.\, \langle g(x), \varepsilon(x)\rangle\big)\\ &\equiv \left\langle \lambda x.\, \pi_1(g(x), \varepsilon(x)), \lambda x.\, \pi_2(g(x), \varepsilon(x))\right\rangle\\ &\equiv \left\langle \lambda x.\, g(x), \lambda x.\, \varepsilon(x)\right\rangle\\ &\equiv \langle g, \varepsilon\rangle.\end{aligned}$$

On the other hand,

$$\begin{aligned}\mathsf{ac}^{-1}(\mathsf{ac}(f)) &\equiv \mathsf{ac}^{-1}\big(\langle\overbrace{\lambda x.\,\pi_1(f(x))}^{g},\overbrace{\lambda x.\,\pi_2(f(x))}^{\varepsilon}\rangle\big)\\ &\equiv \lambda x.\,\langle\pi_1(f(x)),\pi_2(f(x))\rangle.\end{aligned}$$

By Corollary 2.9.6, for each $x\colon X$, the type

$$\langle\pi_1(f(x)),\pi_2(f(x))\rangle =_{\sum_{y\colon A(x)} P(x,y)} f(x)$$

is inhabited. Therefore, by function extensionality,

$$\mathsf{ac}^{-1}(\mathsf{ac}(f)) =_{\prod_{x\colon X}\sum_{y\colon A(x)} P(x,y)} f.$$

□

**Remark 2.18.4.** It is worth noting that the axiom of function extensionality was used to prove that the map $\mathsf{ac}$ is an equivalence. However, no axiom was needed for the construction of the global choice function $g\colon \prod_{x\colon X} A(x)$.

**Theorem 2.18.5.**[FE] *Let $A, B, C\colon \mathcal{U}$ be types. Then the map*

$$\begin{aligned}\mathsf{curry}\colon ((A\times B)\to C)&\to (A\to(B\to C))\\ f&\mapsto \lambda x.\,\lambda y.\,f(x,y)\end{aligned}$$

*is an equivalence.*

*Proof.* Consider the function

$$\begin{aligned}\mathsf{uncurry}\colon (A\to(B\to C))&\to ((A\times B)\to C)\\ g&\mapsto \lambda\omega.\,g(\pi_1(\omega))(\pi_2(\omega)).\end{aligned}$$

Evaluating $\mathsf{curry}\circ\mathsf{uncurry}$ at $g$ yields

$$\begin{aligned}\mathsf{curry}(\mathsf{uncurry}(g)) &\equiv \mathsf{curry}\big(\lambda\omega.\,g(\pi_1(\omega))(\pi_2(\omega))\big)\\ &\equiv \lambda x.\,\lambda y.\,\big(\lambda\omega.\,g(\pi_1(\omega))(\pi_2(\omega))\big)(x,y)\\ &\equiv \lambda x.\,\lambda y.\,g(\pi_1(x,y))(\pi_2(x,y))\\ &\equiv \lambda x.\,\lambda y.\,g(x)(y)\\ &\equiv \lambda x.\,g(x)\\ &\equiv g.\end{aligned}$$

Moreover, evaluating $\mathsf{uncurry}\circ\mathsf{curry}$ at $f$ yields

$$\begin{aligned}\mathsf{uncurry}(\mathsf{curry}(f)) &\equiv \mathsf{uncurry}\big(\lambda x.\,\lambda y.\,f(x,y)\big)\\ &\equiv \lambda\omega.\,\big(\lambda x.\,\lambda y.\,f(x,y)\big)(\pi_1(\omega))(\pi_2(\omega))\\ &\equiv \lambda\omega.\,\big(\lambda y.\,f(\pi_1(\omega)),y)\big)(\pi_2(\omega))\\ &\equiv \lambda\omega.\,f(\pi_1(\omega),\pi_2(\omega)).\end{aligned}$$

By Corollary 2.7.3, for each $\omega\colon A \times B$, there is a path $(\pi_1(\omega), \pi_2(\omega)) =_{A\times B} \omega$. Applying $\mathsf{ap}_f$, we obtain a path $f(\pi_1(\omega), \pi_2(\omega)) =_C f(\omega)$. Therefore, function extensionality implies that

$$\mathsf{uncurry}(\mathsf{curry}(f)) =_{(A\times B)\to C} f,$$

which completes the proof. □

**Theorem 2.18.6.**[FE] *Let $A, B\colon \mathcal{U}$ be types, and let $C\colon A\times B \to \mathcal{U}$ be a dependent type family. Then the map*

$$\begin{aligned}\mathsf{curry}\colon \left(\prod_{\omega\colon A\times B} C(\omega)\right) &\to \prod_{x\colon A}\prod_{y\colon B} C((x,y))\\ f &\mapsto \lambda x.\,\lambda y.\, f((x,y))\end{aligned}$$

*is an equivalence.*

*Proof.* Consider the function[26]

$$\begin{aligned}\mathsf{uncurry}\colon \left(\prod_{x\colon A}\prod_{y\colon B} C((x,y))\right) &\to \prod_{\omega\colon A\times B} C(\omega)\\ g(\omega) &:\equiv \mathsf{transport}^C\big(\mathsf{uniq}_{A\times B}(\omega), g(\pi_1(\omega))(\pi_2(\omega))\big).\end{aligned}$$

Evaluating $\mathsf{curry}\circ\mathsf{uncurry}$ at $g$ yields

$$\begin{aligned}\mathsf{curry}(\mathsf{uncurry}(g)) &\equiv \mathsf{curry}\big(\lambda\omega.\,\mathsf{transport}^C\big(\mathsf{uniq}_{A\times B}(\omega), g(\pi_1(\omega))(\pi_2(\omega))\big)\big)\\ &\equiv \lambda x.\,\lambda y.\, g(x)(y)\\ &\equiv \lambda x.\, g(x)\\ &\equiv g,\end{aligned}$$

where the second definitional equality is justified by the definitional equality

$$\mathsf{uniq}_{A\times B}((x,y)) \equiv \mathsf{refl}_{(x,y)},$$

which causes the transport to vanish.

Moreover, evaluating $\mathsf{uncurry}\circ\mathsf{curry}$ at $f$ yields

$$\begin{aligned}\mathsf{uncurry}(\mathsf{curry}(f)) &\equiv \mathsf{uncurry}\big(\lambda x.\,\lambda y.\, f((x,y))\big)\\ &\equiv \lambda\omega.\,\mathsf{transport}^C\big(\mathsf{uniq}_{A\times B}(\omega), f((\pi_1(\omega),\pi_2(\omega)))\big).\end{aligned}$$

By Lemma 2.3.6, the dependent action on paths

$$\mathsf{apd}_f\big((\pi_1(\omega),\pi_2(\omega)), \omega, \mathsf{uniq}_{A\times B}(\omega)\big)$$

[26] This is none other than the induction principle for products.

is a witness of the identity type

$$\mathsf{transport}^C\big(\mathsf{uniq}_{A\times B}(\omega), f((\pi_1(\omega),\pi_2(\omega)))\big) =_{C(\omega)} f(\omega).$$

Hence, for each $\omega\colon A\times B$, there is a path

$$\mathsf{uncurry}(\mathsf{curry}(f))(\omega) =_{C(\omega)} f(\omega),$$

and the result follows by function extensionality. □

**Theorem 2.18.7.**[FE] *Let $A$ and $B$ be types, and let $C\colon A+B\to\mathcal{U}$ be a type family. Then there is an equivalence*

$$\prod_{z\colon A+B} C(z) \simeq \Big(\prod_{x\colon A} C(\mathsf{inl}(x))\Big)\times\Big(\prod_{y\colon B} C(\mathsf{inr}(y))\Big).$$

*Proof.* The forward map is defined as

$$\mathsf{split} :\equiv \lambda f.\,(f\circ\mathsf{inl}, f\circ\mathsf{inr}),$$

and the backward map as

$$\mathsf{merge} :\equiv \lambda(u,v).\,\mathsf{ind}_{A+B}(C,u,v).$$

To complete the proof, we compute both compositions. For the first composition, we have:

$$\begin{aligned}\mathsf{split}(\mathsf{merge}(u,v)) &\equiv \mathsf{split}(\mathsf{ind}_{A+B}(C,u,v))\\ &\equiv (\mathsf{ind}_{A+B}(C,u,v)\circ\mathsf{inl}, \mathsf{ind}_{A+B}(C,u,v)\circ\mathsf{inr}).\end{aligned}$$

Evaluating these precompositions at $x\colon A$ and $y\colon B$ yields

$$\begin{aligned}\mathsf{ind}_{A+B}(C,u,v,\mathsf{inl}(x)) &\equiv u(x),\\ \mathsf{ind}_{A+B}(C,u,v,\mathsf{inr}(y)) &\equiv v(y).\end{aligned}$$

Thus, the composition is equal to the identity function.

For the other composition, we have:

$$\mathsf{merge}(\mathsf{split}(f)) \equiv \mathsf{ind}_{A+B}(C, f\circ\mathsf{inl}, f\circ\mathsf{inr}),$$

which, by function extensionality, equals $f$ because both functions yield the same results when precomposed with $\mathsf{inl}$ and $\mathsf{inr}$. □

**Theorem 2.18.8.**[FE] *Let $A\colon\mathcal{U}$ be a type and $B\colon A\to\mathcal{U}$ a type family. Given a dependent type family $C\colon\Sigma_A B\to\mathcal{U}$ over the total space $\Sigma_A B$, the map*

$$\begin{aligned}\mathsf{curry}\colon \Big(\prod_{\omega\colon\Sigma_A B} C(\omega)\Big) &\to \prod_{x\colon A}\prod_{y\colon B(x)} C((x,y))\\ f &\mapsto \lambda x.\,\lambda y.\, f((x,y))\end{aligned}$$

*is an equivalence.*

*Proof.* Consider the function[27]

$$\mathsf{uncurry}\colon \left(\prod_{x\colon A}\prod_{y\colon B(x)} C((x,y))\right) \to \prod_{\omega\colon \Sigma_A B} C(\omega)$$
$$\mathsf{uncurry}(g) :\equiv \lambda\omega.\, \mathsf{transport}^C\big(\mathsf{uniq}_{\Sigma_A B}(\omega), g(\pi_1(\omega))(\pi_2(\omega))\big),$$

where the map

$$\mathsf{uniq}_{\Sigma_A B}\colon \prod_{\omega\colon \Sigma_A B} (\pi_1(\omega), \pi_2(\omega)) =_{\Sigma_A B} \omega$$

is the uniqueness principle defined after Corollary 2.9.6.

Evaluating $\mathsf{curry} \circ \mathsf{uncurry}$ at $g$ yields

$$\begin{aligned}
\mathsf{curry}(\mathsf{uncurry}(g)) &\equiv \lambda x.\, \lambda y.\, \mathsf{uncurry}(g)((x,y)) \\
&\equiv \lambda x.\, \lambda y.\, \mathsf{transport}^C\big(\mathsf{uniq}_{\Sigma_A B}((x,y)), g(\pi_1((x,y)))(\pi_2((x,y)))\big) \\
&\equiv \lambda x.\, \lambda y.\, g(x)(y) \\
&\equiv \lambda x.\, g(x) \\
&\equiv g,
\end{aligned}$$

where the third definitional equality is justified by the fact that

$$\mathsf{uniq}_{\Sigma_A B}((x,y)) \equiv \mathsf{refl}_{(x,y)},$$

which causes the transport to vanish.

Moreover, evaluating $\mathsf{uncurry} \circ \mathsf{curry}$ at $f$ yields

$$\begin{aligned}
\mathsf{uncurry}(\mathsf{curry}(f)) &\equiv \mathsf{uncurry}\big(\lambda x.\, \lambda y.\, f((x,y))\big) \\
&\equiv \lambda\omega.\, \mathsf{transport}^C\big(\mathsf{uniq}_{\Sigma_A B}(\omega), f((\pi_1(\omega), \pi_2(\omega)))\big).
\end{aligned}$$

By Lemma 2.3.6, the dependent action on paths

$$\mathsf{apd}_f(\pi_1(\omega), \pi_2(\omega), \mathsf{uniq}_{\Sigma_A B}(\omega))$$

is a witness of the identity type

$$\mathsf{transport}^C\big(\mathsf{uniq}_{\Sigma_A B}(\omega), f((\pi_1(\omega), \pi_2(\omega)))\big) =_{C(\omega)} f(\omega).$$

Hence, for each $\omega\colon \Sigma_A B$, there is a path

$$\mathsf{uncurry}(\mathsf{curry}(f))(\omega) =_{C(\omega)} f(\omega),$$

and the result follows by function extensionality. □

[27] This is the induction principle for dependent pairs.

**Corollary 2.18.9.** *Let $A : \mathcal{U}$ be a type and $B : A \to \mathcal{U}$ a type family. For any type $C : \mathcal{U}$, the map*

$$\mathsf{rec}_{\Sigma_A B}(C) : \left(\prod_{x:A} B(x) \to C\right) \to (\Sigma_A B \to C)$$

*is an equivalence.*

*Proof.* Instantiating the dependent type family in the preceding theorem as the constant family $C(\omega) :\equiv C$ for all $\omega : \Sigma_A B$, we obtain that the map

$$\begin{aligned}\mathsf{curry} : (\Sigma_A B \to C) &\to \prod_{x:A} B(x) \to C \\ f &\mapsto \lambda x.\, \lambda y.\, f((x,y))\end{aligned}$$

is an equivalence. By the definition of the recursion principle for $\Sigma$-types, the map $\mathsf{rec}_{\Sigma_A B}(C)$ is the quasi-inverse of $\mathsf{curry}$, as it takes a curried function and evaluates it on a pair by uncurrying it. Since the quasi-inverse of an equivalence is an equivalence, it follows that $\mathsf{rec}_{\Sigma_A B}(C)$ is an equivalence. □

## 2.19 Exercises

**Exercise 2.19.1.** *Derive the induction principle for products $\mathsf{ind}_{A\times B}$, using only the projections and the propositional uniqueness principle $\mathsf{uniq}_{A\times B}$. Verify that the definitional equalities are valid. Generalize $\mathsf{uniq}_{A\times B}$ to $\Sigma$-types, and do the same for $\Sigma$-types.*

*Solution.* Let $C : A \times B \to \mathcal{U}$ be a type family. We have to define a function

$$\mathsf{ind}_{A\times B} : \left(\prod_{x:A}\prod_{y:B} C((x,y))\right) \to \prod_{\omega : A\times B} C(\omega)$$

such that

$$\mathsf{ind}_{A\times B}(f)((x,y)) \equiv f(x)(y). \tag{2.1}$$

To achieve this, we define

$$\mathsf{ind}_{A\times B}(f) :\equiv \lambda\omega.\, \mathsf{transport}^C\big(\mathsf{uniq}_{A\times B}(\omega), f(\pi_1(\omega))(\pi_2(\omega))\big).$$

Since

$$\mathsf{uniq}_{A\times B}(\omega) : (\pi_1(\omega), \pi_2(\omega)) =_{A\times B} \omega,$$

we have

$$\begin{aligned}&\mathsf{transport}^C\big(\mathsf{uniq}_{A\times B}(\omega)\big) : C((\pi_1(\omega),\pi_2(\omega))) \to C(\omega) \\ &\mathsf{transport}^C\big(\mathsf{uniq}_{A\times B}(\omega), f(\pi_1(\omega))(\pi_2(\omega))\big) : C(\omega),\end{aligned}$$

which shows that $\mathsf{ind}_{A\times B}$ is well defined.

Moreover, given that $\mathsf{uniq}_{A\times B}((x,y)) \equiv \mathsf{refl}_{(x,y)}$, the transport vanishes and equation (2.1) is satisfied because $\pi_1((x,y)) \equiv x$ and $\pi_2((x,y)) \equiv y$.

Let us now do the same for $\Sigma_A B$, where $B\colon A \to \mathcal{U}$ is a type family. We have to define

$$\mathsf{ind}_{\Sigma_A B}\colon \left(\prod_{x\colon A}\prod_{y\colon B(x)} C((x,y))\right) \to \prod_{\zeta\colon \Sigma_A B} C(\zeta)$$

such that

$$\mathsf{ind}_{\Sigma_A B}(f)((x,y)) \equiv f(x)(y). \tag{2.2}$$

To fulfill the requirement, we would like to define the function along the lines of

$$\mathsf{ind}_{\Sigma_A B}(f) :\equiv \lambda\zeta.\, \mathsf{transport}^C\big(\mathsf{uniq}_{\Sigma_A B}(\zeta), f(\pi_1(\zeta))(\pi_2(\zeta))\big).$$

This shows that we first need to assume the existence of a map

$$\mathsf{uniq}_{\Sigma_A B}\colon \prod_{\zeta\colon \Sigma_A B} (\pi_1(\zeta),\pi_2(\zeta)) =_{\Sigma_A B} \zeta$$

satisfying $\mathsf{uniq}_{\Sigma_A B}((a,b)) \equiv \mathsf{refl}_{(a,b)}$ for all $a\colon A$ and all $b\colon B(a)$.

In sum, this exercise shows that $\mathsf{ind}$ and $\mathsf{uniq}$ are deducible from each other for both dependent and non-dependent products.

□

**Exercise 2.19.2.**[FE] *Show that if we define* $A \times B :\equiv \prod_{z\colon 2} \mathsf{rec}_2(\mathcal{U}, A, B, z)$*, then we can give a definition of* $\mathsf{ind}_{A\times' B}$ *for which the definitional equalities stated in* 32. INDUCTION *hold propositionally, i.e., using equality types.*

*Solution.* Let

$$A \times' B :\equiv \prod_{z\colon 2} \mathsf{rec}_2(\mathcal{U}, A, B, z),$$

where $\mathsf{rec}_2(\mathcal{U})$ is defined in 45. THE TYPE OF BOOLEANS as

$$\mathsf{rec}_2(\mathcal{U})\colon \mathcal{U} \to \mathcal{U} \to 2 \to \mathcal{U}.$$

Thus, the elements of $A \times' B$ are dependent functions $\omega$ that satisfy

$$\omega(0_2)\colon A$$
$$\omega(1_2)\colon B.$$

Before proceeding, let us establish that the canonical elements of $A \times' B$ are the dependent functions $\varepsilon_{(a,b)}$ given by pairs $(a,b)\colon A \times B$, such that $\varepsilon_{(a,b)}(0_2) \equiv a$ and $\varepsilon_{(a,b)}(1_2) \equiv b$. Under this convention, for $C\colon A \times' B \to \mathcal{U}$, we have to define

$$\mathsf{ind}_{A\times' B}\colon \left(\prod_{x\colon A}\prod_{y\colon B} C(\varepsilon_{(x,y)})\right) \to \prod_{\omega\colon A\times' B} C(\omega).$$

Let $f$ be a function in the domain. Then, given a pair $(x, y)$ in the official product $A \times B$, we have

$$f(x)(y) \colon C(\varepsilon_{(x,y)}).$$

We want to produce from this a function $\hat{f}$ such that, for every

$$\omega \colon \prod_{z \colon 2} \mathsf{rec}_2(\mathcal{U}, A, B, z),$$

we obtain $\hat{f}(\omega) \colon C(\omega)$. In addition, we require that this function satisfy

$$\hat{f}(\varepsilon_{(a,b)}) =_{C(\varepsilon_{(a,b)})} f(a)(b). \tag{2.3}$$

Before defining $\hat{f}(\omega)$, let us take into account that

$$f(\omega(0_2))(\omega(1_2)) \colon C(\varepsilon_{(\omega(0_2),\omega(1_2))}).$$

For $\omega \colon A \times' B$, introduce

$$E_\omega :\equiv \lambda z \colon 2.\, \varepsilon_{(\omega(0_2),\omega(1_2))}(z) =_{\mathsf{rec}_2(\mathcal{U},A,B,z)} \omega(z).$$

Using the induction principle of booleans, to define a dependent function of this type it is enough to provide two constants $e_0 \colon E_\omega(0_2)$ and $e_1 \colon E_\omega(1_2)$. Since this condition is fulfilled by $e_0 :\equiv \mathsf{refl}_{\omega(0_2)}$ and $e_1 :\equiv \mathsf{refl}_{\omega(1_2)}$, we conclude that there is a dependent function

$$e_\omega :\equiv \mathsf{ind}_2(E_\omega, \mathsf{refl}_{\omega(0_2)}, \mathsf{refl}_{\omega(1_2)}) \colon \prod_{z \colon 2} E_\omega(z).$$

By function extensionality, for $\hat{e}_\omega :\equiv \mathsf{funext}(e_\omega)$, we deduce that

$$\hat{e}_\omega \colon \varepsilon_{(\omega(0_2),\omega(1_2))} =_{A \times' B} \omega.$$

In particular,

$$\hat{e}_{\varepsilon_{(a,b)}} \colon \varepsilon_{(a,b)} =_{A \times' B} \varepsilon_{(a,b)}. \tag{2.4}$$

Therefore, we can define

$$\hat{f}(\omega) :\equiv \mathsf{transport}^C\big(\hat{e}_\omega, f(\omega(0_2))(\omega(1_2))\big),$$

which inhabits $C(\omega)$.

To verify condition (2.3), observe that by boolean induction on the constants, we have

$$e_{\varepsilon_{(a,b)}}(0_2) \equiv \mathsf{refl}_{\varepsilon_{(a,b)}(0_2)}$$
$$e_{\varepsilon_{(a,b)}}(1_2) \equiv \mathsf{refl}_{\varepsilon_{(a,b)}(1_2)}.$$

By induction on $z \colon 2$, we deduce a pointwise propositional equality

$$H(z) \colon e_{\varepsilon_{(a,b)}}(z) =_{E_{\varepsilon_{(a,b)}}(z)} \mathsf{refl}_{\varepsilon_{(a,b)}(z)}.$$

By function extensionality, this implies an equality of dependent functions

$$p\colon e_{\varepsilon_{(a,b)}} =_{\prod_{z:2} E_{\varepsilon_{(a,b)}}(z)} \lambda z.\, \mathsf{refl}_{\varepsilon_{(a,b)}(z)}.$$

Applying $\mathsf{ap}_{\mathsf{funext}}$ to $p$, we obtain

$$\mathsf{funext}(e_{\varepsilon_{(a,b)}}) =_{\varepsilon_{(a,b)}=_{A\times' B}\varepsilon_{(a,b)}} \mathsf{funext}(\lambda z.\, \mathsf{refl}_{\varepsilon_{(a,b)}(z)}).$$

Since $\mathsf{funext}$ maps the constant reflexivity homotopy to the reflexivity path, we have $\mathsf{funext}(\lambda z.\, \mathsf{refl}_{\varepsilon_{(a,b)}(z)}) \equiv \mathsf{refl}_{\varepsilon_{(a,b)}}$. Thus, we have established a path

$$q\colon \hat{e}_{\varepsilon_{(a,b)}} =_{\varepsilon_{(a,b)}=_{A\times' B}\varepsilon_{(a,b)}} \mathsf{refl}_{\varepsilon_{(a,b)}},$$

where $\hat{e}_{\varepsilon_{(a,b)}} :\equiv \mathsf{funext}(e_{\varepsilon_{(a,b)}})$.

For $r$ in this identity type, define

$$T_{(a,b)}(r) :\equiv \mathsf{transport}^C\big(r, f(a)(b)\big).$$

Applying $\mathsf{ap}_{T_{(a,b)}}$ to the path $q$, we obtain

$$\begin{aligned}\hat{f}(\varepsilon_{(a,b)}) &\equiv \mathsf{transport}^C\big(\hat{e}_{\varepsilon_{(a,b)}}, f(a)(b)\big)\\ &=_{C(\varepsilon_{(a,b)})} \mathsf{transport}^C\big(\mathsf{refl}_{\varepsilon_{(a,b)}}, f(a)(b)\big)\\ &\equiv f(a)(b).\end{aligned}$$

□

**Exercise 2.19.3.** *Define, by induction on $n$, a general notion of $n$-dimensional path in a type $A$, simultaneously with the type of boundaries for such paths.*

Hint: If $B\colon \mathbb{N} \to \mathcal{U}$ denotes the type family of boundary types and $P\colon \mathbb{N} \to \mathcal{U}$ the family of path types, then $P(n)\colon B(n) \to \mathcal{U}$.

*Solution.* Let us begin by saying that the boundaries of a path $p\colon x =_A y$ are the terms $x$ and $y$.

With this in mind, what we need to do is to define, for every $n\colon \mathbb{N}$, two types $P(n)$ and $B(n)$, the first one representing $n$-paths and the second $n$-boundaries.

In this regard, $P$ and $B$ will be type families with signature $\mathbb{N} \to \mathcal{U}$, whose definitions will follow from induction on $n$, while satisfying

$$P(n)\colon B(n) \to \mathcal{U}.$$

This means that we have to define two initial types $P(0)$ and $B(0)$, and two step functions. Since we cannot simply define $B(\mathsf{succ}(n))$ as $P(n) \times P(n)$ because

we must ensure that both $n$-paths share the same ends, the actual definition is more involved. The idea comes after considering the first few values of $n$:

$$\begin{cases} B(0) :\equiv \mathsf{1} \\ P(0) :\equiv \lambda u.\, A. \end{cases}$$

$$\begin{cases} B(1) :\equiv \sum_{z:\, B(0)} P(0)(z) \times P(0)(z) \\ P(1) :\equiv \lambda(z, (a_1, a_2)).\, a_1 =_{P(0)(z)} a_2. \end{cases}$$

$$\begin{cases} B(2) :\equiv \sum_{z:\, B(1)} P(1)(z) \times P(1)(z) \\ P(2) :\equiv \lambda(z, (p_1, p_2)).\, p_1 =_{P(1)(z)} p_2. \end{cases}$$

In general,

$$\begin{cases} B(\mathsf{succ}(n)) :\equiv \sum_{z:\, B(n)} P(n)(z) \times P(n)(z) \\ P(\mathsf{succ}(n)) :\equiv \lambda(z, (\alpha_1, \alpha_2)).\, \alpha_1 =_{P(n)(z)} \alpha_2. \end{cases}$$

To define a function

$$c\colon \mathbb{N} \to \Big(\sum_{B:\,\mathcal{U}} B \to \mathcal{U}\Big)$$

by recursion on $n$, we specify

$$\begin{aligned} &c_0 :\equiv \langle \mathsf{1}, \lambda z.\, A \rangle \\ &c_s\colon \mathbb{N} \to \Big(\sum_{B:\,\mathcal{U}} B \to \mathcal{U}\Big) \to \Big(\sum_{B:\,\mathcal{U}} B \to \mathcal{U}\Big) \\ &c_s :\equiv \lambda n.\, \lambda\langle B, h\rangle.\, \Big\langle \Big(\sum_{z:\, B} h(z) \times h(z)\Big), \lambda(\langle z, (h_1, h_2)\rangle).\, h_1 =_{h(z)} h_2 \Big\rangle. \end{aligned}$$

Note that, by abuse of notation, we have written $\lambda\langle B, h\rangle$ instead of $\lambda x$ and $B$ and $h$ instead of $\pi_1(x)$ and $\pi_2(x)$.

We can now define

$$c(n) :\equiv \mathsf{rec}_{\mathbb{N}}\Big(\sum_{B:\,\mathcal{U}} B \to \mathcal{U}, c_0, c_s, n\Big)$$

and with this

$$B(n) :\equiv \pi_1(c(n)) \quad \text{and} \quad P(n) :\equiv \pi_2(c(n)).$$

Since

$$\begin{aligned} \langle B(\mathsf{succ}(n)), P(\mathsf{succ}(n))\rangle &\equiv c(\mathsf{succ}(n)) \\ &\equiv c_s(n, c(n)) \\ &\equiv c_s(n, \langle B(n), P(n)\rangle), \end{aligned}$$

i.e.,

$$\begin{aligned} B(\mathsf{succ}(n)) &\equiv \sum_{z:\, B(n)} P(n)(z) \times P(n)(z) \\ P(\mathsf{succ}(n)) &\equiv \lambda\langle z, (\alpha_1, \alpha_2)\rangle.\, \alpha_1 =_{P(n)(z)} \alpha_2, \end{aligned}$$

as desired. □

**Exercise 2.19.4.** *For any $x, y\colon A$, a path $p\colon x =_A y$, and a function $f\colon A \to B$, show that by concatenating with $\mathsf{transportconst}_p^B(f(x))$ and its inverse, respectively, we obtain functions*

$$(f(x) =_B f(y)) \to (p_*(f(x)) =_B f(y))$$

*and*

$$(p_*(f(x)) =_B f(y)) \to (f(x) =_B f(y)).$$

*Prove that these functions are in fact inverse equivalences.*

*Solution.* Given that

$$\mathsf{transportconst}_p^B(f(x))\colon p_*(f(x)) =_B f(x),$$

we can define

$$\begin{aligned}\phi\colon (f(x) =_B f(y)) &\to (p_*(f(x)) =_B f(y))\\ q &\mapsto \mathsf{transportconst}_p^B(f(x)) \cdot q\end{aligned}$$

and

$$\begin{aligned}\psi\colon (p_*(f(x)) =_B f(y)) &\to (f(x) =_B f(y))\\ r &\mapsto \mathsf{transportconst}_p^B(f(x))^{-1} \cdot r.\end{aligned}$$

To shorten the notation let us introduce

$$T(x) :\equiv \mathsf{transportconst}_p^B(f(x)).$$

Then,

$$\begin{aligned}\phi(\psi(r)) &\equiv T(x) \cdot (T(x)^{-1} \cdot r)\\ &=_{p_*(f(x))=_B f(y)} (T(x) \cdot T(x)^{-1}) \cdot r\\ &=_{p_*(f(x))=_B f(y)} \mathsf{refl}_{p_*(f(x))} \cdot r\\ &\equiv r.\end{aligned}$$

and

$$\begin{aligned}\psi(\phi(q)) &\equiv T(x)^{-1} \cdot (T(x) \cdot q)\\ &=_{f(x)=_B f(y)} (T(x)^{-1} \cdot T(x)) \cdot q\\ &=_{f(x)=_B f(y)} \mathsf{refl}_{f(x)} \cdot q\\ &\equiv q.\end{aligned}$$

□

**Exercise 2.19.5.** *Let $A$ be a type and let $x, y, z\colon A$ with a path $p\colon x =_A y$. Show that the function $(p \cdot -)\colon (y =_A z) \to (x =_A z)$ is an equivalence.*

*Solution.* Consider the type family $E(w) :\equiv w =_A z$. By Lemma 2.14.4 b) applied to the case $a \equiv z$ and $p^{-1}\colon y =_A x$, we know that

$$\begin{aligned}(y =_A z) &\to (x =_A z)\\ q &\mapsto p \cdot q\end{aligned}$$

is an equivalence. □

**Exercise 2.19.6.** *State and prove a generalization of* Theorem 2.7.10 *from cartesian products to* $\Sigma$*-types.*

*Solution.* See Theorem 2.9.19. □

**Exercise 2.19.7.** *State and prove the analogue of* Theorem 2.7.10 *for coproducts.*

*Solution.* See Theorem 2.15.4. □

**Exercise 2.19.8.**[FE] *Prove that coproducts have the expected universal property,*

$$((A + B) \to X) \simeq (A \to X) \times (B \to X).$$

*Can you generalize this to an equivalence involving dependent functions?*

*Solution.* Define

$$\begin{aligned}\varphi\colon (A + B \to X) &\to ((A \to X) \times (B \to X))\\ f &\mapsto (f \circ \mathsf{inl}, f \circ \mathsf{inr}).\end{aligned}$$

For its quasi-inverse, consider

$$\begin{aligned}\psi\colon ((A \to X) \times (B \to X)) &\to (A + B \to X)\\ (u, v) &\mapsto \mathsf{rec}_{A+B}(X, u, v).\end{aligned}$$

For the first composition, the definition of $\mathsf{rec}_{A+B}$ yields

$$\begin{aligned}\varphi(\psi(u, v)) &\equiv \big(\mathsf{rec}_{A+B}(X, u, v) \circ \mathsf{inl}, \mathsf{rec}_{A+B}(X, u, v) \circ \mathsf{inr}\big)\\ &\equiv (u, v).\end{aligned}$$

For the second composition, we have

$$\psi(\varphi(f)) \equiv \mathsf{rec}_{A+B}(X, f \circ \mathsf{inl}, f \circ \mathsf{inr}).$$

From the definition of $\mathsf{rec}_{A+B}$, we obtain

$$\begin{aligned}\psi(\varphi(f))(\mathsf{inl}(a)) &\equiv f(\mathsf{inl}(a)),\\ \psi(\varphi(f))(\mathsf{inr}(b)) &\equiv f(\mathsf{inr}(b)),\end{aligned}$$

for all $a\colon A$ and $b\colon B$. By function extensionality, this pointwise equality constructs paths

$$(\psi(\varphi(f))) \circ \mathsf{inl} =_{A\to X} f \circ \mathsf{inl},$$
$$(\psi(\varphi(f))) \circ \mathsf{inr} =_{B\to X} f \circ \mathsf{inr}.$$

Finally, from the induction principle of $A + B$, we conclude

$$\psi(\varphi(f)) =_{A+B\to X} f.$$

**Generalization.** For dependent functions the previous equivalence becomes

$$\prod_{s\colon A+B} C(s) \simeq \Big(\prod_{a\colon A} C(\mathsf{inl}(a))\Big) \times \Big(\prod_{b\colon B} C(\mathsf{inlr}(b))\Big).$$

Define

$$\varphi\colon \prod_{s\colon A+B} C(s) \to \Big(\prod_{a\colon A} C(\mathsf{inl}(a))\Big) \times \Big(\prod_{b\colon B} C(\mathsf{inlr}(b))\Big)$$
$$f \mapsto (f \circ \mathsf{inl}, f \circ \mathsf{inr}).$$

For the quasi-inverse, consider

$$\psi\colon \Big(\prod_{a\colon A} C(\mathsf{inl}(a))\Big) \times \Big(\prod_{b\colon B} C(\mathsf{inr}(b))\Big) \to \prod_{s\colon A+B} C(s)$$
$$(u, v) \mapsto \mathsf{ind}_{A+B}(C, u, v).$$

The proofs of $\varphi \circ \psi \sim \mathsf{id}$ and $\psi \circ \varphi \sim \mathsf{id}$ are analogous to the previous case. □

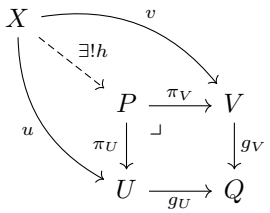
**Exercise 2.19.9.** *Prove that $\Sigma$-types are "associative", in that for any $A\colon \mathcal{U}$ and families $B\colon A \to \mathcal{U}$ and $C\colon \Sigma_A B \to \mathcal{U}$, we have*

$$\sum_{x\colon A} \sum_{y\colon B(x)} C(x, y) \simeq \sum_{p\colon \Sigma_A B} C(p).$$

*Solution.* See Theorem 2.9.14. □

**Exercise 2.19.10.** *Formulate the universal property of a pullback square as an equivalence of types in Homotopy Type Theory.*

*Solution.* Recall the universal property of the pullback captured by the following commutative diagram:



In the context of Homotopy Type Theory, however, trying to translate the categorical definition word for word leads to a complex setting of conditions involving 1-, 2-, and 3-paths.

A better way to address the problem is to express the universal property as an equivalence between types.

To define those types, fix the functions

$$U \xrightarrow{g_U} Q \xleftarrow{g_V} V$$

and observe that a square subtended by $(u, v)\colon X \to U \times V$ is commutative when the type

$$W(u, v) :\equiv g_U \circ u =_{X \to Q} g_V \circ v$$

is inhabited. Thus, the type of commutative squares is

$$\Sigma(X) :\equiv \sum_{(u,v)\colon X \to (U \times V)} W(u, v),$$

where we have identified $X \to (U \times V)$ with $(X \to U) \times (X \to V)$.

On the other hand, the type

$$X \to P$$

represents the type of functions $h$ such that the square induced by $h$ via pre-composition with $\langle(\pi_U, \pi_V), p\rangle\colon \Sigma(P)$ is a pullback.

Therefore, we can define a canonical map

$$\begin{aligned} \varphi\colon (X \to P) &\to \Sigma(X) \\ h &\mapsto \big\langle(\pi_U \circ h, \pi_V \circ h), p \triangleright h\big\rangle, \end{aligned}$$

where $p \triangleright h$ denotes right whiskering.[28]

*Definition.* A commutative square $\langle(\pi_U, \pi_V), p\rangle\colon \Sigma(P)$ is a *pullback* if the canonical map $\varphi$ is an equivalence for every type $X\colon \mathcal{U}$.

Suppose that $\langle(\pi_U, \pi_V), p\rangle$ is a pullback. Since $\varphi$ is an embedding, from equality in Cartesian products and in $\Sigma$-types, we obtain

$$\left.\begin{aligned} q_U &\colon \pi_U \circ h' =_{X \to U} \pi_U \circ h \\ q_V &\colon \pi_V \circ h' =_{X \to V} \pi_V \circ h \\ \tau_{UV} &\colon \mathsf{transport}^W\big(q_{UV}, p \triangleright h'\big) =_{W(\pi_U \circ h, \pi_V \circ h)} p \triangleright h \end{aligned}\right\} \implies h' =_{X \to P} h,$$

where $q_{UV} :\equiv \mathsf{pair}^{=}(q_U, q_V)$. By Proposition 2.12.6, the path $\tau_{UV}$ computes to

$$\tau_{UV}\colon (g_U \triangleleft q_U)^{-1} \cdot (p \triangleright h') \cdot (g_V \triangleleft q_V) =_{W(\pi_U \circ h, \pi_V \circ h)} p \triangleright h,$$

[28] As defined in Naturality of Function Extensionality.

or equivalently (recycling the notation)

$$\tau_{UV} : (p \triangleright h') \cdot (g_V \triangleleft q_V) =_{W(\pi_U \circ h', \pi_V \circ h)} (g_U \triangleleft q_U) \cdot (p \triangleright h).$$

To verify this last equality, note that the LHS starts at $g_U \circ \pi_U \circ h'$ and ends at $g_V \circ v$. The RHS also starts at $g_U \circ \pi_U \circ h'$ and ends at $g_V \circ v$. Therefore, the equality is between two paths of type $g_U \circ \pi_U \circ h' =_{X \to Q} g_V \circ v$, which corresponds exactly to the definition of $W(\pi_U \circ h', v)$.

On the other hand, considering a quasi-inverse map

$$\varphi^{-1} : \Sigma(X) \to (X \to P),$$

for each square $s :\equiv \langle (u, v), q \rangle : \Sigma(X)$, we obtain a function $h : X \to P$ and a homotopy

$$\varepsilon : \varphi(h) =_{\Sigma(X)} s.$$

By definition of the canonical map, this equality expands to

$$\varepsilon : \langle (\pi_U \circ h, \pi_V \circ h), p \triangleright h \rangle =_{\Sigma(X)} \langle (u, v), q \rangle.$$

Unpacking this equality in the $\Sigma$-type $\Sigma(X)$ yields:

$$\begin{aligned} r_U &: \pi_U \circ h =_{X \to U} u, \\ r_V &: \pi_V \circ h =_{X \to V} v, \\ \tau &: \mathsf{transport}^W\big(r_{UV}, p \triangleright h\big) =_{W(u,v)} q, \end{aligned}$$

where $r_{UV} :\equiv \mathsf{pair}^{=}(r_U, r_V)$.

Proposition 2.12.6 applied to this case computes the transport function above as

$$\mathsf{transport}^W(r_{UV}, p \triangleright h) =_{W(u,v)} (g_U \triangleleft r_U)^{-1} \cdot (p \triangleright h) \cdot (g_V \triangleleft r_V),$$

and so we can express the third equality as

$$\tau : (g_U \triangleleft r_U^{-1}) \cdot (p \triangleright h) \cdot (g_V \triangleleft r_V) =_{W(u,v)} q,$$

by Proposition 2.12.5.

□

**Exercise 2.19.11.** *Suppose we are given two commutative squares*

$$\begin{array}{ccccc} A & \longrightarrow & C & \longrightarrow & E \\ \downarrow & & \downarrow & \lrcorner & \downarrow \\ B & \longrightarrow & D & \longrightarrow & F \end{array}$$

*and suppose that the right-hand square is a pullback. Prove that the left-hand square is a pullback square if, and only if, the outer rectangle is a pullback.*

*Solution.* Here we adopt the function names shown in the following commutative diagram:

$$\begin{array}{ccccc} A & \xrightarrow{\pi_C} & C & \xrightarrow{\pi_E} & E \\ {\scriptstyle \pi_B}\downarrow & & {\scriptstyle \pi_D}\downarrow & \lrcorner & \downarrow{\scriptstyle g_E} \\ B & \xrightarrow[g_B]{} & D & \xrightarrow[g_D]{} & F \end{array}$$

where the commutativity of both squares is witnessed by

$$p_L \colon g_B \circ \pi_B =_{A \to D} \pi_D \circ \pi_C \quad \text{and} \quad p_R \colon g_D \circ \pi_D =_{C \to F} g_E \circ \pi_E.$$

We add subscripts $R$ and $L$ to the notation of the previous exercise ($\varphi$, $W$, and $\Sigma$) to adapt it to the right and left squares, while dropping them for the outer rectangle. Thus, our setting defines

$$\begin{aligned} \varphi_R \colon (X \to C) &\to \Sigma_R(X) \\ h &\mapsto \langle (\pi_D \circ h, \pi_E \circ h), p_R \triangleright h \rangle, \\ \varphi_L \colon (X \to A) &\to \Sigma_L(X) \\ h &\mapsto \langle (\pi_B \circ h, \pi_C \circ h), p_L \triangleright h \rangle, \\ \varphi \colon (X \to A) &\to \Sigma(X) \\ h &\mapsto \langle (\pi_B \circ h, \pi_E \circ \pi_C \circ h), p \triangleright h \rangle. \end{aligned}$$

The problem statement becomes:

*If $\varphi_R$ is an equivalence, $\varphi_L$ is an equivalence if, and only if, $\varphi$ is.*

We will establish an equivalence

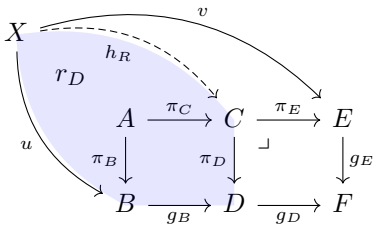
$$\psi \colon \Sigma(X) \to \Sigma_L(X).$$

Let $\langle (u, v), q \rangle \colon \Sigma(X)$. Since $\langle (g_B \circ u, v), q \rangle \colon \Sigma_R(X)$ and $\varphi_R$ is an equivalence, there exists a function $h_R \colon X \to C$, namely $\varphi_R^{-1} \langle (g_B \circ u, v), q \rangle$, such that

$$\varphi_R(h_R) =_{\Sigma_R(X)} \langle (g_B \circ u, v), q \rangle,$$

In particular, the first component of the equality is a path $r_D$ that witnesses the commutativity of the shaded subdiagram

$$r_D \colon \pi_D \circ h_R =_{X \to D} g_B \circ u$$




We define

$$\psi(\langle (u, v), q \rangle) :\equiv \langle (u, h_R), r_D^{-1} \rangle.$$

For the backward function, we set

$$\begin{aligned}\phi &\colon \Sigma_L(X) \to \Sigma(X)\\ \langle (u,k), r\rangle &\mapsto \langle (u, \pi_E \circ k), (g_D \lhd r) \cdot (p_R \rhd k)\rangle,\end{aligned}$$

where $r \colon g_B \circ u =_{X\to D} \pi_D \circ k$.

We claim that $\psi$ and $\phi$ are quasi-inverses.

Using $r^{-1}$ for the first coordinate of the pair and $\mathsf{refl}_{\pi_E\circ k}$ for the second, we obtain

$$\langle (\pi_D \circ k, \pi_E \circ k), p_R \rhd k\rangle =_{\Sigma_R(X)} \langle (g_B \circ u, \pi_E \circ k), \mathsf{transport}^{W_R}(\gamma, p_R \rhd k)\rangle,$$

where

$$\gamma :\equiv \mathsf{pair}^{=}(r^{-1}, \mathsf{refl}_{\pi_E\circ k}).$$

Proposition 2.12.6 yields:

$$\begin{aligned}&\mathsf{transport}^{W_R}\big(\gamma, p_R \rhd k\big)\\ &\qquad =_{W_R(g_B\circ u, \pi_E\circ k)} (g_D \lhd r^{-1})^{-1} \cdot (p_R \rhd k) \cdot (g_E \lhd \mathsf{refl}_{\pi_E\circ k})\\ &\qquad =_{W_R(g_B\circ u, \pi_E\circ k)} (g_D \lhd r) \cdot (p_R \rhd k),\end{aligned}$$

since $(g_E \lhd \mathsf{refl}_{\pi_E\circ k}) =_{g_E\circ\pi_E\circ k = X\to F g_E\circ\pi_E\circ k} \mathsf{refl}_{g_E\circ\pi_E\circ k}$. Therefore,

$$\begin{aligned}\varphi_R(k) &\equiv \langle (\pi_D \circ k, \pi_E \circ k), p_R \rhd k\rangle\\ &\qquad =_{\Sigma_R(X)} \langle (g_B \circ u, \pi_E \circ k), (g_D \lhd r) \cdot (p_R \rhd k)\rangle.\end{aligned}$$

Let us first compute the composition $\psi\circ\phi$. Let $s :\equiv \phi(\langle (u,k), r\rangle)$. By definition,

$$\begin{aligned}\psi(s) &\equiv \psi\big(\langle (u, \pi_E \circ k), (g_D \lhd r) \cdot (p_R \rhd k)\rangle\big)\\ &\qquad \equiv \langle (u, h_R), r_D^{-1}\rangle,\end{aligned}$$

where $h_R$ is defined by the equation

$$e_1 \colon \varphi_R(h_R) =_{\Sigma_R(X)} \langle (g_B \circ u, \pi_E \circ k), (g_D \lhd r) \cdot (p_R \rhd k)\rangle,$$

and $r_D \colon \pi_D \circ h_R = g_B \circ u$ is extracted from the first component of the base path of $e_1$.

From our previous derivation, we also have an explicit equality

$$e_2 \colon \varphi_R(k) =_{\Sigma_R(X)} \langle (g_B \circ u, \pi_E \circ k), (g_D \lhd r) \cdot (p_R \rhd k)\rangle,$$

whose base path is $\gamma :\equiv \mathsf{pair}^{=}(r^{-1}, \mathsf{refl}_{\pi_E\circ k})$.

Concatenating these paths yields $e_1 \cdot e_2^{-1} \colon \varphi_R(h_R) =_{\Sigma_R(X)} \varphi_R(k)$. Because $\varphi_R$ is an equivalence, there is a path $\alpha \colon h_R =_{X\to C} k$ such that $\varphi_R \lhd \alpha = e_1 \cdot e_2^{-1}$.

Concatenating these paths yields $e_1 \cdot e_2^{-1} \colon \varphi_R(h_R) =_{\Sigma_R(X)} \varphi_R(k)$. Because $\varphi_R$ is an equivalence, it is an embedding, which implies there is a unique path $\alpha \colon h_R =_{X \to C} k$ such that $\varphi_R \lhd \alpha = e_1 \cdot e_2^{-1}$.

By the definition of $\varphi_R$, the base path of $\varphi_R \lhd \alpha$ equals $(\pi_D \lhd \alpha, \pi_E \lhd \alpha)$. Since the first component of $e_1 \cdot e_2^{-1}$ is $r_D \cdot (r^{-1})^{-1}$, we deduce that

$$\pi_D \lhd \alpha = r_D \cdot r.$$

To establish the equality $\langle (u, h_R), r_D^{-1} \rangle =_{\Sigma_L(X)} \langle (u, k), r \rangle$, we use the base path $\mathsf{pair}^{=}(\mathsf{refl}_u, \alpha)$ and compute the transport of $r_D^{-1}$ in the family $W_L$. By Proposition 2.12.6,

$$\begin{aligned} &\mathsf{transport}^{W_L}\big(\mathsf{pair}^{=}(\mathsf{refl}_u, \alpha), r_D^{-1}\big) \\ &\qquad = (g_B \lhd \mathsf{refl}_u)^{-1} \cdot r_D^{-1} \cdot (\pi_D \lhd \alpha) \\ &\qquad = \mathsf{refl}_{g_B \circ u}^{-1} \cdot r_D^{-1} \cdot (r_D \cdot r) \\ &\qquad = r. \end{aligned}$$

For the other composition, we have

$$\begin{aligned} \phi(\psi(\langle (u, v), q \rangle)) &\equiv \phi(\langle (u, h_R), r_D^{-1} \rangle) \\ &\equiv \langle (u, \pi_E \circ h_R), (g_D \lhd r_D^{-1}) \cdot (p_R \rhd h_R) \rangle. \end{aligned}$$

By the definition of $h_R$, we have a witness

$$r_E \colon \pi_E \circ h_R =_{X \to E} v.$$

Thus, with $\varepsilon :\equiv \mathsf{pair}^{=}(\mathsf{refl}_u, r_E)$, the question reduces to proving that

$$\mathsf{transport}^{W}(\varepsilon, (g_D \lhd r_D^{-1}) \cdot (p_R \rhd h_R)) = q.$$

By Proposition 2.12.6, we have

$$\begin{aligned} &\mathsf{transport}^{W}\big(\varepsilon, (g_D \lhd r_D^{-1}) \cdot (p_R \rhd h_R)\big) \\ &\qquad = (g_D \circ g_B \lhd \mathsf{refl}_u)^{-1} \cdot \big((g_D \lhd r_D^{-1}) \cdot (p_R \rhd h_R)\big) \cdot (g_E \lhd r_E) \\ &\qquad = \mathsf{refl}_{g_D \circ g_B \circ u}^{-1} \cdot (g_D \lhd r_D^{-1}) \cdot (p_R \rhd h_R) \cdot (g_E \lhd r_E) \\ &\qquad = (g_D \lhd r_D^{-1}) \cdot (p_R \rhd h_R) \cdot (g_E \lhd r_E). \end{aligned}$$

By the definition of $h_R :\equiv \varphi_R^{-1}((g_B \circ u, v), q)$, we have

$$\varphi_R(h_R) =_{\Sigma_R(X)} \langle (g_B \circ u, v), q \rangle$$

and

$$r_D \colon \pi_D \circ h_R =_{X \to D} g_B \circ u.$$

Therefore,

$$q = \mathsf{transport}^{W_R}\big(\mathsf{pair}^{=}(r_D, r_E), p_R \rhd h_R\big).$$

Using Proposition 2.12.6 again, we obtain

$$q = (g_D \triangleleft r_D)^{-1} \cdot (p_R \triangleright h_R) \cdot (g_E \triangleleft r_E),$$

as desired.

□

**Exercise 2.19.12.**[FE] *Show that* $(2 \simeq 2) \simeq 2$.

*Solution.* We will need the following lemma:

**Lemma.** *There is a function* $\mathsf{neq} : (1_2 =_2 0_2) \to 0$.

PROOF: Let $P : 2 \to \mathcal{U}$ be the type family defined by $P(1_2) :\equiv 1$ and $P(0_2) :\equiv 0$. Then, we define

$$\mathsf{neq} :\equiv \lambda x : 1_2 =_2 0_2.\, \mathsf{transport}^P(x, \star).$$

■

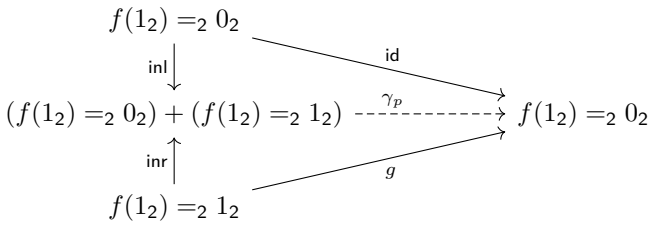
By 46. EXACTLY TWO BOOLEANS, there is a witness

$$\varepsilon : \prod_{x:2} (x =_2 0_2) + (x =_2 1_2).$$

In particular,

$$\varepsilon(f(1_2)) : (f(1_2) =_2 0_2) + (f(1_2) =_2 1_2).$$

Fix $f : 2 \simeq 2$. Suppose that $f(0_2) =_2 1_2$, and let $p$ witness this identity. We want to prove that $f(1_2) =_2 0_2$. Consider the diagram




where $g$ is defined as follows. Given $r : f(1_2) =_2 1_2$, we have

$$p \cdot r^{-1} : f(0_2) =_2 f(1_2).$$

Hence, $\mathsf{ap}_{f^{-1}}(p \cdot r^{-1}) : 0_2 =_2 1_2$, and we can use the lemma to set

$$g(r) :\equiv \mathsf{rec}_0\big(f(1_2) =_2 0_2, \mathsf{neq}(\mathsf{ap}_{f^{-1}}(p \cdot r^{-1}))\big).$$

As the diagram shows, we obtain

$$\gamma_p :\equiv \mathsf{rec}_+\big(f(1_2) =_2 0_2, \mathsf{id}, g\big).$$

It follows that

$$\gamma_p(\varepsilon(f(1_2)))\colon f(1_2) =_2 0_2,$$

which constructs a function

$$\big(\lambda p.\, \gamma_p(\varepsilon(f(1_2)))\big)\colon \big(f(0_2) =_2 1_2\big) \to \big(f(1_2) =_2 0_2\big).$$

Since $\mathsf{not}$[29] is characterized by its evaluation at $0_2$ and $1_2$, the function extensionality axiom yields a function

$$(f(0_2) =_2 1_2) \to (f =_{2\simeq 2} \mathsf{not}).$$

A similar argument yields a function

$$(f(0_2) =_2 0_2) \to (f =_{2\simeq 2} \mathsf{id}).$$

These functions combine to construct a map

$$(f(0_2) =_2 0_2) + (f(0_2) =_2 1_2) \to (f =_{2\simeq 2} \mathsf{id}) + (f =_{2\simeq 2} \mathsf{not}).$$

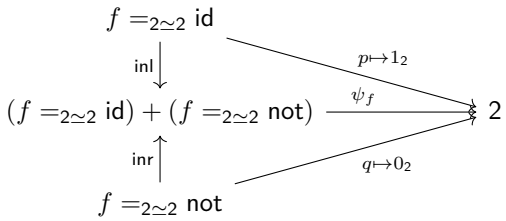
Since

$$\varepsilon(f(0_2))\colon (f(0_2) =_2 0_2) + (f(0_2) =_2 1_2),$$

we deduce the existence of a dependent function

$$\phi\colon \prod_{f\colon 2\simeq 2} (f =_{2\simeq 2} \mathsf{id}) + (f =_{2\simeq 2} \mathsf{not}).$$

Let us now define $\psi_f$ as illustrated in the diagram:




We can define the functions $\alpha$ and $\beta$ as follows:

$$\begin{aligned} \alpha\colon (2 \simeq 2) &\to 2, & \beta\colon 2 &\to (2 \simeq 2),\\ \alpha(f) &:\equiv \psi_f(\phi(f)), & \beta(1_2) &:\equiv \mathsf{id},\\ & & \beta(0_2) &:\equiv \mathsf{not}. \end{aligned}$$

In addition,

$$\begin{aligned} \beta(\psi_f(\mathsf{inl}(p))) &=_{2\simeq 2} \beta(1_2) \equiv \mathsf{id},\\ \beta(\psi_f(\mathsf{inr}(q))) &=_{2\simeq 2} \beta(0_2) \equiv \mathsf{not}. \end{aligned}$$

[29] See Exercise 1.16.2.

To prove that $\beta(\alpha(f)) =_{2\simeq 2} f$, we need to show that

$$\beta(\psi_f(\phi(f))) =_{2\simeq 2} f.$$

To do this, we define the motive $P\colon (f =_{2\simeq 2} \mathsf{id}) + (f =_{2\simeq 2} \mathsf{not}) \to \mathcal{U}$ as

$$P(x) :\equiv \beta(\psi_f(x)) =_{2\simeq 2} f.$$

By the induction principle, the goal reduces to proving the cases for $\mathsf{inl}$ and $\mathsf{inr}$. Using the calculations above for $\beta \circ \psi_f$, we obtain the equalities

$$P(\mathsf{inl}(p)) = (\mathsf{id} =_{2\simeq 2} f),$$
$$P(\mathsf{inr}(q)) = (\mathsf{not} =_{2\simeq 2} f).$$

In the first case, $p\colon f =_{2\simeq 2} \mathsf{id}$; in the second, $q\colon f =_{2\simeq 2} \mathsf{not}$. Therefore, their inverses inhabit the respective types:

$$p^{-1}\colon P(\mathsf{inl}(p)) \quad \text{and} \quad q^{-1}\colon P(\mathsf{inr}(q)).$$

For the backward composition, observe that

$$\begin{aligned}
\alpha(\beta(1_2)) &\equiv \alpha(\mathsf{id}) \\
&\equiv \psi_{\mathsf{id}}(\phi(\mathsf{id})) \\
&\equiv \psi_{\mathsf{id}}(\mathsf{inl}(\mathsf{refl}_{\mathsf{id}})) \\
&\equiv 1_2, \\
\alpha(\beta(0_2)) &\equiv \alpha(\mathsf{not}) \\
&\equiv \psi_{\mathsf{not}}(\phi(\mathsf{not})) \\
&\equiv \psi_{\mathsf{not}}(\mathsf{inr}(\mathsf{refl}_{\mathsf{not}})) \\
&\equiv 0_2.
\end{aligned}$$

□

**Exercise 2.19.13.** *Suppose we add to type theory the equality reflection rule which says that if there is an element $p\colon x = y$, then in fact $x \equiv y$. Prove that for any $p\colon x = x$ we have $p \equiv \mathsf{refl}_x$.*

*Solution.* Under the stated conditions, it suffices to prove that if $p\colon x = x$, then $p = \mathsf{refl}_x$. In other words, we have to prove that the type

$$\prod_{x\colon A} \prod_{p\colon x =_A x} p = \mathsf{refl}_x$$

is inhabited. More generally, we consider the type

$$\prod_{x,y\colon A} \prod_{p\colon x =_A y} p = \mathsf{refl}_x.$$

By path induction on $p$, we can restrict ourselves to the case where $y \equiv x$ and $p \equiv \mathsf{refl}_x$, in which case the goal becomes trivial.

□

**Exercise 2.19.14.** *Show that* Proposition 2.13.6 *can be strengthened to*

$$\mathsf{transport}^B(p,-) =_{B(x)\to B(y)} \mathsf{idtoeqv}(\mathsf{ap}_B(p))$$

*without using function extensionality. (In this and other similar cases, the apparently weaker formulation has been chosen for readability and consistency.)*

*Solution.* The implicit assumptions here are that $B\colon A \to \mathcal{U}$ is a type family, and that $p\colon x =_A y$.

The RHS denotes the equivalence $\mathsf{idtoeqv}(\mathsf{ap}_B(p))$ of type $B(x) \simeq B(y)$. By abuse of notation, it is identified with its underlying function $f\colon B(x) \to B(y)$, which computes exactly as $\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_B(p), -)$.

Therefore, the goal is to show that the type

$$\mathsf{transport}^B(p,-) =_{B(x)\to B(y)} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_B(p), -)$$

is inhabited.

Since both sides are clearly of the same type $B(x) \to B(y)$, to prove the equality we can apply path induction on $p$. Hence, it suffices to consider the case $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In this case, we have

$$\begin{aligned} \text{LHS} &\equiv \mathsf{transport}^B(\mathsf{refl}_x, -) \\ &\equiv \mathsf{id}_{B(x)}. \\ \text{RHS} &\equiv \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{refl}_{B(x)}, -) \\ &\equiv \mathsf{id}_{B(x)}. \end{aligned}$$

Thus, both sides are definitionally equal in the base case, meaning $\mathsf{refl}_{\mathsf{id}_{B(x)}}$ inhabits the goal type. Path induction then yields the equality in the general case.

□

**Exercise 2.19.15.** FE UA

a) *Show that if* $A \simeq B$ *and* $A' \simeq B'$*, then* $(A \times A') \simeq (B \times B')$.

b) *Give two proofs of this fact, one using univalence and one not using it, and show that the two proofs are equal.*

c) *Formulate and prove analogous results for the other type formers:* $\Sigma$*,* $\to$*,* $\Pi$*, and* $+$*.*[30]

[30] Function extensionality is used for Π-types.

*Solution.*

a) Let $(f, (\eta, \vartheta))\colon A \simeq B$ and $(g, (\eta', \vartheta'))\colon A' \simeq B'$. Consider the function

$$\begin{aligned}(f \times g)\colon (A \times A') &\to (B \times B')\\ (a, a') &\mapsto (f(a), g(a')).\end{aligned}$$

Let $f^{-1}$ and $g^{-1}$ be the respective quasi-inverses of $f$ and $g$. Then,

$$(f \times g) \circ (f^{-1} \times g^{-1})(b, b') \equiv \big(f(f^{-1}(b)), g(g^{-1}(b'))\big).$$

Since $\eta(b)\colon f(f^{-1}(b)) =_B b$ and $\eta'(b')\colon g(g^{-1}(b')) =_{B'} b'$, we define

$$(\eta \wedge \eta')(b, b') :\equiv \mathsf{pair}^{=}\big(\eta(b), \eta'(b')\big),$$

which provides the first homotopy

$$(\eta \wedge \eta')\colon (f \times g) \circ (f^{-1} \times g^{-1}) \sim \mathsf{id}_{B \times B'}.$$

By symmetry we obtain

$$(\vartheta \wedge \vartheta')\colon (f^{-1} \times g^{-1}) \circ (f \times g) \sim \mathsf{id}_{A \times A'}.$$

Note that this defines the product of equivalences:

$$(f, (\eta, \vartheta)) \times (g, (\eta', \vartheta')) :\equiv \big(f \times g, (\eta \wedge \eta', \vartheta \wedge \vartheta')\big). \tag{2.5}$$

b) By univalence, we can apply the $\mathsf{ua}$ function to the equivalences $(f, (\eta, \vartheta))$ and $(g, (\eta', \vartheta'))$ to obtain paths $\phi\colon A =_{\mathcal{U}} B$ and $\psi\colon A' =_{\mathcal{U}} B'$.

This transforms the question into proving that $(A \times A') =_{\mathcal{U}} (B \times B')$ is inhabited.

By based path induction, first on $\phi$ and then on $\psi$, the goal reduces to the base case where $B \equiv A$ with $\phi \equiv \mathsf{refl}_A$, and $B' \equiv A'$ with $\psi \equiv \mathsf{refl}_{A'}$. In this case, the target equality is trivially witnessed by $\mathsf{refl}_{A \times A'}$.

Applying $\mathsf{idtoeqv}$ yields the required equivalence.

To show that both proofs are the same, we need to be more precise.

- Fix $B \equiv A$ and $\phi \equiv \mathsf{refl}_A$. Consider the motive

$$\begin{aligned}E\colon \prod_{X:\mathcal{U}} (A' =_{\mathcal{U}} X) &\to \mathcal{U}\\ E(X) &:\equiv \lambda z.\, (A \times A') =_{\mathcal{U}} (A \times X),\end{aligned}$$

where $E(X)$ is constant (it does not depend on $z$).

For the base case $c_{\mathsf{refl}}\colon E(A', \mathsf{refl}_{A'})$, we take reflexivity:

$$c_{\mathsf{refl}} :\equiv \mathsf{refl}_{A \times A'}.$$

The induction principle yields:

$$\gamma_\psi :\equiv \mathsf{ind}_{=}(E, \mathsf{refl}_{A \times A'}, B', \psi)\colon (A \times A') =_{\mathcal{U}} (A \times B').$$

- To complete the based induction on $\phi$, we fix $\psi\colon A' =_{\mathcal{U}} B'$ and introduce the motive

$$F_\psi\colon \prod_{Y:\mathcal{U}} (A =_{\mathcal{U}} Y) \to \mathcal{U}$$
$$F_\psi(Y) :\equiv \lambda z.\, (A \times A') =_{\mathcal{U}} (Y \times B'),$$

where, as before, $F_\psi(Y)$ is constant on $z$. For the base case where $Y \equiv A$, we take $\gamma_\psi$. Thus, we can define

$$\gamma :\equiv \mathsf{ind}_=(F_\psi, \gamma_\psi, B, \phi)\colon (A \times A') =_{\mathcal{U}} (B \times B').$$

Consider the goal type

$$\prod_{\phi:\, A =_{\mathcal{U}} B} \; \prod_{\psi:\, A' =_{\mathcal{U}} B'} \mathsf{idtoeqv}(\gamma) = \mathsf{idtoeqv}(\phi) \times \mathsf{idtoeqv}(\psi),$$

where the product on the RHS is the one defined in (2.5).

Since both sides of the equation inhabit the type $(A \times A') \simeq (B \times B')$, to prove that it is inhabited we may proceed by path induction on $\phi$ and $\psi$. Thus, we assume $B \equiv A$ with $\phi \equiv \mathsf{refl}_A$, and $B' \equiv A'$ with $\psi \equiv \mathsf{refl}_{A'}$. Then,

$$\begin{aligned}
\gamma_\psi &\equiv \mathsf{refl}_{A \times A'},\\
\gamma \equiv \gamma_\psi &\equiv \mathsf{refl}_{A \times A'},\\
\mathsf{idtoeqv}(\gamma) &\equiv (\mathsf{id}_{A \times A'}, (\mathsf{refl}, \mathsf{refl})),\\
\mathsf{idtoeqv}(\phi) &\equiv (\mathsf{id}_A, (\mathsf{refl}, \mathsf{refl})),\\
\mathsf{idtoeqv}(\psi) &\equiv (\mathsf{id}_{A'}, (\mathsf{refl}, \mathsf{refl})),\\
\mathsf{idtoeqv}(\phi) \times \mathsf{idtoeqv}(\psi) &\equiv \big(\mathsf{id}_A \times \mathsf{id}_{A'}, (\mathsf{refl} \wedge \mathsf{refl}, \mathsf{refl} \wedge \mathsf{refl})\big).
\end{aligned}$$

Consequently, we need to verify that

$$\mathsf{id}_{A \times A'} = \mathsf{id}_A \times \mathsf{id}_{A'}$$

and

$$\mathsf{refl} \wedge \mathsf{refl} = \mathsf{refl}.$$

The first equality is actually definitional, modulo abuse of notation. The second follows from function extensionality because

$$\begin{aligned}
\mathsf{refl}(x) &:\equiv \mathsf{refl}_x,\\
(\mathsf{refl} \wedge \mathsf{refl})(x, y) &\equiv \mathsf{pair}^{=}(\mathsf{refl}_x, \mathsf{refl}_y)\\
&\equiv \mathsf{refl}_{(x,y)}\\
&\equiv \mathsf{refl}(x, y).
\end{aligned}$$

c) We proceed by type former

Σ) Let $X : \mathcal{U}$ be a type and $A, B : X \to \mathcal{U}$ be type families. Suppose that for each $x : X$, we have an equivalence $\langle f_x, (\eta_x, \vartheta_x)\rangle : A(x) \simeq B(x)$. We define the induced function

$$f : \Big(\sum_{x:X} A(x)\Big) \to \Big(\sum_{x:X} B(x)\Big)$$
$$\langle x, a\rangle \mapsto \langle x, f_x(a)\rangle.$$

For each $x : X$, let $f_x^{-1} : B(x) \to A(x)$ denote the quasi-inverse of $f_x$. We define the proposed inverse

$$f^{-1} : \Big(\sum_{x:X} B(x)\Big) \to \Big(\sum_{x:X} A(x)\Big)$$
$$\langle x, b\rangle \mapsto \langle x, f_x^{-1}(b)\rangle.$$

Then,

$$\begin{aligned} f \circ f^{-1}(\langle x, b\rangle) &\equiv f(\langle x, f_x^{-1}(b)\rangle) \\ &\equiv \langle x, f_x(f_x^{-1}(b))\rangle \\ &=_{\Sigma_X B} \langle x, b\rangle \qquad \text{(by } \eta_x(b)\text{)}. \end{aligned}$$

By symmetry, $f^{-1} \circ f(\langle x, a\rangle) =_{\Sigma_X A} \langle x, a\rangle$. Thus, we can define

$$\eta : f \circ f^{-1} \sim \mathsf{id}_{\Sigma_X B}$$
$$\eta(\langle x, b\rangle) :\equiv \mathsf{pair}^{=}(\langle \mathsf{refl}_x, \eta_x(b)\rangle),$$

which provides the required homotopy.

Similarly, we obtain

$$\vartheta : f^{-1} \circ f \sim \mathsf{id}_{\Sigma_X A}$$
$$\vartheta(\langle x, a\rangle) :\equiv \mathsf{pair}^{=}(\langle \mathsf{refl}_x, \vartheta_x(a)\rangle).$$

This construction yields an equivalence over the Σ-types:

$$\sum_{x:X} \langle f_x, (\eta_x, \vartheta_x)\rangle :\equiv \langle f, (\eta, \vartheta)\rangle. \tag{2.6}$$

Using the univalence axiom function $\mathsf{ua}$, we may transform, for each $x : X$, the equivalence $f_x$ into a path $p_x : A(x) =_{\mathcal{U}} B(x)$. This allows us to define the homotopy

$$\varepsilon : A \sim B$$
$$\varepsilon(x) :\equiv p_x.$$

We obtain $\mathsf{funext}(\varepsilon) : A =_{\mathcal{U}} B$. In particular, using equality (2.5),

$$\mathsf{funext}(\varepsilon)(x) =_{A(x)=_{\mathcal{U}} B(x)} \mathsf{happly}(\mathsf{funext}(\varepsilon))(x) \equiv \varepsilon(x) \equiv \mathsf{ua}(f_x).$$

Now consider

$$\begin{aligned}\sigma\colon (X \to \mathcal{U}) &\to \mathcal{U}\\ P &\mapsto \sum_{x\colon X} P(x).\end{aligned}$$

Then, $\mathsf{ap}_\sigma(\mathsf{funext}(\varepsilon))\colon \Sigma_X A =_{\mathcal{U}} \Sigma_X B$, as desired.

To verify that both proofs are actually the same, we introduce the goal

$$\mathsf{idtoeqv}\big(\mathsf{ap}_\sigma(p)\big) =_{\Sigma_X A \simeq \Sigma_X B} \langle f, (\eta, \vartheta)\rangle, \tag{2.7}$$

where $p :\equiv \mathsf{funext}(\varepsilon)\colon A =_{\mathcal{U}} B$.

To use based path induction to prove the goal, we must first generalize it to an arbitrary path $p\colon A =_{\mathcal{U}} B$. For an arbitrary $p$, we define $\varepsilon_p :\equiv \mathsf{happly}(p)$, and for each $x\colon X$, the following equivalence between $A(x)$ and $B(x)$:

$$\begin{aligned}f_{p,x} &:\equiv \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\varepsilon_p(x))\\ \eta_{p,x} &:\equiv \mathsf{transportinv}_r^{\mathsf{id}_{\mathcal{U}}}(\varepsilon_p(x))\\ \vartheta_{p,x} &:\equiv \mathsf{transportinv}_l^{\mathsf{id}_{\mathcal{U}}}(\varepsilon_p(x)).\end{aligned}$$

Let $\langle f_p, (\eta_p, \vartheta_p)\rangle$ denote the total equivalence between the $\Sigma$-types induced by this family of equivalences.

We can now apply based path induction on $p$, assuming $B \equiv A$ and $p \equiv \mathsf{refl}_A$. In this base case, the components of the LHS of (2.7) evaluate to

$$\begin{aligned}\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_\sigma(p)) &\equiv \mathsf{id}_{\Sigma_X A}\\ \mathsf{transportinv}_r^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_\sigma(p)) &\equiv \mathsf{refl}\\ \mathsf{transportinv}_l^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ap}_\sigma(p)) &\equiv \mathsf{refl}.\end{aligned}$$

For the RHS of (2.7), the assumption $p \equiv \mathsf{refl}_A$ implies that

$$\varepsilon_p \equiv \mathsf{happly}(\mathsf{refl}_A) \equiv \lambda x.\, \mathsf{refl}_{A(x)},$$

so that $\varepsilon_p(x) \equiv \mathsf{refl}_{A(x)}$. Therefore, by definition of the induced components,

$$\begin{aligned}f_{p,x} &\equiv \mathsf{id}_{A(x)}\\ \eta_{p,x} &\equiv \mathsf{refl}\\ \vartheta_{p,x} &\equiv \mathsf{refl}.\end{aligned}$$

Thus, the total function $f_p(\langle x, a\rangle) :\equiv \langle x, f_{p,x}(a)\rangle$ reduces to $\langle x, a\rangle$, and the global homotopies $\eta_p$ and $\vartheta_p$, which are constructed using $\mathsf{pair}^=$, satisfy

$$\begin{aligned}\eta_p(\langle x, a\rangle) &\equiv \mathsf{pair}^=(\mathsf{refl}_x, \eta_{p,x}(a))\\ &\equiv \mathsf{pair}^=(\mathsf{refl}_x, \mathsf{refl}_a)\\ &\equiv \mathsf{refl}_{\langle x,a\rangle},\end{aligned}$$

and

$$\begin{aligned}\vartheta_p(\langle x, a\rangle) &\equiv \mathsf{pair}^=(\mathsf{refl}_x, \vartheta_{p,x}(a))\\ &\equiv \mathsf{pair}^=(\mathsf{refl}_x, \mathsf{refl}_a)\\ &\equiv \mathsf{refl}_{\langle x,a\rangle}.\end{aligned}$$

Since both sides evaluate to the identity equivalence in the base case, the equality

$$\mathsf{idtoeqv}\big(\mathsf{ap}_\sigma(p)\big) =_{\Sigma_X A \simeq \Sigma_X B} \langle f_p, (\eta_p, \vartheta_p)\rangle$$

holds for an arbitrary path $p$.

Applying this equality to $p :\equiv \mathsf{funext}(\varepsilon)$ yields

$$\begin{aligned}\langle f_{p,x}, (\eta_{p,x}, \vartheta_{p,x})\rangle &\equiv \mathsf{idtoeqv}(\mathsf{happly}(\mathsf{funext}(\varepsilon))(x))\\ &\equiv \mathsf{idtoeqv}(\mathsf{ua}(\langle f_x, (\eta_x, \vartheta_x)\rangle))\\ &= (f_x, (\eta_x, \vartheta_x)).\end{aligned}$$

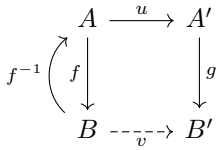
Thus, the generalized equivalence $\langle f_p, (\eta_p, \vartheta_p)\rangle$ evaluates precisely to the arbitrarily chosen equivalence $\langle f, (\eta, \vartheta)\rangle$, completing the proof.

$\to$) Here we have to prove that, given equivalences $\langle f, (\eta, \vartheta)\rangle\colon A \simeq B$ and $\langle g, (\eta', \vartheta')\rangle\colon A' \simeq B'$, the type $(A \to A') \simeq (B \to B')$ is inhabited.

Consider the function

$$\begin{aligned}(f \to g)\colon (A \to A') &\to (B \to B')\\ u &\mapsto v,\end{aligned}$$

where $v$ is defined by the commutativity of the diagram




That is, $(f \to g) :\equiv \lambda u.\, g \circ u \circ f^{-1}$. We define the quasi-inverse as

$$(f \to g)^{-1} :\equiv \lambda v.\, g^{-1} \circ v \circ f.$$

Note that, for $b\colon B$, we have

$$\begin{aligned}(f \to g) \circ (f \to g)^{-1}(v)(b) & \\ &\equiv (f \to g)(g^{-1} \circ v \circ f)(b) \\ &\equiv (g \circ g^{-1} \circ v)(f(f^{-1}(b))) \\ &=_{B'} (g \circ g^{-1})(v(b)) && ; ((g \circ g^{-1} \circ v) \triangleleft \eta)(b) \\ &=_{B'} v(b) && ; (\eta' \triangleright v)(b).\end{aligned}$$

Thus,

$$((g \circ g^{-1} \circ v) \triangleleft \eta) \cdot (\eta' \triangleright v)\colon (f \to g) \circ (f \to g)^{-1}(v) \sim v.$$

Since Proposition 2.12.8 implies

$$((g \circ g^{-1} \circ v) \triangleleft \eta) \cdot (\eta' \triangleright v) = \eta' \bullet (v \triangleleft \eta),$$

we can express the global right homotopy as

$$\begin{aligned}(f \to g) \circ (f \to g)^{-1} &\sim \mathsf{id}_{B \to B'} \\ v &\mapsto \eta' \bullet (v \triangleleft \eta).\end{aligned}$$

By symmetry, we deduce

$$\begin{aligned}(f \to g)^{-1} \circ (f \to g) &\sim \mathsf{id}_{A \to A'} \\ u &\mapsto \vartheta' \bullet (u \triangleleft \vartheta).\end{aligned}$$

Using the univalence axiom, we can transform the equivalences into equalities:

$$\mathsf{ua}(\langle f, (\eta, \vartheta)\rangle)\colon A =_{\mathcal{U}} B \quad \text{and} \quad \mathsf{ua}(\langle g, (\eta', \vartheta')\rangle)\colon A' =_{\mathcal{U}} B'.$$

Now suppose we have generic paths $p\colon A =_{\mathcal{U}} B$ and $q\colon A' =_{\mathcal{U}} B'$. To prove that

$$(A \to A') =_{\mathcal{U}} (B \to B'),$$

by based path induction on $p$, we can assume $B \equiv A$ and $p \equiv \mathsf{refl}_A$. The goal becomes

$$(A \to A') =_{\mathcal{U}} (A \to B').$$

Thus, we may further assume $B' \equiv A'$ and $q \equiv \mathsf{refl}_{A'}$. In this case, the goal becomes

$$(A \to A') =_{\mathcal{U}} (A \to A'),$$

which is inhabited by $\mathsf{refl}_{A \to A'}$. Combining both inductions we obtain

$$\mathsf{rec}_{=_{\mathcal{U}}}\big(A' \to B', \mathsf{rec}_{=_{\mathcal{U}}}(A \to B', \mathsf{refl}_{A \to A'}, A \to A'), A' \to B'\big)$$

as a witness of our goal.

Let $\varepsilon(p,q)\colon (A \to A') =_{\mathcal{U}} (B \to B')$ denote the path obtained by our double recursion. Applying $\mathsf{idtoeqv}$ to the equivalences we have yields:

$$\mathsf{idtoeqv}\big(\varepsilon(\mathsf{ua}(\langle f, (\eta, \vartheta)\rangle), \mathsf{ua}(\langle g, (\eta', \vartheta')\rangle))\big)\colon (A \to A') \simeq (B \to B').$$

To verify that both proofs are equivalent, we have to show that

$$\begin{aligned}
\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\epsilon) &= (f \to g)\\
\mathsf{transportinv}_r^{\mathsf{id}_{\mathcal{U}}}(\epsilon) &= \lambda v.\, \eta' \bullet (v \triangleleft \eta)\\
\mathsf{transportinv}_l^{\mathsf{id}_{\mathcal{U}}}(\epsilon) &= \lambda u.\, \vartheta' \bullet (u \triangleleft \vartheta),
\end{aligned}$$

where

$$\epsilon :\equiv \varepsilon\big(\mathsf{ua}(\langle f, (\eta, \vartheta)\rangle), \mathsf{ua}(\langle g, (\eta', \vartheta')\rangle)\big).$$

If we generalize to $\epsilon_{p,q} :\equiv \varepsilon(p,q)$, as we defined it above, we can introduce

$$\begin{aligned}
\langle f_p, (\eta_p, \vartheta_p)\rangle &:\equiv \mathsf{idtoeqv}(p)\\
\langle g_q, (\eta'_q, \vartheta'_q)\rangle &:\equiv \mathsf{idtoeqv}(q).
\end{aligned}$$

We claim that

$$\begin{aligned}
\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\epsilon_{p,q}) &= (f_p \to g_q)\\
\mathsf{transportinv}_r^{\mathsf{id}_{\mathcal{U}}}(\epsilon_{p,q}) &= \lambda v.\, \eta'_q \bullet (v \triangleleft \eta_p)\\
\mathsf{transportinv}_l^{\mathsf{id}_{\mathcal{U}}}(\epsilon_{p,q}) &= \lambda u.\, \vartheta'_q \bullet (u \triangleleft \vartheta_p).
\end{aligned} \tag{2.8}$$

In fact, by based path induction first on $p$ and then on $q$, we only need to consider the case $B \equiv A$, $p \equiv \mathsf{refl}_A$ and $B' \equiv A'$, $q \equiv \mathsf{refl}_{A'}$. In that case, $\epsilon_{p,q}$ becomes $\mathsf{refl}_{A\to A'}$, $\langle f_p, (\eta_p, \vartheta_p)\rangle \equiv \langle \mathsf{id}_A, (\mathsf{refl}, \mathsf{refl})\rangle$, and $\langle g_q, (\eta'_q, \vartheta'_q)\rangle \equiv \langle \mathsf{id}_{A'}, (\mathsf{refl}, \mathsf{refl})\rangle$. Consequently,

$$\begin{aligned}
\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\epsilon_{p,q}) &\equiv \mathsf{id}_{A\to A'},\\
(f_p \to g_q) &\equiv \mathsf{id}_{A\to A'};
\end{aligned}$$

moreover, Lemma 2.3.11 and Proposition 2.12.8 yield

$$\begin{aligned}
\mathsf{transportinv}_r^{\mathsf{id}_{\mathcal{U}}}(\epsilon_{p,q}) &\equiv \mathsf{refl}\\
\lambda v.\, \eta'_q \bullet (v \triangleleft \eta_p) &= \mathsf{refl}\\
\mathsf{transportinv}_l^{\mathsf{id}_{\mathcal{U}}}(\epsilon_{p,q}) &\equiv \mathsf{refl}\\
\lambda u.\, \vartheta'_q \bullet (u \triangleleft \vartheta_p) &= \mathsf{refl},
\end{aligned}$$

proving the claim.

Specializing (2.8) to $p \equiv \mathsf{ua}(\langle f, (\eta, \vartheta)\rangle)$ and $q \equiv \mathsf{ua}(\langle g, (\eta', \vartheta')\rangle)$ reduces the question to proving that the $p$- and $q$-equivalences equal the original ones. But

$$\begin{aligned}\langle f_p, (\eta_p, \vartheta_p)\rangle &\equiv \mathsf{idtoeqv}(\mathsf{ua}(\langle f, (\eta, \vartheta)\rangle))\\ &=_{A\simeq B} \langle f, (\eta, \vartheta)\rangle\end{aligned}$$

and

$$\begin{aligned}\langle g_q, (\eta'_q, \vartheta'_q)\rangle &\equiv \mathsf{idtoeqv}(\mathsf{ua}(\langle g, (\eta', \vartheta')\rangle))\\ &=_{A'\simeq B'} \langle g, (\eta', \vartheta')\rangle,\end{aligned}$$

which complete the proof.

Π) Suppose that there are equivalences $(f_x, (\eta_x, \vartheta_x))\colon A(x) \simeq B(x)$. We have to prove that $\Pi_X A \simeq \Pi_X B$. Consider the function

$$\begin{aligned}f\colon \Big(\prod_{x\colon X} A(x)\Big) &\to \prod_{x\colon X} B(x)\\ u &\mapsto \lambda x.\, f_x(u(x)).\end{aligned}$$

For the back function we define

$$\begin{aligned}f^{-1}\colon \Big(\prod_{x\colon X} B(x)\Big) &\to \prod_{x\colon X} A(x)\\ v &\mapsto \lambda x.\, f_x^{-1}(v(x)).\end{aligned}$$

Clearly, these functions are mutual quasi-inverses. To construct the required homotopies, we apply function extensionality:

$$\begin{aligned}\eta\colon f \circ f^{-1} &\sim \mathsf{id}_{\Pi_X B}\\ \eta(v) &:\equiv \mathsf{funext}(\lambda x.\, \eta_x(v(x))),\end{aligned}$$

and

$$\begin{aligned}\vartheta\colon f^{-1} \circ f &\sim \mathsf{id}_{\Pi_X A}\\ \vartheta(u) &:\equiv \mathsf{funext}(\lambda x.\, \vartheta_x(u(x))).\end{aligned}$$

Applying the univalence axiom to the point-wise equivalences yields

$$\mathsf{ua}(\langle f_x, (\eta_x, \vartheta_x)\rangle)\colon A(x) =_{\mathcal{U}} B(x).$$

Let $\varepsilon(x) :\equiv \mathsf{ua}(\langle f_x, (\eta_x, \vartheta_x)\rangle)$. This means that $\varepsilon\colon A \sim B$ and so, $\mathsf{funext}(\varepsilon)\colon A =_{X\to\mathcal{U}} B$.

Define the function

$$\begin{aligned}\Pi_X\colon (X \to \mathcal{U}) &\to \mathcal{U}\\ P &\mapsto \Pi_x P.\end{aligned}$$

Using the path above, we obtain

$$\mathsf{ap}_{\Pi_X}(\mathsf{funext}(\varepsilon)) \colon \Pi_X A =_{\mathcal{U}} \Pi_X B.$$

The conclusion follows by applying $\mathsf{idtoeqv}$ to the resulting path.

To verify that both proofs are equivalent, we have to show that

$$\mathsf{idtoeqv}(\mathsf{ap}_{\Pi_X}(\mathsf{funext}(\varepsilon))) = \langle f, (\eta, \vartheta)\rangle.$$

Fix a generic path $p \colon A =_{X\to\mathcal{U}} B$. For each $x \colon X$, define

$$\langle f^p_x, (\eta^p_x, \vartheta^p_x)\rangle :\equiv \mathsf{idtoeqv}\big(\mathsf{happly}(p)(x)\big) \colon A(x) \simeq B(x).$$

We can then construct a global equivalence with functions

$$\begin{aligned} f^p &\colon \Pi_X A \to \Pi_X B & g^p &\colon \Pi_X B \to \Pi_X A \\ u &\mapsto \lambda x.\, f^p_x(u(x)) & v &\mapsto \lambda x.\, (f^p_x)^{-1}(v(x)) \end{aligned}$$

and homotopies

$$\begin{aligned} \eta^p &\colon f^p \circ g^p \sim \mathsf{id}_{\Pi_X B} & \vartheta^p &\colon g^p \circ f^p \sim \mathsf{id}_{\Pi_X A} \\ v &\mapsto \mathsf{funext}(\lambda x.\, \eta^p_x(v(x))) & u &\mapsto \mathsf{funext}(\lambda x.\, \vartheta^p_x(u(x))). \end{aligned}$$

Thus, the global equivalence $E(p)$ is defined as

$$E(p) :\equiv \langle f^p, (\eta^p, \vartheta^p)\rangle.$$

We claim that

$$\mathsf{idtoeqv}(\mathsf{ap}_{\Pi_X}(p)) = E(p).$$

By based path induction on $p$, it suffices to consider $B \equiv A$ and $p \equiv \mathsf{refl}_A$. In this case, we have

$$\begin{aligned} \text{LHS} &\equiv \mathsf{idtoeqv}(\mathsf{ap}_{\Pi_X}(\mathsf{refl}_A)) \\ &\equiv \mathsf{idtoeqv}(\mathsf{refl}_{\Pi_X A}) \\ &\equiv \langle \mathsf{id}_{\Pi_X A}, (\mathsf{refl}_{\mathsf{id}_{\Pi_X A}}, \mathsf{refl}_{\mathsf{id}_{\Pi_X A}})\rangle. \end{aligned}$$

On the other hand, given $x \colon X$, by equation (2.3)

$$\mathsf{happly}(\mathsf{refl}_A)(x) \equiv \mathsf{refl}_{A(x)}.$$

Then

$$\langle f^{\mathsf{refl}_A}_x, (\eta^{\mathsf{refl}_A}_x, \vartheta^{\mathsf{refl}_A}_x)\rangle \equiv \langle \mathsf{id}_{A(x)}, (\mathsf{refl}_{\mathsf{id}_{A(x)}}, \mathsf{refl}_{\mathsf{id}_{A(x)}})\rangle.$$

Substituting these into the definitions of our global functions $f^{\mathsf{refl}_A}$ and $g^{\mathsf{refl}_A}$ yields

$$\begin{aligned} f^{\mathsf{refl}_A}(u) &\equiv \lambda x.\, \mathsf{id}_{A(x)}(u(x)) \equiv \lambda x.\, u(x) \equiv u, \\ g^{\mathsf{refl}_A}(v) &\equiv \lambda x.\, \mathsf{id}_{A(x)}(v(x)) \equiv \lambda x.\, v(x) \equiv v. \end{aligned}$$

Thus, both global functions reduce definitionally to the identity function on the Π-type. Substituting the pointwise homotopies into the global ones yields

$$\eta^{\mathsf{refl}_A}(v) \equiv \mathsf{funext}(\lambda x.\, \mathsf{refl}_{v(x)}),$$
$$\vartheta^{\mathsf{refl}_A}(u) \equiv \mathsf{funext}(\lambda x.\, \mathsf{refl}_{u(x)}).$$

Hence,

$$\eta^{\mathsf{refl}_A}(v) \equiv \mathsf{refl}_v,$$
$$\vartheta^{\mathsf{refl}_A}(u) \equiv \mathsf{refl}_u,$$

which implies $\eta^{\mathsf{refl}_A} \equiv \mathsf{refl}_{\mathsf{id}_{\Pi_X A}}$ and $\vartheta^{\mathsf{refl}_A} \equiv \mathsf{refl}_{\mathsf{id}_{\Pi_X A}}$. It follows that

$$\begin{aligned}\textsc{rhs} &\equiv E(\mathsf{refl}_A)\\ &\equiv \langle f^{\mathsf{refl}_A}, (\eta^{\mathsf{refl}_A}, \vartheta^{\mathsf{refl}_A})\rangle\\ &\equiv \langle \mathsf{id}_{\Pi_X A}, (\mathsf{refl}_{\mathsf{id}_{\Pi_X A}}, \mathsf{refl}_{\mathsf{id}_{\Pi_X A}})\rangle.\end{aligned}$$

In conclusion, $\textsc{lhs} \equiv \textsc{rhs}$, and the claimed equality follows. Specializing it to the path $p :\equiv \mathsf{funext}(\varepsilon)$ yields the desired equality because, for each $x\colon X$,

$$\begin{aligned}\mathsf{idtoeqv}\big(\mathsf{happly}(\mathsf{funext}(\varepsilon))(x)\big) &= \mathsf{idtoeqv}(\varepsilon(x))\\ &\equiv \mathsf{idtoeqv}\big(\mathsf{ua}(\langle f_x, (\eta_x, \vartheta_x)\rangle)\big)\\ &= \langle f_x, (\eta_x, \vartheta_x)\rangle.\end{aligned}$$

Since the pointwise components defining $E(\mathsf{funext}(\varepsilon))$ are propositionally equal to the original point-wise equivalences, the assembled global equivalence is equal to $\langle f, (\eta, \vartheta)\rangle$.

+) In this case, we are given two equivalences $\langle f, (\eta, \vartheta)\rangle\colon A \simeq B$ and $\langle g, (\eta', \vartheta')\rangle\colon A' \simeq B'$, and we have to construct an equivalence between $(A + A')$ and $(B + B')$. To do that, we define

$$\begin{aligned}(f + g)\colon (A + A') &\to (B + B')\\ \mathsf{inl}(a) &\mapsto \mathsf{inl}(f(a))\\ \mathsf{inr}(a') &\mapsto \mathsf{inr}(g(a')).\end{aligned}$$

For the inverse, we introduce

$$(f + g)^{-1} :\equiv (f^{-1} + g^{-1}),$$

and for the homotopies,

$$\begin{aligned}(\eta + \eta')(\mathsf{inl}(b)) &:\equiv \mathsf{inl}(\eta(b))\\ (\eta + \eta')(\mathsf{inr}(b')) &:\equiv \mathsf{inr}(\eta'(b'))\\ (\vartheta + \vartheta')(\mathsf{inl}(a)) &:\equiv \mathsf{inl}(\vartheta(a))\\ (\vartheta + \vartheta')(\mathsf{inr}(a')) &:\equiv \mathsf{inr}(\vartheta'(a')).\end{aligned}$$

It is clear that

$$\langle f, (\eta, \vartheta)\rangle + \langle g, (\eta', \vartheta')\rangle :\equiv \langle f + g, (\eta + \eta', \vartheta + \vartheta')\rangle \tag{2.9}$$

is the desired equivalence.

Using the univalence axiom, we obtain paths $\mathsf{ua}(\langle f, (\eta, \vartheta)\rangle)\colon A =_{\mathcal{U}} B$ and $\mathsf{ua}(\langle g, (\eta', \vartheta')\rangle)\colon A' =_{\mathcal{U}} B'$. Suppose we have generic paths $p\colon A =_{\mathcal{U}} B$ and $q\colon A' =_{\mathcal{U}} B'$. We can construct a path

$$p + q\colon (A + A') =_{\mathcal{U}} (B + B')$$

by based path induction, first on $p$, and then on $q$, as follows: considering the case where $B \equiv A$ and $p \equiv \mathsf{refl}_A$, and $B' \equiv A'$ and $q \equiv \mathsf{refl}_{A'}$, we can simply define $p + q :\equiv \mathsf{refl}_{A+A'}$.

Thus, specializing this construction to the paths $p :\equiv \mathsf{ua}(\langle f, (\eta, \vartheta)\rangle)$ and $q :\equiv \mathsf{ua}(\langle g, (\eta', \vartheta')\rangle)$, and then applying $\mathsf{idtoeqv}$ to the resulting path, we arrive at the desired equivalence.

To show that both proofs are the same, we need to be more precise about the second one.

- Fix $B \equiv A$ and $p \equiv \mathsf{refl}_A$. Consider the motive

$$E\colon \prod_{X:\mathcal{U}} (A' =_{\mathcal{U}} X) \to \mathcal{U}$$
$$E(X) :\equiv \lambda z.\, (A + A') =_{\mathcal{U}} (A + X),$$

  where $E(X)$ is constant (it does not depend on $z$).

  For the base case $c_{\mathsf{refl}}\colon E(A', \mathsf{refl}_{A'})$, we take reflexivity:

$$c_{\mathsf{refl}} :\equiv \mathsf{refl}_{A+A'}.$$

  The induction principle yields:

$$\gamma_q :\equiv \mathsf{ind}_{=}(E, \mathsf{refl}_{A+A'}, B', q)\colon (A + A') =_{\mathcal{U}} (A + B').$$

- To complete the based induction on $p$, we fix $q\colon A' =_{\mathcal{U}} B'$ and introduce the motive

$$F_q\colon \prod_{Y:\mathcal{U}} (A =_{\mathcal{U}} Y) \to \mathcal{U}$$
$$F_q(Y) :\equiv \lambda z.\, (A + A') =_{\mathcal{U}} (Y + B'),$$

  where, as before, $F_q(Y)$ is constant on $z$. For the base case where $Y \equiv A$, we take $\gamma_q$. Thus, we can define

$$\gamma :\equiv \mathsf{ind}_{=}(F_q, \gamma_q, B, p)\colon (A + A') =_{\mathcal{U}} (B + B').$$

Consider the goal type

$$\prod_{p:\, A =_{\mathcal{U}} B} \prod_{q:\, A' =_{\mathcal{U}} B'} \mathsf{idtoeqv}(\gamma) = \big(\mathsf{idtoeqv}(p) + \mathsf{idtoeqv}(q)\big),$$

where the coproduct on the RHS is the one defined in (2.9).

Since both sides of the equation are terms of type $(A + A') \simeq (B + B')$, to prove that the identity type is inhabited, we may proceed by path induction on $p$ and $q$. Thus, we assume $B \equiv A$ with $p \equiv \mathsf{refl}_A$, and $B' \equiv A'$ with $q \equiv \mathsf{refl}_{A'}$. Then,

$$\begin{aligned}
\gamma_q &\equiv \mathsf{refl}_{A+A'},\\
\gamma \equiv \gamma_q &\equiv \mathsf{refl}_{A+A'},\\
\mathsf{idtoeqv}(\gamma) &\equiv \langle \mathsf{id}_{A+A'}, (\mathsf{refl}, \mathsf{refl})\rangle,\\
\mathsf{idtoeqv}(p) &\equiv \langle \mathsf{id}_{A}, (\mathsf{refl}, \mathsf{refl})\rangle,\\
\mathsf{idtoeqv}(q) &\equiv \langle \mathsf{id}_{A'}, (\mathsf{refl}, \mathsf{refl})\rangle,\\
\mathsf{idtoeqv}(p) + \mathsf{idtoeqv}(q) &\equiv \langle \mathsf{id}_{A} + \mathsf{id}_{A'}, (\mathsf{refl} + \mathsf{refl}, \mathsf{refl} + \mathsf{refl})\rangle.
\end{aligned}$$

Consequently, we need to verify that

$$\mathsf{id}_{A+A'} = \mathsf{id}_A + \mathsf{id}_{A'}$$

and

$$\mathsf{refl} + \mathsf{refl} = \mathsf{refl}.$$

The first equality is true by coproduct induction. Therefore, it holds by function extensionality. The second also follows from function extensionality because

$$\begin{aligned}
\mathsf{refl}(x) &:\equiv \mathsf{refl}_x,\\
(\mathsf{refl} + \mathsf{refl})(\mathsf{inl}(x)) &\equiv \mathsf{refl}(\mathsf{inl}(x)),\\
(\mathsf{refl} + \mathsf{refl})(\mathsf{inr}(y)) &\equiv \mathsf{refl}(\mathsf{inr}(y)).
\end{aligned}$$

□

**Exercise 2.19.16.** *State and prove a version of equation* (2.2) *from* Types as Categories *for dependent functions.*

*Solution.* Let $P\colon X \to \mathcal{U}$ be a type family. Given two dependent functions $f, g\colon \prod_{x:\, X} P(x)$, a homotopy $\eta\colon f \sim g$, and a path $p\colon x =_X y$, we have

$$\mathsf{apd}_f(p) \cdot \eta(y) =_{P(y)} p_*(\eta(x)) \cdot \mathsf{apd}_g(p).$$

To prove this, first observe that the types match:

$$\underbrace{\mathsf{apd}_f(p)}_{p_*(f(x))=f(y)} \cdot \underbrace{\eta(y)}_{f(y)=g(y)} =_{P(y)} \underbrace{\mathsf{ap}_{p_*}(\underbrace{\eta(x)}_{f(x)=g(x)})}_{p_*(f(x))=p_*(g(x))} \cdot \underbrace{\mathsf{apd}_g(p)}_{p_*(g(x))=g(y)},$$

where

$$p_* :\equiv \mathsf{transport}^P(p)\colon P(x) \to P(y).$$

Therefore, to complete the proof, we may apply based path induction on $p$ and assume that $y \equiv x$ and $p \equiv \mathsf{refl}_x$. In that case, the expression reduces to

$$\mathsf{refl}_{f(x)} \cdot \eta(x) =_{P(x)} \eta(x) \cdot \mathsf{refl}_{g(x)},$$

which is trivially inhabited because

$$\mathsf{refl}_{f(x)} \cdot \eta(x) \equiv \eta(x) \quad \text{and} \quad \mathsf{ru}(\eta(x))^{-1}\colon \eta(x) =_{P(x)} \eta(x) \cdot \mathsf{refl}_{g(x)}.$$

□

**Exercise 2.19.17.**[UA] *The lemma in* Exercise 2.19.12 *shows that* $0_2 \neq_2 1_2$. *Show that, nevertheless,*

$$\langle 2, 0_2\rangle =_{\mathcal{U}_\bullet} \langle 2, 1_2\rangle.$$

*Solution.* Recall that $0_2 \neq_2 1_2$ is a notation for $\neg(0_2 =_2 1_2)$, which in turn is a notation for $(0_2 =_2 1_2) \to 0$.

By Theorem 2.9.5,

$$\big(\langle 2, 0_2\rangle =_{\mathcal{U}_\bullet} \langle 2, 1_2\rangle\big) \simeq \sum_{p\colon 2=_{\mathcal{U}} 2} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p, 0_2) =_2 1_2.$$

By Exercise 2.19.12, we know that $\mathsf{not}\colon 2 \to 2$ defines an equivalence. Therefore, $\mathsf{ua}(\mathsf{not})$ is a path in $2 =_{\mathcal{U}} 2$. Hence,

$$\begin{aligned}
\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(\mathsf{ua}(\mathsf{not}))(0_2) &=_2 \mathsf{idtoeqv}(\mathsf{ua}(\mathsf{not}))(0_2) && \text{; Thm. 2.13.1}\\
&=_2 \mathsf{not}(0_2)\\
&\equiv 1_2,
\end{aligned}$$

which shows that the RHS of the equivalence is inhabited by $\langle \mathsf{ua}(\mathsf{not}), \mathsf{refl}_{1_2}\rangle$. Thus, the LHS is inhabited as well.

□

# Chapter 3

# Sets & Mere Propositions

## 3.1 Sets

**Definition 3.1.1.** A type $A$ is a *set* if

$$\mathsf{isSet}(A) :\equiv \prod_{x,y:\, A} \prod_{p,q:\, x =_A y} p =_{x=_A y} q$$

is inhabited.

**Theorem 3.1.2.**[FE] *The following closure properties of sets hold:*

a) *The empty product* $\mathbf{1}$ *is a set.*

b) *The empty coproduct* $\mathbf{0}$ *is a set.*

c) *The type* $\mathbb{N}$ *of natural numbers is a set.*

d) *If* $A$ *and* $B$ *are sets, then* $A \times B$ *is a set.*

e) *If* $A : \mathcal{U}$ *is a set and* $P : A \to \mathcal{U}$ *is a type family such that* $P(a)$ *is a set for all* $a : A$*, then* $\Sigma_A P$ *is a set.*

f) *If* $A$ *is any type and* $B : A \to \mathcal{U}$ *is a type family such that each* $B(x)$ *is a set, then* $\prod_{x:\, A} B(x)$ *is a set.*[1]

*Proof.*

a) This is a direct consequence of Theorem 2.10.1: for any paths $p, q : x =_{\mathbf{1}} y$, we have $\mathsf{encode}(p) =_{\mathbf{1}} \mathsf{encode}(q)$ because all elements of $\mathbf{1}$ are propositionally equal to $\star$. Since $\mathsf{encode}$ is an equivalence, it is an embedding, which implies that $p =_{x=_{\mathbf{1}} y} q$.

[1] Function extensionality is used here.

b) A witness for $\mathsf{isSet}(\mathsf{0})$ is given by:

$$\lambda x.\, \mathsf{rec}_{\mathsf{0}}\Big(\prod_{y:\,\mathsf{0}} \prod_{p,q:\,x=_{\mathsf{0}}y} p =_{x=_{\mathsf{0}}y} q\Big) :\, \mathsf{isSet}(\mathsf{0}).$$

c) To construct a proof, it suffices to define a witness of the motive

$$E(x) :\equiv \prod_{y:\,\mathbb{N}} \prod_{c,d:\,\mathsf{Code}(x,y)} c =_{\mathsf{Code}(x,y)} d.$$

This requires establishing a base case and an inductive step.

In the base case $x \equiv 0$, the motive becomes

$$E(0) :\equiv \prod_{y:\,\mathbb{N}} \prod_{c,d:\,\mathsf{Code}(0,y)} c =_{\mathsf{Code}(0,y)} d.$$

We proceed by induction on $y$ using the motive

$$E_0(y) :\equiv \prod_{c,d:\,\mathsf{Code}(0,y)} c =_{\mathsf{Code}(0,y)} d.$$

- If $y \equiv 0$, then $\mathsf{Code}(0,0) \equiv \mathsf{1}$. Thus, given $c, d :\, \mathsf{1}$, we have $c =_{\mathsf{1}} d$, as both equal $\star$.
- If $y \equiv \mathsf{succ}(m)$, then $\mathsf{Code}(0, \mathsf{succ}(m)) \equiv \mathsf{0}$. Hence, given an inhabitant $c :\, \mathsf{0}$, we have

$$\mathsf{rec}_{\mathsf{0}}\Big(\prod_{d:\,\mathsf{0}} c =_{\mathsf{0}} d\Big)(c) :\, \prod_{d:\,\mathsf{0}} c =_{\mathsf{0}} d.$$

For the inductive step, we assume the hypothesis $h :\, E(n)$ for a given $n :\, \mathbb{N}$. We must construct a term of type

$$E(\mathsf{succ}(n)) \equiv \prod_{y:\,\mathbb{N}} \prod_{c,d:\,\mathsf{Code}(\mathsf{succ}(n),y)} c =_{\mathsf{Code}(\mathsf{succ}(n),y)} d.$$

We proceed by induction on $y$ with the motive

$$E_{\mathsf{succ}(n)}(y) :\equiv \prod_{c,d:\,\mathsf{Code}(\mathsf{succ}(n),y)} c =_{\mathsf{Code}(\mathsf{succ}(n),y)} d.$$

- If $y \equiv 0$, then $\mathsf{Code}(\mathsf{succ}(n), 0) \equiv \mathsf{0}$. As before, applying the induction principle of the empty type to $c :\, \mathsf{0}$ trivially yields the required path.
- If $y \equiv \mathsf{succ}(m)$, then $\mathsf{Code}(\mathsf{succ}(n), \mathsf{succ}(m)) \equiv \mathsf{Code}(n, m)$. Then, $h(m)(c, d)$ yields the required path of type $c =_{\mathsf{Code}(n,m)} d$.

d) Let $p, q \colon x =_{A \times B} y$. As shown in Equality in Cartesian Products, we have path codifications

$$\mathsf{pair}^{\times}(p) \equiv (\mathsf{ap}_{\pi_1}(p), \mathsf{ap}_{\pi_2}(p)) \quad \text{and} \quad \mathsf{pair}^{\times}(q) \equiv (\mathsf{ap}_{\pi_1}(q), \mathsf{ap}_{\pi_2}(q)).$$

Set

$$p_1 :\equiv \mathsf{ap}_{\pi_1}(p), \quad p_2 :\equiv \mathsf{ap}_{\pi_2}(p), \quad q_1 :\equiv \mathsf{ap}_{\pi_1}(q), \quad q_2 :\equiv \mathsf{ap}_{\pi_2}(q)$$

and

$$x_1 :\equiv \pi_1(x), \quad x_2 :\equiv \pi_2(x), \quad y_1 :\equiv \pi_1(y), \quad y_2 :\equiv \pi_2(y).$$

Since the hypothesis implies that $A$ and $B$ are sets, we obtain

$$p_1 =_A q_1 \quad \text{and} \quad p_2 =_B q_2,$$

by using the functions $\mathsf{in}_b :\equiv a \mapsto (a, b)$ and $\mathsf{in}_a :\equiv b \mapsto (a, b)$ and applying the action on paths, we obtain

$$\begin{aligned}(p_1, p_2) &=_{(x_1 =_A y_1) \times (x_2 =_B y_2)} (p_1, q_2) \\ &=_{(x_1 =_A y_1) \times (x_2 =_B y_2)} (q_1, q_2).\end{aligned}$$

Thus, $\mathsf{pair}^{\times}(p) =_{(x_1 =_A y_1) \times (x_2 =_B y_2)} \mathsf{pair}^{\times}(q)$, and the result follows by decoding with $\mathsf{pair}^{=}$.

e) Let $\omega, \omega' \colon \Sigma_A P$. The codification of a path $p \colon \omega =_{\Sigma_A P} \omega'$ is defined as

$$\mathsf{encode}(p) \equiv \langle \mathsf{ap}_{\pi_1}(p), \mathsf{apd}_{\pi_2}(p) \rangle.$$

If $q$ is another path with the same ends,

$$\mathsf{encode}(q) \equiv \langle \mathsf{ap}_{\pi_1}(q), \mathsf{apd}_{\pi_2}(q) \rangle.$$

Introduce

$$x :\equiv \pi_1(\omega), \quad y :\equiv \pi_2(\omega), \quad x' :\equiv \pi_1(\omega'), \quad y' :\equiv \pi_2(\omega').$$

Given that

$$\mathsf{ap}_{\pi_1}(p),\ \mathsf{ap}_{\pi_1}(q) \colon x =_A x',$$

the hypothesis on $A$ implies the existence of a witness

$$\epsilon \colon \mathsf{ap}_{\pi_1}(p) =_{x =_A x'} \mathsf{ap}_{\pi_1}(q).$$

Set

$$p_1 :\equiv \mathsf{ap}_{\pi_1}(p), \quad p_2 :\equiv \mathsf{apd}_{\pi_2}(p), \quad q_1 :\equiv \mathsf{ap}_{\pi_1}(q), \quad q_2 :\equiv \mathsf{apd}_{\pi_2}(q).$$

Then,

$$p_2 \colon (p_1)_*(y) =_{P(x')} y' \quad \text{and} \quad q_2 \colon (q_1)_*(y) =_{P(x')} y'.$$

If we define

$$R\colon (x =_A x') \to \mathcal{U}$$
$$R(r) :\equiv r_*(y) =_{P(x')} y',$$

then, $p_2\colon R(p_1)$ and $q_2\colon R(q_1)$.

Since $\epsilon\colon p_1 =_{x=_A x'} q_1$, it follows that $\epsilon_*\colon R(p_1) \to R(q_1)$ satisfies

$$\epsilon_*(p_2)\colon (q_1)_*(y) =_{P(x')} y'.$$

Thus, the hypothesis on $P(x')$ implies the existence of a witness

$$\rho\colon \epsilon_*(p_2) =_{R(q_1)} q_2.$$

Therefore, the pair of paths $(\epsilon, \rho)$ constructs a path

$$\langle p_1, p_2\rangle =_{\Sigma_{x=_A x'} R} \langle q_1, q_2\rangle,$$

which definitionally means that $\mathsf{encode}(p) = \mathsf{encode}(q)$. Since $\mathsf{encode}$ is an equivalence, we conclude that $\Sigma_A P$ is a set.

f) Set $\Pi_A B :\equiv \prod_{x\colon A} B(x)$ and consider two paths $p, q\colon f =_{\Pi_A B} g$. Then,

$$\mathsf{happly}(p), \mathsf{happly}(q)\colon \prod_{x\colon A} f(x) =_{B(x)} g(x).$$

Hence, for any $x\colon A$, we have

$$\mathsf{happly}(p)(x), \mathsf{happly}(q)(x)\colon f(x) =_{B(x)} g(x).$$

Since $B(x)$ is a set, any two paths in it are equal, so we obtain a path

$$\eta(x)\colon \mathsf{happly}(p)(x) =_{f(x)=_{B(x)} g(x)} \mathsf{happly}(q)(x)$$

that defines a homotopy $\eta\colon \mathsf{happly}(p) \sim \mathsf{happly}(q)$. Thus, by function extensionality, we obtain a path

$$\mathsf{funext}(\eta)\colon \mathsf{happly}(p) =_{f\sim g} \mathsf{happly}(q).$$

Applying the action on paths of $\mathsf{funext}$ yields

$$\mathsf{ap}_{\mathsf{funext}}(\mathsf{funext}(\eta))\colon \mathsf{funext}(\mathsf{happly}(p)) =_{f=_{\Pi_A B} g} \mathsf{funext}(\mathsf{happly}(q)).$$

Since $\mathsf{funext}$ is the quasi-inverse of $\mathsf{happly}$, this directly decodes to a path $p =_{f=_{\Pi_A B} g} q$, proving that $\Pi_A B$ is a set.

□

**Definition 3.1.3.** A type $A$ is a 1-*type* if for all $x, y\colon A$, $p, q\colon x =_A y$, and $r, s\colon p =_{x=_A y} q$, we have $r =_{p=_{x=_A y} q} s$.

**Remark 3.1.4.** Equivalently, the definition means that $A$ is a 1-type when the identity type $x =_A y$ is a set for all $x, y : A$.

**Lemma 3.1.5.** *Every set is a* 1*-type.*

*Proof.* Let $A$ be a set. We have to prove that, given $x, y : A$ and $p, q : x =_A y$, any two paths $r, s : p =_{x=_A y} q$ are equal.

By definition, there is a dependent function

$$f : \prod_{x,y:A} \prod_{p,q:x=_A y} p =_{x=_A y} q.$$

Fix $x, y : A$ and $p : x =_A y$ and define

$$g : \prod_{q:x=_A y} p =_{x=_A y} q$$
$$g(q) :\equiv f(x, y, p, q).$$

Suppose that $q : x =_A y$ and that $r : p =_{x=_A y} q$. By Lemma 2.3.6 applied to $x =_A y$, $g$, and $P(\zeta) :\equiv p =_{x=_A y} \zeta$, we obtain

$$\mathsf{apd}_g(r) : r_*(g(p)) =_{p=_{x=_A y} q} g(q),$$

where

$$r_* :\equiv \mathsf{transport}^P(r) \equiv \mathsf{transport}^{\zeta \mapsto (p=_{x=_A y} \zeta)}(r).$$

By Proposition 2.11.6, we have

$$r_*(g(p)) =_{p=_{x=_A y} q} g(p) \cdot r,$$

which implies that

$$g(p) \cdot r =_{p=_{x=_A y} q} g(q)$$

is inhabited. Since this conclusion is also valid for $s$, we deduce that

$$g(p) \cdot r =_{p=_{x=_A y} q} g(q) \quad \text{and} \quad g(p) \cdot s =_{p=_{x=_A y} q} g(q)$$

are inhabited. It follows that

$$g(p) \cdot r =_{p=_{x=_A y} q} g(p) \cdot s,$$

is inhabited, which implies that $r =_{p=_{x=_A y} q} s$ is inhabited as well.

□

**Example 3.1.6.**[UA] The universe is not a set.

To see this, consider the Boolean type $\mathsf{2}$. As shown in Exercise 2.19.12, we have an equivalence $(\mathsf{2} \simeq \mathsf{2}) \simeq \mathsf{2}$. Since the lemma in that exercise shows that $0_{\mathsf{2}} \neq 1_{\mathsf{2}}$, we deduce that there are two distinct equivalences from $\mathsf{2}$ to $\mathsf{2}$ (namely, the identity and the swap function). By the univalence axiom, these two equivalences correspond to two distinct paths in the identity type $\mathsf{2} =_{\mathcal{U}} \mathsf{2}$, contradicting the definition of a set.

**Lemma 3.1.7.** *For any type $A$, there is a map $(A + \neg A) \to (\neg\neg A \to A)$.*

*Proof.* To construct a map from $A + \neg A$ to $\neg\neg A \to A$, we first define the constant function

$$\begin{aligned} c &: A \to (\neg\neg A \to A) \\ a &\mapsto (\beta \mapsto a). \end{aligned}$$

Second, fix $b : \neg A$. Using recursion, we obtain

$$\begin{aligned} \rho_b &: \neg\neg A \to A \\ \beta &\mapsto \mathsf{rec}_0(A)(\beta(b)). \end{aligned}$$

This yields a function

$$\begin{aligned} \rho &: \neg A \to (\neg\neg A \to A) \\ b &\mapsto \rho_b. \end{aligned}$$

By recursion on $A + \neg A$, we arrive at the desired map:

$$\begin{aligned} &\varphi_A : (A + \neg A) \to (\neg\neg A \to A) \\ &\varphi_A(x) :\equiv \mathsf{rec}_{A+\neg A}(\neg\neg A \to A, c, \rho, x). \end{aligned}$$

□

**Theorem 3.1.8.**[FE] [UA] *It is not the case that for all $A : \mathcal{U}$ we have $\neg\neg A \to A$.*

*Proof.* The statement means that

$$\Big(\prod_{A:\mathcal{U}} \neg\neg A \to A\Big) \to \mathsf{0} \tag{3.1}$$

is inhabited. Consider the type family

$$\begin{aligned} &P : \mathcal{U} \to \mathcal{U} \\ &P(A) :\equiv \neg\neg A \to A \end{aligned}$$

and the function

$$\begin{aligned} \mathsf{not} &: \mathsf{2} \to \mathsf{2} \\ 0_\mathsf{2} &\mapsto 1_\mathsf{2} \\ 1_\mathsf{2} &\mapsto 0_\mathsf{2}, \end{aligned}$$

introduced in Exercise 1.16.2.

Being an involution, $\mathsf{not}$ defines an equivalence $\mathsf{2} \simeq \mathsf{2}$. Hence, by the univalence axiom, we obtain $p :\equiv \mathsf{ua}(\mathsf{not}) : \mathsf{2} =_\mathcal{U} \mathsf{2}$.

Given $f\colon \prod_{A:\mathcal{U}} \neg\neg A \to A \equiv \prod_{A:\mathcal{U}} P(A)$, we have $f(2)\colon P(2) \equiv \neg\neg 2 \to 2$ and

$$\mathsf{apd}_f(p)\colon \mathsf{transport}^P(p, f(2)) =_{P(2)} f(2). \tag{3.2}$$

By Proposition 2.11.5 for $A(X) :\equiv \neg\neg X$, $B :\equiv \mathsf{id}_{\mathcal{U}}$, and $g \equiv f(2)$, we have $P \equiv A \to B$, and so

$$\mathsf{transport}^P(p, f(2)) =_{P(2)} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p) \circ f(2) \circ \mathsf{transport}^{X \mapsto \neg\neg X}(p^{-1}).$$

Evaluating this equality at any $\zeta\colon \neg\neg 2$, we obtain

$$\mathsf{transport}^P\big(p, f(2)\big)(\zeta) =_2 \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}\big(p, f(2)(\xi)\big),$$

where

$$\xi :\equiv \mathsf{transport}^{X \mapsto \neg\neg X}(p^{-1}, \zeta).$$

We claim that $\xi =_{\neg\neg 2} \zeta$. To see this, note that for any $c\colon \neg 2$, both $\xi(c)$ and $\zeta(c)$ are elements of $\mathsf{0}$. Applying $\mathsf{rec}_{\mathsf{0}}$ we deduce that $\xi(c) =_{\mathsf{0}} \zeta(c)$. By function extensionality, $\xi =_{\neg\neg 2} \zeta$.

Applying $\mathsf{happly}$ to (3.2) and evaluating at $\zeta$, we also know that

$$\mathsf{transport}^P\big(p, f(2)\big)(\zeta) =_2 f(2)(\zeta).$$

Combining these paths yields

$$\mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}\big(p, f(2)(\zeta)\big) =_2 f(2)(\zeta).$$

By Theorem 2.13.1, we deduce that

$$\mathsf{not}\big(f(2)(\zeta)\big) =_2 f(2)(\zeta).$$

Take $u\colon \neg 2$. Then $u\colon 2 \to \mathsf{0}$ and we can evaluate $u$ at $\mathsf{0}_2$, i.e., we can define

$$e\colon \neg\neg 2, \quad e :\equiv \lambda u.\, u(\mathsf{0}_2).$$

Then $b :\equiv f(2)(e)$ is an element of $2$. The equality above constructs a path

$$\epsilon\colon \mathsf{not}(b) =_2 b.$$

We can define

$$\begin{aligned} &\psi\colon \prod_{x:2} (\mathsf{not}(x) =_2 x) \to \mathsf{0} \\ &\psi(\mathsf{0}_2) :\equiv \mathsf{neq} \\ &\psi(\mathsf{1}_2) :\equiv \lambda p.\, \mathsf{neq}(p^{-1}), \end{aligned} \tag{3.3}$$

where $\mathsf{neq}\colon (\mathsf{1}_2 =_2 \mathsf{0}_2) \to \mathsf{0}$ is the function defined in the lemma of Exercise 2.19.12.

In consequence, the function

$$f \mapsto \psi(b, \epsilon)$$

is a witness of the type given in (3.1).

□

**Remark 3.1.9.** The function $\psi$ of (3.3) allows us to define

$$\begin{aligned}\mathsf{notid}\colon \Big(\sum_{x\colon 2} \mathsf{not}(x) =_2 x\Big) &\to 0\\ \langle x, p\rangle &\mapsto \psi(\langle x, p\rangle).\end{aligned}$$

**Corollary 3.1.10.**[FE UA] *It is not the case that for all $A\colon \mathcal{U}$ we have $A + \neg A$.*

*Proof.* The statement means that

$$\Big(\prod_{A\colon \mathcal{U}} A + \neg A\Big) \to 0$$

is inhabited.

By Lemma 3.1.7, for any $A\colon \mathcal{U}$ there is a map $\varphi_A\colon (A + \neg A) \to (\neg\neg A \to A)$. This family of functions leads to

$$\begin{aligned}\Phi\colon \Big(\prod_{A\colon \mathcal{U}} A + \neg A\Big) &\to \prod_{A\colon \mathcal{U}} \neg\neg A \to A\\ g &\mapsto \lambda A.\, \varphi_A(g(A)).\end{aligned}$$

The conclusion follows by composing $\Phi$ with the function given in (3.1). □

## 3.2 Mere Propositions

**Definition 3.2.1.** A type $P$ is a *mere proposition* (a.k.a. *subsingleton*, a.k.a. *h-proposition*) if the type

$$\mathsf{isProp}(P) :\equiv \prod_{x,y\colon P} x =_P y$$

is inhabited.[2]

**Remark 3.2.2.** If $A$ is a set, then $x =_A y$ is a mere proposition for all $x, y\colon A$.

**Lemma 3.2.3.** *Every inhabited mere proposition is equivalent to the unit type* $1$.

*Proof.* Fix $\pi\colon \mathsf{isProp}(P)$ and $a\colon P$. Define

$$\begin{aligned}\phi\colon P &\to 1 & \qquad \psi\colon 1 &\to P\\ x &\mapsto \star & \star &\mapsto a.\end{aligned}$$

Then,

$$\phi(\psi(\star)) \equiv \phi(a) \equiv \star,$$

[2] We have adopted the function name given in [The Univalent Foundations Program, 2013].

which implies by induction on $\mathsf{1}$ that $\phi(\psi(z)) =_{\mathsf{1}} z$ for all $z\colon \mathsf{1}$. Conversely, for any $x\colon P$,

$$\psi(\phi(x)) \equiv \psi(\star) \equiv a \overset{\pi(a,x)}{=_P} x,$$

showing that $\lambda x.\, \pi(a, x)\colon \psi \circ \phi \sim \mathsf{id}_P$. □

**Lemma 3.2.4.** *Let $P$ and $Q$ be mere propositions. Then any pair of functions $P \to Q$ and $Q \to P$ yields an equivalence $P \simeq Q$.*

*Proof.* Let $\phi\colon P \to Q$ and $\psi\colon Q \to P$, and fix two witnesses $\pi\colon \mathsf{isProp}(P)$ and $\sigma\colon \mathsf{isProp}(Q)$.

To show that $\phi$ is an equivalence with quasi-inverse $\psi$, we must provide homotopies $\eta\colon \phi \circ \psi \sim \mathsf{id}_Q$ and $\vartheta\colon \psi \circ \phi \sim \mathsf{id}_P$.

Given any $y\colon Q$, by hypothesis,

$$\sigma(\phi(\psi(y)), y)\colon \phi(\psi(y)) =_Q y,$$

and we can define the homotopy $\eta$.

Similarly,

$$\pi(\psi(\phi(x)), x)\colon \psi(\phi(x)) =_P x,$$

which yields the homotopy $\vartheta$. □

**Remark 3.2.5.** Lemma 3.2.3 could have been proved as a consequence of Lemma 3.2.4.

**Lemma 3.2.6.** *Every mere proposition is a set.*

*Proof.* Let $A$ be a mere proposition. Fix $\pi\colon \mathsf{isProp}(A)$, two points $x, y\colon A$, and paths $p, q\colon x =_A y$.

Consider the dependent function

$$f :\equiv \lambda z.\, \pi(x, z)\colon \prod_{z\colon A} P(z),$$

where $P\colon A \to \mathcal{U}$ is given by $P(z) :\equiv x =_A z$. We obtain

$$\mathsf{apd}_f(p)\colon \mathsf{transport}^P\big(p, \pi(x, x)\big) =_{x =_A y} \pi(x, y).$$

By Proposition 2.11.6, we have

$$\mathsf{transport}^P\big(p, \pi(x, x)\big) =_{x =_A y} \pi(x, x) \cdot p.$$

Therefore, by path concatenation, we have

$$\pi(x, x) \cdot p =_{x =_A y} \pi(x, y).$$

By symmetry, we also obtain

$$\pi(x,x) \cdot q =_{x=_A y} \pi(x,y).$$

Therefore,

$$\begin{aligned} p &=_{x=_A y} \pi(x,x)^{-1} \cdot \pi(x,y) \\ &=_{x=_A y} q. \end{aligned}$$

□

**Remark 3.2.7.** Combining the lemma with Remark 3.2.2, we deduce that, if $P$ is a mere proposition, $x =_P y$ is also a mere proposition for all $x, y\colon P$.

**Theorem 3.2.8.** *The boolean type* $\mathsf{2}$ *is a set.*

*Proof.* Let $x, y\colon \mathsf{2}$ and consider two paths $p, q\colon x =_{\mathsf{2}} y$.

By Theorem 2.8.1, the map $\mathsf{encode}(x,y)$ is an equivalence, and therefore an embedding. Thus, to prove $p =_{x=_{\mathsf{2}} y} q$, it suffices to show that $\mathsf{Code}(x,y)$ is a mere proposition for any $x$ and $y$; that is, we need to construct an inhabitant of

$$\prod_{x,y\colon \mathsf{2}} \mathsf{isProp}(\mathsf{Code}(x,y)).$$

We proceed to construct this witness by double induction on $x$ and $y$:

- If $x \equiv 0_{\mathsf{2}}$ and $y \equiv 0_{\mathsf{2}}$, or $x \equiv 1_{\mathsf{2}}$ and $y \equiv 1_{\mathsf{2}}$, the type $\mathsf{Code}(x,y)$ is definitionally equal to $\mathsf{1}$, which is a mere proposition.
- If $x \equiv 0_{\mathsf{2}}$ and $y \equiv 1_{\mathsf{2}}$, or $x \equiv 1_{\mathsf{2}}$ and $y \equiv 0_{\mathsf{2}}$, the type $\mathsf{Code}(x,y)$ is definitionally equal to $\mathsf{0}$, which is trivially a mere proposition.

□

**Proposition 3.2.9.**[FE] *For any type* $A$*, the types* $\mathsf{isProp}(A)$ *and* $\mathsf{isSet}(A)$ *are mere propositions.*

*Proof.* Consider $f, g\colon \mathsf{isProp}(A)$. To show that $\mathsf{isProp}(A)$ is a mere proposition, we must show that $f =_{\mathsf{isProp}(A)} g$. By function extensionality, it suffices to fix $x, y\colon A$ and show that $f(x,y) =_{x=_A y} g(x,y)$. The terms evaluate as

$$f(x,y)\colon x =_A y \quad \text{and} \quad g(x,y)\colon x =_A y.$$

Since $f$ is a witness that $A$ is a mere proposition, Lemma 3.2.6 implies that $A$ is a set. Therefore, $x =_A y$ is a mere proposition and so $f(x,y) =_{x=_A y} g(x,y)$, as desired.

In the case where $f, g\colon \mathsf{isSet}(A)$, fix $x, y\colon A$ and $p, q\colon x =_A y$. Then,

$$f(x,y,p,q),\ g(x,y,p,q)\colon p =_{x=_A y} q.$$

The existence of $f$ implies that $A$ is a set. So, by Lemma 3.1.5, $x =_A y$ is a set. Thus, $p =_{x=_A y} q$ is a mere proposition. Therefore,

$$f(x, y, p, q) =_{p=_{x=_A y} q} g(x, y, p, q).$$

The proof concludes by function extensionality. □

**Definitions 3.2.10.**

- A type $A$ is *decidable* if $A + \neg A$ is inhabited.
- A family $C\colon A \to \mathcal{U}$ is *decidable* if $\prod_{a:\, A} C(a) + \neg C(a)$ is inhabited.
- The *law of excluded middle* is the type

$$\textsc{lem} :\equiv \prod_{A:\mathcal{U}} \mathsf{isProp}(A) \to (A + \neg A).$$

  In other words, the law of excluded middle states that all mere propositions are decidable.

**Proposition 3.2.11.**

a) *Equality on* $\mathbb{N}$ *is decidable; that is, given* $n, m\colon \mathbb{N}$*, we have*

$$(n =_{\mathbb{N}} m) + \neg(n =_{\mathbb{N}} m).$$

b) *Given* $n, m\colon \mathbb{N}$*, if* $\neg(n < m)$ *and* $\neg(m < n)$ *then* $n =_{\mathbb{N}} m$*.*

c) *Given* $n, m\colon \mathbb{N}$*, if* $n < \mathsf{succ}(m)$ *then* $n \leq m$*.*

d) *Inequality on* $\mathbb{N}$ *is decidable; that is, given* $n, m\colon \mathbb{N}$*, we have*

$$(n < m) + (m \leq n).$$

*Proof.*

a) Using the encoding and decoding of equalities in $\mathbb{N}$ introduced in § 2.16, it is straightforward to construct a witness

$$\epsilon\colon (\mathsf{Code}(n, m) \simeq \mathsf{0}) + (\mathsf{Code}(n, m) \simeq \mathsf{1}).$$

Let $E$ be the coproduct above and $C :\equiv \mathsf{Code}(n, m) + \neg\mathsf{Code}(n, m)$. To construct an inhabitant of $C$, it suffices to define a map from $E$ to $C$. We proceed by recursion on the coproduct $E$.

- For the left case, given an equivalence $\langle f_0, (\eta_0, \vartheta_0)\rangle\colon \mathsf{Code}(n, m) \simeq \mathsf{0}$, since $f_0\colon \mathsf{Code}(n, m) \to \mathsf{0}$, we can specify

$$g_0(\langle f_0, (\eta_0, \vartheta_0)\rangle) :\equiv \mathsf{inr}(f_0).$$

- For the right case, given $\langle f_1, (\eta_1, \vartheta_1)\rangle \colon \mathsf{Code}(n, m) \simeq \mathsf{1}$, we specify

$$g_1(\langle f_1, (\eta_1, \vartheta_1)\rangle) :\equiv \mathsf{inl}(f_1^{-1}(\star)).$$

As a result, we obtain

$$\mathsf{rec}_E(C, g_0, g_1, \epsilon) \colon C.$$

b) Let $u \colon \neg(n < m)$ and $v \colon \neg(m < n)$ be two witnesses. By part a) the type $(n =_{\mathbb{N}} m) + \neg(n =_{\mathbb{N}} m)$ is inhabited. Therefore, to prove that $n =_{\mathbb{N}} m$ it suffices to define a map

$$\big((n =_{\mathbb{N}} m) + \neg(n =_{\mathbb{N}} m)\big) \to (n =_{\mathbb{N}} m).$$

We proceed by recursion on the coproduct.

- For the left case, we take the identity $\mathsf{id}_{n=_{\mathbb{N}}m}$.
- For the right case, consider $z \colon \neg(n =_{\mathbb{N}} m)$. By Lemma 1.12.4, there is a term

$$w \colon (n \leq m) + (m \leq n).$$

We will use $z$ to define a map

$$\zeta_z \colon ((n \leq m) + (m \leq n)) \to (n =_{\mathbb{N}} m),$$

and $z \mapsto \zeta_z(w)$ will provide the required map

$$\neg(n =_{\mathbb{N}} m) \to (n =_{\mathbb{N}} m).$$

We proceed by recursion on the coproduct domain:

  - If $r \colon n \leq m$, the pair $(r, z)$ inhabits $(n \leq m) \times \neg(n =_{\mathbb{N}} m)$, which is definitionally $n < m$. Applying $u$ to this pair yields a term in $\mathsf{0}$. Hence, we can apply $\mathsf{rec}_{\mathsf{0}}(n =_{\mathbb{N}} m)$ to obtain our goal.
  - If $s \colon m \leq n$, then $(s, z')$, with $z' :\equiv \lambda q \colon (m =_{\mathbb{N}} n).\, z(q^{-1})$, is a term of $m < n$. Thus, $v(s, z') \colon \mathsf{0}$ and

$$\mathsf{rec}_{\mathsf{0}}(n =_{\mathbb{N}} m, v(s, z')) \colon (n =_{\mathbb{N}} m).$$

c) By definition $(n < \mathsf{succ}(m)) \equiv (n \leq \mathsf{succ}(m)) \times \neg(n =_{\mathbb{N}} \mathsf{succ}(m))$. Take $(p, u)$ in the RHS with

$$p \equiv \langle k, q\rangle, \quad q \colon n + k =_{\mathbb{N}} \mathsf{succ}(m), \quad \text{and } u \colon (n =_{\mathbb{N}} \mathsf{succ}(m)) \to \mathsf{0}.$$

Since $(k =_{\mathbb{N}} 0) + \neg(k =_{\mathbb{N}} 0)$, we deduce from $u$ and $q$ that $\neg(k =_{\mathbb{N}} 0)$. Then, by Lemma 1.12.4, $k =_{\mathbb{N}} \mathsf{succ}(\mathsf{pred}(k))$. It follows that

$$\mathsf{succ}(n + \mathsf{pred}(k)) =_{\mathbb{N}} \mathsf{succ}(m).$$

Hence, $n + \mathsf{pred}(k) =_{\mathbb{N}} m$, which proves that $n \leq m$.

d) The coproduct is definitionally equal to $(n < m) + \neg(n < m)$. We proceed by induction on $n : \mathbb{N}$.

BASE CASE: $n \equiv 0$. We proceed by induction on $m : \mathbb{N}$.

- If $m \equiv 0$, then $\neg(0 =_{\mathbb{N}} 0)$ holds, so we inject into the right summand.
- If $m \equiv \mathsf{succ}(k)$ for some $k : \mathbb{N}$, then $0 < \mathsf{succ}(k)$ holds, so we inject into the left summand.

INDUCTIVE STEP: $n \mapsto \mathsf{succ}(n)$. Assume the property holds for $n$ and all $m : \mathbb{N}$. We again proceed by induction on $m : \mathbb{N}$.

- If $m \equiv 0$, then $0 \leq \mathsf{succ}(n)$ holds, yielding the right summand.
- If $m \equiv \mathsf{succ}(k)$ for some $k : \mathbb{N}$, we apply the outer inductive hypothesis to $n$ and $k$, which yields $(n < k) + \neg(n < k)$. We perform case analysis on this coproduct:
  - Assume $n < k$. Then, $\mathsf{succ}(n) < \mathsf{succ}(k)$, meaning $\mathsf{succ}(n) < m$. This injects into the left summand.
  - Assume $\neg(n < k)$. Then $k \leq n$ and so $\mathsf{succ}(k) \leq \mathsf{succ}(n)$, meaning $\neg(\mathsf{succ}(n) < m)$. This injects into the right summand.

□

**Proposition 3.2.12.**

a) *The types* $\mathsf{0}$ *and* $\mathsf{1}$ *are both decidable.*

b) *If* $A$ *and* $B$ *are decidable, then* $A + B$ *is decidable.*

c) *If* $A$ *and* $B$ *are decidable, then* $A \times B$ *is decidable.*

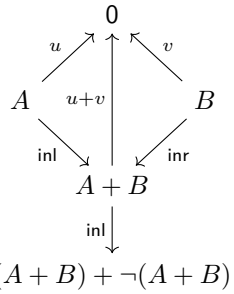
d) *If* $A$ *and* $B$ *are decidable, then* $A \to B$ *is decidable.*

*Proof.*

a) We have the terms $\mathsf{inr}(\mathsf{id}_{\mathsf{0}}) : \mathsf{0} + \neg\mathsf{0}$ and $\mathsf{inl}(\star) : \mathsf{1} + \neg\mathsf{1}$.

b) The diagram below captures the cases needed to define a map

$$\big((A + \neg A) \times (B + \neg B)\big) \to \big((A + B) + \neg(A + B)\big).$$



More precisely, let $C :\equiv (A + B) + \neg(A + B)$. By the universal property of the product, we can equivalently define a curried map

$$f\colon (A + \neg A) \to (B + \neg B) \to C.$$

To construct $f$, we apply the recursion principle for $A + \neg A$.

For the left branch, we take

$$\phi :\equiv \lambda a.\, \lambda y.\, \mathsf{inl}(\mathsf{inl}(a))\colon A \to ((B + \neg B) \to C).$$

For the right branch, we define

$$\begin{aligned}\psi &\colon \neg A \to ((B + \neg B) \to C)\\ \psi(u) &:\equiv \mathsf{rec}_{B+\neg B}(C, \mathsf{inl} \circ \mathsf{inr}, \lambda v.\, \mathsf{inr}(\mathsf{rec}_{A+B}(\mathsf{0}, u, v))).\end{aligned}$$

Combining these branches, we obtain

$$f :\equiv \mathsf{rec}_{A+\neg A}((B + \neg B) \to C, \phi, \psi).$$

c) Since $A$ and $B$ are decidable, we have elements of the coproducts $A + \neg A$ and $B + \neg B$. We proceed by case analysis on $A + \neg A$:
   a) Assume $a\colon A$. By case analysis on $B + \neg B$:
      - If $b\colon B$, then $(a, b)$ injects into the left summand $(A \times B)$.
      - If $\zeta\colon \neg B$, then $\zeta \circ \pi_2\colon \neg(A \times B)$ injects into the right summand.
   b) If $\xi\colon \neg A$, then $\xi \circ \pi_1\colon \neg(A \times B)$ injects into the right summand.

d) Since $A$ and $B$ are decidable, we have elements of the coproducts $A + \neg A$ and $B + \neg B$. We proceed by case analysis on $A + \neg A$:
   - Assume $a\colon A$. By case analysis on $B + \neg B$:
     - If $b\colon B$, then the constant function $\lambda x.\, b\colon A \to B$ injects into the left summand.
     - If $\zeta\colon \neg B$, given $f\colon A \to B$, we have $f(a)\colon B$, and so $\zeta(f(a))\colon \mathsf{0}$. Hence, $\lambda f.\, \zeta(f(a))\colon \neg(A \to B)$ injects into the right summand.
   - Assume $\xi\colon \neg A$. Given $x\colon A$, we have $\xi(x)\colon \mathsf{0}$, so $\mathsf{rec}_{\mathsf{0}}(B, \xi(x))\colon B$. Hence, $\lambda x.\, \mathsf{rec}_{\mathsf{0}}(B, u(x))\colon A \to B$ injects into the left summand.

□

**Lemma 3.2.13.** *Let $P\colon A \to \mathcal{U}$ be a type family such that $P(x)$ is a mere proposition for all $x\colon A$. For any $u, v\colon \Sigma_A P$, the first projection induces an equivalence between their identity types:*

$$(u =_{\Sigma_A P} v) \simeq (\pi_1(u) =_A \pi_1(v)).$$

*Proof.* Consider

$$f\colon (u =_{\Sigma_A P} v) \to (\pi_1(u) =_A \pi_1(v))$$
$$p \mapsto \mathsf{ap}_{\pi_1}(p).$$

To construct a backward map

$$g\colon (\pi_1(u) =_A \pi_1(v)) \to (u =_{\Sigma_A P} v),$$

given a path $q\colon \pi_1(u) =_A \pi_1(v)$, take a witness $\epsilon\colon \mathsf{isProp}(P(\pi_1(v)))$. Then,

$$\gamma :\equiv \epsilon(q_*(\pi_2(u)), \pi_2(v))$$

inhabits $q_*(\pi_2(u)) =_{P(\pi_1(v))} \pi_2(v)$. Therefore, we can define the backward map as

$$g(q) :\equiv \mathsf{pair}^{=}(q, \gamma).$$

To conclude that $f$ and $g$ are quasi-inverses, we must verify that their compositions are homotopic to the respective identity maps:

- For any $q\colon \pi_1(u) =_A \pi_1(v)$, it is clear that $f(g(q)) = q$.
- For any $p\colon u =_{\Sigma_A P} v$, we can write $p = \mathsf{pair}^{=}(p_1, p_2)$ for some transport path $p_2\colon (p_1)_*(\pi_2(u)) =_{P(\pi_1(v))} \pi_2(v)$, where $p_1 :\equiv \mathsf{ap}_{\pi_1}(p)$.

  Evaluating $g(f(p))$ yields $\mathsf{pair}^{=}(p_1, \gamma')$, where $\gamma' :\equiv \epsilon((p_1)_*(\pi_2(u)), \pi_2(v))$. Because $P(\pi_1(v))$ is a mere proposition, it is also a set, meaning any two parallel paths within it are propositionally equal. In particular, $\gamma' = p_2$. Applying the function $\mathsf{ap}$ to this equality yields

  $$\mathsf{pair}^{=}(p_1, \gamma') = \mathsf{pair}^{=}(p_1, p_2),$$

  which implies $g(f(p)) = p$.

□

**Notation 3.2.14.** *When $P\colon A \to \mathcal{U}$ is a family of mere propositions, an alternative notation for $\Sigma_A P$ will be*

$$\{x\colon A \mid P(x)\}. \tag{3.1}$$

*If, in addition, $A$ is a set, we will say that* (3.1) *is a subset of $A$. In the general case, we will refer to this type as a subtype of $A$.*

*By* Proposition 3.2.9 *we can use this notation to introduce*

$$\mathsf{Set}_{\mathcal{U}} :\equiv \{A\colon \mathcal{U} \mid \mathsf{isSet}(A)\}$$
$$\mathsf{Prop}_{\mathcal{U}} :\equiv \{A\colon \mathcal{U} \mid \mathsf{isProp}(A)\}$$

*In particular, we can define mere proposition families, a.k.a. predicates of $A$ by*

$$\mathsf{Pred}(A) :\equiv A \to \mathsf{Prop}_{\mathcal{U}}.$$

**Theorem 3.2.15.**[FE UA] *Mere propositions are closed under the following constructions:*[3]

a) DEPENDENT FUNCTIONS: *For any arbitrary type $A$ and any family of mere propositions $P\colon A \to \mathcal{U}$, the dependent product $\prod_{x:A} P(x)$ is a mere proposition.*

b) FUNCTIONS & NEGATION: *If $Q$ is a mere proposition, the type $A \to Q$ is a mere proposition for any arbitrary type $A$. In particular, the negation $\neg A$ is always a mere proposition.*

c) PRODUCTS: *If $P$ and $Q$ are mere propositions, then their Cartesian product $P \times Q$ is a mere proposition.*

d) DEPENDENT PAIRS: *If $A$ is a mere proposition and $P\colon A \to \mathcal{U}$ is a family of mere propositions, then the total space $\Sigma_A P$ is a mere proposition.*

e) MUTUALLY EXCLUSIVE COPRODUCTS: *If $P$ and $Q$ are mere propositions, and they are mutually exclusive (i.e., we have a function $P \to \neg Q$), then the coproduct $P + Q$ is a mere proposition.*

f) EQUIVALENCES & IDENTITY: *If $P$ and $Q$ are mere propositions, the type of equivalences $P \simeq Q$ and their propositional equality $P =_{\mathsf{Prop}_\mathcal{U}} Q$ are mere propositions.*

*Proof.*

a) Let $f, g\colon \prod_{x:A} P(x)$. Given $x\colon A$, since $P(x)$ is a mere proposition, we have $f(x) =_{P(x)} g(x)$. Thus, the conclusion follows by function extensionality.

b) This is a direct consequence of part a) for the particular case where $P(x) \equiv Q$ for all $x\colon A$. Since $\mathsf{0}$ is clearly a mere proposition (witnessed by $\mathsf{ind}_{\mathsf{0}}(\prod_{x:\mathsf{0}} \prod_{y:\mathsf{0}} x =_{\mathsf{0}} y)$), we deduce that $\neg A \equiv A \to \mathsf{0}$ is a mere proposition.

c) Let $(a, b), (c, d)\colon P \times Q$. Then $a =_P c$ and $b =_Q d$ and so $(a, b) =_{P\times Q} (c, b) =_{P\times Q} (c, d)$. The conclusion follows because, for all $x\colon P \times Q$, we have $x =_{P\times Q} (\pi_1(x), \pi_2(x))$.

d) This is a direct consequence of Lemma 3.2.13.

e) Fix $f\colon P \to \neg Q$, and let $\epsilon\colon \mathsf{isProp}(P)$ and $\delta\colon \mathsf{isProp}(Q)$. To construct a dependent function of type

$$\mathsf{isProp}(P + Q) \equiv \prod_{u,v:\, P+Q} u =_{P+Q} v,$$

we proceed by double induction on $u, v\colon P + Q$.

[3] Function extensionality is used in parts a) and b). Univalance in part f).

For the cross-cases, given $x\colon P$ and $y\colon Q$, define:

$$g\colon (P \times Q) \to \mathsf{0}, \qquad (x,y) \mapsto f(x)(y),$$

$$\mathsf{rec}\colon \prod_{x\colon P} \prod_{y\colon Q} \mathsf{0} \to (\mathsf{inl}(x) =_{P+Q} \mathsf{inr}(y)), \quad (x,y) \mapsto \mathsf{rec}_{\mathsf{0}}(\mathsf{inl}(x) =_{P+Q} \mathsf{inr}(y)),$$

$$\zeta\colon \prod_{x\colon P} \prod_{y\colon Q} \mathsf{inl}(x) =_{P+Q} \mathsf{inr}(y), \qquad (x,y) \mapsto \mathsf{rec}(x,y)(g(x,y)).$$

Take $x, x'\colon P$. We have $\epsilon(x,x')\colon x =_P x'$ and so

$$\epsilon_x\colon \prod_{x'\colon P} \mathsf{inl}(x) =_{P+Q} \mathsf{inl}(x'), \quad \epsilon_x(x') :\equiv \mathsf{ap}_{\mathsf{inl}}(\epsilon(x,x')).$$

Let the target family for the inner induction be

$$C_x(v) :\equiv \mathsf{inl}(x) =_{P+Q} v.$$

Then

$$\epsilon_x\colon \prod_{x'\colon P} C_x(\mathsf{inl}(x')).$$

Moreover,

$$\begin{aligned}\zeta(x,-)\colon \prod_{y\colon Q} \mathsf{inl}(x) &=_{P+Q} \mathsf{inr}(y)\\ &\equiv \prod_{y\colon Q} C_x(\mathsf{inr}(y)).\end{aligned}$$

Thus, when $u \equiv \mathsf{inl}(x)$, induction on $v\colon P+Q$ yields

$$c_{\mathsf{inl}}(x) :\equiv \mathsf{ind}_{P+Q}\big(C_x, \epsilon_x, \zeta(x,-)\big)\colon \prod_{v\colon P+Q} \mathsf{inl}(x) =_{P+Q} v.$$

Similarly, given $y\colon Q$, define the family

$$D_y(v) :\equiv \mathsf{inr}(y) =_{P+Q} v.$$

Then,

$$\delta_y\colon \prod_{y'\colon Q} D_y(\mathsf{inr}(y')), \quad \delta_y(y') :\equiv \mathsf{ap}_{\mathsf{inr}}(\delta(y,y')).$$

and

$$\begin{aligned}\zeta(-,y)^{-1}\colon \prod_{x\colon P} \mathsf{inr}(y) &=_{P+Q} \mathsf{inl}(x)\\ &\equiv \prod_{x\colon P} D_y(\mathsf{inl}(x)).\end{aligned}$$

Thus, when $u \equiv \mathsf{inr}(y)$, induction on $v\colon P + Q$ yields

$$c_{\mathsf{inr}}(y) :\equiv \mathsf{ind}_{P+Q}\big(D_y, \zeta(-,y)^{-1}, \delta_y\big)\colon \prod_{v\colon P+Q} \mathsf{inr}(y) =_{P+Q} v.$$

Finally, define the family for the outer induction as

$$E(u) :\equiv \prod_{v\colon P+Q} u =_{P+Q} v.$$

By induction on $u\colon P + Q$, we obtain

$$\mathsf{ind}_{P+Q}\big(E, c_{\mathsf{inl}}, c_{\mathsf{inr}}\big)\colon \prod_{u,v\colon P+Q} u =_{P+Q} v,$$

which is a witness of $\mathsf{isProp}(P + Q)$.

f) By definition,

$$P \simeq Q \equiv \sum_{f\colon P\to Q} \mathsf{isequiv}(f).$$

By part b), the base type $P \to Q$ is a mere proposition. By part d), to prove that the total space $P \simeq Q$ is a mere proposition, it suffices to show that the fiber $\mathsf{isequiv}(f)$ is a mere proposition for all $f\colon P \to Q$.

Fix $f\colon P \to Q$. Recall that $\mathsf{isequiv}(f)$ is the product of two $\Sigma$-types:

$$\Big(\sum_{h\colon Q\to P} f \circ h \sim \mathsf{id}_Q\Big) \times \Big(\sum_{k\colon Q\to P} k \circ f \sim \mathsf{id}_P\Big).$$

By part c), we need only show that each factor is a mere proposition. Consider the first factor. By part b), the base type $Q \to P$ is a mere proposition. Hence, any two elements in this factor have equal first components. Furthermore, the fiber family is given by

$$(f \circ h \sim \mathsf{id}_Q) \equiv \prod_{y\colon Q} f(h(y)) =_Q \mathsf{id}_Q(y).$$

Because $Q$ is a mere proposition, Remark 3.2.7 ensures that the identity type $f(h(y)) =_Q \mathsf{id}_Q(y)$ is a mere proposition. Part a) then guarantees that the homotopy is a mere proposition. Since the fibers are mere propositions and the base components are necessarily equal, Lemma 3.2.13 implies that any two elements in this first factor are equal.

A symmetric argument applies to the second factor. Hence, $\mathsf{isequiv}(f)$ is a mere proposition, which concludes the first claim.

For the second claim, let $P \equiv \langle P_0, f\rangle$ and $Q \equiv \langle Q_0, g\rangle$ be two types in $\mathsf{Prop}_{\mathcal{U}} \equiv \sum_{A:\mathcal{U}} \mathsf{isProp}(A)$. By the characterization of paths in $\Sigma$-types, we have an equivalence

$$(P =_{\mathsf{Prop}_{\mathcal{U}}} Q) \simeq \sum_{h\colon P_0 =_{\mathcal{U}} Q_0} h_*(f) =_{\mathsf{isProp}(Q_0)} g.$$

By the univalence axiom, $(P_0 =_{\mathcal{U}} Q_0) \simeq (P_0 \simeq Q_0)$. Since $P_0$ and $Q_0$ are mere propositions, the first claim of this part implies that $P_0 \simeq Q_0$ is a mere proposition. Thus, its equivalent type $P_0 =_{\mathcal{U}} Q_0$ is also a mere proposition.

Furthermore, by Proposition 3.2.9, $\mathsf{isProp}(Q_0)$ is a mere proposition, which implies by Remark 3.2.7 that the identity type $h_*(f) =_{\mathsf{isProp}(Q_0)} g$ is a mere proposition for any $h$.

Since both the base type and the fibers of this $\Sigma$-type are mere propositions, part d) dictates that the total space is a mere proposition. Because the property of being a mere proposition is preserved under equivalence, we conclude that the identity type $P =_{\mathsf{Prop}_{\mathcal{U}}} Q$ is a mere proposition. □

**Remark 3.2.16.** The double induction on $u, v\colon P + Q$ in the proof of part (e) is geometrically analogous to integrating over the two-dimensional space

$$(P + Q) \times (P + Q).$$

The induction principle naturally partitions this space into four distinct *quadrants*:

$$P \times P, \quad P \times Q, \quad Q \times P, \quad \text{and} \quad Q \times Q.$$

The proof effectively *integrated* over the cross-quadrants $P \times Q$ and $Q \times P$ using $\zeta$ and $\zeta^{-1}$, and over the squares $P \times P$ and $Q \times Q$ using the internal mere proposition witnesses $\epsilon$ and $\delta$.

**Corollary 3.2.17.**[FE] *Let $f, g\colon A \to P$ be two functions. If $P$ is a mere proposition, then $f \sim g$ is a mere proposition as well.*

*Proof.* This is actually contained in the proof of the last part of the theorem. Take two inhabitants

$$\eta, \vartheta\colon f \sim g \equiv \prod_{x\colon A} f(x) =_P g(x).$$

By Remark 3.2.7, we deduce that $\eta(x) =_{f(x)=_P g(x)} \vartheta(x)$ for all $x\colon A$. Then, function extensionality yields $\eta =_{f \sim g} \vartheta$. The conclusion follows. □

**Corollary 3.2.18.**[FE] *Let $A$ and $B$ be sets, and let $f\colon A \to B$ be a function. Then the type $\mathsf{isequiv}(f)$ is a set. If $A$ and $B$ are mere propositions, then $\mathsf{isequiv}(f)$ is a mere proposition.*

*Proof.* To prove that $\mathsf{isequiv}(f)$ is a set, we have to prove that the type

$$\Big(\sum_{g\colon B \to A} f \circ g \sim \mathsf{id}_B\Big) \times \Big(\sum_{h\colon B \to A} h \circ f \sim \mathsf{id}_A\Big)$$

is a set.

By the symmetry between $A$ and $B$, and the fact that the Cartesian product of sets is a set given in part d) of Theorem 3.1.2, it suffices to show that the first $\Sigma$-type is a set.

By part f) of the same theorem, the type $B \to A$ is a set. Therefore, to reach the conclusion, it suffices to show that we can apply part e), i.e., that $f \circ g \sim \mathsf{id}_B$ is a set for every $g\colon B \to A$.

To see this, take $\eta_1, \eta_2\colon f \circ g \sim \mathsf{id}_B$. Given $y\colon B$, we have

$$\eta_1(y), \eta_2(y)\colon f(g(y)) =_B y.$$

Since $B$ is a set, we obtain $\eta_1(y) = \eta_2(y)$ in $f(g(y)) =_B y$. Therefore, function extensionality produces $\eta_1 = \eta_2$ in $f \circ g \sim \mathsf{id}_B$. Thus, $f \circ g \sim \mathsf{id}_B$ is a mere proposition, hence a set. The conclusion follows.

In the case where $A$ and $B$ are mere propositions, since they are also sets, the fact that $f \circ g \sim \mathsf{id}_B$ is a mere proposition allows us to invoke part d) of Theorem 3.2.15, because $B \to A$ is a mere proposition by part b) of the same theorem. By symmetry, the second $\Sigma$-type is also a mere proposition, and since the product of mere propositions is a mere proposition (part c), the type $\mathsf{isequiv}(f)$ is a mere proposition. □

**Corollary 3.2.19.**[FE] *Let $A$ and $B$ be sets. Then the type $A \simeq B$ is a set.*

*Proof.* By definition, the type of equivalences is given by the $\Sigma$-type:

$$A \simeq B :\equiv \sum_{f\colon A \to B} \mathsf{isequiv}(f).$$

To show that a $\Sigma$-type is a set, by part e) of Theorem 3.1.2, it suffices to show that $A \to B$ is a set and that each $\mathsf{isequiv}(f)$ is a set.

The first requirement is covered by part f) of said theorem. The second by Corollary 3.2.18. □

**Proposition 3.2.20.**[FE] *Let $A$ and $B$ be sets, and let $f\colon A \to B$ be a function. Then the type $\mathsf{qinv}(f)$ is a set.*

*Proof.* By definition,

$$\mathsf{qinv}(f) \equiv \sum_{g\colon B \to A} (f \circ g \sim \mathsf{id}_B) \times (g \circ f \sim \mathsf{id}_A).$$

By part e) of Theorem 3.1.2, to show that $\mathsf{qinv}(f)$ is a set, it suffices to show that the base type $B \to A$ is a set, and that for each $g\colon B \to A$, the fiber $(f \circ g \sim \mathsf{id}_B) \times (g \circ f \sim \mathsf{id}_A)$ is a set.

The former type is a set by part f) of the same theorem .

For the latter type, since the Cartesian product of sets is a set (same theorem part d), we need to show that both factors are set. Consider the first one:

$$f \circ g \sim \mathsf{id}_B \equiv \prod_{y:\, B} f(g(y)) =_B y.$$

Because $B$ is a set, given $y\colon B$, the identity type $f(g(y)) =_B y$ is a mere proposition by Remark 3.2.2. By part a) of Theorem 3.2.15, a dependent product of mere propositions is a mere proposition, hence, a set. Thus, $f \circ g \sim \mathsf{id}_B$ is a set.

By symmetry, the second factor

$$g \circ f \sim \mathsf{id}_A$$

is a set as well. □

## 3.3 Propositional Truncation

We can define a new type former that converts any type $A\colon \mathcal{U}$ into $\|A\|\colon \mathcal{U}$, whose canonical terms are of the form $|a|$ for $a\colon A$. The distinguishing characteristic of $\|A\|$ is that it is a mere proposition; that is, for any $x, y\colon \|A\|$, we have $x =_{\|A\|} y$. More precisely, there exists a dependent function

$$\mathsf{squash}\colon \prod_{x,y:\, \|A\|} x =_{\|A\|} y.$$

The induction principle, which can be applied to mere proposition families, is given by:

$$\mathsf{ind}_{\|A\|}\colon \prod_{C:\, \mathsf{Pred}(\|A\|)} \Big(\prod_{a:\, A} C(|a|)\Big) \to \prod_{z:\, \|A\|} C(z), \tag{3.1}$$

satisfying $\mathsf{ind}_{\|A\|}(C, f, |a|) :\equiv f(a)$ for $a\colon A$.

In the case of non-dependent mere proposition families, we have the recursion principle

$$\mathsf{rec}_{\|A\|}\colon \prod_{B:\, \mathsf{Prop}_{\mathcal{U}}} (A \to B) \to (\|A\| \to B) \tag{3.2}$$
$$\mathsf{rec}_{\|A\|}(B, f) :\equiv \mathsf{ind}_{\|A\|}(a \mapsto B, f).$$

**Remark 3.3.1.** One might be tempted to simplify the defining requirement $x =_{\|A\|} y$ for all $x, y\colon \|A\|$, to merely $|a| =_{\|A\|} |b|$ for all $a, b\colon A$, and then attempt to use induction on $\|A\|$ to deduce the former. However, to apply the induction principle, we must first show that the target family $C(x, y) :\equiv x =_{\|A\|} y$ is a predicate. This requires already knowing that the identity type $x =_{\|A\|} y$ is a

mere proposition, which is equivalent to knowing that $\|A\|$ is a set. Because we do not yet know this, we are trapped in a circular reasoning.

Imposing paths only on canonical elements fails to define a set. For instance, consider the type $1 + 2$. This type has exactly three discrete elements, say $x, y, z$, derived from injecting $\star$ on the left, and $0_2$ and $1_2$ on the right. If we explicitly adjoin paths $p_{xy} : x = y$, $p_{yz} : y = z$, and $p_{xz} : x = z$, we obtain a hollow triangle. In this space, there are two distinct ways to travel from $x$ to $z$: the path $p_{xz}$, and the concatenation $p_{xy} \cdot p_{yz}$. Because we have not explicitly stated that $p_{xz} = p_{xy} \cdot p_{yz}$, these two paths are not equal. Consequently, the identity type $x = z$ is not a mere proposition, and the resulting type is not a set.

**Lemma 3.3.2.** *Truncation is a functorial operation. That is,*

a) *Every map $f : A \to B$ produces a map $\|f\| : \|A\| \to \|B\|$.*

b) *For any type $A$, we have $\|\mathsf{id}_A\| \sim \mathsf{id}_{\|A\|}$.*

c) *For any $f : A \to B$ and $g : B \to C$, we have $\|g \circ f\| \sim \|g\| \circ \|f\|$.*

*Consequently, if $A \simeq B$, then $\|A\| \simeq \|B\|$.*

*Proof.* Given $f : A \to B$, we define the map $\|f\|$ using the recursion principle $\mathsf{rec}_{\|A\|}$. Since the target $\|B\|$ is a mere proposition, it suffices to define its action on the canonical constructor:

$$\|f\| :\equiv \mathsf{rec}_{\|A\|}(\|B\|, a \mapsto |f(a)|).$$

For b), let $x : \|A\|$. We must show that $\|\mathsf{id}_A\|(x) =_{\|A\|} x$. By part d) of Theorem 3.2.15 applied to the constant family $P(x) :\equiv A$, the identity type of a mere proposition is itself a mere proposition. Therefore, we can proceed by induction on $x$. It suffices to consider the case $x \equiv |a|$ for some $a : A$. In this case, $\|\mathsf{id}_A\|(|a|)$ reduces definitionally to $|\mathsf{id}_A(a)| \equiv |a|$, so the equality holds by reflexivity.

For c), by induction on $x : \|A\|$, we only need to check the base case $x \equiv |a|$. Since both $\|g \circ f\|(|a|)$ and $\|g\|(\|f\|(|a|))$ compute definitionally to $|g(f(a))|$, the required path is simply reflexivity.

Finally, suppose that $f : A \to B$ induces an equivalence and let $f^{-1} : B \to A$ be a quasi-inverse. By a), we obtain $\|f\| : \|A\| \to \|B\|$ and $\|f^{-1}\| : \|B\| \to \|A\|$. Since the target spaces are mere propositions, invoking Theorem 3.2.15 as before, we deduce that both compositions $\|f^{-1}\| \circ \|f\|$ and $\|f\| \circ \|f^{-1}\|$ are trivially homotopic to $\mathsf{id}_{\|A\|}$ and $\mathsf{id}_{\|B\|}$, respectively. Thus, $\|f\|$ is an equivalence.

□

**Lemma 3.3.3.**[UA] *If $A$ is a mere proposition, then $A =_{\mathcal{U}} \|A\|$.*

*Proof.* Since $A$ is a mere proposition, recursion yields a function

$$\mathsf{rec}_{\|A\|}(A, \mathsf{id}_A) \colon \|A\| \to A.$$

Conversely, the canonical constructor gives:

$$|-| \colon A \to \|A\|, \quad a \mapsto |a|.$$

The conclusion follows from Lemma 3.2.4 and the univalence axiom. □

**Lemma 3.3.4.**[FE] *Let $S$ be a set, and let $A \colon \mathcal{U}$ be a type. If $\|A =_{\mathcal{U}} S\|$ is inhabited, then $A$ is a set.*

*Proof.* We must construct an inhabitant of the type $\mathsf{isSet}(A)$.

By recursion for propositional truncation, to define a function $\|X\| \to P$, where $P$ is a mere proposition, it suffices to construct a map from $X$ to $P$. By Proposition 3.2.9, we know that $\mathsf{isSet}(C)$ is a mere proposition for every $C \colon \mathcal{U}$. Hence, we can apply recursion to the case $X :\equiv A =_{\mathcal{U}} S$ and $P :\equiv \mathsf{isSet}(A)$.

To provide the function $f \colon (A =_{\mathcal{U}} S) \to \mathsf{isSet}(A)$, fix a witness $\epsilon \colon \mathsf{isSet}(S)$ and define:

$$f(q) :\equiv \mathsf{transport}^{\mathsf{isSet}}(q^{-1}, \epsilon).$$

Thus, if $h \colon \mathsf{isProp}(\mathsf{isSet}(A))$, we obtain the function

$$\mathsf{rec}_{\|A =_{\mathcal{U}} S\|}\big(\mathsf{isSet}(A), h, f\big) \colon \|A =_{\mathcal{U}} S\| \to \mathsf{isSet}(A).$$

The conclusion follows. □

**Notation 3.3.5.** *Truncation allows us to interpret traditional logical notation via the following type-theoretic operations. Although we will not adopt this notation extensively —reserving it primarily for contexts strictly focused on sets and propositions— it establishes a precise dictionary between classical logic and type theory. Here, $P$ and $Q$ denote mere propositions (or mere proposition families), and $A$ is an arbitrary type.*

*For operations that inherently preserve mere propositions, no truncation is necessary:*

$$\top :\equiv \mathsf{1} \quad \& \quad \bot :\equiv \mathsf{0} \qquad\qquad \neg P :\equiv P \to \mathsf{0}$$

$$P \wedge Q :\equiv P \times Q \qquad\qquad \forall (x \colon A).\, P(x) :\equiv \prod_{x \colon A} P(x)$$

$$P \Rightarrow Q :\equiv P \to Q \qquad\qquad P \Leftrightarrow Q :\equiv P =_{\mathcal{U}} Q$$

*For operations that can construct spaces with multiple distinct points, we must apply propositional truncation to strictly yield a mere proposition:*

$$P \vee Q :\equiv \|P + Q\|$$

$$\exists (x \colon A).\, P(x) :\equiv \left\| \sum_{x \colon A} P(x) \right\|$$

## 3.4 Contractibility

**Definitions 3.4.1.**

- A type $A$ is *contractible* (a.k.a. a *singleton*) if there is an element $a\colon A$, called the *center of contraction*, such that every element in $A$ is equal to $a$. We define the type of such witnesses as

$$\mathsf{isContr}(A) :\equiv \sum_{a\colon A} \prod_{x\colon A} a =_A x.$$

  Identifying the element $a\colon A$ with the constant function $\lambda x.\, a$, we can rewrite the definition as

$$\mathsf{isContr}(A) :\equiv \sum_{a\colon A} a \sim \mathsf{id}_A.$$

  If $a$ is a center of contraction, i.e., there is a pair $\langle a, \eta\rangle\colon \mathsf{isContr}(A)$, the homotopy $\eta$ is called the *contraction homotopy*.

- For any type $A$ and any type family $P\colon A \to \mathcal{U}$, the *weak function extensionality axiom* asserts the existence of a function

$$\Big(\prod_{x\colon A} \mathsf{isContr}(P(x))\Big) \to \mathsf{isContr}\Big(\prod_{x\colon A} P(x)\Big).$$

**Notation 3.4.2.** *From now on we will tag with* WE *results that rely on weak function extensionality.*

**Lemma 3.4.3.** *For a type $A$, the following are logically equivalent.*

a) *$A$ is contractible.*

b) *$A$ is a mere proposition, and there is a point $a\colon A$.*

c) *$A$ is equivalent to $\mathsf{1}$.*

*Proof.*

a) $\Rightarrow$ b) The hypothesis provides a witness $h\colon \mathsf{isContr}(A)$. Composition with first projection yields a witness $\pi_1(h))\colon A$. Hence, $A$ is inhabited.

b) $\Rightarrow$ c) See Lemma 3.2.3.

c) $\Rightarrow$ a) The canonical element $\star\colon \mathsf{1}$ is a center of contraction as shown by the $\mathsf{uniq}_{\mathsf{1}}$ function [cf. 31. DEPENDENT FUNCTIONS OVER PRODUCT].

□

**Lemma 3.4.4.** *Let $A$ be a type, $a, b\colon A$, and define the type family*

$$H\colon A \to \mathcal{U}, \quad H(z) :\equiv z \sim \mathsf{id}_A.$$

*Then, given $\eta\colon H(a)$ and $\gamma\colon a =_A b$, we have for any $x\colon A$*

$$\mathsf{transport}^H(\gamma, \eta)(x) =_{b=_A x} \gamma^{-1} \cdot \eta(x).$$

*Proof.* The transport on the LHS is an element of $H(b)$. Its evaluation at $x$ yields a path in $b =_A x$. On the other hand, the RHS concatenates a path from $b$ to $a$ with a path from $a$ to $x$, yielding a path from $b$ to $x$. Thus, both sides inhabit the type $b =_A x$.

By based path induction on $\gamma\colon a =_A b$ it suffices to assume that $b \equiv a$ and $\gamma \equiv \mathsf{refl}_a$. Under this assumption, the LHS reduces trivially:

$$\mathsf{transport}^H(\mathsf{refl}_a, \eta)(x) \equiv \eta(x).$$

For the RHS, the inverse of reflexivity is definitionally reflexivity, and since reflexivity acts trivially on the left, the equality with $\eta(x)$ follows. □

**Theorem 3.4.5.**[WE] *For any type $A$, the type $\mathsf{isContr}(A)$ is a mere proposition.*

*Proof.* Let $c, d\colon \mathsf{isContr}(A)$. Set

$$a :\equiv \pi_1(c), \quad \eta :\equiv \pi_2(c) \quad \text{and} \quad b :\equiv \pi_1(d), \quad \vartheta :\equiv \pi_2(d).$$

Then,

$$\eta\colon a \sim \mathsf{id}_A \quad \text{and} \quad \vartheta\colon b \sim \mathsf{id}_A.$$

Evaluating $\eta$ at $b$ yields a path $\eta(b)\colon a =_A b$, which provides the equality of the first components.

Let

$$P\colon A \to \mathcal{U}, \quad P(z) :\equiv z \sim \mathsf{id}_A.$$

To prove $c =_{\mathsf{isContr}(A)} d$ we need to show that

$$\mathsf{transport}^P(\eta(b), \eta) =_{P(b)} \vartheta.$$

Since $d$ witnesses that $A$ is contractible with center $b$, for any $x\colon A$, the identity type $b =_A x$ is contractible, by Remark 3.2.7 and Lemma 3.4.3. By the weak function extensionality axiom, the homotopy type $b \sim \mathsf{id}_A$, which is exactly $P(b)$, is also contractible.

Because every contractible type is a mere proposition, any two of its elements are equal. Consequently, the equality $\mathsf{transport}^P(\eta(b), \eta) =_{P(b)} \vartheta$ holds. □

**Corollary 3.4.6.**[WE] *If $A$ is contractible, $\mathsf{isContr}(A)$ is also contractible.*

*Proof.* By the theorem, $\mathsf{isContr}(A)$ is a mere proposition. It is also inhabited because $A$ is contractible. The conclusion follows from part b) of Lemma 3.4.3. □

**Remark 3.4.7.** Let $A : \mathcal{U}$ be a type and $P : A \to \mathcal{U}$ a type family. Claiming that *the type $P(x)$ is contractible for every $x : A$* means that we are given a dependent function

$$h : \prod_{x : A} \mathsf{isContr}(P(x)),$$

i.e.,

$$h : \prod_{x : A} \sum_{c : P(x)} c \sim \mathsf{id}_{P(z)}.$$

In particular, given $x : A$, the term $c_x :\equiv \pi_1(h(x))$ is a center of contraction for $P(x)$. In other words, picking the center of contraction for every contractible fiber of $P$ is a constructive, choice-free operation.

**Theorem 3.4.8.**[FE UA] *Contractible types are closed under the following type-theoretic constructions:*[4]

a) Dependent Functions: *For any type $A$ and any family of contractible types $P : A \to \mathcal{U}$, the dependent product $\prod_{x : A} P(x)$ is contractible.*

b) Functions: *If $Q$ is contractible, the type $A \to Q$ is contractible for any type $A$.*

c) Products: *If $P$ and $Q$ are contractible, then their Cartesian product $P \times Q$ is contractible.*

d) Dependent Pairs: *If $A$ is contractible and $P : A \to \mathcal{U}$ is a family of contractible types, then the total space $\Sigma_A P$ is contractible.*

e) Identity Types: *If $A$ is contractible, then for any $x, y : A$, the identity type $x =_A y$ is contractible.*

f) Equivalences & Identity: *If $P$ and $Q$ are contractible, the type of equivalences $P \simeq Q$ and their propositional equality $P =_{\mathcal{U}} Q$ are contractible.*

g) Homotopies: *If $f, g : \prod_{x : A} B(x)$, where $B(x)$ is contractible for $x : A$, then the type $f \sim g$ is contractible.*

*Proof.*

a) Set $\Pi_A P :\equiv \prod_{x : A} P(x)$. Given $x : A$, let $c_x : P(x)$ be the center of contraction, and let $\eta_x : c_x \sim \mathsf{id}_{P(x)}$ be the corresponding contraction homotopy. Define $c :\equiv \lambda x.\, c_x$. For any function $f : \Pi_A P$, the pointwise equality $\eta_x \triangleright f : c \sim f$ induces, by function extensionality, a path $c =_{\Pi_A P} f$.

[4] Function extensionality is needed in parts a), b), f), and g). Univalence in part f).

b) This is a direct consequence of part a), applied to the constant type family $P(x) :\equiv Q$ for all $x\colon A$.

c) Let $c$ and $d$ be the centers of contraction for $P$ and $Q$, respectively. Given $(p, q)\colon P \times Q$, we have paths $\alpha\colon c =_P p$ and $\beta\colon d =_Q q$. Therefore, $\mathsf{pair}^{=}(\alpha, \beta)$ is a path $(c, d) =_{P\times Q} (p, q)$.

d) Let $c$ be the center of $A$, and let $d$ be the center of $P(c)$. Given $\langle x, u\rangle\colon \Sigma_A P$, the contractibility of $A$ yields a path $\alpha\colon c =_A x$. Since the fiber $P(x)$ is contractible, there is a path $\beta\colon \mathsf{transport}^P(\alpha, d) =_{P(x)} u$. Therefore, $\mathsf{pair}^{=}(\alpha, \beta)$ is a path $\langle c, d\rangle =_{\Sigma_A P} \langle x, u\rangle$.

e) Let $c$ be the center of $A$. For any $x, y\colon A$, there exist paths $p\colon c =_A x$ and $q\colon c =_A y$. Then $p^{-1} \cdot q\colon x =_A y$, meaning the identity type is inhabited. Since $A$ is a mere proposition, the identity type $x =_A y$ is also a mere proposition by Remark 3.2.7. Because it is an inhabited mere proposition, it is contractible by Lemma 3.4.3.

f) By part b), the type $P \to Q$ is contractible. The type of equivalences is defined as $P \simeq Q \equiv \sum_{f\colon P\to Q} \mathsf{isequiv}(f)$. By Corollary 3.2.18, the type $\mathsf{isequiv}(f)$ is a mere proposition. Because $P$ and $Q$ are both contractible, any function between them is an equivalence [cf. Lemma 3.2.4]. Hence, $\mathsf{isequiv}(f)$ is inhabited and thus contractible for any $f$. By part d), the total space $P \simeq Q$ is contractible. Finally, by the univalence axiom, the identity type $P =_{\mathcal{U}} Q$ is equivalent to $P \simeq Q$. Since contractibility is preserved under equivalence by Proposition 3.4.10, the type $P =_{\mathcal{U}} Q$ is also contractible.

g) By definition, $f \sim g \equiv \prod_{x\colon A} f(x) =_{B(x)} g(x)$. Since each $B(x)$ is contractible, part e) implies that the identity type $f(x) =_{B(x)} g(x)$ is contractible for every $x\colon A$. The conclusion follows from part a).

□

**Definition 3.4.9.** A *retraction* (a.k.a. *split surjection*) is a function $r\colon A \to B$ such that there exists a function $s\colon B \to A$, called its *section*, and a homotopy $\eta\colon r \circ s \sim \mathsf{id}_B$. When a retraction exists, we say that $B$ is a *retract* of $A$.

**Proposition 3.4.10.** *Every retract of a contractible type is contractible.*

*Proof.* Let $A$ be contractible and $r\colon A \to B$ a retraction. If $a\colon A$ denotes its center of contraction, we claim that $b :\equiv r(a)$ is a center of contraction for $B$. This means that we must construct a witness of

$$\prod_{y\colon B} b =_B y.$$

Let $s\colon B \to A$ be a section of $r$, and $\eta\colon r \circ s \sim \mathsf{id}_B$ the corresponding homotopy.

The hypothesis implies that for every $y\colon B$, the identity type $a =_A s(y)$ is inhabited. Applying $\mathsf{ap}_r$ yields a witness of $b =_B r(s(y))$, whose concatenation with $\eta(y)$ constructs a witness of $b =_B y$. The conclusion follows.

□

**Lemma 3.4.11.** *Let $A$ be a type. Given $a\colon A$, the type $\sum_{x\colon A} a =_A x$ is contractible with center of contraction $\langle a, \mathsf{refl}_a\rangle$.*

*Proof.* Let $\Sigma$ denote this $\Sigma$-type. Given $\langle x, p\rangle\colon \Sigma$, we have $p\colon a =_A x$. Since

$$\mathsf{transport}^{\zeta\mapsto(a=_A\zeta)}(p, \mathsf{refl}_a) = \mathsf{refl}_a \cdot p \equiv p,$$

by part a) of Proposition 2.11.6, the identity type $\langle a, \mathsf{refl}_a\rangle =_\Sigma \langle x, p\rangle$ is inhabited. □

**Remark 3.4.12.** Suppose that $A\colon \mathcal{U}$ is a contractible type. If $c$ is a center of contraction for $A$, then for every $x\colon A$ we have a path $p_x\colon c =_A x$. Even though we cannot assume that $p_c \equiv \mathsf{refl}_c$, we do have

$$p_c =_{c=_A c} \mathsf{refl}_c,$$

because every contractible type is a mere proposition, and every mere proposition is a set.

**Theorem 3.4.13.**[FE] *The principle that contractible data can be freely ignored up to equivalence manifests in the following equivalences. Let $A$ and $B$ be types, and let $P\colon A \to \mathcal{U}$ be a type family.*[5]

a) Dependent Pairs (Right): *If for every $x\colon A$, the type $P(x)$ is contractible, then $\sum_{x\colon A} P(x) \simeq A$.*

b) Dependent Pairs (Left): *If $A$ is contractible with center $c\colon A$, then $\sum_{x\colon A} P(x) \simeq P(c)$.*

c) Dependent Functions (Left): *If $A$ is contractible with center $c\colon A$, then $\prod_{x\colon A} P(x) \simeq P(c)$.*

d) Dependent Functions (Right): *If for every $x\colon A$, the type $P(x)$ is contractible, then $\prod_{x\colon A} P(x)$ is contractible, hence $\prod_{x\colon A} P(x) \simeq \mathsf{1}$.*

e) Cartesian Products: *If $B$ is contractible, then $A \times B \simeq A$. By symmetry, if $A$ is contractible, then $A \times B \simeq B$.*

f) Functions (Domain): *If $A$ is contractible, then $(A \to B) \simeq B$.*

g) Functions (Codomain): *If $B$ is contractible, then $A \to B$ is contractible, hence $(A \to B) \simeq \mathsf{1}$.*

[5] Function extensionality is needed in parts c), d), f), and g).

*Proof.*

a) Consider the function

$$\begin{aligned} \iota\colon A &\to \sum_{x\colon A} P(x) \\ x &\mapsto \langle x, c_x\rangle, \end{aligned}$$

where $c_x$ is the center of contraction for $P(x)$ [cf. Remark 3.4.7]. Then $\pi_1 \circ \iota \equiv \mathsf{id}_A$, and

$$\iota(\pi_1(\langle x, u\rangle)) \equiv \langle x, c_x\rangle.$$

By equation (2.1) of Remark 2.3.2, to verify that $\langle x, c_x\rangle =_{\Sigma_A P} \langle x, u\rangle$, we must construct a witness of $c_x =_{P(x)} u$. But this is trivial because the identity type $c_x =_{P(x)} u$ is inhabited by hypothesis.

b) Let

$$h\colon \mathsf{isContr}(A) \equiv \sum_{c\colon A} c \sim \mathsf{id}_A.$$

Then $c :\equiv \pi_1(h)$ is a center for $A$, and given $x\colon A$, $p_x :\equiv \pi_2(h)(x)$ satisfies $p_x\colon c =_A x$. Thus, we can introduce

$$\begin{aligned} f\colon \sum_{x\colon A} P(x) &\to P(c) \\ \langle x, u\rangle &\mapsto \mathsf{transport}^P(p_x^{-1}, u), \end{aligned}$$

which is well-defined because $p_x^{-1}\colon x =_A c$. On the other hand, we have

$$\begin{aligned} \iota\colon P(c) &\to \sum_{x\colon A} P(x) \\ u &\mapsto \langle c, u\rangle. \end{aligned}$$

Then,

$$\begin{aligned} f(\iota(u)) &\equiv f(\langle c, u\rangle) \\ &\equiv \mathsf{transport}^P(p_c^{-1}, u) \\ &=_{P(c)} \mathsf{transport}^P(\mathsf{refl}_c^{-1}, u) \\ &\equiv u, \end{aligned}$$

where the equality follows from Remark 3.4.12 and $\mathsf{ap}_{\mathsf{transport}^P(-,u)}$.

Moreover,

$$\iota(f(\langle x, u\rangle)) \equiv \langle c, \mathsf{transport}^P(p_x^{-1}, u)\rangle.$$

Since $p_x\colon c =_A x$, to prove that $\iota \circ f(\langle x, u\rangle) =_{\Sigma_A P} (\langle x, u\rangle)$, we need to verify that

$$\mathsf{transport}^P\big(p_x, \mathsf{transport}^P(p_x^{-1}, u)\big) =_{P(x)} u,$$

which is witnessed by $\mathsf{transportinv}_r(p_x, u)$ [cf. Lemma 2.3.11].

c) Let $c\colon A$ be a center of contraction. Evaluation at $c$ gives us a map

$$\begin{aligned}\mathrm{ev}_c\colon \prod_{x\colon A} P(x) &\to P(c)\\ f &\mapsto f(c).\end{aligned}$$

For the backward function, define

$$\begin{aligned}\gamma\colon P(c) &\to \prod_{x\colon A} P(x)\\ u &\mapsto \lambda x.\, \mathsf{transport}^P(p_x, u),\end{aligned}$$

where $p_x\colon c =_A x$ is constructed as in part b). These functions satisfy:

$$\begin{aligned}\mathrm{ev}_c(\gamma(u)) &\equiv \mathsf{transport}^P(p_c, u)\\ &=_{P(c)} \mathsf{transport}^P(\mathsf{refl}_c, u)\\ &\equiv u,\end{aligned}$$

where the equality follows from Remark 3.4.12 and $\mathsf{ap}_{\mathsf{transport}^P(-,u)}$. In addition, for $x\colon A$,

$$\begin{aligned}\gamma(f(c))(x) &\equiv \mathsf{transport}^P(p_x, f(c))\\ &=_{P(x)} f(x) \qquad\qquad ;\text{ by } \mathsf{apd}_f(p_x),\end{aligned}$$

where the last equality follows from Lemma 2.3.6. By function extensionality, $\gamma(f(c)) = f$ in $\prod_{x\colon A} P(x)$.

d) For $x\colon A$, let $(f_x, \eta_x, \vartheta_x)\colon P(x) \simeq \mathsf{1}$ be an equivalence with quasi-inverse $f_x^{-1}$. Define

$$\begin{aligned}\gamma\colon \prod_{x\colon A} P(x) &\to \mathsf{1}\\ u &\mapsto \star,\end{aligned}$$

and

$$\begin{aligned}\epsilon\colon \mathsf{1} &\to \prod_{x\colon A} P(x)\\ z &\mapsto \lambda x.\, f_x^{-1}(z).\end{aligned}$$

Then,

$$\gamma(\epsilon(\star)) \equiv \star.$$

Since $P(x)$ is a mere proposition, there is a path $p_x\colon f_x^{-1}(\star) =_{P(x)} u(x)$. Hence,

$$\begin{aligned}\epsilon(\gamma(u))(x) &\equiv f_x^{-1}(\star)\\ &=_{P(x)} u(x) \qquad\qquad ;\text{ by } p_x.\end{aligned}$$

Thus, by function extensionality,

$$\epsilon(\gamma(u)) =_{\prod_{x:\,A} P(x)} u.$$

e) Suppose that $B$ is contractible. Define

$$\begin{aligned} \iota &: A \to A \times B \\ a &\mapsto (a, c), \end{aligned}$$

where $c$ is a center of contraction for $B$. Then, $\iota$ is a quasi-inverse of $\pi_1$, because $\pi_1(\iota(a)) \equiv \pi_1(a, c) \equiv a$, and

$$\begin{aligned} \iota(\pi_1(a, b)) &\equiv (a, c) \\ &=_{A\times B} (a, b) \qquad\qquad ; \text{ by } \mathsf{ap}_{y\mapsto(a,y)}(p_b), \end{aligned}$$

where $p_b : c =_B b$.

f) This is a particular case of part c) for the constant family $B$.

g) This is a particular case of part d) for the constant family $B$.

□

**Lemma 3.4.14.** *A type $A$ is a mere proposition if, and only if, for all $x, y : A$, the type $x =_A y$ is contractible.*

*Proof.* If $A$ is a mere proposition, by Lemma 3.2.6, it is also a set. Therefore, by definition, $x =_A y$ is a mere proposition. Since $x$ (as well as $y$) is a witness of $A$, it follows from Lemma 3.4.3 that $A$ is contractible. Hence, it has a center, say $c$, and we can take $p : c =_A x$ and $q : c =_A y$ to construct $p^{-1} \cdot q : x =_A y$.

Conversely, if $x =_A y$ is contractible for all $x, y : A$, then $A$ is a mere proposition by definition. □

**Definitions 3.4.15.** Let $f : A \to B$ be a function.

- The *fiber of $f$* is the type family

$$\begin{aligned} \mathsf{fib}_f &: B \to \mathcal{U} \\ \mathsf{fib}_f(y) &:\equiv \sum_{x:\,A} f(x) =_B y. \end{aligned}$$

- The function $f$ is *contractible* if, for every $y : B$, the fiber $\mathsf{fib}_f(y)$ of $f$ over $y$ is contractible. In other words, the type

$$\mathsf{isContrMap}(f) :\equiv \prod_{y:\,B} \mathsf{isContr}(\mathsf{fib}_f(y)) \tag{3.1}$$

is inhabited.

**Lemma 3.4.16.** *For any function $f\colon A \to B$, the type* $\mathsf{isContrMap}(f)$ *is a mere proposition.*

*Proof.* By definition, the type of contractible maps is the dependent type given in (3.1).

By Theorem 3.4.5, the type $\mathsf{isContr}(\mathsf{fib}_f(y))$ is a mere proposition for every $y\colon B$. Then, the conclusion follows from part d) of Theorem 3.2.15.

□

## 3.5 The Axiom of Choice

**Remark 3.5.1.** A consequence of part d) of Theorem 3.2.15 applies to the universes of sets and mere propositions, $\mathsf{Set}_{\mathcal{U}}$ and $\mathsf{Prop}_{\mathcal{U}}$. By definition, elements of these universes are dependent pairs consisting of a base type and a witness to its particular characteristic.

Because the types $\mathsf{isSet}(A)$ and $\mathsf{isProp}(P)$ are themselves mere propositions, part d) guarantees that the second coordinates of these pairs add no new information. In the language of type theory, being a set or a mere proposition is a *property* of a type, rather than an additional *structure* placed upon it.

Consequently, the first projections from $\mathsf{Set}_{\mathcal{U}}$ and $\mathsf{Prop}_{\mathcal{U}}$ into $\mathcal{U}$ are embeddings. This rigorously justifies a standard, ubiquitous abuse of notation: it is perfectly valid to systematically identify pairs $A\colon \mathsf{Set}_{\mathcal{U}}$ and $P\colon \mathsf{Prop}_{\mathcal{U}}$ with their first coordinates, safely treating them as plain types, e.g., writing $x\colon A$ instead of $x\colon \pi_1(A)$.

In particular, a type family $A\colon X \to \mathcal{U}$ such that $\mathsf{isSet}(A(x))$ is inhabited for every $x\colon X$, will often be denoted as a family $A\colon X \to \mathsf{Set}_{\mathcal{U}}$. Similarly, the notation $P\colon X \to \mathsf{Prop}_{\mathcal{U}}$ refers to a mere proposition family, i.e., a family such that $P(x)$ is a mere proposition for all $x\colon X$.

---

As we have seen in 39. Type-theoretic Axiom of Choice, the "naive" extraction of a choice function from a dependent pair is a direct consequence of the rules for Π- and Σ-types, requiring no axioms. However, this relies on having the explicit data of the elements.

To formulate the classical Axiom of Choice, we must express the proposition that a choice function exists even when the specific witnesses have been truncated into mere propositions.

**Definition 3.5.2.** Let $X$ be a set, $A\colon X \to \mathsf{Set}_{\mathcal{U}}$ be a family of sets, and $P\colon \prod_{x\colon X} A(x) \to \mathsf{Prop}_{\mathcal{U}}$ be a family of mere propositions. The *axiom of choice*

is the following inhabitancy assertion:

$$\mathsf{ac}_0 : \Big(\prod_{x:\,X} \Big\| \sum_{a:\,A(x)} P(x,a)\Big\|\Big) \to \Big\| \sum_{f:\,\prod_{x:\,X} A(x)} \prod_{x:\,X} P(x, f(x))\Big\|. \tag{3.1}$$

**Lemma 3.5.3.**[FE] *The axiom of choice is equivalent to the statement that for any set $X$ and any family of sets $C : X \to \mathsf{Set}_{\mathcal{U}}$, the function type*

$$\Big(\prod_{x:\,X} \|C(x)\|\Big) \to \Big\|\prod_{x:\,X} C(x)\Big\| \tag{3.2}$$

*is inhabited.*

*Proof.* Let $X, A, P$ be as in Definition 3.5.2. By Theorem 2.18.3, the RHS of (3.1) is equivalent to

$$\Big\|\prod_{x:\,X} \sum_{y:\,A(x)} P(x,y)\Big\|. \tag{3.3}$$

Let

$$C(x) :\equiv \sum_{y:\,A(x)} P(x,y).$$

By Theorem 3.1.2 e), $C(x)$ is a set for $x : X$. Hence, if (3.2) holds, applying it to $C$ yields a map to (3.3). Since the equivalence of Theorem 2.18.3 lifts through propositional truncation, composing these maps yields a witness for (3.1).

To show that (3.1) yields (3.2), take $A(x) :\equiv C(x)$ and $P(x,y) :\equiv \mathsf{1}$. Then

$$\sum_{a:\,A(x)} P(x,a) \equiv \Big(\sum_{a:\,C(x)} \mathsf{1}\Big) \simeq C(x)$$

where the equivalence is given by $\pi_1$ and $c \mapsto \langle c, \star\rangle$.

On the other hand,

$$\prod_{x:\,X} P(x, f(x)) \equiv \Big(\prod_{x:\,X} \mathsf{1}\Big) \equiv (X \to \mathsf{1}) \simeq \mathsf{1},$$

where the equivalence follows from part b) of Theorem 3.2.15 and Lemma 3.2.3 because $X \to \mathsf{1}$ is inhabited by $x \mapsto \star$.

Consequently, the total space inside the truncation on the RHS of (3.1) reduces to

$$\Big(\sum_{f:\,\prod_{x:\,X} C(x)} \prod_{x:\,X} P(x, f(x))\Big) \simeq \Big(\sum_{f:\,\prod_{x:\,X} C(x)} \mathsf{1}\Big) \simeq \Big(\prod_{x:\,X} C(x)\Big).$$

Because equivalences are preserved under truncation [cf. Lemma 3.3.2], the map provided by (3.1) constructs an inhabitant of (3.2). □

**Theorem 3.5.4.**[UA] *Let $P : \mathcal{U} \to \mathsf{Prop}_{\mathcal{U}}$ be a mere proposition family. Then, for any $u \equiv \langle A, p\rangle$ and $v \equiv \langle B, q\rangle$ in $\Sigma_{\mathcal{U}} P$, there is an equivalence:*

$$(u =_{\Sigma_{\mathcal{U}} P} v) \simeq (A \simeq B).$$

*Proof.* By Lemma 3.2.13, there is an equivalence:

$$(u =_{\Sigma_{\mathcal{U}} P} v) \simeq (A =_{\mathcal{U}} B).$$

The conclusion follows from the univalence axiom by composition of the former equivalence with:

$$(A =_{\mathcal{U}} B) \simeq (A \simeq B).$$

□

**Corollary 3.5.5.**[UA FE WE] *The following type is inhabited:*

$$\sum_{X : \mathcal{U}} \sum_{C : X \to \mathsf{Set}_{\mathcal{U}}} \left( \left( \prod_{x : X} \| C(x) \| \right) \to \left\| \prod_{x : X} C(x) \right\| \right) \to \mathsf{0}.$$

*In other words, there exists a type $X$ and a family $C : X \to \mathsf{Set}_{\mathcal{U}}$ such that (3.2) is false.*

*Proof.* Define

$$X :\equiv \sum_{A : \mathcal{U}} \| \mathsf{2} =_{\mathcal{U}} A \|.$$

Since $\mathsf{2}$ is a set [cf. Theorem 3.2.8], it follows by Lemma 3.3.4 that if $\langle A, p\rangle : X$, then $A$ must be a set as well.

By Theorem 3.5.4, for any $u :\equiv \langle A, p\rangle$ and $v :\equiv \langle B, q\rangle$ in $X$, there is an equivalence:

$$(u =_X v) \simeq (A \simeq B). \tag{3.4}$$

From (3.4), we infer using Corollary 3.2.19 that $A \simeq B$ is a set. Hence, $u =_X v$ is a set as well.

Let $x_0 :\equiv \langle \mathsf{2}, |\mathsf{refl}_{\mathsf{2}}| \rangle$. It follows that the family

$$\begin{aligned} C : X &\to \mathcal{U} \\ x &\mapsto x_0 =_X x, \end{aligned}$$

has the property that $C(x)$ is a set for all $x : X$.

In the particular case where $v :\equiv u :\equiv x_0$, (3.4) yields

$$(x_0 =_X x_0) \simeq (\mathsf{2} \simeq \mathsf{2}).$$

As Exercise 2.19.12 shows, $(\mathsf{2} \simeq \mathsf{2}) \simeq \mathsf{2}$. Using the lemma from that exercise, we deduce that $x_0 =_X x_0$ has two distinct inhabitants. Hence, $X$ is not a set. More precisely, there is a map $\mathsf{isSet}(X) \to \mathsf{0}$.

Suppose

$$f\colon \Big(\prod_{x\colon X} \|C(x)\|\Big) \to \Big\|\prod_{x\colon X} C(x)\Big\|.$$

Given

$$\zeta\colon \prod_{x\colon X} \|x_0 =_X x\|,$$

we obtain

$$f(\zeta)\colon \Big\|\prod_{x\colon X} x_0 =_X x\Big\|.$$

Since $\mathsf{isContr}(X)$ is a mere proposition by Theorem 3.4.5, using recursion on the RHS, to construct a map

$$\Big\|\prod_{x\colon X} x_0 =_X x\Big\| \to \mathsf{isContr}(X),$$

it suffices to provide a map

$$\iota\colon \Big(\prod_{x\colon X} x_0 =_X x\Big) \to \mathsf{isContr}(X),$$

which we can do by setting $\iota(g) :\equiv \langle x_0, g\rangle$.

Applying the recursion principle of the propositional truncation, we obtain an element of $\mathsf{isContr}(X)$. Since $\mathsf{isContr}(X)$ is a mere proposition, and every mere proposition is a set [cf. Proposition 3.2.9], by composition we obtain a witness

$$\epsilon\colon \Big\|\prod_{x\colon X} x_0 =_X x\Big\| \to \mathsf{0}.$$

Consequently, the map $\lambda f.\, f(\zeta) \circ \epsilon$ fulfills the requirement of the statement.

□

## 3.6 Hedberg's Theorem

**Definition 3.6.1.** A type family of mere propositions $R\colon X \times X \to \mathsf{Prop}_{\mathcal{U}}$ is *reflexive* if there is an inhabitant

$$\rho\colon \prod_{x\colon X} R(x,x). \tag{3.1}$$

It *implies identity* if there is a map

$$f\colon \prod_{x,y\colon X} R(x,y) \to (x =_X y). \tag{3.2}$$

**Theorem 3.6.2.** *Suppose that $R$ is a reflexive mere relation on a type $X$ implying identity. Then $X$ is a set.*

*Proof.* We must prove that for any $x, y\colon X$ and any paths $p, q\colon x =_X y$, we have $p =_{x=_X y} q$.

Let $\rho$ and $f$ as in (3.1) and (3.2).

By path induction on $q$, we may assume $y \equiv x$ and $q \equiv \mathsf{refl}_x$. Then the goal reduces to showing that for any $p\colon x =_X x$, we have $p =_{x=_X x} \mathsf{refl}_x$.

Consider the dependent function

$$f(x)\colon \prod_{y\colon X} (R(x,y) \to (x =_X y)).$$

According to Lemma 2.3.6, we have

$$\mathsf{apd}_{f(x)}(p)\colon p_*(f(x,x)) =_{R(x,x)\to(x=_X x)} f(x,x).$$

We can apply Theorem 2.11.15 by taking the type families $A(y) \equiv R(x,y)$ and $B(y) \equiv (x =_X y)$, with the functions $f$ and $g$ both being $f(x,x)$. The theorem establishes an equivalence between paths of the form

$$p_*(f(x,x)) =_{R(x,x)\to(x=_X x)} f(x,x)$$

and homotopies

$$p_* \circ f(x,x) \sim f(x,x) \circ p_*.$$

Therefore, the path $\mathsf{apd}_{f(x)}(p)$ corresponds under this equivalence to a homotopy. Evaluating this homotopy at any $r\colon R(x,x)$ yields the path

$$p_*(f(x,x,r)) =_{x=_X x} f(x,x,p_*(r)).$$

The LHS computes as follows

$$\begin{aligned}
\text{LHS} &\equiv p_*(f(x,x,r)) \\
&\equiv \mathsf{transport}^B(p, f(x,x,r)) \\
&= f(x,x,r) \cdot p && \text{; Prop. 2.11.6 a)}
\end{aligned}$$

The RHS is propositionally equal to $f(x,x,r)$ because both $r$ and

$$p_*(r) \equiv \mathsf{transport}^A(p, r)$$

inhabit $A(x) \equiv R(x,x)$, which is a mere proposition.

Substituting $r \equiv \rho(x)$, we obtain

$$f(x,x,\rho(x)) \cdot p =_{x=_X x} f(x,x,\rho(x)).$$

Hence, $p =_{x=_X x} \mathsf{refl}_x$, which completes the proof.

□

**Corollary 3.6.3.** *If a type $X$ satisfies $\neg\neg(x =_X y) \to (x =_X y)$ for any $x, y\colon X$, then $X$ is a set.*

*Proof.* Let $R(x, y) \equiv \neg\neg(x =_X y)$. Then $R(x, y)$ is a mere proposition by Theorem 3.2.15. Moreover $R$ is reflexive because $\mathsf{ev}_{\mathsf{refl}_x}\colon \neg(x =_X x) \to \mathsf{0}$. By hypothesis, $R$ implies identity. Thus, by Theorem 3.6.2, $X$ is a set. □

**Theorem 3.6.4** [Hedberg]. *If equality is decidable in $X$, then $X$ is a set.*

*Proof.* The hypothesis means that for all $x, y\colon X$ we have $(x =_X y)+\neg(x =_X y)$. By Lemma 3.1.7, we obtain $\neg\neg(x =_X y) \to (x =_X y)$. The result follows from Corollary 3.6.3.

□

## 3.7 Exercises

**Exercise 3.7.1.**[FE] *Prove that if $A \simeq B$ and $A$ is a set, then so is $B$.*

*Solution.* Let $\langle f, (\eta, \vartheta)\rangle\colon A \simeq B$. Take $x, y\colon B$. Given $p, q\colon x =_B y$, we have

$$\mathsf{ap}_{f^{-1}}(p), \mathsf{ap}_{f^{-1}}(q)\colon f^{-1}(x) =_A f^{-1}(y).$$

Since $A$ is a set, its identity types are mere propositions. Therefore,

$$\mathsf{ap}_{f^{-1}}(p) =_{f^{-1}(x)=_A f^{-1}(y)} \mathsf{ap}_{f^{-1}}(q).$$

Applying $\mathsf{ap}_f$ and using Proposition 2.1.1, we obtain

$$\mathsf{ap}_{f\circ f^{-1}}(p) =_{f(f^{-1}(x))=_B f(f^{-1}(y))} \mathsf{ap}_{f\circ f^{-1}}(q).$$

On the other hand, the naturality of the homotopy $\eta\colon f \circ f^{-1} \sim \mathsf{id}_B$ [cf. (2.2)] yields

$$\mathsf{ap}_{f\circ f^{-1}}(p) \cdot \eta_y = \eta_x \cdot \mathsf{ap}_{\mathsf{id}_B}(p) = \eta_x \cdot p, \quad \text{in } f(f^{-1}(x)) =_B y,$$

and similarly for $q$. By concatenating $\eta_y$ on the right it follows that

$$\eta_x \cdot p =_{f(f^{-1}(x))=_B y} \eta_x \cdot q.$$

Thus, concatenating with $\eta_x^{-1}$ on the left constructs the desired path

$$p =_{x=_B y} q.$$

□

**Exercise 3.7.2.** *Prove that if $A$ and $B$ are sets, then so is $A + B$.*

*Solution.* We have to construct a witness

$$\sigma\colon \prod_{u,v\colon A+B} \prod_{p,q\colon u=_{A+B} v} p =_{u=_{A+B} v} q.$$

Let $\sigma_A\colon \mathsf{isSet}(A)$ and $\sigma_B\colon \mathsf{isSet}(B)$. By double induction on $u$ and $v$, we first consider the case where both are in the left component. With the homotopy

$$\eta(p)\colon \mathsf{decode}(\mathsf{encode}(p)) =_{u=_{A+B} v} p$$

established by path induction, we define

$$\sigma(\mathsf{inl}(x), \mathsf{inl}(y))(p,q) :\equiv \eta(p)^{-1} \cdot \mathsf{ap}_{\mathsf{decode}}\big(\sigma_A(x,y)(\mathsf{encode}(p), \mathsf{encode}(q))\big) \cdot \eta(q),$$

where $\mathsf{encode}$ and $\mathsf{decode}$ are the functions defined in 2.15. Equality in Coproducts. This is well-defined because, in this case,

$$\mathsf{encode}(p), \mathsf{encode}(q)\colon x =_A y,$$

and so

$$\sigma_A(x,y)(\mathsf{encode}(p), \mathsf{encode}(q))\colon \mathsf{encode}(p) =_{x=_A y} \mathsf{encode}(q).$$

A similar construction yields

$$\sigma(\mathsf{inr}(x), \mathsf{inr}(y))(p,q) :\equiv \eta(p)^{-1} \cdot \mathsf{ap}_{\mathsf{decode}}\big(\sigma_B(x,y)(\mathsf{encode}(p), \mathsf{encode}(q))\big) \cdot \eta(q).$$

In the cases $u \equiv \mathsf{inl}(x)$ and $v \equiv \mathsf{inr}(y)$, or $u \equiv \mathsf{inr}(x)$ and $v \equiv \mathsf{inl}(y)$, considering that $\mathsf{encode}(p), \mathsf{encode}(q)\colon \mathsf{0}$, we simply define

$$\sigma(u,v)(p,q) :\equiv \mathsf{rec}_{\mathsf{0}}(p =_{u=_{A+B} v} q)(\mathsf{encode}(p)).$$

□

**Exercise 3.7.3.**[FE] *Show that $A$ is a mere proposition if, and only if, $A \to A$ is contractible.*

*Solution.* Suppose that $A$ is a mere proposition. By part b) of Theorem 3.2.15, the type $A \to A$ is also a mere proposition. Since $\mathsf{id}_A\colon A \to A$, it is inhabited. Therefore, it is contractible by Lemma 3.4.3.

Conversely, assume that $A \to A$ is contractible. Define

$$\begin{aligned} \iota\colon A &\to (A \to A) \\ x &\mapsto \lambda z.\, x. \end{aligned}$$

We have a witness

$$h\colon \prod_{a\colon A} \prod_{f\colon A \to A} \iota(a) =_{A \to A} f.$$

Then the dependent function

$$k\colon \prod_{a\colon A}\prod_{x\colon A} a =_A x$$
$$a \mapsto \lambda x.\, \mathsf{ap}_{\mathsf{ev}_a}(h(a, \iota(x)))$$

is well-defined because $h(a, \iota(x))\colon \iota(a) =_{A\to A} \iota(x)$ and so

$$\mathsf{ap}_{\mathsf{ev}_a}(h(a, \iota(x)))\colon a =_A x.$$

□

**Exercise 3.7.4.**[FE] *Show that* $\mathsf{isProp}(A) \simeq (A \to \mathsf{isContr}(A))$.

*Solution.* By Theorem 3.4.5, $\mathsf{isContr}(A)$ is a mere proposition. By part b) of Theorem 3.2.15, $A \to \mathsf{isContr}(A)$ is a mere proposition.

Consider

$$\mathsf{isProp}(A) \to (A \to \mathsf{isContr}(A))$$
$$u \mapsto \lambda a.\, \langle a, \lambda x.\, u(a, x)\rangle,$$

which is well-defined because $u(a, x)\colon a =_A x$ for all $a, x\colon A$.

On the other hand, we have

$$(A \to \mathsf{isContr}(A)) \to \mathsf{isProp}(A)$$
$$f \mapsto \lambda x, y.\, \pi_2(f(x))(x)^{-1} \cdot \pi_2(f(x))(y),$$

also well-defined because

$$f(x)\colon \sum_{a\colon A}\prod_{y\colon A} a =_A y$$

and so, $\pi_2(f(x))\colon \prod_{y\colon A} \pi_1(f(x)) =_A y$. Hence,

$$\pi_2(f(x))(x)^{-1} \cdot \pi_2(f(x))(y)\colon x =_A y.$$

The result follows from Lemma 3.2.4. □

**Exercise 3.7.5.**[FE] *Show that if $A$ is a mere proposition, then so is $A + \neg A$. In particular, in the context of* Notation 3.3.5*, there is no need to apply propositional truncation to define $A \vee \neg A :\equiv A + \neg A$.*

*Solution.* To apply part e) of Theorem 3.2.15, we must first ensure that both components of the coproduct are mere propositions. We are given that $A$ is a mere proposition. The second component is $\neg A \equiv A \to \mathsf{0}$. To see that this is a mere proposition, let $f, g\colon A \to \mathsf{0}$. For any $x\colon A$, the terms $f(x)$ and $g(x)$ belong

to $\mathsf{0}$. Since $\mathsf{0}$ is a mere proposition, we have a pointwise equality $f(x) =_{\mathsf{0}} g(x)$. By function extensionality, this pointwise equality yields a path $f =_{A\to\mathsf{0}} g$. Thus, $\neg A$ is a mere proposition.

Since both $A$ and $\neg A$ are mere propositions, it suffices to show that they are mutually exclusive. That is, we must provide a map

$$A \to ((A \to \mathsf{0}) \to \mathsf{0}).$$

Given $a\colon A$ and $f\colon A \to \mathsf{0}$, we define such a map by

$$\mathsf{ev}(a)(f) :\equiv f(a).$$

□

**Exercise 3.7.6.** *Assume that some type* $\mathsf{isequiv}(f)$ *satisfies that, for any two inhabitants* $e_1, e_2\colon \mathsf{isequiv}(f)$*, we have* $e_1 =_{\mathsf{isequiv}(f)} e_2$*. Show that the type* $\|\mathsf{qinv}(f)\|$ *satisfies the same condition and is equivalent to* $\mathsf{isequiv}(f)$*.*

*Solution.* The condition on $\mathsf{isequiv}(f)$ means that it is a mere proposition. By the universal property of propositional truncation, since $\mathsf{isequiv}(f)$ is a mere proposition, the map $\mathsf{qinv}(f) \to \mathsf{isequiv}(f)$ from Proposition 2.5.6 induces a map $\|\mathsf{qinv}(f)\| \to \mathsf{isequiv}(f)$.

On the other hand, the map $\mathsf{isequiv}(f) \to \mathsf{qinv}(f)$ from the same proposition, followed by the constructor $\mathsf{qinv}(f) \to \|\mathsf{qinv}(f)\|$, $e \mapsto |e|$, is a function $\mathsf{isequiv}(f) \to \|\mathsf{qinv}(f)\|$.

By Lemma 3.2.4, having functions in both directions between mere propositions establishes an equivalence, so we deduce that $\|\mathsf{qinv}(f)\| \simeq \mathsf{isequiv}(f)$. Moreover, $\|\mathsf{qinv}(f)\|$ trivially satisfies the condition because propositional truncations are mere propositions by definition. □

**Exercise 3.7.7.**[UA FE] *Show that if* LEM [Definition 3.2.10] *holds, then*

$$\Big(\sum_{A:\mathcal{U}} \mathsf{isProp}(A)\Big) \simeq 2.$$

*Solution.* IDEA: The $\Sigma$-type classifies mere propositions into two groups: the empty ones and the contractible ones.

Fix $A\colon \mathcal{U}$. By hypothesis, there exists a function $\lambda\colon$ LEM satisfying

$$\lambda(A)\colon \mathsf{isProp}(A) \to (A + \neg A).$$

Let $c_1\colon A \to 2$ and $c_0\colon \neg A \to 2$ be the constant maps taking values $1_2$ and $0_2$, respectively. These maps induce a function $c_A\colon A + \neg A \to 2$ via the coproduct

recursor. Then, we can define

$$f\colon \Big(\sum_{A:\mathcal{U}} \mathsf{isProp}(A)\Big) \to 2$$
$$\langle A, \epsilon\rangle \mapsto c_A(\lambda(A)(\epsilon)).$$

For the backward function, consider

$$\tau\colon 2 \to \sum_{A:\mathcal{U}} \mathsf{isProp}(A)$$
$$0_2 \mapsto \langle \mathsf{0}, \epsilon_0\rangle,$$
$$1_2 \mapsto \langle \mathsf{1}, \epsilon_1\rangle,$$

where $\epsilon_0\colon \mathsf{isProp}(\mathsf{0})$ and $\epsilon_1\colon \mathsf{isProp}(\mathsf{1})$ are the canonical dependent functions defined by induction on $\mathsf{0}$ and $\mathsf{1}$, respectively, as follows:

$$\epsilon_0 :\equiv \mathsf{ind}_{\mathsf{0}}\Big(\lambda x.\prod_{y:\mathsf{0}} x =_{\mathsf{0}} y\Big), \quad \epsilon_1 :\equiv \mathsf{ind}_{\mathsf{1}}\Big(\lambda x.\prod_{y:\mathsf{1}} x =_{\mathsf{1}} y, \mathsf{ind}_{\mathsf{1}}\Big(\lambda y.\star =_{\mathsf{1}} y, \mathsf{refl}_\star\Big)\Big).$$

Let us now evaluate both compositions. For $f\circ\tau$, we claim that $(f\circ\tau)(0_2) = 0_2$ and $(f\circ\tau)(1_2) = 1_2$. Since $\lambda$ is an abstract function, the term $\lambda(\mathsf{0})(\epsilon_0)$ does not compute definitionally. However, we can prove $c_{\mathsf{0}}(z) = 0_2$ for all $z\colon \mathsf{0}+\neg\mathsf{0}$ by induction on the coproduct. If $z \equiv \mathsf{inl}(x)$, the context is contradictory since $x\colon \mathsf{0}$. If $z \equiv \mathsf{inr}(y)$, then $c_{\mathsf{0}}(\mathsf{inr}(y)) \equiv c_{\mathsf{0}}(y) \equiv 0_2$. Thus,

$$(f\circ\tau)(0_2) \equiv f(\langle \mathsf{0}, \epsilon_0\rangle) \equiv c_{\mathsf{0}}(\lambda(\mathsf{0})(\epsilon_0)) = 0_2.$$

By a symmetric argument on $\mathsf{1}+\neg\mathsf{1}$, any left injection maps to $1_2$ definitionally, and the right injection is contradictory since $\neg\mathsf{1}$ is empty. Thus, $c_{\mathsf{1}}(z) = 1_2$ for all $z\colon \mathsf{1}+\neg\mathsf{1}$, which yields

$$(f\circ\tau)(1_2) \equiv f(\langle \mathsf{1}, \epsilon_1\rangle) \equiv c_{\mathsf{1}}(\lambda(\mathsf{1})(\epsilon_1)) = 1_2.$$

For the other composition, we need to prove that

$$\tau(f(\langle A, \epsilon\rangle)) \equiv \tau(c_A(\lambda(A)(\epsilon))) = \langle A, \epsilon\rangle.$$

Therefore, it suffices to prove that $\tau(c_A(z)) = \langle A, \epsilon\rangle$ for all $z\colon A+\neg A$. By induction on $A+\neg A$, we are reduced to the cases $z \equiv \mathsf{inl}(a)$ and $z \equiv \mathsf{inr}(\zeta)$, where $a\colon A$ and $\zeta\colon A\to\mathsf{0}$.

In the first case, the goal reduces to showing that $\langle \mathsf{1}, \epsilon_1\rangle = \langle A, \epsilon\rangle$ is inhabited. Since $\epsilon\colon \mathsf{isProp}(A)$, $A$ is a mere proposition. By Lemma 3.4.3, it is contractible because $a\colon A$. By the same lemma, it follows that $A \simeq \mathsf{1}$. Hence, the univalence axiom implies the existence of a path $p\colon \mathsf{1} =_{\mathcal{U}} A$. To prove the goal, we have to show that

$$\mathsf{transport}^{\mathsf{isProp}}(p, \epsilon_1) =_{\mathsf{isProp}(A)} \epsilon,$$

which holds trivially because, by Proposition 3.2.9, $\mathsf{isProp}(A)$ is a mere proposition.

In the second case, the goal reduces to proving that $\langle \mathsf{0}, \epsilon_0 \rangle = \langle A, \epsilon \rangle$.

To see that $\mathsf{0} =_{\mathcal{U}} A$, we need the following lemma:

**Lemma.** *If $A \to \mathsf{0}$ is inhabited, then $A =_{\mathcal{U}} \mathsf{0}$.*

PROOF: By univalence, it suffices to find an equivalence $A \simeq \mathsf{0}$.

Let $f : A \to \mathsf{0}$ be the function given by hypothesis. We claim that $g :\equiv \mathsf{rec}_{\mathsf{0}}(A)$ is a quasi-inverse of $f$.

To prove this, we must construct a right homotopy $\eta : f \circ g \sim \mathsf{id}_{\mathsf{0}}$ and a left homotopy $\vartheta : g \circ f \sim \mathsf{id}_A$.

By 43. COPRODUCT INDUCTION, for $\eta$ we have $\mathsf{ind}_{\mathsf{0}}(P)$, where $P : \mathsf{0} \to \mathcal{U}$ is defined as $P(y) :\equiv f(g(y)) =_{\mathsf{0}} y$. For $\vartheta$, fix $x : A$ and consider the type

$$Q_x :\equiv g(f(x)) =_A x.$$

This defines the function $\mathsf{rec}_{\mathsf{0}}(Q_x) : \mathsf{0} \to Q_x$. Applying it to $f(x)$ yields a witness for $g(f(x)) =_A x$.

Since we have maps in both directions and their corresponding homotopies, we conclude that $A \simeq \mathsf{0}$. The result follows from the univalence axiom. ■

It remains to be shown that $\mathsf{transport}^{\mathsf{isProp}}(p, \epsilon_0) =_{\mathsf{isProp}(A)} \epsilon$, where $p : \mathsf{0} =_{\mathcal{U}} A$. Again, this is trivial because $\mathsf{isProp}(A)$ is a mere proposition. □

**Exercise 3.7.8.**[UA FE] *Show that if $\mathcal{U}_{i+1}$ satisfies* LEM*, then the canonical inclusion $\mathsf{Prop}_{\mathcal{U}_i} \to \mathsf{Prop}_{\mathcal{U}_{i+1}}$ is an equivalence.*

*Solution.* Let us recall that, by definition,

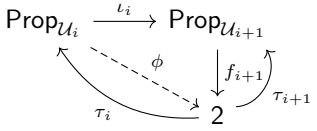
$$\mathsf{Prop}_{\mathcal{U}} \equiv \sum_{A:\mathcal{U}} \mathsf{isProp}(A),$$

and that the canonical inclusion is

$$\begin{aligned} \iota_i : \mathsf{Prop}_{\mathcal{U}_i} &\to \mathsf{Prop}_{\mathcal{U}_{i+1}} \\ \langle A, \epsilon \rangle &\mapsto \langle A, \epsilon \rangle, \end{aligned}$$

where $A$ is considered as a type in $\mathcal{U}_{i+1}$ on the RHS, by THE CUMULATIVE PROPERTY.

By Exercise 3.7.7, the hypothesis means that $\mathsf{Prop}_{\mathcal{U}_{i+1}} \simeq \mathsf{2}$. In addition, the triangle



where $\tau_i$, $\tau_{i+1}$, and $f_{i+1}$ are the maps constructed in that exercise, defines by composition a function $\phi\colon \mathsf{Prop}_{\mathcal{U}_i} \to 2$.

Moreover, using the notation of Exercise 3.7.7, we have

$$\begin{aligned}(\phi \circ \tau_i)(0_2) &\equiv f_{i+1}(\iota_i(\langle 0, \epsilon_0\rangle))\\ &= f_{i+1}(\tau_{i+1}(0_2))\\ &\equiv 0_2,\end{aligned}$$

and similarly for $1_2$. Hence, by induction on $2$, we deduce that $\phi \circ \tau_i =_{2\to 2} \mathsf{id}_2$.

For the other composition,

$$\begin{aligned}(\tau_i \circ \phi)(\langle A, \epsilon\rangle) &\equiv \tau_i(f_{i+1}(\iota_i(\langle A, \epsilon\rangle)))\\ &= \tau_i(f_{i+1}(\langle A, \epsilon\rangle))\\ &\equiv \tau_i(c_A(\ell(A)(\epsilon))),\end{aligned}$$

where $\ell$ is the witness of $\textsc{lem}\colon \mathcal{U}_{i+1}$ given by hypothesis, and $c_A\colon (A + \neg A) \to 2$ is the map of the previous exercise.

We claim that $\tau_i(c_A(z)) = \langle A, \epsilon\rangle$ for all $A\colon \mathcal{U}_i$ and $z\colon A + \neg A$. But this was proved in the previous exercise by induction on $z$, using the fact that $\mathsf{isProp}(A)$ is a mere proposition (and not that $\textsc{lem}\colon \mathcal{U}_i$ was inhabited).

Therefore, $\tau_i$ and $\phi$ are quasi-inverses, which implies that $\iota_i$ induces an equivalence.

□

**Exercise 3.7.9.**[UA] *Prove that, in general, there is no map $\|A\| \to A$ for an arbitrary type $A\colon \mathcal{U}$. (Note, however, that such maps do exist for specific types; for example,* Exercise 3.7.6 *implies that $\mathsf{qinv}(f)$ is one such type.)*

*Solution.* Note that if $A$ has a witness $a\colon A$, then $\|A\|$ is an inhabited mere proposition, hence equivalent to $1$. Thus, we can define a map $\|A\| \to A$ as the composition $\|A\| \to 1 \to A$, where the second map is defined by mapping $\star\colon 1$ to $a$. What the exercise emphasizes is the nonexistence of a general function that would identify a witness for each inhabited type.

To solve the exercise, we have to define a map

$$\epsilon\colon \Big(\prod_{X:\mathcal{U}} \|X\| \to X\Big) \to 0.$$

Take $u$ in the domain. Then $u(2)\colon \|2\| \to 2$.

Suppose that $p\colon A =_{\mathcal{U}} B$ is a path between two types $A$ and $B$. Then

$$\mathsf{apd}_u(p)\colon \mathsf{transport}^P(p, u(A)) =_{\|B\|\to B} u(B),$$

where $P\colon \mathcal{U} \to \mathcal{U}$ is defined as $P(X) :\equiv \|X\| \to X$. Applying Proposition 2.11.5 to the case $(A \to B) \equiv (\|-\| \to \mathsf{id}_{\mathcal{U}})$ and $f \equiv u(A)$, we obtain

$$\mathsf{transport}^{P}(p, u(A)) =_{\|B\| \to B} \mathsf{transport}^{\mathsf{id}_{\mathcal{U}}}(p) \circ u(A) \circ \mathsf{transport}^{\|-\|}(p^{-1}).$$

By Theorem 2.13.1, the first transport on the RHS induces the equivalence $\mathsf{idtoeqv}(p)\colon A \simeq B$. The second transport is an equivalence between $\|B\|$ and $\|A\|$.

In particular, if $p \equiv \mathsf{ua}(e)$ for some equivalence $e\colon A \simeq B$, then $\mathsf{idtoeqv}(p) \equiv e$. Applying $\mathsf{happly}$ to the path $\mathsf{apd}_u(p)$ yields, for any $y\colon \|B\|$, a pointwise path

$$q_y\colon e\big(u(A)(\|e\|^{-1}(y))\big) =_B u(B)(y).$$

Specializing this to the case where $B :\equiv A :\equiv 2$, $e \equiv \mathsf{not}$, and $y \equiv |1_2|$, we obtain a path

$$q_{|1_2|}\colon \mathsf{not}\big(u(2)(\|\mathsf{not}\|^{-1}(|1_2|))\big) =_2 u(2)(|1_2|).$$

Since $\|2\|$ is a mere proposition, there is a path $s\colon \|\mathsf{not}\|^{-1}(|1_2|) =_{\|2\|} |1_2|$. Applying the function $\lambda z.\, \mathsf{not}(u(2)(z))$ to $s$ yields a path

$$s'\colon \mathsf{not}\big(u(2)(\|\mathsf{not}\|^{-1}(|1_2|))\big) =_2 \mathsf{not}(u(2)(|1_2|)).$$

Concatenating $(s')^{-1}$ with $q_{|1_2|}$, we obtain a path

$$r\colon \mathsf{not}(u(2)(|1_2|)) =_2 u(2)(|1_2|).$$

Thus, we can define

$$\epsilon(u) :\equiv \mathsf{notid}(u(2)(|1_2|), r),$$

where $\mathsf{notid}$ is the function defined in Remark 3.1.9. □

**Exercise 3.7.10.** *Show that if* LEM *holds, then for all* $A\colon \mathcal{U}$ *we have*

$$\big\|\|A\| \to A\big\|.$$

*Solution.* We have to construct a dependent function

$$\epsilon\colon \prod_{A:\mathcal{U}} \|P(A)\|,$$

where $P(A) :\equiv \|A\| \to A$.

By hypothesis, we have a witness of LEM, say

$$\ell\colon \prod_{A:\mathcal{U}} \mathsf{isProp}(A) \to (A + \neg A).$$

Fix $A\colon \mathcal{U}$. We have

$$\ell(\|A\|)(\mathsf{squash})\colon \|A\| + \neg\|A\|.$$

To define a map from $\|A\| + \neg\|A\|$ to $\|P(A)\|$, we have to specify two maps: $\alpha\colon \|A\| \to \|P(A)\|$ and $\beta\colon \neg\|A\| \to \|P(A)\|$.

To define $\alpha$ we use recursion:

$$\alpha :\equiv \mathsf{rec}_{\|A\|}(\|P(A)\|, a \mapsto |z \mapsto a|).$$

For $\beta$, take $u\colon \|A\| \to \mathsf{0}$. Then,

$$\beta(u) :\equiv |z \mapsto \mathsf{rec}_{\mathsf{0}}(A)(u(z))|.$$

Since $\alpha$ and $\beta$ are well defined, they induce a map

$$\gamma(A)\colon (\|A\| + \neg\|A\|) \to \|P(A)\|.$$

To conclude, we define

$$\epsilon(A) :\equiv \gamma(A)(\ell(\|A\|)(\mathsf{squash})).$$

**Note.** Saying that $\|P(A)\|$ is inhabited means that there is some witness of $P(A)$, i.e., some way to choose an element of $A$, provided that $A$ is inhabited. Since HoTT requires that every element be constructed, this claim is not generally true.

□

**Exercise 3.7.11.**[FE] Corollary 3.1.10 *shows that the following naive form of* LEM *is inconsistent with univalence:*

$$\prod_{A:\mathcal{U}} A + \neg A.$$

*Assuming this axiom (which is known to be consistent in the absence of univalence), show that it implies the axiom of choice.*

*Solution.* Let $X$, $A$, and $P$ be as in Definition 3.5.2.

According to Lemma 3.5.3, given a set $X$ and a family of sets $C\colon X \to \mathcal{U}$, we have to construct a witness

$$\epsilon\colon \Big(\prod_{x:X} \|C(x)\|\Big) \to \Big\|\prod_{x:X} C(x)\Big\|.$$

Let $\gamma$ be a witness of the axiom and let $f$ be a term of the domain type for $\epsilon$.

Fix $x\colon X$. By hypothesis, there is a witness $\gamma(C(x))\colon C(x) + \neg C(x)$.

Since $f(x)\colon \|C(x)\|$, we can define

$$\begin{aligned}\zeta\colon \neg C(x) &\to C(x)\\ \beta &\mapsto \mathsf{rec}_{\mathsf{0}}\big(C(x), \tilde{\beta}(f(x))\big),\end{aligned}$$

where $\tilde{\beta} :\equiv \mathsf{rec}_{\|C(x)\|}(0, \beta)$.

By induction on the coproduct, $\mathsf{id}_{C(x)}$ and $\zeta$ construct a function

$$v\colon (C(x) + \neg C(x)) \to C(x).$$

Thus, the evaluation

$$v(\gamma(C(x)))$$

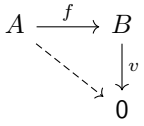
inhabits $C(x)$. Hence, we can define $\epsilon :\equiv \|\lambda x.\, v(\gamma(C(x)))\|$.

□

**Exercise 3.7.12.** *Show that assuming* LEM*, the double negation $\neg\neg A$ has the same recursion principle as the propositional truncation $\|A\|$ but with a propositional computation rule rather than a judgmental one. In other words, prove that assuming* LEM*, if $B$ is a mere proposition and we have $f\colon A \to B$, then there is an induced $g\colon \neg\neg A \to B$ such that $g(\lambda u.\, u(a)) =_B f(a)$ for all $a\colon A$. Deduce that, assuming* LEM*, we have $\neg\neg A \simeq \|A\|$. Thus, under* LEM*, the propositional truncation can be defined rather than taken as a separate type former.*

*Solution.* As we saw in Exercise 1.15.8, given $a\colon A$, the evaluation at $a$, defined by $\mathrm{ev}_a :\equiv \lambda u\colon \neg A.\, u(a)$, is a term of $\neg\neg A$.

As the following diagram shows, $f\colon A \to B$ induces a map $\neg f\colon \neg B \to \neg A$ defined by $v \mapsto v \circ f$.



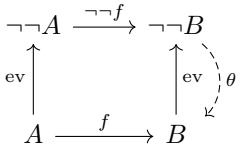

Applying this once again yields a map

$$\begin{aligned} \neg\neg f\colon \neg\neg A &\to \neg\neg B \\ \nu &\mapsto \nu \circ \neg f. \end{aligned}$$

Note moreover that

$$\begin{aligned} (\neg\neg f(\mathrm{ev}_a))(v) &\equiv \mathrm{ev}_a(\neg f(v)) \\ &\equiv \mathrm{ev}_a(v \circ f) \\ &\equiv (v \circ f)(a) \\ &\equiv \mathrm{ev}_{f(a)}(v), \end{aligned}$$

i.e., $\neg\neg f \circ \mathrm{ev} \equiv \mathrm{ev} \circ f$, which means that the following diagram commutes definitionally:



Thus, to obtain $g\colon \neg\neg A \to B$, it suffices to construct $\theta\colon \neg\neg B \to B$.

By Lemma 3.1.7, there is a map

$$\varphi\colon (B + \neg B) \to (\neg\neg B \to B).$$

Therefore, given witnesses $\ell\colon \mathrm{LEM}$ and $\pi\colon \mathsf{isProp}(B)$, we obtain a witness

$$\ell(\pi)\colon B + \neg B.$$

Applying $\varphi$, we arrive at the desired map $\theta$:

$$\theta :\equiv \varphi(\ell(\pi)).$$

Hence, we can define $g :\equiv \theta \circ \neg\neg f$.

To verify that $g(\mathrm{ev}_a) =_B f(a)$, i.e., that the square above commutes for $\theta$, recall that $B$ is a mere proposition and $\pi(g(\mathrm{ev}_a), f(a))$ provides the required path.

Note that since $\neg\neg X \equiv (\neg X \to \mathsf{0})$, both $\neg\neg A$ and $\neg\neg B$ are mere propositions. In particular, ev and $\theta$ are quasi-inverses of each other by Lemma 3.2.4.

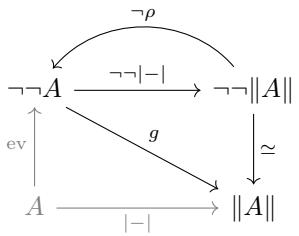
In the case where $B :\equiv \|A\|$ and $f$ is the canonical constructor $a \mapsto |a|$, the map $g\colon \neg\neg A \to \|A\|$ satisfies $g(\mathrm{ev}_a) =_{\|A\|} |a|$ for all $a\colon A$.

Take an element $u\colon \neg A$. Since $u\colon A \to \mathsf{0}$ and $\mathsf{0}$ is a mere proposition, we obtain an element $\mathsf{rec}_{\|A\|}(\mathsf{0}, u)\colon \|A\| \to \mathsf{0}$. This defines a map

$$\begin{aligned} \rho\colon \neg A &\to \neg\|A\| \\ u &\mapsto \mathsf{rec}_{\|A\|}(\mathsf{0}, u), \end{aligned}$$

which, in turn, yields a map $\neg\rho\colon \neg\neg\|A\| \to \neg\neg A$, as we saw above.

Summarizing in a diagram, we have




Now Lemma 3.2.4 implies that the top horizontal functions are mutual quasi-inverses. In particular, $g$ induces the desired equivalence $\neg\neg A \simeq \|A\|$.

□

**Exercise 3.7.13.**[FE]

a) *Show that for any $A : \mathcal{U}$, the type*

$$\prod_{P : \mathsf{Prop}_{\mathcal{U}}} (A \to P) \to P$$

*has the same recursion principle as $\|A\|$, at least relative to propositions in $\mathcal{U}$, with the same judgmental computation rule.*

b) *The type considered in the previous part does not lie in the same universe $\mathcal{U}$, and its recursion principle only applies to mere propositions in $\mathcal{U}$. Show that if we assume propositional resizing, we can define a type that does lie in the same universe $\mathcal{U}$ and satisfies the same recursion principle as $\|A\|$, albeit with only a propositional computation rule. Thus, we can also define the propositional truncation in this case.*

*Solution.*

a) Let $\Pi$ denote the $\Pi$-type defined in the statement. We have to prove that $(\Pi \to Q) \simeq (A \to Q)$ for all mere propositions $Q : \mathcal{U}$.

Fix a mere proposition $Q$. Define the forward map as

$$\begin{aligned} \varphi : (\Pi \to Q) &\to (A \to Q) \\ \alpha &\mapsto \lambda a.\, \alpha(\lambda P.\, \mathrm{ev}_a^P), \end{aligned}$$

where $\mathrm{ev}_a^P : (A \to P) \to P$ is the evaluation at $a$. This is well-defined because $(\lambda P.\, \mathrm{ev}_a^P) : \Pi$.

For the backward map, consider

$$\begin{aligned} \psi : (A \to Q) &\to (\Pi \to Q) \\ f &\mapsto \lambda \pi.\, \pi(Q)(f). \end{aligned}$$

Since $\pi(Q) : (A \to Q) \to Q$, its value at $f$ inhabits $Q$.

By part b) of Theorem 3.2.15, since $Q$ is a mere proposition, the function types $\Pi \to Q$ and $A \to Q$ are also mere propositions. Given that we have constructed maps in both directions, the equivalence follows directly from Lemma 3.2.4.

It remains to be shown that the recursion principle of $\Pi$ evaluates judgmentally in the same way as that of the truncation.

Before verifying this property, let us emphasize that, since $\Pi$ quantifies over all mere propositions of $\mathcal{U}$, it cannot belong to $\mathcal{U}$ and becomes an element of a universe above it. This differs from the type former $\|-\|$, which produces a type $\|A\|$ in the same universe as $A$.

To check the recursion principle of $\Pi$, let us define

$$\begin{aligned} \mathrm{ev} : A &\to \Pi \\ a &\mapsto \lambda P.\, \mathrm{ev}_a^P. \end{aligned}$$

Given $f\colon A \to Q$, the function $\psi(f)$ makes the following diagram judgmentally commutative:

$$\begin{array}{ccc} A & \xrightarrow{f} & Q \\ {\scriptstyle \mathrm{ev}}\downarrow & \nearrow_{\psi(f)} & \\ \Pi & & \end{array} \qquad \begin{aligned} \psi(f) \circ \mathrm{ev}(a) &\equiv (\lambda\pi.\, \pi(Q)(f))(\lambda P.\, \mathrm{ev}_a^P) \\ &\equiv (\lambda P.\, \mathrm{ev}_a^P)(Q)(f) \\ &\equiv \mathrm{ev}_a^Q(f) \\ &\equiv f(a). \end{aligned}$$

b) To better understand this part, we need to introduce the following axiom.

**Axiom** [Propositional resizing] The canonical map $\iota_i\colon \mathsf{Prop}_{\mathcal{U}_i} \to \mathsf{Prop}_{\mathcal{U}_{i+1}}$ is an equivalence.

Using the notation of Exercise 3.7.8, note that if $\iota_i^{-1}$ is a quasi-inverse of $\iota_i$, then given $Q'\colon \mathcal{U}_{i+1}$, the homotopy between $\iota_i \circ \iota_i^{-1}$ and the identity implies that the type $Q :\equiv \iota_i^{-1}(Q')$ satisfies $\iota_i(Q) =_{\mathcal{U}_{i+1}} Q'$. In particular, $\iota_i(Q) \simeq Q'$.

It is also worth noting that the universe level of a dependent product $\prod_{x\colon X} Y(x)$ is the maximum of the universe levels of its domain $X$ and its family of codomains $Y(x)$.

Let $\mathcal{U} \equiv \mathcal{U}_i$ and define $\mathcal{U}' :\equiv \mathcal{U}_{i+1}$. In the case of $\Pi$, since $(A \to P) \to P\colon \mathcal{U}$ for all $P\colon \mathsf{Prop}_{\mathcal{U}}$, and the domain $\mathsf{Prop}_{\mathcal{U}}\colon \mathcal{U}'$, we deduce that $\Pi\colon \mathcal{U}'$.

By part a) of Theorem 3.2.15, a product of mere propositions is a mere proposition, so $\Pi$ is a mere proposition. This means that it represents an element of $\mathsf{Prop}_{\mathcal{U}'}$.

If propositional resizing is assumed, the equivalence $\mathsf{Prop}_{\mathcal{U}} \simeq \mathsf{Prop}_{\mathcal{U}'}$ provides a mere proposition $\Pi'\colon \mathsf{Prop}_{\mathcal{U}}$ that is equivalent to $\Pi$. Its underlying type belongs to $\mathcal{U}$. Because $\Pi'$ and $\Pi$ are equivalent, maps out of $\Pi'$ into any proposition $Q\colon \mathcal{U}$ are equivalent to maps out of $\Pi$, meaning $\Pi'$ satisfies the same recursion principle.

Because the recursion for $\Pi'$ is inherited from its equivalence with $\Pi$, its computation rule only holds propositionally.

□

**Exercise 3.7.14.**[FE] *Assuming* LEM*, show that double negation commutes with universal quantification of mere propositions over sets. That is, show that if $X$ is a set and each $C(x)$ is a mere proposition, then* LEM *implies*

$$\Big(\prod_{x\colon X} \neg\neg C(x)\Big) \simeq \neg\neg \prod_{x\colon X} C(x). \tag{†}$$

*Observe that if we assume instead that each $C(x)$ is a set, then* (†) *becomes equivalent to the axiom of choice.*

*Proof.* Let $\ell\colon \textsc{lem}$ be a witness of the least excluded middle type. By definition, given a type $A\colon \mathcal{U}$, we have

$$\ell(A)\colon \mathsf{isProp}(A) \to (A + \neg A).$$

Applying this to $C(x)$, we obtain a witness

$$\ell(C(x))(\zeta(x))\colon C(x) + \neg C(x),$$

where $\zeta(x)$ inhabits $\mathsf{isProp}(C(x))$. Applying the map

$$(C(x) + \neg C(x)) \to (\neg\neg C(x) \to C(x))$$

from Lemma 3.1.7 yields a map

$$e(x)\colon \neg\neg C(x) \to C(x).$$

To define a forward function for (†), fix an element $f$ of the LHS. For $x\colon X$, we have

$$f(x)\colon \neg\neg C(x).$$

Then, to define a map

$$\varphi(f)\colon \Big(\neg \prod_{x\colon X} C(x)\Big) \to \mathsf{0},$$

for $u\colon \Big(\prod_{x\colon X} C(x)\Big) \to \mathsf{0}$, we specify

$$\varphi(f)(u) :\equiv u\big(\lambda x.\, e(x)(f(x))\big),$$

which is well-typed because $e(x)(f(x))\colon C(x)$.

Let $\Pi$ be the dependent product on the RHS of (†). By part a) of Theorem 3.2.15, we know that $\Pi$ is a mere proposition. Hence,

$$\ell(\Pi)(\zeta)\colon \Pi + \neg\Pi,$$

where $\zeta$ is an inhabitant of $\mathsf{isProp}(\Pi)$. Applying the map

$$(\Pi + \neg\Pi) \to (\neg\neg\Pi \to \Pi)$$

from Lemma 3.1.7 yields a map

$$e\colon \Big(\neg\neg \prod_{x\colon X} C(x)\Big) \to \prod_{x\colon X} C(x).$$

Thus, the canonical map $\mathrm{ev}\colon C(x) \to \neg\neg C(x)$ [cf. Exercise 3.7.12] yields a map

$$\psi\colon \Big(\neg\neg \prod_{x\colon X} C(x)\Big) \to \prod_{x\colon X} \neg\neg C(x).$$

Given that both sides of (†) are mere propositions, their equivalence follows from Lemma 3.2.4.

As proved in Exercise 3.7.12, double negation and truncation are equivalent if LEM is assumed. Therefore, (†) is equivalent to

$$\Big(\prod_{x:\,X} \|C(x)\|\Big) \simeq \Big\|\prod_{x:\,X} C(x)\Big\|.$$

In the case where each $C(x)$ is a set for $x\colon X$, since the canonical backward map

$$\Big\|\prod_{x:\,X} C(x)\Big\| \to \prod_{x:\,X} \|C(x)\|$$

always exists (even if the types $C(x)$ are not sets), the equivalence (†) reduces to the existence of a forward map. By Lemma 3.5.3, the existence of such a map is equivalent to the axiom of choice.

□

**Exercise 3.7.15.** *Show that, in the case of propositional truncation, the existence of* $\mathsf{rec}_{\|A\|}$ *for* $A\colon \mathcal{U}$*, implies the existence of* $\mathsf{ind}_{\|A\|}$.

*Solution.* Take $A\colon \mathcal{U}$. We must construct the function $\mathsf{ind}_{\|A\|}$ of (3.1).

Let $C\colon \|A\| \to \mathsf{Prop}_{\mathcal{U}}$ be a predicate. Given a dependent map

$$f\colon \prod_{a:\,A} C(|a|)$$

and an element $z\colon \|A\|$, we need to specify an inhabitant $g(z)\colon C(z)$.

For any $a\colon A$, there is a path $p_z\colon |a| =_{\|A\|} z$. Hence, we have the transport map

$$(p_z)_* :\equiv \mathsf{transport}^C(p_z)\colon C(|a|) \to C(z).$$

In particular, $(p_z)_*(f(a))\colon C(z)$. This yields a map

$$\begin{aligned} v\colon A &\to C(z) \\ a &\mapsto (p_z)_*(f(a)). \end{aligned}$$

Thus, the recursion principle provides

$$\rho :\equiv \mathsf{rec}_{\|A\|}(C(z), v)\colon \|A\| \to C(z),$$

and we can define $g(z) :\equiv \rho(|a|)$.

□

**Exercise 3.7.16.**[FE] *Show that the law of excluded middle* [cf. Definition 3.2.10] *and the law of double negation, which states that*

$$\prod_{A:\mathcal{U}} \mathsf{isProp}(A) \to (\neg\neg A \to A), \tag{3.1}$$

*are logically equivalent.*

Hint: To derive LEM use the hypothesis on $A + \neg A$.

*Solution.* If $\ell\colon$ LEM, given $A\colon \mathcal{U}$, we have

$$\ell(A)\colon \mathsf{isProp}(A) \to (A + \neg A).$$

Following this with the map $(A + \neg A) \to (\neg\neg A \to A)$ of Lemma 3.1.7 yields a map from LEM to the type of (3.1).

Conversely, given an inhabitant $d$ of (3.1), we must use the function

$$h(A)\colon \mathsf{isProp}(A) \to (\neg\neg A \to A)$$

to construct a map

$$\mathsf{isProp}(A) \to (A + \neg A).$$

For any $\epsilon\colon \mathsf{isProp}(A)$, we have $h(A)(\epsilon)\colon \neg\neg A \to A$. By Exercise 3.7.5, the type $A + \neg A$ is a mere proposition. Therefore, applying the hypothesis $h$ to $A + \neg A$ yields a map

$$h(A + \neg A)\colon \neg\neg(A + \neg A) \to (A + \neg A).$$

For any $u\colon (A + \neg A) \to \mathsf{0}$, define

$$\begin{aligned}&\rho(u)\colon \neg(A + \neg A) \to \mathsf{0}\\ &\rho(u) :\equiv \lambda v\colon (A + \neg A) \to \mathsf{0}.\, \mathsf{rec}_{A+\neg A}(\mathsf{0}, v \circ \mathsf{inl}, u \circ \mathsf{inr}).\end{aligned}$$

Setting $\rho :\equiv \lambda u.\, \rho(u)$, we obtain a term $\rho\colon \neg\neg(A + \neg A)$, which gives us the element $h(A + \neg A)(\rho)\colon A + \neg A$.

□

**Exercise 3.7.17.**[FE] *Suppose $P\colon \mathbb{N} \to \mathcal{U}$ is a decidable family*[6] *of mere propositions. Prove that*

$$\left\| \sum_{n:\mathbb{N}} P(n) \right\| \to \sum_{n:\mathbb{N}} P(n).$$

Hint: If $F\colon \mathbb{N} \to \mathcal{U}$ is a mere proposition family such that $F(n) =_{\mathcal{U}} F(m)$ only when $n =_{\mathbb{N}} m$, then $\Sigma_{\mathbb{N}} F$ is a mere proposition.

[6] Definition 3.2.10.

*Solution.* By hypothesis, there exists

$$\epsilon\colon \prod_{n\colon \mathbb{N}} P(n) + \neg P(n).$$

The Π-type

$$Q(n) :\equiv \prod_{k\colon \mathbb{N}} (k < n) \to \neg P(k)$$

is a mere proposition by parts b) and a) of Theorem 3.2.15. The dependent type

$$F(n) :\equiv Q(n) \times P(n),$$

which is also a mere proposition by the same theorem, is inhabited when $n$ is the first natural number such that $P(n)$ is true.

Now consider

$$S :\equiv \sum_{n\colon \mathbb{N}} F(n).$$

We claim that $S$ is a mere proposition. Intuitively, it is clear that $S$ can have at most one inhabitant. Take two elements $\langle n, f\rangle$ and $\langle k, g\rangle$ of $S$. By Lemma 3.2.13, to show that they are equal it suffices to show that $n =_{\mathbb{N}} k$. If $u\colon k < n$, then

$$\pi_1(f)(u)\colon \neg P(k).$$

However, $\pi_2(g)\colon P(k)$, which yields $\pi_1(f)(u)(\pi_2(g))\colon \mathsf{0}$.

By symmetry, $n < k$ also yields a contradiction. Hence, by part b) of Proposition 3.2.11, we deduce that $n =_{\mathbb{N}} k$, proving our claim that $S$ is a mere proposition.

Consequently, by recursion on the truncated type, to define a map $\|\Sigma_{\mathbb{N}} P\| \to S$, it suffices to define a map $\Sigma_{\mathbb{N}} P \to S$. And since the second projection induces a map from $S$ to $\Sigma_{\mathbb{N}} P$, the solution would be complete if we define said map.

The induction principle of Σ-types reduces our goal further to defining a map $P(n) \to S$ for $n\colon \mathbb{N}$.

The idea is to construct the desired maps by induction on $n$. However, to enable that, we need an additional property, which we will also prove by induction, namely, that the coproduct

$$S + Q(n)$$

is inhabited for all $n\colon \mathbb{N}$.

In the case $n \equiv 0$, by part g) of Lemma 1.12.4, for each $k\colon \mathbb{N}$, there is a witness $\zeta_k\colon (k < 0) \to \mathsf{0}$. Therefore, the map $\lambda k.\, \lambda u.\, \mathsf{rec}_{\mathsf{0}}(\neg P(k), \zeta_k(u))$ inhabits $Q(0)$.

In the inductive step, suppose that we have a witness $h\colon S + Q(n)$. To construct a witness of $S + Q(\mathsf{succ}(n))$ it suffices to define a map from the former type to the latter. We proceed by recursion on the coproduct:

- For the left branch $S \to S + Q(\mathsf{succ}(n))$, we use the canonical injection $\mathsf{inl}$.
- For the right branch, given $q\colon Q(n)$ we define a function

$$\sigma_n\colon (P(n) + \neg P(n)) \to (S + Q(\mathsf{succ}(n)))$$

using recursion once again:

For the left sub-branch, we define

$$\begin{aligned} P(n) &\to S + Q(\mathsf{succ}(n)) \\ u &\mapsto \mathsf{inl}(\langle n, (q, u)\rangle), \end{aligned}$$

which is well-defined because $(q, u)\colon Q(n) \times P(n)$, the type definitionally equal to $F(n)$, and so $\langle n, (q, u)\rangle\colon S$.

For the right sub-branch, if $z\colon \neg P(n)$, using recursion for the third time, we define a map $v_z\colon (k < n) + (k =_{\mathbb{N}} n) \to \neg P(k)$ via the functions:

$$\begin{aligned} q(k)&\colon (k < n) \to \neg P(k), \\ \zeta_z&\colon (k =_{\mathbb{N}} n) \to \neg P(k), \quad p \mapsto \mathsf{transport}^{\neg P}(p^{-1}, z). \end{aligned}$$

Thus, precomposing $v_z$ with the canonical map[7]

$$(k < \mathsf{succ}(n)) \to (k < n) + (k =_{\mathbb{N}} n),$$

we obtain a function $k \mapsto ((k < \mathsf{succ}(n)) \to \neg P(k))$, which is exactly an element of $Q(\mathsf{succ}(n))$. Composing this with

$$\mathsf{inr}\colon Q(\mathsf{succ}(n)) \to S + Q(\mathsf{succ}(n))$$

yields the desired function with domain $\neg P(n)$.

This finishes the construction of $\sigma_n$, which proves inductively that $S + Q(n)$ is inhabited for all $n\colon \mathbb{N}$.

Now we are in a position to construct the desired maps $P(n) \to S$. Indeed, given $u\colon P(n)$, the functions

$$\begin{aligned} \mathsf{id}_S&\colon S \to S \\ \gamma_{u,n}&\colon Q(n) \to S, \quad q \mapsto \langle n, (q, u)\rangle, \end{aligned}$$

define a map $\mathsf{rec}_{S+Q(n)}(S, \mathsf{id}_S, \gamma_{u,n})\colon S + Q(n) \to S$. Thus, we can define

$$\begin{aligned} P(n) &\to S \\ u &\mapsto \mathsf{rec}_{S+Q(n)}(S, \mathsf{id}_S, \gamma_{u,n})(\tau_n), \end{aligned}$$

where $\tau_n\colon S + Q(n)$ is the witness obtained from our previous induction.

□

[7] This is easily deduced from part c) of Proposition 3.2.11.

**Exercise 3.7.18.**[FE] *Prove that* $\mathsf{isProp}(P) \simeq (P \simeq \|P\|)$.

*Solution.* As shown in Lemma 3.3.3, if $P$ is a mere proposition, we have a map $\mathsf{rec}_{\|P\|}(P, \mathsf{id}_P) \colon \|P\| \to P$. Since we also have the canonical injection $P \to \|P\|$, by Lemma 3.2.4, we deduce that there is a map $\mathsf{isProp}(P) \to (P \simeq \|P\|)$.

By Proposition 3.2.9 and part f) of Theorem 3.2.15, both $\mathsf{isProp}(P)$ and $P \simeq \|P\|$ are mere propositions. Consequently, by Lemma 3.2.4, to prove their equivalence it suffices to construct a backward map.

Let $(f, \eta, \vartheta) \colon P \simeq \|P\|$, where $f \colon P \to \|P\|$ is the forward map and $g \colon \|P\| \to P$ is its quasi-inverse. Given $x, y \colon P$, we have a path

$$\mathsf{squash}(f(x), f(y)) \colon f(x) =_{\|P\|} f(y).$$

Therefore, we can concatenate the following paths:

$$\begin{aligned} x &=_P g(f(x)) && ; \text{ by } \vartheta(x)^{-1}, \\ &=_P g(f(y)) && ; \text{ by } \mathsf{ap}_g(\mathsf{squash}(f(x), f(y))), \\ &=_P y && ; \text{ by } \eta(y), \end{aligned}$$

which proves that $P$ is a mere proposition. Thus, we can define the map

$$(P \simeq \|P\|) \to \mathsf{isProp}(P), \quad \langle f, (\eta, \vartheta) \rangle \mapsto \epsilon,$$

where $\epsilon \colon \mathsf{isProp}(P)$ is the witness constructed from the equality chain above. □

**Exercise 3.7.19.**[FE] *As in classical set theory, the finite version of the axiom of choice is a theorem. Prove that the axiom of choice* (3.2) *holds when* $X$ *is a finite type* $\mathsf{Fin}(n)$*, as defined in* Exercise 1.15.6.

*Solution.* Let us introduce

$$A_n :\equiv \prod_{x \colon \mathsf{Fin}(n)} \|C(x)\| \quad \text{and} \quad B_n :\equiv \prod_{x \colon \mathsf{Fin}(n)} C(x).$$

We must construct a map

$$f_n \colon A_n \to \|B_n\|.$$

We proceed by induction on $n$.

The case $n \equiv 0$ is trivial because $\mathsf{Fin}(0) =_{\mathcal{U}} \mathbf{0}$, and so $A_0 \simeq \mathbf{1}$ and $B_0 \simeq \mathbf{1}$. We can compose the forward map of the first equivalence with the inverse of the second to obtain a map $A_0 \to B_0$. Hence, we complete the base case by postcomposing this with the canonical injection $B_0 \to \|B_0\|$.

For the inductive step, let $C' :\equiv C \circ \mathsf{inl}$ and set

$$A'_n :\equiv \prod_{x \colon \mathsf{Fin}(n)} \|C'(x)\|, \quad B'_n :\equiv \prod_{x \colon \mathsf{Fin}(n)} C'(x).$$

Since $\mathsf{Fin}(\mathsf{succ}(n)) \equiv \mathsf{Fin}(n) + 1$, we can apply the universal property of dependent functions over coproducts, Theorem 2.18.7, to obtain the equivalences

$$A_{\mathsf{succ}(n)} \simeq A'_n \times \|C(\mathsf{inr}(\star))\|, \quad B_{\mathsf{succ}(n)} \simeq B'_n \times C(\mathsf{inr}(\star)).$$

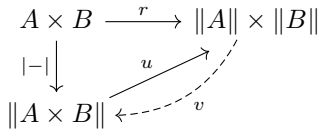
Thus, to complete the induction we have to define a map

$$A'_n \times \|C(\mathsf{inr}(\star))\| \to \|B'_n \times C(\mathsf{inr}(\star))\|.$$

Since the inductive hypothesis gives us a map $A'_n \to \|B'_n\|$, the conclusion is a direct consequence of the following

**Lemma.** *If $A$ and $B$ are types, then $\|A \times B\| \simeq (\|A\| \times \|B\|)$.*

Proof: There is a commutative diagram




where

$$\begin{aligned} r &:\equiv \lambda(a, b).\, (|a|, |b|), \\ u &:\equiv \mathsf{rec}_{\|A\times B\|}(\|A\| \times \|B\|, r), \end{aligned}$$

which is well-defined because $r$ factors through $|-|$ by part c) of Theorem 3.2.15.

To construct $v$, define the curried function

$$\begin{aligned} A &\to (B \to \|A \times B\|) \\ a &\mapsto (b \mapsto |(a, b)|). \end{aligned}$$

Since the codomain is a function type into a mere proposition, by part b) of Theorem 3.2.15, it is a mere proposition itself, and so the outer function factors through $\|A\|$. In turn, for each $a\colon A$, the inner function $b \mapsto |(a, b)|$ factors through $\|B\|$, yielding a map

$$\|A\| \to (\|B\| \to \|A \times B\|),$$

whose uncurried form gives $v$.

The conclusion follows from Lemma 3.2.4.

□

**Exercise 3.7.20.**[FE] *Show that the conclusion of* Exercise 3.7.17 *if $P\colon \mathbb{N} \to \mathcal{U}$ is any decidable family.*

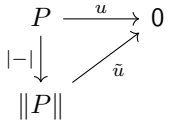
*Solution.* We open our reasoning by establishing the following

**Lemma.** *For any type $P$, there is a map $(P + \neg P) \to (\|P\| + \neg\|P\|)$. In particular, if $P$ is decidable, then $\|P\|$ is decidable as well.*

Proof: We proceed by induction on the coproduct $P + \neg P$.

For the left branch, we take the composition $\mathsf{inl} \circ |-|$.

For the right branch, observe that since $\mathsf{0}$ is a mere proposition, every map $u\colon P \to \mathsf{0}$ factors through the canonical map $|-|\colon P \to \|P\|$; i.e., there is a commutative triangle




Therefore, we can define the mapping for this branch as $u \mapsto \mathsf{inr}(\tilde{u})$. ∎

In our case, since the family $P(n)$ is decidable, we deduce that the family of types $\|P(n)\|$ satisfies the hypothesis of Exercise 3.7.17. Therefore, we have a map

$$\left\|\sum_{n\colon \mathbb{N}} \|P(n)\|\right\| \to \sum_{n\colon \mathbb{N}} \|P(n)\|.$$

Before proceeding, we need a second

**Lemma.** *Let $P\colon A \to \mathcal{U}$ be any type family. Then, there is an equivalence*

$$\left\|\sum_{x\colon A} \|P(x)\|\right\| \simeq \left\|\sum_{x\colon A} P(x)\right\|.$$

Proof: Since the type on the LHS is a mere proposition, to construct a forward map, it suffices to define it with domain in the untruncated $\Sigma$-type. The induction principle for $\Sigma$-types reduces the goal to defining, for each $a\colon A$, a map

$$\|P(a)\| \to \left\|\sum_{x\colon A} P(x)\right\|.$$

Applying the recursion principle for propositional truncation once again, the goal reduces further to defining a map $P(a) \to \left\|\sum_{x\colon A} P(x)\right\|$, which is trivially constructed by $u \mapsto |\langle a, u\rangle|$.

The backward map is trivially derived from the canonical map $P(x) \to \|P(x)\|$.

Lemma 3.2.4 completes the proof. ∎

As a result, by composing our derived maps, we have a witness

$$\epsilon\colon \left\|\sum_{n\colon \mathbb{N}} P(n)\right\| \to \sum_{n\colon \mathbb{N}} \|P(n)\|.$$

Let

$$\langle n, \zeta \rangle : \sum_{n:\mathbb{N}} \|P(n)\|.$$

Then $\zeta : \|P(n)\|$. By hypothesis, we are given an element $\epsilon : P(n) + \neg P(n)$. To derive from $\epsilon$ an element of $P(n)$, thereby establishing a map $\|P(n)\| \to P(n)$, it suffices to construct a map $(P(n) + \neg P(n)) \to P(n)$ and evaluate it at $\epsilon$. We proceed by induction on the coproduct:

LEFT BRANCH: Here we specify the identity map $\mathsf{id}_{P(n)}$.

RIGHT BRANCH: Given $u : \neg P(n)$, we factor it through $\|P(n)\|$. This is possible because $u : P(n) \to \mathsf{0}$ yields an extension $\tilde{u} : \|P(n)\| \to \mathsf{0}$, since $\mathsf{0}$ is a mere proposition. Thus, $\tilde{u}(\zeta) : \mathsf{0}$, which allows us to specify $\mathsf{rec}_{\mathsf{0}}(P(n), \tilde{u}(\zeta))$ for the right branch.

□

**Exercise 3.7.21.** *Simplify the proof of* Theorem 2.16.2 *by first proving that* $\mathsf{Code}(n, m)$ *is a mere proposition for all* $n, m : \mathbb{N}$.

*Solution.* To prove that $\mathsf{Code}(n, m)$ is a mere proposition for all $n, m : \mathbb{N}$, we proceed by double induction on $n$ and $m$.

- For $n \equiv 0$, we use induction on $m$ to show that $\mathsf{Code}(0, m)$ is a mere proposition:
  - For $m \equiv 0$, this is trivial because, by definition, $\mathsf{Code}(0, 0) \equiv \mathsf{1}$.
  - For $\mathsf{succ}(m)$, this is also trivial because $\mathsf{Code}(0, \mathsf{succ}(m)) \equiv \mathsf{0}$ by definition.
- For $\mathsf{succ}(n)$, suppose that we have $h_m : \mathsf{isProp}(\mathsf{Code}(n, m))$ for every $m : \mathbb{N}$. We prove by induction on $m$ that $\mathsf{isProp}(\mathsf{Code}(\mathsf{succ}(n), m))$ is inhabited:
  - For $m \equiv 0$, this is trivial because $\mathsf{Code}(\mathsf{succ}(n), 0) \equiv \mathsf{0}$.
  - For $\mathsf{succ}(m)$, since $\mathsf{Code}(\mathsf{succ}(n), \mathsf{succ}(m)) \equiv \mathsf{Code}(n, m)$, our goal follows from the inductive hypothesis.

Let $n, m : \mathbb{N}$ and $c : \mathsf{Code}(n, m)$. Since both $\mathsf{encode}(\mathsf{decode}(c))$ and $c$ are inhabitants of $\mathsf{Code}(n, m)$, and we just proved that this type is a mere proposition, they must be equal. As a result, the homotopy $\mathsf{encode} \circ \mathsf{decode} \sim \mathsf{id}_{\mathsf{Code}(n,m)}$ is inhabited.

The proof of the other composition proceeds as presented in Theorem 2.16.3.

□

# Chapter 4

# Equivalences

## 4.1 Quasi-Inverses

**Lemma 4.1.1.** *Let $X$ be a type. Given a function $u\colon X \to X$ and a homotopy $\theta\colon u \sim \mathsf{id}_X$, there is a homotopy*

$$\theta \triangleright u \sim u \triangleleft \theta.$$

*Proof.* Let $p\colon x =_X y$ be a path. The naturality identity (2.2) of $\theta$ evaluated on $p$ yields

$$\theta(x) \cdot p =_{u(x)=_X y} \mathsf{ap}_u(p) \cdot \theta(y).$$

By choosing $p :\equiv \theta(x)\colon u(x) =_X x$, we obtain

$$\theta(u(x)) \cdot \theta(x) =_{u(u(x))=_X x} \mathsf{ap}_u(\theta(x)) \cdot \theta(x).$$

Right-canceling $\theta(x)$ yields the stated homotopy. □

**Proposition 4.1.2.** *Given two functions $f\colon A \to B$ and $g\colon B \to A$ and two homotopies $\eta\colon f \circ g \sim \mathsf{id}_B$ and $\vartheta\colon g \circ f \sim \mathsf{id}_A$, the types*

$$(f \triangleleft \vartheta \sim \eta \triangleright f) \to (g \triangleleft \eta \sim \vartheta \triangleright g) \quad \textit{and} \quad (g \triangleleft \eta \sim \vartheta \triangleright g) \to (f \triangleleft \vartheta \sim \eta \triangleright f)$$

*are inhabited.*

*Proof.* By symmetry, it suffices to construct a map from the first to the second homotopy.

Fix a homotopy $\tau\colon f \triangleleft \vartheta \sim \eta \triangleright f$. We must construct $\rho\colon g \triangleleft \eta \sim \vartheta \triangleright g$.

Applying Lemma 4.1.1 to $\eta\colon f \circ g \sim \mathsf{id}_B$ and $\vartheta\colon g \circ f \sim \mathsf{id}_A$, we obtain two homotopies:

$$N_\eta\colon \eta \triangleright (f \circ g) \sim (f \circ g) \triangleleft \eta,$$
$$N_\vartheta\colon \vartheta \triangleright (g \circ f) \sim (g \circ f) \triangleleft \vartheta.$$

By whiskering $\tau$ with $g$ on both sides, we obtain

$$g \triangleleft \tau \triangleright g\colon (g \circ f) \triangleleft \vartheta \triangleright g \sim g \triangleleft \eta \triangleright (f \circ g).$$

Then

$$\begin{aligned} \vartheta \triangleright (g \circ f \circ g) &\sim (g \circ f) \triangleleft \vartheta \triangleright g && ;\text{ via } N_\vartheta \triangleright g \\ &\sim g \triangleleft \eta \triangleright (f \circ g) && ;\text{ via } g \triangleleft \tau \triangleright g \\ &\sim (g \circ f \circ g) \triangleleft \eta && ;\text{ via } g \triangleleft N_\eta, \end{aligned}$$

yielding a homotopy $\alpha\colon \vartheta \triangleright (g \circ f \circ g) \sim (g \circ f \circ g) \triangleleft \eta$.

Now consider the naturality of $\vartheta\colon g \circ f \sim \mathsf{id}_A$ evaluated on the homotopy $g \triangleleft \eta\colon g \circ f \circ g \sim g$, as described in § 2.4.4. In this case, diagram (2.2) yields the following commutative square of homotopies:

$$\begin{array}{ccc} g \circ f \circ g \circ f \circ g & \xrightarrow{\vartheta \triangleright (g \circ f \circ g)} & g \circ f \circ g \\ \Big\downarrow{\scriptstyle (g \circ f \circ g) \triangleleft \eta} & & \Big\downarrow{\scriptstyle g \triangleleft \eta} \\ g \circ f \circ g & \xrightarrow[\vartheta \triangleright g]{} & g \end{array}$$

Since $\alpha$ establishes that the top edge is homotopic to the left edge, we obtain the required homotopy

$$g \triangleleft \eta \sim \vartheta \triangleright g.$$

□

**Corollary 4.1.3.** *If $f\colon A \to B$ is an equivalence with quasi-inverse $\langle g, (\eta, \vartheta)\rangle$, there is a homotopy $\vartheta'\colon g \circ f \sim \mathsf{id}_A$ such that $\langle g, (\eta, \vartheta')\rangle$ is a coherent quasi-inverse.*

*Proof.* The hypothesis implies that $\langle f, (\vartheta, \eta)\rangle$ is a quasi-inverse of $g$. By Theorem 2.5.16, there is a homotopy $\vartheta'$ such that $\langle f, (\vartheta', \eta)\rangle$ is a coherent quasi-inverse. This means that we have $\tau\colon g \triangleleft \eta \sim \vartheta' \triangleright g$. Applying Proposition 4.1.2, we deduce that $f \triangleleft \vartheta' \sim \eta \triangleright f$ is inhabited, as desired. □

**Theorem 4.1.4.** *If $f\colon A \to B$ is an equivalence with quasi-inverse $\langle g, (\eta, \vartheta)\rangle$, then for all $y\colon B$ the fiber $\mathsf{fib}_f(y)$ is contractible with center $\langle g(y), \eta(y)\rangle$.*

*Proof.* By Corollary 4.1.3, after replacing $\vartheta$ with $\vartheta'$, we can assume that there exists a homotopy

$$\tau\colon f \triangleleft \vartheta \sim \eta \triangleright f.$$

Given $y\colon B$, note that

$$\langle g(y), \eta(y)\rangle\colon \mathsf{fib}_f(y)$$

is well-typed since $\eta(y)\colon f(g(y)) =_B y$.

Fix $\langle x, p\rangle\colon \mathsf{fib}_f(y)$, where $p\colon f(x) =_B y$. According to Definition 3.4.15, we must provide a path $\gamma\colon g(y) =_A x$ such that

$$\mathsf{transport}^{z\mapsto f(z)=_B y}(\gamma, \eta(y)) =_{f(x)=_B y} p.$$

Using Lemma 2.3.13, the LHS equals

$$\mathsf{transport}^{z\mapsto z=_B y}\big(\mathsf{ap}_f(\gamma), \eta(y)\big).$$

Therefore, by Proposition 2.11.6, our goal becomes

$$\mathsf{ap}_f(\gamma)^{-1} \cdot \eta(y) =_{f(x)=_B y} p,$$

which is equivalent to

$$\mathsf{ap}_f(\gamma) \cdot p =_{f(g(y))=_B y} \eta(y). \tag{$\dagger$}$$

Let

$$\gamma :\equiv \mathsf{ap}_g(p)^{-1} \cdot \vartheta_x.$$

Since $\mathsf{ap}_g(p)\colon g(f(x)) =_A g(y)$ and $\vartheta_x\colon g(f(x)) =_A x$, we see that $\gamma$ is of the required type. Moreover,

$$\mathsf{ap}_f(\gamma) \cdot p =_{f(g(y))=_B y} \mathsf{ap}_f(\mathsf{ap}_g(p))^{-1} \cdot \mathsf{ap}_f(\vartheta_x) \cdot p.$$

Using the coherence condition $\tau_x\colon \mathsf{ap}_f(\vartheta_x) =_{f(g(f(x)))=_B f(x)} \eta_{f(x)}$, we obtain

$$\mathsf{ap}_f(\gamma) \cdot p =_{f(g(y))=_B y} \mathsf{ap}_f(\mathsf{ap}_g(p))^{-1} \cdot \eta_{f(x)} \cdot p. \tag{$\ddagger$}$$

The naturality path diagram (2.1) applied to $p\colon f(x) =_B y$ and $\eta\colon f \circ g \sim \mathsf{id}_B$ yields

$$\begin{array}{ccc} f(g(f(x))) & \xrightarrow{\mathsf{ap}_{f\circ g}(p)} & f(g(y)) \\ {\scriptstyle \eta(f(x))}\downarrow & & \downarrow{\scriptstyle \eta(y)} \\ f(x) & \xrightarrow[\mathsf{ap}_{\mathsf{id}_B}(p)]{} & y \end{array}$$

Using that $\mathsf{ap}_{f\circ g} \equiv \mathsf{ap}_f \circ \mathsf{ap}_g$ and $\mathsf{ap}_{\mathsf{id}_B} \equiv \mathsf{id}$, the diagram translates into

$$\mathsf{ap}_f(\mathsf{ap}_g(p))^{-1} \cdot \eta(f(x)) \cdot p =_{f(g(y))=_B y} \eta(y).$$

Substituting this into ($\ddagger$), we arrive at ($\dagger$). □

**Definition 4.1.5.** Given a function $f\colon A \to B$, we define the types

$$\mathsf{linv}(f) :\equiv \sum_{g\colon B\to A} g \circ f \sim \mathsf{id}_A$$
$$\mathsf{rinv}(f) :\equiv \sum_{g\colon B\to A} f \circ g \sim \mathsf{id}_B$$

of *left* and *right inverses* to $f$, respectively. We call $f$ *left invertible* if $\mathsf{linv}(f)$ is inhabited, and *right invertible* if $\mathsf{rinv}(f)$ is inhabited.

**Lemma 4.1.6.** *Let $B\colon \mathcal{U}$ be a type and $P\colon B \to \mathcal{U}$ a type family. Consider two functions $f, g\colon B \to \Sigma_B P$, a path $p\colon f = g$, and the type family*

$$Q\colon (B \to \Sigma_B P) \to \mathcal{U}$$
$$h \mapsto \pi_1 \circ h \sim \mathsf{id}_B.$$

*Then, for any $\eta\colon Q(f)$ and any $b\colon B$, we have*

$$\mathsf{transport}^Q(p,\eta)(b) =_{\pi_1(g(b))=_B b} \mathsf{transport}^{Q_b}(p,\eta(b)),$$

*where $Q_b :\equiv \lambda h.\, \pi_1(h(b)) =_B b$.*

*Proof.* By path induction on $p$, it suffices to consider the case where $g \equiv f$ and $p \equiv \mathsf{refl}_f$. Since transport along reflexivity is the identity, the goal reduces to

$$\eta(b) =_{\pi_1(f(b))=_B b} \eta(b),$$

which is witnessed by $\mathsf{refl}_{\eta(b)}$. □

**Theorem 4.1.7.**[FE] *Let $B\colon \mathcal{U}$ be a type and $P\colon B \to \mathcal{U}$ a type family. The type of dependent functions $\prod_{b\colon B} P(b)$ is equivalent to the type of right inverses of the first projection $\pi_1\colon \Sigma_B P \to B$. That is, there is an equivalence*

$$\Big(\prod_{b\colon B} P(b)\Big) \simeq \mathsf{rinv}(\pi_1).$$

*Proof.* We define the forward map by

$$\Phi\colon \Big(\prod_{b\colon B} P(b)\Big) \to \mathsf{rinv}(\pi_1)$$
$$\sigma \mapsto \langle \underbrace{\lambda b.\, \langle b, \sigma(b)\rangle}_{B\to\Sigma_B P}, \lambda b.\, \mathsf{refl}_b\rangle,$$

and the backward map by

$$\Psi\colon \mathsf{rinv}(\pi_1) \to \prod_{b\colon B} P(b)$$
$$\langle g, \eta\rangle \mapsto \lambda b.\, \mathsf{transport}^P(\eta(b), \pi_2(g(b))).$$

This definition is well-typed because $\eta(b)\colon \pi_1(g(b)) =_B b$, and so the transport map takes $P(\pi_1(g(b)))$ to $P(b)$.

We must show that these maps are mutually inverse. For the forward-backward composition, we have

$$\begin{aligned}\Psi(\Phi(\sigma))(b) &\equiv \mathsf{transport}^P(\mathsf{refl}_b, \pi_2(\langle b, \sigma(b)\rangle)) \\ &\equiv \mathsf{transport}^P(\mathsf{refl}_b, \sigma(b)) \\ &\equiv \sigma(b).\end{aligned}$$

Hence, $\Psi(\Phi(\sigma)) \equiv \sigma$, which implies $\Psi \circ \Phi \sim \mathsf{id}$.

For the backward-forward composition, we obtain

$$\Phi(\Psi(\langle g, \eta\rangle)) \equiv \big\langle \lambda b.\, \langle b, \mathsf{transport}^P(\eta(b), \pi_2(g(b)))\rangle, \lambda b.\, \mathsf{refl}_b \big\rangle.$$

Let

$$f :\equiv \lambda b.\, \langle b, \mathsf{transport}^P(\eta(b), \pi_2(g(b)))\rangle$$

denote the function in the first component of the RHS. We must show that

$$\langle g, \eta\rangle =_{\mathsf{rinv}(\pi_1)} \langle f, \lambda b.\, \mathsf{refl}_b\rangle.$$

To establish a path $g =_{B\to\Sigma_B P} f$, we will construct a homotopy $\vartheta\colon g \sim f$ and then apply function extensionality to get $\mathsf{funext}(\vartheta)$.

Fix $b\colon B$. To construct a path $\vartheta(b)\colon g(b) =_{\Sigma_B P} f(b)$, we need paths for both components:

- a path in the first component given by
$$\eta(b)\colon \pi_1(g(b)) =_B b,$$
- a path in the second component between the transport of $\pi_2(g(b))$ along $\eta(b)$ and $\pi_2(f(b))$. Since $\pi_2(f(b)) \equiv \mathsf{transport}^P(\eta(b), \pi_2(g(b)))$, this path is trivially produced by reflexivity.

This establishes the path $\vartheta(b)$, yielding $\mathsf{funext}(\vartheta)\colon g =_{B\to\Sigma_B P} f$.

To complete the proof, we must show that transporting the second component $\eta$ along $\mathsf{funext}(\vartheta)$ in the family

$$\begin{aligned}Q\colon (B \to \Sigma_B P) &\to \mathcal{U} \\ h &\mapsto \pi_1 \circ h \sim \mathsf{id}_B\end{aligned}$$

yields $\lambda b.\, \mathsf{refl}_b$. By Lemma 4.1.6, we deduce

$$\mathsf{transport}^Q(\mathsf{funext}(\vartheta), \eta)(b) =_{Q_b(f)} \mathsf{transport}^{Q_b}(\mathsf{funext}(\vartheta), \eta(b)),$$

where $Q_b :\equiv \lambda h.\, \pi_1(h(b)) =_B b$.

Applying Transport in Identity Types (Lemma 2.14.2) to the map $\lambda h.\, \pi_1(h(b))$ and the constant $\lambda h.\, b$, the RHS becomes

$$\begin{aligned}\text{RHS} &=_{Q_b(f)} \mathsf{ap}_{\lambda h.\, \pi_1(h(b))}(\mathsf{funext}(\vartheta))^{-1} \cdot \eta(b) \cdot \mathsf{ap}_{\lambda h.\, b}(\mathsf{funext}(\vartheta)) \\ &=_{Q_b(f)} \left(\mathsf{ap}_{\pi_1} \circ \mathsf{ap}_{\mathrm{ev}_b}(\mathsf{funext}(\vartheta))\right)^{-1} \cdot \eta(b) \cdot \cancel{\mathsf{refl}_b} \\ &=_{Q_b(f)} \mathsf{ap}_{\pi_1}(\vartheta(b))^{-1} \cdot \eta(b).\end{aligned}$$

Since the path in the first component of $\vartheta(b)$ is $\eta(b)$, Lemma 2.9.11 implies that $\mathsf{ap}_{\pi_1}(\vartheta(b)) = \eta(b)$. Thus, we conclude that

$$\text{RHS} =_{Q_b(f)} \eta(b)^{-1} \cdot \eta(b) = \mathsf{refl}_b.$$

By function extensionality, the transported homotopy is equal to $\lambda b.\, \mathsf{refl}_b$. This completes the proof.

□

**Proposition 4.1.8.**[FE] *If $f\colon A \to B$ has a quasi-inverse $\langle g, (\eta, \vartheta)\rangle$, then the types $\mathsf{rinv}(f)$ and $\mathsf{linv}(f)$ are contractible with respective centers $\langle g, \eta\rangle$ and $\langle g, \vartheta\rangle$.*

*Proof.* Consider the maps

$$\begin{aligned}(f \circ -)&\colon (C \to A) \to (C \to B) \\ (- \circ f)&\colon (B \to C) \to (A \to C).\end{aligned}$$

If $g$ is a quasi-inverse of $f$, then $(g \circ -)$ and $(- \circ g)$ are quasi-inverses of $(f \circ -)$ and $(- \circ f)$, respectively. Thus, they are equivalences and, by Theorem 4.1.4, their fibers are contractible.

Therefore, the same theorem implies that $\mathsf{rinv}(f)$ and $\mathsf{linv}(f)$ are contractible because, by function extensionality, we have

$$\begin{aligned}\mathsf{rinv}(f) &\simeq \Big(\sum_{g\colon B\to A} f \circ g =_{B\to B} \mathsf{id}_B\Big) \equiv \mathsf{fib}_{f\circ -}(\mathsf{id}_B) \\ \mathsf{linv}(f) &\simeq \Big(\sum_{g\colon B\to A} g \circ f =_{A\to A} \mathsf{id}_A\Big) \equiv \mathsf{fib}_{-\circ f}(\mathsf{id}_A).\end{aligned}$$

Since $\langle g, \mathsf{funext}(\eta)\rangle$ and $\langle g, \mathsf{funext}(\vartheta)\rangle$ are the respective centers of $\mathsf{fib}_{f\circ -}(\mathsf{id}_B)$ and $\mathsf{fib}_{-\circ f}(\mathsf{id}_A)$, the statement on the centers of $\mathsf{rinv}(f)$ and $\mathsf{linv}(f)$ follows. □

**Lemma 4.1.9.** *Let $f\colon A \to B$ be a function. Then there is an equivalence*

$$\mathsf{qinv}(f) \simeq \sum_{\omega\colon \mathsf{rinv}(f)} \pi_1(\omega) \circ f \sim \mathsf{id}_A.$$

*Proof.* By definition, the type $\mathsf{qinv}(f)$ is given by

$$\mathsf{qinv}(f) \equiv \sum_{h:\, B\to A} (f \circ h \sim \mathsf{id}_B) \times (h \circ f \sim \mathsf{id}_A).$$

By expanding the cartesian product into a non-dependent $\Sigma$-type, this becomes definitionally equivalent to

$$\sum_{h:\, B\to A} \; \sum_{\theta:\, f\circ h\sim \mathsf{id}_B} h \circ f \sim \mathsf{id}_A.$$

Using $\Sigma$-associativity (see Proposition 2.9.13), this nested $\Sigma$-type is equivalent to

$$\sum_{\omega:\, \sum_{h:\, B\to A} f\circ h\sim \mathsf{id}_B} \pi_1(\omega) \circ f \sim \mathsf{id}_A.$$

This completes the proof because the index type is precisely $\mathsf{rinv}(f)$. □

**Lemma 4.1.10.** *Let $X : \mathcal{U}$ be a type, and let $A, B : X \to \mathcal{U}$ be two type families over $X$. If $\Sigma_X A$ is contractible with center $\langle z, c\rangle$, then there is an equivalence*

$$\sum_{x:\, X} A(x) \times B(x) \simeq B(z).$$

*Proof.* By rewriting the cartesian product as a non-dependent $\Sigma$-type, the left-hand side equals

$$\sum_{x:\, X} \; \sum_{u:\, A(x)} B(x).$$

Using $\Sigma$-associativity (see Proposition 2.9.13), this total space is equivalent to

$$\sum_{\omega:\, \Sigma_X A} B(\pi_1(\omega)).$$

By part b) of Theorem 3.4.13, we obtain

$$\Big(\sum_{\omega:\, \Sigma_X A} B(\pi_1(\omega))\Big) \simeq B(\pi_1(\langle z, c\rangle)) \equiv B(z).$$

□

**Proposition 4.1.11.**[FE] *If $f : A \to B$ is such that $\mathsf{qinv}(f)$ is inhabited, then*

$$\mathsf{qinv}(f) \simeq (\mathsf{id}_A \sim \mathsf{id}_A).$$

*Proof.* Since $\mathsf{qinv}(f)$ is inhabited, $f$ has a quasi-inverse $\langle g, (\eta, \vartheta)\rangle$. Define

$$\begin{aligned} \sigma : (\mathsf{id}_A \sim \mathsf{id}_A) &\to (g \circ f \sim \mathsf{id}_A) \\ \epsilon &\mapsto \vartheta \cdot \epsilon. \end{aligned}$$

The function $\sigma$ is an equivalence with inverse

$$\begin{aligned}(g \circ f \sim \mathsf{id}_A) &\to (\mathsf{id}_A \sim \mathsf{id}_A)\\ \delta &\mapsto \vartheta^{-1} \cdot \delta.\end{aligned}$$

By Proposition 4.1.8, $\mathsf{rinv}(f)$ is contractible with center $\langle g, \eta\rangle$. Thus, applying Lemma 4.1.10 to the definition

$$\mathsf{qinv}(f) \equiv \sum_{h:\, B\to A} (f \circ h \sim \mathsf{id}_B) \times (h \circ f \sim \mathsf{id}_A)$$

yields an equivalence $\mathsf{qinv}(f) \simeq (g \circ f \sim \mathsf{id}_A)$, which, combined with the equivalence induced by $\sigma$, completes the proof. □

**Lemma 4.1.12.**[FE] *Suppose we have a type $A$ with $a\colon A$ and $q\colon a =_A a$ such that*

a) *The type $a =_A a$ is a set.*

b) *For all $x\colon A$ we have $\|a =_A x\|$.*

c) *For all $p\colon a =_A a$ we have $p \cdot q =_{a=_A a} q \cdot p$.*

*Then there exists a homotopy $\eta\colon \mathsf{id}_A \sim \mathsf{id}_A$ with $\eta(a) \equiv q$.*

*Proof.* The proof consists of three parts.

Part 1. *Given $x, y\colon A$, the type $x =_A y$ is a set.*

We proceed by defining a map

$$\gamma\colon \big((a =_A x) \times (a =_A y)\big) \to \mathsf{isSet}(x =_A y).$$

Given $r\colon a =_A x$ and $s\colon a =_A y$, we define

$$\begin{aligned}g_{r,s}\colon (a =_A a) &\to (x =_A y)\\ p &\mapsto r^{-1} \cdot p \cdot s.\end{aligned}$$

Since this map has an inverse, it induces an equivalence

$$(a =_A a) \simeq (x =_A y).$$

By Exercise 3.7.1, condition a) implies the existence of a witness

$$\omega_{r,s}\colon \mathsf{isSet}(x =_A y).$$

Hence, we can define $\gamma(r, s) :\equiv \omega_{r,s}$.

Given that $\mathsf{isSet}(x =_A y)$ is a mere proposition [cf. Remark 3.2.2], the map $\gamma$ factors through

$$\mathsf{rec}_{\|\cdots\|}(\mathsf{isSet}(x =_A y), \gamma)\colon \|(a =_A x) \times (a =_A y)\| \to \mathsf{isSet}(x =_A y).$$

By the lemma included in the solution to Exercise 3.7.19, condition b) implies that this truncated type is inhabited. Consequently, the type $\mathsf{isSet}(x =_A y)$ is inhabited as well.

PART 2. *Given $x\colon A$, the type*

$$B(x) :\equiv \sum_{r\colon x=_A x} \prod_{s\colon a=_A x} r =_{x=_A x} s^{-1} \cdot q \cdot s$$

*is a mere proposition.*

To prove this, it suffices to construct a map

$$(a =_A x) \to \mathsf{isProp}(B(x)).$$

Let $t\colon a =_A x$. If $\langle r, u\rangle$ and $\langle r', u'\rangle$ are elements of $B(x)$, we must construct from $t$ a path between them.

Since

$$u(t)\colon r =_{x=_A x} t^{-1} \cdot q \cdot t \quad \text{and} \quad u'(t)\colon r' =_{x=_A x} t^{-1} \cdot q \cdot t,$$

we obtain the path

$$e :\equiv u(t) \cdot u'(t)^{-1}\colon r =_{x=_A x} r'.$$

To establish the equality $\langle r, u\rangle =_{B(x)} \langle r', u'\rangle$, we must show that for the type family

$$S(p) :\equiv \prod_{w\colon a=_A x} p =_{x=_A x} w^{-1} \cdot q \cdot w,$$

we have $\mathsf{transport}^S(e, u) = u'$. By function extensionality, this reduces to showing that for all $w\colon a =_A x$,

$$\mathsf{transport}^S(e, u)(w) = u'(w)$$

in $r' =_{x=_A x} w^{-1} \cdot q \cdot w$. But, since $x =_A x$ is a set, this identity type is a mere proposition, and the equality holds automatically.

PART 3. *There is a homotopy $\eta\colon \mathsf{id}_A \sim \mathsf{id}_A$ with $\eta(a) \equiv q$.*

Because $B(x)$ is a mere proposition for any $x\colon A$, to prove that $B(x)$ is inhabited, it suffices to construct a map $(a =_A x) \to B(x)$, and then invoke condition b).

Consider

$$D\colon \prod_{x\colon A} (a =_A x) \to \mathcal{U}$$
$$D(x, p) :\equiv B(x).$$

By BASED PATH INDUCTION, to define a map

$$f\colon \prod_{x\colon A} \prod_{p\colon a=_A x} B(x),$$

it suffices to construct an element of $D(a, \mathsf{refl}_a) \equiv B(a)$.

By definition,

$$B(a) \equiv \sum_{r:\, a =_A a} \prod_{s:\, a =_A a} r =_{a =_A a} s^{-1} \cdot q \cdot s.$$

Condition c) implies that for $r :\equiv q$, we have a path

$$\beta(s) \colon q =_{a =_A a} s^{-1} \cdot q \cdot s.$$

Thus, $\langle q, \beta \rangle \colon B(a)$, as required. As a result, for every $x \colon A$, we obtain a map $f_x \colon (a =_A x) \to B(x)$. Given that $B(x)$ is a mere proposition, $f_x$ factors through $\|a =_A x\|$. And since this type is inhabited, we obtain a witness $b_x \colon B(x)$. Moreover, $b_a \equiv f_a(\mathsf{refl}_a) \equiv \langle q, \beta \rangle$.

Therefore, we can define the homotopy $\eta :\equiv \lambda x.\, \pi_1(b_x)$ that satisfies

$$\eta(a) \equiv \pi_1(b_a) \equiv \pi_1(\langle q, \beta \rangle) \equiv q.$$

□

**Theorem 4.1.13.**[FE UA] *There exists a type $A$ such that $\mathsf{qinv}(\mathsf{id}_A)$ is not a mere proposition.*

*Proof.* By Proposition 4.1.11, it suffices to construct a type $A$ and a homotopy $\eta \colon \mathsf{id}_A \sim \mathsf{id}_A$ with $\eta \neq (\lambda x.\, \mathsf{refl}_x)$.

For the type we set

$$A :\equiv \sum_{X:\, \mathcal{U}} \|2 =_{\mathcal{U}} X\|.$$

Let $a :\equiv \langle 2, |\mathsf{refl}_2| \rangle \colon A$, and consider the equivalence $e \colon 2 \simeq 2$ induced by the map $\mathsf{not}$ introduced in Exercise 1.16.2. The lemma included in the solution to Exercise 2.19.12 shows that $\mathsf{not} \neq \mathsf{id}_2$.

Let $e_{\mathsf{id}} \colon 2 \simeq 2$ be the identity equivalence. Since $\pi_1(e) \equiv \mathsf{not}$ and $\pi_1(e_{\mathsf{id}}) \equiv \mathsf{id}_2$, we deduce that $e \neq e_{\mathsf{id}}$.

On the other hand, by definition, we have $\mathsf{idtoeqv}(\mathsf{refl}_2) \equiv e_{\mathsf{id}}$. Because $\mathsf{ua}$ and $\mathsf{idtoeqv}$ are quasi-inverses, this yields the propositional equality

$$\mathsf{refl}_2 = \mathsf{ua}(\mathsf{idtoeqv}(\mathsf{refl}_2)) \equiv \mathsf{ua}(e_{\mathsf{id}}).$$

Thus, defining $q :\equiv \mathsf{ua}(e)$, we obtain $q \neq \mathsf{refl}_2$.

Since we intend to use Lemma 4.1.12, we need to prove the three conditions stated there.

Condition a) By Lemma 3.2.13 and the univalence axiom, there is an equivalence

$$(a =_A a) \simeq (2 \simeq 2).$$

Since $(2 \simeq 2) \simeq 2$ by Exercise 2.19.12, and $2$ is a set by Theorem 3.2.8, condition a) holds.

Condition b) Given $x :\equiv \langle X, u\rangle$ with $u\colon \|2 =_{\mathcal{U}} X\|$, we need to verify that the truncated type

$$\|\langle 2, |\mathsf{refl}_2|\rangle =_A \langle X, u\rangle\|$$

is inhabited.

We proceed by recursion on the truncated type $\|2 =_{\mathcal{U}} X\|$. Let $P\colon \mathcal{U} \to \mathcal{U}$ be the type family $X \mapsto \|2 =_{\mathcal{U}} X\|$. Given a path $p\colon 2 =_{\mathcal{U}} X$, we can apply Lemma 3.2.13 to obtain a path $\epsilon_p\colon a =_A x$, because $P(X)$ is a mere proposition.

It follows that $p \mapsto |\epsilon_p|$ is a map of type $(2 =_{\mathcal{U}} X) \to \|a =_A x\|$. Since the codomain is a mere proposition, this map factors through $\|2 =_{\mathcal{U}} X\|$. Evaluating this factored map at the given $u$ provides the desired witness in $\|a =_A x\|$.

Condition c) As established in the verification of condition a), we know that $(a =_A a) \simeq 2$ [cf. Proposition 2.5.13 c)]. Thus, the identity type has exactly two inhabitants. Besides $q$, which trivially commutes with itself, the only other element corresponds to $\mathsf{ua}(\mathsf{id}_2)$, which is the reflexivity path and also commutes with $q$. Hence, condition c) is satisfied.

Since all three conditions hold, Lemma 4.1.12 provides a homotopy $\eta\colon \mathsf{id}_A \sim \mathsf{id}_A$ such that $\eta(a) \equiv q$. Since $q \neq \mathsf{refl}_2$, it follows that $\eta \neq (\lambda x.\, \mathsf{refl}_x)$.

To complete the theorem, define $B :\equiv A$ and $f :\equiv \mathsf{id}_A$. By Proposition 4.1.11, the existence of the non-trivial self-homotopy $\eta$ of the identity map on $A$ implies that the type $\mathsf{qinv}(\mathsf{id}_A)$ is not a mere proposition.

□

## 4.2 Half-Adjoint Equivalences

Definition 2.5.15 introduces the concept of a coherent quasi-inverse of an equivalence $f\colon A \to B$. Then, Theorem 2.5.16 shows that such a quasi-inverse always exists. Here, we continue the study of this condition, introducing the type

$$\mathsf{ishae}(f) :\equiv \sum_{g\colon B\to A} \sum_{\eta\colon f\circ g\sim \mathsf{id}_B} \sum_{\vartheta\colon g\circ f\sim \mathsf{id}_A} f \triangleleft \vartheta \sim \eta \triangleright f. \tag{4.1}$$

The need for this definition arises from the failure of $\mathsf{qinv}(f)$ to be a mere proposition. The type $\mathsf{qinv}(f)$ is formed by three pieces of data: a map $g$, a right homotopy $\eta$, and a left homotopy $\vartheta$. As stated in Proposition 4.1.8, when $f$ is an equivalence, the space $\mathsf{linv}(f)$ of its left inverses is contractible. As a consequence, the total space $\mathsf{qinv}(f)$ reduces to the space of right homotopies $\eta$ [cf. Theorem 3.4.8 b)].

As established in Proposition 4.1.11, the space $\mathsf{qinv}(f)$ is equivalent to $\mathsf{id}_A \sim \mathsf{id}_A$. If the space $A$ is not a set, there are multiple distinct, non-homotopic loops at

each point, meaning there are multiple valid choices for $\eta$. Consequently, $\mathsf{qinv}(f)$ can have multiple inhabitants.

To ensure that the property of being an equivalence is a mere proposition, we must eliminate these extra degrees of freedom. This is achieved by introducing the coherence datum in (4.1), which tethers the right homotopy $\eta$ to the left homotopy $\vartheta$. While the space of all possible right homotopies is potentially large, the space of pairs $\langle \eta, \tau \rangle$, where $\tau \colon f \triangleleft \vartheta \sim \eta \triangleright f$, is contractible. By pairing the contractible space of left inverses $\langle g, \vartheta \rangle$ with the contractible space of coherent right homotopies $\langle \eta, \tau \rangle$, all higher degrees of freedom are eliminated, and $\mathsf{ishae}(f)$ turns out to be a mere proposition.

On a practical level, if $\mathsf{isequiv}(f)$ were not a mere proposition, every time we used the fact that $f$ is an equivalence, we would have to explicitly keep track of which specific proof we were using.

By constructing $\mathsf{ishae}(f)$ as a mere proposition, we attain *proof irrelevance*. We can construct an equivalence and immediately forget the specific construction of the right/left homotopies, because any two inhabitants of the mere proposition are propositionally equal. This saves the theory from collapsing into an unmanageable infinite tower of coherence conditions.

**Remark 4.2.1.** As we discussed at the beginning of this section, the type of quasi-inverses reduces to the space of choices for the right homotopy $\eta$. This leaves $\eta$ as an unbound "loose end," meaning the type $\mathsf{qinv}(f)$ is not generally a mere proposition.

The left coherence condition $\tau \colon f \triangleleft \vartheta \sim \eta \triangleright f$ fixes this loose end by tethering $\eta$ to $\vartheta$. Since the space of such coherent pairs $\langle \eta, \tau \rangle$ is contractible, the entire structure becomes a sequence of contractible extensions, leaving no degrees of freedom. Thus, the type $\mathsf{ishae}(f)$ is a mere proposition.

If we wanted to reproduce the standard categorical adjunction by demanding both coherence conditions, we would have to define a type containing the data $\langle g, \vartheta, \eta, \tau, \nu \rangle$, where $\nu \colon g \triangleleft \eta \sim \vartheta \triangleright g$.

Since the base space of half-adjoint data $\langle g, \vartheta, \eta, \tau \rangle$ is contractible, the total space of this two-sided structure would reduce entirely to the space of choices for $\nu$.

However, $\nu$ is a homotopy between homotopies (a 2-path). Just as with $\eta$ in the case of quasi-inverses, there can be multiple choices for $\nu$, which re-introduces degrees of freedom in higher homotopy levels. In other words, the type of full two-sided adjoint equivalences generally fails to be a mere proposition.

This is why the type-theoretic definition specifies only one coherence condition, rather than redundantly including both.

**Theorem 4.2.2.** *Given $f \colon A \to B$, there is a map $\mathsf{qinv}(f) \to \mathsf{ishae}(f)$.*

*Proof.* This is Theorem 2.5.16 restated after the introduction of $\mathsf{ishae}(f)$. □

**Lemma 4.2.3.** *Let $f\colon A \to B$ be a function. Then, given $y\colon B$, and elements $\langle x, p\rangle, \langle x', p'\rangle\colon \mathsf{fib}_f(y)$, we have an equivalence*

$$\big(\langle x, p\rangle =_{\mathsf{fib}_f(y)} \langle x', p'\rangle\big) \simeq \sum_{q\colon x = x'} \mathsf{ap}_f(q) \cdot p' =_{f(x)=_B y} p.$$

*Proof.* By the characterization of equality in $\Sigma$-types [cf. § 2.9], we have an equivalence

$$\big(\langle x, p\rangle =_{\mathsf{fib}_f(y)} \langle x', p'\rangle\big) \simeq \sum_{q\colon x = x'} \mathsf{transport}^P(q, p) =_{f(x')=_B y} p',$$

where $P(x) :\equiv f(x) =_B y$.

Since $P \equiv (z \mapsto z =_B y) \circ f$, we can use Lemma 2.3.13 and Proposition 2.11.6 to compute the transport as

$$\mathsf{transport}^P(q, p) =_{f(x')=_B y} \mathsf{ap}_f(q)^{-1} \cdot p.$$

Thus, for any fixed $q\colon x =_A x'$, the identity above involving the transport is equivalent to

$$\mathsf{ap}_f(q)^{-1} \cdot p =_{f(x')=_B y} p'.$$

Left-concatenating both sides with $\mathsf{ap}_f(q)$ yields an equivalence to

$$p =_{f(x)=_B y} \mathsf{ap}_f(q) \cdot p'.$$

Taking the path inverse provides a further equivalence to

$$\mathsf{ap}_f(q) \cdot p' =_{f(x)=_B y} p.$$

Since this establishes an equivalence between the fibers for each $q\colon x =_A x'$, the induced map on the total spaces is an equivalence as well. □

**Remark 4.2.4.** By Theorem 4.1.4, if $f\colon A \to B$ is a half-adjoint equivalence, then for any $y\colon B$, the fiber $\mathsf{fib}_f(y)$ is contractible.

**Definition 4.2.5.** For any function $f\colon A \to B$, a right inverse $\langle g, \eta\rangle\colon \mathsf{rinv}(f)$, and a left inverse $\langle g, \vartheta\rangle\colon \mathsf{linv}(f)$, we define the types of *right coherences* and *left coherences* as

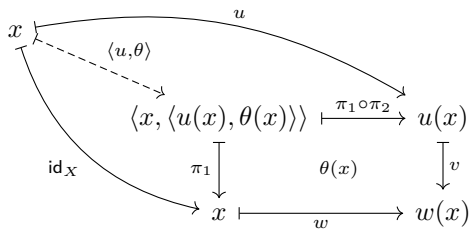
$$\mathsf{rcoh}_f(\langle g, \eta\rangle) :\equiv \sum_{\vartheta\colon g\circ f \sim \mathsf{id}_A} f \triangleleft \vartheta \sim \eta \triangleright f,$$

$$\mathsf{lcoh}_f(\langle g, \vartheta\rangle) :\equiv \sum_{\eta\colon f\circ g \sim \mathsf{id}_B} g \triangleleft \eta \sim \vartheta \triangleright g.$$

**Notation 4.2.6.** *Given $u\colon X \to Z$, $v\colon Z \to Y$, $w\colon X \to Y$, and $\theta\colon v \circ u \sim w$, we introduce the function*

$$\langle u, \theta\rangle\colon \prod_{x\colon X} \mathsf{fib}_v(w(x))$$
$$x \mapsto \langle u(x), \theta(x)\rangle.$$

*Representing this in a homotopy diagram, the total space of the fibration acts as the categorical pullback $X \times_Y Z$. The homotopy $\theta$ witnesses the commutativity of the quadrangle, and $\langle u, \theta\rangle$ is the map induced by the pair $(\mathsf{id}_X, u)$:*




**Lemma 4.2.7.** *Let $f\colon A \to B$, $g\colon B \to A$, $\eta\colon f \circ g \sim \mathsf{id}_B$, and $\vartheta\colon g \circ f \sim \mathsf{id}_A$. Using the preceding notation we have*

$$\mathsf{rcoh}_f(\langle g, \eta\rangle) \simeq \big(\langle g \circ f, \eta \triangleright f\rangle \sim \langle \mathsf{id}_A, \mathsf{refl} \triangleright f\rangle\big),$$
$$\mathsf{lcoh}_f(\langle g, \vartheta\rangle) \simeq \big(\langle f \circ g, \vartheta \triangleright g\rangle \sim \langle \mathsf{id}_B, \mathsf{refl} \triangleright g\rangle\big).$$

*Proof.* By symmetry, it suffices to prove the first equivalence.

First note that the expressions above are well-typed:

a) For $\langle g \circ f, \eta \triangleright f\rangle$, take $u :\equiv g \circ f$, $\theta :\equiv \eta \triangleright f$, $v :\equiv f$, and $w :\equiv f$.

b) For $\langle \mathsf{id}_A, \mathsf{refl} \triangleright f\rangle$, take $u :\equiv \mathsf{id}_A$, $\theta :\equiv \mathsf{refl} \triangleright f$, $v :\equiv f$, and $w :\equiv f$.

For the forward map of the equivalence, fix $\langle \alpha, \beta\rangle\colon \mathsf{rcoh}_f(\langle g, \eta\rangle)$. We have to construct the corresponding homotopy.

By definition, $\alpha\colon g \circ f \sim \mathsf{id}_A$ and $\beta\colon f \triangleleft \alpha \sim \eta \triangleright f$.

Given $x\colon A$, we need a path

$$\langle g(f(x)), \eta_{f(x)}\rangle =_{\mathsf{fib}_f(f(x))} \langle x, \mathsf{refl}_{f(x)}\rangle.$$

Let $p :\equiv \alpha(x)$ and $q :\equiv \beta(x)$. Then

$$p\colon g(f(x)) =_A x \quad \text{and} \quad q\colon \mathsf{ap}_f(p) =_{f(g(f(x)))=_B f(x)} \eta_{f(x)}.$$

By Lemma 4.2.3, constructing the required path in $\mathsf{fib}_f(f(x))$ is equivalent to providing an element of the $\Sigma$-type

$$\sum_{r\colon g(f(x))=_A x} \mathsf{ap}_f(r) \cdot \mathsf{refl}_{f(x)} =_{f(g(f(x)))=_B f(x)} \eta_{f(x)}.$$

Since $\mathsf{ru}(\mathsf{ap}_f(p))\colon \mathsf{ap}_f(p) \cdot \mathsf{refl}_{f(x)} =_{f(g(f(x)))=_B f(x)} \mathsf{ap}_f(p)$, we can take $r :\equiv p$, form the required dependent pair $\langle p, \mathsf{ru}(\mathsf{ap}_f(p)) \cdot q\rangle$.

For the backward map, consider $\tau\colon \langle g \circ f, \eta \triangleright f\rangle \sim \langle \mathsf{id}_A, \mathsf{refl} \triangleright f\rangle$, and let us construct a witness of $\mathsf{rcoh}_f(\langle g, \eta\rangle)$.

According to the definition, we need two homotopies

$$\varepsilon\colon g \circ f \sim \mathsf{id}_A \quad \text{and} \quad \nu\colon f \triangleleft \varepsilon \sim \eta \triangleright f.$$

Evaluating at $x\colon A$ and following Notation 4.2.6, we obtain

$$\tau(x)\colon \langle g(f(x)), \eta_{f(x)}\rangle =_{\mathsf{fib}_f(f(x))} \langle x, \mathsf{refl}_{f(x)}\rangle.$$

Applying the forward direction of Lemma 4.2.3 to $\tau(x)$, we obtain a pair $\langle p, q\rangle$ consisting of two paths

$$p\colon g(f(x)) =_A x \quad \text{and} \quad q\colon \mathsf{ap}_f(p) \cdot \mathsf{refl}_{f(x)} =_{f(g(f(x)))=_B f(x)} \eta_{f(x)}.$$

Thus, we can define

$$\varepsilon(x) :\equiv p \quad \text{and} \quad \nu(x) :\equiv \mathsf{ru}(\mathsf{ap}_f(p))^{-1} \cdot q.$$

Verifying that these maps are quasi-inverses of each other is straightforward. □

**Lemma 4.2.8.**[FE] *If $f\colon A \to B$ is an equivalence, then for any $\langle g, \eta\rangle\colon \mathsf{rinv}(f)$, the type $\mathsf{rcoh}_f(\langle g, \eta\rangle)$ is contractible.*

*Proof.* According to Notation 4.2.6, we have

$$\langle g \circ f, \eta\rangle\colon \prod_{x\colon A} \mathsf{fib}_f(f(x)).$$

Since the fibers $\mathsf{fib}_f$ of $f$ are contractible by Theorem 4.1.4, we deduce from part a) of Theorem 3.4.8 that the Π-type above is contractible.

For the same reasons, $\langle \mathsf{id}_A, \mathsf{refl} \triangleright f\rangle$ is contractible. Hence, by part g) of the same theorem, we deduce that $\langle g \circ f, \eta\rangle \sim \langle \mathsf{id}_A, \mathsf{refl} \triangleright f\rangle$ is contractible. The conclusion follows from Lemma 4.2.7. □

**Theorem 4.2.9.**[FE] *For any $f\colon A \to B$, the type $\mathsf{ishae}(f)$ is a mere proposition.*

*Proof.* By Σ-associativity, Proposition 2.9.13, we have

$$\begin{aligned}\mathsf{ishae}(f) &:\equiv \sum_{g\colon B\to A} \sum_{\eta\colon f\circ g\sim \mathsf{id}_B} \mathsf{rcoh}_f(\langle \mathsf{g}, \eta\rangle) \\ &= \sum_{\omega\colon \mathsf{rinv}(f)} \mathsf{rcoh}_f(\omega).\end{aligned}$$

If $\mathsf{ishae}(f)$ is inhabited, then $\mathsf{rinv}(f)$ is contractible by Proposition 4.1.8, and $\mathsf{rcoh}(f)$ is contractible by Lemma 4.2.8. Thus, the conclusion follows from part d) of Theorem 3.4.8. □

**Corollary 4.2.10.** *For any function $f\colon A \to B$, the type* $\mathsf{isequiv}(f)$ *is a mere proposition.*

*Proof.* By Definitions 2.5.1 and 4.1.5, we have the definitional equality

$$\mathsf{isequiv}(f) \equiv \mathsf{rinv}(f) \times \mathsf{linv}(f).$$

To show that $\mathsf{isequiv}(f)$ is a mere proposition, it suffices to prove that if it is inhabited, it is contractible. If $\mathsf{isequiv}(f)$ is inhabited, $f$ has both a right and a left inverse, which implies that it is quasi-invertible. Hence, by Proposition 4.1.8, both $\mathsf{rinv}(f)$ and $\mathsf{linv}(f)$ are contractible. The conclusion follows from part c) of Theorem 3.4.8. □

## 4.3 Bi-invertible Maps

**Definition 4.3.1.** We say that $f\colon A \to B$ is *bi-invertible* if it has both a left inverse and a right inverse, i.e., if the type

$$\mathsf{biinv}(f) :\equiv \mathsf{linv}(f) \times \mathsf{rinv}(f)$$

is inhabited.

**Theorem 4.3.2.**[FE] *For any $f\colon A \to B$, the type* $\mathsf{biinv}(f)$ *is a mere proposition.*

*Proof.* It suffices to show that, in the case where $\mathsf{biinv}(f)$ is inhabited, it is contractible. If $\mathsf{biinv}(f)$ is inhabited, $f$ has both a left and a right inverse. This implies that $f$ is quasi-invertible. Hence, by Proposition 4.1.8, both $\mathsf{rinv}(f)$ and $\mathsf{linv}(f)$ are contractible. Thus, the conclusion follows from part c) of Theorem 3.4.8.

□

**Remark 4.3.3.** While $\mathsf{ishae}(f)$ requires the addition of a 2-dimensional coherence homotopy to reach the status of a mere proposition, $\mathsf{biinv}(f)$ achieves the same using only 1-dimensional homotopies.

**Corollary 4.3.4.**[FE] *Let $f\colon A \to B$ be a function. Then* $\mathsf{biinv}(f) \simeq \mathsf{ishae}(f)$.

*Proof.* Given that both types are mere propositions, by Lemma 3.2.4, it suffices to show a forward and a backward map.

By Theorem 4.2.2, there is a map $\mathsf{qinv}(f) \to \mathsf{ishae}(f)$. Composing this with the trivial map $\mathsf{biinv}(f) \to \mathsf{qinv}(f)$, one obtains a forward map.

□

## 4.4 Contractible Fibers

Recall from Definition 3.4.15 that a map is contractible when its fibers are contractible. As observed in Remark 4.2.4, the fibers of a half-adjoint equivalence are contractible, which means that there is a map $\mathsf{ishae}(f) \to \mathsf{isContrMap}(f)$.

**Theorem 4.4.1.** *For any $f\colon A \to B$, we have $\mathsf{isContrMap}(f) \to \mathsf{ishae}(f)$.*

*Proof.* By abuse of notation,

$$\mathsf{isContrMap}(f) \equiv \prod_{y\colon B} \sum_{\langle a,p\rangle\colon \mathsf{fib}_f(y)} \prod_{\langle x,q\rangle\colon \mathsf{fib}_f(y)} \langle a,p\rangle =_{\mathsf{fib}_f(y)} \langle x,q\rangle.$$

On the other hand,

$$\mathsf{ishae}(f) \equiv \sum_{g\colon B\to A} \sum_{\eta\colon f\circ g\sim \mathsf{id}_B} \sum_{\vartheta\colon g\circ f\sim \mathsf{id}_A} f \triangleleft \vartheta \sim \eta \triangleright f.$$

Therefore, we can define the map as

$$\begin{aligned} \mathsf{isContrMap}(f) &\to \mathsf{ishae}(f) \\ \alpha &\mapsto \langle g, \langle \eta, \langle \vartheta, \tau\rangle\rangle\rangle, \end{aligned}$$

where

$$\begin{aligned} g(y) &:\equiv \pi_1(\pi_1(\alpha(y))) \\ &\equiv \pi_1(\langle a,p\rangle) \\ &\equiv a, \\ \eta(y) &:\equiv \pi_2(\pi_1(\alpha(y))) \\ &\equiv \pi_2(\langle a,p\rangle) \\ &\equiv p. \end{aligned}$$

To construct $\vartheta(x)\colon g(f(x)) =_A x$ and $\tau(x)\colon \mathsf{ap}_f(\vartheta(x)) =_{f(g(f(x)))=_B f(x)} \eta(f(x))$, set

$$a :\equiv \pi_1(\pi_1(\alpha(f(x)))) \quad \text{and} \quad p :\equiv \pi_2(\pi_1(\alpha(f(x)))),$$

and observe that $a \equiv g(f(x))$ and $p \equiv \eta(f(x))$.

Evaluating the contraction homotopy $\pi_2(\alpha(f(x)))$ at the canonical element $\langle x, \mathsf{refl}_{f(x)}\rangle$ yields a path

$$s\colon \langle a,p\rangle =_{\mathsf{fib}_f(f(x))} \langle x, \mathsf{refl}_{f(x)}\rangle.$$

By Lemma 4.2.3, we can use $s$ to construct a dependent pair $\langle r,q\rangle$, where

$$r\colon a =_A x \quad \text{and} \quad q\colon \mathsf{ap}_f(r) \cdot \mathsf{refl}_{f(x)} =_{f(a)=_B f(x)} p.$$

After replacing $q$ with $\mathsf{ru}(\mathsf{ap}_f(r))^{-1} \cdot q$, we may assume that

$$r\colon g(f(x)) =_A x \quad \text{and} \quad q\colon \mathsf{ap}_f(r) =_{f(a)=_B f(x)} \eta(f(x)),$$

which we can use to define $\vartheta(x) :\equiv r$ and $\tau(x) :\equiv q$.

□

**Lemma 4.4.2.** *For any type $A\colon \mathcal{U}$, the types $\mathsf{isProp}(A)$ and $A \to \mathsf{isContr}(A)$ are logically equivalent.*

*Proof.* For the forward map, consider the assignment

$$\begin{aligned} \mathsf{isProp}(A) &\to (A \to \mathsf{isContr}(A)) \\ p &\mapsto \lambda x.\, \langle x, p(x)\rangle, \end{aligned}$$

which is well-defined because

$$p\colon \prod_{x\colon A} \prod_{y\colon A} x =_A y.$$

For the backward map, fix $f\colon A \to \mathsf{isContr}(A)$. To produce a witness of $\mathsf{isProp}(A)$, we must construct a path $x =_A y$ for any given elements $x, y\colon A$.

Since $x\colon A$, evaluating $f$ at $x$ yields a proof that $A$ is contractible. Therefore, $c :\equiv \pi_1(f(x))$ is a center of contraction and

$$\pi_2(f(x))\colon \prod_{z\colon A} c =_A z.$$

Hence,

$$\underbrace{\pi_2(f(x))(x)^{-1}}_{x=_A c} \cdot \underbrace{\pi_2(f(x))(y)}_{c=_A y}\colon x =_A y$$

is the required path. □

**Lemma 4.4.3.**[WE] *For any $f\colon A \to B$, the type $\mathsf{isContrMap}(f)$ is a mere proposition.*

*Proof.* By Lemma 4.4.2, to prove that $\mathsf{isContrMap}(f)$ is a mere proposition, we assume that there is an element

$$c\colon \mathsf{isContrMap}(f).$$

By definition,

$$\mathsf{isContrMap}(f) \equiv \prod_{y\colon B} \mathsf{isContr}(\mathsf{fib}_f(y)).$$

Hence, $c(y)$ is a proof that $\mathsf{fib}_f(y)$ is contractible for every $y\colon B$.

By Theorem 3.4.5, the type $\mathsf{isContr}(\mathsf{fib}_f(y))$ is a mere proposition for each $y\colon B$.

Since $\mathsf{isContr}(\mathsf{fib}_f(y))$ is both a mere proposition and inhabited by $c(y)$, it is contractible for every $y\colon B$.

We can now apply the weak function extensionality axiom to the family of types $P(y) \equiv \mathsf{isContr}(\mathsf{fib}_f(y))$. Since each fiber $P(y)$ is contractible, the product type $\prod_{y\colon B} P(y)$ is contractible.

Because this product type is definitionally equal to $\mathsf{isContrMap}(f)$, we conclude that $\mathsf{isContrMap}(f)$ is contractible, hence, is a mere proposition.

□

**Theorem 4.4.4.**[WE] *For any $f\colon A \to B$, we have $\mathsf{isContrMap}(f) \simeq \mathsf{ishae}(f)$.*

*Proof.* This follows from the previous lemma, Lemma 3.2.4, and the fact that we have maps $\mathsf{isContrMap}(f) \to \mathsf{ishae}(f)$ and $\mathsf{ishae}(f) \to \mathsf{isContrMap}(f)$.

□

**Corollary 4.4.5.**[WE] *Let $f\colon A \to B$ be a map, and suppose we have another map $B \to \mathsf{isequiv}(f)$. Then $f$ is an equivalence.*

*Proof.* By the theorem, to prove that $f$ is an equivalence, it suffices to construct a witness of

$$\mathsf{isContrMap}(f) \equiv \prod_{y\colon B} \mathsf{isContr}(\mathsf{fib}_f(y)).$$

Fix $y\colon B$. If $h\colon B \to \mathsf{isContrMap}(f)$ is the given map, then $h(y)\colon \mathsf{isContrMap}(f)$. In particular,

$$h(y)(y)\colon \mathsf{isContr}(\mathsf{fib}_f(y)).$$

Thus, $\lambda y.\, h(y)(y)$ is exactly the required witness of $\mathsf{isContrMap}(f)$.

□

## 4.5 Surjections and Embeddings

**Definition 4.5.1.** Let $f\colon A \to B$. We say that $f$ is

- *surjective* (or a *surjection*) if $\|\mathsf{fib}_f(b)\|$ is inhabited for every $b\colon B$;
- an *embedding* if for every $x, y\colon A$ the function

$$\mathsf{ap}_f\colon (x =_A y) \to (f(x) =_B f(y))$$

  induces an equivalence.

**Remarks 4.5.2.** Suppose that $A$ and $B$ are sets, and let $f\colon A \to B$ be a function.

- If $f$ is a surjection, the hypothesis that every fiber is merely inhabited is expressed by the type $\prod_{b:\,B}\|\mathsf{fib}_f(b)\|$. In contrast, a retraction requires an explicit, constructive selection of preimages, corresponding to the untruncated product $\prod_{b:\,B}\mathsf{fib}_f(b)$.

  Since we cannot extract the explicit data required to build a retraction from a truncated type, AC (for sets) bridges this exact gap by asserting the implication

  $$\Big(\prod_{b:\,B}\|\mathsf{fib}_f(b)\|\Big) \to \Big\|\prod_{b:\,B}\mathsf{fib}_f(b)\Big\|.$$

  This states precisely that if every fiber of a function between sets is merely inhabited, then the type of its retractions is merely inhabited as well.

  It should be noted that for every $y\colon B$, the fiber $\mathsf{fib}_f(y)$ is a set. This follows from part e) of Theorem 3.1.2, because $A$ is a set and, since $B$ is a set, the identity type $f(x) =_B y$ is a mere proposition (and hence a set) for any $x\colon A$.

- By Lemma 3.2.4, $f$ is an embedding if, and only if, the function type

  $$(f(x) =_B f(y)) \to (x =_A y)$$

  is inhabited.

**Lemma 4.5.3.** *If $f\colon A \to B$ is an embedding, then its fibers are mere propositions.*

*Proof.* Let $b\colon B$, and let $\langle x, p\rangle$ and $\langle y, q\rangle$ be two elements in $\mathsf{fib}_f(b)$. To show that $\mathsf{fib}_f(b)$ is a mere proposition, it suffices to prove that the identity type $\langle x, p\rangle =_{\mathsf{fib}_f(b)} \langle y, q\rangle$ is contractible. By Lemma 4.2.3, we have an equivalence

$$\big(\langle x, p\rangle =_{\mathsf{fib}_f(b)} \langle y, q\rangle\big) \simeq \sum_{r:\,x=_A y} \mathsf{ap}_f(r)\cdot q =_{f(x)=_B b} p.$$

Therefore,

$$\begin{aligned}\big(\langle x, p\rangle =_{\mathsf{fib}_f(b)} \langle y, q\rangle\big) &\simeq \sum_{r:\,x=_A y} \mathsf{ap}_f(r) =_{f(x)=_B f(y)} p\cdot q^{-1}\\ &\equiv \mathsf{fib}_{\mathsf{ap}_f}(p\cdot q^{-1}).\end{aligned}$$

Since $\mathsf{ap}_f$ is an equivalence, it has contractible fibers.

□

**Theorem 4.5.4.**[FE] *A function $f\colon A \to B$ induces an equivalence if, and only if, it is both surjective and an embedding.*

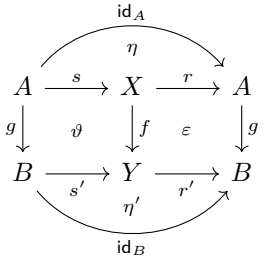
*Proof.* If $f$ induces an equivalence, it is a surjection because, by Remark 4.2.4, its fibers are contractible, hence $\|\mathsf{fib}_f(b)\|$ is contractible and inhabited by its center. It is an embedding because, by Proposition 2.1.1, $\mathsf{ap}_{f^{-1}}$ is a quasi-inverse of $\mathsf{ap}_f$.

To prove that $f$ is an equivalence, by Theorem 4.4.4, it suffices to show that its fibers are contractible. By Lemma 4.5.3, the fibers of $f$ are mere propositions. Therefore, the canonical map $\mathsf{squash}\colon \mathsf{fib}_f(b) \to \|\mathsf{fib}_f(b)\|$ has a quasi-inverse. Since $f$ is a surjection, it follows that $\mathsf{fib}_f(b)$ is inhabited, hence contractible by Lemma 3.4.3.

□

## 4.6 Closure Properties of Equivalences

**Definition 4.6.1.** A function $g\colon A \to B$ is said to be a *retract* of a function $f\colon X \to Y$ if there is a diagram




$$\begin{aligned} &\eta : r \circ s \sim \mathsf{id}_A \\ &\eta'\colon r' \circ s' \sim \mathsf{id}_B \\ &\vartheta : f \circ s \sim s' \circ g \\ &\varepsilon : g \circ r \sim r' \circ f \end{aligned} \tag{4.1}$$

and a homotopy $\tau$ witnessing the commutativity of the square

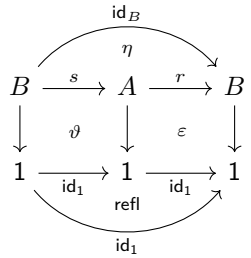
$$\begin{array}{ccc} g \circ r \circ s & \xrightarrow{\varepsilon \triangleright s} & r' \circ f \circ s \\ {\scriptstyle g \triangleleft \eta}\downarrow & \tau & \downarrow{\scriptstyle r' \triangleleft \vartheta} \\ g & \xrightarrow[(\eta' \triangleright g)^{-1}]{} & r' \circ s' \circ g. \end{array} \tag{4.2}$$

**Proposition 4.6.2.** *Let $A$ and $B$ be types. Then $B$ is a retract of $A$ if, and only if, the constant map $B \to \mathsf{1}$ is a retract of $A \to \mathsf{1}$.*

*Proof.* Suppose that $B$ is a retract of $A$. By Definition 3.4.9, there exist a section $s\colon B \to A$, a retraction $r\colon A \to B$, and a homotopy $\eta\colon r \circ s \sim \mathsf{id}_B$.

The data for the map retraction translates into the following diagram, which

commutes trivially:




where $\vartheta$ and $\varepsilon$ are the constant homotopies defined by $b \mapsto \mathsf{refl}_\star$ and $a \mapsto \mathsf{refl}_\star$, respectively.

The coherence condition (4.2) is satisfied automatically since all paths in $\mathbf{1}$ are trivially equal by contractibility. Thus, by Definition 4.6.1, the map $B \to \mathbf{1}$ is a retract of $A \to \mathbf{1}$.

Conversely, if $B \to \mathbf{1}$ is a retract of $A \to \mathbf{1}$, the retraction data inherently provides maps $s\colon B \to A$ and $r\colon A \to B$, along with a homotopy $\eta\colon r \circ s \sim \mathsf{id}_B$. This is precisely the data required by Definition 3.4.9 to establish that $B$ is a retract of $A$. □

**Lemma 4.6.3.** *Suppose that a function $g\colon A \to B$ is a retract of a function $f\colon X \to Y$, and let $s'\colon B \to Y$ be the section provided by* Definition 4.6.1. *Then for every $b\colon B$, the fiber $\mathsf{fib}_g(b)$ is a retract of the fiber $\mathsf{fib}_f(s'(b))$.*

*Proof.* Fix $b\colon B$. To define the forward function between the fibers, consider

$$\sigma\colon \mathsf{fib}_g(b) \to \mathsf{fib}_f(s'(b)), \qquad \langle a, p\rangle \mapsto \langle s(a), \vartheta(a) \cdot \mathsf{ap}_{s'}(p)\rangle.$$

This is well-defined because $\vartheta(a)\colon f(s(a)) =_Y s'(g(a))$ and $p\colon g(a) =_B b$. For the backward map, consider

$$\rho\colon \mathsf{fib}_f(s'(b)) \to \mathsf{fib}_g(b), \qquad \langle x, q\rangle \mapsto \langle r(x), \varepsilon(x) \cdot \mathsf{ap}_{r'}(q) \cdot \eta'(b)\rangle,$$

which is well-defined because

$$\begin{aligned} q&\colon f(x) =_Y s'(b),\\ \mathsf{ap}_{r'}(q)&\colon r'(f(x)) =_B r'(s'(b)),\\ \eta'(b)&\colon r'(s'(b)) =_B b,\\ \varepsilon(x)&\colon g(r(x)) =_B r'(f(x)). \end{aligned}$$

It remains to be shown that $\rho \circ \sigma \sim \mathsf{id}_{\mathsf{fib}_g(b)}$.

Take $\langle a, p\rangle\colon \mathsf{fib}_g(b)$. The first component of $\rho \circ \sigma(\langle a, p\rangle)$ is $r(s(a))$. The second

component, which we will call $p'$, yields the following computation:

$$\begin{aligned}
p' &\equiv \varepsilon(s(a)) \cdot \mathsf{ap}_{r'}(\vartheta(a) \cdot \mathsf{ap}_{s'}(p)) \cdot \eta'(b) \\
&=_{g(r(s(a)))=_B b} \varepsilon \triangleright s(a) \cdot r' \triangleleft \vartheta(a) \cdot \mathsf{ap}_{r'}(\mathsf{ap}_{s'}(p)) \cdot \eta'(b) \\
&=_{g(r(s(a)))=_B b} (\varepsilon \triangleright s \cdot r' \triangleleft \vartheta)(a) \cdot \mathsf{ap}_{r' \circ s'}(p) \cdot \eta'(b) \\
&=_{g(r(s(a)))=_B b} (g \triangleleft \eta \cdot (\eta' \triangleright g)^{-1})(a) \cdot \mathsf{ap}_{r' \circ s'}(p) \cdot \eta'(b) \qquad ; \text{ by } \tau \\
&=_{g(r(s(a)))=_B b} \mathsf{ap}_g(\eta(a)) \cdot \eta'(g(a))^{-1} \cdot \mathsf{ap}_{r' \circ s'}(p) \cdot \eta'(b).
\end{aligned}$$

The naturality equation (2.2) of the homotopy $\eta' \colon r' \circ s' \sim \mathsf{id}_B$ applied to $p \colon g(a) =_B b$ yields

$$\mathsf{ap}_{r' \circ s'}(p) \cdot \eta'(b) =_{r'(s'(g(a)))=_B b} \eta'(g(a)) \cdot p.$$

Substituting this into the expression for $p'$ above gives:

$$p' =_{g(r(s(a)))=_B b} \mathsf{ap}_g(\eta(a)) \cdot p.$$

Given that $\eta(a) \colon r(s(a)) =_A a$, the equality $\langle r(s(a)), p' \rangle =_{\mathsf{fib}_g(b)} \langle a, p \rangle$ is a direct consequence of Lemma 4.2.3. Hence, $\rho \circ \sigma(\langle a, p \rangle) =_{\mathsf{fib}_g(b)} \langle a, p \rangle$.

□

**Theorem 4.6.4.** *A retract of an equivalence is an equivalence.*

*Proof.* Let $g$ be a retract of an equivalence $f$. By Theorem 4.1.4, every fiber of $f$ is contractible. Furthermore, Lemma 4.6.3 establishes that each fiber of $g$ is a retract of a corresponding fiber of $f$. Since retracts of contractible types are themselves contractible by Proposition 3.4.10, it follows that all fibers of $g$ are contractible. Thus, $g$ is an equivalence by Theorem 4.4.1.

□

**Definition 4.6.5.** A type family $P \colon A \to \mathcal{U}$ defines the *fibration* over $A$ given by the projection

$$\pi_1 \colon \Big(\sum_{x \colon A} P(x)\Big) \to A.$$

From this point of view, given two type families $P, Q \colon A \to \mathcal{U}$, we refer to a function

$$f \colon \prod_{x \colon A} P(x) \to Q(x)$$

as a *fiberwise map* or *fiberwise transformation*. Furthermore, the *total* function of such a map is defined as

$$\mathsf{total}(f) :\equiv \lambda w.\, \langle \pi_1(w), f(\pi_1(w), \pi_2(w)) \rangle \colon \Big(\sum_{x \colon A} P(x)\Big) \to \Big(\sum_{x \colon A} Q(x)\Big).$$

The map $f$ is a *fiberwise equivalence* if $f(x) \colon P(x) \to Q(x)$ is an equivalence for every $x \colon A$.

**Theorem 4.6.6.** *Suppose that $f$ is a fiberwise transformation between families $P$ and $Q$ over a type $A$, and let $x : A$ and $v : Q(x)$. Then we have an equivalence*

$$\mathsf{fib}_{\mathsf{total}(f)}(\langle x, v\rangle) \simeq \mathsf{fib}_{f(x)}(v).$$

*Proof.* By definition, the fiber of $\mathsf{total}(f)$ over $\langle x, v\rangle$ is

$$\mathsf{fib}_{\mathsf{total}(f)}(\langle x, v\rangle) \equiv \sum_{w:\, \Sigma_A P} \mathsf{total}(f)(w) =_{\Sigma_A Q} \langle x, v\rangle.$$

By expanding $w$ as a pair $\langle y, u\rangle$ and using the characterization of equality in $\Sigma$-types, we have

$$\begin{aligned}\mathsf{fib}_{\mathsf{total}(f)}(\langle x, v\rangle) &\simeq \sum_{y:\, A} \sum_{u:\, P(y)} \langle y, f(y, u)\rangle =_{\Sigma_A Q} \langle x, v\rangle \\ &\simeq \sum_{y:\, A} \sum_{u:\, P(y)} \sum_{p:\, y =_A x} p_*(f(y, u)) =_{Q(x)} v.\end{aligned}$$

Applying $\Sigma$-commutativity and -associativity (Propositions 2.9.12 and 2.9.13), we obtain

$$\mathsf{fib}_{\mathsf{total}(f)}(\langle x, v\rangle) \simeq \sum_{\langle y,p\rangle:\, \sum_{y:\, A} y =_A x} \sum_{u:\, P(y)} p_*(f(y, u)) =_{Q(x)} v.$$

By Lemma 3.4.11, the based path space $\sum_{y:\, A} y =_A x$ is contractible with center of contraction $\langle x, \mathsf{refl}_x\rangle$. Consequently, the entire $\Sigma$-type is equivalent to evaluating the inner sum at this center:

$$\begin{aligned}\mathsf{fib}_{\mathsf{total}(f)}(\langle x, v\rangle) &\simeq \sum_{u:\, P(x)} (\mathsf{refl}_x)_*(f(x, u)) =_{Q(x)} v \\ &\equiv \sum_{u:\, P(x)} f(x, u) =_{Q(x)} v \\ &\equiv \mathsf{fib}_{f(x)}(v).\end{aligned}$$

□

**Corollary 4.6.7.** *Let $f$ be a fiberwise transformation. Then $f$ is a fiberwise equivalence if, and only if, $\mathsf{total}(f)$ is an equivalence.*

*Proof.* This is a direct consequence of the preceding theorem and Theorem 4.4.1.

□

**Corollary 4.6.8.** *Let $A$ and $B$ be types, and let $\phi : A \simeq B$ be an equivalence. Given type families $P : A \to \mathcal{U}$ and $Q : B \to \mathcal{U}$, along with a family of equivalences*

$$f : \prod_{x:\, A} P(x) \simeq Q(\phi(x)),$$

*there is an equivalence*

$$\sum_{x:\,A} P(x) \simeq \sum_{y:\,B} Q(y)$$

*whose underlying function maps* $\langle x, p\rangle$ *to* $\langle \phi(x), f(x)(p)\rangle$.

*Proof.* By the theorem, the family of equivalences $f$ induces an equivalence of total spaces over $A$,

$$\mathsf{total}(f)\colon \Big(\sum_{x:\,A} P(x)\Big) \simeq \Big(\sum_{x:\,A} Q(\phi(x))\Big).$$

Furthermore, since $\phi\colon A \simeq B$ is an equivalence, reindexing the total space along $\phi$ yields another equivalence,

$$e\colon \Big(\sum_{x:\,A} Q(\phi(x))\Big) \simeq \Big(\sum_{y:\,B} Q(y)\Big),$$

defined by $e(\langle x, q\rangle) :\equiv \langle \phi(x), q\rangle$.

By composition we obtain the equivalence

$$e \circ \mathsf{total}(f)\colon \Big(\sum_{x:\,A} P(x)\Big) \simeq \Big(\sum_{y:\,B} Q(y)\Big).$$

Evaluating this composition at an arbitrary pair $\langle x, p\rangle$ gives

$$(e \circ \mathsf{total}(f))(\langle x, p\rangle) \equiv e(\langle x, f(x)(p)\rangle) \equiv \langle \phi(x), f(x)(p)\rangle,$$

as stated. □

## 4.7 The Object Classifier

In set theory, $\{0, 1\}$ is the *subobject classifier*, representing the set of truth values. Given a base set $A$ and a subset $S \subseteq A$, this subset is perfectly described by its characteristic function $\chi_S\colon A \to \{0, 1\}$. The function $\chi_S$ acts as a classifier: it points to 1 if an element belongs to $S$, and 0 otherwise. The subset $S$ is mathematically recovered as the preimage of 1 under the function $\chi_S$.

In Homotopy Type Theory, the focus expands from classifying subsets to classifying arbitrary type families. Instead of a subset inclusion $S \hookrightarrow A$, we work with a total space $\Sigma_A B$ equipped with the projection map $\pi_1\colon \Sigma_A B \to A$.

Here, the universe $\mathcal{U}$ generalizes $\{0, 1\}$, becoming the *object classifier*. The type family $B\colon A \to \mathcal{U}$ acts precisely like the characteristic function $\chi_S$. Rather than returning a simple truth value, evaluating $B$ at a point $a\colon A$ returns the entire fiber over that point. Just as $\chi_S$ completely determines the subset $S$, the classifying function $B$ completely determines the structure of the fibration defined by the projection map $\pi_1$.

**Proposition 4.7.1.** *Let $A : \mathcal{U}$ be a type and $B : A \to \mathcal{U}$ a type family over $A$. Then, the fiber of $\pi_1 : \Sigma_A B \to A$ over $a : A$ is equivalent to $B(a)$:*

$$\mathsf{fib}_{\pi_1}(a) \simeq B(a).$$

*Proof.*

$$\begin{aligned}
\mathsf{fib}_{\pi_1}(a) &\equiv \sum_{u:\, \Sigma_A B} \pi_1(u) =_A a \\
&\simeq \sum_{x:\, A} \sum_{u:\, B(x)} x =_A a && \text{; Prop. 2.9.13} \\
&\simeq \sum_{x:\, A} \sum_{p:\, x =_A a} B(x) && \text{; Prop. 2.9.12} \\
&\simeq \sum_{\zeta :\, \sum_{x:\, A} x =_A a} B(\pi_1(\zeta)) && \text{; Prop. 2.9.13} \\
&\simeq B(\pi_1(\langle a, \mathsf{refl}_a \rangle)) && \text{; Lem. 3.4.11 \& Thm. 3.4.8 d)} \\
&\equiv B(a).
\end{aligned}$$

□

**Lemma 4.7.2.** *For any function $f : A \to B$, we have*

$$A \simeq \sum_{b:\, B} \mathsf{fib}_f(b).$$

*Proof.* The map that induces the equivalence is defined by

$$\gamma : A \to \sum_{b:\, B} \mathsf{fib}_f(b), \quad a \mapsto \langle f(a), \langle a, \mathsf{refl}_{f(a)} \rangle \rangle.$$

Its quasi-inverse is given by

$$\pi : \left( \sum_{b:\, B} \mathsf{fib}_f(b) \right) \to A, \quad \langle b, \langle x, p \rangle \rangle \mapsto x.$$

Indeed, the compositions compute to

$$a \overset{\gamma}{\mapsto} \langle f(a), \langle a, \mathsf{refl}_{f(a)} \rangle \rangle \overset{\pi}{\mapsto} a$$

and

$$\langle b, \langle x, p \rangle \rangle \overset{\pi}{\mapsto} x \overset{\gamma}{\mapsto} \langle f(x), \langle x, \mathsf{refl}_{f(x)} \rangle \rangle.$$

Since $p : f(x) =_B b$, showing that

$$\langle f(x), \langle x, \mathsf{refl}_{f(x)} \rangle \rangle =_{\Sigma_B \mathsf{fib}_f} \langle b, \langle x, p \rangle \rangle$$

amounts to proving that

$$\mathsf{transport}^{\mathsf{fib}_f}(p, \langle x, \mathsf{refl}_{f(x)}\rangle) =_{\mathsf{fib}_f(b)} \langle x, p\rangle.$$

By applying based path induction on $p$, the goal reduces to the case $b \equiv f(x)$ and $p \equiv \mathsf{refl}_{f(x)}$, which yields the definitional equality

$$\mathsf{transport}^{\mathsf{fib}_f}(\mathsf{refl}_{f(x)}, \langle x, \mathsf{refl}_{f(x)}\rangle) \equiv \langle x, \mathsf{refl}_{f(x)}\rangle.$$

□

**Theorem 4.7.3.**[FE UA] *For any type $B$, there is an equivalence*

$$\chi\colon \Big(\sum_{A:\mathcal{U}} A \to B\Big) \simeq (B \to \mathcal{U}).$$

*Proof.* We define the forward function by

$$\begin{aligned}\chi\colon \Big(\sum_{A:\mathcal{U}} A \to B\Big) &\to (B \to \mathcal{U})\\ \langle A, f\rangle &\mapsto \mathsf{fib}_f,\end{aligned}$$

where $\mathsf{fib}_f :\equiv \lambda b.\, \mathsf{fib}_f(b)$. The backward function is given by

$$\begin{aligned}\chi^{-1}\colon (B \to \mathcal{U}) &\to \Big(\sum_{A:\mathcal{U}} A \to B\Big)\\ g &\mapsto \Big\langle \sum_{b:B} g(b), \pi_1\Big\rangle.\end{aligned}$$

To verify that these maps are mutually inverse, we compute both compositions. Given $g\colon B \to \mathcal{U}$, we have

$$\begin{aligned}(\chi \circ \chi^{-1})(g) &\equiv \chi(\chi^{-1}(g))\\ &\equiv \chi\Big(\Big\langle \sum_{b:B} g(b), \pi_1\Big\rangle\Big)\\ &\equiv \mathsf{fib}_{\pi_1}.\end{aligned}$$

By Proposition 4.7.1, for each $b\colon B$ there is an equivalence $e_b\colon \mathsf{fib}_{\pi_1}(b) \simeq g(b)$. Therefore,

$$\mathsf{ua}(e_b)\colon \mathsf{fib}_{\pi_1}(b) =_{\mathcal{U}} g(b).$$

By function extensionality, this family of paths induces an equality of functions

$$\mathsf{funext}(\lambda b.\, \mathsf{ua}(e_b))\colon \mathsf{fib}_{\pi_1} =_{B\to\mathcal{U}} g,$$

which yields the required path $(\chi \circ \chi^{-1})(g) =_{B\to\mathcal{U}} g$.

On the other hand, given $A : \mathcal{U}$ and $f : A \to B$, we have

$$\begin{aligned}\chi^{-1} \circ \chi(\langle A, f\rangle) &\equiv \chi^{-1}(\mathsf{fib}_f) \\ &\equiv \left\langle \sum_{b:\, B} \mathsf{fib}_f(b), \pi_1 \right\rangle\end{aligned}$$

To complete the proof, we must show that

$$\left\langle \sum_{b:\, B} \mathsf{fib}_f(b), \pi_1 \right\rangle =_{\sum_{A:\, \mathcal{U}} (A \to B)} \langle A, f\rangle.$$

For the first component, we take $p :\equiv \mathsf{ua}(e^{-1})$, where $e : A \simeq \sum_{b:\, B} \mathsf{fib}_f(b)$ is the equivalence established in Lemma 4.7.2. For the second component, we must show that

$$\mathsf{transport}^{\lambda X.\, (X \to B)}(p, \pi_1) =_{A \to B} f.$$

As in the proof of Lemma 4.7.2, let $\gamma$ and $\pi$ be the quasi-inverses inducing the equivalence $e$. Proposition 2.13.5 implies that

$$\mathsf{transport}^{\lambda X.\, (X \to B)}(p, \pi_1) \sim \pi_1 \circ \gamma.$$

Evaluating the RHS at an arbitrary $a : A$, we get

$$\begin{aligned}\pi_1(\gamma(a)) &\equiv \pi_1\big(\langle f(a), \langle a, \mathsf{refl}_{f(a)}\rangle\rangle\big) \\ &\equiv f(a).\end{aligned}$$

Now function extensionality gives the desired equality. □

**Remark 4.7.4.**[UA] Given a type $B : \mathcal{U}$, the total space

$$E :\equiv \sum_{A:\, \mathcal{U}} A \to B$$

is the type-theoretic equivalent of the slice category $\mathbf{Set}_B$.

To understand this correspondence at the level of morphisms, let us consider how mappings between objects are defined in both contexts. As established in § B.3, a morphism in $\mathbf{Set}_B$ between two objects $f : A \to B$ and $f' : A' \to B$ is a function $h : A \to A'$ such that the corresponding triangle commutes, i.e., $f' \circ h = f$. In Homotopy Type Theory, the type of maps over $B$ from $\langle A, f\rangle$ to $\langle A', f'\rangle$ is given precisely by

$$\sum_{h:\, A \to A'} f' \circ h \sim f.$$

Furthermore, the characterization of equality for $\Sigma$-types dictates that a path between $\langle A, f\rangle$ and $\langle A', f'\rangle$ consists of:

- a path $A =_{\mathcal{U}} A'$, which the univalence axiom reduces to an equivalence $h\colon A \simeq A'$, and
- a homotopy $\mathsf{transport}^{\lambda X.\,(X\to B)}(\mathsf{ua}(h), f) \sim f'$, which Proposition 2.13.5 renders equivalent to a homotopy $f \circ h^{-1} \sim f'$ or $f \sim f' \circ h$.

Thus, the type-theoretic structure naturally captures the objects, the general morphisms (via the dependent pair type), and the isomorphisms (via the identity type) of the categorical slice construction.

As discussed in § 2.4.2 Types as Categories, we can view types as categories where the terms are the objects and the identity paths are the morphisms. Functors in this context are functions between types, which act on paths via the $\mathsf{ap}$ operator.

However, when we let the universe $\mathcal{U}$ play the role of **Set**, our correspondence requires a categorical shift in perspective. In **Set**, the morphisms are arbitrary functions, not just bijections. Therefore, when viewing $\mathcal{U}$ as a category, we treat types as objects, functions as morphisms, and equivalences as isomorphisms. Consequently, a type family $P\colon B \to \mathcal{U}$ acts as a functor by mapping:

- objects: a term $b\colon B$ is mapped to the type $P(b)\colon \mathcal{U}$.
- isomorphisms: a path $p\colon b =_B b'$ (which is an isomorphism in the groupoid $B$) is mapped to an equivalence in $\mathcal{U}$. Specifically, $P$ transforms $p$ into the equivalence $\mathsf{idtoeqv}(\mathsf{ap}_P(p))\colon P(b) \simeq P(b')$. By Proposition 2.13.6, the underlying function of this equivalence is exactly the transport map $\mathsf{transport}^P(p, -)\colon P(b) \to P(b')$.

In category theory, a natural transformation between two functors consists of two separate pieces of data:

a) a family of morphisms representing the object-level components, and

b) a proof that these components make the associated naturality squares commute for every morphism in the base category.

Given families $P, Q\colon B \to \mathcal{U}$, a dependent function $\eta\colon \prod_{b:\,B} P(b) \to Q(b)$ provides exactly the object-level data required by part (a). For every $b\colon B$, it gives a function $\eta(b)\colon P(b) \to Q(b)$.

The profound difference in type theory is that part (b) is an intrinsic consequence of path induction. To verify this naturality, we must show that for any path $p\colon b = b'$ and any element $x\colon P(b)$, the evaluation of $\eta(b')$ after transporting $x$ equals the transport of the evaluation of $\eta(b)$ at $x$. This is expressed by the propositional equality

$$\eta(b')(\mathsf{transport}^P(p, x)) = \mathsf{transport}^Q(p, \eta(b)(x)).$$

We can construct a path witnessing this equality by induction on $p$. Assuming $b' \equiv b$ and $p \equiv \mathsf{refl}_b$, the transport operations reduce definitionally to the identity function. The equation then simplifies to

$$\eta(b)(x) = \eta(b)(x),$$

which is trivially inhabited by $\mathsf{refl}_{\eta(b)(x)}$. By path induction, this commutativity holds for all paths $p$.

Therefore, simply providing the dependent function $\eta$ is entirely sufficient. The rules of identity types guarantee that any such function is natural with respect to paths.

Under these considerations it is now clear that Theorem 4.7.3 is the type-theoretic version of the Grothendieck Construction Theorem B.3.6.

**Notation 4.7.5.** *Let $U, V, Q : \mathcal{U}$ be types, and consider functions $g_U : U \to Q$ and $g_V : V \to Q$. The canonical pullback type $U \times_Q V$ is defined as*

$$U \times_Q V :\equiv \sum_{u:\, U} \sum_{v:\, V} g_U(u) =_Q g_V(v),$$

*where $\pi_U : (U \times_Q V) \to U$ is the first projection $\pi_1$, and $\pi_V : (U \times_Q V) \to V$ is the composition $\pi_1 \circ \pi_2$.*

**Lemma 4.7.6.**[FE] *With the preceding notation, the canonical construction $U \times_Q V$, equipped with the projections $\pi_U$ and $\pi_V$, is the pullback of $g_U$ and $g_V$.*

*Proof.* By Exercise 2.19.10, we have to show that, for any $X : \mathcal{U}$, there is an equivalence

$$\big(X \to (U \times_Q V)\big) \simeq \left( \sum_{(u,v):\, X \to (U \times V)} g_U \circ u =_{X \to Q} g_V \circ v \right).$$

By 39. Type-theoretic Axiom of Choice applied twice,

$$\begin{aligned}
\big(X \to (U \times_Q V)\big) &\equiv \prod_{x:\, X} \sum_{u:\, U} \sum_{v:\, V} g_U(u) =_Q g_V(v) \\
&\simeq \sum_{u:\, X \to U} \prod_{x:\, X} \sum_{v:\, V} g_U(u(x)) =_Q g_V(v) \\
&\simeq \sum_{u:\, X \to U} \sum_{v:\, X \to V} \prod_{x:\, X} g_U(u(x)) =_Q g_V(v(x)).
\end{aligned}$$

Then, by function extensionality,

$$\text{RHS} \simeq \sum_{u:\, X \to U} \sum_{v:\, X \to V} g_U \circ u =_{X \to Q} g_V \circ v.$$

Grouping $u$ and $v$ into a single map of type $X \to (U \times V)$, we obtain

$$\text{RHS} \simeq \sum_{(u,v):\, X \to (U \times V)} g_U \circ u =_{X \to Q} g_V \circ v,$$

as required. □

**Theorem 4.7.7.**[FE] *Let $f : A \to B$ be a function. Then the diagram*

$$\begin{array}{ccc} A & \xrightarrow{\vartheta_f} & \mathcal{U}_\bullet \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle \pi_1} \\ B & \xrightarrow[\mathsf{fib}_f]{} & \mathcal{U} \end{array}$$

*is a pullback square, where*

$$\vartheta_f :\equiv \lambda a.\, \langle \mathsf{fib}_f(f(a)), \langle a, \mathsf{refl}_{f(a)} \rangle \rangle.$$

*Proof.* First observe that Lemma 3.4.11 implies that, for any $b : B$, the type

$$C_b :\equiv \sum_{X : \mathcal{U}} \mathsf{fib}_f(b) =_{\mathcal{U}} X$$

is contractible with center $c :\equiv \langle \mathsf{fib}_f(b), \mathsf{refl}_{\mathsf{fib}_f(b)} \rangle$. By associativity of $\Sigma$-types, we have

$$\sum_{X : \mathcal{U}} \sum_{p :\, \mathsf{fib}_f(b) = X} X \simeq \sum_{\omega :\, C_b} \pi_1(\omega).$$

Therefore, by part b) of Theorem 3.4.8, evaluating the family $\pi_1$ at the center of contraction yields the equivalence

$$\sum_{\omega :\, C_b} \pi_1(\omega) \simeq \pi_1(c) \equiv \mathsf{fib}_f(b).$$

It follows that

$$\mathsf{fib}_f(b) \simeq \sum_{X : \mathcal{U}} \sum_{p :\, \mathsf{fib}_f(b) = X} X. \tag{$*$}$$

On the other hand, since the type family

$$\begin{aligned} P :\, (\mathsf{fib}_f(b) =_{\mathcal{U}} X) &\to \mathcal{U} \\ p &\mapsto X \end{aligned}$$

is constant, we can swap its components and obtain the equivalence

$$\begin{aligned} \Big( \sum_{p :\, \mathsf{fib}_f(b) = X} X \Big) &\xrightarrow{\simeq} \sum_{x :\, X} \mathsf{fib}_f(b) =_{\mathcal{U}} X \\ \langle p, x \rangle &\mapsto \langle x, p \rangle. \end{aligned} \tag{$**$}$$

Then,

$$\begin{aligned}
A &\simeq \sum_{b:\,B} \mathsf{fib}_f(b) && \text{; Lem. 4.7.2}\\
&\simeq \sum_{b:\,B} \sum_{X:\mathcal{U}} \sum_{p:\,\mathsf{fib}_f(b)=_{\mathcal{U}} X} X && \text{; } (*)\\
&\simeq \sum_{b:\,B} \sum_{X:\mathcal{U}} \sum_{x:\,X} \mathsf{fib}_f(b) =_{\mathcal{U}} X && \text{; } (**)\\
&\simeq \sum_{b:\,B} \sum_{Y:\mathcal{U}_\bullet} \mathsf{fib}_f(b) =_{\mathcal{U}} \pi_1(Y) && \text{; Def. of } \mathcal{U}_\bullet\\
&\equiv B \times_{\mathcal{U}} \mathcal{U}_\bullet, && \text{; Lem. 4.7.6}
\end{aligned}$$

Let $e\colon A \xrightarrow{\simeq} B \times_{\mathcal{U}} \mathcal{U}_\bullet$ be the function inducing the equivalence above. Following the composition step by step, we obtain

$$\begin{aligned}
a &\mapsto \langle \underbrace{f(a)}_{B}, \underbrace{\langle a, \mathsf{refl}_{f(a)}\rangle}_{\mathsf{fib}_f(f(a))}\rangle\\
&\mapsto \langle \underbrace{f(a)}_{B}, \langle \underbrace{\mathsf{fib}_f(f(a))}_{\mathcal{U}}, \langle \underbrace{\mathsf{refl}_{\mathsf{fib}_f(f(a))}}_{\mathsf{fib}_f(f(a))=_{\mathcal{U}} X}, \underbrace{\langle a, \mathsf{refl}_{f(a)}\rangle\rangle}_{X}\rangle\rangle\rangle\\
&\mapsto \langle \underbrace{f(a)}_{B}, \langle \underbrace{\mathsf{fib}_f(f(a))}_{\mathcal{U}}, \langle \underbrace{\langle a, \mathsf{refl}_{f(a)}\rangle}_{X}, \underbrace{\mathsf{refl}_{\mathsf{fib}_f(f(a))}}_{\mathsf{fib}_f(f(a))=_{\mathcal{U}} X}\rangle\rangle\rangle\\
&\mapsto \langle \underbrace{f(a)}_{B}, \langle \underbrace{\langle \mathsf{fib}_f(f(a)), \langle a, \mathsf{refl}_{f(a)}\rangle\rangle}_{\mathcal{U}_\bullet}, \underbrace{\mathsf{refl}_{\mathsf{fib}_f(f(a))}}_{\mathsf{fib}_f(f(a))=_{\mathcal{U}} \pi_1(Y)}\rangle\rangle.
\end{aligned}$$

Therefore, we get homotopies $f \sim \pi_1 \circ e$ and $\vartheta_f \sim \pi_1 \circ \pi_2 \circ e$. □

**Remark 4.7.8.** The pullback square of the universal fibration $\pi_1\colon \mathcal{U}_\bullet \to \mathcal{U}$ establishes a direct link between the geometric universal property of limits and the logical equivalence of the Grothendieck construction. Suppose two functions $f, g\colon A \to B$ have propositionally equal fibers, meaning $\mathsf{fib}_f =_{B\to\mathcal{U}} \mathsf{fib}_g$. They therefore share the same classifying map $\chi\colon B \to \mathcal{U}$ defined by their common fibers.

Because both $f$ and $g$ can be recovered as pullbacks of the universal bundle $\pi_1\colon \mathcal{U}_\bullet \to \mathcal{U}$ along $\chi$, they both exhibit the domain $A$ as the limit of the exact same diagram. By the universal property of pullbacks, limits are unique up to equivalence. This guarantees the existence of a canonical automorphism $\alpha\colon A \simeq A$ over $B$, yielding the homotopy $f \sim g \circ \alpha$.

This geometric uniqueness perfectly mirrors the algebraic equality within the Grothendieck construction. Under the equivalence

$$\Big(\sum_{X:\mathcal{U}} X \to B\Big) \simeq (B \to \mathcal{U}),$$

mapping both functions to the same family implies the equality $\langle A, f\rangle = \langle A, g\rangle$ as points in the $\Sigma$-type. Equality in this total space consists of an equivalence $\alpha\colon A \simeq A$ in the first coordinate and a path $f =_{A\to B} g \circ \alpha^{-1}$ in the second.

Thus, the universal property of the pullback acts as the local geometric mechanism that computes the global structural equivalence between ordinary functions mapping into a base type $B$ and dependent type families indexed by $B$.

## 4.8 Univalence Implies Function Extensionality

In this section we prove that UA $\Rightarrow$ WE $\Rightarrow$ FE.

**Remark 4.8.1.** The following lemma is proved without using the function extensionality axiom.

**Lemma 4.8.2.**[UA] *Assuming the univalence axiom, for any types $A, B, X\colon \mathcal{U}$ and any equivalence $e\colon A \simeq B$, post-composition with $e$ induces an equivalence*

$$(X \to A) \simeq (X \to B).$$

*Proof.* By the univalence axiom, the map $\mathsf{idtoeqv}\colon (A =_{\mathcal{U}} B) \to (A \simeq B)$ is an equivalence. Therefore, to prove a property for an arbitrary equivalence $e\colon A \simeq B$, it suffices to prove it for an equivalence of the form $\mathsf{idtoeqv}(p)$ for some path $p\colon A =_{\mathcal{U}} B$.

By path induction on $p$, we may assume that $B \equiv A$ and $p \equiv \mathsf{refl}_A$. Under this assumption, the equivalence $e \equiv \mathsf{idtoeqv}(\mathsf{refl}_A)$ is the identity equivalence on $A$, whose underlying function is $\mathsf{id}_A\colon A \to A$.

For any function $h\colon X \to A$, post-composition with the underlying map of $e$ yields

$$\mathsf{id}_A \circ h \equiv h.$$

Thus, the post-composition map is definitionally equal to the identity function $\mathsf{id}_{X\to A}$. Since the identity function is an equivalence, the proof is complete.

□

**Proposition 4.8.3.**[UA] *Let $P\colon A \to \mathcal{U}$ be a family of contractible types. Then the projection $\pi_1\colon \Sigma_A P \to A$ is an equivalence. If $\mathcal{U}$ is univalent, then post-composition with $\pi_1$ gives an equivalence*

$$(A \to \Sigma_A P) \simeq (A \to A).$$

*Proof.* By Proposition 4.7.1, given $x\colon A$, we have an equivalence

$$\mathsf{fib}_{\pi_1}(x) \simeq P(x).$$

In particular, $\mathsf{fib}_{\pi_1}(x)$ is contractible itself by Lemma 3.4.3. Thus, Theorem 4.4.1 implies that $\pi_1$ induces an equivalence. The second conclusion is a direct consequence of Lemma 4.8.2. □

**Theorem 4.8.4.**[UA] *The univalence axiom implies the weak function extensionality principle.*

*Proof.* Suppose that $\mathcal{U}$ is univalent, and let $P\colon A \to \mathcal{U}$ be a family of contractible types. We must show that the dependent product $\prod_{x\colon A} P(x)$ is contractible.

Let

$$\alpha\colon (A \to \Sigma_A P) \simeq (A \to A)$$

be the equivalence of Proposition 4.8.3. We will show that the product is a retract of $\mathsf{fib}_\alpha(\mathsf{id}_A)$.

Define the maps

$$\sigma\colon \Big(\prod_{x\colon A} P(x)\Big) \to \mathsf{fib}_\alpha(\mathsf{id}_A)$$
$$\sigma(f) :\equiv \langle \lambda x.\, \langle x, f(x)\rangle, \mathsf{refl}_{\mathsf{id}_A}\rangle,$$

and

$$\rho\colon \mathsf{fib}_\alpha(\mathsf{id}_A) \to \prod_{x\colon A} P(x)$$
$$\rho(\langle g, p\rangle) :\equiv \lambda x.\, \mathsf{happly}(p, x)_*(\pi_2(g(x))).$$

Evaluating their composition yields

$$\begin{aligned}
\rho(\sigma(f)) &\equiv \rho(\langle \lambda x.\, \langle x, f(x)\rangle, \mathsf{refl}_{\mathsf{id}_A}\rangle) \\
&\equiv \lambda x.\, \mathsf{happly}(\mathsf{refl}_{\mathsf{id}_A}, x)_*(f(x)) \\
&\equiv \lambda x.\, (\mathsf{refl}_{\mathsf{id}_A(x)})_*(f(x)) && \text{; Eq. (2.3)} \\
&\equiv \lambda x.\, f(x) \\
&\equiv f,
\end{aligned}$$

as desired. Since the fiber is contractible by Remark 4.2.4, the result follows from Proposition 3.4.10.

□

**Theorem 4.8.5.**[WE] *The weak function extensionality principle implies the function extensionality axiom.*

*Proof.* Let

$$\Pi_A P :\equiv \prod_{x\colon A} P(x).$$

Fix $f : \Pi_A P$ and define

$$\gamma : \Big( \sum_{g : \Pi_A P} f =_{\Pi_A P} g \Big) \to \sum_{g : \Pi_A P} f \sim g$$
$$\langle g, p \rangle \mapsto \langle g, \mathsf{happly}(p) \rangle.$$

By Lemma 3.4.11, the domain type of this map is contractible. For the codomain type we can apply 39. Type-theoretic Axiom of Choice and obtain an equivalence

$$\sum_{g : \Pi_A P} f \sim g \equiv \sum_{g : \Pi_A P} \prod_{x : A} f(x) =_{P(x)} g(x)$$
$$\simeq \prod_{x : A} \sum_{u : P(x)} f(x) =_{P(x)} u.$$

Note that the inner $\Sigma$-type is contractible by the same lemma as before. From weak function extensionality, it follows that the outer $\Pi$-type is contractible as well. Hence the codomain type of $\gamma$ is contractible. Using Lemmas 3.4.3 and 3.2.4, we see that $\gamma$ induces an equivalence.

Consider the fiberwise transformation

$$h : \prod_{g : \Pi_A P} \big( (f =_{\Pi_A P} g) \to (f \sim g) \big)$$

defined by $h(g) :\equiv \mathsf{happly}(f, g)$. According to Definition 4.6.5,

$$\gamma \equiv \mathsf{total}(h).$$

By Theorem 4.6.6, for $\eta : f \sim g$, we have

$$\mathsf{fib}_\gamma(\langle g, \eta \rangle) \simeq \mathsf{fib}_{h(g)}(\eta).$$

Since the fibers of $\gamma$ are contractible by Theorem 4.4.4, we deduce that the fibers of $h(g)$ are contractible. Hence, by the same theorem, $h(g)$ induces an equivalence.

□

**Remark 4.8.6.** To facilitate the verification that the deduction of function extensionality from the univalence axiom is not circular, we provide the combined dependency tree for the proofs of Theorems 4.8.4 and 4.8.5. Nodes rendered in gray indicate repeated dependencies. Note also that there is no need to list results previous to section § 2.11.

- ◇ **Theorem 4.8.4** (UA ⇒ WE)
  - ▷ Proposition 4.8.3 (UA)
    - ⋆ Proposition 4.7.1

- ∘ Lemma 3.4.11
  - Proposition 2.11.6
- ∘ Theorem 3.4.8 (part d)
  - Theorem 3.4.5 (not used in part d)
  - Theorem 3.2.15 (part d)
  - Lemma 3.4.3
    - Lemma 3.2.3
- ⋆ Lemma 3.4.3
  - ∘ Lemma 3.2.3
- ⋆ Theorem 4.4.1
  - ∘ Lemma 4.2.3
    - Proposition 2.11.6
- ⋆ Lemma 4.8.2 (UA)

▷ Equation (2.3)
▷ Equation (2.1)
▷ Remark 4.2.4

- ∗ Theorem 4.1.4
  - ∘ Corollary 4.1.3
    - Proposition 4.1.2
      - Lemma 4.1.1
  - ∘ Proposition 2.11.6

▷ Proposition 3.4.10

It is easy to check that none of these results depend on WE or FE.

♦ **Theorem 4.8.5** (WE ⇒ FE)

- ▷ Lemma 3.4.11
- ▷ Lemma 3.4.3
  - Lemma 3.2.3
- ▷ Lemma 3.2.4
- ▷ Definition 4.6.5
- ▷ Theorem 4.6.6
  - ∘ Lemma 3.4.11
- ▷ Theorem 4.4.4 (WE)

As we can see, the dependency on WE happens in Theorem 4.4.4.[1]

## 4.9 Exercises

**Exercise 4.9.1.**[FE] *Consider the type of "two-sided adjoint equivalence data" for* $f : A \to B$*,*

$$\sum_{g:\, B\to A} \; \sum_{\eta:\, f\circ g \sim \mathsf{id}_B} \; \sum_{\vartheta:\, g\circ f \sim \mathsf{id}_A} (f \triangleleft \vartheta \sim \eta \triangleright f) \times (g \triangleleft \eta \sim \vartheta \triangleright g).$$

[1] For further revision see § A.1.

*By* Proposition 4.1.2*, we know that if $f$ is an equivalence, then this type is inhabited. Give a characterization of this type analogous to* Proposition 4.1.11*.*

*Can you give an example showing that this type is not generally a mere proposition? (This will be easier after* Chapter 6*.)*

*Solution.* Recall that

$$\begin{aligned}\mathsf{rcoh}_f &: \mathsf{rinv}(f) \to \mathcal{U} \\ &\langle g, \eta \rangle \mapsto \sum_{\vartheta :\, g \circ f \sim \mathsf{id}_A} f \triangleleft \vartheta \sim \eta \triangleright f.\end{aligned}$$

Let $G$ denote the target type. By $\Sigma$-associativity, we can expand $G$ as

$$G \equiv \sum_{\langle g,\eta \rangle :\, \mathsf{rinv}(f)} \; \sum_{\vartheta :\, g \circ f \sim \mathsf{id}_A} (f \triangleleft \vartheta \sim \eta \triangleright f) \times (g \triangleleft \eta \sim \vartheta \triangleright g).$$

Consider the type of right-half-adjoint equivalence data,

$$G' :\equiv \sum_{\langle g,\eta \rangle :\, \mathsf{rinv}(f)} \mathsf{rcoh}_f(\langle g, \eta \rangle).$$

There is an equivalence

$$G \simeq \sum_{\langle \langle g,\eta \rangle, \langle \vartheta, \tau \rangle \rangle :\, G'} g \triangleleft \eta \sim \vartheta \triangleright g,$$

induced by the canonical mapping

$$\big\langle \langle g, \eta \rangle, \langle \vartheta, (\tau, \sigma) \rangle \big\rangle \mapsto \big\langle \langle \langle g, \eta \rangle, \langle \vartheta, \tau \rangle \rangle, \sigma \big\rangle.$$

As we saw in the proof of Theorem 4.2.9, $G' \simeq \mathsf{ishae}(f)$. In particular, if $f$ is an equivalence, then $G'$ is contractible. By evaluating the sum at the center of contraction $\langle \langle g, \eta \rangle, \langle \vartheta, \tau \rangle \rangle$ of $G'$, we obtain the equivalence

$$G \simeq (g \triangleleft \eta \sim \vartheta \triangleright g).$$

Thus, $G$ is equivalent to the space of choices for this secondary coherence condition—a "loose end," as described in Remark 4.2.1.

Regarding the second part of the exercise, a counterexample can be provided by any type whose identity type is not always a set. Since we have not yet seen such a type, we postpone a complete solution to a later chapter.

□

**Exercise 4.9.2.**[FE] *Show that for any types $A, B : \mathcal{U}$, the following type is equivalent to $A \simeq B$.*

$$\sum_{R :\, A \to B \to \mathcal{U}} \Big( \prod_{a :\, A} \mathsf{isContr}\Big( \sum_{b :\, B} R(a,b) \Big) \Big) \times \Big( \prod_{b :\, B} \mathsf{isContr}\Big( \sum_{a :\, A} R(a,b) \Big) \Big).$$

*Can you extract from this a definition of a type satisfying the three desiderata of $\mathsf{isequiv}(f)$? Given a type family $\mathsf{E} : (A \to B) \to \mathcal{U}$, these are:*

i) *For each* $f\colon A \to B$ *there is a function* $\mathsf{qinv}(f) \to \mathsf{E}(f)$.

ii) *For each* $f\colon A \to B$ *there is a function* $\mathsf{E}(f) \to \mathsf{qinv}(f)$.

iii) $\mathsf{E}(f)$ *is a mere proposition.*

*Solution.* With abbreviated notation we can rewrite the target type as

$$\sum_{R\colon A\to B\to\mathcal{U}} \Big(\prod_{a\colon A} \mathsf{isContr}(\Sigma_B R(a,-))\Big) \times \Big(\prod_{b\colon B} \mathsf{isContr}(\Sigma_A R(-,b))\Big).$$

Expanding the definition of contractibility, the first part of the Cartesian product is given by

$$\begin{aligned}\prod_{a\colon A} \mathsf{isContr}(\Sigma_B R(a,-)) &\equiv \prod_{a\colon A} \sum_{u\colon \Sigma_B R(a,-)} \prod_{v\colon \Sigma_B R(a,-)} u = v \\ &\simeq \prod_{a\colon A} \sum_{b\colon B} \sum_{r\colon R(a,b)} \prod_{v\colon \Sigma_B R(a,-)} \langle b, r\rangle = v \quad ;\ \Sigma\text{-assoc.}.\end{aligned}$$

Applying 39. Type-theoretic Axiom of Choice, the RHS satisfies

$$\text{RHS} \simeq \sum_{f\colon A\to B} \prod_{a\colon A} \sum_{r\colon R(a,f(a))} \prod_{v\colon \Sigma_B R(a,-)} \langle f(a), r\rangle = v, \tag{1}$$

which allows us to extract, via the first projection of the outer $\Sigma$-type, a function $f\colon A \to B$ and a proof that the type

$$\prod_{a\colon A} R(a, f(a))$$

is inhabited.[2]

Repeating this same reasoning for the second part of the product, we extract a function $g\colon B \to A$ and proof that the type

$$\prod_{b\colon B} R(g(b), b) \tag{2}$$

is inhabited.

Fix $b\colon B$. From (2) we obtain a term $s\colon R(g(b), b)$. the evaluation of (1) at $a :\equiv g(b)$ produces a term $r\colon R(g(b), f(g(b)))$ and a map

$$\rho\colon \prod_{v\colon \Sigma_B R(g(b),-)} \langle f(g(b)), r\rangle = v.$$

Because $s\colon R(g(b), b)$, we can form the pair $v :\equiv \langle b, s\rangle\colon \Sigma_B R(g(b), -)$. Evaluating $\rho$ at this $v$, we arrive at the equality

$$\langle f(g(b)), r\rangle = \langle b, s\rangle.$$

[2]Given $a\colon A$, since $P(a) :\equiv \Sigma_B R(a,-)$ is contractible we might have defined $f(a)$ as its center of contraction. Similarly for the center $g(b)$ of $Q(b) :\equiv \Sigma_A R(-, b)$.

In particular, $f(g(b)) =_B b$. Since $b\colon B$ was arbitrarily chosen, this constructs the right homotopy $f \circ g \sim \mathsf{id}_B$. The left homotopy $g \circ f \sim \mathsf{id}_A$ follows symmetrically.

Conversely, if $g\colon B \to A$ induces an equivalence, we have a relation

$$R\colon A \to B \to \mathcal{U}, \quad R(a,b) :\equiv g(b) =_A a.$$

Since the fibers of $g$ are contractible, we obtain a witness for the first part of the product:

$$\prod_{a\colon A} \mathsf{isContr}\Big(\sum_{b\colon B} g(b) =_A a\Big).$$

For the second part, observe that for any fixed $b\colon B$, the sum

$$\sum_{a\colon A} g(b) =_A a$$

is also contractible by Lemma 3.4.11 applied to $g(b)\colon A$. Therefore, the second part of the Cartesian product is inhabited for all $b\colon B$. This shows that the entire target type has a witness.

For the final question, fix $f\colon A \to B$, and consider the relation

$$R(a,b) :\equiv f(a) =_B b.$$

By definition, the target type computes to

$$\mathsf{E}(f) :\equiv \prod_{a\colon A} \mathsf{isContr}\Big(\sum_{b\colon B} f(a) =_B b\Big) \times \prod_{b\colon B} \mathsf{isContr}\Big(\sum_{a\colon A} f(a) =_B b\Big).$$

By Lemma 3.4.11, the first part of this cartesian product is inhabited. Thus, by Corollary 3.4.6 and part a) of Theorem 3.4.8, it is contractible. The second part of the product is definitionally equal to $\mathsf{isContrMap}(f)$. Therefore, we obtain an equivalence $\mathsf{E}(f) \simeq \mathsf{isContrMap}(f)$, which is equivalent to $\mathsf{ishae}(f)$ by Theorem 4.4.1 and its preceding comment. □

**Exercise 4.9.3.** *Proof* Proposition 4.1.11 *without using univalence.*[3]

**Exercise 4.9.4.** *Let $f\colon A \to B$ and $g\colon B \to C$ be functions, and fix $b\colon B$.*

a) *Construct a natural map $\mathsf{fib}_{g\circ f}(g(b)) \to \mathsf{fib}_g(g(b))$ and prove that its fiber over the point $\langle b, \mathsf{refl}_{g(b)}\rangle$ is equivalent to $\mathsf{fib}_f(b)$.*

b) *Show that for any $c\colon C$, there is an equivalence*

$$\mathsf{fib}_{g\circ f}(c) \simeq \sum_{w\colon \mathsf{fib}_g(c)} \mathsf{fib}_f(\pi_1(w)).$$

[3] Our proof doesn't use the univalence axiom.

*Solution.*

a) Let $\langle x, p\rangle : \mathsf{fib}_{g\circ f}(g(b))$. By definition, $p : g(f(x)) =_C g(b)$. It follows that $\langle f(x), p\rangle : \mathsf{fib}_g(g(b))$, and we can define

$$\begin{aligned}\phi : \mathsf{fib}_{g\circ f}(g(b)) &\to \mathsf{fib}_g(g(b))\\ \langle x, p\rangle &\mapsto \langle f(x), p\rangle.\end{aligned}$$

In particular,

$$\begin{aligned}\mathsf{fib}_\phi(\langle b, \mathsf{refl}_{g(b)}\rangle) &\equiv \sum_{w:\, \mathsf{fib}_{g\circ f}(g(b))} \langle f(\pi_1(w)), \pi_2(w)\rangle =_{\mathsf{fib}_g(g(b))} \langle b, \mathsf{refl}_{g(b)}\rangle\\ &\simeq \sum_{x:\, A} \sum_{p:\, g(f(x))=_C g(b)} \langle f(x), p\rangle =_{\mathsf{fib}_g(g(b))} \langle b, \mathsf{refl}_{g(b)}\rangle\\ &\simeq \sum_{x:\, A} \sum_{p:\, g(f(x))=_C g(b)} \sum_{q:\, f(x)=_B b} q_*(p) =_{g(b)=_C g(b)} \mathsf{refl}_{g(b)}\end{aligned}$$

where

$$\begin{aligned}q_*(p) &\equiv \mathsf{transport}^{y\mapsto g(y)=_C g(b)}(q, p)\\ &=_{g(b)=_C g(b)} \mathsf{transport}^{z\mapsto z=_C g(b)}(\mathsf{ap}_g(q), p) && ;\ \text{Lem. 2.3.13}\\ &=_{g(b)=_C g(b)} \mathsf{ap}_g(q)^{-1} \cdot p && ;\ \text{Prop. 2.11.6.}\end{aligned}$$

Hence,

$$\begin{aligned}\mathsf{fib}_\phi(\langle b, \mathsf{refl}_{g(b)}\rangle) &\simeq \sum_{x:\, A} \sum_{p:\, g(f(x))=_C g(b)} \sum_{q:\, f(x)=_B b} p =_{g(f(x))=_C g(b)} \mathsf{ap}_g(q)\\ &\simeq \sum_{x:\, A} \sum_{p:\, g(f(x))=_C g(b)} \sum_{q:\, f(x)=_B b} \mathsf{ap}_g(q) =_{g(f(x))=_C g(b)} p\\ &\simeq \sum_{x:\, A} \sum_{q:\, f(x)=_B b} \underbrace{\sum_{p:\, g(f(x))=_C g(b)} \mathsf{ap}_g(q) =_{g(f(x))=_C g(b)} p}_{\text{Contractible by Lem. 3.4.11}}\\ &\simeq \sum_{x:\, A} \sum_{q:\, f(x)=_B b} 1\\ &\simeq \sum_{x:\, A} f(x) =_B b\\ &\equiv \mathsf{fib}_f(b).\end{aligned}$$

**Note.** The sequence

$$\mathsf{fib}_f(b) \to \mathsf{fib}_{g\circ f}(g(b)) \xrightarrow{\phi} \mathsf{fib}_g(g(b))$$

shows that $\mathsf{fib}_f(b)$ behaves as the "kernel" of $\phi$. It is what in algebraic topology is called a *fiber sequence*, the geometric analogue of an algebraic exact sequence.

b) Given $c\colon C$, we have

$$\begin{aligned}\sum_{w\colon \mathsf{fib}_g(c)} \mathsf{fib}_f(\pi_1(w)) &\simeq \sum_{y\colon B} \sum_{q\colon g(y)=_C c} \sum_{x\colon A} f(x) =_B y \\ &\simeq \sum_{x\colon A} \sum_{y\colon B} \sum_{q\colon g(y)=_C c} f(x) =_B y \\ &\simeq \sum_{x\colon A} \sum_{y\colon B} (f(x) =_B y) \times (g(y) =_C c).\end{aligned}$$

Since

$$\sum_{y\colon B} f(x) =_B y$$

is contractible with center $\langle f(x), \mathsf{refl}_{f(x)}\rangle$ by Lemma 3.4.11, we can apply Lemma 4.1.10 and replace the inner $\Sigma$-type with

$$g(f(x)) =_C c,$$

which yields $\mathsf{fib}_{g\circ f}(c)$, as desired.

**Note.** The equivalence

$$\mathsf{fib}_{g\circ f}(c) \simeq \sum_{w\colon \mathsf{fib}_g(c)} \mathsf{fib}_f(\pi_1(w))$$

geometrically describes how the fibers of $g \circ f$ can be recovered from the fibers of $f$ and $g$.

□

**Exercise 4.9.5.** *Prove that equivalences satisfy the* 2*-out-of-*6 *property: given* $f\colon A \to B$ *and* $g\colon B \to C$ *and* $h\colon C \to D$*, if* $g \circ f$ *and* $h \circ g$ *are equivalences, so are* $f$*,* $g$*,* $h$*, and* $h \circ g \circ f$*. Use this to give a higher-level proof of* Theorem 2.5.18*.*

*Solution.*

First part. Suppose that $\langle u, (\eta, \vartheta)\rangle$ is a quasi-inverse of $g \circ f$ and $\langle v, (\rho, \theta)\rangle$ is a quasi-inverse of $h \circ g$. Then

$$\eta\colon g \circ f \circ u \sim \mathsf{id}_C \quad \text{and} \quad \theta\colon v \circ h \circ g \sim \mathsf{id}_A,$$

which show that

$$(\langle f \circ u, \eta\rangle, \langle v \circ h, \theta\rangle)\colon \mathsf{isequiv}(g).$$

Hence, $g$ has a quasi-inverse $g^{-1}$, and so $f \sim g^{-1} \circ (g \circ f)$ and $h \sim (h \circ g) \circ g^{-1}$ have quasi-inverses as well.

Second part. We give two proofs. The first one following the approach suggested by the exercise. The second, derived from the previous exercise. In both cases we suppose that $\langle g, (\eta, \vartheta)\rangle$ is a quasi-inverse of $f$.

*First proof.* Given $x, y\colon A$ and $p\colon x =_A y$, the naturality of $\vartheta\colon g \circ f \sim \mathsf{id}_A$ yields

$$\vartheta_x \cdot p = \mathsf{ap}_{g\circ f}(p) \cdot \vartheta_y.$$

By right-multiplying by $\vartheta_y^{-1}$, we obtain

$$\begin{aligned}\mathsf{ap}_g(\mathsf{ap}_f(p)) &= \mathsf{ap}_{g\circ f}(p)\\ &= \vartheta_x \cdot p \cdot \vartheta_y^{-1}.\end{aligned}$$

Thus, the function $\mathsf{ap}_g \circ \mathsf{ap}_f$ is given by pre- and post-composition with fixed paths, which is an equivalence with quasi-inverse

$$q \mapsto \vartheta_x^{-1} \cdot q \cdot \vartheta_y.$$

By a symmetric argument using the right homotopy $\eta\colon f \circ g \sim \mathsf{id}_B$, the composition $\mathsf{ap}_f \circ \mathsf{ap}_g$ is also an equivalence.

The conclusion follows by applying the 2-out-of-6 property to the following sequence of maps:

$$\begin{array}{ccc} f(x) =_B f(y) & \xrightarrow{\mathsf{ap}_g} & g(f(x)) =_A g(f(y)) \\ {\scriptstyle \mathsf{ap}_f}\uparrow & & \downarrow{\scriptstyle \mathsf{ap}_f} \\ x =_A y & & f(g(f(x)) =_B f(g(f(y)))). \end{array}$$

*Second proof.* As established in part a) of the previous exercise, we have

$$\mathsf{fib}_\phi(\langle b, \mathsf{refl}_{g(b)}\rangle) \simeq \sum_{x\colon A} \sum_{p\colon g(f(x))=_A g(b)} \sum_{q\colon f(x)=_B b} p =_{g(f(x))=_A g(b)} \mathsf{ap}_g(q).$$

By path inversion, the innermost $\Sigma$-type is equivalent to the fiber $\mathsf{fib}_{\mathsf{ap}_g}(p)$. Hence, we obtain

$$\begin{aligned}\mathsf{fib}_\phi(\langle b, \mathsf{refl}_{g(b)}\rangle) &\simeq \sum_{x\colon A} \sum_{p\colon g(f(x))=_A g(b)} \mathsf{fib}_{\mathsf{ap}_g}(p)\\ &\simeq \sum_{w\colon \mathsf{fib}_{g\circ f}(g(b))} \mathsf{fib}_{\mathsf{ap}_g}(\pi_2(w)) \qquad ;\ \Sigma\text{-assoc.}\end{aligned}$$

As $f$ is an equivalence, $\mathsf{fib}_f(b)$ is contractible. Since $\mathsf{fib}_\phi(\langle b, \mathsf{refl}_{g(b)}\rangle) \simeq \mathsf{fib}_f(b)$, it follows that the $\Sigma$-type above is also contractible. Because $g\circ f$ is an equivalence, the base space $\mathsf{fib}_{g\circ f}(g(b))$ is contractible as well. Hence, the projection

$$\pi_1\colon \Big(\sum_{w\colon \mathsf{fib}_{g\circ f}(g(b))} \mathsf{fib}_{\mathsf{ap}_g}(\pi_2(w))\Big) \to \mathsf{fib}_{g\circ f}(g(b))$$

is a map between contractible spaces, which implies it is an equivalence. Consequently, it has contractible fibers. By Proposition 4.7.1, the fiber over $w$ is equivalent to $\mathsf{fib}_{\mathsf{ap}_g}(\pi_2(w))$.

Let $w :\equiv \langle x, p\rangle$. Then the fiber $\mathsf{fib}_{\mathsf{ap}_g}(p)$ is contractible for all $x\colon A$ and $p\colon g(f(x)) =_A g(b)$.

To show that this holds for an arbitrary $y\colon B$ and $q\colon g(y) =_A g(b)$, consider $x :\equiv g(y)$. Then, $\eta(y)\colon f(x) =_B y$. We can define the property of having contractible fibers as a type family over $B$:

$$P(z) :\equiv \prod_{p\colon g(z)=_A g(b)} \mathsf{isContr}(\mathsf{fib}_{\mathsf{ap}_g}(p)).$$

Since we established that $P(f(x))$ is inhabited, we can transport this witness along the path $\eta(y)$ to obtain a witness for $P(y)$. Because $y$ was arbitrarily chosen, the fibers of $\mathsf{ap}_g$ are contractible everywhere. Hence, $\mathsf{ap}_g$ is an equivalence. By symmetry, $\mathsf{ap}_f$ is also an equivalence.

□

**Exercise 4.9.6.** *For $A, B\colon \mathcal{U}$, define*

$$\mathsf{idtoqinv}_{A,B}\colon (A =_{\mathcal{U}} B) \to \sum_{f\colon A\to B} \mathsf{qinv}(f)$$

*by path induction in the obvious way. Let* qinv*-univalence denote the modified form of the univalence axiom which asserts that for all $A, B\colon \mathcal{U}$ the function* $\mathsf{idtoqinv}_{A,B}$ *has a quasi-inverse.*

a) *Show that* qinv*-univalence can be used instead of univalence in the proof of function extensionality in* § 4.8*.*

b) *Show that* qinv*-univalence can be used instead of univalence in the proof of* Theorem 4.1.13*.*

c) *Show that* qinv*-univalence is inconsistent (i.e. allows construction of an inhabitant of* $\mathsf{0}$*).*

   Hint: Fix $A$ and apply the dependent functor $(\sum_{B:\mathcal{U}}, \mathsf{total})$.

*Thus, the use of a "good" version of* isequiv *is essential in the statement of univalence.*

*Solution.* Let us begin by observing that idtoqinv is defined by path induction because, in the case $B \equiv A$ and $p \equiv \mathsf{refl}_A$, the identity map $\mathsf{id}_A\colon A \to A$ has the quasi-inverse $\langle \mathsf{id}_A, (\mathsf{refl}, \mathsf{refl})\rangle$.

a) We need to analyze the following results:

   - Lemma 4.8.2. To derive this lemma from qinv-univalence, it suffices to show that $e\colon A \simeq B$ implies $A =_{\mathcal{U}} B$, because the rest of the proof proceeds exactly as it does with standard univalence. From $e$, we can

extract its underlying function $f : A \to B$ along with a quasi-inverse $\langle g, (\eta, \vartheta)\rangle : \mathsf{qinv}(f)$. This gives us the pair

$$\langle f, \langle g, (\eta, \vartheta)\rangle\rangle : \sum_{f : A \to B} \mathsf{qinv}(f).$$

Applying the inverse of $\mathsf{idtoqinv}_{A,B}$ to this pair produces the desired path $A =_{\mathcal{U}} B$.

- Proposition 4.8.3. This proposition holds under qinv-univalence because it relies on univalence exclusively through Lemma 4.8.2.
- Theorem 4.8.4. The proof of this theorem invokes univalence only via Proposition 4.8.3.
- Theorem 4.8.5. The proof of this theorem does not rely on univalence at all.

b) Let $\mathsf{ua}'$ denote the inverse of $\mathsf{idtoqinv}_{2,2}$ given by qinv-univalence. First, observe that the same deduction included in the proof of Theorem 4.1.13 showing that $q \neq \mathsf{refl}_2$ works for $q :\equiv \mathsf{ua}'(e)$.

Therefore, to prove that condition a) of the theorem holds under qinv-univalence, we must show that the identity type $2 =_{\mathcal{U}} 2$ is a set. Under qinv-univalence, this goal reduces to proving that the type

$$\sum_{f : 2 \to 2} \mathsf{qinv}(f)$$

is a set.

Since 2 is a set by Theorem 3.2.8, the function type $2 \to 2$ is a set by part f) of Theorem 3.1.2. Note that we can use this part of the theorem because it relies on function extensionality, which we derived from qinv-univalence in the previous part of this exercise.

In addition, $\mathsf{qinv}(f)$ is a set by Proposition 3.2.20, which also requires function extensionality.

Thus, condition a) follows from part e) of Theorem 3.1.2.

Condition b) does not depend on univalence.

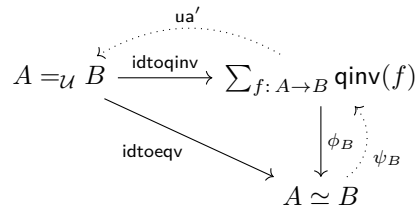
Condition c) can be proved as in the theorem because, in our case, the definitional equality $\mathsf{idtoqinv}_{2,2}(\mathsf{refl}_2) \equiv e_{\mathsf{id}}$ implies the propositional equality $\mathsf{ua}'(e_{\mathsf{id}}) = \mathsf{refl}_2$.

c) Fix $A : \mathcal{U}$ and let $B$ run over $\mathcal{U}$.

Let $\phi_f : \mathsf{qinv}(f) \to \mathsf{ishae}(f)$ and $\psi_f : \mathsf{ishae}(f) \to \mathsf{qinv}(f)$ be the maps given by Theorem 4.2.2. Note that

$$\phi_B(\langle f, q\rangle) \equiv \langle f, \phi_f(q)\rangle \quad \text{and} \quad \psi_B(\langle f, e\rangle) \equiv \langle f, \psi_f(e)\rangle.$$

Consider the triangle




The propositional commutativity of the solid triangle follows from path induction on $A =_{\mathcal{U}} B$.

Since $\mathsf{ishae}(f)$ is a mere proposition, we have

$$\phi_f(\psi_f(e)) =_{\mathsf{ishae}(f)} e,$$

for $e\colon \mathsf{ishae}(f)$. This means that $\phi_f \circ \psi_f \sim \mathsf{id}_{\mathsf{ishae}(f)}$. From part a) we know that $\mathsf{qinv}$-univalence implies function extensionality. It follows that $\phi_f \circ \psi_f = \mathsf{id}_{\mathsf{ishae}(f)}$.

Therefore,

$$\begin{aligned}\phi_B(\psi_B(\langle f, e\rangle)) &\equiv \phi_B(\langle f, \psi_f(e)\rangle)\\ &\equiv \langle f, \phi_f(\psi_f(e))\rangle\\ &=_{A\simeq B} \langle f, e\rangle.\end{aligned}$$

By function extensionality,

$$\phi_B \circ \psi_B = \mathsf{id}_{A\simeq B} \quad \text{in } (A \simeq B) \to (A \simeq B).$$

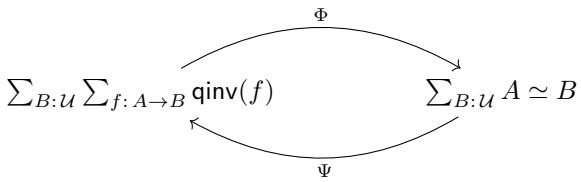
If we fix $A$ and let $B$ run over $\mathcal{U}$, the type

$$E :\equiv \sum_{B:\mathcal{U}} A =_{\mathcal{U}} B$$

is contractible with center $\langle A, \mathsf{refl}_A\rangle$ by Lemma 3.4.11. It follows that the total space

$$\sum_{B:\mathcal{U}} \sum_{f:\, A\to B} \mathsf{qinv}(f)$$

is contractible as well, because of the equivalence with the type $E$ induced by $\mathsf{idtoqinv}$. Similarly to what we did above, $\phi_B$ and $\psi_B$ induce maps



that satisfy, by function extensionality,

$$\Phi \circ \Psi = \mathsf{id}_{\sum_{B:\mathcal{U}} A \simeq B}.$$

Thus, $\Phi$ is a retraction out of a contractible type. By Proposition 3.4.10, the type $\sum_{B:\mathcal{U}} A \simeq B$ must be contractible as well.[4] It follows that $\Phi$ is quasi-invertible.

Now, according to Definition 4.6.5,

$$\Phi \equiv \mathsf{total}(\lambda B.\, \phi_B) \quad \text{and} \quad \phi_B \equiv \mathsf{total}(\lambda f.\, \phi_f).$$

Since $\Phi$ induces an equivalence, Corollary 4.6.7, applied twice, implies that $\phi_f$ is an equivalence for all $f$.

By part b), there is a type $A : \mathcal{U}$ and a map $f : A \to A$ such that $\mathsf{qinv}(f)$ is not a mere proposition. Since the type $A$ was fixed arbitrarily earlier, we can now specialize to this particular $A$ for which there exist elements $q, q' : \mathsf{qinv}(f)$ and a map

$$\epsilon : (q =_{\mathsf{qinv}(f)} q') \to \mathsf{0}.$$

Let $e :\equiv \phi_f(q)$ and $e' :\equiv \phi_f(q')$. Since $\mathsf{ishae}(f)$ is a mere proposition, there is a witness $p : e =_{\mathsf{ishae}(f)} e'$. By the action on paths, we obtain a witness

$$\mathsf{ap}_{\psi_f}(p) : \psi_f(e) =_{\mathsf{qinv}(f)} \psi_f(e').$$

Because $\psi_f$ is the quasi-inverse of $\phi_f$, we have paths $\psi_f(e) =_{\mathsf{qinv}(f)} q$ and $\psi_f(e') =_{\mathsf{qinv}(f)} q'$. Concatenating these paths yields a witness of $q =_{\mathsf{qinv}(f)} q'$, which $\epsilon$ maps to a witness of $\mathsf{0}$.

□

**Exercise 4.9.7.** *Show that a function $f : A \to B$ is an embedding if, and only if, the following two conditions hold:*

a) *The function $f$ is left-cancellable, i.e., for any $x, y : A$, if $f(x) =_B f(y)$ then $x =_A y$.*

b) *For any $x : A$, the map $\mathsf{ap}_f : \Omega(\langle A, x\rangle) \to \Omega(\langle B, f(x)\rangle)$ is an equivalence.*[5]

*(In particular, if $A$ is a set, then $f$ is an embedding if, and only if, it is left-cancellable and $\Omega(\langle B, f(x)\rangle)$ is contractible for all $x : A$.) Give examples to show that neither part* a) *nor part* b) *implies the other.*

*Solution.* Suppose that $f : A \to B$ is an embedding. By definition

$$\mathsf{ap}_f : (x =_A y) \to (f(x) =_B f(y))$$

[4] In other words, we have proved that $\mathsf{qinv}$-univalence implies univalence.

[5] See Notation 2.2.5.

is an equivalence. Let $\mathsf{ap}_f^{-1}$ be a quasi-inverse of $\mathsf{ap}_f$.

For part a), given $q\colon f(x) =_B f(y)$, by definition, $\mathsf{ap}_f^{-1}(q)\colon x =_A y$. Part a) follows.

Part b) is trivial because embeddings produce equivalences from all pairs of elements $x, y\colon A$.

Conversely, suppose that $f\colon A \to B$ satisfies conditions a) and b). We have to prove that $\mathsf{ap}_f\colon (x =_A y) \to (f(x) =_B f(y))$ is an equivalence.

Fix $x\colon A$, and let $y$ run over $A$. We have

$$\mathsf{total}(\mathsf{ap}_f)\colon \Big(\sum_{y\colon A} x =_A y\Big) \to \Big(\sum_{y\colon A} f(x) =_B f(y)\Big).$$

Since part a) establishes the existence of a map $u_{x,y}\colon (f(x) =_B f(y)) \to (x =_A y)$, the map $\mathsf{total}(\lambda y.\, u_{x,y})$ goes in the backward direction.

Because $\sum_{y\colon A} x =_A y$ is contractible, the composition $\mathsf{total}(\lambda y.\, u_{x,y}) \circ \mathsf{total}(\mathsf{ap}_f)$ is an equivalence.

By the functoriality of $\mathsf{total}$ and Corollary 4.6.7, we deduce that $u_{x,y} \circ \mathsf{ap}_f$ is an equivalence. It follows that $\mathsf{ap}_f$ is an embedding.

By Theorem 4.5.4, it suffices to show that $\mathsf{ap}_f\colon (x =_A y) \to (f(x) =_B f(y))$ is surjective for all $x, y\colon A$.

Let $q\colon f(x) =_B f(y)$ be a target path. We have to prove that $\|\mathsf{fib}_{\mathsf{ap}_f}(q)\|$ is inhabited.

Let $p :\equiv u_{x,y}(q)$. Then $p\colon x =_A y$ and $\mathsf{ap}_f(p)\colon f(x) =_B f(y)$. It follows that $q \cdot \mathsf{ap}_f(p)^{-1}\colon f(x) =_B f(x)$. By part b), there is a path $r\colon x =_A x$ such that

$$\mathsf{ap}_f(r) =_B q \cdot \mathsf{ap}_f(p)^{-1}.$$

Consequently, $\mathsf{ap}_f(r \cdot p) =_B q$. Thus, $\mathsf{fib}_{\mathsf{ap}_f}(q)$ is inhabited, and the conclusion follows.

(In the case where $A$ is a set, the type $\Omega(\langle A, x\rangle)$ is, by definition, contractible. Hence, the map $\Omega(\langle A, x\rangle) \to \Omega(\langle B, f(x)\rangle)$ is an equivalence if, and only if, the target type is contractible.)

Let $A$ be any inhabited non-contractible type. Define the function

$$\begin{aligned} f\colon 2 &\to \mathcal{U} \\ 1_2 &\mapsto A \\ 0_2 &\mapsto 0. \end{aligned}$$

To see that $f$ is left-cancellable, fix $a\colon A$ and let

$$\begin{aligned}\zeta &\colon (A =_{\mathcal{U}} \mathsf{0}) \to \mathsf{0}\\ &\qquad p \mapsto \mathsf{idtoeqv}(p)(a),\\ \zeta' &\colon (\mathsf{0} =_{\mathcal{U}} A) \to \mathsf{0}\\ &\qquad p \mapsto \zeta(p^{-1}).\end{aligned}$$

Next define the dependent functions

$$\begin{aligned}\epsilon_1 &\colon \prod_{y\colon \mathsf{2}} (A =_{\mathcal{U}} f(y)) \to (1_{\mathsf{2}} =_{\mathsf{2}} y)\\ \epsilon_1(y) &:\equiv \mathsf{ind}_{\mathsf{2}}(P_1, \mathsf{rec}_{\mathsf{0}}(1_{\mathsf{2}} =_{\mathsf{2}} 0_{\mathsf{2}}) \circ \zeta, \lambda q.\, \mathsf{refl}_{1_{\mathsf{2}}}, y),\\ \epsilon_0 &\colon \prod_{y\colon \mathsf{2}} (\mathsf{0} =_{\mathcal{U}} f(y)) \to (0_{\mathsf{2}} =_{\mathsf{2}} y)\\ \epsilon_0(y) &:\equiv \mathsf{ind}_{\mathsf{2}}(P_0, \lambda q.\, \mathsf{refl}_{0_{\mathsf{2}}}, \mathsf{rec}_{\mathsf{0}}(0_{\mathsf{2}} =_{\mathsf{2}} 1_{\mathsf{2}}) \circ \zeta', y),\end{aligned}$$

where $P_0$ and $P_1$ are the type families defined in the products. Then we can define

$$\begin{aligned}\epsilon &\colon \prod_{x\colon \mathsf{2}} \prod_{y\colon \mathsf{2}} (f(x) =_{\mathcal{U}} f(y)) \to (x =_{\mathsf{2}} y)\\ \epsilon(x, y) &:\equiv \mathsf{ind}_{\mathsf{2}}(P, \epsilon_0(y), \epsilon_1(y), x),\end{aligned}$$

with

$$P :\equiv \lambda x.\, \prod_{y\colon \mathsf{2}} (f(x) =_{\mathcal{U}} f(y)) \to (x =_{\mathsf{2}} y).$$

Thus, $f$ satisfies condition a). However, it does not satisfy b) because, in that case, it would be an embedding, which is generally false. The reason is that $A =_{\mathcal{U}} A$ would have to be contractible, which fails to be true for $A :\equiv \mathsf{2}$ because $(\mathsf{2} =_{\mathcal{U}} \mathsf{2}) \simeq \mathsf{2}$.

For the other example, consider $f\colon \mathsf{2} \to \mathsf{1}$. This function is clearly not left-cancellable. However, it satisfies condition b) because $\mathsf{2}$ is a set.

□

**Exercise 4.9.8.** *Show that the type of left-cancellable functions $\mathsf{2} \to B$ (see* Exercise 4.9.7*) is equivalent to $\sum_{x,y\colon B} x \neq y$. Give a similar explicit characterization of the type of embeddings $\mathsf{2} \to B$.*

*Solution.* We have to define an equivalence

$$\Big(\sum_{f\colon \mathsf{2}\to B} \prod_{x,y\colon \mathsf{2}} (f(x) =_B f(y)) \to (x =_{\mathsf{2}} y)\Big) \simeq \sum_{u,v\colon B} u \neq v.$$

Intuitively, the LHS represents the functions that satisfy $f(0_2) \neq f(1_2)$. Since a function from $2$ to $B$ is nothing but a pair of terms in $B$, the property translates into pairs of distinct elements.

Consider the map

$$\begin{aligned} \alpha \colon (2 \to B) &\to B \times B \\ f &\mapsto (f(1_2), f(0_2)). \end{aligned}$$

Its quasi-inverse is

$$\begin{aligned} \beta \colon B \times B &\to (2 \to B) \\ (u, v) &\mapsto \mathsf{rec}_2(B, v, u). \end{aligned}$$

The recursion principle for the boolean type $2$ implies that the composition $\alpha \circ \beta$ is the identity. Moreover,

$$\begin{aligned} \beta(\alpha(f)) &\equiv \mathsf{rec}_2(B, f(0_2), f(1_2)) \\ &=_{2 \to B} f. \end{aligned}$$

Fix $\langle f, c \rangle$ in the LHS. Let $\mathsf{neq} \colon (1_2 =_2 0_2) \to 0$ be the function defined in the lemma of Exercise 2.19.12. Then

$$\mathsf{neq} \circ c(1_2, 0_2) \colon (f(1_2) =_B f(0_2)) \to 0,$$

and we can define the forward function as

$$\begin{aligned} \phi(f) \colon \Big( \prod_{x,y \colon 2} (f(x) =_B f(y)) \to (x =_2 y) \Big) &\to \big( (f(1_2) =_B f(0_2)) \to 0 \big) \\ c &\mapsto \mathsf{neq} \circ c(1_2, 0_2). \end{aligned}$$

For the backward map, fix $\zeta \colon (f(1_2) =_B f(0_2)) \to 0$. Define the dependent function $c(x, y) \colon (f(x) =_B f(y)) \to (x =_2 y)$ by double induction on $x, y \colon 2$:

$$\begin{aligned} c(1_2, 1_2) &:\equiv \lambda p.\, \mathsf{refl}_{1_2} \\ c(0_2, 0_2) &:\equiv \lambda p.\, \mathsf{refl}_{0_2} \\ c(1_2, 0_2) &:\equiv \mathsf{rec}_0(1_2 =_2 0_2) \circ \zeta \\ c(0_2, 1_2) &:\equiv \lambda p.\, \mathsf{rec}_0(0_2 =_2 1_2)(\zeta(p^{-1})). \end{aligned}$$

The LHS is a mere proposition by part a) of Theorem 3.2.15 because of part b) of the same theorem and the fact that $x =_2 y$ is a mere proposition since $2$ is a set. The RHS is also a mere proposition because $0$ is a mere proposition. Consequently, $\phi(f)$ is an equivalence because we have a map in the other direction.

The conclusion follows by Corollary 4.6.8. □

**Exercise 4.9.9.** *The naïve non-dependent function extensionality axiom says that for $A, B : \mathcal{U}$ and $f, g : A \to B$ there is a function $(f \sim g) \to (f =_{A\to B} g)$. Modify the argument of §4.8 to show that this axiom implies the full function extensionality axiom 2.4.*

*Solution.* Let us first prove that Lemma 4.8.2 can be derived from naïve extensionality (NE).

Let $f : A \to B$ be quasi-invertible and let $\langle g, (\eta, \vartheta)\rangle : \mathsf{qinv}(f)$. Since $\eta : f \circ g \sim \mathsf{id}_B$ and $\vartheta : g \circ f \sim \mathsf{id}_A$, we deduce from NE that $f \circ g =_{B\to B} \mathsf{id}_B$ and $g \circ f =_{A\to A} \mathsf{id}_A$.

Define

$$\begin{aligned} \phi : (X \to A) &\to (X \to B) \\ u &\mapsto f \circ u, \end{aligned}$$

and

$$\begin{aligned} \psi : (X \to B) &\to (X \to A) \\ v &\mapsto g \circ v. \end{aligned}$$

Then, for any $u : X \to A$, we have

$$\begin{aligned} (\psi \circ \phi)(u) &\equiv \psi(f \circ u) \\ &\equiv g \circ f \circ u \\ &=_{X\to A} \mathsf{id}_A \circ u && ; \text{ by } \mathsf{ap}_{-\circ u} \\ &\equiv u. \end{aligned}$$

Thus, $\psi \circ \phi \sim \mathsf{id}_{X\to A}$ and, by NE, we obtain $\psi \circ \phi =_{(X\to A)\to(X\to A)} \mathsf{id}_{X\to A}$. Equality for the other composition follows by symmetry.

By the dependency tree for Theorem 4.8 shown in Remark 4.8.6, we deduce NE $\Rightarrow$ WE.

Since the implication WE $\Rightarrow$ FE does not depend on UA (as shown in the dependency tree of Theorem 4.8.5) but rather on WE, which we have already deduced from NE, the proof is complete.

□

# Chapter 5

# Induction

## 5.1 Intuition for Uniqueness

The principles of induction and recursion do more than guarantee the existence of certain functions; they completely characterize them by providing a uniqueness principle. Here we present two examples: natural numbers and lists.

Recall that the type $\mathbb{N}$ of natural numbers has two constructors

- $0 : \mathbb{N}$
- $\mathsf{succ} : \mathbb{N} \to \mathbb{N}$.

A similar example is the type $\mathsf{L}_A$ (a.k.a. $\mathsf{List}(A)$) of finite lists of elements of some type $A$, which has constructors

- $\mathsf{nil} : \mathsf{L}_A$
- $\mathsf{cons} : A \to \mathsf{L}_A \to \mathsf{L}_A$,

where $\mathsf{nil}$ represents the empty list, and $\mathsf{cons}(a, \ell)$ the list with head $a$ and tail $\ell$.

The induction principle for this type states that, given a family $E : \mathsf{L}_A \to \mathcal{U}$, to construct a dependent function

$$f : \prod_{\ell : \mathsf{L}_A} E(\ell),$$

it is sufficient to provide the following data:

- a term $e_{\mathsf{nil}} : E(\mathsf{nil})$ and
- a dependent function

$$e_{\mathsf{cons}} : \prod_{a : A} \prod_{\ell : \mathsf{L}_A} E(\ell) \to E(\mathsf{cons}(a, \ell)).$$

This principle also specifies that the constructed function $f$ computes as expected on the constructors, meaning that we have the definitional equalities

$$f(\mathsf{nil}) \equiv e_{\mathsf{nil}} \quad \text{and} \quad f(\mathsf{cons}(a,\ell)) \equiv e_{\mathsf{cons}}(a,\ell,f(\ell)), \tag{5.1}$$

which correspond to

$$\begin{aligned} \mathsf{ind}_{\mathsf{L}_A}(E,e_{\mathsf{nil}},e_{\mathsf{cons}})(\mathsf{nil}) &\equiv e_{\mathsf{nil}}, \\ \mathsf{ind}_{\mathsf{L}_A}(E,e_{\mathsf{nil}},e_{\mathsf{cons}})(\mathsf{cons}(a,\ell)) &\equiv e_{\mathsf{cons}}\big(a,\ell,\mathsf{ind}_{\mathsf{L}_A}(E,e_{\mathsf{nil}},e_{\mathsf{cons}})(\ell)\big). \end{aligned} \tag{5.2}$$

**Example 5.1.1.** To define the function

$$\mathsf{append}\colon \mathsf{L}_A \to \mathsf{L}_A \to \mathsf{L}_A$$

that joins two lists, we define:

- the motive $E(\ell) :\equiv \mathsf{L}_A \to \mathsf{L}_A$,
- $e_{\mathsf{nil}} :\equiv \mathsf{id}_{\mathsf{L}_A}$, and
- $e_{\mathsf{cons}}(a,\ell,g) :\equiv \lambda y.\, \mathsf{cons}(a,g(y))$.

Then $\mathsf{append} :\equiv \mathsf{ind}_{\mathsf{L}_A}(E,e_{\mathsf{nil}},e_{\mathsf{cons}})$ satisfies

$$\mathsf{append}(\mathsf{nil},y) \equiv y$$

and

$$\mathsf{append}(\mathsf{cons}(a,\ell),y) \equiv \mathsf{cons}(a,\mathsf{append}(\ell,y)).$$

**Theorem 5.1.2.** *Let $f,g\colon \prod_{x:\mathbb{N}} E(x)$ be functions satisfying the recurrences*

$$e_z\colon E(0) \quad \textit{and} \quad e_s\colon \prod_{n:\mathbb{N}} E(n) \to E(\mathsf{succ}(n))$$

*up to propositional equality. That is, assume that*

$$f(0) =_{E(0)} e_z \quad \textit{and} \quad g(0) =_{E(0)} e_z,$$

*and that*

$$\prod_{n:\mathbb{N}} f(\mathsf{succ}(n)) =_{E(\mathsf{succ}(n))} e_s(n,f(n))$$

*and*

$$\prod_{n:\mathbb{N}} g(\mathsf{succ}(n)) =_{E(\mathsf{succ}(n))} e_s(n,g(n)).$$

*Then $f \sim g$.*

*Proof.* Let $C\colon \mathbb{N} \to \mathcal{U}$ be the type family defined by $C(n) :\equiv f(n) =_{E(n)} g(n)$. To prove the theorem, it suffices to produce a dependent function in $\prod_{n:\mathbb{N}} C(n)$.

By the induction principle of $\mathbb{N}$, we need to provide the following data:

- a term $c_z : C(0)$ and
- a function $c_s : \prod_{n:\mathbb{N}} C(n) \to C(\mathsf{succ}(n))$.

The first part of our hypothesis gives us paths

$$p_0 : f(0) =_{E(0)} e_z \quad \text{and} \quad q_0 : g(0) =_{E(0)} e_z,$$

which we can use to produce a path from $f(0)$ to $g(0)$, defining $c_z :\equiv p_0 \cdot q_0^{-1} : C(0)$.

To define $c_s$, given $n : \mathbb{N}$ and $h : C(n)$, we must construct $c_s(n,h) : C(\mathsf{succ}(n))$. Since $h : f(n) =_{E(n)} g(n)$, we can use the action on paths to obtain

$$\mathsf{ap}_{e_s(n,-)}(h) : e_s(n, f(n)) =_{E(\mathsf{succ}(n))} e_s(n, g(n)).$$

The second part of the hypothesis yields paths

$$p_n : f(\mathsf{succ}(n)) =_{E(\mathsf{succ}(n))} e_s(n, f(n))$$

and

$$q_n : g(\mathsf{succ}(n)) =_{E(\mathsf{succ}(n))} e_s(n, g(n)).$$

Concatenating these paths yields

$$p_n \cdot \mathsf{ap}_{e_s(n,-)}(h) \cdot q_n^{-1} : f(\mathsf{succ}(n)) =_{E(\mathsf{succ}(n))} g(\mathsf{succ}(n)),$$

which yields a term of type $C(\mathsf{succ}(n))$. To complete the proof, we set $c_s(n,h)$ to be this concatenation. □

**Remark 5.1.3.** A similar theorem holds for lists: *If* $f, g : \prod_{x:\mathsf{L}_A} E(x)$ *propositionally satisfy the recurrences* (5.1)*, then* $f \sim g$.

The proof proceeds exactly as in the theorem, introducing

$$C(\ell) :\equiv f(\ell) =_{E(\ell)} g(\ell),$$

and then defining $c_{\mathsf{nil}}$ by concatenating the paths

$$f(\mathsf{nil}) =_{E(\mathsf{nil})} e_{\mathsf{nil}} \quad \text{and} \quad g(\mathsf{nil}) =_{E(\mathsf{nil})} e_{\mathsf{nil}}.$$

The definition of $c_{\mathsf{cons}}(a, \ell, \eta)$ is obtained by concatenating the path

$$\mathsf{ap}_{e_{\mathsf{cons}}(a,\ell,-)}(\eta) : e_{\mathsf{cons}}(a, \ell, f(\ell)) =_{E(\mathsf{cons}(a,\ell))} e_{\mathsf{cons}}(a, \ell, g(\ell))$$

with the paths provided by the equalities

$$f(\mathsf{cons}(a, \ell)) =_{E(\mathsf{cons}(a,\ell))} e_{\mathsf{cons}}(a, \ell, f(\ell))$$

and

$$g(\mathsf{cons}(a, \ell)) =_{E(\mathsf{cons}(a,\ell))} e_{\mathsf{cons}}(a, \ell, g(\ell)).$$

Suppose we have a different construction of the natural numbers:

- $0' : \mathbb{N}'$
- $\mathsf{succ}' : \mathbb{N}' \to \mathbb{N}'$,

along with an induction principle

$$\mathsf{ind}_{\mathbb{N}'} : \prod_{E : \mathbb{N}' \to \mathcal{U}} E(0') \to \Big( \prod_{n' : \mathbb{N}'} E(n') \to E(\mathsf{succ}'(n')) \Big) \to \prod_{n' : \mathbb{N}'} E(n')$$

and the definitional equalities

$$\begin{aligned} \mathsf{ind}_{\mathbb{N}'}(E, e_0, e_s, 0') &:\equiv e_0, \\ \mathsf{ind}_{\mathbb{N}'}(E, e_0, e_s, \mathsf{succ}'(n')) &:\equiv e_s(n', \mathsf{ind}_{\mathbb{N}'}(E, e_0, e_s, n')). \end{aligned}$$

We can use the constant families $E(n) :\equiv \mathbb{N}'$ and $E'(n') :\equiv \mathbb{N}$ to define

$$f : \mathbb{N} \to \mathbb{N}' :\equiv \mathsf{ind}_{\mathbb{N}}(E, 0', \lambda n.\, \lambda r.\, \mathsf{succ}'(r))$$

and

$$g : \mathbb{N}' \to \mathbb{N} :\equiv \mathsf{ind}_{\mathbb{N}'}(E', 0, \lambda n'.\, \lambda r.\, \mathsf{succ}(r)).$$

First, observe that

$$g \circ f(0) \equiv g(0') \equiv 0 \equiv \mathsf{id}_{\mathbb{N}}(0)$$

and also

$$g \circ f(\mathsf{succ}(n)) \equiv g(\mathsf{succ}'(f(n))) \equiv \mathsf{succ}(g(f(n))) \equiv \mathsf{succ}(g \circ f(n)).$$

By setting $e_s(x, y) :\equiv \mathsf{succ}(y)$ and noting that $\mathsf{id}_{\mathbb{N}}(\mathsf{succ}(n)) \equiv \mathsf{succ}(\mathsf{id}_{\mathbb{N}}(n))$, we can rewrite the equivalences above as

$$g \circ f(\mathsf{succ}(n)) \equiv e_s(n, g \circ f(n)) \quad \text{and} \quad \mathsf{id}_{\mathbb{N}}(\mathsf{succ}(n)) \equiv e_s(n, \mathsf{id}_{\mathbb{N}}(n)).$$

Therefore, by Theorem 5.1.2, we obtain a homotopy $\vartheta : g \circ f \sim \mathsf{id}_{\mathbb{N}}$.

A symmetric argument yields a homotopy $\eta : f \circ g \sim \mathsf{id}_{\mathbb{N}'}$. Consequently, $f$ and $g$ are quasi-inverses and so $\mathbb{N} \simeq \mathbb{N}'$. Thus, the univalence axiom constructs a path $\mathbb{N} =_{\mathcal{U}} \mathbb{N}'$.

**Example 5.1.4.** A concrete example of an alternative construction of $\mathbb{N}$ is the following:

- $\mathbb{N}' :\equiv \mathsf{L}_1$,
- $0' :\equiv \mathsf{nil}$, and
- $\mathsf{succ}'(\ell) :\equiv \mathsf{cons}(\star, \ell)$.

The induction principle for the target type $\mathbb{N}'$ has the signature:

$$\mathsf{ind}_{\mathbb{N}'}: \prod_{E:\,\mathbb{N}'\to\mathcal{U}} E(\mathsf{nil}) \to \Big(\prod_{\ell:\,\mathbb{N}'} E(\ell) \to E(\mathsf{succ}'(\ell))\Big) \to \prod_{y:\,\mathbb{N}'} E(y).$$

To define this using the list eliminator $\mathsf{ind}_{\mathsf{L}_1}$, we must supply a step function

$$e_{\mathsf{cons}}: \prod_{z:\,1}\ \prod_{\ell:\,\mathsf{L}_1} E(\ell) \to E(\mathsf{cons}(z,\ell)).$$

Given our inductive step argument $g: \prod_{\ell:\,\mathbb{N}'} E(\ell) \to E(\mathsf{succ}'(\ell))$, we construct $e_{\mathsf{cons}}$ as follows. For any $z:1$, there is a canonical path $p_z: \star =_1 z$. The action on paths yields

$$\mathsf{ap}_{\mathsf{cons}(-,\ell)}(p_z): \mathsf{cons}(\star,\ell) =_{\mathbb{N}'} \mathsf{cons}(z,\ell).$$

Since $\mathsf{cons}(\star,\ell) \equiv \mathsf{succ}'(\ell)$, we can define $e_{\mathsf{cons}}(z,\ell)$ by composition:

$$e_{\mathsf{cons}}(z,\ell) :\equiv \mathsf{transport}^{E}(\mathsf{ap}_{\mathsf{cons}(-,\ell)}(p_z), -) \circ g(\ell)$$

as depicted in the diagram

$$\begin{array}{ccc} E(\mathsf{cons}(\star,\ell)) & \xrightarrow{\mathsf{transport}^{E}(\mathsf{ap}_{\mathsf{cons}(-,\ell)}(p_z),-)} & E(\mathsf{cons}(z,\ell)) \\ {\scriptstyle g(\ell)}\uparrow & & \downarrow{\scriptstyle \equiv} \\ E(\ell) & \xrightarrow[e_{\mathsf{cons}}(z,\ell)]{} & E(\mathsf{succ}'(\ell)) \end{array}$$

Finally, we can define:

$$\mathsf{ind}_{\mathbb{N}'}(E, e_{\mathsf{nil}}, g) :\equiv \mathsf{ind}_{\mathsf{L}_1}(E, e_{\mathsf{nil}}, e_{\mathsf{cons}}).$$

**Remark 5.1.5.** The same ideas can be applied to other inductive types. For instance, as shown in Exercise 1.15.4, the coproduct $A + B$ can be implemented as

$$A \oplus B :\equiv \sum_{x:\,2} \mathsf{rec}_2(\mathcal{U}, A, B, x)$$

with constructors

- $\mathsf{in}_A(x) :\equiv \langle 0_2, x\rangle$ and
- $\mathsf{in}_B(y) :\equiv \langle 1_2, y\rangle$,

which are well-defined because $\mathsf{rec}_2(\mathcal{U}, A, B, 0_2) \equiv A$ and $\mathsf{rec}_2(\mathcal{U}, A, B, 1_2) \equiv B$.

In this case, the induction principle for $A \oplus B$ is given by

$$\mathsf{ind}_{A\oplus B}(C, g_0, g_1) :\equiv \mathsf{ind}_{\sum_{t:\,2} \mathsf{rec}_2(\mathcal{U},A,B,t)}\big(C, \mathsf{ind}_2(D, g_0, g_1)\big),$$

where the motive $D\colon \mathsf{2} \to \mathcal{U}$ is defined as

$$D(t) :\equiv \prod_{z\colon \mathsf{rec}_{\mathsf{2}}(\mathcal{U},A,B,t)} C(\langle t, z\rangle).$$

The associated computation rules evaluate as follows:

$$\mathsf{ind}_{A\oplus B}(C, g_0, g_1, \mathsf{in}_A(x)) \equiv g_0(x),$$
$$\mathsf{ind}_{A\oplus B}(C, g_0, g_1, \mathsf{in}_B(y)) \equiv g_1(y).$$

Since $A \oplus B$ satisfies the same induction principle as $A + B$, they are canonically equivalent.

## 5.2 Well-Founded Trees

The so-called *well-founded trees*, a.k.a. $\mathsf{W}$-*trees*, generalize inductive data structures such as natural numbers and lists. The term "well-founded" means that there are no infinite downward paths in these trees.

A $\mathsf{W}$-tree has two basic ingredients:

- A *label* type $A$ and
- An *arity* type family $B\colon A \to \mathcal{U}$.

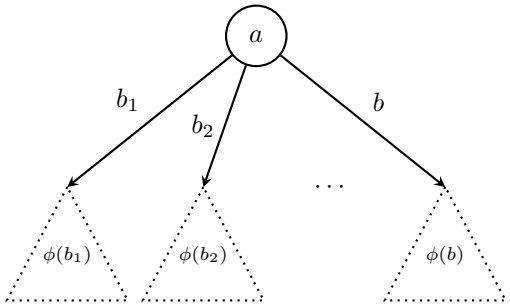
The $\mathsf{W}$-type models $\mathsf{W}$-trees. It is denoted $\mathsf{W}_{x\colon A}B(x)$, or simply $\mathsf{W}_A B$.

The term-construction process has two steps. First, we choose a label $a\colon A$ for the root. The type $A$ encodes the possible node constructors.

Second, the type family $B(a)$ dictates the branches hanging from that node. If $B(a)$ is the empty type, the node is a leaf: it has zero branches. If $B(a)$ is the boolean type, the node has two branches, and so forth.

If the node has branches, a fully formed subtree hangs from each of them. As illustrated below, the branches are indexed by the elements $b_1, b_2, \ldots, b$ of $B(a)$.

The function $\phi\colon B(a) \to \mathsf{W}_{x\colon A}B(x)$ attaches a subtree $\phi(b)$ to each branch, represented by the dotted triangles.




(5.1)

The induction principle of the $\mathsf{W}$-type ensures that each of these triangles is itself a well-founded tree. Thus, the branching process repeats inside each triangle and terminates at leaves after a finite number of steps.

Because $\mathsf{W}$-types serve as a universal generalization of inductive types, all the distinct constructors of a given type are unified into a single constructor, namely the function:

$$\mathsf{sup}\colon \prod_{a\colon A} \big(B(a) \to \mathsf{W}_A B\big) \to \mathsf{W}_A B. \tag{5.2}$$

When a term is built using $\mathsf{sup}(a, \phi)$, the label $a\colon A$ acts as the indicator for which constructor is being applied, while the function

$$\phi\colon B(a) \to \mathsf{W}_A B$$

supplies the subtrees required by that specific constructor.

In this way, the single constructor $\mathsf{sup}$ expresses the behavior of any number of individual constructors by delegating the choice of the operation to the label type $A$.

**Remark 5.2.1.** The constructor $\mathsf{sup}$ operates as follows: Given a label $a$ and a function $\phi\colon B(a) \to \mathsf{W}_A B$ representing an indexed family of sub-trees (5.1), the constructor gathers this entire family and joins them under a single new node. In the hierarchy of the tree, this node sits above all of its children. Therefore, the operation of creating this new node is conceptually identical to taking the supremum of all the sub-trees in the family.

**Remark 5.2.2.** According to 37. Recursion in Σ-types, the constructor $\mathsf{sup}$ given by (5.2) determines a function

$$\mathsf{rec}_{\sum_{a\colon A} B(a)\to \mathsf{W}_A B}(\mathsf{W}_A B, \mathsf{sup})\colon \Big(\sum_{a\colon A} B(a) \to \mathsf{W}_A B\Big) \to \mathsf{W}_A B. \tag{5.3}$$

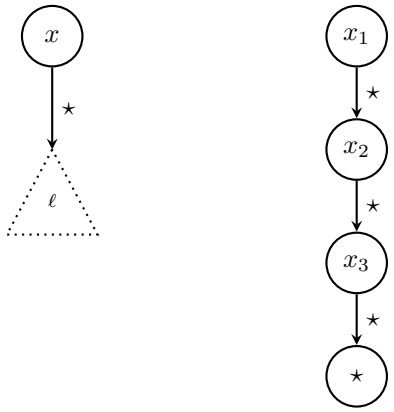
**Example 5.2.3.** To illustrate how $\mathsf{W}$-trees are appropriate for generalizing inductive types, let us use them to produce a new definition of $\mathsf{L}_X$.

The core idea is that, when using $\mathsf{W}$-trees, **constructors are labeled by elements, and their recursive arguments (if any) by types**. This strictly separates the non-recursive data of a constructor from its recursive structure. The elements act as labels belonging to a type $A$, and the shape of the recursive arguments is dictated by an *arity* type family $B\colon A \to \mathcal{U}$.

In our case, the constructors are $\mathsf{nil}$ and $\mathsf{cons}$. Since $\mathsf{nil}$ carries no non-recursive data, we can associate it to the nullary label $\star\colon \mathsf{1}$. For $\mathsf{cons}$, we have two arguments: $x\colon X$ and $\ell\colon \mathsf{L}_X$. The first one is not recursive, so we must tag it directly as a label; thus, we use a whole family of unary labels provided by the type $X$ to represent the constructor family $x \mapsto \mathsf{cons}(x, -)$.

The second argument $\ell\colon \mathsf{L}_X$ is recursive. Since each constructor $\mathsf{cons}(x, -)$ expects one recursive argument, it is assigned an arity of the unit type $\mathsf{1}$. Similarly, since $\mathsf{nil}$ expects zero recursive arguments, its arity is the empty type $\mathsf{0}$.

Summarizing, our proposal is to define $A :\equiv \mathsf{1} + X$ and specify the arity family $B\colon A \to \mathcal{U}$ by cases: $B(\mathsf{inl}(\star)) :\equiv \mathsf{0}$ and $B(\mathsf{inr}(x)) :\equiv \mathsf{1}$. In other words, $B(a) :\equiv \mathsf{rec}_{\mathsf{1}+X}(\mathcal{U}, \lambda\star.\, \mathsf{0}, \lambda x.\, \mathsf{1}, a)$.




The first graph depicts the recursive step for the constructor $\mathsf{cons}(x, \ell)$. The root is labeled by $x$, representing $\mathsf{inr}(x)\colon A$. Since the arity of this label is $B(\mathsf{inr}(x)) :\equiv \mathsf{1}$, there is exactly one downward edge emerging from the node. This edge is indexed by $\star$, and the subtree-assigning function $\phi\colon \mathsf{1} \to \mathsf{W}_A B$ attaches the recursive argument by defining $\phi(\star) :\equiv \ell$.

With this, we can formally redefine:

- $\mathsf{L}'_X :\equiv \mathsf{W}_{\mathsf{1}+X}\mathsf{rec}_{\mathsf{1}+X}(\mathcal{U}, \lambda\star.\, \mathsf{0}, \lambda x.\, \mathsf{1})$,
- $\mathsf{nil}' :\equiv \mathsf{sup}(\mathsf{inl}(\star), \mathsf{rec}_{\mathsf{0}}(\mathsf{L}'_X))$, and
- $\mathsf{cons}'(x, \ell') :\equiv \mathsf{sup}(\mathsf{inr}(x), \lambda\star.\, \ell')$.

The second graph illustrates a list of three elements. Each node $x_i$ has arity $\mathsf{1}$, creating a single downward branch. The list terminates at the node representing the $\mathsf{nil}$ constructor. This leaf node is labeled by $\star$, which stands for $\mathsf{inl}(\star)\colon A$. Because its arity is the empty type, no edges depart from it.

**Example 5.2.4.** To formalize the natural numbers using $\mathsf{W}$-trees, we must define the label type $A$ to represent two constructors: 0 and $\mathsf{succ}$.

Since the former has no recursive arguments, we can consider $\mathsf{1}$ for labeling it. The latter has a single argument $n$, which is recursive. Consequently, $\mathsf{1}$ is also appropriate for labeling the $\mathsf{succ}$ constructor. Thus, $\mathsf{1} + \mathsf{1}$ is appropriate for $A$, and this can be abbreviated by setting $A :\equiv \mathsf{2}$.

The arity family $B\colon A \to \mathcal{U}$ needs to represent the recursive arguments of each constructor. Then,

$$B(0_2) :\equiv 0,$$
$$B(1_2) :\equiv 1.$$

Thus, the arity function can be specified as

$$B :\equiv \mathsf{rec}_2(\mathcal{U}, 0, 1).$$

The subtree-assigning function $\phi\colon B(a) \to \mathsf{W}_2 B$ depends on the constructor being encoded.

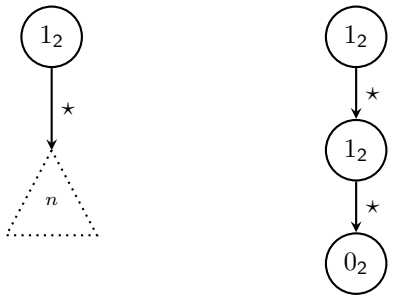
For the base case 0, we have $\phi_0 :\equiv \mathsf{rec}_0(\mathsf{W}_2 B)$.

For the inductive step $\mathsf{succ}(n)$, we have $\phi_{\mathsf{succ}(n)} :\equiv \lambda\star.\, n$.

In summary:

- $\mathbb{N}' :\equiv \mathsf{W}_2\mathsf{rec}_2(\mathcal{U}, 0, 1)$,
- $0' :\equiv \mathsf{sup}(0_2, \mathsf{rec}_0(\mathbb{N}'))$, and
- $\mathsf{succ}'(n) :\equiv \mathsf{sup}(1_2, \lambda\star.\, n)$.

Here is a graph showing the recursive constructor $\mathsf{succ}$ on the left. On the right, we have a representation of $2 :\equiv \mathsf{succ}(\mathsf{succ}(0))$:




## 5.3 W-Types

**Definition 5.3.1.** The formal definition of the W-type is given by its rules for formation, introduction, elimination, and computation. Let $A\colon \mathcal{U}$ be a type and $B\colon A \to \mathcal{U}$ be a type family.

Formation: The type is formed by the binder $\mathsf{W}$, yielding a type in the universe:

$$\mathsf{W}_{x:\,A} B(x)\colon \mathcal{U}.$$

Introduction: The elements of the W-type are built using the constructor $\mathsf{sup}$. Given a label $a\colon A$ and a subtree-assigning function $\phi\colon B(a) \to \mathsf{W}_A B$, we

can form a new tree $\mathsf{sup}(a, \phi)$, where

$$\mathsf{sup}\colon \prod_{a\colon A} \big(B(a) \to \mathsf{W}_A B\big) \to \mathsf{W}_A B. \tag{5.1}$$

Elimination: To construct a dependent function over the W-type, we use its induction principle. Given a motive $E\colon \mathsf{W}_A B \to \mathcal{U}$, we must provide an inductive step function $e_s$. This function constructs a term in $E(\mathsf{sup}(a, \phi))$, assuming we already have terms in $E(\phi(b))$ for all $b\colon B(a)$. The required signature for $e_s$ is

$$e_s\colon \prod_{a\colon A} \prod_{\phi\colon B(a)\to \mathsf{W}_A B} \Big( \prod_{b\colon B(a)} E(\phi(b))\Big) \to E(\mathsf{sup}(a, \phi)). \tag{5.2}$$

With this data, the induction principle produces a dependent function

$$\mathsf{ind}_{\mathsf{W}}(E, e_s)\colon \prod_{w\colon \mathsf{W}_A B} E(w). \tag{5.3}$$

As a special case, suppose that the motive is a constant type family $E(w) :\equiv C$ and that we are given a map

$$s_C\colon \prod_{a\colon A} (B(a) \to \mathsf{W}_A B) \to (B(a) \to C) \to C. \tag{5.4}$$

We claim that, in this case, the induction principle reduces to the non-dependent *recursor*

$$\mathsf{rec}_{\mathsf{W}}(C, s_C)\colon \mathsf{W}_A B \to C. \tag{5.5}$$

To see this, fix $a\colon A$. We have

$$\begin{aligned} \prod_{\phi\colon B(a)\to \mathsf{W}_A B} \Big( \prod_{b\colon B(a)} E(\phi(b))\Big) \to E(\mathsf{sup}(a, \phi)) & \\ \equiv \prod_{\phi\colon B(a)\to \mathsf{W}_A B} \Big( \prod_{b\colon B(a)} C\Big) \to C & \\ \equiv (B(a) \to \mathsf{W}_A B) \to (B(a) \to C) \to C. & \end{aligned}$$

Hence,

$$\mathsf{rec}_{\mathsf{W}}(C, s_C) :\equiv \mathsf{ind}_{\mathsf{W}}(C, s_C).$$

Computation: The eliminators compute by evaluating the step function on the components of the sup constructor. For the induction principle, the computation rule is the definitional equality

$$\mathsf{ind}_{\mathsf{W}}(E, e_s)(\mathsf{sup}(a, \phi)) \equiv e_s(a, \phi, \mathsf{ind}_{\mathsf{W}}(E, e_s) \circ \phi). \tag{5.6}$$

Accordingly, the computation rule for the recursor is

$$\mathsf{rec}_{\mathsf{W}}(C, s_C)(\mathsf{sup}(a, \phi)) \equiv s_C(a, \phi, \mathsf{rec}_{\mathsf{W}}(C, s_C) \circ \phi). \tag{5.7}$$

**Example 5.3.2.** To see how this general framework operates in a concrete inductive type, let us continue with Example 5.2.3. The goal is to derive the induction principle for $\mathsf{L}_X$ from the induction principle for $\mathsf{W}$-types. For simplicity, we omit the primes from $\mathsf{L}_X$, $\mathsf{nil}$, and $\mathsf{cons}$.

We must construct the data required by $\mathsf{W}$-induction to yield standard list induction. This involves case analysis over the label $a : 1 + X$.

Suppose $a \equiv \mathsf{inl}(\star)$. In this case, we have $B(a) \equiv 0$. Consequently, the associated subtree function is $\phi \equiv \mathsf{rec}_0(\mathsf{L}_X)$. In particular, by definition [↑],

$$\mathsf{sup}(a, \phi) \equiv \mathsf{sup}(a, \mathsf{rec}_0(\mathsf{L}_X)) \equiv \mathsf{nil}.$$

The inductive hypothesis is a dependent function $g : \prod_{b:0} E(\phi(b))$. Because the product runs over the empty type, it is canonically equivalent to $1$ and thus contains a unique, trivial inhabitant.

For this case, the inductive step function of $\mathsf{ind}_\mathsf{W}$ needs to map this trivial inhabitant $g$ to a term in $E(\mathsf{sup}(a, \phi)) \equiv E(\mathsf{nil})$. This is precisely the data required by standard list induction: a term $e_\mathsf{nil} : E(\mathsf{nil})$.

In the case where $a \equiv \mathsf{inr}(x)$, we have $B(a) \equiv 1$. Thus, the subtree-assigning function has the signature $\phi : 1 \to \mathsf{L}_X$. As previously established [↑], the constructor application $\mathsf{sup}(a, \phi)$ is definitionally equal to $\mathsf{cons}(x, \phi(\star))$.

The inductive hypothesis is a dependent function $h : \prod_{b:1} E(\phi(b))$, which provides a single term: $h(\star) : E(\phi(\star))$.

To complete the definition of $e_s$, we must construct a term in $E(\mathsf{sup}(a, \phi))$, which evaluates to $E(\mathsf{cons}(x, \phi(\star)))$. But this is exactly the data required by the standard list induction[↑]: a dependent function $e_\mathsf{cons}$ that produces a term in $E(\mathsf{cons}(x, \phi(\star)))$ from the head $x$, the tail $\phi(\star)$, and the inductive hypothesis $h(\star)$.

Consequently, we can fully define the inductive step function $e_s$ by pattern matching on the label $a : 1 + X$:

$$\begin{aligned} e_s(\mathsf{inl}(\star), \phi, g) &:\equiv e_\mathsf{nil}, \\ e_s(\mathsf{inr}(x), \phi, h) &:\equiv e_\mathsf{cons}(x, \phi(\star), h(\star)). \end{aligned}$$

It follows that the induction principle for lists is an instance of the induction principle for $\mathsf{W}$-types:

$$\mathsf{ind}_\mathsf{List}(E, e_\mathsf{nil}, e_\mathsf{cons}) :\equiv \mathsf{ind}_\mathsf{W}(E, e_s).$$

**Theorem 5.3.3.** *Let $A$ be a type and $B : A \to \mathcal{U}$ a type family. Then, for every $w : \mathsf{W}_A B$, there is a propositional equality*

$$w =_{\mathsf{W}_A B} \mathsf{sup}(a, \phi)$$

*for some $a : A$ and $\phi : B(a) \to \mathsf{W}_A B$.*

*Proof.* Let $\mathsf{W} :\equiv \mathsf{W}_A B$. We want to construct an inhabitant of the type

$$\prod_{w:\,\mathsf{W}} \sum_{a:\,A} \sum_{\phi:\,B(a)\to\mathsf{W}} w =_{\mathsf{W}} \mathsf{sup}(a,\phi).$$

Let $E$ be the type family over $\mathsf{W}$ defined by

$$E(w) :\equiv \sum_{x:\,A} \sum_{\psi:\,B(x)\to\mathsf{W}} w =_{\mathsf{W}} \mathsf{sup}(x,\psi).$$

By the induction principle for $\mathsf{W}$-types, to prove $\Pi_{\mathsf{W}} E$, it suffices to construct a step function (5.2)

$$\prod_{a:\,A} \prod_{\phi:\,B(a)\to\mathsf{W}} \Big( \prod_{b:\,B(a)} E(\phi(b)) \Big) \to E(\mathsf{sup}(a,\phi)).$$

Fix $a\colon A$ and $\phi\colon B(a)\to\mathsf{W}$, and assume the induction hypothesis

$$h\colon \prod_{b:\,B(a)} E(\phi(b)).$$

We must provide an inhabitant of $E(\mathsf{sup}(a,\phi))$, which is definitionally equivalent to

$$\sum_{x:\,A} \sum_{\psi:\,B(x)\to\mathsf{W}} \mathsf{sup}(a,\phi) =_{\mathsf{W}} \mathsf{sup}(x,\psi).$$

This type is trivially inhabited by the tuple $\langle a, \langle \phi, \mathsf{refl}_{\mathsf{sup}(a,\phi)} \rangle \rangle$.

□

**Theorem 5.3.4.** *Let $f, g\colon \prod_{w:\,\mathsf{W}_A B} E(w)$ be dependent functions that satisfy the computation rule* (5.6) *associated with the recurrence* (5.2)

$$e\colon \prod_{a:\,A} \prod_{\phi:\,B(a)\to\mathsf{W}_A B} \Big( \prod_{b:\,B(a)} E(\phi(b)) \Big) \to E(\mathsf{sup}(a,\phi))$$

*up to propositional equality; that is, such that*

$$\prod_{a:\,A} \prod_{\phi:\,B(a)\to\mathsf{W}_A B} f(\mathsf{sup}(a,\phi)) =_{E(\mathsf{sup}(a,\phi))} e(a,\phi,f\circ\phi),$$
$$\prod_{a:\,A} \prod_{\phi:\,B(a)\to\mathsf{W}_A B} g(\mathsf{sup}(a,\phi)) =_{E(\mathsf{sup}(a,\phi))} e(a,\phi,g\circ\phi).$$

*Then $f \sim g$.*

*Proof.* Let $C\colon \mathsf{W}_A B \to \mathcal{U}$ be the type family defined by

$$C(w) :\equiv f(w) =_{E(w)} g(w).$$

We must construct a witness for $\prod_{w:\mathsf{W}_A B} C(w)$.

By the induction principle of W-types, this reduces to defining a step function $c_s(a,\phi)$ as specified in (5.2):

$$c_s(a,\phi)\colon (f\circ\phi\sim g\circ\phi)\to \big(f(\mathsf{sup}(a,\phi))=_{E(\mathsf{sup}(a,\phi))} g(\mathsf{sup}(a,\phi))\big)$$

for $a\colon A$ and $\phi\colon B(a)\to \mathsf{W}_A B$.

Let $\eta\colon f\circ\phi\sim g\circ\phi$. Applying the action on paths of the function

$$e(a,\phi)\colon \Big(\prod_{b:\,B(a)} E(\phi(b))\Big)\to E(\mathsf{sup}(a,\phi))$$

to the path

$$\mathsf{funext}(\eta)\colon f\circ\phi=_{\prod_{b:\,B(a)} E(\phi(b))} g\circ\phi$$

yields

$$\mathsf{ap}_{e(a,\phi)}(\mathsf{funext}(\eta))\colon e(a,\phi,f\circ\phi)=_{E(\mathsf{sup}(a,\phi))} e(a,\phi,g\circ\phi).$$

By hypothesis, we also have paths

$$u(a,\phi)\colon f(\mathsf{sup}(a,\phi))=_{E(\mathsf{sup}(a,\phi))} e(a,\phi,f\circ\phi)$$

and

$$v(a,\phi)\colon g(\mathsf{sup}(a,\phi))=_{E(\mathsf{sup}(a,\phi))} e(a,\phi,g\circ\phi).$$

Therefore, we can define the target path by concatenation:

$$c_s(a,\phi,\eta):\equiv u(a,\phi)\cdot \mathsf{ap}_{e(a,\phi)}(\mathsf{funext}(\eta))\cdot v(a,\phi)^{-1}.$$

□

**Theorem 5.3.5.** *Let $A\colon\mathcal{U}$ be a type and $B\colon A\to\mathcal{U}$ a type family. If $A$ is a mere proposition then $\mathsf{W}_A B$ is a mere proposition as well.*

*Proof.* Define $E\colon \mathsf{W}\to\mathcal{U}$ by

$$E(w):\equiv \prod_{w':\,\mathsf{W}} w=_{\mathsf{W}} w'.$$

By the induction principle, it suffices to specify a map in conformance with (5.2), namely

$$e_s\colon \prod_{a:\,A}\prod_{\phi:\,B(a)\to\mathsf{W}} \Big(\prod_{b:\,B(a)} E(\phi(b))\Big)\to E(\mathsf{sup}(a,\phi)),$$

where

$$\mathsf{sup}\colon \prod_{a:\,A}\big(B(a)\to\mathsf{W}\big)\to\mathsf{W}.$$

By the definition of $E$, this expands definitionally to

$$e_s : \prod_{a:A} \prod_{\phi: B(a)\to \mathsf{W}} \Big( \prod_{b: B(a)} \prod_{w:\mathsf{W}} \phi(b) =_{\mathsf{W}} w \Big) \to \prod_{w:\mathsf{W}} \mathsf{sup}(a,\phi) =_{\mathsf{W}} w.$$

To define $e_s$, let us fix $a$ and $\phi$. Given an inductive term

$$h : \prod_{b: B(a)} \prod_{w:\mathsf{W}} \phi(b) =_{\mathsf{W}} w,$$

we will map it to a dependent function of type

$$\prod_{w:\mathsf{W}} \mathsf{sup}(a,\phi) =_{\mathsf{W}} w.$$

Using the induction principle for the motive $D(w) :\equiv \mathsf{sup}(a,\phi) =_{\mathsf{W}} w$, our goal reduces to defining

$$d_s : \prod_{x:A} \prod_{\psi: B(x)\to \mathsf{W}} \Big( \prod_{y: B(x)} \mathsf{sup}(a,\phi) =_{\mathsf{W}} \psi(y) \Big) \to \mathsf{sup}(a,\phi) =_{\mathsf{W}} \mathsf{sup}(x,\psi).$$

Fix $x : A$ and $\psi : B(x) \to \mathsf{W}$. The inner inductive term is now a dependent map

$$g : \prod_{y: B(x)} \mathsf{sup}(a,\phi) =_{\mathsf{W}} \psi(y).$$

By hypothesis, since $A$ is a mere proposition, there exists a path $p : a =_A x$. This path induces a backwards transport map

$$f :\equiv \mathsf{transport}^B(p^{-1}) : B(x) \to B(a).$$

Combining this map with $h$, we obtain

$$h(f(y), \psi(y)) : \phi(f(y)) =_{\mathsf{W}} \psi(y)$$

for all $y : B(x)$. By Lemma 2.13.4, there is a path

$$q : \mathsf{transport}^{\lambda z.\, B(z)\to \mathsf{W}}(p,\phi) =_{B(x)\to \mathsf{W}} \phi \circ f.$$

Applying $\mathsf{happly}$ to $q$ and concatenating the result with the point-wise equality provided by $h$, we obtain a homotopy $\eta : \mathsf{transport}^{\lambda z.\, B(z)\to \mathsf{W}}(p,\phi) \sim \psi$. Then, function extensionality yields

$$\mathsf{funext}(\eta) : \mathsf{transport}^{\lambda z.\, B(z)\to \mathsf{W}}(p,\phi) =_{B(x)\to \mathsf{W}} \psi.$$

By the characterization of equality in dependent pairs, we construct

$$\mathsf{pair}^{=}(p, \mathsf{funext}(\eta)) : \langle a,\phi\rangle =_{\sum_{z:A} B(z)\to \mathsf{W}} \langle x,\psi\rangle.$$

By Remark 5.2.2, the action on paths $\mathsf{ap}_{\mathsf{sup}}$ transforms this equality into the desired path

$$\mathsf{sup}(a,\phi) =_{\mathsf{W}} \mathsf{sup}(x,\psi).$$

□

## 5.4 Initial Algebras

**Definitions 5.4.1.**

- An $\mathsf{L}_X$*-algebra* is a type $C$ equipped with an element $c_{\mathsf{nil}} : C$ and a function $c_{\mathsf{cons}} : X \to C \to C$. The type of such algebras is

$$\mathsf{L}_X\mathsf{Alg} :\equiv \sum_{C:\mathcal{U}} C \times (X \to C \to C).$$

- Let $\mathcal{C} :\equiv \langle C, (c_{\mathsf{nil}}, c_{\mathsf{cons}})\rangle$ and $\mathcal{D} :\equiv \langle D, (d_{\mathsf{nil}}, d_{\mathsf{cons}})\rangle$ be two $\mathsf{L}_X$-algebras. An $\mathsf{L}_X$*-morphism* between them is a map $h : C \to D$ such that $h(c_{\mathsf{nil}}) =_D d_{\mathsf{nil}}$ and $h(c_{\mathsf{cons}}(x, c)) =_D d_{\mathsf{cons}}(x, h(c))$ for all $x : X$ and $c : C$. The type of such morphisms is

$$\mathsf{L}_X\mathrm{Hom}(\mathcal{C}, \mathcal{D}) :\equiv \sum_h \big(h(c_{\mathsf{nil}}) =_D d_{\mathsf{nil}}\big) \times \prod_{x,c} h(c_{\mathsf{cons}}(x, c)) =_D d_{\mathsf{cons}}(x, h(c)).$$

- An $\mathsf{L}_X$-algebra $\mathcal{I}$ is *homotopy-initial* (a.k.a. *h-initial*) if for any $\mathsf{L}_X$-algebra $\mathcal{C}$ the type of $\mathsf{L}_X$-morphisms from $\mathcal{I}$ to $\mathcal{C}$ is contractible, i.e.,

$$\mathsf{isHinit}_{\mathsf{L}_X}(\mathcal{I}) :\equiv \prod_{\mathcal{C}:\mathsf{L}_X\mathsf{Alg}} \mathsf{isContr}(\mathsf{L}_X\mathrm{Hom}(\mathcal{I}, \mathcal{C}))$$

  is inhabited.

**Lemma 5.4.2.** *Given* $\mathsf{L}_X$*-algebras* $\mathcal{A} :\equiv \langle A, (a_{\mathsf{nil}}, a_{\mathsf{cons}})\rangle$, $\mathcal{B} :\equiv \langle B, (b_{\mathsf{nil}}, b_{\mathsf{cons}})\rangle$, *and* $\mathcal{C} :\equiv \langle C, (c_{\mathsf{nil}}, c_{\mathsf{cons}})\rangle$, *the following properties hold:*

a) *The identity function* $\mathsf{id}_A : A \to A$ *is an* $\mathsf{L}_X$*-morphism from* $\mathcal{A}$ *to* $\mathcal{A}$.

b) *If* $f : A \to B$ *and* $g : B \to C$ *are* $\mathsf{L}_X$*-morphisms, then* $g \circ f : A \to C$ *is an* $\mathsf{L}_X$*-morphism from* $\mathcal{A}$ *to* $\mathcal{C}$.

c) *If* $f : A \to B$ *is an* $\mathsf{L}_X$*-morphism and* $g : B \to A$ *is a quasi-inverse of* $f$, *then the quasi-inverse* $g$ *is an* $\mathsf{L}_X$*-morphism from* $\mathcal{B}$ *to* $\mathcal{A}$.

*Proof.*

a) We need to show that $\mathsf{id}_A(a_{\mathsf{nil}}) = a_{\mathsf{nil}}$ and $\mathsf{id}_A(a_{\mathsf{cons}}(x, a)) = a_{\mathsf{cons}}(x, \mathsf{id}_A(a))$ for all $x : X$ and $a : A$. Both equalities hold definitionally.

b) For the base case, we have

$$(g \circ f)(a_{\mathsf{nil}}) \equiv g(f(a_{\mathsf{nil}})) =_C g(b_{\mathsf{nil}}) =_C c_{\mathsf{nil}}.$$

For the inductive step, let $x : X$ and $a : A$. Then,

$$\begin{aligned}(g \circ f)(a_{\mathsf{cons}}(x, a)) &\equiv g(f(a_{\mathsf{cons}}(x, a)))\\ &=_C g(b_{\mathsf{cons}}(x, f(a)))\\ &=_C c_{\mathsf{cons}}(x, g(f(a)))\\ &\equiv c_{\mathsf{cons}}(x, (g \circ f)(a)).\end{aligned}$$

c) Since $g$ is a quasi-inverse of $f$, there exist homotopies $\eta\colon f \circ g \sim \mathsf{id}_B$ and $\vartheta\colon g \circ f \sim \mathsf{id}_A$. We must show that $g$ preserves the algebra constructors.

For the base case, we apply $\mathsf{ap}_g$ to the equality $f(a_{\mathsf{nil}}) =_B b_{\mathsf{nil}}$ to obtain $g(f(a_{\mathsf{nil}})) =_A g(b_{\mathsf{nil}})$. By the homotopy $\vartheta$, we know $g(f(a_{\mathsf{nil}})) =_A a_{\mathsf{nil}}$. Transitivity of equality thus yields $g(b_{\mathsf{nil}}) =_A a_{\mathsf{nil}}$.

For the inductive step, let $x\colon X$ and $b\colon B$. Since $f$ is an $\mathsf{L}_X$-morphism, we have $f(a_{\mathsf{cons}}(x, g(b))) =_B b_{\mathsf{cons}}(x, f(g(b)))$. The homotopy $\eta$ implies $f(g(b)) =_B b$. Then substitution gives

$$f(a_{\mathsf{cons}}(x, g(b))) =_B b_{\mathsf{cons}}(x, b).$$

Applying $g$ to both sides yields

$$g(f(a_{\mathsf{cons}}(x, g(b)))) =_A g(b_{\mathsf{cons}}(x, b)).$$

By the homotopy $\vartheta$, the LHS is equal to $a_{\mathsf{cons}}(x, g(b))$. Therefore, we conclude

$$a_{\mathsf{cons}}(x, g(b)) =_A g(b_{\mathsf{cons}}(x, b)),$$

which shows that $g$ is an $\mathsf{L}_X$-morphism. □

**Theorem 5.4.3.** *The type of h-initial $\mathsf{L}_X$-algebras is a mere proposition.*

*Proof.* Let $\mathcal{I}$ and $\mathcal{J}$ be two $h$-initial $\mathsf{L}_X$-algebras. Then, $\mathsf{L}_X\mathrm{Hom}(\mathcal{I}, \mathcal{J})$ is contractible, hence inhabited by some $f\colon \mathcal{I} \to \mathcal{J}$. By symmetry, there is also $g\colon \mathcal{J} \to \mathcal{I}$. Since Lemma 5.4.2 implies that $g \circ f$ and $\mathsf{id}_{\mathcal{I}}$ inhabit $\mathsf{L}_X\mathrm{Hom}(\mathcal{I}, \mathcal{I})$, which is contractible, $g \circ f =_{\mathsf{L}_X\mathrm{Hom}(\mathcal{I},\mathcal{I})} \mathsf{id}_{\mathcal{I}}$. The same reasoning proves that $f \circ g =_{\mathsf{L}_X\mathrm{Hom}(\mathcal{J},\mathcal{J})} \mathsf{id}_{\mathcal{J}}$.

It follows that $f$ and $g$ are quasi-inverses and so $\mathcal{I} \simeq \mathcal{J}$. The conclusion follows by the univalence axiom. □

**Theorem 5.4.4.** *The $\mathsf{L}_X$-algebra $\langle \mathsf{L}_X, (\mathsf{nil}, \mathsf{cons})\rangle$ is homotopy initial.*

*Proof.* Let $\mathcal{C} :\equiv \langle C, (c_{\mathsf{nil}}, c_{\mathsf{cons}})\rangle$ be an $\mathsf{L}_X$-algebra.

Define $u\colon \mathsf{L}_X \to C$ inductively by

- $u(\mathsf{nil}) :\equiv c_{\mathsf{nil}}$,
- $u(\mathsf{cons}(a, \ell)) :\equiv c_{\mathsf{cons}}(a, u(\ell))$.

By definition, $u$ is an $\mathsf{L}_X$-morphism.

Given any other $\mathsf{L}_X$-morphism $v\colon \mathsf{L}_X \to C$, we have

- $v(\mathsf{nil}) =_C c_{\mathsf{nil}}$,
- $v(\mathsf{cons}(a, \ell)) =_C c_{\mathsf{cons}}(a, v(\ell))$.

Therefore, by Remark 5.1.3, we deduce that $u \sim v$, hence $u =_{\mathsf{L}_X \mathrm{Hom}(\mathsf{L}_X, C)} v$. Consequently, $\mathsf{L}_X\mathrm{Hom}(\mathsf{L}_X, C)$ is contractible with center $u$.

□

**Corollary 5.4.5.** *The* $\mathsf{L}_1$*-algebra* $\langle \mathbb{N}, (0, \mathsf{succ})\rangle$ *is homotopy initial.*

*Proof.* By the theorem, the $\mathsf{L}_1$-algebra $\langle \mathsf{L}_1, (\mathsf{nil}, \mathsf{cons})\rangle$ is $h$-initial. Since we have established in Example 5.1.4 the equality $\mathbb{N} =_{\mathcal{U}} \mathsf{L}_1$, which identifies 0 with $\mathsf{nil}$ and $\mathsf{succ}(n)$ with $\mathsf{cons}(\star, n)$, this yields a path

$$q\colon \langle \mathbb{N}, (0, \mathsf{succ})\rangle =_{\mathsf{L}_1\mathsf{Alg}} \langle \mathsf{L}_1, (\mathsf{nil}, \mathsf{cons})\rangle.$$

If we define the type family $Q\colon \mathsf{L}_1\mathsf{Alg} \to \mathcal{U}$ by $Q(\mathcal{A}) :\equiv \mathsf{isHinit}_{\mathsf{L}_1}(\mathcal{A})$, then

$$\mathsf{transport}^Q(q^{-1})\colon \mathsf{isHinit}_{\mathsf{L}_1}(\langle \mathsf{L}_1, (\mathsf{nil}, \mathsf{cons})\rangle) \to \mathsf{isHinit}_{\mathsf{L}_1}(\langle \mathbb{N}, (0, \mathsf{succ})\rangle).$$

Applying this function to the fact that $\langle \mathsf{L}_1, (\mathsf{nil}, \mathsf{cons})\rangle$ is $h$-initial provides the desired inhabitant. □

**Definition 5.4.6.** Let $A\colon \mathcal{U}$ be a type of labels and $B\colon A \to \mathcal{U}$ an arity family. For any type $X : \mathcal{U}$, the *polynomial functor* (in normal form) is the type family defined by:

$$\begin{aligned} P(X) &:\equiv \sum_{u\colon 1+A} B(u) \to X \\ &=_{\mathcal{U}} \Big(\sum_{u\colon 1} \mathsf{0} \to X\Big) + \sum_{a\colon A} \mathsf{1} \to X \\ &=_{\mathcal{U}} \mathsf{1} + A \times X. \end{aligned}$$

**Remark 5.4.7.** The notion of polynomial functor can be seen as a generalization of (5.3). The polynomial aspect of $P(X)$ becomes clear when $B(a) \to X$ is written as $X^{B(a)}$.

**Example 5.4.8.** As shown in Example 5.3.2, $\mathsf{L}_A$ can be seen as $W_{1+A}B$, where $B(\mathsf{inl}(\star)) \equiv \mathsf{0}$ and $B(\mathsf{inr}(a)) \equiv \mathsf{1}$. Thus, the associated polynomial functor is

$$\begin{aligned} P(X) &:\equiv \sum_{u\colon 1+A} B(u) \to X \\ &=_{\mathcal{U}} (\mathsf{0} \to X) + \sum_{a\colon A} (\mathsf{1} \to X) \\ &=_{\mathcal{U}} \mathsf{1} + A \times X. \end{aligned}$$

In particular, the constructors $c_{\mathsf{nil}}$ and $c_{\mathsf{cons}}$ can be encoded in a single constructor

$$\mathsf{c}\colon P(\mathsf{L}_A) \to \mathsf{L}_A,$$

with $P(\mathsf{L}_A)$ replaced by $\mathsf{1} + A \times \mathsf{L}_A$, $\mathsf{c}(\mathsf{inl}(\star)) :\equiv \mathsf{nil}$, and $\mathsf{c}(\mathsf{inr}(a, \ell)) :\equiv \mathsf{cons}(a, \ell)$.

**Definition 5.4.9.** The type of $\mathsf{W}$-*algebras* is

$$\mathsf{WAlg}(A, B) :\equiv \sum_{C:\mathcal{U}} \prod_{a:A} (B(a) \to C) \to C.$$

Its terms are dependent pairs $\langle C, \alpha_C \rangle$, where $\alpha_C : \prod_{a:A}(B(a) \to C) \to C$ is the *structure map* of the $\mathsf{W}$-algebra.

**Remark 5.4.10.** A $\mathsf{W}$-algebra $\langle C, \alpha_C \rangle$ admits an *uncurried variant* $\langle C, \bar{\alpha}_C \rangle$, where

$$\bar{\alpha}_C :\equiv \mathsf{rec}_{P(C)}(C, \alpha_C) : P(C) \to C.$$

We will refer to this form as a *$P$-algebra*.

These two views are interchangeable since $\alpha_C$ can be recovered from $\bar{\alpha}_C$ by the computation rule of the $\Sigma$-type.

**Definitions 5.4.11.**

- Given $P$-algebras $\langle C, \bar{\alpha}_C \rangle$ and $\langle D, \bar{\alpha}_D \rangle$, any map $f : C \to D$ induces a function $Pf : P(C) \to P(D)$ defined by

  $$Pf(\langle a, \phi \rangle) :\equiv \langle a, f \circ \phi \rangle,$$

  for $a : A$ and $\phi : B(a) \to C$.

- A *$P$-morphism* from a $P$-algebra $\langle C, \bar{\alpha}_C \rangle$ to another $\langle D, \bar{\alpha}_D \rangle$ is a dependent pair $\langle f, \eta_f \rangle$, where $f : C \to D$ and $\eta_f : f \circ \bar{\alpha}_C \sim \bar{\alpha}_D \circ Pf$. This homotopy can be visualized in the square:

  $$\begin{array}{ccc} P(C) & \xrightarrow{Pf} & P(D) \\ {\scriptstyle \bar{\alpha}_C}\downarrow & \eta_f & \downarrow{\scriptstyle \bar{\alpha}_D} \\ C & \xrightarrow[f]{} & D \end{array}$$

  Thus, the type of $P$-morphisms is

  $$\mathsf{PHom}(\langle C, \bar{\alpha}_C \rangle, \langle D, \bar{\alpha}_D \rangle) :\equiv \sum_{f : C \to D} f \circ \bar{\alpha}_C \sim \bar{\alpha}_D \circ Pf.$$

  Equivalently, using the curried variants $\alpha_C$ and $\alpha_D$, the homotopy condition is given by a family of identities, making the type of morphisms definitionally equivalent to

  $$\sum_{f : C \to D} \prod_{a:A} \prod_{\phi : B(a) \to C} f(\alpha_C(a, \phi)) =_D \alpha_D(a, f \circ \phi). \tag{5.1}$$

  We will denote the type of $\mathsf{W}$-morphisms by $\mathsf{WHom}(\langle C, \alpha_C \rangle, \langle D, \alpha_D \rangle)$, or $\mathsf{WHom}(C, D)$ for short, if the context permits.

- A $\mathsf{W}$-algebra $\langle I, \alpha_I \rangle$ is *h-initial* if the type

$$\mathsf{isHinit}_{\mathsf{W}}(A, B, \langle I, \alpha_I \rangle) :\equiv \prod_{\langle C, \alpha_C \rangle : \mathsf{WAlg}(A,B)} \mathsf{isContr}\big(\mathsf{WHom}(I, C)\big)$$

  is inhabited.

**Lemma 5.4.12.** *Recursion based on the long step function*

$$s_C : \prod_{a:\, A} (B(a) \to \mathsf{W}) \to (B(a) \to C) \to C$$

*of* (5.4) *is equivalent to recursion based on the short step function*

$$s'_C : \prod_{a:\, A} (B(a) \to C) \to C.$$

*That is, defining functions out of the* $\mathsf{W}$*-type* $\mathsf{W}_A B$ *using either method yields the same recursive function.*

*Proof.* Let $\mathsf{rec}_{\mathsf{W}}$ denote the recursion based on $s_C$ and $\mathsf{rec}'_{\mathsf{W}}$ the recursion based on $s'_C$.

We can obtain $\mathsf{rec}'_{\mathsf{W}}$ from $\mathsf{rec}_{\mathsf{W}}$ as

$$\mathsf{rec}'_{\mathsf{W}}(C, s'_C) :\equiv \mathsf{rec}_{\mathsf{W}}(C, s_C),$$

by setting

$$s_C(a, \phi, h) :\equiv s'_C(a, h).$$

Since the computation rule (5.7) for $\mathsf{rec}_{\mathsf{W}}$ establishes

$$\mathsf{rec}_{\mathsf{W}}(C, s_C)(\mathsf{sup}(a, \phi)) \equiv s_C(a, \phi, \mathsf{rec}_{\mathsf{W}}(C, s_C) \circ \phi),$$

we deduce the computation rule for the short recursor $\mathsf{rec}'_{\mathsf{W}}$:

$$\mathsf{rec}'_{\mathsf{W}}(C, s'_C)(\mathsf{sup}(a, \phi)) \equiv s'_C(a, \mathsf{rec}'_{\mathsf{W}}(C, s'_C) \circ \phi). \tag{5.2}$$

Conversely, suppose we are equipped with the short recursor, and we are given a step function $s_C$.

To construct $\mathsf{rec}_{\mathsf{W}}(C, s_C) : \mathsf{W} \to C$, we will use $\mathsf{rec}'_{\mathsf{W}}$ to define a map $\mathsf{W} \to C'$, where $C' :\equiv \mathsf{W} \times C$. We must provide a short step function $s'_{C'}$:

$$s'_{C'} : \prod_{a:\, A} (B(a) \to \mathsf{W} \times C) \to \mathsf{W} \times C.$$

Fix $a : A$ and let $\psi : B(a) \to \mathsf{W} \times C$. By composing with the standard projections, we obtain $\psi_1 :\equiv \pi_1 \circ \psi : B(a) \to \mathsf{W}$ and $\psi_2 :\equiv \pi_2 \circ \psi : B(a) \to C$. Then, we can specify

$$s'_{C'}(a, \psi) :\equiv \big(\mathsf{sup}(a, \psi_1), s_C(a, \psi_1, \psi_2)\big). \tag{$*$}$$

Applying the short recursor to this structure yields the map

$$\mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}) \colon \mathsf{W} \to \mathsf{W} \times C,$$

which produces the long recursor:

$$\mathsf{rec}_{\mathsf{W}}(C, s_C) :\equiv \pi_2 \circ \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}).$$

From the computation rule (5.2) for the short recursor, we deduce

$$\begin{aligned}
\mathsf{rec}_{\mathsf{W}}(C, s_C)(\mathsf{sup}(a, \phi)) &\equiv \pi_2\big(\mathsf{rec}'_{\mathsf{W}}(C', s'_{C'})(\mathsf{sup}(a, \phi))\big) \\
&\equiv \pi_2\big(s'_{C'}(a, \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}) \circ \phi)\big) \\
&\equiv s_C\big(a, \pi_1 \circ \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}) \circ \phi, \pi_2 \circ \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}) \circ \phi\big) \\
&\equiv s_C\big(a, \pi_1 \circ \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}) \circ \phi, \mathsf{rec}_{\mathsf{W}}(C, s_C) \circ \phi\big).
\end{aligned}$$

We claim that $\pi_1 \circ \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'})$ is propositionally equal to $\mathsf{id}_{\mathsf{W}}$.

To see this, let $\rho :\equiv \pi_1 \circ \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'})$. By function extensionality, it suffices to construct a homotopy of type $\rho \sim \mathsf{id}_{\mathsf{W}}$. Define the motive $E(w) :\equiv \rho(w) =_{\mathsf{W}} w$. To provide the inductive step function, let us fix $a \colon A$ and $\phi \colon B(a) \to \mathsf{W}$. We assume the inductive hypothesis

$$h \colon \prod_{b \colon B(a)} \rho(\phi(b)) =_{\mathsf{W}} \phi(b).$$

This hypothesis is a homotopy of type $\rho \circ \phi \sim \phi$. By function extensionality, we obtain a path

$$\mathsf{funext}(h) \colon \rho \circ \phi =_{B(a) \to \mathsf{W}} \phi.$$

Now, we evaluate $\rho(\mathsf{sup}(a, \phi))$ using the computation rule (5.2) for $\mathsf{rec}'_{\mathsf{W}}$ and the definitional equality $(*)$:

$$\begin{aligned}
\rho(\mathsf{sup}(a, \phi)) &\equiv \pi_1\big(\mathsf{rec}'_{\mathsf{W}}(C', s'_{C'})(\mathsf{sup}(a, \phi))\big) \\
&\equiv \pi_1\big(s'_{C'}(a, \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}) \circ \phi)\big) \\
&\equiv \mathsf{sup}(a, \pi_1 \circ \mathsf{rec}'_{\mathsf{W}}(C', s'_{C'}) \circ \phi) \qquad ; (*) \\
&\equiv \mathsf{sup}(a, \rho \circ \phi).
\end{aligned}$$

Applying the action on paths of $\mathsf{sup}(a, -)$, we obtain

$$\mathsf{ap}_{\mathsf{sup}(a,-)}(\mathsf{funext}(h)) \colon \mathsf{sup}(a, \rho \circ \phi) =_{\mathsf{W}} \mathsf{sup}(a, \phi).$$

Since $\rho(\mathsf{sup}(a, \phi)) \equiv \mathsf{sup}(a, \rho \circ \phi)$, this path establishes $E(\mathsf{sup}(a, \phi))$, completing the induction.

Consequently,

$$\mathsf{rec}_{\mathsf{W}}(C, s_C)(\mathsf{sup}(a, \phi)) =_C s_C(a, \phi, \mathsf{rec}_{\mathsf{W}}(C, s_C) \circ \phi),$$

i.e., the constructed map satisfies the computation rule of the long recursor up to propositional equality. □

**Notation 5.4.13.** *Let* $\mathsf{W}$ *denote the* $\mathsf{W}$*-type* $\mathsf{W}_A B$.

- *The short recursor* $\mathsf{rec}'_{\mathsf{W}}$ *of* Lemma 5.4.12 *will be denoted by* $\mathsf{hom}_{\mathsf{W}}$*, i.e.,*

$$\mathsf{hom}_{\mathsf{W}} \colon \prod_{C:\mathcal{U}} \Big(\prod_{a:\, A} (B(a) \to C) \to C\Big) \to \mathsf{W} \to C.$$

- *For any type* $C \colon \mathcal{U}$ *the short structure map* $s'_C$ *of said lemma will be denoted by* $\alpha_C$*, i.e.,*

$$\alpha_C \colon \prod_{a:\, A} (B(a) \to C) \to C.$$

  *Thus, the recursion map* $\mathsf{hom}_{\mathsf{W}}(C, \alpha_C) \colon \mathsf{W} \to C$ *is defined inductively on constructors by the rule*

$$\mathsf{hom}_{\mathsf{W}}(C, \alpha_C)(\mathsf{sup}(a, \phi)) \equiv \alpha_C(a, \mathsf{hom}_{\mathsf{W}}(C, \alpha_C) \circ \phi),$$

  *for any* $a \colon A$ *and* $\phi \colon B(a) \to \mathsf{W}$.

- *The term*

$$\mathsf{comp}_{\mathsf{W}}(C, \alpha_C, a, \phi) :\equiv \mathsf{refl}_{\mathsf{hom}_{\mathsf{W}}(C,\alpha_C)(\mathsf{sup}(a,\phi))}$$

  *will denote the witness for the computation rule* (5.7)*. Specifically, it is the trivial inhabitant of the identity type*

$$\mathsf{hom}_{\mathsf{W}}(C, \alpha_C)(\mathsf{sup}(a, \phi)) =_C \alpha_C(a, \mathsf{hom}_{\mathsf{W}}(C, \alpha_C) \circ \phi). \tag{5.3}$$

**Lemma 5.4.14.** *Let* $H, C \colon \mathcal{U}$ *be types and* $L, R \colon H \to C$ *functions. Given a path* $p \colon h_1 =_H h_2$ *and a path* $q \colon L(h_1) =_C R(h_1)$*, we have*

$$\mathsf{transport}^{h \mapsto L(h) =_C R(h)}(p, q) =_{L(h_2) =_C R(h_2)} \mathsf{ap}_L(p)^{-1} \cdot q \cdot \mathsf{ap}_R(p).$$

*Proof.* First observe that, in fact, both sides of the target identity inhabit the type $L(h_2) =_C R(h_2)$. Hence, we can apply path induction on $p$. It suffices to consider the case where $h_2 \equiv h_1$ and $p \equiv \mathsf{refl}_{h_1}$.

In this case, the LHS becomes definitionally $q$:

$$\mathsf{transport}^{h \mapsto L(h) =_C R(h)}(\mathsf{refl}_{h_1}, q) \equiv q.$$

Since $\mathsf{ap}_L(\mathsf{refl}_{h_1}) \equiv \mathsf{refl}_{L(h_1)}$ and $\mathsf{ap}_R(\mathsf{refl}_{h_1}) \equiv \mathsf{refl}_{R(h_1)}$, the RHS reduces to

$$\mathsf{refl}^{-1}_{L(h_1)} \cdot q \cdot \mathsf{refl}_{R(h_1)}.$$

The conclusion follows by the unit laws for path concatenation. □

**Lemma 5.4.15.** *Let* $\langle D, \alpha_D \rangle$ *and* $\langle C, \alpha_C \rangle$ *be two* $\mathsf{W}$*-algebras. Let* $\langle f, \eta_f \rangle$ *and* $\langle g, \eta_g \rangle$ *be two* $\mathsf{W}$*-morphisms from* $\langle D, \alpha_D \rangle$ *to* $\langle C, \alpha_C \rangle$*. The identity type*

$$\langle f, \eta_f \rangle =_{\mathsf{WHom}(D,C)} \langle g, \eta_g \rangle$$

*is equivalent to the type of dependent pairs* $\langle e, \sigma_e \rangle$*, where* $e\colon f \sim g$ *is a homotopy and* $\sigma_e$ *is a family of paths assigning to each* $a\colon A$ *and* $\phi\colon B(a) \to D$ *a path*

$$\sigma_e(a,\phi)\colon e(\alpha_D(a,\phi)) = \eta_f(a,\phi) \cdot \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi)) \cdot \eta_g(a,\phi)^{-1}$$

*in* $f(\alpha_D(a,\phi)) =_C g(\alpha_D(a,\phi))$*. In other words, the path diagram*

$$\begin{array}{ccc} f(\alpha_D(a,\phi)) & \xrightarrow{\quad e(\alpha_D(a,\phi)) \quad} & g(\alpha_D(a,\phi)) \\ {\scriptstyle \eta_f(a,\phi)}\big\downarrow & \sigma_e(a,\phi) \quad \eta_g(a,\phi)^{-1}\big\uparrow & \circlearrowright\ {\scriptstyle \eta_g(a,\phi)} \\ \alpha_C(a, f \circ \phi) & \xrightarrow[\mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi))]{} & \alpha_C(a, g \circ \phi) \end{array}$$

*commutes propositionally if, and only if,* $\langle f, \eta_f \rangle =_{\mathsf{WHom}(D,C)} \langle g, \eta_g \rangle$*.*

*Proof.* Equating the dependent pairs $\langle f, \eta_f \rangle$ and $\langle g, \eta_g \rangle$ in the $\Sigma$-type of W-morphisms $\mathsf{WHom}(D, C)$ is equivalent to providing a path $p\colon f =_{D \to C} g$ and a path $q$ witnessing the identity between the transport of $\eta_f$ along $p$ and $\eta_g$.

By function extensionality, the path $p$ is equivalent to a homotopy $e\colon f \sim g$, giving $p :\equiv \mathsf{funext}(e)$.

For any $a\colon A$ and $\phi\colon B(a) \to D$, let us define the local terms

$$L_{a,\phi}(h) :\equiv h(\alpha_D(a,\phi)) \quad \text{and} \quad R_{a,\phi}(h) :\equiv \alpha_C(a, h \circ \phi).$$

This allows us to define the dependent type family $E$ as

$$E(h) :\equiv \prod_{a\colon A} \prod_{\phi\colon B(a) \to D} L_{a,\phi}(h) =_C R_{a,\phi}(h).$$

According to (5.1), we have $\eta_f\colon E(f)$ and $\eta_g\colon E(g)$, and the transport condition mentioned above translates into

$$\mathsf{transport}^E(p, \eta_f) = \eta_g.$$

By applying Proposition 2.11.11 to the outer product over $A$, we obtain

$$\mathsf{transport}^E(p, \eta_f) = \lambda a.\, \mathsf{transport}^{\lambda h.\, \prod_{\phi\colon B(a) \to D} L_{a,\phi}(h) =_C R_{a,\phi}(h)}(p, \eta_f(a)).$$

Applying the proposition a second time to the inner product over $B(a) \to D$ yields

$$\mathsf{transport}^E(p, \eta_f) = \lambda a.\, \lambda \phi.\, \mathsf{transport}^{\lambda h.\, L_{a,\phi}(h) =_C R_{a,\phi}(h)}(p, \eta_f(a,\phi)).$$

By Lemma 5.4.14, we obtain

$$\mathsf{transport}^{h \mapsto L_{a,\phi}(h) =_C R_{a,\phi}(h)}(p, \eta_f(a,\phi)) = \mathsf{ap}_{L_{a,\phi}}(p)^{-1} \cdot \eta_f(a,\phi) \cdot \mathsf{ap}_{R_{a,\phi}}(p). \quad (*)$$

Computing the action of $\mathsf{ap}_{L_{a,\phi}}$ and $\mathsf{ap}_{R_{a,\phi}}$ for $p :\equiv \mathsf{funext}(e)$ gives:

a) $\mathsf{ap}_{L_{a,\phi}}(\mathsf{funext}(e)) = e(\alpha_D(a,\phi))$.

b) $\mathsf{ap}_{R_{a,\phi}}(\mathsf{funext}(e)) = \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi))$.

Substituting these into the transport equation $(*)$ yields

$$e(\alpha_D(a,\phi))^{-1} \cdot \eta_f(a,\phi) \cdot \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi)).$$

The coherence condition requires this transported path to equal $\eta_g(a,\phi)$:

$$e(\alpha_D(a,\phi))^{-1} \cdot \eta_f(a,\phi) \cdot \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi)) = \eta_g(a,\phi).$$

Isolating $e(\alpha_D(a,\phi))$ yields the required coherence condition:

$$e(\alpha_D(a,\phi)) = \eta_f(a,\phi) \cdot \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi)) \cdot \eta_g(a,\phi)^{-1}.$$

This completes the proof of the *if* part.

For the *only if* part, note that the equality $\langle f, \eta_f \rangle = \langle g, \eta_g \rangle$ means the existence of paths

$$\begin{aligned} &p\colon f =_{D \to C} g, \\ &q\colon \mathsf{transport}^E(p, \eta_f) =_{E(g)} \eta_g. \end{aligned}$$

Thus, we can apply $\mathsf{happly}$ to $q$ to translate this global equality into the local path:

$$\mathsf{happly}(q,a)\colon \mathsf{transport}^E(p,\eta_f)(a) =_{B(a) \to D} \eta_g(a).$$

By applying $\mathsf{happly}$ a second time we obtain

$$\mathsf{happly}(\mathsf{happly}(q,a),\phi)\colon \mathsf{transport}^E(p,\eta_f)(a,\phi) = \eta_g(a,\phi).$$

According to Proposition 2.11.11 and Lemma 5.4.14, the left-hand side reduces point-wise. Thus, this double application of $\mathsf{happly}$ yields the local path

$$e(\alpha_D(a,\phi))^{-1} \cdot \eta_f(a,\phi) \cdot \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi)) = \eta_g(a,\phi),$$

which we can use to derive $\sigma_e(a,\phi)$. □

**Theorem 5.4.16.** *For any type $A\colon \mathcal{U}$ and type family $B\colon A \to \mathcal{U}$, the $\mathsf{W}$-algebra $\langle \mathsf{W}_A B, \mathsf{sup} \rangle$ is h-initial.*

*Proof.* Let $\mathsf{W} :\equiv \mathsf{W}_A B$. We must show that for any $\mathsf{W}$-algebra $\langle C, \alpha_C \rangle$, the type $\mathsf{WHom}(\mathsf{W}, C)$ is contractible.

For the center of contraction $\langle f, \eta_f \rangle$, let $f\colon \mathsf{W} \to C$ be defined by (5.5), i.e.,

$$f :\equiv \mathsf{hom}_{\mathsf{W}}(C, \alpha_C).$$

The homotopy condition (5.1) for $\eta_f$ requires that, for $a\colon A$ and $\phi\colon B(a) \to \mathsf{W}$, we have

$$f(\mathsf{sup}(a,\phi)) =_C \alpha_C(a, f \circ \phi).$$

This equality is exactly the computation rule (5.3) for the $\mathsf{W}$-recursor. Thus, using Notation 5.4.13, we define $\eta_f(a,\phi) :\equiv \mathsf{comp}_{\mathsf{W}}(C, \alpha_C, a, \phi)$.

To prove contractibility, it remains to show that for any other $\mathsf{W}$-morphism $\langle g, \eta_g\rangle$, there is an identity $\langle f, \eta_f\rangle =_{\mathsf{WHom}(\mathsf{W},C)} \langle g, \eta_g\rangle$.

By Lemma 5.4.15, providing such an identity is equivalent to providing a dependent pair $\langle e, \sigma_e\rangle$, where $e\colon f \sim g$ and $\sigma_e(a,\phi)$ witnesses the equality

$$e(\mathsf{sup}(a,\phi)) = \eta_f(a,\phi) \cdot \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(e \triangleright \phi)) \cdot \eta_g(a,\phi)^{-1}.$$

To construct the homotopy $e$, we proceed by $\mathsf{W}$-induction for the type family $E\colon \mathsf{W} \to \mathcal{U}$ defined by

$$E(w) :\equiv f(w) =_C g(w).$$

According to (5.2), given

$$\epsilon\colon \prod_{b\colon B(a)} f(\phi(b)) =_C g(\phi(b)),$$

we need to provide a path in $f(\mathsf{sup}(a,\phi)) =_C g(\mathsf{sup}(a,\phi))$. We define

$$e_s(a,\phi,\epsilon) :\equiv \eta_f(a,\phi) \cdot \mathsf{ap}_{\alpha_C(a,-)}(\mathsf{funext}(\epsilon)) \cdot \eta_g(a,\phi)^{-1}.$$

By the computation rule (5.6), the resulting homotopy $e\colon f \sim g$ satisfies

$$e(\mathsf{sup}(a,\phi)) = e_s(a,\phi, e \triangleright \phi),$$

since $e \triangleright \phi \equiv e \circ \phi$.

Thus, the required coherence condition $\sigma_e$ of Lemma 5.4.15 is satisfied, yielding the required identity $\langle f, \eta_f\rangle =_{\mathsf{WHom}(\mathsf{W},C)} \langle g, \eta_g\rangle$.

□

## 5.5 General Recursion

Historically, the recursion principles for inductive types were formulated using syntactic text-replacement schemas. While intuitive for simple types, these schemas proved logically inconsistent when extended to operations with negative occurrences. Modern type theory resolves this by grounding recursion in the categorical semantics of endofunctors. In this section we provide both definitions.

**Definition 5.5.1.** Given an arbitrary type-level operation $\Phi : \mathcal{U} \to \mathcal{U}$, the *step-function domain* $\Phi'(X, P)$ is defined by structural induction on $\Phi$:

a) Constants and parameter types remain unchanged.

b) Negative occurrences of $X$ remain $X$.

c) Strictly positive occurrences of $X$ are replaced by $P$.

When $\Phi$ is strictly positive everywhere, $\Phi'(X, P) \equiv \Phi(P)$.

**Definition 5.5.2.** Let $\Phi : \mathcal{U} \to \mathcal{U}$ be an arbitrary type-level operation representing the signature of a constructor. The *syntactic recursion schema* for a type $X$ with constructor $\mathsf{c} : \Phi(X) \to X$ postulates a recursor

$$\mathsf{rec}_X^{\mathrm{syn}} : \prod_{P:\mathcal{U}} \big(\Phi(X) \to \Phi'(X, P) \to P\big) \to X \to P,$$

with computation rule

$$f(\mathsf{c}(x)) \equiv s\big(x, \mathsf{map}_\Phi(f)(x)\big) \tag{5.1}$$

for all $x : \Phi(X)$.

**Remark 5.5.3.** The syntactic schema operates by blind textual substitution. For strictly positive types with constant parameters, it forces the step function to accept redundant copies of the non-recursive data. More critically, as we will see in the exercises, if $X$ occurs negatively in $\Phi(X)$, the substitution $\Phi'(X, P)$ generates a spurious inductive hypothesis that leads to contradictions.

**Definition 5.5.4.** Let $\Phi$ be a strictly positive type-level operation and let $\Phi'(X, P)$ denote its associated step-function domain. The *categorical recursion principle* for a type $X$ recursively defined by $\mathsf{c} : \Phi(X) \to X$ postulates a recursor

$$\mathsf{rec}_X : \prod_{P:\mathcal{U}} \big(\Phi'(X, X \times P) \to P\big) \to X \to P.$$

For any target type $P$ and step function $s : \Phi'(X, X \times P) \to P$, the function $f :\equiv \mathsf{rec}_X(P, s)$ satisfies the computation rule

$$f(\mathsf{c}(u)) \equiv s\big(\mathsf{map}_\Phi(\lambda x.\, (x, f(x)))(u)\big) \tag{5.2}$$

for all $u : \Phi(X)$, where $\mathsf{map}_\Phi$ denotes the functorial action of $\Phi$. This computation rule is well-typed because:

1. $\mathsf{c} : \Phi(X) \to X$, which forces $u : \Phi(X)$,
2. $f : X \to P$,
3. $g :\equiv \lambda x.\, (x, f(x)) : X \to X \times P$,
4. $\mathsf{map}_\Phi(g) : \Phi(X) \to \Phi(X \times P)$,

5. 5. by Definition 5.5.1, the strict positivity of $\Phi$ guarantees that $\Phi'(X, X \times P) \equiv \Phi(X \times P)$.

**Example 5.5.5.** The constructor of $\mathsf{W} :\equiv \mathsf{W}_A B$

$$\mathsf{sup}\colon \prod_{a\colon A}(B(a) \to \mathsf{W}) \to \mathsf{W}$$

can be rewritten as

$$\Big(\sum_{a\colon A} B(a) \to \mathsf{W}\Big) \to \mathsf{W}.$$

Thus,

$$\Phi(X) \equiv \sum_{a\colon A} B(a) \to X,$$

and

$$\Phi(\mathsf{W} \times C) \equiv \sum_{a\colon A} B(a) \to \mathsf{W} \times C.$$

Therefore, the step function becomes

$$s\colon \Big(\sum_{a\colon A} B(a) \to \mathsf{W} \times C\Big) \to C,$$

which can be rewritten as

$$s\colon \prod_{a\colon A}(B(a) \to \mathsf{W} \times C) \to C.$$

Since $B(a) \to \mathsf{W} \times C$ equals $(B(a) \to \mathsf{W}) \times (B(a) \to C)$, by currying we recover the specification given in (5.4).

Finally, Definition 5.5.4 yields

$$\mathsf{rec}_{\mathsf{W}}(C, s)\colon \mathsf{W} \to C.$$

## 5.6 Homotopy-Inductive Types

Let us get back to the alternative definition $\mathbb{N}'$ of the natural numbers given in Example 5.2.4, namely

- $\mathbb{N}' :\equiv \mathsf{W}_2\mathsf{rec}_2(\mathcal{U}, 0, 1)$,
- $0' :\equiv \mathsf{sup}(0_2, \mathsf{rec}_0(\mathbb{N}'))$, and
- $\mathsf{succ}'(n) :\equiv \mathsf{sup}(1_2, \lambda\star.\, n)$.

To see if the standard computation rules for natural numbers can be derived from the rules for W-types, we must define the step function $e_s$ as specified in (5.2), namely:

$$e_s\colon \prod_{a\colon 2} \prod_{\phi\colon B(a)\to\mathbb{N}'} \Big(\prod_{b\colon B(a)} E(\phi(b))\Big) \to E(\mathsf{sup}(a, \phi)).$$

Consequently, we assume we are given the standard inductive data for the natural numbers: a motive $E$, a base case $e_{0'} : E(0')$, and a successor step function

$$e_{\mathsf{succ}'} : \prod_{n:\,\mathbb{N}'} E(n) \to E(\mathsf{sup}(1_2, \lambda\star.\, n)).$$

To derive the standard induction principle for $\mathbb{N}'$ from the $\mathsf{W}$-induction, we must define $\mathsf{ind}_{\mathbb{N}'}$ in terms of $\mathsf{ind}_{\mathsf{W}}$:

$$\mathsf{ind}_{\mathbb{N}'}(E, e_{0'}, e_{\mathsf{succ}'}) :\equiv \mathsf{ind}_{\mathsf{W}}(E, e_s).$$

Moreover, this definition must satisfy:

$$\mathsf{ind}_{\mathbb{N}'}(E, e_{0'}, e_{\mathsf{succ}'})(\mathsf{succ}'(n)) \equiv e_{\mathsf{succ}'}\big(n, \mathsf{ind}_{\mathbb{N}'}(E, e_{0'}, e_{\mathsf{succ}'})(n)\big).$$

If we substitute our definition of $\mathsf{ind}_{\mathbb{N}'}$ into this expected identity, we obtain:

$$\mathsf{ind}_{\mathsf{W}}(E, e_s)(\mathsf{succ}'(n)) \equiv e_{\mathsf{succ}'}\big(n, \mathsf{ind}_{\mathsf{W}}(E, e_s)(n)\big). \tag{5.1}$$

Given $\phi : 1 \to \mathbb{N}'$ and $h : \prod_{b:\,1} E(\phi(b))$, to define $e_s(1_2, \phi, h)$, we must construct a term of type $E(\mathsf{sup}(1_2, \phi))$.

Since the domain of both $\phi$ and $h$ is $1$, we can apply them to $\star : 1$, obtaining

$$\begin{aligned} &\phi(\star) : \mathbb{N}', \\ &h(\star) : E(\phi(\star)). \end{aligned}$$

We can now pass these terms to $e_{\mathsf{succ}'}$, which yields

$$e_{\mathsf{succ}'}(\phi(\star), h(\star)) : E(\mathsf{sup}(1_2, \lambda\star.\, \phi(\star))).$$

This codomain is definitionally equal to $E(\mathsf{sup}(1_2, \phi))$. Since this matches the required type, we can define the step function as

$$e_s(1_2, \phi, h) :\equiv e_{\mathsf{succ}'}(\phi(\star), h(\star)). \tag{5.2}$$

Substituting this definition into the $\mathsf{W}$-type computation rule for $\mathsf{succ}'(n)$, we obtain

$$\begin{aligned} \mathsf{ind}_{\mathsf{W}}(E, e_s)(\mathsf{succ}'(n)) &\equiv e_s\big(1_2, \lambda\star.\, n, \mathsf{ind}_{\mathsf{W}}(E, e_s) \circ (\lambda\star.\, n)\big) \\ &\equiv e_{\mathsf{succ}'}\big((\lambda\star.\, n)(\star), (\mathsf{ind}_{\mathsf{W}}(E, e_s) \circ (\lambda\star.\, n))(\star)\big) \\ &\equiv e_{\mathsf{succ}'}\big(n, \mathsf{ind}_{\mathsf{W}}(E, e_s)(n)\big), \end{aligned}$$

which reconstructs the expected recurrence (5.1).

However, the problem arises because the base case fails. This requirement translates into the definitional equality

$$\mathsf{ind}_{\mathbb{N}'}(E, e_{0'}, e_{\mathsf{succ}'})(0') \equiv e_{0'}.$$

In our setup, substituting the definition of $\mathsf{ind}_{\mathbb{N}'}$ translates this requirement into

$$\mathsf{ind}_{\mathsf{W}}(E, e_s)(0') \equiv e_{0'}.$$

To see why this fails, let us evaluate the left-hand side. We have

$$\begin{aligned}\mathsf{ind}_{\mathsf{W}}(E, e_s)(0') &\equiv \mathsf{ind}_{\mathsf{W}}(E, e_s)(\mathsf{sup}(0_2, \mathsf{rec}_0(\mathbb{N}'))) \\ &\equiv e_s\big(0_2, \mathsf{rec}_0(\mathbb{N}'), \mathsf{ind}_{\mathsf{W}}(E, e_s) \circ \mathsf{rec}_0(\mathbb{N}')\big) \qquad ; (5.6).\end{aligned}$$

The required signature for the base case is

$$e_s(0_2, -, -) \colon \prod_{\phi \colon 0 \to \mathbb{N}'} \Big( \prod_{b \colon 0} E(\phi(b)) \Big) \to E(\mathsf{sup}(0_2, \phi)).$$

We are given the standard base case data

$$e_{0'} \colon E(0') \equiv E(\mathsf{sup}(0_2, \mathsf{rec}_0(\mathbb{N}'))).$$

To define $e_s(0_2, \phi, h) :\equiv e_{0'}$, we would need the strict syntactic match between codomains:

$$E(\mathsf{sup}(0_2, \phi)) \equiv E(\mathsf{sup}(0_2, \mathsf{rec}_0(\mathbb{N}'))).$$

However, this would require

$$\phi \equiv \mathsf{rec}_0(\mathbb{N}'),$$

which does not hold because not all functions out of $\mathsf{0}$ are judgmentally equal.

As a result, the direct assignment fails to typecheck natively.

### 5.6.1 Bridging the Gap

Of course, given $\phi \colon \mathsf{0} \to \mathbb{N}'$ there is a trivial homotopy $\phi \sim \mathsf{rec}_0(\mathbb{N}')$. By function extensionality, this gives a path $p_\phi \colon \phi =_{0 \to \mathbb{N}'} \mathsf{rec}_0(\mathbb{N}')$.

Thus, to bridge the gap described above, we must transport the given term $e_{0'}$ along this path:

$$e_s(0_2, \phi, h) :\equiv \mathsf{transport}^{E(\mathsf{sup}(0_2, -))}(p_\phi^{-1}, e_{0'}).$$

When we compute the $\mathsf{W}$-type induction for $0'$, we obtain

$$\mathsf{ind}_{\mathsf{W}}(E, e_s)(0') \equiv \mathsf{transport}^{E(\mathsf{sup}(0_2, -))}(p_\phi^{-1}, e_{0'}).$$

Because $p_\phi$ is constructed using the function extensionality axiom, which is not a constructor that can be evaluated, the transport cannot reduce any further. Therefore, the judgmental equality $\mathsf{ind}_{\mathsf{W}}(E, e_s)(0') \equiv e_{0'}$ fails, holding only propositionally.

**Note.** Although relaxing these computation rules to hold only propositionally might seem like a viable workaround, the resulting loss of judgmental equality is

precisely why modern proof assistants abandon the W-type encoding altogether. By elevating *all* inductive types to primitive status, these systems ensure that base constructors like nil and zero satisfy their computation rules judgmentally.

As Chapter 6 demonstrates, this foundational shift is also a structural necessity for accommodating higher-dimensional path constructors.

Nevertheless, it is important to emphasize that while the W-type encoding is a pragmatic dead-end for language designers, it provides a crucial foundational framework for homotopy type theorists.

## 5.7 Strict Positivity

The aim of this section is to establish the requirements that constructors of inductive types must comply with in order to ensure valid definitions of new types.

The main idea is that a proper constructor may admit arguments that are functions, but the type being defined cannot be the domain of those functional arguments.

A good way to illustrate the necessity of this restriction is by showing how ignoring it leads to the Russell paradox.

Suppose we define an inductive type $X$ by means of a single constructor that violates our restriction:

$$\mathsf{c}\colon (X \to 2) \to X.$$

Because $X$ is an inductive type, we should be able to define, for any $C\colon \mathcal{U}$, a function $u\colon X \to C$ by specifying how $u$ operates on points of $X$ constructed from functions $f\colon X \to 2$ via $\mathsf{c}$.

For example, in the case $C :\equiv X \to 2$, we could define

$$u\colon X \to (X \to 2)$$

by establishing

$$u(\mathsf{c}(f)) :\equiv f. \tag{1}$$

Following Russell's paradox, this would allow us to build a diagonal function $d\colon X \to 2$ as

$$d(x) :\equiv \mathsf{not}(u(x)(x)). \tag{2}$$

In consequence, we could define

$$x_0 :\equiv \mathsf{c}(d). \tag{3}$$

The contradiction explodes when we try to evaluate the function $d$ on the element $x_0$. Indeed,

$$d(x_0) \overset{(3)}{\equiv} d(\mathsf{c}(d)) \overset{(2)}{\equiv} \mathsf{not}(u(\mathsf{c}(d))(x_0)) \overset{(1)}{\equiv} \mathsf{not}(d(x_0)),$$

which is impossible in the type 2 [cf. Exercise 2.19.12].

### 5.7.1 Free Variables

**Definition 5.7.1.** Let $\Phi$ be a type expression, and let $\mathcal{F}(\Phi)$ denote the set of *free variables* in $\Phi$, defined recursively on the syntactic structure of the expression as follows:

- $\mathcal{F}(V) :\equiv \{V\}$ for any variable $V$.
- $\mathcal{F}(C) :\equiv \emptyset$ for any constant type $C$.
- $\mathcal{F}(A + B) :\equiv \mathcal{F}(A) \cup \mathcal{F}(B)$.
- $\mathcal{F}\big(\prod_{y:\,A} B(y)\big) :\equiv \mathcal{F}(A) \cup (\mathcal{F}(B(y)) \setminus \{y\})$.
- $\mathcal{F}\big(\sum_{y:\,A} B(y)\big) :\equiv \mathcal{F}(A) \cup (\mathcal{F}(B(y)) \setminus \{y\})$.

**Remarks 5.7.2.**

- The variable $y$ in the last two parts of the definition is excluded from the set of free variables because it is free in $B(y)$ but gets *bound* in the entire expression.
- The last two parts of the definition imply that $\mathcal{F}(A \to B)$ and $\mathcal{F}(A \times B)$ equal $\mathcal{F}(A) \cup \mathcal{F}(B)$.
- Using this precise syntactic extraction, the informal statement "$X$ does not occur in $\Phi$" translates directly to the set-theoretic condition:

  $$X \notin \mathcal{F}(\Phi).$$

  Conversely, "$X$ occurs in $\Phi$" simply means $X \in \mathcal{F}(\Phi)$.

**Definition 5.7.3.** Let $X$ be a type variable and $\Phi$ a type expression. The predicate $\mathsf{Pos}_X(\Phi)$, denoting that $X$ occurs *strictly positively* in $\Phi$, holds when one of the following conditions is met:

a) $X \notin \mathcal{F}(\Phi)$,

b) $\Phi \equiv X$,

c) $\Phi \equiv \prod_{y:\,A} B(y)$, $X \notin \mathcal{F}(A)$, and $\mathsf{Pos}_X(B(y))$ holds,

d) $\Phi \equiv \sum_{y:\,A} B(y)$, $X \notin \mathcal{F}(A)$, and $\mathsf{Pos}_X(B(y))$ holds,

e) $\Phi \equiv A + B$, and $\mathsf{Pos}_X(A)$ and $\mathsf{Pos}_X(B)$ both hold.

**Remark 5.7.4.** Clauses c) and d) require $X \notin \mathcal{F}(A)$ to prevent $X$ from appearing anywhere inside the domain $A$, while recursively checking $\mathsf{Pos}_X(B(y))$ to allow $X$ to occur strictly positively in the codomain.

**Theorem 5.7.5.** *Let $X$ be a type variable and let $\Phi(X)$ be a type expression such that $\mathsf{Pos}_X(\Phi(X))$ holds. Given any two types $U$ and $V$, and a function $f\colon U \to V$, there exists a canonically defined function*

$$\mathsf{map}_\Phi(f)\colon \Phi(U) \to \Phi(V).$$

*Furthermore, this mapping preserves identities and composition, making $\Phi$ a covariant functor.*

*Proof.* We construct the function $\mathsf{map}_\Phi(f)$ by structural induction on the derivation of $\mathsf{Pos}_X(\Phi(X))$.

a) NON-OCCURRENCE: If $X \notin \mathcal{F}(\Phi)$, then $\Phi(U) \equiv \Phi(V) \equiv \Phi$. We define $\mathsf{map}_\Phi(f) :\equiv \mathsf{id}_\Phi$, the identity function on $\Phi$.

b) BASE VARIABLE: If $\Phi(X) \equiv X$, then $\Phi(U) \equiv U$ and $\Phi(V) \equiv V$. We define $\mathsf{map}_\Phi(f) :\equiv f$.

c) COPRODUCT: Suppose $\Phi(X) \equiv A(X) + B(X)$, where both $\mathsf{Pos}_X(A(X))$ and $\mathsf{Pos}_X(B(X))$ hold. By the inductive hypothesis, we have

$$\mathsf{map}_A(f)\colon A(U) \to A(V) \quad \text{and} \quad \mathsf{map}_B(f)\colon B(U) \to B(V).$$

We define $\mathsf{map}_\Phi(f)$ by pattern matching on the coproduct:

$$\begin{aligned}\mathsf{map}_\Phi(f)(\mathsf{inl}(a)) &:\equiv \mathsf{inl}(\mathsf{map}_A(f)(a)),\\ \mathsf{map}_\Phi(f)(\mathsf{inr}(b)) &:\equiv \mathsf{inr}(\mathsf{map}_B(f)(b)).\end{aligned}$$

d) DEPENDENT PRODUCT: Suppose $\Phi(X) \equiv \prod_{y:\,A} B(X, y)$, where $X \notin \mathcal{F}(A)$ and $\mathsf{Pos}_X(B(X, y))$ holds. By the inductive hypothesis, for any $y\colon A$, we have a function $\mathsf{map}_{B(y)}(f)\colon B(U, y) \to B(V, y)$. Given a dependent function $g\colon \prod_{y:\,A} B(U, y)$, we define

$$\mathsf{map}_\Phi(f)(g) :\equiv \lambda y.\, \mathsf{map}_{B(y)}(f)(g(y)).$$

e) DEPENDENT SUM: Suppose $\Phi(X) \equiv \sum_{y:\,A} B(X, y)$, where $X \notin \mathcal{F}(A)$ and $\mathsf{Pos}_X(B(X, y))$ holds. By the inductive hypothesis, we again have $\mathsf{map}_{B(y)}(f)$ for any $y\colon A$. Given a pair $p\colon \sum_{y:\,A} B(U, y)$, we define

$$\mathsf{map}_\Phi(f)(p) :\equiv \big(\pi_1(p), \mathsf{map}_{B(\pi_1(p))}(f)(\pi_2(p))\big).$$

This completes the construction. To complete the proof of functoriality, it remains to show that $\mathsf{map}_\Phi(\mathsf{id}_U)$ is propositionally equal to $\mathsf{id}_{\Phi(U)}$ and $\mathsf{map}_\Phi(g \circ f)$ is propositionally equal to $\mathsf{map}_\Phi(g) \circ \mathsf{map}_\Phi(f)$. We omit the details because these properties follow by structural induction under the same case analysis as above. □

**Example 5.7.6.** The converse statement, functoriality implies strict positivity, is false. Consider the double continuation type former, using the boolean type $2$:

$$\Phi(X) :\equiv (X \to 2) \to 2.$$

This type expression is *not* strictly positive. According to our definition, for $\mathsf{Pos}_X((X \to 2) \to 2)$ to hold, we would need $X$ not to occur in the domain $X \to 2$. Because $X \in \mathcal{F}(X \to 2)$, the strict positivity check fails. In this expression, $X$ is nested inside two negative positions (left sides of arrows), which makes it an "ordinarily positive" position, but not a strictly positive one.

However, $\Phi$ natively defines a covariant functor. Given any function $f : U \to V$, we can define the mapping $\mathsf{map}_\Phi(f) : \Phi(U) \to \Phi(V)$ as follows:

$$\mathsf{map}_\Phi(f)(g) :\equiv \lambda h.\, g(h \circ f),$$

where $g : (U \to 2) \to 2$ and $h : V \to 2$. This mapping also preserves identities and compositions.

**Remark 5.7.7.** The example highlights why type theory specifically demands *strict* positivity rather than just ordinary positivity. If we have $\Phi(X) :\equiv X \to A$, which is contravariant in the sense that it maps $f : U \to V$ to $f^* : \Phi(V) \to \Phi(U)$, then composing it with a second contravariant expression $\Phi'(X) :\equiv X \to A'$ results by double inversion of the arrows in the covariant behavior $f \mapsto f^{**}$.

## 5.8 Inductive Type Families

In this section we will see how a collection of types indexed by some other type can be simultaneously defined by induction. Instead of defining a single type $X$, we define a family of types $X : I \to \mathcal{U}$, where $I$ is the index type.

For a standard inductive type $X$, every constructor must output a value of type $X$. For an inductive type family $X : I \to \mathcal{U}$, the constructors can target a specific index $i : I$ to produce a value in $X(i)$.

**Example 5.8.1.** Let $A$ be a type. The inductive type family of *vectors over* $A$, denoted $\mathsf{Vec}_A : \mathbb{N} \to \mathcal{U}$, is generated by the following two constructors:

- $\mathsf{nil} : \mathsf{Vec}_A(0)$
- $\mathsf{cons} : \prod_{n:\mathbb{N}} A \to \mathsf{Vec}_A(n) \to \mathsf{Vec}_A(\mathsf{succ}(n))$

For a standard type $X$, the motive is a family of types $P : X \to \mathcal{U}$. For a type family $X : I \to \mathcal{U}$, the elements are distributed across different indices. Therefore, our motive must depend on both the index and the element. It becomes $P : \prod_{i:I} X(i) \to \mathcal{U}$.

**Example 5.8.2.** Following with Example 5.8.1, let $P\colon \prod_{n:\mathbb{N}} \mathsf{Vec}_A(n) \to \mathcal{U}$ be a family of types.

The eliminator for the vector family $\mathsf{ind}_{\mathsf{Vec}_A}$ takes two arguments corresponding to the constructors:

- a base case: $p_{\mathsf{nil}}\colon P(0, \mathsf{nil})$,
- an inductive step:

$$p_{\mathsf{cons}}\colon \prod_{n:\mathbb{N}} \prod_{a:A} \prod_{v:\mathsf{Vec}_A(n)} P(n, v) \to P(\mathsf{succ}(n), \mathsf{cons}(n, a, v)).$$

Given these, the eliminator produces a dependent function for all vectors at all indices:

$$\mathsf{ind}_{\mathsf{Vec}_A}(P, p_{\mathsf{nil}}, p_{\mathsf{cons}})\colon \prod_{n:\mathbb{N}} \prod_{v:\mathsf{Vec}_A(n)} P(n, v),$$

with computation rules:

$$\mathsf{ind}_{\mathsf{Vec}_A}(P, p_{\mathsf{nil}}, p_{\mathsf{cons}}, 0, \mathsf{nil}) \equiv p_{\mathsf{nil}}$$
$$\mathsf{ind}_{\mathsf{Vec}_A}\big(P, p_{\mathsf{nil}}, p_{\mathsf{cons}}, \mathsf{succ}(n), \mathsf{cons}(n, a, v)\big) \equiv p_{\mathsf{cons}}\big(n, a, v, \mathsf{ind}_{\mathsf{Vec}_A}(P, p_{\mathsf{nil}}, p_{\mathsf{cons}}, n, v)\big).$$

**Eliminator Methods.** The eliminator $\mathsf{ind}_X$ is a machine that builds proofs by induction. To operate this machine, we have to supply it with the *eliminator methods*.

Consider the natural numbers $\mathbb{N}$ with constructors $\mathsf{zero}\colon \mathbb{N}$ and $\mathsf{succ}\colon \mathbb{N} \to \mathbb{N}$. Let our motive be a property $P\colon \mathbb{N} \to \mathcal{U}$ that we want to prove for all natural numbers.

The eliminator $\mathsf{ind}_{\mathbb{N}}$ requires two methods, one for each constructor:

- *The method for* $\mathsf{zero}$*:* Since the $\mathsf{zero}$ constructor takes no arguments, this method reduces to proving the base case:

$$p_{\mathsf{zero}}\colon P(0).$$

- *The method for* $\mathsf{succ}$*:* The constructor $\mathsf{succ}$ takes a number $n$ and produces $n + 1$. The method must show that *if* the property holds for $n$, it holds for $n + 1$. This is the inductive step:

$$p_{\mathsf{succ}}\colon \prod_{n:\mathbb{N}} P(n) \to P(\mathsf{succ}(n)).$$

The eliminator consumes these two methods and outputs the global proof for all natural numbers:

$$\mathsf{ind}_{\mathbb{N}}(P, p_{\mathsf{zero}}, p_{\mathsf{succ}})\colon \prod_{n:\mathbb{N}} P(n).$$

**Remark 5.8.3.** From a programming perspective, the eliminator is a switch statement that branches based on how the data was constructed. The eliminator method is simply the block of code of one particular branch. If the branch involves recursion, the method receives as an argument the result of the recursive call.

**Definition 5.8.4.** Let $I$ be an index type and $X : I \to \mathcal{U}$ an inductive type family. A constructor $\mathsf{c}$ with no recursive arguments (a *base constructor*) is specified by two components:

- CONTEXT: A type $C$ for all non-recursive arguments.
- TARGET INDEX: A function $g : C \to I$ that determines the index of the constructed element.

The signature of the constructor is:

$$\mathsf{c} : \prod_{z:C} X(g(z)). \tag{5.1}$$

Given a motive $P : \prod_{i:I} X(i) \to \mathcal{U}$, the eliminator method $p_{\mathsf{c}}$ corresponding to this base constructor must satisfy:

$$p_{\mathsf{c}} : \prod_{z:C} P(g(z), \mathsf{c}(z)).$$

**Example 5.8.5.** For the $\mathsf{nil}$ constructor of the vector family $\mathsf{Vec}_A : \mathbb{N} \to \mathcal{U}$, there are no non-recursive arguments either. Therefore,

- CONTEXT: The context type is the unit type $\mathbf{1}$.
- TARGET INDEX: The empty vector has length 0, so $g(\star) :\equiv 0$.

This gives the signature $\mathsf{c} : \prod_{z:\mathbf{1}} \mathsf{Vec}_A(0)$, which is equivalent to giving the constant element:

$$\mathsf{nil} : \mathsf{Vec}_A(0).$$

**Definition 5.8.6.** Let $I$ be an index type and $X : I \to \mathcal{U}$ be an inductive type family. A *constructor* $\mathsf{c}$ with exactly one recursive argument is specified by three components:

- CONTEXT: A type $C$ for all non-recursive arguments.
- SOURCE INDEX: A function $h : C \to I$ that determines the index required for the recursive premise.
- TARGET INDEX: A function $g : C \to I$ that determines the index of the constructed element.

The signature of the constructor is:

$$\mathsf{c}\colon \prod_{z\colon C} X(h(z)) \to X(g(z)). \tag{5.2}$$

Given a motive $P\colon \prod_{i\colon I} X(i) \to \mathcal{U}$, the *eliminator method* $p_{\mathsf{c}}$ corresponding to this constructor must provide a proof for the new element:

$$p_{\mathsf{c}}\colon \prod_{z\colon C} \prod_{x\colon X(h(z))} P(h(z), x) \to P(g(z), \mathsf{c}(z, x))$$

**Example 5.8.7.** Applying this to the $\mathsf{cons}$ constructor of the vector family $\mathsf{Vec}_A\colon \mathbb{N} \to \mathcal{U}$, we have:

- Context: The non-recursive data is a length $n\colon \mathbb{N}$ and a coordinate $a\colon A$. Hence, we define $C :\equiv \mathbb{N} \times A$.
- Source Index: Since the recursive vector must have length $n$, we have $h(n, a) :\equiv n$.
- Target Index: The resulting vector has length $n + 1$. Thus, the target index function is $g(n, a) :\equiv \mathsf{succ}(n)$.

Plugging these into the signature of the generic constructor yields:

$$\mathsf{c}\colon \prod_{(n,a)\colon \mathbb{N}\times A} \mathsf{Vec}_A(n) \to \mathsf{Vec}_A(\mathsf{succ}(n))$$

By currying the dependent pair into separate arguments, we recover the standard signature:

$$\mathsf{cons}\colon \prod_{n\colon \mathbb{N}} \prod_{a\colon A} \mathsf{Vec}_A(n) \to \mathsf{Vec}_A(\mathsf{succ}(n)).$$

**Example 5.8.8.** Consider the inductive predicate defining even numbers, represented as a family $\mathsf{IsEven}\colon \mathbb{N} \to \mathcal{U}$. It has a step constructor that says "if $n$ is even, then $n + 2$ is even".

- Context: We only need the number $n$, so $C :\equiv \mathbb{N}$.
- Source Index: The premise is about $n$, so $h(n) :\equiv n$.
- Target Index: The conclusion is about $n + 2$, so $g(n) :\equiv \mathsf{succ}(\mathsf{succ}(n))$.

This directly produces the signature:

$$\mathsf{stepEven}\colon \prod_{n\colon \mathbb{N}} \mathsf{IsEven}(n) \to \mathsf{IsEven}(\mathsf{succ}(\mathsf{succ}(n))).$$

**Definition 5.8.9.** Let $I$ be an index type and $X\colon I \to \mathcal{U}$ an inductive type family. A general constructor $\mathsf{c}$ is specified by four components:

- CONTEXT: The type $C$ of all non-recursive arguments.
- BRANCHING TYPE: A family of types $W : C \to \mathcal{U}$ where, for any context element $z : C$, the type $W(z)$ indexes the recursive arguments.
- SOURCE INDEX: A function $h : \prod_{z:C} W(z) \to I$ that determines the index required for the recursive premise at branch $w$.
- TARGET INDEX: A function $g : C \to I$ that determines the index of the final constructed element.

The signature of the general constructor is:

$$\mathsf{c} : \prod_{z:C} \Big( \prod_{w:W(z)} X(h(z,w)) \Big) \to X(g(z)). \tag{5.3}$$

Let $P : \prod_{i:I} X(i) \to \mathcal{U}$ be a motive. The eliminator method $p_{\mathsf{c}}$ corresponding to this general constructor must provide a proof for the newly constructed element $\mathsf{c}(u, z)$.

To do this, the method receives three pieces of data:

- The non-recursive context data $z : C$.
- The bundle of recursive elements $x : \prod_{w:W(z)} X(h(z,w))$.
- The bundle of inductive hypotheses, which proves the property $P$ for every recursive element across all branches. This is a function that, for any branch $w : W(z)$, returns a proof of $P(h(z,w), x(w))$.

The signature of the general eliminator method is therefore:

$$p_{\mathsf{c}} : \prod_{z:C} \prod_{x:\prod_{w:W(z)} X(h(z,w))} \Big( \prod_{w:W(z)} P(h(z,w), x(w)) \Big) \to P(g(z), \mathsf{c}(z,x)).$$

**Remark 5.8.10.** When we evaluate the global eliminator $\mathsf{ind}_X$ on the constructed term $\mathsf{c}(z,x)$, the computation rule establishes that

$$\mathsf{ind}_X(P, \dots, g(z), \mathsf{c}(z,x)) \equiv p_{\mathsf{c}}\big(z, x, \lambda w.\, \mathsf{ind}_X\big(P, \dots, h(z,w), x(w)\big)\big).$$

## The Accessibility Predicate (Well-Foundedness)

In classical mathematics, a relation $R$ on a set $A$ is *well-founded* if every non-empty subset has an $R$-minimal element, or equivalently, if there are no infinite descending chains:

$$x_0 \leftarrow x_1 \leftarrow x_2 \leftarrow \cdots,$$

where $x \leftarrow y$ stands for $R(x,y)$.

Constructive type theory defines well-foundedness differently, using an inductive approach. An element is *accessible* if it is built up solely from other accessible elements.

**Definition 5.8.11.** Given a type $A$ and a relation $R\colon A \times A \to \mathcal{U}$, the *accessibility predicate* is an inductive type family $\mathsf{Acc}\colon A \to \mathcal{U}$ generated by a single constructor:

$$\mathsf{accIntro}\colon \prod_{x\colon A}\Big(\prod_{y\colon A} R(y,x) \to \mathsf{Acc}(y)\Big) \to \mathsf{Acc}(x)$$

This constructor embodies a single rule: an element $x$ is *accessible* if every strict predecessor of $x$ is already accessible.

**Remark 5.8.12.** The base cases are naturally captured by *minimal* elements.

We say that an element $x_0$ is *minimal* if, for every $y\colon A$, there is a witness $e_y\colon R(y,x_0) \to \mathsf{0}$. Consequently, the composition $\mathsf{rec_0}(\mathsf{Acc}(y)) \circ e_y$ is a function of type $R(y,x_0) \to \mathsf{Acc}(y)$. Applying $\mathsf{accIntro}(x_0)$ to this family of functions yields a proof of $\mathsf{Acc}(x_0)$ without requiring any actual predecessors.

## Well-Founded Recursion

The idea behind the accessibility predicate is to provide a structural foundation for well-founded recursion. A termination checker usually performs *structural* recursion: it calls a function on a syntactically smaller piece of an inductive datatype.

To write a recursive function where the argument decreases according to some arbitrary relation $R$, it suffices to prove that the relation is well-founded by showing that every element in $A$ is accessible:

$$\mathsf{isWellFounded}\colon \prod_{x\colon A} \mathsf{Acc}(x)$$

Constructing the function $\mathsf{isWellFounded}$ proves that recursive functions that decrease according to $R$ will always hit a base case and terminate.

This is perhaps the most important example of an indexed family requiring the full schema. To prove that a relation $R\colon A \times A \to \mathcal{U}$ is well-founded (allowing us to write terminating recursive functions over it), we define the inductive family $\mathsf{Acc}\colon A \to \mathcal{U}$.

The constructor says: an element $x$ is "accessible" if every $y$ strictly preceding $x$ according to $R$ is accessible.

- Context: We define $C :\equiv A$ and let $z \equiv x$, the element we want to prove accessible.
- Branching Type: We need one recursive premise for every $y$ that is smaller than $x$. The "shape" of the recursion depends entirely on the

context $x$. We define the branching type as the type of all elements related to $x$:

$$W(x) :\equiv \sum_{y:\, A} R(y, x)$$

- Source Index: For a given branch $w \equiv (y, r) : W(x)$, the recursive premise must be about $y$. So, $h(x, (y, r)) :\equiv y$.
- Target Index: Since the constructor proves that $x$ is accessible, we define $g(x) :\equiv x$.

Plugging these into the general schema:

$$\mathsf{accIntro} : \prod_{x:\, A} \Big( \prod_{\langle y,r\rangle :\, \sum_{y:\, A} R(y,x)} \mathsf{Acc}(y) \Big) \to \mathsf{Acc}(x)$$

Currying the inner product gives the standard signature:

$$\mathsf{accIntro} : \prod_{x:\, A} \Big( \prod_{y:\, A} R(y, x) \to \mathsf{Acc}(y) \Big) \to \mathsf{Acc}(x)$$

Here, the number of recursive arguments dynamically changes based on how many elements relate to the specific $x$ we provide.

**Note.** Suppose we have an algorithm that calculates integer division, producing a remainder $y$ from a number $x$. The compiler knows nothing about integer arithmetic, so to prove termination we cannot rely on the mathematical fact that $y$ is smaller than $x$. Instead, the strategy is to structurally traverse a proof tree $a$ of $\mathsf{Acc}(x)$.

When the algorithm computes $y$, it simultaneously computes a witness $r$ of $R(y, x)$. Since the proof $a$ is constructed by means of $\mathsf{accIntro}$, its argument is a function of type $\prod_{z:\, A} R(z, x) \to \mathsf{Acc}(z)$. By applying this function to $y$ and $r$, we extract a proof $a_y$ of $\mathsf{Acc}(y)$. Since $a_y$ was extracted directly from the structure of $a$, the compiler recognizes it as syntactically smaller, ensuring that successive recursive calls will terminate.

**Remark 5.8.13.** To derive the definition of a base constructor from the general constructor schema, we formalize the absence of recursive arguments by setting the branching type to the empty type.

Step 1. For a constructor with no recursive arguments, there are no recursive premises to evaluate for any context $z : C$. Therefore, we define the branching type as the empty type:

$$W(z) :\equiv \mathsf{0}.$$

Consequently, the source index function $h : \prod_{z:\, C} \mathsf{0} \to I$ is vacuously defined by the induction principle of the empty type.

STEP 2. We substitute $W(z) :\equiv \mathsf{0}$ into the general signature given in Equation (5.3):

$$\mathsf{c}\colon \prod_{z\colon C}\Big(\prod_{w\colon \mathsf{0}} X(h(z,w))\Big) \to X(g(z)).$$

The dependent product over the empty type is inhabited by the induction function $\mathsf{ind}_{\mathsf{0}}$, making any $\Pi$-type over $\mathsf{0}$ equivalent to $\mathsf{1}$. Thus, the signature simplifies to:

$$\mathsf{c}\colon \prod_{z\colon C} \mathsf{1} \to X(g(z)).$$

Since $(\mathsf{1} \to A) \simeq A$, we arrive at the exact signature for a base constructor:

$$\mathsf{c}\colon \prod_{z\colon C} X(g(z)).$$

STEP 3. Invoking the same reasons as above, the general eliminator method $p_{\mathsf{c}}$ reduces to:

$$p_{\mathsf{c}}\colon \prod_{z\colon C} P(g(z), \mathsf{c}(z)).$$

**Example 5.8.14.** The type $\mathsf{Fin}(n)$ represents the canonical type with $n$ elements.[1] Rather than being a primitive notion, it is formally defined as an indexed inductive type family over the natural numbers, $\mathsf{Fin}\colon \mathbb{N} \to \mathcal{U}$, generated by two constructors:

- *Finite zero:* $\mathsf{fz}\colon \prod_{k\colon \mathbb{N}} \mathsf{Fin}(\mathsf{succ}(k))$
- *Finite successor:* $\mathsf{fs}\colon \prod_{k\colon \mathbb{N}} \mathsf{Fin}(k) \to \mathsf{Fin}(\mathsf{succ}(k))$

The constructor $\mathsf{fz}$ (finite zero) provides the "first" element of any non-empty finite type, and $\mathsf{fs}$ (finite successor) injects the $k$ elements of $\mathsf{Fin}(k)$ into the "next" $k$ slots of $\mathsf{Fin}(\mathsf{succ}(k))$.

1. *The Base Constructor.* The constructor $\mathsf{fz}$ takes a natural number as a parameter, and has no recursive premises. Therefore, we can apply Definition 5.8.4 directly:

   - CONTEXT: The non-recursive argument is a natural number $k$. Thus, $C :\equiv \mathbb{N}$.
   - TARGET INDEX: The constructed element has type $\mathsf{Fin}(\mathsf{succ}(k))$, so $g(k) :\equiv \mathsf{succ}(k)$.

   Applying the base constructor signature from Equation (5.1) yields:

   $$\mathsf{fz}\colon \prod_{k\colon \mathbb{N}} \mathsf{Fin}(g(k)),$$

   which evaluates to:

   $$\mathsf{fz}\colon \prod_{k\colon \mathbb{N}} \mathsf{Fin}(\mathsf{succ}(k)).$$

[1] See also Exercise 1.15.6.

2. *The Recursive Constructor.* The constructor $\mathsf{fs}$ takes a natural number parameter, followed by one recursive argument from the $\mathsf{Fin}$ family.

   - CONTEXT: The non-recursive argument is a natural number $k$. Thus, $C :\equiv \mathbb{N}$.
   - BRANCHING TYPE: There is one recursive argument, so $W(k) :\equiv \mathsf{1}$.
   - SOURCE INDEX: The single recursive premise must belong to $\mathsf{Fin}(k)$. Thus, evaluated at the single branch $\star$, we have $h(k, \star) :\equiv k$.
   - TARGET INDEX: The constructed element belongs to $\mathsf{Fin}(\mathsf{succ}(k))$, so $g(k) :\equiv \mathsf{succ}(k)$.

   Applying the general schema yields:

$$\mathsf{fs}\colon \prod_{k\colon \mathbb{N}} \Big( \prod_{w\colon \mathsf{1}} \mathsf{Fin}(k) \Big) \to \mathsf{Fin}(\mathsf{succ}(k)),$$

   which reduces to:

$$\mathsf{fs}\colon \prod_{k\colon \mathbb{N}} \mathsf{Fin}(k) \to \mathsf{Fin}(\mathsf{succ}(k)).$$

## 5.9 Mutual Inductive Types

Suppose we want to define two types, $\mathsf{Even}$ and $\mathsf{Odd}$, representing the even and odd natural numbers. We would like to think of them as simultaneously defined by three constructors:

- $\mathsf{zeroEven}\colon \mathsf{Even}$
- $\mathsf{succEven}\colon \mathsf{Odd} \to \mathsf{Even}$
- $\mathsf{succOdd}\colon \mathsf{Even} \to \mathsf{Odd}$

However, that would require introducing a new primitive notion of mutual induction. A simpler way to achieve the same result is to define an index type $I$ with two tags and use it to obtain an inductive type family.

The index type $I$ can be introduced as the inductive type generated by:

- $I :\equiv 2$,
- $\mathsf{even} :\equiv 0_2$,
- $\mathsf{odd} :\equiv 1_2$.

We then define a single inductive type family $X\colon I \to \mathcal{U}$, where $X(\mathsf{even})$ represents $\mathsf{Even}$, and $X(\mathsf{odd})$ represents $\mathsf{Odd}$.

1. *The Base Constructor.* This constructor defines the base case for even numbers. It has no recursive arguments, so we apply Definition 5.8.4:

   - CONTEXT: $C :\equiv \mathsf{1}$.

- TARGET INDEX: The function $g\colon 1 \to I$ is given by $g(\star) :\equiv \mathsf{even}$.

Applying the base constructor signature, we obtain:

$$\mathsf{c}_1\colon \prod_{z\colon 1} X(\mathsf{even}),$$

which is equivalent to $X(\mathsf{even})$.

2. *The Recursive Constructor for Even.* This constructor builds an even number from a predecessor, which must be odd. We apply the general constructor schema:

   - CONTEXT: $C :\equiv 1$.
   - BRANCHING TYPE: There is one recursive argument, so $W(\star) :\equiv 1$.
   - SOURCE INDEX: The single recursive premise comes from the odd index. Thus, $h(\star, \star) :\equiv \mathsf{odd}$.
   - TARGET INDEX: The resulting element is even, so $g(\star) :\equiv \mathsf{even}$.

   Plugging these into the general schema yields:

$$\mathsf{c}_2\colon \prod_{z\colon 1}\Big(\prod_{w\colon 1} X(\mathsf{odd})\Big) \to X(\mathsf{even}),$$

   which reduces to:

$$\mathsf{c}_2\colon X(\mathsf{odd}) \to X(\mathsf{even}).$$

3. *The Recursive Constructor for Odd.* Analogously, this constructor is derived as:

$$\mathsf{c}_3\colon X(\mathsf{even}) \to X(\mathsf{odd}).$$

## The General Approach

Encoding mutual induction is fully captured by the general definition of an inductive type family.

To define a collection of mutually inductive types we generalize the Even/Odd approach in the following way:

1. *The Index Type:* Instead of a two-element type, we define an index type $I$ that contains a unique tag for every type we wish to define in our mutual block. For instance, if we are defining $n$ mutual types, we define an index type with $n$ base constructors, which we can label $I :\equiv \mathsf{Fin}(n)$.

2. *The Unified Family:* We define a single inductive type family $X\colon I \to \mathcal{U}$. For each $i\colon I$, the application $X(i)$ represents the $i$th type in our mutual collection.

3. *The Constructors:* In the formal family $X$, a constructor targeting the $i$th type is simply a constructor whose target index function is the constant function $g(z) :\equiv i$.

4. *The Cross-References:* The mutual dependencies are entirely handled by the source index function $h\colon \prod_{z\colon C} W(z) \to I$. Whenever a constructor requires a recursive premise from the $j$th type in the collection, we simply set $h(z, w) :\equiv j$ for that specific branch $w$.

Because the general constructor schema

$$\mathsf{c}\colon \prod_{z\colon C}\Big(\prod_{w\colon W(z)} X(h(z,w))\Big) \to X(g(z))$$

allows the source index $h$ to output any element of $I$ independently for each branch $w$, a single constructor can require premises from arbitrarily many different types in the mutual collection. Thus, mutual inductive types are a direct consequence of indexed inductive families.

## 5.10 Inductive Type Families & W-Types

Inductive type families generalize W-types in such a way that W-types can be viewed as their unindexed variants.

To show that a W-type $\mathsf{W}_A B$ is exactly an inductive type family indexed by the unit type, we set $I :\equiv \mathsf{1}$ and define our family $X\colon \mathsf{1} \to \mathcal{U}$ that consists of a single type $X(\star)$.

a) *Components.* A W-type is specified by a type of shapes $A\colon \mathcal{U}$ and an arity type family $B\colon A \to \mathcal{U}$. We map these directly to the four components of a general constructor:

   - Context: The non-recursive arguments determine the "shape" of the node. Thus, $C :\equiv A$. Let $x\colon A$.
   - Branching Type: The recursive premises for $x$ correspond to the "positions" where subtrees can be attached. Thus, $W(x) :\equiv B(x)$.
   - Source Index: Since there is only the $\star$ index, $h(x, w) :\equiv \star$.
   - Target Index: $g(x) :\equiv \star$.

b) *Constructor.* The signature of the general inductive family constructor is:

$$\mathsf{c}\colon \prod_{z\colon C}\Big(\prod_{w\colon W(z)} X(h(z,w))\Big) \to X(g(z)),$$

which becomes

$$\begin{aligned}\mathsf{c}\colon \prod_{x\colon A}\Big(\prod_{w\colon B(x)} X(\star)\Big) &\to X(\star)\\ &\equiv \prod_{x\colon A}(B(x)\to X(\star))\to X(\star).\end{aligned}$$

If we set $\mathsf{W} :\equiv X(\star)$, the constructor signature becomes:

$$\prod_{x\colon A}(B(x)\to \mathsf{W})\to \mathsf{W},$$

which reproduces the canonical introduction rule (5.2) for $\mathsf{W}_A B$.

## 5.11 Inductive-Inductive Definitions

In standard mutual induction, we define two (or more) types simultaneously. However, while their constructors depend on one another, their existence does not, and so their formation rules are independent.

In an *inductive-inductive* definition, we define a type $A$ and a type family $B$ that depends on $A$. The formation rules of these entities look like this:

$$\begin{aligned}A&\colon \mathcal{U}\\ B&\colon A\to\mathcal{U}\end{aligned}$$

The issue is that we cannot even write down the type signature for $B$ without having defined $A$ first as a well-formed type. In addition, because they are defined together, the constructors that build elements of $A$ might need to use elements of $B$.

**Example 5.11.1.** [Contexts & Types] As we saw in 8. ASSUMPTIONS, whenever we write a mathematical statement, we are always working under some set of assumptions or available variables.

The inductive-inductive definition of $\mathsf{Con}$ and $\mathsf{Ty}$ is a way to formalize this process inside type theory itself.

A *context*, modeled by the type $\mathsf{Con}$, represents the space of assumptions. It can be regarded as a list of variable declarations. An example would be:

$$\Gamma :\equiv [x\colon \mathsf{Nat},\ y\colon \mathsf{Vector}(x)]$$

We build contexts one step at a time. We start with the empty context, represented by the $\mathsf{nil}$ constructor, which has no variables. Then, we extend it by adding one variable at a time, represented by the $\mathsf{ext}$ constructor.

The family $\mathsf{Ty}(\Gamma)$ models all the valid type expressions that make sense under the specific assumptions of the context $\Gamma$. The double dependency arises because:

- To add a new variable to a context, we must specify its type, which must already be known to be valid in the current context. Thus, building an element of $\mathsf{Con}$ requires an element of $\mathsf{Ty}(\Gamma)$.
- To know if a dependent type is valid, we need to check those variables against our current environment. Thus, building an element of $\mathsf{Ty}(\Gamma)$ requires a context $\Gamma : \mathsf{Con}$.

To model this, we need an inductive-inductive definition. The idea works as follows:

a) First, we define the base case for contexts. The empty context requires no prerequisites and does not depend on any types:

$$\mathsf{nil} : \mathsf{Con}.$$

b) Now, the intertwining begins. To extend a context with a new variable, we need a valid type to assign to that variable. However, because of the formation rules, a type is only valid with respect to an existing context. So, the context-extension constructor requires both a prior context $\Gamma$ and a type valid in $\Gamma$:

$$\mathsf{ext} : \prod_{\Gamma : \mathsf{Con}} \mathsf{Ty}(\Gamma) \to \mathsf{Con}$$

Notice how a constructor for the base type $\mathsf{Con}$ consumes an element of the dependent family $\mathsf{Ty}$.

Conversely, the constructors for $\mathsf{Ty}$ must take elements of $\mathsf{Con}$ as arguments, because every element of $\mathsf{Ty}$ must be indexed by a context.

The circularity is saved by the existence of the *primitive types*. These are precisely those types that form the base cases of the recursive structure. They require only a context and no prior types or variables to be well-formed.

Primitive types are usually restricted to $\mathsf{0}$, $\mathsf{1}$, $\mathsf{2}$, $\mathbb{N}$, and the universe $\mathcal{U}$. More precisely, we have

$$\mathsf{empty}, \mathsf{unit}, \mathsf{bool}, \mathsf{nat}, \mathsf{univ} : \prod_{\Gamma : \mathsf{Con}} \mathsf{Ty}(\Gamma),$$

which establishes that these types are valid in any context.

In standard type theory, this is often written as an inference rule stating that if $\Gamma$ is a valid context, then $\mathbb{N}$ is a valid type under $\Gamma$:

$$\frac{\vdash \Gamma \ \mathrm{ctx}}{\Gamma \vdash \mathbb{N} \ \mathrm{type}}$$

## The General Schema

In general, an inductive-inductive definition consists of the simultaneous specification of a type $A$ and a type family $B$ indexed by $A$:

$$A : \mathcal{U}$$
$$B : A \to \mathcal{U}$$

along with a set of constructors for $A$ and a set of constructors for $B$.

The constructors for $A$ specify how to build elements of $A$, and they may take arguments of type $A$ as well as arguments of type $B(a)$ for previously constructed elements $a : A$.

The constructors for $B$ specify how to build elements of $B(a)$ for a given $a : A$. These constructors may depend on elements of $A$ and elements of $B(a')$ for previously constructed elements $a' : A$.

The elimination principle for such a definition must be mutually defined: we simultaneously specify a motive for $A$ and a motive for $B$, and provide methods for all constructors of both $A$ and $B$.

To understand the formal structure, let us look at a canonical simplified schema consisting of one prototypical constructor for $A$ and one for $B$:

**Formation Rules:**

$$A : \mathcal{U} \quad \text{and} \quad B : A \to \mathcal{U}.$$

**Constructors:**

$$\mathsf{c}_A : \prod_{x : A} B(x) \to A,$$
$$\mathsf{c}_B : \prod_{x : A} \prod_{y : B(x)} B(c_A(x, y)).$$

**Induction Principle:** We must specify two motives:

The motive for $A$ is standard:

$$P_A : A \to \mathcal{U}.$$

However, the motive for $B$ is specialized:

$$P_B : \prod_{a : A} \prod_{b : B(a)} P_A(a) \to \mathcal{U}.$$

Given these motives, the elimination principle states that if we provide a method for every constructor of $A$ and every constructor of $B$, we obtain a mutually

defined pair of functions:

$$f_A : \prod_{a:\, A} P_A(a)$$
$$f_B : \prod_{a:\, A} \prod_{b:\, B(a)} P_B(a, b, f_A(a))$$

To write the computation rules explicitly, we need to define two additional eliminator methods:

$$m_A : \prod_{x:\, A} \prod_{y:\, B(x)} \prod_{p_x:\, P_A(x)} \prod_{p_y:\, P_B(x,y,p_x)} P_A(\mathsf{c}_A(x, y))$$

and

$$m_B : \prod_{x:\, A} \prod_{y:\, B(x)} \prod_{p_x:\, P_A(x)} \prod_{p_y:\, P_B(x,y,p_x)} P_B\big(\mathsf{c}_A(x, y), \mathsf{c}_B(x, y), m_A(x, y, p_x, p_y)\big)$$

**Computation Rules:**

$$f_A(\mathsf{c}_A(x, y)) \equiv m_A\big(x, y, f_A(x), f_B(x, y)\big)$$
$$f_B(\mathsf{c}_A(x, y), \mathsf{c}_B(x, y)) \equiv m_B\big(x, y, f_A(x), f_B(x, y)\big).$$

Notice how the computation of $f_B$ on the constructor $\mathsf{c}_B$ is synchronized with the computation of $f_A$ on the constructor $\mathsf{c}_A$.

## 5.12 Identity Types and Identity Systems

### Path Induction

Let $A$ be a type. The identity type family

$$- =_A - : A \to A \to \mathcal{U}$$

can be seen as an inductive family with two indices. Following Definition 5.8.4 for a constructor with no recursive arguments, we map the components as follows:

- Index Type: Uncurrying gives $I :\equiv A \times A$.
- Family: The type family $X : A \times A \to \mathcal{U}$ is defined by $X(a, b) :\equiv a =_A b$.
- Context: The constructor $\mathsf{refl}$ requires one non-recursive argument in $A$. Thus, $C :\equiv A$.
- Target Index: The constructor $\mathsf{refl}$ yields an element of $a =_A a$; hence, the function $g : C \to I$ is given by $g(a) :\equiv (a, a)$.

By the general signature (5.1), the constructor is:

$$\mathsf{refl}\colon \prod_{z\colon C} X(g(z)) \equiv \prod_{a\colon A} a =_A a.$$

To derive the induction principle, consider a motive of type:

$$\prod_{(a,b)\colon A\times A} X(a,b) \to \mathcal{U}.$$

By currying, this is equivalent to

$$P\colon \prod_{a,b\colon A} (a =_A b) \to \mathcal{U}.$$

According to Definition 5.8.4, the eliminator method for our constructor has the signature

$$p_{\mathsf{refl}}\colon \prod_{a\colon A} P(a,a,\mathsf{refl}_a).$$

The general eliminator $\mathsf{ind}_X$ produces the dependent function:

$$f\colon \prod_{a,b\colon A} \prod_{p\colon a=_A b} P(a,b,p),$$

which satisfies the computation rule

$$f(a,a,\mathsf{refl}_a) \equiv p_{\mathsf{refl}}(a).$$

**Based Path Induction**

Let us now fix a parameter $a_0\colon A$. We can define the based identity type family

$$a_0 =_A -\colon A \to \mathcal{U}$$

as an inductive family with one index. Following Definition 5.8.4 again, we have:

- Index Type: $I :\equiv A$.
- Family: The type family $X\colon A \to \mathcal{U}$ is defined by $X(b) :\equiv a_0 =_A b$.
- Context: Since the constructor has no arguments, the context is $\mathsf{1}$.
- Target Index: The constructor yields an element of $a_0 =_A a_0$, so the target index function $g\colon \mathsf{1} \to I$ is given by $g(\star) :\equiv a_0$.

By the general signature (5.1), the constructor is:

$$\mathsf{refl}_{a_0}\colon \prod_{z\colon \mathsf{1}} X(g(z)) \simeq a_0 =_A a_0.$$

To derive the based path induction principle, consider a motive of type:

$$P\colon \prod_{b\colon A}(a_0 =_A b) \to \mathcal{U}.$$

According to Definition 5.8.4, the eliminator method for our constructor has the signature

$$p_{\mathsf{refl}}\colon \prod_{z\colon \mathbf{1}} P(g(z), \mathsf{refl}_{a_0}) \equiv P(a_0, \mathsf{refl}_{a_0}).$$

Thus, the method reduces to providing an element $d\colon P(a_0, \mathsf{refl}_{a_0})$.

The general eliminator $\mathsf{ind}_X$ produces the dependent function:

$$f\colon \prod_{b\colon A}\prod_{p\colon a_0 =_A b} P(b,p),$$

which satisfies the computation rule

$$f(a_0, \mathsf{refl}_{a_0}) \equiv d.$$

**Note.** For decades, long before the introduction of HoTT, type theorists believed that $\mathsf{refl}$ could only construct trivial paths. This intuition originated in the behavior of standard inductive types, where, for instance, every natural number is zero or a successor, every boolean term is $0_2$ or $1_2$, etc. However, in 1993, Martin Hofmann and Thomas Streicher constructed a mathematical model in which types are interpreted as groupoids and identity types as the spaces of morphisms between objects. In their model, non-trivial automorphisms existed within the strict syntactic rules of Martin-Löf Type Theory. This showed that the type theory was already describing homotopy spaces.

The example given in Exercise 1.16.2, which shows that $\mathsf{not}\colon 2 \to 2$ is an involution, allows us to conclude via the univalence axiom that the identity type $2 =_{\mathcal{U}} 2$ is not trivial. Specifically, it contains at least two distinct paths: the constant path $\mathsf{refl}_2$ and the path $\mathsf{ua}(\mathsf{not})$ induced by negation.

More generally, while the inductive definition of the identity family provides the architectural scaffolding, the structural richness of identity paths is inextricably tied to univalence. Without this axiom, the identity type acts merely as a formal equality symbol; with it, the type acquires the full expressiveness of symmetries and transformations.

**Remark 5.12.1.** The based path induction principle is the type-theoretic incarnation of the Yoneda lemma B.2.1 from category theory.

As described in § 2.4.2, types can be viewed as categories and identity paths as morphisms between them. Under this analogy, a type family $C\colon A \to \mathcal{U}$ acts as a functor from this category to the universe of spaces. For this correspondence to hold precisely, it is crucial to let $\mathcal{U}$ represent the category whose objects are types (as usual) and whose morphisms are non-dependent functions (rather than identity paths).

Under this interpretation, every type family $P\colon A \to \mathcal{U}$ is a functor. Specifically, for every $x, y\colon A$ and $p\colon x =_A y$, its action on morphisms is given by

$$P(p) :\equiv \mathsf{transport}^P(p, -)\colon P(x) \to P(y).$$

The functoriality of $P$ follows from Theorem 1.11.6 and Lemma 2.3.10. Although path concatenation is written left-to-right, $P$ is covariant.

Consider two type families $R, S\colon A \to \mathcal{U}$. Given a dependent function

$$g\colon \prod_{b\colon A} R(b) \to S(b),$$

for any path $p\colon x =_A y$, we obtain two maps of type $R(x) \to S(y)$:

$$g(y, \mathsf{transport}^R(p, -)) \quad \text{and} \quad \mathsf{transport}^S(p, g(x, -)).$$

By Lemma 2.3.15, for $r\colon R(x)$, these maps satisfy the propositional equality

$$g(y, \mathsf{transport}^R(p, r)) =_{S(y)} \mathsf{transport}^S(p, g(x, r)).$$

This yields the following propositionally commutative diagram:

$$\begin{array}{ccc} R(x) & \xrightarrow{\mathsf{transport}^R(p,-)} & R(y) \\ {\scriptstyle g(x)}\downarrow & & \downarrow{\scriptstyle g(y)} \\ S(x) & \xrightarrow[\mathsf{transport}^S(p,-)]{} & S(y) \end{array}$$

Thus, every dependent function $g\colon \prod_{b\colon A} R(b) \to S(b)$ can be seen as a natural transformation from $R$ to $S$.

In particular, a natural transformation from the representable functor $a_0 =_A -$ to a type family $C$ corresponds to a homotopy $\eta\colon (a_0 =_A -) \sim C$, which is judgmentally equal to a dependent function

$$\eta\colon \prod_{b\colon A} (a_0 =_A b) \to C(b).$$

The following table summarizes the analogy:

| **Category Theory** | **Type Theory** |
|---|---|
| Categories $\mathbf{A}$, $\mathbf{Set}$ | Types $A$, $\mathcal{U}$ |
| Objects $a, b$ | Points $a, b\colon A$ |
| Morphisms $f\colon a \to b$ | Paths $p\colon a =_A b$, or maps $g\colon A \to B$ in $\mathcal{U}$ |
| Functor $C\colon \mathbf{A} \to \mathbf{Set}$ | Type family $C\colon A \to \mathcal{U}$ |
| Representable functor $\mathrm{Hom}(a_0, -)$ | Type family $a_0 =_A -$ |
| Natural transformation $\eta\colon \mathrm{Hom}(a_0, -) \to C$ | Homotopy $\eta\colon (a_0 =_A -) \sim C$ |
| $\mathsf{id}_{a_0} \in \mathrm{Hom}(a_0, a_0)$ | $\mathsf{refl}_{a_0}\colon a_0 =_A a_0$ |
| $\eta_{a_0}(\mathsf{id}_{a_0}) \in C(a_0)$ | $\eta(a_0, \mathsf{refl}_{a_0})\colon C(a_0)$ |
| Yoneda bijection | Based path induction |

**Definitions 5.12.2.** Let $A$ be a type and $a_0 \colon A$ an element.

- A *pointed predicate* over $\langle A, a_0 \rangle \colon \mathcal{U}_\bullet$ is a family $R \colon A \to \mathcal{U}$ equipped with a *basepoint* $r_0 \colon R(a_0)$.
- Given pointed predicates $\langle R, r_0 \rangle$ and $\langle S, s_0 \rangle$, a family of maps

$$g \colon \prod_{b \colon A} R(b) \to S(b)$$

  is said to be *pointed* if $g(a_0, r_0) =_{S(a_0)} s_0$. The type of such pointed maps is defined as

$$\mathsf{ppmap}(R, S) :\equiv \sum_{g \colon \prod_{b \colon A} R(b) \to S(b)} g(a_0, r_0) =_{S(a_0)} s_0.$$

- An *identity system at* $a_0$ is a pointed predicate $\langle R, r_0 \rangle$ such that for any type family

$$D \colon \prod_{b \colon A} R(b) \to \mathcal{U}$$

  and any element $d \colon D(a_0, r_0)$, there exists a dependent function

$$f \colon \prod_{b \colon A} \prod_{r \colon R(b)} D(b, r)$$

  satisfying $f(a_0, r_0) =_{D(a_0, r_0)} d$.

**Remarks 5.12.3.**

- The canonical example of a pointed predicate over $\langle A, a_0 \rangle$ is the identity family $(a_0 =_A -)$ equipped with the basepoint $\mathsf{refl}_{a_0}$.

  An arbitrary pointed predicate $\langle R, r_0 \rangle$ is an identity system if it satisfies the exact same elimination principle (based path induction) as the identity type. In this sense, identity systems generalize the role representable functors play in the Yoneda lemma.
- An alternative way to interpret the type $\mathsf{ppmap}(R, S)$ is by recognizing it as the fiber of the evaluation map at $s_0$, i.e.,

$$\mathsf{ppmap}(R, S) \equiv \mathsf{fib}_{\mathrm{ev}_{(a_0, r_0)}}(s_0). \tag{5.1}$$

**Theorem 5.12.4.** *Let $A$ be a type and $a_0 \colon A$. If $\langle R, r_0 \rangle$ is an identity system at $a_0$, then for all $b \colon A$, there is an equivalence $R(b) \simeq (a_0 =_A b)$.*

*Proof.* Because $\langle a_0 =_A -, \mathsf{refl}_{a_0} \rangle$ is an identity system, we can define a dependent function

$$f \colon \prod_{b \colon A} (a_0 =_A b) \to R(b)$$

by taking $R$ as our motive and $r_0 \colon R(a_0)$ as the base case. The computation rule for based path induction yields the definitional equality $f(a_0, \mathsf{refl}_{a_0}) \equiv r_0$.

Conversely, because $\langle R, r_0 \rangle$ is an identity system, we can define a dependent function

$$g \colon \prod_{b \colon A} R(b) \to (a_0 =_A b)$$

by taking $a_0 =_A -$ as our motive and $\mathsf{refl}_{a_0} \colon a_0 =_A a_0$ as the base case. By the definition of an identity system, we obtain a propositional equality

$$p \colon g(a_0, r_0) =_{a_0 =_A a_0} \mathsf{refl}_{a_0}.$$

To show that $f$ and $g$ are quasi-inverses, we must construct the homotopies

$$g(b, f(b, -)) \sim \mathsf{id}_{a_0 =_A b} \quad \text{and} \quad f(b, g(b, -)) \sim \mathsf{id}_{R(b)}.$$

For the first homotopy, we proceed by based path induction on $q \colon a_0 =_A b$. It suffices to consider the case $b \equiv a_0$ and $q \equiv \mathsf{refl}_{a_0}$. This requires a path

$$g(a_0, f(a_0, \mathsf{refl}_{a_0})) =_{a_0 =_A a_0} \mathsf{refl}_{a_0}.$$

Since $f(a_0, \mathsf{refl}_{a_0}) \equiv r_0$, the goal reduces to $g(a_0, r_0) =_{a_0 =_A a_0} \mathsf{refl}_{a_0}$, which is exactly the path $p$.

For the second homotopy, we proceed by the induction principle of the identity system $R$ on $r \colon R(b)$. It suffices to consider the case $b \equiv a_0$ and $r \equiv r_0$. This requires a path

$$f(a_0, g(a_0, r_0)) =_{R(a_0)} r_0.$$

Applying the action on paths of $f(a_0, -)$ to $p$, we obtain a path

$$f(a_0, g(a_0, r_0)) =_{R(a_0)} f(a_0, \mathsf{refl}_{a_0}).$$

Since $f(a_0, \mathsf{refl}_{a_0}) \equiv r_0$, the required path is reached.

Therefore, $f$ and $g$ are quasi-inverses, completing the proof.

□

**Remarks 5.12.5.**

- The forward map $f \colon \prod_{b \colon A}(a_0 =_A b) \to R(b)$ constructed via based path induction in the theorem is definitionally equal to the transport operation applied to the base point $r_0$. In other words

  $$f(b, p) \equiv \mathsf{transport}^R(p, r_0).$$

  In this sense, an identity system $\langle R, r_0 \rangle$ can be viewed as a space where transporting the root $r_0$ along paths completely and uniquely traces out the entirety of the family $R$. To geometrically visualize this, imagine $R$ as

a fibration over $A$, with a point $r_0$ in the fiber over $a_0$. As $r_0$ is dragged along all possible paths in $A$ with origin at $a_0$, the point $r_0$ transports onto a subspace of $R$. The theorem establishes that this transporting action reaches the entirety of $R$, without ever overlapping itself.

- The Yoneda lemma establishes a bijection between natural transformations $\eta\colon \mathrm{Hom}(a_0, -) \to F$ and elements of $F(a_0)$, given by evaluating the component $\eta_{a_0}$ at the identity morphism $\mathsf{id}_{a_0}$. In the proof above, $r_0\colon R(a_0)$ is precisely the selection for this image of the identity morphism. Just as in the Yoneda lemma, this choice of $r_0$ uniquely determines the forward transformation, which is exactly how we defined the dependent function $f$ via based path induction.

**Theorem 5.12.6.** *A pointed predicate $\langle R, r_0\rangle$ over $\langle A, a_0\rangle$ is an identity system if, and only if, for any pointed predicate $\langle S, s_0\rangle$, the type $\mathsf{ppmap}(R, S)$ is contractible.*

*Proof.* Suppose that $\langle R, r_0\rangle$ is an identity system. The induction principle of $\langle R, r_0\rangle$ applied to the motive $D(b, r) :\equiv S(b)$ and the base element $s_0\colon D(a_0, r_0)$ yields a function

$$f\colon \prod_{b\colon A} \prod_{r\colon R(b)} S(b)$$

and a path $p\colon f(a_0, r_0) =_{S(a_0)} s_0$. Thus, the dependent pair $\langle f, p\rangle$ is an element of $\mathsf{ppmap}(R, S)$. We claim that this pair is a center of contraction.

Given $\langle g, q\rangle\colon \mathsf{ppmap}(R, S)$, we must construct a path

$$\langle f, p\rangle =_{\mathsf{ppmap}(R,S)} \langle g, q\rangle$$

consisting of two parts: a path $\epsilon\colon f = g$, and a path $\zeta\colon \mathsf{transport}^E(\epsilon, p) = q$, where $E(h) :\equiv h(a_0, r_0) =_{S(a_0)} s_0$.

To construct $\epsilon$, it suffices to provide a homotopy

$$\eta\colon \prod_{b\colon A} \prod_{r\colon R(b)} f(b, r) =_{S(b)} g(b, r)$$

and define $\epsilon :\equiv \mathsf{funext}(\lambda b.\, \mathsf{funext}(\eta(b)))$ by applying function extensionality iteratively.

To construct $\eta$, we again use the induction principle of $\langle R, r_0\rangle$. We define the motive as $M(b, r) :\equiv f(b, r) =_{S(b)} g(b, r)$ and use the path $p \cdot q^{-1}\colon M(a_0, r_0)$ for the base case.

By the computation rule, $\eta$ yields a path

$$\beta\colon \eta(a_0, r_0) = p \cdot q^{-1}.$$

To construct $\zeta$, observing that $E \equiv (\lambda u.\, u = s_0) \circ \mathrm{ev}_{(a_0,r_0)}$, we obtain

$$\begin{aligned}
\mathsf{transport}^E(\epsilon, p) &= \mathsf{transport}^{u \mapsto u = s_0}(\mathsf{ap}_{\mathrm{ev}_{(a_0,r_0)}}(\epsilon), p) && ;\ \text{Lem. } 2.3.13 \\
&= \mathsf{ap}_{\mathrm{ev}_{(a_0,r_0)}}(\epsilon)^{-1} \cdot p && ;\ \text{Prop. } 2.11.6 \\
&= \mathsf{happly}(\epsilon, (a_0, r_0))^{-1} \cdot p && ;\ \text{Eq. } 2.5 \\
&\equiv \mathsf{happly}(\mathsf{funext}(\eta), (a_0, r_0))^{-1} \cdot p \\
&= \eta(a_0, r_0)^{-1} \cdot p \\
&= (p \cdot q^{-1})^{-1} \cdot p && ;\ \beta \\
&= q.
\end{aligned}$$

This completes the construction of $\zeta \colon \mathsf{transport}^E(\epsilon, p) = q$.

Conversely, suppose that $\mathsf{ppmap}(R, S)$ is contractible for any pointed predicate $\langle S, s_0 \rangle$. We must show that $\langle R, r_0 \rangle$ is an identity system. Let $D$ be an arbitrary type family over $R$, and let $d_0 \colon D(a_0, r_0)$ be a base point. The pointed predicate $\langle D, d_0 \rangle$ fulfills our hypothesis; thus, according to equation (5.1), the type

$$\mathsf{ppmap}(R, D) \equiv \mathsf{fib}_{\mathrm{ev}_{(a_0,r_0)}}(d_0)$$

is contractible. Since this holds for all base points $d_0$, Remark 4.2.4 ensures that the evaluation map $\mathrm{ev}_{(a_0,r_0)}$ is an equivalence. Therefore, $\langle R, r_0 \rangle$ is an identity system.

□

**Theorem 5.12.7.** *Let $\langle R, r_0 \rangle$ be a pointed predicate over $\langle A, a_0 \rangle$. Then, the following conditions are logically equivalent:*

a) *The pointed predicate $\langle R, r_0 \rangle$ is an identity system at $a_0$.*

b) *For any $b \colon A$, the function $\mathsf{transport}^R(-, r_0) \colon (a_0 =_A b) \to R(b)$ is an equivalence.*

c) *The type $\Sigma_A R$ is contractible.*

d) *For any pointed predicate $\langle S, s_0 \rangle$, the type $\mathsf{ppmap}(R, S)$ is contractible.*

*Proof.*

a) $\Rightarrow$ b) This is a direct consequence of Theorem 5.12.4 and Remark 5.12.5.

b) $\Rightarrow$ c) The equivalence $(a_0 =_A b) \simeq R(b)$ induces an equivalence

$$\Big(\sum_{b \colon A} a_0 =_A b\Big) \simeq \Big(\sum_{b \colon A} R(b)\Big).$$

Hence, the conclusion follows from Lemma 3.4.11.

c) $\Rightarrow$ a) Consider the dependent pair $\langle a_0, r_0\rangle$ in the total space $\Sigma_A R$. Given a type family

$$D\colon \prod_{b\colon A} R(b) \to \mathcal{U}$$

and a term $d\colon D(a_0, r_0)$, we must construct a function

$$f\colon \prod_{b\colon A} \prod_{r\colon R(b)} D(b, r)$$

satisfying $f(a_0, r_0) =_{D(a_0,r_0)} d$.

First, observe that uncurrying produces the family $\bar{D}\colon \Sigma_A R \to \mathcal{U}$, defined by $\bar{D}(\langle b, r\rangle) :\equiv D(b, r)$.

Given $b\colon A$ and $r\colon R(b)$, we obtain the dependent pair $\langle b, r\rangle\colon \Sigma_A R$. By contractibility, there is a path $\omega\colon \langle a_0, r_0\rangle =_{\Sigma_A R} \langle b, r\rangle$. Thus, we can define

$$f(b, r) :\equiv \mathsf{transport}^{\bar{D}}(\omega, d)\colon D(b, r).$$

This map satisfies the computation rule because $\omega$ reduces definitionally to $\mathsf{refl}_{\langle a_0,r_0\rangle}$, making the transport definitionally equal to the identity function.

a) $\Leftrightarrow$ d) This equivalence is established in Theorem 5.12.6.

□

**Definitions 5.12.8.** Let $A$ be a type.

- A *reflexive relation* over $A$ is a binary type family $R\colon A \to A \to \mathcal{U}$ equipped with a *reflexivity function* $r_0\colon \prod_{a\colon A} R(a, a)$.
- Given two reflexive relations $\langle R, r_0\rangle$ and $\langle S, s_0\rangle$ over $A$, a *morphism of reflexive relations* is a family of maps

$$g\colon \prod_{a,b\colon A} R(a, b) \to S(a, b)$$

that propositionally preserves the reflexivity elements, meaning there is a family of paths

$$p\colon \prod_{a\colon A} g(a, a, r_0(a)) =_{S(a,a)} s_0(a).$$

- The type of such morphisms is denoted by

$$\mathsf{rrmap}(R, S) :\equiv \sum_{g\colon \prod_{a,b\colon A} R(a,b)\to S(a,b)} \prod_{a\colon A} g(a, a, r_0(a)) =_{S(a,a)} s_0(a).$$

- An *identity system over* $A$ is a reflexive relation $\langle R, r_0 \rangle$ such that for any type family

$$D\colon \prod_{a,b\colon A} R(a,b) \to \mathcal{U}$$

and any

$$d\colon \prod_{a\colon A} D(a,a,r_0(a)),$$

there exists a function

$$f\colon \prod_{a,b\colon A} \prod_{r\colon R(a,b)} D(a,b,r)$$

satisfying $f(a,a,r_0(a)) =_{D(a,a,r_0(a))} d(a)$ for all $a\colon A$.

**Remark 5.12.9.** Identity systems generalize the concept of based path induction to a global relation over the entire type $A$. Instead of considering the paths departing from a specific base point, the binary family $R\colon A \to A \to \mathcal{U}$ characterizes paths between any two arbitrary points.

As the following theorem shows, this definition axiomatizes equality geometrically. If a binary relation $R$ and a reflexivity term $r_0$ satisfy this induction principle, then the space $R(a,b)$ is equivalent to the built-in identity type $(a =_A b)$.

To formally establish that "isomorphic structures are equal," instead of constructing (potentially complex) paths in the universe, it suffices to define $R(a,b)$ to mean that $a$ and $b$ are isomorphic, and then prove that $\langle R, r_0 \rangle$ forms a global identity system. The induction principle then promotes those isomorphisms to propositional equalities.

**Theorem 5.12.10.** *Let $R\colon A \to A \to \mathcal{U}$ be a type family and $r_0\colon \prod_{a\colon A} R(a,a)$. Then, the following conditions are logically equivalent:*

a) *The pair $\langle R, r_0 \rangle$ is an identity system over $A$.*

b) *For all $a\colon A$, the pointed predicate $\langle R(a), r_0(a) \rangle$ is an identity system at $a$.*

c) *For any $a\colon A$, the type $\sum_{b\colon A} R(a,b)$ is contractible.*

d) *For any $a, b\colon A$, the map $\mathsf{transport}^{R(a)}(-, r_0(a))\colon (a =_A b) \to R(a,b)$ is an equivalence.*

e) *For any reflexive relation $\langle S, s_0 \rangle$ over $A$, the type of morphisms of reflexive relations $\mathsf{rrmap}(R,S)$ is contractible.*

*Proof.*

a) $\Rightarrow$ b) Fix $a\colon A$. Given a local motive $C\colon \prod_{y\colon A} R(a,y) \to \mathcal{U}$ and a base case $c_0\colon C(a, r_0(a))$, we must construct a dependent function of type

$$\prod_{y\colon A} \prod_{r\colon R(a,y)} C(y,r).$$

To apply the global induction principle, we define a motive

$$\begin{aligned} D&\colon \prod_{x,y\colon A} R(x,y) \to \mathcal{U} \\ D(x,y,r) &:\equiv \prod_{p\colon a =_A x} C\big(y, \mathsf{transport}^{R(-,y)}(p^{-1}, r)\big), \end{aligned}$$

which type-checks because $\mathsf{transport}^{R(-,y)}(p^{-1})\colon R(x,y) \to R(a,y)$.

To provide a global base case $d\colon \prod_{x\colon A} D(x,x,r_0(x))$, we construct $d(x)$ for a given $x\colon A$. By path induction on $p\colon a =_A x$, it suffices to consider the case where $x \equiv a$ and $p \equiv \mathsf{refl}_a$. In this case, the required type reduces to $C(a, r_0(a))$, so we may define

$$d(a, \mathsf{refl}_a) :\equiv c_0. \tag{5.2}$$

By the global induction principle of $\langle R, r_0\rangle$, there exists a function

$$f\colon \prod_{x,y\colon A} \prod_{r\colon R(x,y)} D(x,y,r)$$

such that $f(x,x,r_0(x)) = d(x)$ for all $x\colon A$.

Thus, for the required local section we have

$$\begin{aligned} g&\colon \prod_{y\colon A} \prod_{r\colon R(a,y)} C(y,r) \\ g(y,r) &:\equiv f(a,y,r,\mathsf{refl}_a). \end{aligned} \tag{5.3}$$

This is well-defined because the type of the application is

$$f(a,y,r,\mathsf{refl}_a)\colon C\big(y, \mathsf{transport}^{R(-,y)}(\mathsf{refl}_a^{-1}, r)\big),$$

which reduces definitionally to $C(y,r)$ since $\mathsf{refl}_a^{-1} \equiv \mathsf{refl}_a$, making the transport trivially evaluate to $r$.

For the computation rule of $g$, we have

$$\begin{aligned} g(a, r_0(a)) &\equiv f(a,a,r_0(a),\mathsf{refl}_a) && ;\ (5.3) \\ &= d(a,\mathsf{refl}_a) && ;\ \text{comp. rule for } f \\ &\equiv c_0 && ;\ (5.2). \end{aligned}$$

Thus, $\langle R(a), r_0(a)\rangle$ is an identity system at $a$.

b) $\Leftrightarrow$ c) This is a direct consequence of the logical equivalence a) $\Leftrightarrow$ c) of Theorem 5.12.7.

c) $\Leftrightarrow$ d) This is a direct consequence of the logical equivalence c) $\Leftrightarrow$ b) of Theorem 5.12.7.

d) $\Rightarrow$ e) Since d) $\Leftrightarrow$ c) $\Leftrightarrow$ b), we can assume that for all $a\colon A$, the pointed predicate $\langle R(a), r_0(a)\rangle$ is an identity system at $a$.

Now, consider the type from condition e):

$$T :\equiv \sum_{g\colon \prod_{a,b\colon A} R(a,b)\to S(a,b)} \prod_{a\colon A} g(a,a,r_0(a)) =_{S(a,a)} s_0(a)$$

By currying the domain of $g$ and applying 39. Type-theoretic Axiom of Choice, $T$ is equivalent to the type

$$\begin{aligned} T' &:\equiv \prod_{a\colon A} \sum_{h\colon \prod_{b\colon A} R(a,b)\to S(a,b)} h(a,r_0(a)) =_{S(a,a)} s_0(a) \\ &\equiv \prod_{a\colon A} \mathsf{ppmap}(R(a), S(a)) \end{aligned}$$

According to part d) of Theorem 5.12.7, $\mathsf{ppmap}(R(a), S(a))$ is contractible. By Theorem 3.4.8, it follows that $T'$, and hence $T$, is contractible.

e) $\Rightarrow$ a) We will show that condition a) holds by establishing the chain of implications

$$\text{e)} \Rightarrow \text{c)} \overset{\checkmark}{\Leftrightarrow} \text{b)} \Rightarrow \text{a)}.$$

To prove c), we apply condition e) to the type family

$$S(a,b) :\equiv \Sigma_A R(a,-)$$

and the base points $s_0(a) :\equiv \langle a, r_0(a)\rangle$. By part e), the type

$$T :\equiv \sum_{g\colon \prod_{a,b\colon A} R(a,b)\to \Sigma_A R(a,-)} \prod_{a\colon A} g(a,a,r_0(a)) =_{S(a,a)} \langle a, r_0(a)\rangle$$

is contractible.

Consider the elements $\langle g_1, p_1\rangle, \langle g_2, p_2\rangle\colon T$, defined by

$$\begin{aligned} g_1(a,b,r) &:\equiv \langle b, r\rangle && \text{and} && p_1(a) :\equiv \mathsf{refl}_{\langle a, r_0(a)\rangle}, \\ g_2(a,b,r) &:\equiv \langle a, r_0(a)\rangle && \text{and} && p_2(a) :\equiv \mathsf{refl}_{\langle a, r_0(a)\rangle}. \end{aligned}$$

These elements are well-defined because

$$g_1(a,a,r_0(a)),\ g_2(a,a,r_0(a)) \equiv \langle a, r_0(a)\rangle.$$

Since $T$ is contractible, both pairs are equal, which implies $g_1 = g_2$. Hence, for any $a, b\colon A$ and $r\colon R(a,b)$, we have $\langle b, r\rangle = \langle a, r_0(a)\rangle$.

This means that for any $a\colon A$, every element in the total space $\Sigma_A R(a,-)$ is equal to the center $\langle a, r_0(a)\rangle$. Thus, $\Sigma_A R(a,-)$ is contractible, establishing condition c).

Having established condition b), we can now derive condition a).

Let

$$D\colon \prod_{x,y\colon A} R(x,y) \to \mathcal{U}$$

be a global motive with base case

$$d_0\colon \prod_{x\colon A} D(x,x,r_0(x)).$$

For any fixed $a\colon A$, condition b) provides the local induction principle for $R(a)$. We define the local motive $C_a(y,r) :\equiv D(a,y,r)$ and the local base case $c_a :\equiv d_0(a)\colon C_a(a,r_0(a))$. The local eliminator yields a dependent function

$$f_a\colon \prod_{y\colon A} \prod_{r\colon R(a,y)} D(a,y,r)$$

such that $f_a(a,r_0(a)) = d_0(a)$. Thus, we may define the global section $f(x,y,r) :\equiv f_x(y,r)$. The computation rule holds by definition:

$$f(x,x,r_0(x)) \equiv f_x(x,r_0(x)) = d_0(x)$$

for all $x\colon A$. Therefore, $\langle R, r_0\rangle$ is a global identity system over $A$, which concludes the proof.

□

**Corollary 5.12.11.** [Equivalence induction] *Given a type family*

$$D\colon \prod_{A,B:\mathcal{U}} (A \simeq B) \to \mathcal{U}$$

*and function*

$$d\colon \prod_{A:\mathcal{U}} D(A,A,\mathsf{id}_A),$$

*there exists*

$$f\colon \prod_{A,B:\mathcal{U}} \prod_{e\colon A\simeq B} D(A,B,e)$$

*such that* $f(A,A,\mathsf{id}_A) = d(A)$ *for all* $A\colon \mathcal{U}$.

*Proof.* Define $R(A, B) :\equiv A \simeq B$ and $r_0(A) :\equiv \mathsf{id}_A$. By the univalence axiom, for any types $A, B : \mathcal{U}$, the canonical map

$$\mathsf{idtoeqv}_{A,B} : (A =_{\mathcal{U}} B) \to (A \simeq B)$$

is an equivalence.

We claim that $\mathsf{idtoeqv}_{A,B}(p)$ coincides with $\mathsf{transport}^{R(A)}(p, \mathsf{id}_A)$. By path induction on $p : A =_{\mathcal{U}} B$, it suffices to consider the case when $B \equiv A$ and $p \equiv \mathsf{refl}_A$. In that case, $\mathsf{transport}^{R(A)}(\mathsf{refl}_A, \mathsf{id}_A) \equiv \mathsf{id}_A$ and $\mathsf{idtoeqv}_{A,A}(\mathsf{refl}_A) \equiv \mathsf{id}_A$ as well. Consequently, part d) of Theorem 5.12.10 is met.

The conclusion is a direct consequence of part a) of said theorem. □

**Corollary 5.12.12.** [Homotopy induction] *Let* $\Pi_A B :\equiv \prod_{a:A} B(a)$. *Given*

$$D : \prod_{f,g:\Pi_A B} (f \sim g) \to \mathcal{U}$$

*and*

$$d : \prod_{f:\Pi_A B} D(f, f, \lambda x.\, \mathsf{refl}_{f(x)}),$$

*there exists*

$$k : \prod_{f,g:\Pi_A B} \prod_{h:f\sim g} D(f, g, h)$$

*such that*

$$k(f, f, \lambda x.\, \mathsf{refl}_{f(x)}) = d(f)$$

*for all* $f$.

*Proof.* Define

$$R(f, g) :\equiv f \sim g \quad \text{and} \quad r_0(f) :\equiv \lambda x.\, \mathsf{refl}_{f(x)}.$$

By function extensionality, for any $f, g : \Pi_A B$, the canonical map

$$\mathsf{happly}_{f,g} : (f =_{\Pi_A B} g) \to (f \sim g)$$

is an equivalence.

We claim that $\mathsf{happly}_{f,g}(p)$ coincides with $\mathsf{transport}^{R(f)}(p, r_0(f))$. To see this, we proceed by path induction on $p : f =_{\Pi_A B} g$, which reduces the goal to the case where $g \equiv f$ and $p \equiv \mathsf{refl}_f$. In this case, the claim holds trivially because both $\mathsf{happly}_{f,f}(\mathsf{refl}_f)$ and $\mathsf{transport}^{R(f)}(\mathsf{refl}_f, r_0(f))$ are definitionally equal to $r_0(f)$.

Therefore, the function

$$\mathsf{transport}^{R(f)}(-, r_0(f)) : (f =_{\Pi_A B} g) \to R(f, g)$$

is an equivalence for all $f, g : \Pi_A B$.

This establishes condition d) of Theorem 5.12.10 for the type $\Pi_A B$. By the logical equivalence of that theorem, condition a) holds, meaning that $\langle R, r_0 \rangle$ is an identity system over $\Pi_A B$. Expanding this definition yields exactly what the corollary states. □

## 5.13 Exercises

**Exercise 5.13.1.** *Let $A$ be a type, and consider the inductive type $\mathsf{L}_A$ generated by the constructors $\mathsf{nil} : \mathsf{L}_A$ and $\mathsf{cons} : A \to \mathsf{L}_A \to \mathsf{L}_A$.*

*Formulate the induction principle for $\mathsf{L}_A$. Specifically, given a type family $E : \mathsf{L}_A \to \mathcal{U}$, state the following:*

a) *The type of the base case recurrence $e_{\mathsf{nil}}$.*

b) *The type of the inductive step recurrence $e_{\mathsf{cons}}$, assuming the statement $E$ holds for a previously constructed list.*

c) *The full signature of the dependent eliminator function $\mathsf{ind}_{\mathsf{L}}$.*

d) *The definitional computation rules for $\mathsf{ind}_{\mathsf{L}}$ when applied to both $\mathsf{nil}$ and $\mathsf{cons}(a, \ell)$.*

*Solution.*

a) $e_{\mathsf{nil}} : E(\mathsf{nil})$.

b)
$$e_{\mathsf{cons}} : \prod_{a:A} \prod_{\ell:\mathsf{L}_A} E(\ell) \to E(\mathsf{cons}(a, \ell)).$$

c)
$$\mathsf{ind}_{\mathsf{L}_A} : \prod_{E:\mathsf{L}_A \to \mathcal{U}} E(\mathsf{nil}) \to \Big(\prod_{a:A} \prod_{\ell:\mathsf{L}_A} E(\ell) \to E(\mathsf{cons}(a, \ell))\Big) \to \prod_{\ell:\mathsf{L}_A} E(\ell).$$

d)
$$\begin{aligned} \mathsf{ind}_{\mathsf{L}_A}(E, e_{\mathsf{nil}}, e_{\mathsf{cons}}, \mathsf{nil}) &\equiv e_{\mathsf{nil}}, \\ \mathsf{ind}_{\mathsf{L}_A}(E, e_{\mathsf{nil}}, e_{\mathsf{cons}}, \mathsf{cons}(a, \ell)) &\equiv e_{\mathsf{cons}}(a, \ell, \mathsf{ind}_{\mathsf{L}_A}(E, e_{\mathsf{nil}}, e_{\mathsf{cons}}, \ell)). \end{aligned}$$

□

**Exercise 5.13.2.** *Construct two functions on natural numbers which satisfy the same recurrence $(e_z, e_s)$ judgmentally, but are not judgmentally equal.*

*Solution.* Define $C :\equiv \mathbb{N}$ and

- $e_z :\equiv 0$,
- $e_s(n, k) :\equiv \mathsf{succ}(k)$.

Then the function $f\colon \mathbb{N} \to C$ defined by $f(n) :\equiv n + 0$ satisfies, according to 50. Addition of natural numbers,

$$\begin{aligned} f(0) &\equiv 0 + 0 \equiv 0 \equiv e_z, \\ f(\mathsf{succ}(n)) &\equiv \mathsf{succ}(n) + 0 \\ &\equiv \mathsf{succ}(n + 0) \\ &\equiv e_s(n, f(n)). \end{aligned}$$

On the other hand, $g(n) :\equiv n$ satisfies

$$\begin{aligned} g(0) &\equiv 0 \equiv e_z, \\ g(\mathsf{succ}(n)) &\equiv \mathsf{succ}(n) \\ &\equiv e_s(n, g(n)). \end{aligned}$$

The functions $f$ and $g$ are not judgmentally equal because our definition of $\mathsf{add}$ satisfies $0 + n \equiv n$, but not $n + 0 \equiv n$, as explained in 54. Blocked computation.

□

**Exercise 5.13.3.** *Construct two distinct recurrences $(e_z, e_s)$ on the same type $E$ that are judgmentally satisfied by a single function $f\colon \mathbb{N} \to E$.*

*Solution.* Since $e_z \equiv f(0)$, the recurrences must differ in $e_s$. Let $e'_s$ denote the second step function. Thus, we must have

$$e'_s(n, f(n)) \equiv f(\mathsf{succ}(n)) \equiv e_s(n, f(n)). \tag{$*$}$$

For example, if $E :\equiv \mathbb{N}$, $e_z :\equiv 0$, and $f(n) :\equiv 0$ for $n\colon \mathbb{N}$, we can have

$$e_s(n, m) :\equiv m \quad \text{and} \quad e'_s(n, m) :\equiv m + 0.$$

As we saw in the previous exercise, $e_s \not\equiv e'_s$. However, $(*)$ does hold.

□

**Exercise 5.13.4.** *Show that for any type family $E\colon 2 \to \mathcal{U}$, the induction operator*

$$\mathsf{ind}_2(E)\colon E(0_2) \times E(1_2) \to \prod_{b\colon 2} E(b)$$

*is an equivalence.*

*Solution.* Let $f :\equiv \mathsf{ind}_2(E)$. Define

$$\begin{aligned} g\colon \Big(\prod_{b\colon 2} E(b)\Big) &\to E(0_2) \times E(1_2) \\ \epsilon &\mapsto (\epsilon(0_2), \epsilon(1_2)). \end{aligned}$$

We claim that $g$ is a quasi-inverse of $f$. Indeed,

$$\begin{aligned} g \circ f(e_0, e_1) &\equiv g(f(e_0, e_1)) \\ &\equiv \big(f(e_0, e_1)(0_2), f(e_0, e_1)(1_2)\big) \\ &\equiv (e_0, e_1) \end{aligned}$$

by the computation rule of $f$. Thus, by the induction principle for the cartesian product applied to the type family

$$\begin{aligned} C\colon E(0_2) \times E(1_2) &\to \mathcal{U} \\ C(x) &:\equiv g(f(x)) = x, \end{aligned}$$

we obtain a homotopy

$$\mathsf{ind}_\times\big(C, \lambda e_0.\, \lambda e_1.\, \mathsf{refl}_{(e_0,e_1)}\big)\colon g \circ f \sim \mathsf{id}.$$

The other composition computes to

$$f \circ g(\epsilon) \equiv f(\epsilon(0_2), \epsilon(1_2)).$$

By the computation rules of $f$, it follows that

$$\begin{aligned} (f \circ g(\epsilon))(0_2) &\equiv \epsilon(0_2), \\ (f \circ g(\epsilon))(1_2) &\equiv \epsilon(1_2). \end{aligned}$$

Thus, for a fixed $\epsilon$, applying the induction principle for the boolean type to the motive

$$\begin{aligned} P\colon 2 &\to \mathcal{U}, \\ P(b) &:\equiv (f \circ g(\epsilon))(b) = \epsilon(b) \end{aligned}$$

yields the homotopy

$$\mathsf{ind}_2(P, \mathsf{refl}_{\epsilon(0_2)}, \mathsf{refl}_{\epsilon(1_2)})\colon \prod_{b\colon 2} (f \circ g(\epsilon))(b) = \epsilon(b).$$

By function extensionality, this implies $f \circ g(\epsilon) = \epsilon$. Hence, $f \circ g \sim \mathsf{id}$, which completes the proof that $f$ and $g$ are quasi-inverses.

□

**Exercise 5.13.5.** *Show that the analogous statement to* Exercise 5.13.4 *for* $\mathbb{N}$ *fails.*

*Solution.* In analogy with the case of the boolean type, we have to show an example where the induction operator

$$\mathsf{ind}_{\mathbb{N}}(E)\colon E(0) \times \left(\prod_{n:\mathbb{N}} E(n) \to E(\mathsf{succ}(n))\right) \to \prod_{n:\mathbb{N}} E(n)$$

is not an equivalence.

Let $E\colon \mathbb{N} \to \mathcal{U}$ be the constant family $E(n) :\equiv \mathbb{N}$. For the sake of simplicity, let $\mathsf{ind}_{\mathbb{N}}(E)$ be denoted by $f$. Then,

$$f\colon \mathbb{N} \times (\mathbb{N} \to \mathbb{N} \to \mathbb{N}) \to (\mathbb{N} \to \mathbb{N}).$$

Consider the step function

$$\begin{aligned} e_s &\colon \mathbb{N} \to \mathbb{N} \to \mathbb{N} \\ &\quad n \mapsto (m \mapsto m). \end{aligned}$$

By the computation rules of $f$, we have

$$f(0, e_s, \mathsf{succ}(n)) \equiv e_s(n, f(0, e_s, n)) \equiv f(0, e_s, n). \tag{$*$}$$

We claim that $f(0, e_s, n) =_{\mathbb{N}} 0$ for $n\colon \mathbb{N}$. To see this, consider the motive

$$\begin{aligned} &C\colon \mathbb{N} \to \mathcal{U} \\ &C :\equiv \lambda n.\, f(0, e_s, n) =_{\mathbb{N}} 0 \end{aligned}$$

and define

$$\begin{aligned} c_s\colon \prod_{n:\mathbb{N}} C(n) &\to C(\mathsf{succ}(n)) \\ c_s &:\equiv \lambda n.\, \lambda p.\, p, \end{aligned}$$

which is well-typed by $(*)$. Thus, by induction, $f(0, e_s, n) =_{\mathbb{N}} 0$ for all $n\colon \mathbb{N}$.

On the other hand, the step function

$$\begin{aligned} e'_s &\colon \mathbb{N} \to \mathbb{N} \to \mathbb{N} \\ &\quad n \mapsto (m \mapsto 0) \end{aligned}$$

yields the function $f(0, e'_s)$, which can also be inductively shown to satisfy $f(0, e'_s, n) =_{\mathbb{N}} 0$ for all $n\colon \mathbb{N}$.

By function extensionality, $f(0, e_s) =_{\mathbb{N}\to\mathbb{N}} f(0, e'_s)$. However, $e_s$ and $e'_s$ are not propositionally equal because $e_s(0, 1) \equiv 1$ and $e'_s(0, 1) \equiv 0$, and 1 is not propositionally equal to 0.

Since $f$ maps two propositionally distinct elements of its domain to the same function, it is not an embedding. The conclusion that $f$ is not an equivalence then follows from Theorem 4.5.4.

□

**Exercise 5.13.6.** *Show that if we assume simple instead of dependent elimination for* $\mathsf{W}$*-types, the uniqueness property (the analogue of* Theorem 5.3.4*) fails to hold. That is, exhibit a type satisfying the recursion principle of a* $\mathsf{W}$*-type, but for which functions are not determined uniquely by their recurrence.*

*Solution.* We need to define a type $A$, a type family $B\colon A \to \mathcal{U}$, and a type $W$ equipped with a constructor

$$\mathsf{sup}\colon \prod_{a\colon A}(B(a) \to W) \to W,$$

that satisfies the simple recursion principle. That is, for any target type $C$ and step function

$$s_C\colon \prod_{a\colon A}(B(a) \to C) \to C,$$

there must exist a map $f\colon W \to C$ satisfying the computation rule

$$f(\mathsf{sup}(a, \phi)) \equiv s_C(a, f \circ \phi)$$

for all $a\colon A$ and $\phi\colon B(a) \to W$.

To show that the uniqueness property fails, we must exhibit a specific target type $C$ and a specific step function $s_C$ for which there exists a second, propositionally distinct map $g\colon W \to C$ that also satisfies the computation rule

$$g(\mathsf{sup}(a, \phi)) \equiv s_C(a, g \circ \phi).$$

Suppose that $B(a) \equiv \mathsf{0}$ for all $a\colon A$. Then, $\mathsf{sup}$ reduces to a map $u\colon A \to W$ and, similarly, $s_C$ reduces to a map $s_C\colon A \to C$. Thus, the computation rules become

$$f \circ u \equiv s_C,$$
$$g \circ u \equiv s_C.$$

Hence, $f \circ u \equiv g \circ u$. However, this does not suffice to imply in general that $f =_{W\to C} g$.

For instance, consider the case where

$$A \equiv \mathsf{1}, \quad W :\equiv \mathsf{1} + \mathsf{1}, \quad \text{and} \quad u :\equiv \mathsf{inl}.$$

Given any target type $C$ and step function $s_C\colon \mathsf{1} \to C$, the constant map $f(w) :\equiv s_C(\star)$ satisfies the recursion principle. However, if we choose a target type $C$ that contains an element $c\colon C$ propositionally distinct from $s_C(\star)$, we can define a second map $g\colon W \to C$ by

$$g(\mathsf{inl}(\star)) :\equiv s_C(\star), \quad g(\mathsf{inr}(\star)) :\equiv c.$$

Both $f$ and $g$ satisfy the computation rule, but they are not propositionally equal.

□

**Exercise 5.13.7.** *Suppose there is a type $C$ "inductively defined" by the single constructor*

$$\mathsf{c}\colon (C \to \mathsf{0}) \to C.$$

*By* Definition 5.5.2*, the recursion principle for this type states that for any target type $P$, to define a function $f\colon C \to P$, it suffices to provide a step function*

$$s\colon (C \to \mathsf{0}) \to (P \to \mathsf{0}) \to P.$$

*More precisely, assume we are given the recursor*

$$\mathsf{rec}_C\colon \prod_{P:\mathcal{U}} \big((C \to \mathsf{0}) \to (P \to \mathsf{0}) \to P\big) \to C \to P.$$

*Show that even without assuming any computation rules*[2] *for $\mathsf{rec}_C$, the existence of this recursion principle is inconsistent; that is, it implies that $\mathsf{0}$ is inhabited.*

*Solution.* Take $P :\equiv C \to \mathsf{0}$. We can define the step function $s$ by

$$s\colon (C \to \mathsf{0}) \to ((C \to \mathsf{0}) \to \mathsf{0}) \to C \to \mathsf{0}$$
$$(h, \phi) \mapsto (c \mapsto \phi(h)).$$

This is well-typed because, given $h\colon C \to \mathsf{0}$ and $\phi\colon (C \to \mathsf{0}) \to \mathsf{0}$, we obtain $\phi(h)\colon \mathsf{0}$.

Assuming the existence of $\mathsf{rec}_C$, we obtain the map

$$f :\equiv \mathsf{rec}_C(C \to \mathsf{0}, s)\colon C \to (C \to \mathsf{0}).$$

We can now define the diagonal function $d\colon C \to \mathsf{0}$ by $d(x) :\equiv f(x)(x)$. Thus, the constructor yields an element $c_0 :\equiv \mathsf{c}(d)\colon C$.

Finally, evaluating the diagonal function on this element produces $d(c_0)\colon \mathsf{0}$, successfully inhabiting the empty type.

□

**Exercise 5.13.8.** *Suppose we have a type $D$ that is "inductively defined" by a single constructor*

$$\mathsf{scott}\colon (D \to D) \to D.$$

*According to* Definition 5.5.4*, this recursor must be described by means of the type operator*

$$\Phi(X) :\equiv D \to X.$$

*Consequently, to define a function $f\colon D \to P$ by recursion, we must provide a step function $s$ and apply the recursor*

$$\mathsf{rec}_D\colon \prod_{P:\mathcal{U}} \big((D \to D) \to (D \to P) \to P\big) \to D \to P,$$

[2] See equation (5.1).

*subject to the computation rule*

$$\mathsf{rec}_D(P, s, \mathsf{scott}(\alpha)) \equiv s(\alpha, \lambda d.\, \mathsf{rec}_D(P, s, \alpha(d)))$$

*for any* $\alpha\colon D \to D$.

*Show that this recursion principle is inconsistent; that is, it yields an inhabitant of the empty type* $\mathsf{0}$.

*Solution.* Set $P :\equiv D \to \mathsf{0}$. We can define the step function

$$\begin{aligned} s\colon (D \to D) \to (D \to D \to \mathsf{0}) &\to D \to \mathsf{0} \\ (\alpha, \phi) &\mapsto \lambda d.\, \phi(d)(\alpha(d)) \end{aligned}$$

and obtain a recursive function $f\colon D \to D \to \mathsf{0}$.

In particular, setting $\alpha :\equiv \mathsf{id}_D$ and $d_0 :\equiv \mathsf{scott}(\mathsf{id}_D)\colon D$, we obtain

$$f(d_0)\colon D \to \mathsf{0} \quad \text{and} \quad f(d_0)(d_0)\colon \mathsf{0},$$

as desired.

**Note.** Even though it is not used in the solution above, it is worth instantiating the computation rule (5.1) in this case:

$$\begin{aligned} \mathsf{rec}_D(P, s, \mathsf{scott}(\alpha)) &\equiv s\big(\alpha, \mathsf{map}_\Phi(\mathsf{rec}_D(P, s))(\alpha)\big) \\ &\equiv s\big(\alpha, \mathsf{rec}_D(P, s) \circ \alpha\big) \\ &\equiv s\big(\alpha, \lambda d.\, \mathsf{rec}_D(P, s, \alpha(d))\big), \end{aligned}$$

because the functorial action is given by post-composition:

$$\mathsf{map}_\Phi(\mathsf{rec}_D(P, s))(\alpha) \equiv \mathsf{rec}_D(P, s) \circ \alpha.$$

□

**Exercise 5.13.9.** *Let* $A$ *be an arbitrary type and consider generally an "inductive definition" of a type* $L_A$ *with constructor*

$$\mathsf{lawvere}\colon (L_A \to A) \to L_A.$$

*Following* Definition 5.5.4*, to define a function* $f\colon L_A \to P$ *by recursion, we must provide a step function* $s$ *and apply the recursor*

$$\mathsf{rec}_{L_A}(P)\colon ((L_A \to A) \to P) \to L_A \to P,$$

*subject to the judgmental computation rule*

$$\mathsf{rec}_{L_A}(P, s, \mathsf{lawvere}(\alpha)) \equiv s(\alpha)$$

*for any* $\alpha\colon L_A \to A$.

*Using this, show that* $A$ *has the fixed-point property, i.e., for every function* $f\colon A \to A$ *there exists an* $a\colon A$ *such that* $f(a) =_A a$*. In particular,* $L_A$ *is inconsistent if* $A$ *is a type without the fixed-point property, such as* $\mathsf{0}$*,* $\mathsf{2}$*, or* $\mathbb{N}$.

Hint: Set $P :\equiv A$.

*Solution.* In this case, the variable $L_A$ does not occur in any strictly positive position within the operator $\Phi(X) :\equiv X \to A$. Consequently, the type $\Phi'(L_A, L_A \times P)$ evaluates simply to $L_A \to A$, which is identical to $\Phi(L_A)$. This allows us to shorten the step function signature, as established in the following

**Lemma.** *Let $\Phi\colon \mathcal{U} \to \mathcal{U}$ be a type-level operation representing the signature of a constructor $\mathsf{c}\colon \Phi(X) \to X$, such that there are no strictly positive occurrences of $X$ in the expression defining $\Phi(X)$. Under the convention of* Definition 5.5.4*, the general computation rule for the recursor reduces to*

$$\mathsf{rec}_X(P, s, \mathsf{c}(x)) \equiv s(x, x)$$

*for all $x\colon \Phi(X)$. Consequently, the step function signature can be shortened to*

$$s\colon \Phi(X) \to P,$$

*which transforms the computation rule to*

$$\mathsf{rec}_X(P, s, \mathsf{c}(u)) \equiv s(u).$$

Proof. By Definition 5.5.1, all occurrences of $X$ are in negative positions and therefore fixed as parameter types. Hence, $\Phi'(X, X \times P) \equiv \Phi(X)$.

Consequently, the functorial action $\mathsf{map}_\Phi(f)\colon \Phi(X) \to \Phi(X \times P)$ acts trivially as the identity function:

$$\mathsf{map}_\Phi(f) \equiv \mathsf{map}_\Phi(\mathsf{id}_X) \equiv \mathsf{id}_{\Phi(X)}.$$

By (5.2), the general computation rule for the recursor yields the equation

$$f(\mathsf{c}(u)) \equiv s\big(\mathsf{map}_\Phi(\lambda x.\,(x, f(x)))(u)\big) \equiv s(u).$$

■

The goal of the exercise is to construct an inhabitant of the type

$$\prod_{f\colon A \to A} \sum_{a\colon A} f(a) =_A a.$$

Fix $f\colon A \to A$ and set $P :\equiv A$. We can define

$$\begin{aligned} s_f\colon (L_A \to A) &\to A \\ \alpha &\mapsto f \circ \alpha(\mathsf{lawvere}(\alpha)). \end{aligned}$$

By recursion we obtain

$$\mathsf{rec}_{L_A}(A)\colon ((L_A \to A) \to A) \to L_A \to A,$$

satisfying the computation rule

$$\mathsf{rec}_{L_A}(A, s_f, \mathsf{lawvere}(\alpha)) \equiv f \circ \alpha(\mathsf{lawvere}(\alpha)).$$

Let $u :\equiv \mathsf{lawvere}(\alpha)\colon L_A$. Then,

$$\mathsf{rec}_{L_A}(A, s_f, u) \equiv f(\alpha(u)).$$

In particular, setting $\alpha :\equiv \mathsf{rec}_{L_A}(A, s_f)$, we obtain:

$$\alpha(u) \equiv f(\alpha(u)),$$

which proves that $a :\equiv \alpha(u)$ is a *judgmental* fixed point of $f$.

**Note.** Had the computation rule been propositional instead, the fixed point of $f$ would have been propositional as well. This is key because we cannot define a type $L_A$ whose computation rule is strictly judgmental. To see this, consider the case $L_A \equiv A :\equiv 1$ with $\mathsf{lawvere}(\alpha) :\equiv \star$, which is studied in the next exercise.

□

**Exercise 5.13.10.** *Continuing from* Exercise 5.13.9*, consider $L_1$, which is not obviously inconsistent since $1$ does have the fixed-point property. Formulate an induction principle for $L_1$ and its computation rule, analogously to its recursor, and using this, prove that it is contractible.*

*Solution.* The corresponding induction principle would establish the existence of a map

$$\mathsf{ind}_{L_1}\colon \prod_{B\colon L_1\to\mathcal{U}} \Big( \prod_{\alpha\colon L_1\to 1} B(\mathsf{lawvere}(\alpha)) \Big) \to \prod_{u\colon L_1} B(u)$$

along with the computation rule

$$\mathsf{ind}_{L_1}(B, s, \mathsf{lawvere}(\alpha)) \equiv s(\alpha),$$

for all

$$s\colon \prod_{\alpha\colon L_1\to 1} B(\mathsf{lawvere}(\alpha)).$$

First, let us observe that the constant function $\alpha_\star :\equiv \lambda u.\star$ inhabits $L_1 \to 1$. Therefore, $L_1$ is inhabited by $u_0 :\equiv \mathsf{lawvere}(\alpha_\star)$. Thus, the contractibility of $L_1$ reduces to showing that, for every $u\colon L_1$, we can construct a path $p\colon u_0 =_{L_1} u$.

By defining $B :\equiv \lambda u.\, u_0 =_{L_1} u$, our goal reduces to specifying an inhabitant of

$$\prod_{\alpha\colon L_1\to 1} u_0 =_{L_1} \mathsf{lawvere}(\alpha).$$

By the action on paths $\mathsf{ap}_{\mathsf{lawvere}}$, the goal reduces to

$$\prod_{\alpha\colon L_1\to 1} \alpha_\star =_{L_1\to 1} \alpha,$$

which follows from function extensionality because the codomain $1$ is contractible. □

**Exercise 5.13.11.** *In* § 5.1 *we defined the type* $\mathsf{List}(A)$ *of finite lists of elements of type* $A$*. Consider a similar inductive definition of a type* $\mathsf{Lost}(A)$ *whose only constructor is*

$$\mathsf{cons}\colon A \to \mathsf{Lost}(A) \to \mathsf{Lost}(A).$$

*Show that* $\mathsf{Lost}(A)$ *is equivalent to* $\mathsf{0}$*.*

*Solution.* To deduce $\Phi$ from the signature of $\mathsf{cons}$, we need to match it with a map

$$\Phi(\mathsf{Lost}(A)) \to \mathsf{Lost}(A).$$

Since $\mathsf{cons}$ can be uncurried as

$$\mathsf{cons}\colon A \times \mathsf{Lost}(A) \to \mathsf{Lost}(A),$$

we deduce that $\Phi(X) :\equiv A \times X$.

According to Definition 5.5.4, to recursively define a map $f\colon \mathsf{Lost}(A) \to P$, it suffices to specify a step function $s\colon \Phi(\mathsf{Lost}(A) \times P) \to P$. Substituting our functor, this requires

$$s\colon A \times (\mathsf{Lost}(A) \times P) \to P,$$

which, by currying, is equivalent to

$$s\colon A \to \mathsf{Lost}(A) \to P \to P.$$

In the particular case where $P :\equiv \mathsf{0}$, we have

$$\begin{aligned} &s\colon A \to \mathsf{Lost}(A) \to \mathsf{0} \to \mathsf{0} \\ &s :\equiv \lambda a.\, \lambda \ell.\, \lambda z.\, z. \end{aligned}$$

Consequently, we obtain a recursively defined map $f\colon \mathsf{Lost}(A) \to \mathsf{0}$, which proves that $\mathsf{Lost}(A)$ is equivalent to $\mathsf{0}$. □

**Exercise 5.13.12.** *Let* $A$ *be a mere proposition and* $B\colon A \to \mathcal{U}$ *a type family.*

a) *Show that* $\mathsf{W}_A B$ *is a mere proposition.*

b) *Show that* $\mathsf{W}_A B$ *is equivalent to* $\sum_{a\colon A} \neg B(a)$*.*

c) *Without using* $\mathsf{W}_A B$*, show that* $\sum_{a\colon A} \neg B(a)$ *is a homotopy* $\mathsf{W}$*-type* $\mathsf{W}_A^h B$ *in the sense of* § 5.6*.*

Hint: The function type $X \to Y$ is contractible, whenever there is a map $X \to \mathsf{0}$.

*Solution.*

a) This is Theorem 5.3.5.

b) Let $\mathsf{W} :\equiv \mathsf{W}_A B$.

To define a forward map $\mathsf{W} \to \Sigma_A \neg B$, it suffices to specify a short step function

$$\alpha\colon \prod_{a\colon A} (B(a) \to \Sigma_A \neg B) \to \Sigma_A \neg B.$$

Fix $a\colon A$ and $h_a\colon B(a) \to \Sigma_A \neg B$. For every $b\colon B(a)$, the components of $h_a(b)$ define the terms

$$x_b :\equiv \pi_1(h_a(b))\colon A \quad \text{and} \quad \zeta_b :\equiv \pi_2(h_a(b))\colon B(x_b) \to \mathsf{0}.$$

Since $A$ is a mere proposition, there is a witness $\pi\colon \mathsf{isProp}(A)$, which yields the path $p_b :\equiv \pi(a, x_b)\colon a =_A x_b$. Thus, we obtain

$$\mathsf{transport}^B(p_b, b)\colon B(x_b),$$

which constructs

$$\begin{aligned} \epsilon_a\colon B(a) &\to \mathsf{0} \\ b &\mapsto \zeta_b(\mathsf{transport}^B(p_b, b)). \end{aligned}$$

Hence, the desired step function is given by

$$\alpha(a, h_a) :\equiv \langle a, \epsilon_a \rangle.$$

For the backward map, we have

$$\begin{aligned} \Big(\sum_{a\colon A} \neg B(a)\Big) &\to \mathsf{W} \\ \langle a, \zeta \rangle &\mapsto \mathsf{sup}(a, \mathsf{rec}_{\mathsf{0}}(\mathsf{W}) \circ \zeta). \end{aligned}$$

Since $\neg B(a)$ and $\Sigma_A \neg B$ are mere propositions by Theorem 3.2.15, the conclusion follows from part a) and Lemma 3.2.4.

c) Let $\sigma$ denote the short step function

$$\sigma\colon \prod_{a\colon A} (B(a) \to \Sigma_A \neg B) \to \Sigma_A \neg B$$

designated as $\alpha$ in part b). The idea is to show that $\sigma$ plays the role of constructor for $\Sigma_A \neg B$ that $\mathsf{sup}$ plays for $\mathsf{W}$.

Suppose we are given a map

$$\alpha_C\colon \prod_{a\colon A} (B(a) \to C) \to C.$$

We must construct a map $\Sigma_A \neg B \to C$ from $\alpha_C$. Therefore, we define

$$\begin{aligned} \rho_C(\alpha_C)\colon \Sigma_A \neg B &\to C \\ \langle x, \zeta \rangle &\mapsto \alpha_C(x, \mathsf{rec}_{\mathsf{0}}(C) \circ \zeta). \end{aligned}$$

For the computation rule, we have

$$\begin{aligned}\rho_C(\alpha_C)(\sigma(a,h_a)) &\equiv \rho_C(\alpha_C)(\langle a,\epsilon_a\rangle)\\ &\equiv \alpha_C(a,\mathsf{rec}_{\mathsf{0}}(C)\circ\epsilon_a).\end{aligned}$$

On the other hand, according to Notation 5.4.13, we have to show that the above is equal to

$$\alpha_C(a,\rho_C(\alpha_C)\circ h_a).$$

By the action on paths of $\alpha_C(a,-)$, our goal reduces to showing that

$$\mathsf{rec}_{\mathsf{0}}(C)\circ\epsilon_a =_{B(a)\to C} \rho_C(\alpha_C)\circ h_a,$$

which is equivalent to the propositional commutativity of

$$\begin{array}{ccc} B(a) & \xrightarrow{h_a} & \Sigma_A \neg B \\ {\scriptstyle\epsilon_a}\downarrow & & \downarrow{\scriptstyle\rho_C(\alpha_C)} \\ \mathsf{0} & \xrightarrow[\mathsf{rec}_{\mathsf{0}}(C)]{} & C \end{array}$$

But this is direct consequence of the following

**Lemma.** *Let $X$ and $Y$ be types, and suppose there is a map $\epsilon\colon X\to\mathsf{0}$. Then, the type $X\to Y$ is contractible with center $\mathsf{rec}_{\mathsf{0}}(Y)\circ\epsilon$. In other words, for any function $f\colon X\to Y$, the diagram*

$$\begin{array}{ccc} X & & \\ {\scriptstyle\epsilon}\downarrow & \searrow^{f} & \\ \mathsf{0} & \xrightarrow[\mathsf{rec}_{\mathsf{0}}(Y)]{} & Y \end{array}$$

*commutes propositionally.*

Proof: Let $f_0 :\equiv \mathsf{rec}_{\mathsf{0}}(Y)\circ\epsilon$. Given $x\colon X$, we define the motive $E\colon \mathsf{0}\to\mathcal{U}$ by

$$E(z) :\equiv f_0(x) =_Y f(x).$$

Since $\epsilon(x)\colon \mathsf{0}$, the induction principle of the empty type produces

$$\mathsf{ind}_{\mathsf{0}}(E)(\epsilon(x))\colon f_0(x) =_Y f(x).$$

This constructs a homotopy $f_0\sim f$. The conclusion follows by function extensionality. ∎

Since we have provided both the recursion map $\rho_C$ and the computation rule for any structure map $\alpha_C$, we conclude that $\Sigma_A\neg B$ is a homotopy $\mathsf{W}$-type.

□

**Exercise 5.13.13.** *Let $A : \mathcal{U}$ and $B : A \to \mathcal{U}$.*

a) *Show that $\Sigma_A \neg B \to \mathsf{W}_A B$.*

b) *Show that $\mathsf{W}_A B \to \neg\Pi_A B$.*

*Solution.* Let $\mathsf{W} :\equiv \mathsf{W}_A B$.

a) We already defined this map in part b) of the previous exercise:

$$\Big(\sum_{a:\, A} \neg B(a)\Big) \to \mathsf{W}$$
$$\langle a, \zeta\rangle \mapsto \mathsf{sup}(a, \mathsf{rec}_{\mathbf{0}}(\mathsf{W}) \circ \zeta).$$

b) To define the map recursively, it suffices to define a short step function

$$\alpha : \prod_{a:\, A} (B(a) \to \neg\Pi_A B) \to \neg\Pi_A B.$$

Fix $a : A$. Then, we define

$$\alpha(a) : (B(a) \to \neg\Pi_A B) \to \neg\Pi_A B$$
$$\phi \mapsto \lambda f.\, \phi(f(a))(f).$$

□

**Exercise 5.13.14.** *Let $A : \mathcal{U}$ be a type and suppose that $B : A \to \mathcal{U}$ is decidable*[3]*. Show that $\mathsf{W}_A B \to \Sigma_A \neg B$.*

*Solution.* To construct the desired map, it suffices to specify a short step function

$$\alpha : \prod_{a:\, A} (B(a) \to \Sigma_A \neg B) \to \Sigma_A \neg B.$$

Fix $a : A$. In case we have a term $b : B(a)$, we can define

$$\alpha(a) :\equiv \lambda\phi.\, \phi(b),$$

the evaluation at $b$. If we have a term $\zeta : \neg B(a)$, we can set

$$\alpha(a) :\equiv \lambda\phi.\, \langle a, \zeta\rangle,$$

which is well-typed because $\langle a, \zeta\rangle : \Sigma_A \neg B$.

These settings define a map

$$s_a : (B(a) + \neg B(a)) \to (B(a) \to \Sigma_A \neg B) \to \Sigma_A \neg B.$$

[3] See Definition 3.2.10.

Thus, we obtain

$$\alpha :\equiv \lambda a.\, s_a(\partial(a)),$$

where

$$\partial\colon \prod_{a\colon A} B(a) + \neg B(a)$$

is the decidability function provided by the hypothesis. □

**Exercise 5.13.15.** *Show that the following are logically equivalent.*

a) $\mathsf{W}_A B \to \Sigma_A \neg B$ *for any* $A\colon \mathsf{Set}$ *and* $B\colon A \to \mathsf{Prop}$.

b) $\neg\Pi_A B \to \mathsf{W}_A B$ *for any* $A\colon \mathsf{Set}$ *and* $B\colon A \to \mathsf{Prop}$.

c) *The law of excluded middle.*[4]

*Now tweak the two first condition adding truncation, and prove that they are equivalent to condition c) above:*

a') $\mathsf{W}_A B \to \|\Sigma_A \neg B\|$ *for any* $A\colon \mathsf{Set}$ *and* $B\colon A \to \mathsf{Prop}$.

b') $\neg\Pi_A B \to \|\mathsf{W}_A B\|$ *for any* $A\colon \mathsf{Set}$ *and* $B\colon A \to \mathsf{Prop}$.

*Similarly, using* Corollary 3.1.10*, show that it is inconsistent to assume that either untruncated implication in* a) *or* b) *holds for all* $A\colon \mathcal{U}$ *and* $B\colon A \to \mathcal{U}$.

Hint: For a) ⇒ c), given $P\colon \mathsf{Prop}$, consider $A :\equiv P + \mathsf{isProp}(P)$. For b) ⇒ c), take $B$ as the constant $\mathsf{0}$.

*Solution.* In what follows, $\mathsf{W}$ will denote $\mathsf{W}_A B$.

a) ⇒ c) Fix $P\colon \mathsf{Prop}$. By Proposition 3.2.9 and Lemma 3.2.3, $\mathsf{isProp}(P) \simeq \mathsf{1}$. Thus, setting $A :\equiv P + \mathsf{isProp}(P)$ is equivalent to $A :\equiv P + \mathsf{1}$. However, even though $\mathsf{1}$ looks simpler, $P + \mathsf{isProp}(P)$ seems a more natural choice.

Let $B\colon A \to \mathsf{Prop}$ be defined by

$$B(\mathsf{inl}(x)) :\equiv \neg P$$
$$B(\mathsf{inr}(c)) :\equiv P,$$

where $c$ is a center of contraction of $\mathsf{isProp}(P)$ (or $\star$ if we decided to use $\mathsf{1}$ instead).

For $x\colon P$, define $\phi_x\colon \neg P \to \mathsf{W}$ by $\phi_x(\zeta) :\equiv \mathsf{rec}_{\mathsf{0}}(\mathsf{W})(\zeta(x))$. This yields a term $w_x :\equiv \mathsf{sup}(\mathsf{inl}(x), \phi_x)\colon \mathsf{W}$.

Moreover, we can reuse $w_x$ to define $\psi\colon P \to \mathsf{W}$ by $\psi(x) :\equiv w_x$. For the right branch of $A$, take:

$$\omega :\equiv \mathsf{sup}(\mathsf{inr}(c), \psi)\colon \mathsf{W}.$$

[4] Definition 3.2.10.

Since the right branch of $A$ is inhabited, we can applying the hypothesis to $\omega$ and obtain a term $\langle a, \beta \rangle \colon \Sigma_A \neg B$.

If $a \equiv \mathsf{inl}(x)$ for some $x \colon P$, we obtain $\mathsf{inl}(x) \colon P + \neg P$. If $a \equiv \mathsf{inr}(c)$, then $\beta \colon \neg P$ and $\mathsf{inr}(\beta) \colon P + \neg P$. In conclusion, $P$ is decidable.

c) $\Rightarrow$ a) This is a direct consequence of Exercise 5.13.14.

b) $\Rightarrow$ c) Fix $P \colon \mathsf{Prop}$. Let $A :\equiv P$, which is a set, and let $B \colon P \to \mathsf{Prop}$ be the constant family $B(x) :\equiv \mathsf{0}$.

By definition, $\Pi_A B \equiv \neg P$. Hence, $\neg \Pi_A B \equiv \neg\neg P$. By hypothesis, there is a map

$$\omega \colon \neg\neg P \to \mathsf{W}.$$

Define the map $g \colon \mathsf{W} \to P$ via $\mathsf{W}$-induction using the step function:

$$\prod_{x \colon P} (\mathsf{0} \to P) \to P$$
$$(x, \rho) \mapsto x.$$

The composition $g \circ \omega$ is a map of type $\neg\neg P \to P$. Since there is always a map in the other direction, we obtain c) from Exercise 3.7.16.

c) $\Rightarrow$ b) Since $B$ is a type family of mere propositions, the hypothesis implies the existence of a witness

$$\theta \colon \prod_{a \colon A} B(a) + \neg B(a).$$

Fix $\epsilon \colon \neg \Pi_A B$. To prove b), we need to map $\epsilon$ to an element of $\mathsf{W}$.

By the properties of the $\mathsf{sup}$ constructor, to produce an element $w \colon \mathsf{W}$, it suffices to select a specific $a \colon A$ and produce a map $\phi_a \colon B(a) \to \mathsf{W}$.

Given a term $\zeta \colon \neg B(a)$, this map is trivially fulfilled by $\mathsf{rec}_{\mathsf{0}}(\mathsf{W}) \circ \zeta$, which yields a term $w :\equiv \mathsf{sup}(a, \phi_a)$.

Using the decidability witness $\theta$, it is impossible for the left branch to hold for all $a \colon A$, as this would construct a dependent map in $\Pi_A B$, contradicting $\epsilon$. Therefore, the Law of Excluded Middle forces the existence of some $a \colon A$ where $\neg B(a)$ holds, achieving the construction of $w$.

Let us now address the second part of the exercise.

a') $\Rightarrow$ c) Using the notation from a) $\Rightarrow$ c), the new hypothesis yields a term $\sigma \colon \| \Sigma_A \neg B \|$.

As shown in the previous case, there is a map $\Sigma_A \neg B \to P + \neg P$. Since $P + \neg P$ is a mere proposition, this map factors through $\| \Sigma_A \neg B \|$. Applying this factored map to $\sigma$ yields a term of $P + \neg P$.

c) $\Rightarrow$ a') This follows immediately from c) $\Rightarrow$ a), since there is always a canonical map from any type to its truncation.

b') $\Rightarrow$ c) Using the notation from b) $\Rightarrow$ c), this time we obtain a map

$$\omega' \colon \neg\neg P \to \|\mathsf{W}\|.$$

The map $g \colon \mathsf{W} \to P$ can be defined in the same way as before. Since $P$ is a mere proposition, this map factors as $g' \colon \|\mathsf{W}\| \to P$. Thus, the composition $g' \circ \omega' \colon \neg\neg P \to P$ allows us to invoke Exercise 3.7.16 once again, arriving at the desired conclusion.

c) $\Rightarrow$ b') Immediate from c) $\Rightarrow$ b).

Regarding the final requirement of the exercise, as noted in a) $\Rightarrow$ c), if we replace $\mathsf{isProp}(P)$ with $\mathsf{1}$, the exact same argument applied to the untruncated premise a) shows that $P + \neg P$ is inhabited for any type $P \colon \mathcal{U}$. The inconsistency then follows from Corollary 3.1.10.

The same applies to the solution of b) $\Rightarrow$ c) using the untruncated premise b), since without truncation we can directly extract a term of $P$ from $\mathsf{W}$, bypassing the need for $P$ to be a mere proposition. □

**Exercise 5.13.16.** *For $A \colon \mathcal{U}$ and $B \colon A \to \mathcal{U}$, define*

$$\mathsf{W}'_{A,B} :\equiv \prod_{C:\mathcal{U}} \left( \prod_{a:\,A} (B(a) \to C) \to C \right) \to C.$$

*$\mathsf{W}'_{A,B}$ is called the impredicative encoding of $\mathsf{W}_A B$. Note that unlike $\mathsf{W}_A B$, it lives in a higher universe than $A$ and $B$.*

a) *Show that $\mathsf{W}'_{A,B}$ is logically equivalent to $\mathsf{W}_A B$.*

b) *Show that $\mathsf{W}'_{A,B}$ implies $\neg\neg\Sigma_A \neg B$.*

c) *Without using $\mathsf{W}_A B$, show that $\mathsf{W}'_{A,B}$ satisfies the same recursion principle as $\mathsf{W}_A B$ for defining functions into types in the universe $\mathcal{U}$ (to which it itself does not belong).*

d) *Using* LEM*, give an example of an $A \colon \mathcal{U}$ and a $B \colon A \to \mathcal{U}$ such that $\mathsf{W}'_{A,B}$ is not equivalent to $\mathsf{W}_A B$.*

Hint: Consider $A = \mathsf{2}$, $B = \mathsf{0}$. Then use truncation to apply LEM.

*Solution.* In what follows, we denote $\mathsf{W}'_{A,B}$ and $\mathsf{W}_A B$ by $\mathsf{W}'$ and $\mathsf{W}$, respectively.

a) For the forward map, define:

$$\begin{aligned} &\mathsf{W}' \to \mathsf{W} \\ &\quad f \mapsto f(\mathsf{W})(\mathsf{sup}). \end{aligned}$$

For the backward map, we apply Notation 5.4.13 for the recursion principle of $\mathsf{W}$-types:

$$\begin{aligned} &\mathsf{W} \to \mathsf{W}' \\ &\quad w \mapsto \lambda C.\, \lambda \alpha_C.\, \mathsf{hom}_{\mathsf{W}}(C, \alpha_C)(w). \end{aligned}$$

b) Let $w' \colon \mathsf{W}'$. We need to construct a term of $\neg\neg\Sigma_A \neg B$.

Consider the case where $C :\equiv \mathsf{0}$. Then,

$$w'(\mathsf{0}) \colon \Big(\prod_{x \colon A} \neg\neg B(x)\Big) \to \mathsf{0}.$$

By the recursion principle of $\Sigma$-types[5], the map $\mathsf{rec}_{\Sigma_A \neg B}(\mathsf{0})$ is an equivalence from $\prod_{x \colon A} \neg\neg B(x)$ to $\Sigma_A \neg B \to \mathsf{0} \equiv \neg\Sigma_A \neg B$ (Corollary 2.18.9).

Consequently, the type of $w'(\mathsf{0})$ is equivalent to $\neg\neg\Sigma_A \neg B$. Mapping $w'(\mathsf{0})$ along this equivalence yields the required term.

c) According to Notation 5.4.13, we have to prove without invoking $\mathsf{W}$-types that, given $C \colon \mathcal{U}$, there is a constructor

$$\mathsf{sup}' \colon \prod_{a \colon A} (B(a) \to \mathsf{W}') \to \mathsf{W}'$$

and a recursor map

$$\mathsf{hom}'_C \colon \Big(\prod_{a \colon A} (B(a) \to C) \to C\Big) \to \mathsf{W}' \to C$$

such that for every

$$\alpha'_C \colon \prod_{a \colon A} (B(a) \to C) \to C$$

we have the computation rule

$$\mathsf{hom}'_C(\alpha'_C)(\mathsf{sup}'(a, \phi)) \equiv \alpha'_C(a, \mathsf{hom}'_C(\alpha'_C) \circ \phi).$$

*Constructor:* Fix $a \colon A$ and $\phi \colon B(a) \to \mathsf{W}'$.

To construct a term $w' \colon \mathsf{W}'$ from $a$ and $\phi$, we fix a type $C \colon \mathcal{U}$ and aim to construct a map

$$\Big(\prod_{x \colon A} (B(x) \to C) \to C\Big) \to C.$$

To this end, let $f \colon \prod_{x \colon A} (B(x) \to C) \to C$.

Given $b \colon B(a)$, we obtain the term $\phi(b) \colon \mathsf{W}'$. By the definition of $\mathsf{W}'$,

$$\phi(b)(C) \colon \Big(\prod_{x \colon A} (B(x) \to C) \to C\Big) \to C.$$

In particular, $\phi(b)(C)(f) \colon C$. This allows us to define a map

$$\psi :\equiv \lambda b.\, \phi(b)(C)(f) \colon B(a) \to C.$$

It follows that $f(a)(\psi) \colon C$. Hence, the required map is

$$\mathsf{sup}'(a, \phi)(C)(f) :\equiv f(a)(\psi). \tag{5.1}$$

[5] 37. Recursion in $\Sigma$-types.

*Recursor:* To define the recursor, we have to fix $C : \mathcal{U}$ and produce, for every $f : \prod_{a:A}(B(a) \to C) \to C$, a map $\mathsf{W}' \to C$.

Let $w' : \mathsf{W}'$. By definition,

$$w'(C) : \Big(\prod_{a:A}(B(a) \to C) \to C\Big) \to C.$$

Therefore, given $f$ as above, $w'(C)(f)$ is a term of type $C$. Consequently, we can define the desired map as $w' \mapsto w'(C)(f)$. In other words

$$\mathsf{hom}'_C(f)(w') :\equiv w'(C)(f). \tag{5.2}$$

*Computation:* Using the notation and specifications above, we obtain

$$\begin{aligned}
\mathsf{hom}'_C(\underbrace{\alpha'_C}_{f})(\underbrace{\mathsf{sup}'(a,\phi)}_{w'}) &\equiv \underbrace{\mathsf{sup}'(a,\phi)(C)(\alpha'_C)}_{w'(C)(f)} && ;\ (5.2)\\
&\equiv \alpha'_C(a)(\underbrace{\lambda b.\ \underbrace{\phi(b)}_{w''}(C)(\alpha'_C)}_{\psi}) && ;\ (5.1)\\
&\equiv \alpha'_C(a)(\lambda b.\, \mathsf{hom}'_C(\alpha'_C)(\phi(b)))) && ;\ (5.2)\\
&\equiv \alpha'_C(a, \mathsf{hom}'_C(\alpha'_C) \circ \phi),
\end{aligned}$$

which is exactly the definitional identity we wanted to prove.

d) As proposed in the hint, let us compare $\mathsf{W} :\equiv \mathsf{W}_{2,0}$ and $\mathsf{W}' :\equiv \mathsf{W}'_{2,0}$.

By Exercise 5.13.14,

$$\begin{aligned}
\mathsf{W} &\to \Sigma_2 \neg 0\\
&\simeq \Sigma_2 1\\
&\simeq 2 \times 1\\
&\simeq 2.
\end{aligned}$$

Combining this with Exercise 5.13.13, we deduce that $\mathsf{W}$ is logically equivalent to $2$. On the other hand,

$$\begin{aligned}
\mathsf{W}' &\equiv \prod_{C:\mathcal{U}} \Big(\prod_{z:2}(0 \to C) \to C\Big) \to C\\
&\simeq \prod_{C:\mathcal{U}} \Big(\prod_{z:2} 1 \to C\Big) \to C\\
&\simeq \prod_{C:\mathcal{U}} (2 \to C) \to C\\
&\simeq \prod_{C:\mathcal{U}} (C \times C) \to C.
\end{aligned}$$

Let $\ell\colon \mathrm{LEM}$ be a witness of the law of excluded middle. Given $C\colon \mathcal{U}$, the type $\|C =_{\mathcal{U}} 2\|$ is a mere proposition. Thus, we obtain

$$\ell(\|C =_{\mathcal{U}} 2\|)\colon \|C =_{\mathcal{U}} 2\| + \neg\|C =_{\mathcal{U}} 2\|.$$

Define

$$\begin{aligned}\epsilon_C\colon \|C =_{\mathcal{U}} 2\| + \neg\|C =_{\mathcal{U}} 2\| \to (C \times C \to C)\\ \mathsf{inl}(t) &\mapsto \pi_2\\ \mathsf{inr}(\zeta) &\mapsto \pi_1.\end{aligned}$$

Consider the function $f :\equiv \lambda C.\, \epsilon_C(\ell(C))$.

- For $C :\equiv 2$, we have $f(2) = \pi_2$.
- For $C :\equiv 2\times 2$, its truncation $\|2\times 2 =_{\mathcal{U}} 2\|$ is empty because the type $(2\times 2 =_{\mathcal{U}} 2) \to 0$ is inhabited and $0$ is a mere proposition. Therefore, $f(2\times 2) = \pi_1$.

It follows that $f \neq \lambda C.\, \pi_1$ and $f \neq \lambda C.\, \pi_2$. Since $\pi_1, \pi_2\colon 2\times 2 \to 2$ are not equal either, the functions $f$, $\lambda C.\, \pi_1$, and $\lambda C.\, \pi_2$ yield three distinct elements in $\mathsf{W}'$.

□

**Exercise 5.13.17.** *Show that for any $A\colon \mathcal{U}$ and $B\colon A \to \mathcal{U}$, we have*

$$\neg \mathsf{W}_A B \simeq \neg\Sigma_A \neg B.$$

*In other words, $\mathsf{W}_A B$ is inhabited if, and only if, at least one constructor has an empty arity.*[6]

*Solution.* Let $\mathsf{W}$ denote the $\mathsf{W}$-type $\mathsf{W}_A B$. By Exercise 5.13.13, there is a map $\Sigma_A \neg B \to \mathsf{W}$. Applying the contravariant functoriality of $\neg$, we obtain a map $\neg\mathsf{W} \to \neg\Sigma_A \neg B$. More precisely, this function is defined by

$$\begin{aligned}f\colon \neg\mathsf{W} &\to \neg\Sigma_A \neg B\\ \epsilon &\mapsto \lambda\langle a, \zeta\rangle.\, \epsilon\big(\mathsf{sup}(a, \mathsf{rec}_0(\mathsf{W}) \circ \zeta)\big).\end{aligned}$$

For the backward function, we need to fix $\theta\colon \Sigma_A \neg B \to 0$ and produce a map $\mathsf{W} \to 0$. As established in Notation 5.4.13, it suffices to construct a map

$$\alpha_0\colon \prod_{a\colon A} \neg B(a) \to 0,$$

obtaining $\mathsf{hom}_{\mathsf{W}}(0, \alpha_0)$. To fulfill this requirement, we specify

$$\alpha_0 :\equiv \lambda a.\, \lambda\zeta.\, \theta(\langle a, \zeta\rangle).$$

[6] Compare to Exercise 5.13.11.

More precisely, we define

$$\begin{aligned} g &: \neg\Sigma_A \neg B \to \neg \mathsf{W} \\ &\qquad \theta \mapsto \mathsf{hom}_{\mathsf{W}}(\mathsf{0}, \alpha_0). \end{aligned}$$

To complete the proof, we have to verify that $f$ and $g$ are quasi-inverses:

$$\begin{aligned} g \circ f(\epsilon) &\equiv g\Big(\underbrace{\lambda\langle a, \zeta\rangle.\, \epsilon\big(\mathsf{sup}(a, \mathsf{rec}_{\mathsf{0}}(\mathsf{W}) \circ \zeta)\big)}_{\theta}\Big) \\ &\equiv \mathsf{hom}_{\mathsf{W}}\big(\mathsf{0}, \lambda a.\, \lambda\zeta.\, \epsilon\big(\mathsf{sup}(a, \mathsf{rec}_{\mathsf{0}}(\mathsf{W}) \circ \zeta)\big)\big) \\ &=_{\neg \mathsf{W}} \epsilon. \end{aligned}$$

On the other hand

$$\begin{aligned} f \circ g(\theta) &\equiv f\big(\underbrace{\mathsf{hom}_{\mathsf{W}}(\mathsf{0}, \alpha_0)}_{\epsilon}\big) \\ &\equiv \lambda\langle a, \zeta\rangle.\, \underbrace{\mathsf{hom}_{\mathsf{W}}(\mathsf{0}, \alpha_0)}_{\epsilon}\big(\mathsf{sup}(a, \mathsf{rec}_{\mathsf{0}}(\mathsf{W}) \circ \zeta)\big) \\ &\equiv \lambda\langle a, \zeta\rangle.\, \alpha_0\big(a, \mathsf{hom}_{\mathsf{W}}(\mathsf{0}, \alpha_0) \circ \mathsf{rec}_{\mathsf{0}}(\mathsf{W}) \circ \zeta\big) \\ &\equiv \lambda\langle a, \zeta\rangle.\, \theta\big(a, \underbrace{\mathsf{hom}_{\mathsf{W}}(\mathsf{0}, \alpha_0) \circ \mathsf{rec}_{\mathsf{0}}(\mathsf{W})}_{\mathsf{0} \to \mathsf{0}} \circ \zeta\big) \\ &=_{\neg\Sigma_A \neg B} \lambda\langle a, \zeta\rangle.\, \theta(\langle a, \zeta\rangle) \\ &\equiv \theta. \end{aligned}$$

where the final propositional equality holds trivially because the codomain of both functions is $\mathsf{0}$.

□

# Chapter 6

# Higher Inductive Types

## 6.1 Initial Algebras Revisited

**Theorem 6.1.1.** [Map Lifting Property] *Let $P\colon A \to \mathcal{U}$ be a type family over $A$ and $E :\equiv \sum_{x\colon A} P(x)$ its total space. Given a type $B$ and a map $f\colon B \to A$, there exists a function*

$$\mathsf{maplift}\colon \Big(\prod_{y\colon B} P(f(y))\Big) \to \sum_{\hat{f}\colon B\to E} \pi_1^E \circ \hat{f} \sim f$$

*satisfying, for every section $s\colon \prod_{y\colon B} P(f(y))$, the judgmental identities*

$$\pi_1(\mathsf{maplift}(s)) \equiv \lambda y.\, \langle f(y), s(y)\rangle \quad \textit{and} \quad \pi_2(\mathsf{maplift}(s)) \equiv \lambda y.\, \mathsf{refl}_{f(y)},$$

*where $\pi_1^E\colon E \to A$ is the first projection of the total space, and $\pi_1$ and $\pi_2$ denote the canonical projections of the target type of* maplift.

*Proof.* Given $s\colon \prod_{y\colon B} P(f(y))$, define

$$\begin{aligned} \hat{f}&\colon B \to E \\ &Y \mapsto \langle f(y), s(y)\rangle. \end{aligned}$$

This is well-typed because, for $y\colon B$, we have $f(y)\colon A$ and $s(y)\colon P(f(y))$, and produces the judgmentally commutative diagram

$$\begin{array}{ccc} & & E \\ & \overset{\hat{f}}{\nearrow} & \big\downarrow{\scriptstyle \pi_1} \\ B & \xrightarrow[f]{} & A \end{array}$$

By the definitional equality of the first projection on a dependent pair, we have

$$\pi_1^E(\hat{f}(y)) \equiv \pi_1^E(\langle f(y), s(y)\rangle) \equiv f(y).$$

Because these are judgmentally equal, the required homotopy $\eta$ between $\pi_1^E \circ \hat{f}$ and $f$ can be given by reflexivity at each point:

$$\eta(y) :\equiv \mathsf{refl}_{f(y)}.$$

Thus, we can define

$$\mathsf{maplift}(s) :\equiv \langle \hat{f}, \eta\rangle.$$

By construction, we have

$$\pi_1(\mathsf{maplift}(s)) \equiv \hat{f} \equiv \lambda y.\, \langle f(y), s(y)\rangle$$

and

$$\pi_2(\mathsf{maplift}(s)) \equiv \eta \equiv \lambda y.\, \mathsf{refl}_{f(y)}.$$

□

**Remark 6.1.2.** Since the theorem's proof actually establishes a strict, judgmental equality $\pi_1^E \circ \hat{f} \equiv f$ rather than a simple homotopy, one might wonder why the statement claims a weaker conclusion.

This is necessary because judgmental equalities are meta-theoretic relations used by the formal system; they do not produce any type in the universe $\mathcal{U}$.

An expression such as $\sum_{\hat{f}:\, B \to E} \pi_1^E \circ \hat{f} \equiv f$ is syntactically ill-formed. The homotopy instead acts as a certificate of commutativity that is visible to the internal logic of the theory and can be invoked in subsequent theorems.

**Example 6.1.3.** The Map Lifting Property bridges the type-theoretic presentation of the induction principle with its categorical interpretation as an initial algebra.

The induction principle for the natural numbers $\mathbb{N}$ with respect to a type family $P\colon \mathbb{N} \to \mathcal{U}$ requires two pieces of data:

a) A base case element $b\colon P(0)$.

b) A step function $s\colon \prod_{n:\,\mathbb{N}} P(n) \to P(\mathsf{succ}(n))$.

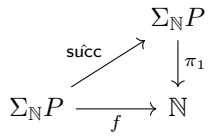
We can reinterpret this data by lifting the constructor maps to the total space $\Sigma_{\mathbb{N}} P$. Following the theorem, we define the base map

$$\begin{aligned} f\colon \underbrace{\textstyle\sum_{\mathbb{N}} P}_{B} &\to \underbrace{\mathbb{N}}_{A} \\ \langle n, u\rangle &\mapsto \mathsf{succ}(n). \end{aligned}$$

The uncurried step function $s$ assigns to every pair $\langle n, u\rangle \colon \Sigma_{\mathbb{N}} P$ an element in the fiber $P(\mathsf{succ}(n))$, which is exactly $P(f(\langle n, u\rangle))$. Therefore, the step function can be seen as a section

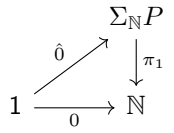
$$s \colon \prod_{e \colon \Sigma_{\mathbb{N}} P} P(f(e)).$$

By applying the Map Lifting Property, this section $s$ lifts the map $f$ to an endomorphism of the total space bundle, yielding a new map $\hat{f} \colon \Sigma_{\mathbb{N}} P \to \Sigma_{\mathbb{N}} P$. Denoting $\hat{f}$ by $\widehat{\mathsf{succ}}$ to stress the categorical interpretation, we obtain the propositionally commutative triangle




where the lifted map sends every pair $\langle n, u\rangle$ to the pair $\big\langle \mathsf{succ}(n), s(\langle n, u\rangle)\big\rangle$.

Similarly, we can apply the Map Lifting Property to the base constructor: the constant map $0 \colon 1 \to \mathbb{N}$. The base element $b \colon P(0)$ acts as a section over this map. Lifting it yields a point in the total space bundle given by the map $\hat{0} \colon 1 \to \Sigma_{\mathbb{N}} P$, which selects the pair $\langle 0, b\rangle$.

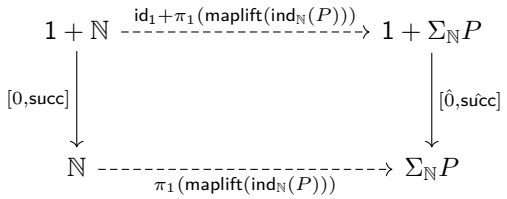



Categorically, this demonstrates that both the base space $\mathbb{N}$ and the total space $\Sigma_{\mathbb{N}} P$ are algebras for the polynomial functor $F(X) :\equiv 1 + X$.

The induction principle of $\mathbb{N}$ is equivalent to the statement that $\mathbb{N}$ is the initial algebra in this category: there exists a unique algebra homomorphism $\mathbb{N} \to \Sigma_{\mathbb{N}} P$ commuting with the constructors. This unique homomorphism is exactly the global dependent section

$$\mathsf{ind}_{\mathbb{N}}(P) \colon \prod_{n \colon \mathbb{N}} P(n).$$

Here is the commutative diagram illustrating this initial algebra homomorphism:




The commutativity of this square translates categorical properties into computation rules for the induction principle.

## 6.2 Induction Principles and Dependent Paths

Like the general inductive types, higher inductive types provide a schema for defining new types generated by constructors. The difference is that higher inductive types may have constructors that produce paths and higher paths. For instance, the constructors of the circle $\mathbb{S}^1$ are:

- $\mathsf{base}\colon \mathbb{S}^1$,
- $\mathsf{loop}\colon \mathsf{base} =_{\mathbb{S}^1} \mathsf{base}$.

This is analogous to the definition of $\mathbb{N}$, whose constructors are $0\colon \mathbb{N}$ and $\mathsf{succ}\colon \mathbb{N} \to \mathbb{N}$. In the circle, however, the generation extends to higher dimensions, which is natural when types are seen as higher groupoids.

Ordinary constructors, such as $\mathsf{base}$, are referred as *point constructors*; the others, such as $\mathsf{loop}$, as *path* or *higher constructors*.

Each path constructor must specify its source and target; for $\mathsf{loop}$, both are $\mathsf{base}$. A path constructor generates a new inhabitant of an identity type, which is not *a priori* equal to any existing inhabitant. This distinguishes $\mathbb{S}^1$ from the inductive type $\mathbf{1}$.

There are some important points regarding this generalization:

- The operations may create paths that do not come directly from the constructors. For example, in $\mathbb{S}^1$, we also obtain nontrivial paths like $\mathsf{loop}\cdot\mathsf{loop}$ and $\mathsf{loop}^{-1}$.
- To make two paths equal up to homotopy, the inductive definition must add a new 2-dimensional constructor relating them. This ensures that the propositional equality between paths comes equipped with the necessary elimination and computation rules.
- Constructors without higher types in their inputs can generate higher paths. For instance, the 2-dimensional sphere $\mathbb{S}^2$, generated by a point $\mathsf{base}\colon \mathbb{S}^2$ and a 2-dimensional path $\mathsf{surf}\colon \mathsf{refl}_{\mathsf{base}} = \mathsf{refl}_{\mathsf{base}}$, contains a nontrivial 3-dimensional path corresponding to the Hopf fibration.

## 6.3 Elimination Rules

To define operations on the circle $\mathbb{S}^1$, we must specify their behavior on the fibers above each constructor. We begin by defining the notion of a dependent path, which formalizes what it means for a path to lie over the loop constructor.

**Definition 6.3.1.** Let $A$ be a type and $P\colon A \to \mathcal{U}$ a type family. A dependent path from $u\colon P(x)$ to $v\colon P(y)$ lying over a path $p\colon x =_A y$ is defined as

$$u =_p^P v :\equiv \mathsf{transport}^P(p, u) =_{P(y)} v.$$

**Proposition 6.3.2.** *Let $P\colon A \to \mathcal{U}$ be a type family. The following properties of dependent paths hold.*

a) Reduction: *For any $x\colon A$ and $u, w\colon P(x)$,*

$$(u =^{P}_{\mathsf{refl}_x} w) \equiv (u =_{P(x)} w).$$

b) Reflexivity: *For any $x\colon A$ and $u\colon P(x)$,*

$$\mathsf{refl}_u\colon u =^{P}_{\mathsf{refl}_x} u.$$

c) Symmetry: *For any $p\colon x =_A y$, $u\colon P(x)$, $v\colon P(y)$, and*

$$\ell\colon u =^{P}_{p} v,$$

*there is a dependent path*

$$\ell^{-1}\colon v =^{P}_{p^{-1}} u.$$

d) Transitivity: *For any $p\colon x =_A y$ and $q\colon y =_A z$, if*

$$\ell_1\colon u =^{P}_{p} v \qquad \text{and} \qquad \ell_2\colon v =^{P}_{q} w,$$

*then*

$$\ell_1 \cdot \ell_2\colon u =^{P}_{p\cdot q} w.$$

e) Fiberwise concatenation: *For any $p\colon x =_A y$, $u\colon P(x)$, $v, w\colon P(y)$, if*

$$\ell\colon u =^{P}_{p} v \qquad \text{and} \qquad r\colon v =_{P(y)} w,$$

*then*

$$\ell \cdot r\colon u =^{P}_{p} w.$$

*Proof.*

a) *Reduction:* By definition,

$$(u =^{P}_{\mathsf{refl}_x} w) \equiv \Big(\mathsf{transport}^{P}(\mathsf{refl}_x, u) =_{P(x)} w\Big).$$

Since

$$\mathsf{transport}^{P}(\mathsf{refl}_x, u) \equiv u,$$

the desired definitional equality follows.

b) *Reflexivity:* By (a), the type

$$u =^{P}_{\mathsf{refl}_x} u$$

is definitionally equal to $u =_{P(x)} u$. Hence ordinary reflexivity gives

$$\mathsf{refl}_u\colon u =^{P}_{\mathsf{refl}_x} u.$$

c) *Symmetry:* By path induction on $p$, we may assume that

$$y \equiv x \qquad \text{and} \qquad p \equiv \mathsf{refl}_x.$$

Thus $v\colon P(x)$, and

$$\ell\colon u =^P_{\mathsf{refl}_x} v.$$

By (a), this type is definitionally equal to $u =_{P(x)} v$. Moreover,

$$p^{-1} \equiv \mathsf{refl}_x^{-1} \equiv \mathsf{refl}_x.$$

Hence the required type

$$v =^P_{p^{-1}} u$$

is definitionally equal to $v =_{P(x)} u$, which is inhabited by the ordinary inverse $\ell^{-1}$.

d) *Transitivity:* By path induction on $p$ and then on $q$, we may assume that

$$y \equiv x, \qquad z \equiv x, \qquad p \equiv \mathsf{refl}_x, \qquad q \equiv \mathsf{refl}_x.$$

Consequently,

$$\ell_1\colon u =_{P(x)} v \qquad \text{and} \qquad \ell_2\colon v =_{P(x)} w$$

by (a). Also,

$$p \cdot q \equiv \mathsf{refl}_x.$$

Therefore the target

$$u =^P_{p\cdot q} w$$

reduces, by (a), to $u =_{P(x)} w$, which is inhabited by the ordinary concatenation $\ell_1 \cdot \ell_2$.

e) *Fiberwise concatenation:* By path induction on $p$, we may assume that

$$y \equiv x \qquad \text{and} \qquad p \equiv \mathsf{refl}_x.$$

In this case, both $v$ and $w$ are elements of $P(x)$, and by (a),

$$\ell\colon u =_{P(x)} v.$$

Thus the ordinary concatenation $\ell \cdot r$ has type $u =_{P(x)} w$. By (a), this is definitionally equal to

$$u =^P_{\mathsf{refl}_x} w,$$

which is the required target.

□

With Definition 6.3.1, the fundamental elimination rule for the circle is given by its induction principle:

INDUCTION: Given a type family $P\colon \mathbb{S}^1 \to \mathcal{U}$, along with the following data:

- $b\colon P(\mathsf{base})$.
- $\ell\colon b =^P_{\mathsf{loop}} b$,

there exists a dependent function

$$\mathsf{ind}_{\mathbb{S}^1}(P)\colon \prod_{x\colon \mathbb{S}^1} P(x)$$

satisfying the computation rules:[1]

$$\mathsf{ind}_{\mathbb{S}^1}(P, \mathsf{base}) \equiv b \quad \text{and} \quad \mathsf{apd}_{\mathsf{ind}_{\mathbb{S}^1}(P)}(\mathsf{loop}) :\equiv \ell.$$

**Remark 6.3.3.** In the computation rule above, the evaluation at the point constructor is a strict judgmental equality. In contrast, the evaluation at the path constructor is a propositional equality. This *propositional equality by definition* is denoted using the hybridized symbol ':='.

The recursion principle is the non-dependent version of the induction principle when the type family is constant:

RECURSION: Given a type $B$ equipped with a point $b\colon B$ and a path $\ell\colon b =_B b$, there exists a function $\mathsf{rec}_{\mathbb{S}^1}(B, b, \ell)\colon \mathbb{S}^1 \to B$ satisfying the computation rules:

$$\mathsf{rec}_{\mathbb{S}^1}(B, b, \ell)(\mathsf{base}) \equiv b \quad \text{and} \quad \mathsf{ap}_{\mathsf{rec}_{\mathbb{S}^1}(B,b,\ell)}(\mathsf{loop}) :\equiv \ell.$$

The following lemma formalizes the derivation of the recursion principle directly from the induction principle.

**Lemma 6.3.4.** *If $B$ is a type together with $b\colon B$ and $p\colon b =_B b$, then there is a function $f\colon \mathbb{S}^1 \to B$ with*

a) $f(\mathsf{base}) \equiv b$, *and*

b) $\mathsf{ap}_f(\mathsf{loop}) :\equiv p$.

*Proof.* By Lemma 2.3.7, we have

$$\begin{aligned}\mathsf{transportconst}^B(\mathsf{loop}, b)\colon \mathsf{transport}^{x\mapsto B}(\mathsf{loop}, b) &=_B b\\ &\equiv b =^{x\mapsto B}_{\mathsf{loop}} b.\end{aligned}$$

Let

$$\ell :\equiv \mathsf{transportconst}^B(\mathsf{loop}, b).$$

From Proposition 6.3.2 e) we deduce

$$\ell \cdot p\colon b =^{x\mapsto B}_{\mathsf{loop}} b.$$

[1] See Lemma 2.3.6

Applying the induction principle of $\mathbb{S}^1$ to the constant type family $x \mapsto B$, we obtain a function

$$f\colon \mathbb{S}^1 \to B$$

such that

$$f(\mathsf{base}) \equiv b \quad \text{and} \quad \mathsf{apd}_f(\mathsf{loop}) \coloneqq \ell \cdot p.$$

Furthermore,

$$\begin{aligned} \mathsf{apd}_f(\mathsf{loop}) &= \mathsf{transportconst}^B(\mathsf{loop}, f(\mathsf{base})) \cdot \mathsf{ap}_f(\mathsf{loop}) && ;\ \text{Lem 2.3.8} \\ &= \mathsf{transportconst}^B(\mathsf{loop}, b) \cdot \mathsf{ap}_f(\mathsf{loop}) && ;\ f(\mathsf{base}) \equiv b \\ &\equiv \ell \cdot \mathsf{ap}_f(\mathsf{loop}). \end{aligned}$$

Hence,

$$\ell \cdot \mathsf{ap}_f(\mathsf{loop}) = \ell \cdot p.$$

The conclusion follows by left cancellation of $\ell$.

□

**Remark 6.3.5.** Suppose that $h, k\colon A \to B$ are two functions. Then, if $P\colon A \to \mathcal{U}$ is defined as $P(x) :\equiv h(x) =_B k(x)$ for $x\colon A$, Lemmas 2.14.2 and 2.14.4 b) ii) (or Exercise 2.19.5) imply

$$(u =^P_\gamma v) \simeq \big(\mathsf{ap}_h(\gamma)^{-1} \cdot u \cdot \mathsf{ap}_k(\gamma) =_B v\big).$$

**Lemma 6.3.6.** *If $A$ is a type and $f, g\colon \mathbb{S}^1 \to A$ are two maps together with two equalities $p, q$:*

$$\begin{aligned} &p\colon f(\mathsf{base}) =_A g(\mathsf{base}) \\ &q\colon \mathsf{ap}_f(\mathsf{loop}) =^{x \mapsto x =_A x}_p \mathsf{ap}_g(\mathsf{loop}), \end{aligned}$$

*then for all $z\colon \mathbb{S}^1$ we have $f(z) =_A g(z)$.*

*Proof.* Let $P\colon \mathbb{S}^1 \to \mathcal{U}$ be the type family defined by $P(z) :\equiv f(z) =_A g(z)$. Then, $p\colon P(\mathsf{base})$. On the other hand, by Remark 6.3.5, we have

$$(p =^P_{\mathsf{loop}} p) = \big(\mathsf{ap}_f(\mathsf{loop})^{-1} \cdot p \cdot \mathsf{ap}_g(\mathsf{loop}) = p\big) \tag{6.1}$$

By Proposition 2.11.6, we have

$$\mathsf{transport}^{x \mapsto (x =_A x)}(p, \mathsf{ap}_f(\mathsf{loop})) = p^{-1} \cdot \mathsf{ap}_f(\mathsf{loop}) \cdot p.$$

By definition, $q$ witnesses the equality

$$q\colon \mathsf{transport}^{x \mapsto (x =_A x)}(p, \mathsf{ap}_f(\mathsf{loop})) = \mathsf{ap}_g(\mathsf{loop}),$$

producing an inhabitant of

$$\mathsf{ap}_g(\mathsf{loop}) = p^{-1} \cdot \mathsf{ap}_f(\mathsf{loop}) \cdot p.$$

Hence,

$$p \cdot \mathsf{ap}_g(\mathsf{loop}) = \mathsf{ap}_f(\mathsf{loop}) \cdot p.$$

Left-concatenating with $\mathsf{ap}_f(\mathsf{loop})^{-1}$ gives a proof of

$$\mathsf{ap}_f(\mathsf{loop})^{-1} \cdot p \cdot \mathsf{ap}_g(\mathsf{loop}) = p.$$

Plugging this proof into (6.1), we obtain an element of $p =^P_{\mathsf{loop}} p$. Then, the induction principle of the circle yields a section of $P$, proving that $f(z) =_A g(z)$ for all $z : \mathbb{S}^1$.

□

**Theorem 6.3.7.** *For any type $A$, we have a natural equivalence*

$$(\mathbb{S}^1 \to A) \simeq \sum_{x:\,A} x =_A x.$$

*Proof.* We have two canonical maps

$$\begin{aligned} f : (\mathbb{S}^1 \to A) &\to \sum_{x:\,A} x =_A x \\ \rho &\mapsto \langle \rho(\mathsf{base}), \mathsf{ap}_\rho(\mathsf{loop}) \rangle \end{aligned}$$

$$\begin{aligned} g : \Big(\sum_{x:\,A} x =_A x\Big) &\to (\mathbb{S}^1 \to A) \\ \langle a, \ell \rangle &\mapsto \mathsf{rec}_{\mathbb{S}^1}(A, a, \ell). \end{aligned}$$

To prove that they define an equivalence, we evaluate both compositions:

$$\begin{aligned} f \circ g(\langle a, \ell \rangle) &\equiv f\big(\mathsf{rec}_{\mathbb{S}^1}(A, a, \ell)\big) \\ &\equiv \big\langle \mathsf{rec}_{\mathbb{S}^1}(A, a, \ell)(\mathsf{base}), \mathsf{ap}_{\mathsf{rec}_{\mathbb{S}^1}(A,a,\ell)}(\mathsf{loop}) \big\rangle \\ &= \langle a, \ell \rangle, \end{aligned}$$

by the computation rules of $\mathsf{rec}_{\mathbb{S}^1}$.

$$\begin{aligned} g \circ f(\rho) &\equiv g(\langle \rho(\mathsf{base}), \mathsf{ap}_\rho(\mathsf{loop}) \rangle) \\ &\equiv \mathsf{rec}_{\mathbb{S}^1}(A, \rho(\mathsf{base}), \mathsf{ap}_\rho(\mathsf{loop})) \\ &\sim \rho, \end{aligned}$$

by Lemma 6.3.6. The equality follows from function extensionality.

Regarding naturality, suppose that $\phi : A \to B$. Then, $\phi$ induces a commutative diagram because, by Proposition 2.1.1, $\mathsf{ap}_\phi \circ \mathsf{ap}_\rho = \mathsf{ap}_{\phi \circ \rho}$:

$$\begin{array}{ccc} \mathbb{S}^1 \to A & \longrightarrow & \sum_{x:\,A} x =_A x \\ \big\downarrow & & \big\downarrow \\ \mathbb{S}^1 \to B & \longrightarrow & \sum_{y:\,B} y =_B y \end{array} \qquad \begin{array}{ccc} \rho & \longmapsto & \langle \rho(\mathsf{base}), \mathsf{ap}_\rho(\mathsf{loop}) \rangle \\ \big\downarrow & & \big\downarrow \\ \phi \circ \rho & \longmapsto & \langle \phi(\rho(\mathsf{base})), \mathsf{ap}_\phi(\mathsf{ap}_\rho(\mathsf{loop})) \rangle \end{array}$$

□

## 6.4 The Interval

The type *interval*, denoted by I, is defined inductively as

- $0_{\mathrm{I}}\colon \mathrm{I}$,
- $1_{\mathrm{I}}\colon \mathrm{I}$,
- $\mathsf{seg}\colon 0_{\mathrm{I}} =_{\mathrm{I}} 1_{\mathrm{I}}$.

The recursion principle for the interval says that given a type $B$ along with

- a point $b_0\colon B$,
- a point $b_1\colon B$, and
- a path $s\colon b_0 =_B b_1$,

there is a function

$$\mathsf{rec}_{\mathrm{I}}(B)\colon \mathrm{I} \to B$$

such that

$$f(0_{\mathrm{I}}) \equiv b_0, \quad f(1_{\mathrm{I}}) \equiv b_1, \quad \text{and} \quad \mathsf{ap}_f(\mathsf{seg}) \coloneqq s.$$

The induction principle says that given $P\colon \mathrm{I} \to \mathcal{U}$ along with

- a point $b_0\colon P(0_{\mathrm{I}})$,
- a point $b_1\colon P(1_{\mathrm{I}})$, and
- a path $s\colon b_0 =^P_{\mathsf{seg}} b_1$,

there is a function

$$\mathsf{ind}_{\mathrm{I}}(P)\colon \prod_{x\colon \mathrm{I}} P(x)$$

such that

$$\mathsf{ind}_{\mathrm{I}}(P)(0_{\mathrm{I}}) \equiv b_0, \quad \mathsf{ind}_{\mathrm{I}}(P)(1_{\mathrm{I}}) \equiv b_1, \quad \text{and} \quad \mathsf{apd}_{\mathsf{ind}_{\mathrm{I}}(P)}(\mathsf{seg}) \coloneqq s.$$

**Proposition 6.4.1.** *The type* I *is contractible.*

*Proof.* We choose $1_{\mathrm{I}}$ as the center of contraction. We must find a contraction map $f\colon \prod_{x\colon \mathrm{I}} x =_{\mathrm{I}} 1_{\mathrm{I}}$. To define this by induction on $x$ we must provide:

- a point $b_0\colon 0_{\mathrm{I}} =_{\mathrm{I}} 1_{\mathrm{I}}$,
- a point $b_1\colon 1_{\mathrm{I}} =_{\mathrm{I}} 1_{\mathrm{I}}$, and
- a dependent path $s\colon \mathsf{seg} =^{x \mapsto x =_{\mathrm{I}} 1_{\mathrm{I}}}_{\mathsf{seg}} \mathsf{refl}_{1_{\mathrm{I}}}$.

We can choose $b_0 :\equiv \mathsf{seg}$ and $b_1 :\equiv \mathsf{refl}_{1_\mathrm{I}}$. Therefore, by Proposition 2.11.6, the type of $s$ is equivalent to

$$\mathsf{seg}^{-1} \cdot \mathsf{seg} =_{1_\mathrm{I} =_\mathrm{I} 1_\mathrm{I}} \mathsf{refl}_{1_\mathrm{I}},$$

which is trivially inhabited after path cancellation. Therefore, the required dependent path $s$ exists, completing the definition of $f$. □

**Theorem 6.4.2.** [Function Extensionality] *The recursion principle for the interval implies function extensionality.*

*Proof.* Let $f, g\colon A \to B$ be two functions such that $f(x) =_B g(x)$ for every $x\colon A$. We have to prove that $f =_{A\to B} g$.

Let $\eta\colon \prod_{x:A} f(x) =_B g(x)$ be the given pointwise equality. For each $x\colon A$, we apply the recursion principle of the interval to the type $B$, the points $f(x)$ and $g(x)$, and the path $\eta(x)$. This yields a function $h_x\colon \mathrm{I} \to B$ such that $h_x(0_\mathrm{I}) \equiv f(x)$, $h_x(1_\mathrm{I}) \equiv g(x)$, and $\mathsf{ap}_{h_x}(\mathsf{seg}) =_{f(x)=_B g(x)} \eta(x)$.

Let $\epsilon\colon \mathrm{I} \to (A \to B)$ be defined by

$$\epsilon(i) :\equiv \lambda x.\, h_x(i).$$

Evaluating $\epsilon$ at the endpoints of I, we obtain

$$\epsilon(0_\mathrm{I}) \equiv (\lambda x.\, h_x(0_\mathrm{I})) \equiv (\lambda x.\, f(x)) \equiv f$$

and

$$\epsilon(1_\mathrm{I}) \equiv (\lambda x.\, h_x(1_\mathrm{I})) \equiv (\lambda x.\, g(x)) \equiv g.$$

Consequently,

$$\mathsf{ap}_\epsilon(\mathsf{seg})\colon (\epsilon(0_\mathrm{I}) =_{A\to B} \epsilon(1_\mathrm{I})) \equiv (f =_{A\to B} g).$$

□

## 6.5 Circles & Spheres

**Lemma 6.5.1.** *The circle* $\mathbb{S}^1$ *satisfies* $\mathsf{loop} \neq \mathsf{refl}_{base}$. *In particular, is not equivalent to the unit type* $\mathsf{1}$.

*Proof.* We must construct a map

$$\mathsf{loop} =_{\mathsf{base}=_{\mathbb{S}^1}\mathsf{base}} \mathsf{refl}_{\mathsf{base}} \to \mathsf{0}.$$

Let $\zeta\colon \mathsf{loop} =_{\mathsf{base}=_{\mathbb{S}^1}\mathsf{base}} \mathsf{refl}_{\mathsf{base}}$. For any $A\colon \mathcal{U}$, any $a\colon A$, and any $\ell\colon a =_A a$, consider the function

$$f_a :\equiv \mathsf{rec}_{\mathbb{S}^1}(A, a, \ell)\colon \mathbb{S}^1 \to A.$$

By definition, $f_a(\mathsf{base}) \equiv a$ and $\mathsf{ap}_{f_a}(\mathsf{loop}) =_{a=_A a} \ell$. It follows that

$$\mathsf{ap}_{\mathsf{ap}_{f_a}}(\zeta) \colon \mathsf{ap}_{f_a}(\mathsf{loop}) =_{a=_A a} \mathsf{ap}_{f_a}(\mathsf{refl}_{\mathsf{base}}).$$

Since $\mathsf{ap}_{f_a}(\mathsf{refl}_{\mathsf{base}}) \equiv \mathsf{refl}_{f_a(\mathsf{base})} \equiv \mathsf{refl}_a$, we obtain a witness of $\ell =_{a=_A a} \mathsf{refl}_a$.

As a consequence, given two paths $p, q \colon a =_A x$, because

$$p \cdot q^{-1} \colon a =_A a,$$

we deduce that $p \cdot q^{-1} =_{a=_A a} \mathsf{refl}_a$, which implies $p =_{a=_A x} q$.

Thus, $A$ would be a Set. Since the same reasoning can be applied to a higher universe $\mathcal{U}'$, we would deduce that $\mathcal{U} : \mathcal{U}'$ is a Set. However, Example 3.1.6 yields a contradiction. □

**Remark 6.5.2.** The application of the recursion principle for the circle to a target type $A : \mathcal{U}'$ is justified by 33. Universe Polymorphism. Although the elimination rule is commonly stated for a target type in $\mathcal{U}$, it permits the standard recursor $\mathsf{rec}_{\mathbb{S}^1}$ to eliminate into types residing in any universe above $\mathcal{U}$, without the need of a separate definition of the circle at each universe level.

**Corollary 6.5.3.** *The type $\mathbb{S}^1 =_{\mathcal{U}} \mathbb{S}^1$ is not a set.*

*Proof.* By univalence, it suffices to prove that $\mathbb{S}^1 \simeq \mathbb{S}^1$ is not a set. By Definition 2.5.7,

$$\mathbb{S}^1 \simeq \mathbb{S}^1 \equiv \sum_{f : \mathbb{S}^1 \to \mathbb{S}^1} \mathsf{isequiv}(f).$$

Because

$$\langle \mathsf{id}_{\mathbb{S}^1}, \mathsf{isequiv}(\mathsf{id}_{\mathbb{S}^1}) \rangle \colon \sum_{f : \mathbb{S}^1 \to \mathbb{S}^1} \mathsf{isequiv}(f),$$

it suffices to show that $\mathsf{id}_{\mathbb{S}^1} =_{\mathbb{S}^1 \to \mathbb{S}^1} \mathsf{id}_{\mathbb{S}^1}$ is not a mere proposition.

By function extensionality, this identity type is equivalent to $\mathsf{id}_{\mathbb{S}^1} \sim \mathsf{id}_{\mathbb{S}^1}$. Since $\lambda x. \mathsf{refl}_x$ is a homotopy from $\mathsf{id}_{\mathbb{S}^1}$ to itself, it suffices to construct another homotopy of this type distinct from this one.

Let $P \colon \mathbb{S}^1 \to \mathcal{U}$ be the type family defined by $P(x) :\equiv x =_{\mathbb{S}^1} x$. We define a dependent function

$$\eta \colon \prod_{x : \mathbb{S}^1} x =_{\mathbb{S}^1} x$$

by circle induction. We choose $\mathsf{loop} \colon P(\mathsf{base})$ as the base point. To complete the definition, we need a dependent path $\ell \colon \mathsf{loop} =^P_{\mathsf{loop}} \mathsf{loop}$. By Proposition 2.11.6,

$$\mathsf{transport}^P(\mathsf{loop}, \mathsf{loop}) = \mathsf{loop}^{-1} \cdot \mathsf{loop} \cdot \mathsf{loop} = \mathsf{loop}.$$

Let $\ell$ denote the witness of this equality. We can then define

$$\eta :\equiv \mathsf{ind}_{\mathbb{S}^1}(P, \mathsf{loop}, \ell).$$

By the computation rule, $\eta(\mathsf{base}) \equiv \mathsf{loop}$. Since $\mathsf{loop} \neq \mathsf{refl}_{\mathsf{base}}$, we conclude that $\eta \neq \lambda x.\, \mathsf{refl}_x$, as required. □

**Lemma 6.5.4.** *Given $f\colon A \to B$, $x, y\colon A$, $p, q\colon x =_A y$, and $\gamma\colon p =_{x=_A y} q$, we have a path*

$$\mathsf{ap}^2_f(\gamma)\colon \mathsf{ap}_f(p) =_{f(x)=_B f(y)} \mathsf{ap}_f(q).$$

*Proof.* By induction on $\gamma$ it suffices to assume $q \equiv p$ and $\gamma \equiv \mathsf{refl}_p$. Thus, our goal reduces to $\mathsf{ap}_f(p) =_{f(x)=_B f(y)} \mathsf{ap}_f(p)$, which is trivially inhabited by $\mathsf{ap}^2_f(\mathsf{refl}_p) :\equiv \mathsf{refl}_{\mathsf{ap}_f(p)}$. □

**Lemma 6.5.5.** *Given $P\colon A \to \mathcal{U}$, $x, y\colon A$, $p, q\colon x =_A y$, and $\gamma\colon p =_{x=_A y} q$, for any $u\colon P(x)$ we have a path*

$$\mathsf{transport}^{2P}(\gamma, u)\colon \mathsf{transport}^P(p, u) =_{P(y)} \mathsf{transport}^P(q, u).$$

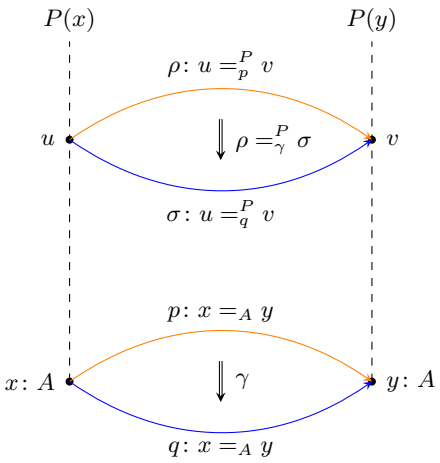
*Proof.* By induction on $\gamma$ we obtain our goal and the definitional identity

$$\mathsf{transport}^{2P}(\mathsf{refl}_p, u) \equiv \mathsf{refl}_{\mathsf{transport}^P(p,u)}.$$ □

**Definition 6.5.6.** Let $P\colon A \to \mathcal{U}$ be a type family, $x, y\colon A$, $p, q\colon x =_A y$, and $\gamma\colon p =_{x=_A y} q$. Given points $u\colon P(x)$ and $v\colon P(y)$, along with dependent paths $\rho\colon u =^P_p v$ and $\sigma\colon u =^P_q v$, we define the dependent 2-path over $\gamma$ as

$$\rho =^P_\gamma \sigma :\equiv \big(\rho = \mathsf{transport}^{2P}(\gamma, u) \cdot \sigma\big).$$

**Remark 6.5.7.** The elements of the definition above can be illustrated as follows:



The term

$$\mathsf{transport}^{2P}(\gamma, u)$$

is the path in the fiber $P(y)$ from

$$\mathsf{transport}^{P}(p, u) \quad \text{to} \quad \mathsf{transport}^{P}(q, u).$$

Thus, the definition says that the dependent 2-path

$$\rho =^{P}_{\gamma} \sigma$$

witnesses the equality between $\rho$ and the concatenation

$$\mathsf{transport}^{2P}(\gamma, u) \cdot \sigma$$

in the fiber $P(y)$.

**Lemma 6.5.8.** *Given $P\colon A \to \mathcal{U}$, $x, y\colon A$, $p, q\colon x =_A y$, $\gamma\colon p =_{x=_A y} q$, and a function $f\colon \prod_{x\colon A} P(x)$, we have a dependent 2-path*

$$\mathsf{apd}^2_f(\gamma)\colon \mathsf{apd}_f(p) =^{P}_{\gamma} \mathsf{apd}_f(q).$$

*Proof.* By path induction on the 2-path $\gamma$, we only need to address the case where $q \equiv p$ and $\gamma \equiv \mathsf{refl}_p$. In this case, the target type becomes

$$\mathsf{apd}_f(p) =^{P}_{\mathsf{refl}_p} \mathsf{apd}_f(p),$$

which is definitionally equal to

$$\mathsf{apd}_f(p) = \mathsf{apd}_f(p).$$

This identity type is trivially inhabited by $\mathsf{refl}_{\mathsf{apd}_f(p)}$. □

**Definition 6.5.9.** The 2-sphere $\mathbb{S}^2$ is the higher inductive type generated by

- $\mathsf{base}\colon \mathbb{S}^2$ and
- $\mathsf{surf}\colon \mathsf{refl}_{\mathsf{base}} =_{\mathsf{base}=_{\mathbb{S}^2}\mathsf{base}} \mathsf{refl}_{\mathsf{base}}$.

**Remark 6.5.10.** Think of a 1-path as a string from one point to another, and a 2-path as the surface swept out as one string continuously deforms into another. Because $\mathsf{refl}_{\mathsf{base}}$ is a constant path of zero length that remains stationary at the base point, a 2-path from $\mathsf{refl}_{\mathsf{base}}$ to itself is a surface whose entire outer boundary is collapsed into that single point.

The recursion principle for $\mathbb{S}^2$ says that given $B$ with

- $b\colon B$ and
- $s\colon \mathsf{refl}_b =_{b=_B b} \mathsf{refl}_b$,

we have

$$\mathsf{rec}_{\mathbb{S}^2}(B, b, s) \colon \mathbb{S}^2 \to B$$

with

$$\mathsf{rec}_{\mathbb{S}^2}(B, b, s)(\mathsf{base}) \equiv b \quad \text{and} \quad \mathsf{ap}^2_{\mathsf{rec}_{\mathbb{S}^2}(P,b,s)}(\mathsf{surf}) = s.$$

The induction principle for $\mathbb{S}^2$ says that, given $P \colon \mathbb{S}^2 \to \mathcal{U}$ with

- $b \colon P(\mathsf{base})$ and
- $s \colon \mathsf{refl}_b =^Q_{\mathsf{surf}} \mathsf{refl}_b$,

where $Q :\equiv \lambda p.\, b =^P_p b$, there is a function

$$\mathsf{ind}_{\mathbb{S}^2}(P, b, s) \colon \prod_{x \colon \mathbb{S}^2} P(x)$$

such that

$$\mathsf{ind}_{\mathbb{S}^2}(P, b, s)(\mathsf{base}) \equiv b \quad \text{and} \quad \mathsf{apd}^2_{\mathsf{ind}_{\mathbb{S}^2}(P,b,s)}(\mathsf{surf}) = s.$$

### 6.5.1 Higher Dependent Loops

**Definition 6.5.11.** Let $A \colon \mathcal{U}$, $a \colon A$, and $P \colon A \to \mathcal{U}$. Given a term $p$ of the $n$th loop space $\pi_1(\Omega^n(\langle A, a\rangle))$, we define:

- for $u \colon P(a)$, the type $\Omega^n_a(P, u, p)$ of *dependent $n$-loops* over $p$,
- the *trivial dependent loop*

$$d_n \colon \prod_{u \colon P(a)} \Omega^n_a(P, u, c_n), \tag{1}$$

  where $c_n :\equiv \pi_2(\Omega^n(\langle A, a\rangle))$ is the trivial ordinary $n$-loop,[2] and
- the *action* $\mathsf{apd}^n_f(p)$ *on $n$-loops* of a dependent function $f \colon \prod_{x \colon A} P(x)$, with

$$\mathsf{apd}^n_f \colon \prod_{p \colon \pi_1(\Omega^n(\langle A,a\rangle))} \Omega^n_a(P, f(a), p), \tag{2}$$

  satisfying the invariant

$$\mathsf{apd}^n_f(c_n) \equiv d_n(f(a)), \tag{3}$$

by induction on $n \colon \mathbb{N}$ as follows:

Base Case ($n = 0$): Since $\Omega^0(\langle A, a\rangle) \equiv \langle A, a\rangle$, the term $p$ reduces to an element $x \colon A$. Then,

- $\Omega^0_a(P, u, x) :\equiv P(x)$.

[2] See Remark 2.2.4

- $d_0(u) :\equiv u$, $c_0 :\equiv a$.
- $\mathsf{apd}^0_f(x) :\equiv f(x)$.

Conditions (1), (2), and (3) are trivially satisfied:

(1) $d_0(u) \equiv u \colon P(a) \equiv \Omega^0_a(P, u, a) \equiv \Omega^0_a(P, u, c_0)$.

(2) $\mathsf{apd}^0_f(x) \equiv f(x) \colon P(x) \equiv \Omega^0_a(P, f(a), x)$.

(3) $\mathsf{apd}^0_f(c_0) \equiv f(a) \equiv d_0(f(a))$.

INDUCTIVE STEP $(n \mapsto n+1)$: Assume defined $\Omega^n_a(P, u, p)$ with trivial dependent loop $d_n$, and $\mathsf{apd}^n_f$.

- $\Omega^{n+1}_a(P, u, p) :\equiv \left(d_n(u) =_p^{x \mapsto \Omega^n_a(P,u,x)} d_n(u)\right)$.
- $d_{n+1}(u) :\equiv \mathsf{refl}_{d_n(u)}$, $c_{n+1} :\equiv \mathsf{refl}_{c_n}$ (Remark 2.2.4).
- $\mathsf{apd}^{n+1}_f(p) :\equiv \mathsf{apd}_{\mathsf{apd}^n_f}(p)$.

Let $Q(u, x) :\equiv \Omega^n_a(P, u, x)$. Conditions (1), (2), and (3) can be verified as follows:

(1) By condition (1) of the inductive hypothesis, $d_n(u) \colon Q(u, c_n)$. Hence,

$$\begin{aligned} d_{n+1}(u) &\equiv \mathsf{refl}_{d_n(u)} \\ &\quad : \left(d_n(u) =_{\mathsf{refl}_{c_n}}^{Q(u,-)} d_n(u)\right) \\ &\equiv \Omega^{n+1}_a(P, u, \mathsf{refl}_{c_n}) \\ &\equiv \Omega^{n+1}_a(P, u, c_{n+1}). \end{aligned}$$

(2) Applying $\mathsf{apd}$ to the dependent function

$$\mathsf{apd}^n_f \colon \prod_{p \colon c_n = c_n} Q(f(a), p)$$

yields

$$\begin{aligned} \mathsf{apd}^{n+1}_f(p) &\equiv \mathsf{apd}_{\mathsf{apd}^n_f}(p) \\ &\quad : \left(\mathsf{apd}^n_f(c_n) =_p^{Q(f(a),-)} \mathsf{apd}^n_f(c_n)\right). \end{aligned}$$

By the inductive invariant (3), the type above translates into

$$d_n(f(a)) =_p^{Q(f(a),-)} d_n(f(a)),$$

which is definitionally $\Omega^{n+1}_a(P, f(a), p)$, confirming that the type signature matches (2).

(3) Evaluating the action on the trivial loop $c_{n+1} \equiv \mathsf{refl}_{c_n}$ yields:

$$\begin{aligned}
\mathsf{apd}_f^{n+1}(c_{n+1}) &\equiv \mathsf{apd}_f^{n+1}(\mathsf{refl}_{c_n}) \\
&\equiv \mathsf{apd}_{\mathsf{apd}_f^n}(\mathsf{refl}_{c_n}) \\
&\equiv \mathsf{refl}_{\mathsf{apd}_f^n(c_n)} \\
&\equiv \mathsf{refl}_{d_n(f(a))} && \text{; IH (3)} \\
&\equiv d_{n+1}(f(a)) && \text{; Defn.}
\end{aligned}$$

**Induction Principle for $\mathbb{S}^n$.** Given a type family $P\colon \mathbb{S}^n \to \mathcal{U}$, along with the following data:

- $b\colon P(\mathsf{base})$,
- $\ell\colon \Omega^n_{\mathsf{base}}(P, b, \mathsf{loop}_n)$,

there is a dependent function

$$\mathsf{ind}_{\mathbb{S}^n}(P, b, \ell)\colon \prod_{x\colon \mathbb{S}^n} P(x)$$

satisfying the computation rules

$$\mathsf{ind}_{\mathbb{S}^n}(P, b, \ell)(\mathsf{base}) \equiv b$$

and

$$\mathsf{apd}^n_{\mathsf{ind}_{\mathbb{S}^n}(P,b,\ell)}(\mathsf{loop}_n) \coloneqq \ell.$$

**Example 6.5.12.** To illustrate general induction, consider the evaluation of Definition 6.5.11 at $n = 1$. The loop $p$ resides in $\pi_1(\Omega^1(\langle A, a\rangle))$, which definitionally evaluates to $a =_A a$. In this case, the definition translates into:

- $\Omega^1_a(P, u, p) :\equiv (u =^P_p u)$.
- $d_1(u) :\equiv \mathsf{refl}_{d_0(u)} \equiv \mathsf{refl}_u$, and $c_1 :\equiv \mathsf{refl}_{c_0} \equiv \mathsf{refl}_a$.
- $\mathsf{apd}^1_f(p) :\equiv \mathsf{apd}_{\mathsf{apd}^0_f}(p) \equiv \mathsf{apd}_f(p)$.

Regarding the structural conditions, we observe that condition (1) reduces to

$$d_1(u) \equiv \mathsf{refl}_u\colon (u =^P_{\mathsf{refl}_a} u) \equiv \Omega^1_a(P, u, \mathsf{refl}_a) \equiv \Omega^1_a(P, u, c_1).$$

Condition (2) reduces to the standard action of a dependent function on a path,

$$\mathsf{apd}^1_f(p) \equiv \mathsf{apd}_f(p)\colon f(a) =^P_p f(a),$$

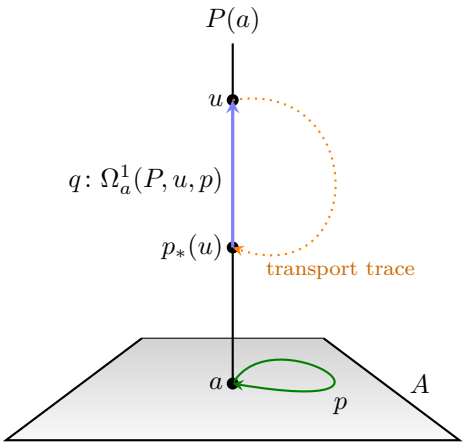
which is satisfied because

$$\Omega^1_a(P, f(a), p) \equiv (f(a) =^P_p f(a)).$$

The invariant (3) follows from the computation rule for $\mathsf{apd}$ over reflexivity:

$$\mathsf{apd}^1_f(c_1) \equiv \mathsf{apd}_f(\mathsf{refl}_a) \equiv \mathsf{refl}_{f(a)} \equiv d_1(f(a)).$$

We can represent $q\colon \Omega^1_a(P,u,p)$ as follows.




For $n = 2$, an ordinary 2-loop $p$ inhabits $\pi_1(\Omega^2(\langle A, a\rangle))$, which evaluates to $\mathsf{refl}_a =_{a=_A a} \mathsf{refl}_a$. Using our computations for $n = 1$, we obtain:

- $\Omega^2_a(P,u,p) :\equiv \left(d_1(u) =^{\Omega^1_a(P,u,-)}_{p} d_1(u)\right) \equiv \left(\mathsf{refl}_u =^{x\mapsto(u=^P_x u)}_{p} \mathsf{refl}_u\right)$.
- $d_2(u) :\equiv \mathsf{refl}_{d_1(u)} \equiv \mathsf{refl}_{\mathsf{refl}_u}$, and $c_2 :\equiv \mathsf{refl}_{c_1} \equiv \mathsf{refl}_{\mathsf{refl}_a}$.
- $\mathsf{apd}^2_f(p) :\equiv \mathsf{apd}_{\mathsf{apd}^1_f}(p) \equiv \mathsf{apd}_{\mathsf{apd}_f}(p)$.

As a concrete instance, consider the 2-sphere $\mathbb{S}^2$, introduced in Definition 6.5.9. The generator $\mathsf{surf}$ (corresponding to $\mathsf{loop}_2$) is an element of $\pi_1(\Omega^2(\langle \mathbb{S}^2, \mathsf{base}\rangle))$. Given a type family $P\colon \mathbb{S}^2 \to \mathcal{U}$ and an element $b\colon P(\mathsf{base})$, a dependent 2-loop over $\mathsf{surf}$ anchored at $b$ has the type

$$\Omega^2_{\mathsf{base}}(P,b,\mathsf{surf}) \equiv \left(\mathsf{refl}_b =^{x\mapsto(b=^P_x b)}_{\mathsf{surf}} \mathsf{refl}_b\right).$$

Furthermore, for any dependent function $f\colon \prod_{x:\mathbb{S}^2} P(x)$, its action on $\mathsf{surf}$ has the type

$$\mathsf{apd}^2_f(\mathsf{surf}) \equiv \mathsf{apd}_{\mathsf{apd}_f}(\mathsf{surf})\colon \Omega^2_{\mathsf{base}}(P, f(\mathsf{base}), \mathsf{surf}).$$

Consequently, the induction principle for $\mathbb{S}^n$ at $n = 2$ with input data $b\colon P(\mathsf{base})$ and $\ell\colon \Omega^2_{\mathsf{base}}(P,b,\mathsf{surf})$, includes the computation rule $f(\mathsf{base}) \equiv b$ for the induced function $f :\equiv \mathsf{ind}_{\mathbb{S}^2}(P,b,\ell)$. This yields the judgmental equality of types

$$\Omega^2_{\mathsf{base}}(P, f(\mathsf{base}), \mathsf{surf}) \equiv \Omega^2_{\mathsf{base}}(P,b,\mathsf{surf}),$$

which ensures that both sides of the computation rule $\mathsf{apd}_{\mathsf{apd}_f}(\mathsf{surf}) := \ell$ inhabit the type $\mathsf{refl}_b =^{x\mapsto(b=^P_x b)}_{\mathsf{surf}} \mathsf{refl}_b$.

### 6.5.2 Recursion

**Definition 6.5.13.** Let $B, C : \mathcal{U}$, $b : B$, and $g : B \to C$. Denote the respective trivial $n$-loops by

$$c_n^B :\equiv \pi_2(\Omega^n(\langle B, b\rangle)) \quad \text{and} \quad c_n^C :\equiv \pi_2(\Omega^n(\langle C, g(b)\rangle)).$$

By induction on $n : \mathbb{N}$, we define the *action* $\mathsf{ap}_g^n$ *on* $n$*-loops*,

$$\mathsf{ap}_g^n : \pi_1(\Omega^n(\langle B, b\rangle)) \to \pi_1(\Omega^n(\langle C, g(b)\rangle)), \tag{1}$$

satisfying the invariant

$$\mathsf{ap}_g^n(c_n^B) \equiv c_n^C, \tag{2}$$

as follows:

Base Case ($n = 0$): $\mathsf{ap}_g^0(x) :\equiv g(x)$.

Conditions (1) and (2) hold trivially because $\Omega^0 \equiv \mathsf{id}_{\mathcal{U}_\bullet}$.

Inductive Step ($n \mapsto n+1$): $\mathsf{ap}_g^{n+1}(p) :\equiv \mathsf{ap}_{\mathsf{ap}_g^n}(p)$.

Conditions (1) and (2) are verified as follows:

(1) Applying $\mathsf{ap}$ to $\mathsf{ap}_g^n$ along $p : c_n^B = c_n^B$ yields a path

$$\begin{aligned} \mathsf{ap}_{\mathsf{ap}_g^n}(p) : \mathsf{ap}_g^n(c_n^B) &= \mathsf{ap}_g^n(c_n^B) \\ &\equiv c_n^C = c_n^C && ;\ \text{IH (2)} \\ &\equiv \pi_1(\Omega^{n+1}(\langle C, g(b)\rangle)). && ;\ \text{Rem. 2.2.4} \end{aligned}$$

(2) Evaluating $\mathsf{ap}_g^{n+1}$ at the trivial loop $c_{n+1}^B \equiv \mathsf{refl}_{c_n^B}$ gives

$$\begin{aligned} \mathsf{ap}_g^{n+1}(c_{n+1}^B) &\equiv \mathsf{ap}_{\mathsf{ap}_g^n}(\mathsf{refl}_{c_n^B}) \\ &\equiv \mathsf{refl}_{\mathsf{ap}_g^n(c_n^B)} \\ &\equiv \mathsf{refl}_{c_n^C} && ;\ \text{IH (2)} \\ &\equiv c_{n+1}^C. && ;\ \text{Rem. 2.2.4} \end{aligned}$$

**Proposition 6.5.14.** *Let $B, C : \mathcal{U}$, $b : B$, and let $P : B \to \mathcal{U}$ be the constant type family $P(x) :\equiv C$. Write $c_n^B :\equiv \pi_2(\Omega^n(\langle B, b\rangle))$ and $c_n(u) :\equiv \pi_2(\Omega^n(\langle C, u\rangle))$ for $u : C$. Then for every $n : \mathbb{N}$, $u : C$, and $p : \pi_1(\Omega^n(\langle B, b\rangle))$, there is an equivalence*

$$\phi_n(u, p) : \Omega_b^n(P, u, p) \simeq \pi_1(\Omega^n(\langle C, u\rangle)) \tag{1}$$

*satisfying the invariant*

$$\phi_n(u, c_n^B)(d_n(u)) \equiv c_n(u), \tag{2}$$

*such that for any function $g : B \to C$,*

$$\phi_n(g(b), p)\big(\mathsf{apd}_g^n(p)\big) =_{\pi_1(\Omega^n(\langle C, g(b)\rangle))} \mathsf{ap}_g^n(p). \tag{3}$$

*Proof.* We proceed by induction on $n\colon \mathbb{N}$.

Base Case ($n = 0$): In this case, $p$ is simply an element $x\colon B$. Since

$$\Omega^0_b(P, u, x) \equiv P(x) \equiv C \equiv \pi_1(\Omega^0(\langle C, u\rangle)),$$

we define $\phi_0(u, x) :\equiv \mathsf{id}_C$.

Then $\phi_0(u, x)$ is an equivalence, and conditions (2) and (3) hold judgmentally.

Inductive Step ($n \mapsto n + 1$): Introduce

$$Q(u, x) :\equiv \Omega^n_b(P, u, x) \quad \text{and} \quad D(u) :\equiv \pi_1(\Omega^n(\langle C, u\rangle)).$$

For an $(n + 1)$-loop $p\colon c^B_n = c^B_n$ and a dependent loop $q\colon \Omega^{n+1}_b(P, u, p)$, Definition 6.5.11 yields

$$q\colon \mathsf{transport}^{Q(u,-)}(p, d_n(u)) =_{Q(u,c^B_n)} d_n(u).$$

First, we construct a path

$$\tau_n(u, p)\colon \phi_n(u, c^B_n)\big(\mathsf{transport}^{Q(u,-)}(p, d_n(u))\big) =_{D(u)} c_n(u)$$

by based path induction on $p\colon c^B_n = x$ for a generic $x\colon \pi_1(\Omega^n(\langle B, b\rangle))$.

Replacing $c^B_n$ with $x$, the left-hand side of the identity type above becomes

$$\underbrace{\phi_n(u, x)(\mathsf{transport}^{Q(u,-)}(p, \underbrace{d_n(u)}_{Q(u,c^B_n)}))}_{Q(u,x)}.$$

When $p \equiv \mathsf{refl}_{c^B_n}$, the inductive condition (2) reduces judgmentally this expression to

$$\phi_n(u, c^B_n)(d_n(u)) \equiv c_n(u).$$

Hence, we can set

$$\tau_n(u, \mathsf{refl}_{c^B_n}) :\equiv \mathsf{refl}_{c_n(u)}.$$

We then define

$$\begin{aligned} \phi_{n+1}(u, p)\colon \Omega^{n+1}_b(P, u, p) &\to \big(c_n(u) =_{D(u)} c_n(u)\big) \\ q &\mapsto \tau_n(u, p)^{-1} \cdot \mathsf{ap}_{\phi_n(u,c^B_n)}(q). \end{aligned} \tag{6.1}$$

This is well-typed in $(c_n(u) =_{D(u)} c_n(u)) \equiv \pi_1(\Omega^{n+1}(\langle C, u\rangle))$ because condition (2) implies that $\mathsf{ap}_{\phi_n(u,c^B_n)}(q)$ has right endpoint $c_n(u)$.

We now verify the three conditions for $n + 1$:

a) By the inductive hypothesis, $\phi_n(u, c^B_n)$ is an equivalence. Hence, $\mathsf{ap}_{\phi_n(u,c^B_n)}$ is an equivalence, and post-concatenation with $\tau_n(u, p)^{-1}$ is also an equivalence.

b) At the trivial loop $c^B_{n+1} \equiv \mathsf{refl}_{c^B_n}$ and $d_{n+1}(u) \equiv \mathsf{refl}_{d_n(u)}$, both $\tau_n(u, \mathsf{refl}_{c^B_n})$ and $\mathsf{ap}_{\phi_n(u,c^B_n)}(\mathsf{refl}_{d_n(u)})$ reduce judgmentally to $\mathsf{refl}_{c_n(u)}$. Thus,

$$\phi_{n+1}(u, c^B_{n+1})(d_{n+1}(u)) \equiv \mathsf{refl}^{-1}_{c_n(u)} \cdot \mathsf{refl}_{c_n(u)} \equiv \mathsf{refl}_{c_n(u)} \equiv c_{n+1}(u).$$

c) To simplify the notation, let us introduce $X :\equiv \pi_1(\Omega^n(\langle B, b\rangle))$ and put $s :\equiv \mathsf{apd}^n_g$. Define $f_1, f_2 \colon X \to D(u)$ by

$$f_1(x) :\equiv \phi_n(u, x)(s(x)),$$
$$f_2(x) :\equiv \mathsf{ap}^n_g(x).$$

The inductive hypothesis (3) provides a homotopy $\eta \colon f_1 \sim f_2$. By function extensionality, we obtain a path $f_1 = f_2$. Consequently, for an $(n+1)$-loop $p \colon c^B_n = c^B_n$, we have

$$\mathsf{ap}_{f_1}(p) = \mathsf{ap}_{f_2}(p).$$

Since the RHS is definitionally equal to $\mathsf{ap}^{n+1}_g(p)$, our goal reduces to proving that $\mathsf{ap}_{f_1}(p)$ is equal to the LHS of the target equation. We proceed by path induction on $p$.

Let us generalize $p$ to a path with a free right endpoint, $p \colon c^B_n =_X x$. We want to establish a path between:

a) $\mathsf{ap}_{f_1}(p) \colon f_1(c^B_n) =_{D(u)} f_1(x)$

b) $\tau_n(u, p)^{-1} \cdot \mathsf{ap}_{\phi_n(u,x)}\big(\mathsf{apd}_s(p)\big) \colon f_1(c^B_n) =_{D(u)} f_1(x)$

Let us verify that the endpoints of the second expression match the first:

- $\mathsf{apd}_s(p) \colon \mathsf{transport}^{Q(u,-)}(p, s(c^B_n)) = s(x)$.
- $s(c^B_n) \equiv \mathsf{apd}^n_g(c^B_n) \equiv d_n(u)$.
- $\mathsf{ap}_{\phi_n(u,x)}\big(\mathsf{apd}_s(p)\big) \colon \phi_n(u, x)\big(\mathsf{transport}^{Q(u,-)}(p, d_n(u))\big) = f_1(x)$.
- $\tau_n(u, p) \colon \phi_n(u, x)\big(\mathsf{transport}^{Q(u,-)}(p, d_n(u))\big) = c_n(u)$.
- Since $c_n(u) \equiv f_1(c^B_n)$ by (2), $\tau_n(u, p)^{-1}$ bridges $f_1(c^B_n)$ to the start of the second path, making the concatenation in b) well-typed.

To prove these two paths are equal for all $x$ and $p$, by path induction it suffices to check the case where $x \equiv c^B_n$ and $p \equiv \mathsf{refl}_{c^B_n}$. Evaluating both sides at reflexivity, we obtain:

- The path in a) reduces to $\mathsf{ap}_{f_1}(\mathsf{refl}) \equiv \mathsf{refl}_{f_1(c^B_n)}$.
- The path in b) reduces judgmentally to $\mathsf{refl} \cdot \mathsf{refl} \equiv \mathsf{refl}_{c_n(u)}$.

Since $f_1(c_n^B) \equiv c_n(u)$, both sides reduce to the same reflexivity path. Therefore, the equality holds for all paths $p$.

In conclusion, when we restrict $p$ back to being an $(n+1)$-loop by setting $x :\equiv c_n^B$, the equality just proven yields

$$\mathsf{ap}_{f_1}(p) = \tau_n(u,p)^{-1} \cdot \mathsf{ap}_{\phi_n(u,c_n^B)}\big(\mathsf{apd}_s(p)\big).$$

According to (6.1), the RHS is definitionally equal to $\phi_{n+1}(u,p)$ applied to $\mathsf{apd}_s(p)$. Since

$$\mathsf{apd}_s(p) \equiv \mathsf{apd}_{\mathsf{apd}_g^n}(p) \equiv \mathsf{apd}_g^{n+1}(p),$$

we arrive at

$$\phi_{n+1}(u,p)\big(\mathsf{apd}_g^{n+1}(p)\big) = \mathsf{ap}_{f_1}(p) = \mathsf{ap}_{f_2}(p) \equiv \mathsf{ap}_g^{n+1}(p),$$

proving condition (3).

□

**Theorem 6.5.15.** *For any type $C$ and any $n\colon \mathbb{N}$, the $n$-sphere $\mathbb{S}^n$ satisfies the non-dependent recursion principle. That is, given a point $u\colon C$ and an $n$-loop $\ell\colon \pi_1(\Omega^n(\langle C, u\rangle))$, there exists a function $g\colon \mathbb{S}^n \to C$ such that*

$$g(\mathsf{base}_n) \equiv u \quad \textit{and} \quad \mathsf{ap}_g^n(\mathsf{loop}_n) =_{\pi_1(\Omega^n(\langle C,u\rangle))} \ell.$$

*Proof.* Let $P\colon \mathbb{S}^n \to \mathcal{U}$ be the constant type family $P(x) :\equiv C$. To construct a non-dependent function $g\colon \mathbb{S}^n \to C$, which is equivalent to a dependent function $\prod_{x:\mathbb{S}^n} P(x)$, we apply the induction principle of the $n$-sphere 6.5.1. To that end we must specify

- $u\colon P(\mathsf{base}_n) \equiv C$,
- $q\colon \Omega_b^n(P, u, \mathsf{loop}_n)$.

Since $q$ is given, it remains to provide $q$.

By Proposition 6.5.14, we have an equivalence

$$\phi_n(u, \mathsf{loop}_n)\colon \Omega_b^n(P, u, \mathsf{loop}_n) \simeq \pi_1(\Omega^n(\langle C, u\rangle)).$$

From the ordinary $n$-loop $\ell\colon \pi_1(\Omega^n(\langle C, u\rangle))$, we can derive the required dependent loop:

$$q :\equiv \phi_n(u, \mathsf{loop}_n)^{-1}(\ell).$$

Thus, we obtain

$$g\colon \mathbb{S}^n \to C$$
$$g \equiv \mathsf{ind}_{\mathbb{S}^n}(P, u, q)$$

satisfying the computation rules:

i) $g(\mathsf{base}_n) \equiv u$,

ii) $\mathsf{apd}^n_g(\mathsf{loop}_n) =_{\Omega^n_b(P,u,\mathsf{loop}_n)} q$.

The first rule matches our goal for $\mathsf{base}_n$. Therefore, we need to prove the second requirement.

By condition (3) of Proposition 6.5.14, we have

$$\phi_n(g(\mathsf{base}_n), \mathsf{loop}_n)\big(\mathsf{apd}^n_g(\mathsf{loop}_n)\big) =_{\pi_1(\Omega^n(\langle C,u\rangle))} \mathsf{ap}^n_g(\mathsf{loop}_n).$$

Using rules i) and ii) above, we obtain

$$\phi_n(u, \mathsf{loop}_n)(q) =_{\pi_1(\Omega^n(\langle C,u\rangle))} \mathsf{ap}^n_g(\mathsf{loop}_n),$$

which, by the definition of $q$, computes to

$$\phi_n(u, \mathsf{loop}_n)\big(\phi_n(u, \mathsf{loop}_n)^{-1}(\ell)\big) =_{\pi_1(\Omega^n(\langle C,u\rangle))} \ell.$$

Concatenating with the previous path yields $\mathsf{ap}^n_g(\mathsf{loop}_n) =_{\pi_1(\Omega^n(\langle C,u\rangle))} \ell$.

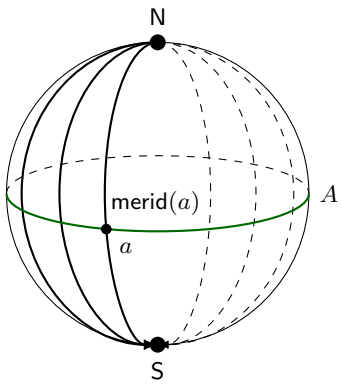
□

## 6.6 Suspensions

**Definition 6.6.1.** Given a type $A$, the *suspension* of $A$ is the type inductively defined by

- $\mathsf{N} : \mathsf{Susp}(A)$,
- $\mathsf{S} : \mathsf{Susp}(A)$, and
- $\mathsf{merid} : A \to (\mathsf{N} =_{\mathsf{Susp}(A)} \mathsf{S})$.

The suspension $\mathsf{Susp}(A)$ represents the universal method for elevating the points of a type $A$ into paths. Consequently, the 1-paths of $A$ are promoted to 2-paths, and so forth across higher dimensions. Geometrically, this construction evokes a globe, featuring a north pole and a south pole connected by a family of meridians parameterized by the elements of $A$.



The recursion principle for $\mathsf{Susp}(A)$ says that given a type $B$ with

- $n\colon B$,
- $s\colon B$, and
- $m\colon A \to (n =_B s)$,

there is a function

$$\mathsf{rec}_{\mathsf{Susp}(A)}(B, n, s, m)\colon \mathsf{Susp}(A) \to B$$

such that

$$\begin{aligned}\mathsf{rec}_{\mathsf{Susp}(A)}(B, n, s, m)(\mathsf{N}) &\equiv n,\\ \mathsf{rec}_{\mathsf{Susp}(A)}(B, n, s, m)(\mathsf{S}) &\equiv s, \text{ and}\\ \mathsf{ap}_{\mathsf{rec}_{\mathsf{Susp}(A)}(B,n,s,m)}(\mathsf{merid}(a)) &= m(a).\end{aligned}$$

The induction principle says that given $P\colon \mathsf{Susp}(A) \to \mathcal{U}$ together with

- $n\colon P(\mathsf{N})$,
- $s\colon P(\mathsf{S})$, and
- for each $a\colon A$, a dependent path $m(a)\colon n =^P_{\mathsf{merid}(a)} s$,

there exists a function

$$\mathsf{ind}_{\mathsf{Susp}(A)}(P, n, s, m)\colon \prod_{x\colon \mathsf{Susp}(A)} P(x)$$

such that

$$\mathsf{ind}_{\mathsf{Susp}(A)}(P, n, s, m)(\mathsf{N}) \equiv n \quad \text{and} \quad \mathsf{ind}_{\mathsf{Susp}(A)}(P, n, s, m)(\mathsf{S}) \equiv s,$$

and for each $a\colon A$ we have

$$\mathsf{apd}_{\mathsf{ind}_{\mathsf{Susp}(A)}(P,n,s,m)}(\mathsf{merid}(a)) = m(a).$$

**Lemma 6.6.2.** *Let $A, B\colon \mathcal{U}$ and $f, g\colon \mathsf{Susp}(A) \to B$. Suppose we have*

- $p\colon f(\mathsf{N}) =_B g(\mathsf{N})$,
- $q\colon f(\mathsf{S}) =_B g(\mathsf{S})$,
- $m\colon \prod_{a\colon A} \mathsf{ap}_f(\mathsf{merid}(a))^{-1} \cdot p \cdot \mathsf{ap}_g(\mathsf{merid}(a)) = q$.

*Then $f = g$.*

*Proof.* By function extensionality, it suffices to construct a homotopy

$$\eta\colon \prod_{x\colon \mathsf{Susp}(A)} f(x) =_B g(x).$$

We define $\eta$ using the induction principle of $\mathsf{Susp}(A)$ on the type family

$$P\colon \mathsf{Susp}(A) \to \mathcal{U}$$
$$P(x) :\equiv f(x) =_B g(x).$$

For the point constructors, we simply provide the given paths $p\colon P(\mathsf{N})$ and $q\colon P(\mathsf{S})$.

For the path constructors, we must provide, for each $a\colon A$, a dependent path $\mu(a)\colon p =^P_{\mathsf{merid}(a)} q$. By Remark 6.3.5), there is an equivalence

$$(p =^P_{\mathsf{merid}(a)} q) \simeq \Big(\mathsf{ap}_f(\mathsf{merid}(a))^{-1} \cdot p \cdot \mathsf{ap}_g(\mathsf{merid}(a)) = q\Big).$$

Since the right-hand side is the type of $m(a)$, we can define $\mu(a)$ as the path corresponding to $m(a)$ under this equivalence.

Now the induction principle yields the required homotopy

$$\eta :\equiv \mathsf{ind}_{\mathsf{Susp}(A)}(P, p, q, \mu). \qquad \square$$

**Proposition 6.6.3.** *Given types $A, B\colon \mathcal{U}$, the function*

$$\phi\colon \big(\mathsf{Susp}(A) \to B\big) \to \sum_{n,s\colon B} A \to n =_B s$$
$$f \mapsto \langle (f(\mathsf{N}), f(\mathsf{S})), \mathsf{ap}_f \circ \mathsf{merid}\rangle$$

*is an equivalence.*

*Proof.* For the backward map, define

$$\psi\colon \Big(\sum_{n,s\colon B} A \to n =_B s\Big) \to \big(\mathsf{Susp}(A) \to B\big)$$
$$\langle (n, s), m\rangle \mapsto \mathsf{rec}_{\mathsf{Susp}(A)}(B, n, s, m).$$

To complete the proof, we have to compute both compositions.

$$\begin{aligned}\phi \circ \psi(\langle (n, s), m\rangle) &\equiv \phi\big(\mathsf{rec}_{\mathsf{Susp}(A)}(B, n, s, m)\big)\\ &\equiv \langle (n, s), \mathsf{ap}_\rho \circ \mathsf{merid}\rangle,\end{aligned}$$

where

$$\rho :\equiv \mathsf{rec}_{\mathsf{Susp}(A)}(B, n, s, m).$$

Because the first component of the dependent pair is definitionally equal to $(n, s)$ and transport along reflexivity is the identity, the goal reduces, via function extensionality, to showing that $\mathsf{ap}_\rho(\mathsf{merid}(a)) = m(a)$, which is precisely the third computation rule of the recursor.

With regard to the other composition, we have

$$\begin{aligned}\psi \circ \phi(f) &\equiv \psi(B, f(\mathsf{N}), f(\mathsf{S}), \mathsf{ap}_f \circ \mathsf{merid}) \\ &\equiv \mathsf{rec}_{\mathsf{Susp}(A)}\big(B, f(\mathsf{N}), f(\mathsf{S}), \mathsf{ap}_f \circ \mathsf{merid}\big).\end{aligned}$$

The conclusion follows from Lemma 6.6.2. □

**Lemma 6.6.4.** $\mathsf{Susp}(2) \simeq \mathbb{S}^1$.

*Proof.* Define $m :\equiv \mathsf{rec}_2(\mathsf{base} =_{\mathbb{S}^1} \mathsf{base}, \mathsf{loop}, \mathsf{refl}_{\mathsf{base}})$, that is,

$$\begin{aligned} m\colon 2 &\to (\mathsf{base} =_{\mathbb{S}^1} \mathsf{base}) \\ 0_2 &\mapsto \mathsf{loop} \\ 1_2 &\mapsto \mathsf{refl}_{\mathsf{base}}.\end{aligned}$$

Then, recursion on $\mathsf{Susp}(2)$ gives us the forward map

$$f :\equiv \mathsf{rec}_{\mathsf{Susp}(2)}(\mathbb{S}^1, \mathsf{base}, \mathsf{base}, m).$$

For the backward map, consider

$$g :\equiv \mathsf{rec}_{\mathbb{S}^1}\big(\mathsf{Susp}(2), \mathsf{N}, \mathsf{merid}(0_2) \cdot \mathsf{merid}(1_2)^{-1}\big),$$

which is the function defined by recursion on $\mathbb{S}^1$ satisfying

$$\begin{aligned} g\colon \mathbb{S}^1 &\to \mathsf{Susp}(2) \\ g(\mathsf{base}) &\equiv \mathsf{N} \\ \mathsf{ap}_g(\mathsf{loop}) &= \mathsf{merid}(0_2) \cdot \mathsf{merid}(1_2)^{-1}.\end{aligned}$$

To complete the proof, we must show that $f$ and $g$ are quasi-inverses. For the composition $f \circ g$, we have

$$\begin{aligned} f \circ g(\mathsf{base}) &\equiv f(\mathsf{N}) \\ &\equiv \mathsf{base}.\end{aligned}$$

$$\begin{aligned} \mathsf{ap}_{f\circ g}(\mathsf{loop}) &= \mathsf{ap}_f(\mathsf{merid}(0_2)) \cdot \mathsf{ap}_f(\mathsf{merid}(1_2))^{-1} \\ &= m(0_2) \cdot m(1_2)^{-1} \\ &= \mathsf{loop} \cdot \mathsf{refl}_{\mathsf{base}} \\ &= \mathsf{loop}.\end{aligned}$$

Therefore, $f \circ g \sim \mathsf{id}_{\mathbb{S}^1}$ by Lemma 6.3.6.

For the other composition,

$$\begin{aligned} g \circ f(\mathsf{N}) &\equiv g(\mathsf{base}) \\ &\equiv \mathsf{N}\end{aligned}$$

$$\begin{aligned} g \circ f(\mathsf{S}) &\equiv g(\mathsf{base}) \\ &\equiv \mathsf{N} \\ &= \mathsf{S} \qquad\qquad ;\ \mathsf{merid}(1_2).\end{aligned}$$

This means that $n :\equiv \mathsf{refl}_{\mathsf{N}}$ and $s :\equiv \mathsf{merid}(1_2)$ satisfy

- $n\colon g \circ f(\mathsf{N}) = \mathsf{N}$,
- $s\colon g \circ f(\mathsf{S}) = \mathsf{S}$.

Let $P\colon \mathsf{Susp}(2) \to \mathcal{U}$ be defined by $P(x) :\equiv g \circ f(x) =_{\mathsf{Susp}(2)} x$. The conditions above translate into

- $n\colon P(\mathsf{N})$,
- $s\colon P(\mathsf{S})$.

Therefore, to construct a witness of $g \circ f \sim \mathsf{id}_{\mathsf{Susp}(2)}$, it suffices to define, for each $b\colon 2$, a dependent path $\mathfrak{m}(b)\colon n =^{P}_{\mathsf{merid}(b)} s$, and then invoke the induction principle of $\mathsf{Susp}(2)$.

The induction principle of 2 allows us to reduce the construction to the cases $b \equiv 0_2$ and $b \equiv 1_2$. By Remark 6.3.5, the type of the dependent path is equivalent to an equality of paths in $\mathsf{Susp}(2)$:

$$(n =^{P}_{\mathsf{merid}(0_2)} s) \simeq \Big(\mathsf{ap}_{g\circ f}(\mathsf{merid}(0_2))^{-1} \cdot n \cdot \mathsf{ap}_{\mathsf{id}_{\mathsf{Susp}(2)}}(\mathsf{merid}(0_2)) = s\Big).$$

To compute this, observe that

$$\begin{aligned}\mathsf{ap}_{g\circ f}(\mathsf{merid}(0_2)) &= \mathsf{ap}_g(\mathsf{ap}_f(\mathsf{merid}(0_2)))\\ &= \mathsf{ap}_g(m(0_2))\\ &= \mathsf{ap}_g(\mathsf{loop})\\ &= \mathsf{merid}(0_2) \cdot \mathsf{merid}(1_2)^{-1}.\end{aligned}$$

Hence,

$$\mathsf{ap}_{g\circ f}(\mathsf{merid}(0_2))^{-1} = \mathsf{merid}(1_2) \cdot \mathsf{merid}(0_2)^{-1}.$$

Therefore, using that $n \equiv \mathsf{refl}_{\mathsf{N}}$ and $\mathsf{ap}_{\mathsf{id}_{\mathsf{Susp}(2)}}(p) \equiv p$, the goal reduces to

$$(n =^{P}_{\mathsf{merid}(0_2)} s) \simeq \mathsf{merid}(1_2) = s,$$

which is trivial since, by definition, $s \equiv \mathsf{merid}(1_2)$.

Similarly, for the case $b \equiv 1_2$, the required dependent path type translates into

$$(n =^{P}_{\mathsf{merid}(1_2)} s) \simeq \Big(\mathsf{ap}_{g\circ f}(\mathsf{merid}(1_2))^{-1} \cdot n \cdot \mathsf{ap}_{\mathsf{id}_{\mathsf{Susp}(2)}}(\mathsf{merid}(1_2)) = s\Big),$$

where

$$\begin{aligned}\mathsf{ap}_{g\circ f}(\mathsf{merid}(1_2)) &= \mathsf{ap}_g(\mathsf{ap}_f(\mathsf{merid}(1_2)))\\ &= \mathsf{ap}_g(\mathsf{refl}_{\mathsf{base}})\\ &= \mathsf{refl}_{g(\mathsf{base})}\\ &= \mathsf{refl}_{\mathsf{N}}.\end{aligned}$$

Hence,

$$\begin{aligned}\mathsf{ap}_{g\circ f}(\mathsf{merid}(1_2))^{-1}\cdot n\cdot \mathsf{ap}_{\mathsf{id}_{\mathsf{Susp}(2)}}(\mathsf{merid}(1_2)) &= \mathsf{refl}_{\mathsf{N}}^{-1}\cdot \mathsf{refl}_{\mathsf{N}}\cdot \mathsf{merid}(1_2)\\ &= \mathsf{merid}(1_2)\\ &\equiv s.\end{aligned}$$

This provides the witnesses $\mathfrak{m}(0_2)$ and $\mathfrak{m}(1_2)$, completing the proof. □

**Lemma 6.6.5.** $\mathsf{Susp}(0) \simeq 2$.

*Proof.* For the forward map, we define

$$\begin{aligned}&f\colon \mathsf{Susp}(0)\to 2\\ &f :\equiv \mathsf{rec}_{\mathsf{Susp}(0)}(2, 0_2, 1_2, \mathsf{rec}_0(0_2 =_2 1_2)).\end{aligned}$$

For the backward map, let

$$g :\equiv \mathsf{rec}_2(\mathsf{Susp}(0), \mathsf{N}, \mathsf{S}).$$

To verify that $f\circ g \sim \mathsf{id}_2$, observe that

$$\begin{aligned}f\circ g(0_2) &\equiv f(\mathsf{N}) \equiv 0_2,\\ f\circ g(1_2) &\equiv f(\mathsf{S}) \equiv 1_2.\end{aligned}$$

For the other composition, let $P\colon \mathsf{Susp}(0)\to\mathcal{U}$ be the type family defined by $P(x) :\equiv g\circ f(x) =_{\mathsf{Susp}(0)} x$. Since

$$\begin{aligned}g\circ f(\mathsf{N}) &\equiv g(0_2) \equiv \mathsf{N},\\ g\circ f(\mathsf{S}) &\equiv g(1_2) \equiv \mathsf{S},\end{aligned}$$

we define $n :\equiv \mathsf{refl}_{\mathsf{N}}\colon P(\mathsf{N})$ and $s :\equiv \mathsf{refl}_{\mathsf{S}}\colon P(\mathsf{S})$. Thus, for any $z\colon 0$, we have

$$\mathsf{ind}_0(Q, z)\colon n =^P_{\mathsf{merid}(z)} s,$$

where $Q\colon 0\to\mathcal{U}$ is defined by $Q(z) :\equiv n =^P_{\mathsf{merid}(z)} s$. Therefore, we define $m(z) :\equiv \mathsf{ind}_0(Q, z)$ and complete the proof by providing

$$\mathsf{ind}_{\mathsf{Susp}(0)}(P, n, s, m)$$

as the witness for $g\circ f \sim \mathsf{id}_{\mathsf{Susp}(0)}$. □

**Remark 6.6.6.** If we define

- $\mathbb{S}^{-1} :\equiv 0$ and
- $\mathbb{S}^0 :\equiv 2$,

then Lemmas 6.6.5 and 6.6.4 establish that

- $\mathsf{Susp}(\mathbb{S}^{-1}) \simeq \mathbb{S}^0$ and
- $\mathsf{Susp}(\mathbb{S}^0) \simeq \mathbb{S}^1$.

**Definition 6.6.7.** Given two pointed types $\langle A, a_0\rangle, \langle B, b_0\rangle : \mathcal{U}_\bullet$, we introduce

$$\mathsf{Map}_\bullet(\langle A, a_0\rangle, \langle B, b_0\rangle) :\equiv \sum_{f:\, A\to B} f(a_0) =_B b_0.$$

In addition, we define

$$\begin{aligned}(-)_+ &: \mathcal{U} \to \mathcal{U}_\bullet\\ A_+ &:\equiv \langle A + \mathsf{1}, \mathsf{inr}(\star)\rangle.\end{aligned}$$

**Proposition 6.6.8.** *Let $A$ and $B$ be types. Given functions $f, g : A \to B$ and a path $\gamma : f =_{A\to B} g$, and elements $x_0 : A$ and $y_0 : B$, consider the type family*

$$\begin{aligned}P &: (A \to B) \to \mathcal{U}\\ P(h) &:\equiv h(x_0) =_B y_0.\end{aligned}$$

*Then, for any path $p : f(x_0) =_B y_0$, we have*

$$\mathsf{transport}^P(\gamma, p) = \mathsf{happly}(\gamma, x_0)^{-1} \cdot p.$$

*Proof.* By path induction on $\gamma$, it suffices to consider the case where $g \equiv f$ and $\gamma \equiv \mathsf{refl}_f$. In this case, the left-hand side evaluates definitionally to

$$\mathsf{transport}^P(\mathsf{refl}_f, p) \equiv p.$$

For the right-hand side, since $\mathsf{happly}(\mathsf{refl}_f, x_0) \equiv \mathsf{refl}_{f(x_0)}$, we can evaluate the expression as

$$\begin{aligned}\mathsf{happly}(\mathsf{refl}_f, x_0)^{-1} \cdot p &\equiv \mathsf{refl}_{f(x_0)}^{-1} \cdot p\\ &\equiv \mathsf{refl}_{f(x_0)} \cdot p\\ &\equiv p.\end{aligned}$$

This establishes the equality for any path $\gamma$. □

**Corollary 6.6.9.** *Under the conditions of the propositions, if $\gamma \equiv \mathsf{funext}(\eta)$ for a homotopy $\eta : f \sim g$, then*

$$\mathsf{transport}^P(\mathsf{funext}(\eta), p) = \eta(x_0)^{-1} \cdot p.$$

*Proof.* The result is a direct consequence of the equality

$$\mathsf{happly}(\mathsf{funext}(\eta), x_0) = \eta(x_0).$$ □

**Theorem 6.6.10.** *Given $A : \mathcal{U}$ and $\langle B, b_0\rangle : \mathcal{U}_\bullet$, we have*

$$\mathsf{Map}_\bullet(A_+, \langle B, b_0\rangle) \simeq (A \to B).$$

*Proof.* For the forward map, consider

$$\begin{aligned}\phi : \mathsf{Map}_\bullet(A_+, \langle B, b_0\rangle) &\to (A \to B)\\ \langle f, p\rangle &\mapsto f \circ \mathsf{inl},\end{aligned}$$

where $f : A + 1 \to B$ with $p : f(\mathsf{inr}(\star)) =_B b_0$, and $\mathsf{inl} : A \to A + 1$.

For the backward map, we have

$$\begin{aligned}\psi : (A \to B) &\to \mathsf{Map}_\bullet(A_+, \langle B, b_0\rangle)\\ g &\mapsto \langle \mathsf{rec}_{A+1}(B, g, \lambda\star.\, b_0), \mathsf{refl}_{b_0}\rangle.\end{aligned}$$

We can now compute both compositions:

$$\phi \circ \psi(g) \equiv \mathsf{rec}_{A+1}(B, g, \lambda\star.\, b_0) \circ \mathsf{inl} \equiv g.$$

The other composition yields

$$\psi \circ \phi(\langle f, p\rangle) \equiv \langle \mathsf{rec}_{A+1}(B, f \circ \mathsf{inl}, \lambda\star.\, b_0), \mathsf{refl}_{b_0}\rangle.$$

By the equality in $\Sigma$-types, constructing a path $\langle f, p\rangle = \psi \circ \phi(\langle f, p\rangle)$ translates into providing paths

$$\alpha : f =_{A+1\to B} \mathsf{rec}_{A+1}(B, f \circ \mathsf{inl}, \lambda\star.\, b_0)$$

and

$$\beta : \mathsf{transport}^P(\alpha, p) = \mathsf{refl}_{b_0},$$

where $P(h) :\equiv h(\mathsf{inr}(\star)) =_B b_0$.

Let

$$\rho :\equiv \mathsf{rec}_{A+1}(B, f \circ \mathsf{inl}, \lambda\star.\, b_0).$$

To construct $\alpha$, we need a homotopy

$$\eta : \prod_{x:\, A+1} f(x) =_B \rho(x).$$

We define $\eta$ by induction on the coproduct $A + 1$. By the computation rule, we have

$$\rho \circ \mathsf{inl} \equiv f \circ \mathsf{inl}.$$

Hence, we can set $\eta \circ \mathsf{inl} :\equiv \mathsf{refl}_{f\circ\mathsf{inl}}$.

For the right constructor, the computation rule gives $\rho \circ \mathsf{inr} \equiv \lambda\star.\, b_0$, which implies $\rho(\mathsf{inr}(\star)) \equiv b_0$.

Because we need a path of type $f(\mathsf{inr}(\star)) =_B \rho(\mathsf{inr}(\star))$ and $\rho(\mathsf{inr}(\star)) \equiv b_0$, we can define $\eta(\mathsf{inr}(\star)) :\equiv p$.

Applying function extensionality, we obtain $\alpha :\equiv \mathsf{funext}(\eta)$.

Now we must construct $\beta$. We need to evaluate the transport of $p$ along $\alpha$ within the type family $P(h) :\equiv h(\mathsf{inr}(\star)) =_B b_0$. By Corollary 6.6.9, we obtain

$$\begin{aligned}\mathsf{transport}^P(\alpha, p) &= \eta(\mathsf{inr}(\star))^{-1} \cdot p \\ &\equiv p^{-1} \cdot p \\ &= \mathsf{refl}_{b_0},\end{aligned}$$

which provides the required witness $\beta$. □

**Lemma 6.6.11.** *Given a type $A\colon \mathcal{U}$, an element $a_0\colon A$, and a type family $P\colon A \to \mathcal{U}$, there is an equivalence*

$$\Big(\sum_{a\colon A}(a =_A a_0) \times P(a)\Big) \simeq P(a_0).$$

*Proof.* First, we rewrite the cartesian product as a non-dependent Σ-type:

$$\begin{aligned}\text{LHS} &\simeq \sum_{a\colon A}\ \sum_{p\colon a =_A a_0} P(a) \\ &\simeq \sum_{\omega\colon \sum_{a\colon A} a =_A a_0} P(\pi_1(\omega)) && \text{; Prop. 2.9.13.} \\ &\simeq P(\pi_1(\langle a_0, \mathsf{refl}_{a_0}\rangle)), && \text{; Lem. 3.4.11 \& Thm. 3.4.8 b)}\end{aligned}$$

which is definitionally equal to $P(a_0)$. □

**Lemma 6.6.12.** *Given types $X, Y\colon \mathcal{U}$ and a map $f\colon X \to Y$, there is an equivalence*

$$X \simeq \sum_{x\colon X}\ \sum_{y\colon Y} f(x) =_Y y.$$

*In particular, given a type $A\colon \mathcal{U}$ and a point $a_0\colon A$, we have*

$$(A \to Y) \simeq \sum_{g\colon A\to Y}\ \sum_{y\colon Y} g(a_0) =_Y y.$$

*Proof.* For each $x\colon X$, let $P\colon Y \to \mathcal{U}$ be the constant family $P(y) :\equiv \mathsf{1}$. Applying Lemma 6.6.11 to the type $Y$, the element $f(x)\colon Y$, and $P$, we have

$$\Big(\sum_{y\colon Y} f(x) =_Y y\Big) \simeq \Big(\sum_{y\colon Y}(y =_Y f(x)) \times \mathsf{1}\Big) \simeq \mathsf{1} \qquad \text{; Lem. 6.6.11.}$$

Summing over all $x\colon X$, we obtain

$$\Big(\sum_{x\colon X}\sum_{y\colon Y} f(x) =_Y y\Big) \simeq \sum_{x\colon X} \mathsf{1} \simeq X,$$

which proves the first equivalence. The second equivalence follows from the first by setting $X :\equiv A \to Y$ and $f :\equiv \mathrm{ev}_{a_0}\colon (A \to Y) \to Y$. □

**Remark 6.6.13.** In algebraic topology for any pointed spaces $A$ and $B$, there is a natural bijection between continuous based maps from the suspension of $A$ to $B$, and continuous based maps from $A$ to the loop space of $B$. The following theorem shows that this topological adjunction holds within type theory.

**Theorem 6.6.14.** *For pointed types $\langle A, a_0\rangle$ and $\langle B, b_0\rangle$ we have*

$$\mathsf{Map}_\bullet(\langle \mathsf{Susp}(A), \mathsf{N}\rangle, \langle B, b_0\rangle) \simeq \mathsf{Map}_\bullet(\langle A, a_0\rangle, \Omega(\langle B, b_0\rangle)).$$

*Proof.* We construct the equivalence between the LHS and the RHS via the following chain of equivalences:

$$\begin{aligned}
\text{LHS} &\equiv \sum_{f\colon \mathsf{Susp}(A)\to B} f(\mathsf{N}) =_B b_0 \\
&\simeq \sum_{n,s\colon B}\ \sum_{m\colon A\to n=_B s} n =_B b_0 && \text{; Prop. 6.6.3} \\
&\simeq \sum_{s\colon B}\sum_{n\colon B}\ \sum_{m\colon A\to n=_B s} n =_B b_0 && \text{; Prop. 2.9.12} \\
&\simeq \sum_{s\colon B}\sum_{n\colon B} (A \to n =_B s) \times (n =_B b_0) && \text{; Non-dependent } \Sigma \\
&\simeq \sum_{s\colon B}\sum_{n\colon B} (n =_B b_0) \times (A \to n =_B s) \\
&\simeq \sum_{s\colon B} A \to b_0 =_B s && \text{; Lem. 6.6.11} \\
&\simeq \sum_{s\colon B}\ \sum_{g\colon A\to b_0=_B s}\ \sum_{p\colon b_0=_B s} g(a_0) = p && \text{; Lem. 6.6.12} \\
&\simeq \sum_{s\colon B}\ \sum_{p\colon b_0=_B s}\ \sum_{g\colon A\to b_0=_B s} g(a_0) = p && \text{; Prop. 2.9.12} \\
&\simeq \sum_{\omega\colon \sum_{s\colon B} b_0=_B s}\ \sum_{g\colon A\to b_0=_B \pi_1(\omega)} g(a_0) = \pi_2(\omega) && \text{; Prop. 2.9.13} \\
&\simeq \sum_{g\colon A\to b_0=_B b_0} g(a_0) = \mathsf{refl}_{b_0} && \text{; Thm. 3.4.8 b)} \\
&\equiv \text{RHS} && \text{; Defns. of } \Omega \text{ and } \mathsf{Map}_\bullet,
\end{aligned}$$

where the application of Theorem 3.4.8 b) is justified because

$$\sum_{s:\,B} b_0 =_B s$$

is contractible with center $\langle b_0, \mathsf{refl}_{b_0}\rangle$ by Lemma 3.4.11. □

**Lemma 6.6.15.** *Let $\langle \mathbb{S}^0, \mathsf{base}_0\rangle :\equiv \langle 2, 0_2\rangle$. For any pointed type $\langle X, x_0\rangle$, evaluating a pointed map at $1_2$ induces an equivalence*

$$\mathsf{Map}_\bullet(\langle \mathbb{S}^0, \mathsf{base}_0\rangle, \langle X, x_0\rangle) \simeq \pi_1(\langle X, x_0\rangle).$$

*Proof.* By the universal property of the coproduct $2 \simeq 1 + 1$, an unpointed function $g\colon 2 \to X$ is equivalent to the pair of its values $(g(0_2), g(1_2))\colon X \times X$. Therefore,

$$\begin{aligned}
\mathsf{Map}_\bullet(\langle 2, 0_2\rangle, \langle X, x_0\rangle) &:\equiv \sum_{g:\,2\to X} g(0_2) =_X x_0 \\
&\simeq \sum_{\langle x,y\rangle:\,X\times X} x =_X x_0 \\
&\simeq \Big(\sum_{x:\,X} x =_X x_0\Big) \times X \\
&\simeq X && \text{; Contractible sum} \\
&\equiv \pi_1(\langle X, x_0\rangle),
\end{aligned}$$

where the projection to the second coordinate of the Cartesian product corresponds to the evaluation of pointed maps at $1_2$. □

**Proposition 6.6.16.** *Define the pointed $n$-sphere recursively for $n\colon \mathbb{N}$ by*

- $\langle \mathbb{S}^0, \mathsf{base}_0\rangle :\equiv \langle 2, 0_2\rangle$ *and*
- $\langle \mathbb{S}^{n+1}, \mathsf{base}_{n+1}\rangle :\equiv \langle \mathsf{Susp}(\mathbb{S}^n), \mathsf{N}\rangle$.

*For any pointed type $\langle B, b\rangle$ and $n\colon \mathbb{N}$, we have an equivalence*

$$\mathsf{Map}_\bullet(\langle \mathbb{S}^n, \mathsf{base}_n\rangle, \langle B, b\rangle) \simeq \pi_1(\Omega^n(\langle B, b\rangle)).$$

*Proof.* We first show by induction on $n\colon \mathbb{N}$ that for all $\langle B, b\rangle\colon \mathcal{U}_\bullet$,

$$\mathsf{Map}_\bullet(\langle \mathbb{S}^n, \mathsf{base}_n\rangle, \langle B, b\rangle) \simeq \mathsf{Map}_\bullet(\langle \mathbb{S}^0, \mathsf{base}_0\rangle, \Omega^n(\langle B, b\rangle)).$$

For the base case $n \equiv 0$, since $\Omega^0(\langle B, b\rangle) \equiv \langle B, b\rangle$, both sides are judgmentally equal, and the equivalence is given by the identity equivalence $\mathsf{id}$.

For the inductive step, assume that the equivalence holds for $n$ and every pointed type in $\mathcal{U}_\bullet$. Given $\langle B, b\rangle : \mathcal{U}_\bullet$, we have the composite equivalence

$$\begin{aligned}
\mathsf{Map}_\bullet(\langle \mathbb{S}^{n+1}, \mathsf{base}_{n+1}\rangle, \langle B, b\rangle) &\equiv \mathsf{Map}_\bullet(\langle \mathsf{Susp}(\mathbb{S}^n), \mathsf{N}\rangle, \langle B, b\rangle) \\
&\simeq \mathsf{Map}_\bullet(\langle \mathbb{S}^n, \mathsf{base}_n\rangle, \Omega(\langle B, b\rangle)) && \text{; Thm 6.6.14} \\
&\simeq \mathsf{Map}_\bullet(\langle \mathbb{S}^0, \mathsf{base}_0\rangle, \Omega^n(\Omega(\langle B, b\rangle))) && \text{; I.H.} \\
&\equiv \mathsf{Map}_\bullet(\langle \mathbb{S}^0, \mathsf{base}_0\rangle, \Omega^{n+1}(\langle B, b\rangle)) && \text{; Def 2.2.2} \\
&\simeq \pi_1(\Omega^n(\langle B, b\rangle)) && \text{; Lem 6.6.15}
\end{aligned}$$

□

**Corollary 6.6.17.** *For any type $B$, the unpointed mapping space out of the recursively defined $n$-sphere satisfies the universal property*

$$(\mathbb{S}^n \to B) \simeq \sum_{b:\, B} \pi_1(\Omega^n(\langle B, b\rangle)).$$

*Proof.* Grouping functions by their value at $\mathsf{base}_n$ gives

$$\begin{aligned}
(\mathbb{S}^n \to B) &\simeq \sum_{f:\, \mathbb{S}^n \to B} \sum_{b:\, B} f(\mathsf{base}_n) =_B b && \text{; Lem. 3.4.11} \\
&\simeq \sum_{b:\, B} \sum_{f:\, \mathbb{S}^n \to B} f(\mathsf{base}_n) =_B b && \text{; Prop. 2.9.12} \\
&\equiv \sum_{b:\, B} \mathsf{Map}_\bullet(\langle \mathbb{S}^n, \mathsf{base}_n\rangle, \langle B, b\rangle) && \text{; Defn. 6.6.7} \\
&\simeq \sum_{b:\, B} \pi_1(\Omega^n(\langle B, b\rangle)) && \text{; Prop. 6.6.16.}
\end{aligned}$$

□

### 6.6.1 Functoriality

**Proposition 6.6.18.** *For $n : \mathbb{N}$, let $F : \mathcal{U} \to \mathcal{U}$ be the operation defined on a type $B : \mathcal{U}$ by*

$$F(B) :\equiv \sum_{b:\, B} \pi_1(\Omega^n(\langle B, b\rangle)),$$

*and on a function $g : B \to C$ by*

$$\begin{aligned}
F(g) : F(B) &\to F(C) \\
\langle b, \ell\rangle &\mapsto \langle g(b), \mathsf{ap}^n_g(\ell)\rangle.
\end{aligned}$$

*Then $F$ satisfies the functoriality laws up to homotopy:*

- $F(\mathsf{id}_B) \sim \mathsf{id}_{F(B)}$.
- *If $h : C \to D$, then $F(h \circ g) \sim F(h) \circ F(g)$.*

*Proof.* Let us first establish, by induction on $n\colon \mathbb{N}$, the action laws

- $\mathsf{ap}^n_{\mathsf{id}_B}(\ell) = \ell$ and
- $\mathsf{ap}^n_{h\circ g}(\ell) = \mathsf{ap}^n_h\big(\mathsf{ap}^n_g(\ell)\big)$.

For $n = 0$, both hold judgmentally.

For the inductive step, the functoriality of standard $\mathsf{ap}$ yields

$$\mathsf{ap}^{n+1}_{\mathsf{id}_B}(\ell) \equiv \mathsf{ap}_{\mathsf{ap}^n_{\mathsf{id}_B}}(\ell) = \mathsf{ap}_{\mathsf{id}}(\ell) = \ell$$

and

$$\mathsf{ap}^{n+1}_{h\circ g}(\ell) \equiv \mathsf{ap}_{\mathsf{ap}^n_{h\circ g}}(\ell) = \mathsf{ap}_{\mathsf{ap}^n_h\circ\mathsf{ap}^n_g}(\ell) = \mathsf{ap}_{\mathsf{ap}^n_h}\big(\mathsf{ap}_{\mathsf{ap}^n_g}(\ell)\big) \equiv \mathsf{ap}^{n+1}_h\big(\mathsf{ap}^{n+1}_g(\ell)\big).$$

Now, using trivial transport along reflexivity in the base components, we deduce the functoriality laws as follows:

$$\begin{aligned} F(\mathsf{id}_B)(\langle b,\ell\rangle) &\equiv \langle b, \mathsf{ap}^n_{\mathsf{id}_B}(\ell)\rangle \\ &= \langle b,\ell\rangle \\ &\equiv \mathsf{id}_{F(B)}\langle b,\ell\rangle). \end{aligned}$$

and

$$\begin{aligned} F(h\circ g)(\langle b,\ell\rangle) &\equiv \langle h(g(b)), \mathsf{ap}^n_{h\circ g}(\ell)\rangle \\ &= \langle h(g(b)), \mathsf{ap}^n_h(\mathsf{ap}^n_g(\ell))\rangle \\ &\equiv (F(h)\circ F(g))(\langle b,\ell\rangle)). \end{aligned}$$

□

**Remark 6.6.19.** The operator $F\colon \mathcal{U}\to\mathcal{U}$ defined by

$$F(B) :\equiv \sum_{b\colon B} \pi_1(\Omega^n(\langle B, b\rangle))$$

is functorial:

- If $g\colon B\to C$, then $F(g)\colon F(B)\to F(C)$ maps $\langle b,\ell\rangle$ to $\langle g(b), \mathsf{ap}^n_g(\ell)\rangle$.

A type $X$ co-represents $F$ if there is a family of equivalences $\Phi_B\colon (X\to B)\simeq F(B)$ natural in $B$. By the Yoneda lemma, any such family is completely determined by the *universal element* $u_X :\equiv \Phi_X(\mathsf{id}_X)\colon F(X)$, since naturality forces $\Phi_B(f) = F(f)(u_X)$ for every $f\colon X\to B$.

Both definitions of the $n$-sphere co-represent $F$:

- When the $n$-sphere is defined directly as a Higher Inductive Type $\mathbb{S}^n_{\mathrm{HIT}}$ generated by a point $\mathsf{base}\colon \mathbb{S}^n_{\mathrm{HIT}}$ and an $n$-loop $\mathsf{loop}_n\colon \pi_1(\Omega^n(\langle \mathbb{S}^n_{\mathrm{HIT}}, \mathsf{base}\rangle))$, the constructors form the universal element $\langle \mathsf{base}, \mathsf{loop}_n\rangle\colon F(\mathbb{S}^n_{\mathrm{HIT}})$, and its recursion principle states that the canonical evaluation map $\Phi_B(f) :\equiv \langle f(\mathsf{base}), \mathsf{ap}^n_f(\mathsf{loop}_n)\rangle$ is a natural equivalence $(\mathbb{S}^n_{\mathrm{HIT}}\to B)\simeq F(B)$.

- When the $n$-sphere is defined via iterated suspensions as $\mathbb{S}^n_{\mathrm{Susp}}$, Corollary 6.6.17 provides a natural equivalence $\Psi_B \colon (\mathbb{S}^n_{\mathrm{Susp}} \to B) \simeq F(B)$, whose universal element $\Psi_{\mathbb{S}^n_{\mathrm{Susp}}}(\mathsf{id}_{\mathbb{S}^n_{\mathrm{Susp}}})$ extracts the canonical basepoint $\mathsf{base}_n$ and the suspended $n$-loop.

Composing these equivalences for arbitrary $X$ and $Y$ co-representing $F$ yields a natural equivalence of mapping spaces

$$\Theta_B :\equiv \Psi_B^{-1} \circ \Phi_B \colon (X \to B) \simeq (Y \to B),$$

satisfying $\Theta_C(h \circ k) = h \circ \Theta_B(k)$ for all $k \colon X \to B$ and $h \colon B \to C$. Instantiating $B$ at $X$ and $Y$ produces the mutual quasi-inverses

$$g :\equiv \Theta_X(\mathsf{id}_X) \colon Y \to X \quad \text{and} \quad f :\equiv \Theta_Y^{-1}(\mathsf{id}_Y) \colon X \to Y.$$

Indeed, by naturality of $\Theta$ along $f \colon X \to Y$, we have

$$f \circ g \equiv f \circ \Theta_X(\mathsf{id}_X) = \Theta_Y(f \circ \mathsf{id}_X) \equiv \Theta_Y(\Theta_Y^{-1}(\mathsf{id}_Y)) = \mathsf{id}_Y,$$

and symmetrically $g \circ f = \mathsf{id}_X$ by naturality of $\Theta^{-1}$ along $g \colon Y \to X$. Thus, $\mathbb{S}^n_{\mathrm{Susp}} \simeq \mathbb{S}^n_{\mathrm{HIT}}$, and by Univalence, $\mathbb{S}^n_{\mathrm{Susp}} =_{\mathcal{U}} \mathbb{S}^n_{\mathrm{HIT}}$.

# Appendix A

# Axiom Dependencies

## A.1 Function Extensionality Dependencies

The following results rely on the weak function extensionality axiom (WE)


Theorem 3.4.5 . . . . . 232
Corollary 3.4.6 . . . . . 232
Corollary 3.5.5 . . . . . 241
Lemma 4.4.3 . . . . . 283
Theorem 4.4.4 . . . . . 284
Corollary 4.4.5 . . . . . 284
Theorem 4.8.5 . . . . . 299


The following results rely on the strong function extensionality axiom (FE)


Proposition 2.11.2 . . . . . 128
Proposition 2.11.3 . . . . . 128
Corollary 2.11.4 . . . . . 129
Theorem 2.11.15 . . . . . 137
Theorem 2.11.17 . . . . . 140
Theorem 2.12.3 . . . . . 143
Proposition 2.12.8 . . . . . 146
Theorem 2.18.1 . . . . . 171
Theorem 2.18.2 . . . . . 172
Theorem 2.18.3 . . . . . 173
Theorem 2.18.5 . . . . . 174
Theorem 2.18.6 . . . . . 175
Theorem 2.18.7 . . . . . 176
Theorem 2.18.8 . . . . . 176
Exercise 2.19.2 . . . . . 179
Exercise 2.19.8 . . . . . 184
Exercise 2.19.12 . . . . . 191
Exercise 2.19.15 . . . . . 194
Theorem 3.1.2 . . . . . 208
Theorem 3.1.8 . . . . . 213
Corollary 3.1.10 . . . . . 215
Proposition 3.2.9 . . . . . 217
Theorem 3.2.15 . . . . . 223
Corollary 3.2.17 . . . . . 226
Corollary 3.2.18 . . . . . 226
Corollary 3.2.19 . . . . . 227
Proposition 3.2.20 . . . . . 227
Lemma 3.3.4 . . . . . 230
Theorem 3.4.8 . . . . . 233
Theorem 3.4.13 . . . . . 235
Lemma 3.5.3 . . . . . 240
Corollary 3.5.5 . . . . . 241
Exercise 3.7.1 . . . . . 244
Exercise 3.7.3 . . . . . 245
Exercise 3.7.4 . . . . . 246
Exercise 3.7.5 . . . . . 246
Exercise 3.7.7 . . . . . 247
Exercise 3.7.8 . . . . . 249
Exercise 3.7.11 . . . . . 252
Exercise 3.7.13 . . . . . 254

Exercise 3.7.14 . . . . . 256
Exercise 3.7.16 . . . . . 259
Exercise 3.7.17 . . . . . 259
Exercise 3.7.18 . . . . . 262
Exercise 3.7.19 . . . . . 262
Exercise 3.7.20 . . . . . 263
Theorem 4.1.7 . . . . . 269
Proposition 4.1.8 . . . . . 271
Proposition 4.1.11 . . . . . 272
Lemma 4.1.12 . . . . . 273
Theorem 4.1.13 . . . . . 275
Lemma 4.2.8 . . . . . 280
Theorem 4.2.9 . . . . . 280
Theorem 4.3.2 . . . . . 281
Corollary 4.3.4 . . . . . 281
Theorem 4.5.4 . . . . . 285
Theorem 4.7.3 . . . . . 292
Lemma 4.7.6 . . . . . 295
Theorem 4.7.7 . . . . . 296
Exercise 4.9.1 . . . . . 301
Exercise 4.9.2 . . . . . 302

The following results rely on the univalence axiom (UA)

Proposition 2.13.5 . . . . . 149
Theorem 2.17.1 . . . . . 171
Exercise 2.19.15 . . . . . 194
Exercise 2.19.17 . . . . . 207
Theorem 3.1.6 . . . . . 212
Theorem 3.1.8 . . . . . 213
Corollary 3.1.10 . . . . . 215
Theorem 3.2.15 . . . . . 223
Lemma 3.3.3 . . . . . 229
Theorem 3.4.8 . . . . . 233
Theorem 3.5.4 . . . . . 241
Corollary 3.5.5 . . . . . 241
Exercise 3.7.7 . . . . . 247
Exercise 3.7.8 . . . . . 249
Exercise 3.7.9 . . . . . 250
Theorem 4.1.13 . . . . . 275
Theorem 4.7.3 . . . . . 292
Remark 4.7.4 . . . . . 293
Lemma 4.8.2 . . . . . 298
Proposition 4.8.3 . . . . . 298
Theorem 4.8.4 . . . . . 299

# Appendix B

# Additional Notes

## B.1 Universes and Type Families

The definition of a universe has an apparent circularity: a universe is defined using the concept of a type family, while type families are formalized as functions into a universe. The circularity is resolved by distinguishing between the *meta-theoretic* (judgmental) framework and the *object-theoretic* (internal) language of type theory.

### Judgmental Type Families

Before the introduction of any universes, the type theory relies on a primitive notion of type dependence. At this stage, a type family is not a mathematical object or a function that can be evaluated or composed: it is a syntactic rule, a *judgment*. It states that under the assumption of a variable $x$ of type $A$, an expression $P(x)$ forms a valid type. This is denoted using the notation

$$x \colon A \vdash P(x) \text{ type}.$$

These judgmental families are rigidly bound to the deductive rules of the system and cannot be treated as first-class entities.

### Internal Type Families

In order to be able to manipulate type families as mathematical objects (e.g., quantify over them, compose them, and map into them), the judgmental families need to be internalized as functions. A universe $\mathcal{U}$ is introduced to serve as the codomain for these functions

$$P \colon A \to \mathcal{U}.$$

Under this approach, the elements of $\mathcal{U}$ are typically viewed as *codes* (or *names*) for types, rather than types themselves.

## Tarski-Style Universes

To define the universe $\mathcal{U}$ without circularity, we rely on the pre-existing judgmental framework.

**Definition B.1.1.** A *Tarski-style universe* consists of a type $\mathcal{U}$ of *codes* together with a decoding function

$$X : \mathcal{U} \vdash \tau(X) \text{ type}$$

that assigns a type to each code. To accurately reflect the ambient type theory, the universe must be closed under type constructors. This means there exist code-forming operators in $\mathcal{U}$ which $\tau$ decodes definitionally into the standard type formations:

a) Dependent functions: For any code $a : \mathcal{U}$ and any family of codes $b : \tau(a) \to \mathcal{U}$, there is a code $\hat{\Pi}(a, b) : \mathcal{U}$ such that

$$\tau(\hat{\Pi}(a, b)) \equiv \prod_{x : \tau(a)} \tau(b(x)).$$

b) Dependent pairs: For any code $a : \mathcal{U}$ and family of codes $b : \tau(a) \to \mathcal{U}$, there is a code $\hat{\Sigma}(a, b) : \mathcal{U}$ such that

$$\tau(\hat{\Sigma}(a, b)) \equiv \sum_{x : \tau(a)} \tau(b(x)).$$

c) Identity types: For any code $a : \mathcal{U}$ and terms $x, y : \tau(a)$, there is a code $\hat{\mathsf{Id}}(a, x, y) : \mathcal{U}$ such that

$$\tau(\hat{\mathsf{Id}}(a, x, y)) \equiv x =_{\tau(a)} y.$$

d) Coproducts: For any codes $a, b : \mathcal{U}$, there is a code $\hat{+}(a, b) : \mathcal{U}$ such that

$$\tau(\hat{+}(a, b)) \equiv \tau(a) + \tau(b).$$

**Remark B.1.2.** The assertion that the universe $\mathcal{U}$ is itself a type cannot be derived from the decoding function $\tau$. Instead, its existence must be postulated via a primitive formation rule:

$$\vdash \mathcal{U} \text{ type}.$$

One might naturally wonder whether $\mathcal{U}$ is an element of itself, i.e., whether the judgment $\mathcal{U} : \mathcal{U}$ holds. However, positing that a universe contains its own code is notoriously inconsistent. Much like the set of all sets in classical set theory, it leads to a type-theoretic contradiction known as *Girard's paradox*.

To circumvent this inconsistency while maintaining the ability to compute with universes, Homotopy Type Theory introduces an infinite hierarchy of universes:

$$\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \ldots$$

where each universe is a term of the next: $\mathcal{U}_i : \mathcal{U}_{i+1}$. In a strict Tarski-style presentation, each universe $\mathcal{U}_i$ has its own decoding function $\tau_i : \mathcal{U}_i \to \mathcal{U}_{i+1}$.

In mathematical practice, managing these indices explicitly is exceedingly cumbersome. Following a convention known as *typical ambiguity*, we routinely drop the subscripts and simply write $\mathcal{U}$, implicitly trusting that a consistent assignment of universe levels could be reconstructed to avoid paradoxes.

## Russell-Style Universes

While the Tarski-style separation between codes and types is necessary for foundational rigor, managing the decoding function $\tau$ explicitly becomes notationally burdensome. In mathematical practice, it is customary to adopt a *Russell-style* presentation. This is achieved by systematically omitting the decoding function, identifying the code $A : \mathcal{U}$ with its corresponding type $\tau(A)$.

Under this convention, the universe $\mathcal{U}$ is treated as if it were a type whose elements are types. The decoding properties of $\tau$ ensure that this abuse of notation is perfectly safe, as the code-forming operators (such as $\hat{\Pi}$ and $\hat{\Sigma}$) become completely transparent and indistinguishable from the standard type constructors.

**Notation B.1.3.** *With the Russell-style convention in place, an internal type family is simply a function*

$$P : B \to \mathcal{U}.$$

*For any element $b : B$, the application $P(b)$ is an element of $\mathcal{U}$, which we directly regard as a type. Consequently, we can immediately form the dependent types*

$$\prod_{b:\, B} P(b) \qquad \textit{and} \qquad \sum_{b:\, B} P(b)$$

*without writing $\tau(P(b))$.*

## Types by Computation

A profound consequence of having a universe is the ability to define types by *computation*. Because $\mathcal{U}$ is a valid codomain for functions, we can construct type families using the elimination principles (pattern matching and induction) of other types. This is fundamentally impossible with purely judgmental families.

**Example B.1.4.** We can use a computed family over the boolean type 2 to formally prove that its constructors $1_2$ and $0_2$ are distinct. We define a function

$P\colon 2 \to \mathcal{U}$ by pattern matching:

$$P(1_2) :\equiv 1$$
$$P(0_2) :\equiv 0.$$

If there were a path $p\colon 1_2 =_2 0_2$, we could transport the canonical element $\star\colon 1$ along $p$ in the family $P$. This transport would yield an element of the empty type:

$$\mathsf{transport}^P(p, \star)\colon 0.$$

Since $0$ has no elements, such a path $p$ cannot exist. This standard derivation, which shows $1_2 \neq 0_2$, relies essentially on the existence of the universe $\mathcal{U}$: since $\mathcal{U}$ is a type, we can use it as the codomain in the induction principle for $2$.

## B.2 Yoneda's Lema

**Theorem B.2.1.** [Yoneda's Lemma] *Let $A$ be an object in a category $\mathbf{C}$. Let $F\colon \mathbf{C} \to \mathbf{Set}$ be a covariant functor, and let $\boldsymbol{h}_A = \mathrm{Hom}_{\mathbf{C}}(A, -)$ be the representable functor from $\mathbf{C}$ to $\mathbf{Set}$. Then the set $F(A)$ is in a one-to-one correspondence with the collection of natural transformations from $\boldsymbol{h}_A$ to $F$. This correspondence maps each natural transformation $\eta\colon \boldsymbol{h}_A \to F$ to the element $\eta_A(\mathsf{id}_A)$ in $F(A)$.*

*Proof.* Let $c \in F(A)$ be an element. For any object $Y$ define

$$\eta_{c,Y}\colon \boldsymbol{h}_A(Y) \to F(Y)$$
$$\zeta \mapsto F(\zeta)(c).$$

Given $u\colon Y \to Z$ we have $\boldsymbol{h}_A(u)(\zeta) = u \circ \zeta$ for $\zeta \in \boldsymbol{h}_A(Y)$. Put $(\eta_c)_Y = \eta_{c,Y}$. Then $\eta_c\colon \boldsymbol{h}_A \to F$ is natural because the diagram

$$\begin{array}{ccc} \boldsymbol{h}_A(Y) & \xrightarrow{(\eta_c)_Y} & F(Y) \\ {\scriptstyle \boldsymbol{h}_A(u)}\downarrow & & \downarrow{\scriptstyle F(u)} \\ \boldsymbol{h}_A(Z) & \xrightarrow[(\eta_c)_Z]{} & F(Z) \end{array} \qquad \begin{array}{ccc} \zeta & \longmapsto & F(\zeta)(c) \\ \downarrow & & \downarrow \\ u \circ \zeta & \longmapsto & F(u \circ \zeta)(c) \end{array}$$

commutes (since $F(u)(F(\zeta)(c)) = F(u \circ \zeta)(c)$ by the functoriality of $F$).

Conversely, take a natural transformation $\eta\colon \boldsymbol{h}_A \to F$. Since $\eta_A$ takes endomorphisms of $A$ to elements of $F(A)$, we know that $c_\eta = \eta_A(\mathsf{id}_A)$ is such an element.

It remains to be seen that both maps are reciprocal.

$\boldsymbol{F(A) \to \mathbf{Nat} \to F(A)}$ **:** Take $c \in F(A)$. Then,

$$\begin{aligned} c &\mapsto \eta_c \mapsto c_{\eta_c} \\ c_{\eta_c} &= (\eta_c)_A(\mathsf{id}_A) \\ &= \eta_{c,A}(\mathsf{id}_A) \\ &= F(\mathsf{id}_A)(c) \\ &= \mathsf{id}_{F(A)}(c) \\ &= c. \end{aligned}$$

$\mathbf{Nat} \to \boldsymbol{F(A)} \to \mathbf{Nat}$ **:** Take $\eta \colon \boldsymbol{h}_A \to F$. Given $\zeta \colon A \to Y$, we have a commutative diagram

$$\begin{array}{ccc} \boldsymbol{h}_A(A) & \xrightarrow{\eta_A} & F(A) \\ {\scriptstyle \boldsymbol{h}_A(\zeta)}\downarrow & & \downarrow{\scriptstyle F(\zeta)} \\ \boldsymbol{h}_A(Y) & \xrightarrow[\eta_Y]{} & F(Y) \end{array} \qquad \begin{array}{ccc} \mathsf{id}_A & \longmapsto & c_\eta \\ \downarrow & & \downarrow \\ \zeta \circ \mathsf{id}_A = \zeta & \longmapsto & F(\zeta)(c_\eta) \end{array}$$

Therefore,

$$\begin{aligned} (\eta_{c_\eta})_Y(\zeta) &= \eta_{c_\eta, Y}(\zeta) \\ &= F(\zeta)(c_\eta) \\ &= \eta_Y(\zeta), \end{aligned}$$

which shows that $\eta_{c_\eta} = \eta$, as desired.

□

## B.3 The Slice Category

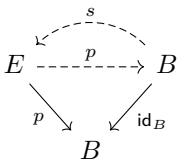
**Definition B.3.1.** Given a set $B$ in **Set**, the *slice category over* $B$ is the category $\mathbf{Set}_B$ whose objects are the function $p \colon E \to B$, and its morphisms $\mathrm{Hom}_{\mathbf{Set}_B}(p \colon E \to B, p' \colon E' \to B)$ are the functions $\phi \colon E \to E'$ that make commutative the triangle

$$\begin{array}{ccc} E & \xrightarrow{\phi} & E' \\ & {\scriptstyle p}\searrow \quad \swarrow{\scriptstyle p'} & \\ & B & \end{array} \tag{B.1}$$

**Remark B.3.2.** Recall that an object $\mathbf{1}$ in a category $\mathbf{C}$ is *terminal* if, for any other object $X$, there is a unique morphism $X \to \mathbf{1}$. If such an object exists, a *global point* in an object $X$ is an arrow $s \colon \mathbf{1} \to X$.

**Proposition B.3.3.** *The identity* $\mathsf{id}_B \colon B \to B$ *is a terminal object in* $\mathbf{Set}_B$*. In particular, the global points of* $p$ *are the sections of* $p$*.*

*Proof.* Let $p\colon E \to B$ be an arbitrary object of $\mathbf{Set}_B$. Then $p\colon E \to B$ is a morphism from $p$ to $\mathsf{id}_B$:




and any morphism $s$ from $\mathsf{id}_B$ to $p$ is, by definition, of section of $p$. □

**Remark B.3.4.** Consider the set $B$ as a so called *discrete category* whose objects are its elements and its morphisms reduce to formal identities $\mathsf{id}_b\colon b \to b$ for $b \in B$. Given two functors $F, G\colon B \to \mathbf{Set}$, since the only morphisms in $B$ are the identities, a natural transformation $\eta\colon F \to G$ consists of a family of maps $\eta_b\colon F(b) \to G(b)$.

**Notation B.3.5.** *As usual, we will denote the functor category from $B$ to* **Set** *by* $\mathsf{Fun}(B, \mathbf{Set})$.

**Theorem B.3.6.** [Grothendieck Construction] *Let $B$ be a set. Then, there is a natural isomorphism* $\mathbf{Set}_B \simeq \mathsf{Fun}(B, \mathbf{Set})$.

*Proof.* The forward functor

$$\mathcal{F}\colon \mathbf{Set}_B \to \mathsf{Fun}(B, \mathbf{Set})$$

is defined on objects by

$$\mathcal{F}(p)(b) = p^{-1}(b), \tag{1}$$

for an object $p$ in $\mathbf{Set}_B$ and $b \in B$. For a morphism $\phi\colon p \to p'$ as in (B.1), we define

$$\mathcal{F}(\phi)_b\colon p^{-1}(b) \to (p')^{-1}(b), \quad x \mapsto \phi(x), \tag{2}$$

for $b \in B$ and $x \in p^{-1}(b)$.

The backward functor

$$\mathcal{G}\colon \mathsf{Fun}(B, \mathbf{Set}) \to \mathbf{Set}_B$$

is defined on objects by the projection map

$$\mathcal{G}(F)\colon \coprod_{b \in B} F(b) \to B, \tag{3}$$

which maps each element of $F(b)$ to $b$ for every $b \in B$. For a natural transformation $\eta\colon F \to G$ between two functors in $\mathsf{Fun}(B, \mathbf{Set})$, we define the morphism

$$\mathcal{G}(\eta)\colon \mathcal{G}(F) \to \mathcal{G}(G), \quad x \mapsto \eta_b(x), \tag{4}$$

for $b \in B$ and $x \in F(b)$.

To see that these functors are mutually inverse, observe that for any functor $F$ and $b \in B$,

$$\begin{aligned}(\mathcal{F} \circ \mathcal{G})(F)(b) &= \mathcal{F}(\mathcal{G}(F))(b) \\ &= \mathcal{G}(F)^{-1}(b) && ; \text{by (1)} \\ &= F(b) && ; \text{by (3).}\end{aligned}$$

For a natural transformation $\eta\colon F \to G$, evaluating at $x \in F(b)$ yields

$$\begin{aligned}(\mathcal{F} \circ \mathcal{G})(\eta)_b(x) &= \mathcal{G}(\eta)(x) && ; \text{by (2)} \\ &= \eta_b(x) && ; \text{by (4),}\end{aligned}$$

so $\mathcal{F} \circ \mathcal{G}$ is the identity functor on $\mathsf{Fun}(B, \mathbf{Set})$.

Conversely, fix an object $p\colon E \to B$ in $\mathbf{Set}_B$. Then,

$$\begin{aligned}(\mathcal{G} \circ \mathcal{F})(p) &= \mathcal{G}(\mathcal{F}(p)) \\ &= \coprod_{b \in B} p^{-1}(b) \xrightarrow{\pi} B && ; \text{by (1) \& (3)} \\ &= E \xrightarrow{p} B \\ &= p.\end{aligned}$$

Now consider a morphism $\phi\colon p \to p'$. For $b \in B$ and $x \in p^{-1}(b)$, we have

$$\begin{aligned}(\mathcal{G} \circ \mathcal{F})(\phi)(x) &= \mathcal{G}(\mathcal{F}(\phi))(x) \\ &= \mathcal{F}(\phi)_b(x) && ; \text{by (4)} \\ &= \phi(x) && ; \text{by (2).}\end{aligned}$$

Thus, $\mathcal{G} \circ \mathcal{F} = \mathsf{Id}$, the identity functor on $\mathbf{Set}_B$. This completes the proof.

□

Even if one can take some $f\colon 1 \to X$ and obtain the commutative diagram:

$$\begin{array}{ccc} 1 & & \\ {\scriptstyle f}\downarrow & \searrow^{y \circ f} & \\ X & \xrightarrow[y]{} & Y, \end{array}$$

there still may be other nonidentical arrows $1 \to X$, which provide nonequivalent 'globalisations' of the local element $y \in Y^X$. Hence, $X$ can have non-global elements which are seen only from some stages of definition ('contexts of observation'). So, while the global elements of an object correspond to the set-theoretic idea of points, there can also be elements of an object that are not points. The importance of this idea cannot be overestimated. It provides a fresh view on many structures.

## B.4 Zorn's Lemma

**Definition B.4.1.** Let $P$ be a partially ordered set. An element $c \in P$ is called **compact** (or **finite**) if, for every directed subset $D \subseteq P$ that has a supremum $\sup D$ in $P$, the following condition holds:

If $c \leq \sup D$, then there exists some $d \in D$ such that $c \leq d$.

# Bibliography

The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, NJ, 2013. URL `https://homotopytypetheory.org/book`.